

FinancePy

Dominic O’Kane

March 11, 2026

Contents

1	financepy.utils	3
1.1	amount	5
1.2	calendar	7
1.3	compounding	23
1.4	currency	24
1.5	date	25
1.6	datelite	37
1.7	date_arrays	39
1.8	date_format	40
1.9	day_count	42
1.10	distribution	49
1.11	error	50
1.12	frequency	52
1.13	global_types	53
1.14	global_vars	55
1.15	helpers	56
1.16	latex	63
1.17	math	64
1.18	polyfit	72
1.19	schedule	76
1.20	singleton	81
1.21	solver_1d	82
1.22	solver_cg	90
1.23	solver_nm	96
1.24	stats	99
1.25	tenor	101
1.26	tension_spline	106
1.27	__init__	107
2	financepy.market.curves	109
2.1	composite_discount_curve	111
2.2	discount_curve	112
2.3	discount_curve_flat	122
2.4	discount_curve_ns	126
2.5	discount_curve_nss	130
2.6	discount_curve_poly	134
2.7	discount_curve_pwf	138

2.8	discount_curve_pwf_onf	141
2.9	discount_curve_pwl	144
2.10	discount_curve_zeros	146
2.11	interpolator	148
2.12	__init__	155
3	financypy.market.volatility	157
3.1	equity_vol_curve	158
3.2	equity_vol_surface	160
3.3	fx_vol_surface	178
3.4	fx_vol_surface_plus	203
3.5	ibor_cap_vol_curve	242
3.6	ibor_cap_vol_curve_fn	247
3.7	swaption_vol_surface	248
3.8	__init__	259
4	financypy.products.equity	261
4.1	equity_american_option	263
4.2	equity_asian_option	265
4.3	equity_barrier_option	288
4.4	equity_basket_option	292
4.5	equity_binomial_tree	298
4.6	equity_chooser_option	305
4.7	equity_cliquet_option	312
4.8	equity_compound_option	316
4.9	equity_digital_option	321
4.10	equity_fixed_lookback_option	329
4.11	equity_float_lookback_option	337
4.12	equity_forward	343
4.13	equity_index_option	346
4.14	equity_model_types	351
4.15	equity_one_touch_option	352
4.16	equity_option	361
4.17	equity_rainbow_option	368
4.18	equity_swap	379
4.19	equity_swap_leg	382
4.20	equity_vanilla_option	387
4.21	equity_variance_swap	403
4.22	__init__	409
5	financypy.products.credit	411
5.1	cds	412
5.2	cds_basket	430
5.3	cds_curve	437
5.4	cds_index_option	443
5.5	cds_index_portfolio	450
5.6	cds_option	457

5.7	<code>cds_tranche</code>	460
5.8	<code>__init__</code>	466
6	<code>financepy.products.bonds</code>	467
6.1	<code>bond</code>	469
6.2	<code>bond_annuity</code>	510
6.3	<code>bond_callable</code>	514
6.4	<code>bond_convertible</code>	519
6.5	<code>bond_frn</code>	528
6.6	<code>bond_future</code>	541
6.7	<code>bond_market</code>	553
6.8	<code>bond_mortgage</code>	556
6.9	<code>bond_option</code>	561
6.10	<code>bond_portfolio</code>	564
6.11	<code>bond_yield_curve</code>	565
6.12	<code>bond_zero</code>	569
6.13	<code>bond_zero_curve</code>	589
6.14	<code>curve_fits</code>	593
6.15	<code>yield_curve</code>	599
6.16	<code>__init__</code>	603
7	<code>financepy.products.rates</code>	605
7.1	<code>callable_swap</code>	608
7.2	<code>dual_curve</code>	609
7.3	<code>ibor_basis_swap</code>	613
7.4	<code>ibor_benchmarks_report</code>	617
7.5	<code>ibor_bermudan_swaption</code>	622
7.6	<code>ibor_cap_floor</code>	626
7.7	<code>ibor_conventions</code>	632
7.8	<code>ibor_curve_risk_engine</code>	633
7.9	<code>ibor_deposit</code>	638
7.10	<code>ibor_dual_curve</code>	643
7.11	<code>ibor_fixed_float_swap</code>	647
7.12	<code>ibor_fra</code>	656
7.13	<code>ibor_future</code>	663
7.14	<code>ibor_lmm_products</code>	666
7.15	<code>ibor_single_curve</code>	678
7.16	<code>ibor_single_curve_par_shocker</code>	685
7.17	<code>ibor_single_curve_smoothing_calibrator</code>	687
7.18	<code>ibor_swap</code>	689
7.19	<code>ibor_swaption</code>	698
7.20	<code>ois</code>	704
7.21	<code>ois_basis_swap</code>	709
7.22	<code>ois_curve</code>	713
7.23	<code>swap_fixed_leg</code>	719
7.24	<code>swap_float_leg</code>	723

7.25	<code>__init__</code>	727
8	<code>financepy.products.fx</code>	729
8.1	<code>fx_barrier_option</code>	730
8.2	<code>fx_digital_option</code>	740
8.3	<code>fx_double_digital_option</code>	743
8.4	<code>fx_double_one_touch_option</code>	746
8.5	<code>fx_fixed_lookback_option</code>	756
8.6	<code>fx_float_lookback_option</code>	762
8.7	<code>fx_forward</code>	768
8.8	<code>fx_mkt_conventions</code>	772
8.9	<code>fx_one_touch_option</code>	773
8.10	<code>fx_option</code>	782
8.11	<code>fx_rainbow_option</code>	785
8.12	<code>fx_vanilla_option</code>	796
8.13	<code>fx_variance_swap</code>	805
8.14	<code>__init__</code>	810
9	<code>financepy.models</code>	811
9.1	<code>bachelier</code>	813
9.2	<code>bdt_tree</code>	815
9.3	<code>bk_tree</code>	833
9.4	<code>black</code>	848
9.5	<code>black_scholes</code>	866
9.6	<code>black_scholes_analytic</code>	874
9.7	<code>black_scholes_mc</code>	892
9.8	<code>black_scholes_mc_tests</code>	902
9.9	<code>black_shifted</code>	905
9.10	<code>cir_montecarlo</code>	908
9.11	<code>equity_asian_option_mc</code>	918
9.12	<code>equity_barrier_option_bs</code>	924
9.13	<code>equity_barrier_option_mc</code>	929
9.14	<code>equity_compound_option_bs</code>	932
9.15	<code>equity_crr_tree</code>	939
9.16	<code>equity_lsmc</code>	942
9.17	<code>finite_difference</code>	945
9.18	<code>finite_difference_psor</code>	949
9.19	<code>gauss_copula</code>	952
9.20	<code>gauss_copula_lhp</code>	953
9.21	<code>gauss_copula_lhplus</code>	959
9.22	<code>gauss_copula_onefactor</code>	966
9.23	<code>gbm_process_simulator</code>	974
9.24	<code>heston</code>	981
9.25	<code>hw_tree</code>	997
9.26	<code>lmm_mc</code>	1032
9.27	<code>loss_dbn_builder</code>	1050

9.28	merton_firm	1052
9.29	merton_firm_mkt	1059
9.30	model	1062
9.31	option_implied_dbn	1063
9.32	process_simulator	1064
9.33	rates_ho_lee	1076
9.34	sabr	1080
9.35	sabr_shifted	1085
9.36	sobol	1093
9.37	student_t_copula	1094
9.38	vasicek_mc	1095
9.39	volatility_fns	1101
9.40	__init__	1116

Chapter 1

financepy.utils

Introduction

This is a collection of modules used across a wide range of FinancePy functions. Examples include date generation, special mathematical functions and useful helper functions for performing some repeated action

- Date is a class for handling dates in a financial setting. Special functions are included for computing IMM dates and CDS dates and moving dates forward by tenors.
- Calendar is a class for determining which dates are not business dates in a specific region or country.
- DayCount is a class for determining accrued interest in bonds and also accrual factors in ISDA swap-like contracts.
- Error is a class which handles errors in the calculations done within FinancePy
- `annual_frequency` takes in a `annual_frequency` type and then returns the number of payments per year
- `global_vars` holds the value of constants used across the whole of FinancePy
- `helper_functions` is a set of helpful functions that can be used in a number of places
- `math` is a set of mathematical functions specific to finance which have been optimised for speed using Numba
- `FinSobol` is the implementation of Sobol quasi-random number generator. It has been speeded up using Numba.
- `FinRateConverter` converts rates for one compounding `annual_frequency` to rates for a different `annual_frequency`
- `FinSchedule` generates a sequence of cashflow payment dates in accordance with financial market standards
- `FinStatistics` calculates a number of statistical variables such as mean, standard deviation and variance
- `FinTestCases` is the code that underlies the test case framework used across FinancePy

FinDayCount

The year fraction function can take up to 3 dates, D1, D2 and D3 and a annual_frequency in specific cases. The current day count methods are listed below.

- THIRTY 360 BOND - 30E/360 ISDA 2006 4.16f, German, Eurobond(ISDA 2000)
- THIRTY E 360 - ISDA 2006 4.16(g) 30/360 ISMA, ICMA
- THIRTY E 360 ISDA - ISDA 2006 4.16(h)
- THIRTY E PLUS 360 - A month has 30 days. It rolls D2 to next month if D2 = 31
- ACT ACT ISDA - Splits accrued period into leap and non-leap year portions.
- ACT ACT ICMA - Used for US Treasury notes and bonds. Takes 3 dates and a annual_frequency.
- ACT 365 F - Denominator is always Fixed at 365, even in a leap year
- ACT 360 - Day difference divided by 360 - always
- ACT 365L - the 29 Feb is counted if it is in the date range

1.1 amount

Class: Amount

Amount

Create Amount object.

```
Amount(notional: float = ONE_MILLION,
currency_type: CurrencyTypes = CurrencyTypes.NONE,
"""Create_Amount_object."""
check_argument_types(.Amount, locals())
.notional = notional
.currency_type = currency_type
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
notional	float	-	ONE_MILLION
currency_type	CurrencyTypes	-	CurrencyTypes.NONE
"""Create Amount object."""	-	-	-
check_argument_types(.Amount, locals())	-	-	-
.notional	-	-	notional
.currency_type	-	-	currency_type

__repr__

Print out amount object.

```
__repr__():
```

The function arguments are described in the following table.

amount

Get the notional amount.

```
amount():
```

The function arguments are described in the following table.

_print

Print out the details of the schedule and the actual dates. This can be used for providing transparency on schedule calculations.

```
_print():
```

The function arguments are described in the following table.

1.2 calendar

Enumerated Type: BusDayAdjustTypes

This enumerated type has the following values:

- NONE
- FOLLOWING
- MODIFIED_FOLLOWING
- PRECEDING
- MODIFIED_PRECEDING

Enumerated Type: CalendarTypes

This enumerated type has the following values:

- NONE
- WEEKEND
- AUSTRALIA
- CANADA
- FRANCE
- GERMANY
- ITALY
- JAPAN
- NEW_ZEALAND
- NORWAY
- SWEDEN
- SWITZERLAND
- TARGET
- UNITED_STATES
- UNITED_KINGDOM

Enumerated Type: DateGenRuleTypes

This enumerated type has the following values:

- FORWARD
- BACKWARD
- self.cal_type
- self.day_in_year
- self.weekday
- @lru_cache(maxsize
- ifself.cal_type
- ifbd_type
- ifbd_type
- dt
- ifbd_type
- d_start
- m_start
- y_start

- dt
- ifdt.m!
- dt
- dt
- ifbd_type
- dt
- ifbd_type
- d_start
- m_start
- y_start
- dt
- ifdt.m!
- dt
- dt
- ifself.cal_type
- ifbd_type
- ifdt.weekday
- dt
- elifdt.weekday
- dt
- dt
- ifbd_type
- d_start,m_start,y_start
- orig_dt
- ifdt.weekday
- dt
- elifdt.weekday
- dt
- dt
- ifdt.m!
- dt
- ifdt.weekday
- dt
- elifdt.weekday
- dt
- dt
- ifbd_type
- ifdt.weekday
- dt
- elifdt.weekday
- dt
- dt
- ifbd_type
- d_start,m_start,y_start
- orig_dt
- ifdt.weekday

- [illegible]

- day_in_year
- weekday
- ifm
- ifm
- ifm
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- m
- d
- y
- weekday
- ifm
- ifm
- ifm
- em
- ifself.day_in_year
- ifself.day_in_year
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- m
- d
- y
- day_in_year
- ifm

- em
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- m
- d
- y
- day_in_year
- weekday
- ifm
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- m
- d
- y
- day_in_year
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifm
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm

- [illegible]

- ifm
- ifm
- m
- d
- y
- day_in_year
- weekday
- ifm
- ifm
- ifm
- ifm
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- m
- d
- y
- day_in_year
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifday_in_year
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm
- m
- d
- weekday
- ifm
- ifm
- ifm
- ifm

- [illegible]

- ifm
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- ifm
- m
- d
- y
- day_in_year
- ifm
- ifm
- em
- ifday_in_year
- ifday_in_year
- ifm
- ifm
- defholiday_none(self,dt
- start_dt
- end_dt
- holiday_list
- start_dt
- em_days
- start_dt
- em
- s
- s

Class: Calendar

Class to manage designation of payment dates as holidays according to a regional or country-specific calendar convention specified by the user. It also supplies an adjustment method which takes in an adjustment convention and then applies that to any date that falls on a holiday in the specified calendar.

Calendar

Create a calendar based on a specified calendar type.

```
Calendar(cal_type: CalendarTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cal_type	CalendarTypes	-	-

adjust_cached

PLEASE ADD A FUNCTION DESCRIPTION

```
adjust_cached(dt: "Date", bd_type: "BusDayAdjustTypes") -> "Date":
return .adjust(dt, bd_type)
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	"Date"	-	-
bd_type	"BusDayAdjustTypes"-ı"Date":	-	-
return .adjust(dt, bd_type)	-	-	-

adjust

Adjust a payment date if it falls on a holiday according to the specified business day convention.

```
adjust(dt: Date, bd_type: BusDayAdjustTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-
bd_type	BusDayAdjustTypes	-	-

fast_adjust

Fast adjust a payment date using business day conventions.

```
fast_adjust(dt: Date, bd_type: BusDayAdjustTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-
bd_type	BusDayAdjustTypes	-	-

add_business_days

Returns a new date that is *num_days* business days after *Date*. All holidays in the chosen calendar are assumed not business days.

```
add_business_days(start_dt: Date, num_days: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	Date	-	-
num_days	int	-	-

is_business_day

Determines if a date is a business day according to the specified calendar. If it is it returns *True*, otherwise *False*.

```
is_business_day(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

is_holiday

Determines if a date is a Holiday according to the specified calendar. Weekends are not holidays unless the holiday falls on a weekend date.

```
is_holiday(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

holiday_weekend

Weekends by themselves are a holiday.

```
holiday_weekend(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

holiday_australia

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_australia(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

holiday_united_kingdom

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_united_kingdom(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_france

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_france(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_sweden

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_sweden(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_germany

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_germany(dt) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_switzerland

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_switzerland(dt) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_japan

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_japan(dt) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_new_zealand

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_new_zealand(dt) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_norway

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_norway(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_united_states

Only bank holidays. Weekends by themselves are not a holiday. This is a generic US calendar that contains the superset of holidays for bond markets, NYSE, and public holidays. For each of these and other categories there will be some variations.

```
holiday_united_states(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_canada

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_canada(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_italy

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_italy(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_target

Only bank holidays. Weekends by themselves are not a holiday.

```
holiday_target(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

holiday_none

No day is a holiday.

```
holiday_none(dt=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	None

get_holiday_list

generates a list of holidays in a specific year for the specified calendar. Useful for diagnostics.

```
get_holiday_list(year: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
year	float	-	-

easter_monday

Get the day in a given year that is Easter Monday. This is not easy to compute, so we rely on a pre-calculated array.

```
easter_monday(year: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
year	float	-	-

__str__*PLEASE ADD A FUNCTION DESCRIPTION*`__str__():`

The function arguments are described in the following table.

__repr__*PLEASE ADD A FUNCTION DESCRIPTION*`__repr__():`

The function arguments are described in the following table.

1.3 compounding

Enumerated Type: CompoundingTypes

This enumerated type has the following values:

- ZERO
- SIMPLE
- ANNUAL
- SEMI_ANNUAL
- TRI_ANNUAL
- QUARTERLY
- MONTHLY
- CONTINUOUS
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type

annual_frequency

PLEASE ADD A FUNCTION DESCRIPTION

```
annual_frequency(freq_type: CompoundingTypes) -> float:
"""
Return the number of payment periods per year for a given FrequencyType.
CONTINUOUS returns -1. ZERO returns 1 (to avoid division by zero).
"""
if not isinstance(freq_type, CompoundingTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description
freq_type	CompoundingTypes)-float:	-
"""	-	-
Return the number of payment periods per year for a given FrequencyType.	-	-
CONTINUOUS returns -1. ZERO returns 1 (to avoid division by zero).	-	-
"""	-	-

1.4 currency

Enumerated Type: CurrencyTypes

This enumerated type has the following values:

- USD
- EUR
- GBP
- CHF
- CAD
- AUD
- NZD
- DKK
- SEK
- HKD
- NONE

1.5 date

Class: Date

A date class to manage dates that is simple to use and includes a number of useful date functions used frequently in Finance.

Class: Date

A date class to manage dates that is simple to use and includes a number of useful date functions used frequently in Finance.

Date

PLEASE ADD A FUNCTION DESCRIPTION

```
Date(d, m, y, hh=0, mm=0, ss=0) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
d	-	-	-
m	-	-	-
y	-	-	-
hh	-	-	0
mm	-	-	0
ss	-	-	0

make

Normal safe constructor.

```
_make(d, m, y, hh=0, mm=0, ss=0) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
d	-	-	-
m	-	-	-
y	-	-	-
hh	-	-	0
mm	-	-	0
ss	-	-	0

make_fast

Fast constructor when excel_dt is already known.

```
_make_fast(cls, d, m, y, excel_dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
d	-	-	-
m	-	-	-
y	-	-	-
excel_dt	-	-	-

from_ymd_excel

PLEASE ADD A FUNCTION DESCRIPTION

```
from_ymd_excel(cls, d, m, y, excel_dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
d	-	-	-
m	-	-	-
y	-	-	-
excel_dt	-	-	-

from_excel

PLEASE ADD A FUNCTION DESCRIPTION

```
from_excel(excel_dt: int) -> "Date":
d, m, y = ymd_from_excel(int(excel_dt))
return Date(d, m, y)
#####
ssmethod
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
excel_dt	int)-i"Date":	-	-
d	-	-	-
m	-	-	-
y	-	-	ymd_from_excel(int(excel_dt))
return Date(d, m, y)	-	-	-
ssmethod	-	-	-

from_string

Create a Date from a date and format string. Example Input: `start_dt = Date('1-1-2018', '`

```
from_string(cls, date_string, format_string):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
date_string	-	-	-
format_string	-	-	-

from_date

Create a Date from a python `datetime.date` object or from a Numpy `datetime64` object. Example Input: `start_dt = Date.from_dt(datetime.date(2022, 11, 8))`

```
from_date(cls, date: Union[datetime.date, np.datetime64]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
date	datetime.date or np.datetime64	-	-

_refresh

Update Excel day count and weekday.

```
_refresh():
```

The function arguments are described in the following table.

__gt__

PLEASE ADD A FUNCTION DESCRIPTION

```
__gt__(other):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__lt__*PLEASE ADD A FUNCTION DESCRIPTION*`__lt__(other):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__ge__*PLEASE ADD A FUNCTION DESCRIPTION*`__ge__(other):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__le__*PLEASE ADD A FUNCTION DESCRIPTION*`__le__(other):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__sub__*PLEASE ADD A FUNCTION DESCRIPTION*`__sub__(other):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__rsub__*PLEASE ADD A FUNCTION DESCRIPTION*`__rsub__(other):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__eq__*PLEASE ADD A FUNCTION DESCRIPTION*`__eq__(other):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__hash__*PLEASE ADD A FUNCTION DESCRIPTION*`__hash__():`

The function arguments are described in the following table.

is_weekend*returns True if the date falls on a weekend.*`is_weekend():`

The function arguments are described in the following table.

is_eom*returns True if this date falls on a month end.*`is_eom():`

The function arguments are described in the following table.

eom

returns last date of month of this date.

```
eom() :
```

The function arguments are described in the following table.

add_hours

Returns a new date that is h hours after the Date.

```
add_hours(hours) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
hours	-	-	-

add_days

PLEASE ADD A FUNCTION DESCRIPTION

```
add_days(num_days: int = 1) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_days	int	-	1

add_weekdays

Returns a new date that is num_days working days after Date. Note that only weekends are taken into account. Other Holidays are not. If you want to include regional holidays then use add_business_days from the FinCalendar class.

```
add_weekdays(num_days: int) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_days	int	-	-

add_months

PLEASE ADD A FUNCTION DESCRIPTION

```
add_months(mm: Union[list, int]) -> Union["Date", list]:
    """Returns a new date that is mm months after the Date. If mm is an
    integer or float you get back a single date. If mm is a vector you get
    back a vector of dates."""
    scalar_flag = False
    if isinstance(mm, (int, float)):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
mm	list or int	-	-
"""Returns a new date that is mm months after the Date. If mm is an integer or float you get back a single date. If mm is a vector you get back a vector of dates."""	-	-	-
scalar_flag	-	-	False

add_years

Returns a new date that is yy years after the Date. If yy is an integer or float you get back a single date. If yy is a list you get back a vector of dates.

```
add_years(yy: Union[np.ndarray, float]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
yy	np.ndarray or float	-	-

next_cds_date

Returns a CDS date that is mm months after the Date. If no argument is supplied then the next CDS date after today is returned.

```
next_cds_date(mm: int = 0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
mm	int	-	0

third_wednesday_of_month

For a specific month and year this returns the day number of the 3rd Wednesday by scanning through dates in the third week.

```
third_wednesday_of_month(m: int, y: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
m	int	-	-
y	int	-	-

next_imm_date

This function returns the next IMM date after the current date This is a 3rd Wednesday of Jun, March, Sep or December. For an IMM contract the IMM date is the First Delivery Date of the futures contract.

```
next_imm_date():
```

The function arguments are described in the following table.

add_tenor

Return the date following the Date by a period given by the tenor which is a string consisting of a number and a letter, the letter being d, w, m, y for day, week, month or year. This is case independent. For example 10Y means 10 years while 120m also means 10 years. The date is NOT weekend or holiday calendar adjusted. This must be done AFTERWARDS.

```
add_tenor(tenor: Union[list, str, Tenor]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
tenor	list or str, Tenor	-	-

datetime

Returns a datetime of the date

```
datetime():
```

The function arguments are described in the following table.

str

returns a formatted string of the date

```
str():
```

The function arguments are described in the following table.

--repr--

returns a formatted string of the date

```
__repr__():
```

The function arguments are described in the following table.

_print

prints formatted string of the date.

```
_print():
```

The function arguments are described in the following table.

is_leap

PLEASE ADD A FUNCTION DESCRIPTION

```
is_leap(y: int) -> bool:
return (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)
#####
t(cache=True, fastmath=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
y	int)-bool:	-	-
return (y % 4	-	-	= 0 and y % 100 != 0) or (y % 400 == 0)
t(cache	-	-	True, fastmath=True)

excel_from_ymd

PLEASE ADD A FUNCTION DESCRIPTION

```
excel_from_ymd(d: int, m: int, y: int) -> int:
y_off = YEAR_OFFSETS[y - START_YEAR]
cum = LEAP_CUM if (y == 1900 or is_leap(y)) else NONLEAP_CUM
return y_off + cum[m - 1] + d
#####
t(cache=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
d	int	-	-
m	int	-	-
y	int)-int:	-	-
y_off	-	-	YEAR_OFFSETS[y - START_YEAR]
cum	-	-	LEAP_CUM if (y == 1900 or is_leap(y)) else NONLEAP_CUM
return y_off + cum[m - 1] + d	-	-	-
t(cache	-	-	True)

ymd_from_excel

Excel day number (1 = 1900-01-01) -> (d, m, y).

```
ymd_from_excel(excel_dt: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
excel_dt	int	-	-

is_leap_year

Test whether year y is a leap year - if so return True, else False

```
is_leap_year(y: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
y	int	-	-

parse_dt

Parse a date string into day, month, and year components.

```
parse_dt(date_str, date_format):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
date_str	-	-	-
date_format	-	-	-

weekday

Converts day count to a weekday based on Excel date

```

weekday(day_count) -> int:
"""Converts day_count to a weekday based on Excel date"""
week_day = (day_count + 5) % 7
return week_day
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
day_count) -> int	-	-	-
"""Converts day count to a weekday based on Excel date"""	-	-	-
week_day	-	-	(day_count + 5) % 7
return week_day	-	-	-

vectorisation_helper

Decorator to allow a function to be vectorised over an iterable

```
vectorisation_helper(func):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
func	-	-	-

daily_working_day_schedule

Returns a list of working dates between start_dt and end_dt. This function should be replaced by dateRange once add_tenor allows for working days.

```
daily_working_day_schedule(start_dt: Date, end_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	Date	-	-
end_dt	Date	-	-

datediff

Calculate the number of days between two Findates.

```
datediff(d1: Date, d2: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
d1	Date	-	-
d2	Date	-	-

from_datetime

Construct a Date from a datetime as this is often needed if we receive inputs from other Python objects such as Pandas dataframes.

```
from_datetime(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

days_in_month

Get the number of days in the month (1-12) of a given year y.

```
days_in_month(m, y):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
m	-	-	-
y	-	-	-

date_range

Returns a list of dates between start_dt (inclusive) and end_dt (inclusive). The tenor represents the distance between two consecutive dates and is set to daily by default.

```
date_range(start_dt: Date, end_dt: Date, tenor: str = "1D"):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	Date	-	-
end_dt	Date	-	-
tenor	str	-	"1D"

1.6 datelite

Class: DateLite

class DateLite:

DateLite

PLEASE ADD A FUNCTION DESCRIPTION

```
DateLite(excel_dt: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
excel_dt	int	-	-

add_days

PLEASE ADD A FUNCTION DESCRIPTION

```
add_days(n: int) -> "DateLite":
return DateLite(.excel_dt + n)
ekday using Excel convention (0=Mon      6=Sun)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
n	int)-ζ"DateLite":	-	-
return DateLite(.excel_dt + n)	-	-	-
ekday using Excel convention (0	-	-	Mon ... 6=Sun)

weekday

PLEASE ADD A FUNCTION DESCRIPTION

```
weekday() -> int:
return (.excel_dt + 5) % 7
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -ζ int	-	-	-
return (.excel_dt + 5) % 7	-	-	-

is_weekend*PLEASE ADD A FUNCTION DESCRIPTION*

```
is_weekend() -> bool:
wd = .weekday()
return wd == 5 or wd == 6
nvert back to full Date when needed
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> bool	-	-	-
wd	-	-	.weekday()
return wd	-	-	= 5 or wd == 6
nvert back to full Date when needed	-	-	-

to_date*PLEASE ADD A FUNCTION DESCRIPTION*

```
to_date() -> "Date":
return Date.from_excel(.excel_dt)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> "Date"	-	-	-
return Date.from_excel(.excel_dt)	-	-	-

__repr__*PLEASE ADD A FUNCTION DESCRIPTION*

```
__repr__():
```

The function arguments are described in the following table.

1.7 date_arrays

1.8 date_format

Enumerated Type: DateFormatTypes

This enumerated type has the following values:

- BLOOMBERG
- US.SHORT
- US.MEDIUM
- US.LONG
- US.LONGEST
- UK.SHORT
- UK.MEDIUM
- UK.LONG
- UK.LONGEST
- DATETIME
- G_DATE_TYPE_FORMAT:DateFormatTypes
- G_DATE_TYPE_FORMAT

set_date_format

Set the global date format.

```
set_date_format(fmt: str) -> None:
"""Set the global date format."""
global G_DATE_TYPE_FORMAT
G_DATE_TYPE_FORMAT = fmt
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fmt	str)-ζNone:	-	-
"""Set the global date format."""	-	-	-
global G_DATE_TYPE_FORMAT	-	-	-
G_DATE_TYPE_FORMAT	-	-	fmt

get_date_format

Return the current global date format.

```
get_date_format() -> str:
"""Return the current global date format."""
return G_DATE_TYPE_FORMAT
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i str		-	-
"""Return the current global date format."""	-	-	-
return G_DATE_TYPE_FORMAT	-	-	-

print_date_format

PLEASE ADD A FUNCTION DESCRIPTION

```
print_date_format():
```

The function arguments are described in the following table.

1.9 day_count

Enumerated Type: DayCountTypes

This enumerated type has the following values:

- ZERO
- THIRTY_360_BOND
- THIRTY_E_360
- THIRTY_E_360_ISDA
- THIRTY_E_PLUS_360
- ACT_ACT_ISDA
- ACT_ACT_ICMA
- ACT_365F
- ACT_360
- ACT_365L
- SIMPLE
- self._type
- dt3:Date
- freq_type:FrequencyTypes
- is_termination_date:bool
- affectnormaldatesforwhichhh
- d1
- m1
- y1
- d2
- m2
- y2
- num
- den
- ifself._type
- ifd1
- d1
- ifd2
- d2
- num
- den
- acc_factor
- elifself._type
- ifd1
- d1
- ifd2
- d2
- num
- den
- acc_factor
- elifself._type

- ifd1
- d1
- last_day_of_feb1
- d1
- ifd2
- d2
- last_day_of_feb2
- d2
- num
- den
- acc_factor
- elifself._type
- ifd1
- d1
- ifd2
- m2
- d2
- num
- den
- acc_factor
- denom1
- denom1
- denom2
- denom2
- ify1
- num
- den
- acc_factor
- day_years_1
- day_years_2
- acc_factor1
- acc_factor2
- year_diff
- num
- den
- acc_factor
- elifself._type
- freq
- num
- den
- acc_factor
- elifself._type
- num
- den
- acc_factor
- elifself._type

- num
- den
- acc_factor
- elifself._type
- frequency
- y3
- y3
- num
- den
- feb29
- feb29
- feb29
- iffrequency
- iff feb29; dt1 and feb29;
- den
- den
- acc_factor
- elifself._type
- num
- den
- acc_factor

Class: DayCount

Calculate the fractional day count between two dates according to a specified day count convention.

DayCount

Create Day Count convention by passing in the Day Count Type.

```
DayCount(dcc_type: DayCountTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dcc_type	DayCountTypes	-	-

year_frac

PLEASE ADD A FUNCTION DESCRIPTION

```
year_frac(dt1: Date, # Start of coupon period
dt2: Date, # Settlement (for bonds) or period end(swaps)
dt3: Date = None, # End of coupon period for accrued
freq_type: FrequencyTypes = FrequencyTypes.ANNUAL,
is_termination_date: bool = False,
# Is dt2 a term date
"""This method performs two functions:
```

```

1) It calculates the year fraction between dates dt1 and dt2 using the
specified day count convention which is useful for calculating year
fractions for Libor products whose flows are day count adjusted. In
this case we will set dt3 to be None
2) This function is also for calculating bond accrued where dt1 is the
last coupon date, dt2 is the settlement date of the bond and date dt3
must be set to the next coupon date. You will also need to provide a
coupon frequency for some conventions.
Note that if the date is intra-day, i.e. hh, mm and ss do not equal zero
then that is used in the calculation of the year frac. This avoids
discontinuities for short-dated intraday products. It should not
affect normal dates for which hh=mm=ss=0.
This seems like a useful source:
https://www.eclipsesoftware.biz/DayCountConventions.html
Wikipedia also has a decent survey of the conventions
https://en.wikipedia.org/wiki/Day\_count\_convention
and
http://data.cbonds.info/files/cbondscalculator/Calculator.pdf
"""
d1 = dt1.d
m1 = dt1.m
y1 = dt1.y
d2 = dt2.d
m2 = dt2.m
y2 = dt2.y
num = 0
den = 0
if ._type == DayCountTypes.THIRTY_360_BOND:
    # It is in accordance with section 4.16(f) of ISDA 2006 Definitions
    # Also known as 30/360, Bond Basis, 30A/360, 30-360 US Municipal
    # This method does not consider February as a special case.
    if d1 == 31:
        d1 = 30
    if d2 == 31 and d1 == 30:
        d2 = 30
    num = 360 * (y2 - y1) + 30 * (m2 - m1) + (d2 - d1)
    den = 360
    acc_factor = num / den
    return acc_factor, num, den
elif ._type == DayCountTypes.THIRTY_E_360:
    # This is in section 4.16(g) of ISDA 2006 Definitions
    # Also known as 30/360 Eurobond, 30/360 ISMA, 30/360 ICMA,
    # 30/360 European, Special German, Eurobond basis (ISDA 2006)
    # Unlike 30/360 BOND the adjustment to dt2 does not depend on dt1
    if d1 == 31:
        d1 = 30
    if d2 == 31:
        d2 = 30
    num = 360 * (y2 - y1) + 30 * (m2 - m1) + (d2 - d1)
    den = 360
    acc_factor = num / den
    return acc_factor, num, den
elif ._type == DayCountTypes.THIRTY_E_360_ISDA:
    # This is 30E/360 (ISDA 2000), 30E/360 (ISDA) section 4.16(h)
    # of ISDA 2006 Definitions, German, Eurobond basis (ISDA 2000)
    if d1 == 31:
        d1 = 30

```

```
last_day_of_feb1 = is_last_day_of_feb(dt1)
if last_day_of_feb1 is True:
    d1 = 30
if d2 == 31:
    d2 = 30
last_day_of_feb2 = is_last_day_of_feb(dt2)
if last_day_of_feb2 is True and is_termination_date is False:
    d2 = 30
num = 360 * (y2 - y1) + 30 * (m2 - m1) + (d2 - d1)
den = 360
acc_factor = num / den
return acc_factor, num, den
elif ._type == DayCountTypes.THIRTY_E_PLUS_360:
    if d1 == 31:
        d1 = 30
    if d2 == 31:
        m2 = m2 + 1 # May roll to 13 but we are doing a difference
        d2 = 1
    num = 360 * (y2 - y1) + 30 * (m2 - m1) + (d2 - d1)
    den = 360
    acc_factor = num / den
    return acc_factor, num, den
elif ._type in [DayCountTypes.ACT_ACT_ISDA, DayCountTypes.ZERO]:
    if is_leap_year(y1):
```

The function arguments are described in the following table.

Argument Name	Type
dt1	Date
dt2	Date
dt3	Date
freq_type	FrequencyTypes
is_termination_date	bool
"""This method performs two functions	
1) It calculates the year fraction between dates dt1 and dt2 using the specified day count convention which is useful for calculating year fractions for Libor products whose flows are day count adjusted. In this case we will set dt3 to be None	-
2) This function is also for calculating bond accrued where dt1 is the last coupon date	-
dt2 is the settlement date of the bond and date dt3 must be set to the next coupon date. You will also need to provide a coupon frequency for some conventions.	-
Note that if the date is intra-day i.e. hh mm and ss do not equal zero then that is used in the calculation of the year frac. This avoids discontinuities for short-dated intraday products. It should not affect normal dates for which hh	-
This seems like a useful source	
https	//www.eclipsesoftware.biz/DayCountConventions
Wikipedia also has a decent survey of the conventions	-
https	//en.wikipedia.org/wiki/Day_count_conventions
and	-
http	//data.cbonds.info/files/cbondscalculator/Calculators
"""	-
d1	-
m1	-
y1	-
d2	-
m2	-
y2	-
num	-
den	-
if .type	-
if d1	-
d1	-
if d2	-
d2	-
num	-
den	-
acc_factor	-
return acc_factor	-
num	-
den	-
elif .type	-
if d1	-
d1	-
if d2	-

__repr__

Returns the calendar type as a string.

```
__repr__():
```

The function arguments are described in the following table.

is_last_day_of_feb

Return True if the date is the last day of February.

```
is_last_day_of_feb(dt: Date) -> bool:
"""Return True if the date is the last day of February."""
if dt.m != 2:
    return False
return dt.d == 29 if is_leap_year(dt.y) else dt.d == 28
#####
THIRTY_360_BOND = 1 # 30E/360 ISDA 2006 4.16f, German, Eurobond(ISDA2000)
THIRTY_E_360 = 2 # ISDA 2006 4.16(g) 30/360 ISMA, ICMA
THIRTY_E_360_ISDA = 3 # ISDA 2006 4.16(h)
THIRTY_E_PLUS_360 = 4 # ROLLS D2 TO NEXT MONTH IF D2 = 31
ACT_ACT_ISDA = 5 # SPLITS ACCRUAL PERIOD INTO LEAP YEAR AND NON LEAP YEAR
ACT_ACT_ICMA = 6 # METHOD FOR ALL US TREASURY NOTES AND BONDS
ACT_365F = 7 # Denominator is always Fixed at 365, even in a leap year
ACT_360 = 8
ACT_365L = 9 # the 29 Feb is counted if it is in the date range
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
dt	Date)-ζbool:	-
"""Return True if the date is the last day of February."""	-	-
if dt.m !	-	-
return False	-	-
return dt.d	-	-
THIRTY_360_BOND	-	30E/360 ISDA 2006 4.16f, German, Eu
THIRTY_E_360	-	ISDA 2006 4.16(g) 30/360 ISM
THIRTY_E_360_ISDA	-	ISDA 2006 4.16(h)
THIRTY_E_PLUS_360	-	ROLLS D2 TO NEXT MONTH
ACT_ACT_ISDA	-	SPLITS ACCRUAL PERIOD INTO LEAP YE
ACT_ACT_ICMA	-	METHOD FOR ALL US TREASURY N
ACT_365F	-	Denominator is always Fixed at 365, c
ACT_360	-	-
ACT_365L	-	the 29 Feb is counted if it is in th

1.10 distribution

Class: *FinDistribution*

FinDistribution

Initialise FinDistribution with x values and associated vector of density times dx values.

```
FinDistribution(x, y):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
y	-	-	-

sum

This should equal 1.0 for the entire distribution.

```
sum() :
```

The function arguments are described in the following table.

1.11 error

Class: FinError(Exception)

Simple error class specific to FinPy. Need to decide how to handle FinancePy errors. Work in progress.

FinError

Create FinError object by passing a message string.

```
FinError(message: str):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
message	str	-	-

_print

PLEASE ADD A FUNCTION DESCRIPTION

```
_print():
```

The function arguments are described in the following table.

_hide_traceback

Avoid long error message inside Jupyter Notebook

```
_hide_traceback(*args, **kwargs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
*args	-	-	-
**kwargs	-	-	-

func_name

Get error message

```
func_name():
```

The function arguments are described in the following table.

suppress_traceback*Avoid long error message*

```
suppress_traceback() :
```

The function arguments are described in the following table.

1.12 frequency

Enumerated Type: FrequencyTypes

This enumerated type has the following values:

- ZERO
- SIMPLE
- ANNUAL
- SEMI_ANNUAL
- TRI_ANNUAL
- QUARTERLY
- MONTHLY
- CONTINUOUS
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type
- iffreq_type

annual_frequency

PLEASE ADD A FUNCTION DESCRIPTION

```
annual_frequency(freq_type: FrequencyTypes) -> float:
"""
Return the number of payment periods per year for a given FrequencyType.
CONTINUOUS returns -1. ZERO returns 1 (to avoid division by zero).
"""
if not isinstance(freq_type, FrequencyTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
freq_type	FrequencyTypes)-float:	-	-
"""	-	-	-
Return the number of payment periods per year for a given FrequencyType.	-	-	-
CONTINUOUS returns -1. ZERO returns 1 (to avoid division by zero).	-	-	-
"""	-	-	-

1.13 global_types

Enumerated Type: FinLongShort

This enumerated type has the following values:

- LONG
- SHORT

Enumerated Type: DoubleBarrierTypes

This enumerated type has the following values:

- KNOCK_IN
- KNOCK_OUT

Enumerated Type: OptionTypes

This enumerated type has the following values:

- EUROPEAN_CALL
- EUROPEAN_PUT
- AMERICAN_CALL
- AMERICAN_PUT
- DIGITAL_CALL
- DIGITAL_PUT
- ASIAN_CALL
- ASIAN_PUT
- COMPOUND_CALL
- COMPOUND_PUT

Enumerated Type: BarrierTypes

This enumerated type has the following values:

- DOWN_AND_OUT_CALL
- DOWN_AND_IN_CALL
- UP_AND_OUT_CALL
- UP_AND_IN_CALL
- UP_AND_OUT_PUT
- UP_AND_IN_PUT
- DOWN_AND_OUT_PUT
- DOWN_AND_IN_PUT

Enumerated Type: CapFloorTypes

This enumerated type has the following values:

- CAP
- FLOOR

Enumerated Type: SwapTypes

This enumerated type has the following values:

- PAY
- RECEIVE

Enumerated Type: ReturnTypes

This enumerated type has the following values:

- TOTAL_RETURN
- PRICE_RETURN

Enumerated Type: ExerciseTypes

This enumerated type has the following values:

- EUROPEAN
- BERMUDAN
- AMERICAN

Enumerated Type: SolverTypes

This enumerated type has the following values:

- CONJUGATE_GRADIENT
- NELDER_MEAD
- NELDER_MEAD_NUMBA

Enumerated Type: TouchOptionTypes

This enumerated type has the following values:

- DOWN_AND_IN_CASH_AT_HIT
- UP_AND_IN_CASH_AT_HIT
- DOWN_AND_IN_CASH_AT_EXPIRY
- UP_AND_IN_CASH_AT_EXPIRY
- DOWN_AND_OUT_CASH_OR_NOTHING
- UP_AND_OUT_CASH_OR_NOTHING
- DOWN_AND_IN_ASSET_AT_HIT
- UP_AND_IN_ASSET_AT_HIT
- DOWN_AND_IN_ASSET_AT_EXPIRY
- UP_AND_IN_ASSET_AT_EXPIRY
- DOWN_AND_OUT_ASSET_OR_NOTHING
- UP_AND_OUT_ASSET_OR_NOTHING

1.14 global_vars

1.15 helpers

`_func_name`

Extract calling function name - using a protected method is not that advisable but calling `inspect.stack` is so slow it must be avoided.

```
_func_name():
```

The function arguments are described in the following table.

`grid_index`

PLEASE ADD A FUNCTION DESCRIPTION

```
grid_index(t, grid_times):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
grid_times	-	-	-

`beta_vector_to_corr_matrix`

Convert a one-factor vector of factor weights to a square correlation matrix.

```
beta_vector_to_corr_matrix(betas):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
betas	-	-	-

`pv01_times`

Calculate a bond style pv01 by calculating remaining coupon times for a bond with t years to maturity and a coupon frequency of f . The order of the list is reverse time order - it starts with the last coupon date and ends with the first coupon date.

```
pv01_times(t: float, f: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float	-	-
f	float	-	-

times_from_dates

PLEASE ADD A FUNCTION DESCRIPTION

```
times_from_dates(
dt: Union[Date, list], value_dt: Date, dc_type: DayCountTypes = None
"""If a single date is passed in then return the year from valuation date
but if a whole vector of dates is passed in then convert to a vector of
times from the valuation date. The output is always a numpy vector of times
which has only one element if the input is only one date."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation date is not a Date")
if dc_type is None:
    dc_counter = None
else:
    dc_counter = DayCount(dc_type)
if isinstance(dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
dt	Date or list	-	-
value_dt	Date	-	-
dc_type	DayCountTypes	-	No
"""If a single date is passed in then return the year from valuation date	-	-	-
but if a whole vector of dates is passed in then convert to a vector of	-	-	-
times from the valuation date. The output is always a numpy vector of times	-	-	-
which has only one element if the input is only one date."""	-	-	-
if isinstance(value_dt, Date) is False	-	-	-
raise FinError("Valuation date is not a Date")	-	-	-
if dc_type is None	-	-	-
dc_counter	-	-	No
else	-	-	-
dc_counter	-	-	DayCount

check_vector_differences

Compare two vectors elementwise to see if they are more different than tolerance.

```
check_vector_differences(x: np.ndarray, y: np.ndarray, tol: float = 1e-6):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-
y	np.ndarray	-	-
tol	float	-	1e-6

check_dt

Check that input d is a Date.

```
check_dt(d: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
d	Date	-	-

dump

PLEASE ADD A FUNCTION DESCRIPTION

```
dump(obj):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
obj	-	-	-

print_tree

Function that prints a binomial or trinomial tree to screen for the purpose of debugging.

```
print_tree(array: np.ndarray, depth: int = None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
array	np.ndarray	-	-
depth	int	-	None

input_time

Validates a time input in relation to a curve. If it is a float then it returns a float as long as it is positive. If it is a Date then it converts it to a float. If it is a Numpy array then it returns the array as long as it is all positive.

```
input_time(dt: Date, curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-
curve	-	-	-

listdiff

Calculate a vector of differences between two equal sized vectors.

```
listdiff(a: np.ndarray, b: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	np.ndarray	-	-
b	np.ndarray	-	-

dotproduct

Fast calculation of dot product using Numba.

```
dotproduct(x_vector: np.ndarray, y_vector: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x_vector	np.ndarray	-	-
y_vector	np.ndarray	-	-

frange

fast range function that takes start value, stop value and step.

```
frange(start: int, stop: int, step: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start	int	-	-
stop	int	-	-
step	int	-	-

normalise_weights

Normalise a vector of weights so that they sum up to 1.0.

```
normalise_weights(wt_vector: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
wt_vector	np.ndarray	-	-

label_to_string

Format label/value pairs for a unified formatting.

```
label_to_string(
label: str,
value: Union[float, str],
separator: str = "\\n",
list_format: bool = False,
"""Format label/value pairs for a unified formatting."""
# Format option for lists such that all values are aligned:
# Label: value1
#       value2
#       ...
label = str(label)
if list_format and isinstance(value, list) and len(value) > 0:
    s = label + ":_"
    label_spacing = "_" * len(s)
    s += str(value[0])
    for v in value[1:]:
        s += "\\n" + label_spacing + str(v)
    s += separator
    return s
return f"{label}:_{value}{separator}"
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
label	str	-	-
value	float or str	-	-
separator	str	-	"
n"			
list_format	bool	-	False
"""Format label/value pairs for a unified formatting."""	-	-	-
label	-	-	str(label)
if list_format and isinstance(value, list) and len(value) > 0	-	-	-
s	-	-	label + ": "
label_spacing	-	-	" " * len(s)
s +	-	-	str(value[0])
for v in value[1	]:	-	-
s +	-	-	"
n" + label_spacing + str(v)			
s +	-	-	separator
return s	-	-	-
return f"label	valueseparator"	-	-

table_to_string

Format a 2D array into a table-like string.

```
table_to_string(header: str, value_table, float_precision="10.7f"):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
header	str	-	-
value_table	-	-	-
float_precision	-	-	"10.7f"

format_table

Format a table without PrettyTable. Left-align text, right-align numbers.

```
format_table(header, rows):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
header	-	-	-
rows	-	-	-

to_usable_type

Convert a type such that it can be used with 'isinstance'

```
to_usable_type(t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-

uniform_to_default_time

Fast mapping of a uniform random variable to a default time given a survival probability curve.

```
uniform_to_default_time(u, t, v):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
u	-	-	-
t	-	-	-
v	-	-	-

accrued_tree

Fast calculation of accrued interest using an Actual/Actual type of convention. This does not calculate according to other conventions.

```
accrued_tree(grid_times: np.ndarray, grid_flows: np.ndarray, face: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
grid_times	np.ndarray	-	-
grid_flows	np.ndarray	-	-
face	float	-	-

check_argument_types

Check that all values passed into a function are of the same type as the function annotations. If a value has not been annotated, it will not be checked.

```
check_argument_types(func, values):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
func	-	-	-
values	-	-	-

1.16 latex

convert_to_latex_table

PLEASE ADD A FUNCTION DESCRIPTION

```
convert_to_latex_table(txt, sep="_", header_list=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
txt	-	-	-
sep	-	-	" "
header_list	-	-	None

1.17 math

accrued_interpolator

PLEASE ADD A FUNCTION DESCRIPTION

```
accrued_interpolator(
t_set: float, # Settlement time in years
cpn_times: np.ndarray,
cpn_amounts: np.ndarray,
"""Fast calculation of accrued interest using an Actual/Actual type of
convention. This does not calculate according to other conventions."""
num_cpns = len(cpn_times)
for i in range(1, num_cpns):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t_set	float	Settlement time in years	-
cpn_times	np.ndarray	-	-
cpn_amounts	np.ndarray	-	-
"""Fast calculation of accrued interest using an Actual/Actual type of convention. This does not calculate according to other conventions."""	-	-	-
num_cpns	-	-	len(cpn.t

is_leap_year

Test whether year y is a leap year - if so return True, else False

```
is_leap_year(y: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
y	int	-	-

scale

Scale all of the elements of an array by the same amount factor.

```
scale(x: np.ndarray, factor: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-
factor	float	-	-

test_monotonicity

Check that an array of doubles is monotonic and strictly increasing.

```
test_monotonicity(x: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-

test_range

Check that all of the values of an array fall between a lower and upper bound.

```
test_range(x: np.ndarray, lower: float, upper: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-
lower	float	-	-
upper	float	-	-

maximum

Determine the array in which each element is the maximum of the corresponding element in two equally length arrays a and b.

```
maximum(a: np.ndarray, b: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	np.ndarray	-	-
b	np.ndarray	-	-

maxaxis

Perform a search for the vector of maximum values over an axis of a 2D Numpy Array

```
maxaxis(s: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	np.ndarray	-	-

minaxis

Perform a search for the vector of minimum values over an axis of a 2D Numpy Array

```
minaxis(s: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	np.ndarray	-	-

covar

Calculate the Covariance of two arrays of numbers. TODO: check that this works well for Numpy Arrays and add NUMBA function signature to code. Do test of timings against Numpy.

```
covar(a: np.ndarray, b: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	np.ndarray	-	-
b	np.ndarray	-	-

pair_gcd

Determine the Greatest Common Divisor of two integers using Euclid's algorithm. TODO - compare this with math.gcd(a,b) for speed. Also examine to see if I should not be declaring inputs as integers for NUMBA.

```
pair_gcd(v1: float, v2: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
v1	float	-	-
v2	float	-	-

heaviside

Calculate the Heaviside function for x

```
heaviside(x: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-

frange

Calculate a range of values from start in steps of size step. Ends as soon as the value equals or exceeds stop.

```
frange(start: int, stop: int, step: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start	int	-	-
stop	int	-	-
step	int	-	-

normcdf_prime

Calculate the first derivative of the Cumulative Normal CDF which is simply the PDF of the Normal Distribution

```
normcdf_prime(x: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-

normpdf

Calculate the probability density function for a Gaussian (Normal) function at value x

```
normpdf(x: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-

normcdf

Fast Normal CDF function based on Hull OFAODS 4th Edition Page 252. This function is accurate to 6 decimal places.

```
normcdf(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

normcdf_vect*PLEASE ADD A FUNCTION DESCRIPTION*`normcdf_vect(x):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

normcdf_prime_vect*PLEASE ADD A FUNCTION DESCRIPTION*`normcdf_prime_vect(x):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

normcdf_integrate

Calculation of Normal Distribution CDF by simple integration which can become exact in the limit of the number of steps tending towards infinity. This function is used for checking as it is slow since the number of integration steps is currently hardcoded to 10,000.

`normcdf_integrate(x: float):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-

normcdf_slow

Calculation of Normal Distribution CDF accurate to 1d-15. This method is faster than integration but slower than other approximations. Reference: J.L. Schonfelder, Math Comp 32(1978), pp 1232-1240.

`normcdf_slow(z: float):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
z	float	-	-

phi3

Bivariate Normal CDF function to upper limits b_1 and b_2 which uses integration to perform the inner-most integral. This may need further refinement to ensure it is optimal as the current range of integration is from -7 and the integration steps are $dx = 0.001$. This may be excessive.

```
phi3(b1: float, b2: float, b3: float, r12: float, r13: float, r23: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
b1	float	-	-
b2	float	-	-
b3	float	-	-
r12	float	-	-
r13	float	-	-
r23	float	-	-

norminvcdf

This algorithm computes the inverse Normal CDF and is based on the algorithm found at (<http://home.online.no/~pjacklam/notes/invnorm/>) which is by John Herrero (3-Jan-03)

```
norminvcdf(p):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
p	-	-	-

M

PLEASE ADD A FUNCTION DESCRIPTION

```
M(a, b, c):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	-	-	-
b	-	-	-
c	-	-	-

phi2

Drezner and Wesolowsky implementation of bi-variate normal

```
phi2(h1, hk, r):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
h1	-	-	-
hk	-	-	-
r	-	-	-

cholesky

Numba-compliant wrapper around Numpy cholesky function.

```
cholesky(rho):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
rho	-	-	-

corr_matrix_generator

Utility function to generate a full rank $n \times n$ correlation matrix with a flat correlation structure and value rho.

```
corr_matrix_generator(rho, n):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
rho	-	-	-
n	-	-	-

npv

Function to calculate the npv given irr and cashflow. It can be used to do root search in IRR. times_cfs is a list of tuples. The tuple is in the form of (years from first date, cashflow)

```
npv(irr: float, times_cfs: list):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
irr	float	-	-
times_cfs	list	-	-

band_matrix_multiplication

PLEASE ADD A FUNCTION DESCRIPTION

```
band_matrix_multiplication(a, m1, m2, b):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	-	-	-
m1	-	-	-
m2	-	-	-
b	-	-	-

solve_tridiagonal_matrix

Solve $A u = r$ for vector u when A is tridiagonal The matrix A is split into vectors a , b , and c contain the three non-zero elements of each row of A , in order. i.e. the vector b is the main diagonal of A , with a and c the elements either side of the main diagonal. Note that $a[0]$ and $c[-1]$ are not used, and so can be any value.

```
solve_tridiagonal_matrix(a_matrix, r):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a_matrix	-	-	-
r	-	-	-

transpose_tridiagonal_matrix

PLEASE ADD A FUNCTION DESCRIPTION

```
transpose_tridiagonal_matrix(a_matrix):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a_matrix	-	-	-

1.18 polyfit

`_coeff_mat_numpy`

PLEASE ADD A FUNCTION DESCRIPTION

```
_coeff_mat_numpy(x: np.ndarray, deg: int) -> np.ndarray:
# fast and stable: column order matches coefficients of increasing power
# (we'll reverse later to match your API)
# vander with increasing=True gives [1, x, x^2, ...]
return np.vander(x, N=deg + 1, increasing=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-
deg	int)-ζnp.ndarray:	-	-
return np.vander(x, N	-	-	deg + 1, increasing=True)

`_fit_poly_numpy`

PLEASE ADD A FUNCTION DESCRIPTION

```
_fit_poly_numpy(x: np.ndarray, y: np.ndarray, deg: int) -> np.ndarray:
a = _coeff_mat_numpy(x, deg)
# numpy lstsq: returns coeffs for increasing powers; reverse to highest-first
p_inc, *_ = np.linalg.lstsq(a, y, rcond=None)
return p_inc[::-1]
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-
y	np.ndarray	-	-
deg	int)-ζnp.ndarray:	-	-
a	-	-	_coeff_mat_numpy(x, deg)
p_inc	-	-	-
*_	-	-	np.linalg.lstsq(a, y, rcond=None)
return p_inc[	:-1]	-	-

`_eval_polynomial_numpy`

PLEASE ADD A FUNCTION DESCRIPTION

```
_eval_polynomial_numpy(p: np.ndarray, x: np.ndarray) -> np.ndarray:
# Horner with highest-order first (your API)
result = np.zeros_like(x, dtype=np.result_type(p.dtype, x.dtype))
for coeff in p:
    result = x * result + coeff
```

```
return result
----- Lazy Numba wiring -----
```

The function arguments are described in the following table.

Argument Name	Type	Description
p	np.ndarray	-
x	np.ndarray)-, np.ndarray:	-
result	-	np.zeros_like(x,
for coeff in p	-	-
result	-	-
return result	-	-
----- Lazy Numba wiring -----	-	-

_enable_numba

Import numba and build jitted implementations on demand.

```
_enable_numba() -> None:
"""Import numba and build jitted implementations on demand."""
global _fit_poly_impl, _eval_poly_impl
import numba as nb # heavy import; done only once, on first enable
# Numba-compatible pieces
@nb.njit("f8[:, :](f8[:, :], i8)", cache=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description
) -, None	-	-
"""Import numba and build jitted implementations on demand."""	-	-
global _fit_poly_impl	-	-
_eval_poly_impl	-	-
import numba as nb	-	heavy import; done only once, on first c
@nb.njit("f8[or :](f8[:, i8	-	-

_ensure_impls

Choose NumPy or Numba impls; import Numba lazily if requested.

```
_ensure_impls():
```

The function arguments are described in the following table.

fit_poly

Fit polynomial of degree 'deg' to points (x, y). Returns coefficients with highest order first (same as original API).

```
fit_poly(x: np.ndarray, y: np.ndarray, deg: int) -> np.ndarray:
"""
Fit_polynomial_of_degree_'deg'_to_points_(x, y).
Returns_coefficients_with_highest_order_first_(same_as_original_API).
"""
_ensure_impls()
return _fit_poly_impl(x.astype(np.float64), y.astype(np.float64), int(deg))
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	np.ndarray	-	-
y	np.ndarray	-	-
deg	int)-ζnp.ndarray:	-	-
"""	-	-	-
Fit polynomial of degree 'deg' to points (x, y).	-	-	-
Returns coefficients with highest order first (same as original API).	-	-	-
"""	-	-	-
_ensure_impls()	-	-	-
return _fit_poly_impl(x.astype(np.float64), y.astype(np.float64), int(deg))	-	-	-

eval_polynomial

Evaluate polynomial with coefficients 'p' (highest-first) at points 'x'.

```
eval_polynomial(p: np.ndarray, x: np.ndarray) -> np.ndarray:
"""
Evaluate_polynomial_with_coefficients_'p'_(highest-first)_at_points_'x'.
"""
_ensure_impls()
p = np.asarray(p, dtype=np.float64)
x = np.asarray(x, dtype=np.float64)
return _eval_poly_impl(p, x)
```

The function arguments are described in the following table.

Argument Name	Type	Description
p	np.ndarray	-
x	np.ndarray)-&np.ndarray:	-
"""	-	-
Evaluate polynomial with coefficients 'p' (highest-first) at points 'x'.	-	-
"""	-	-
_ensure_impls()	-	-
p	-	- np.asarray
x	-	- np.asarray
return _eval_poly_impl(p, x)	-	-

1.19 schedule

Class: Schedule

A schedule is a set of dates generated according to ISDA standard rules which starts on the next date after the effective date and runs up to a termination date. Dates are adjusted to a provided calendar. The zeroth element is the previous coupon date (PCD) and the first element is the Next Coupon Date (NCD). We reference ISDA 2006.

Schedule

Create Schedule object which calculates a sequence of dates following the ISDA convention for fixed income products, mainly swaps. If the date gen rule type is FORWARD we get the unadjusted dates by stepping forward from the effective date in steps of months determined by the period tenor - i.e. the number of months between payments. We stop before we go past the termination date. If the date gen rule type is BACKWARD we get the unadjusted dates by stepping backward from the termination date in steps of months determined by the period tenor - i.e. the number of months between payments. We stop before we go past the effective date. - If the EOM flag is false, and the start date is on the 31st then the the unadjusted dates will fall on the 30 if a 30 is a previous date. - If the EOM flag is false and the start date is 28 Feb then all unadjusted dates will fall on the 28th. - If the EOM flag is false and the start date is 28 Feb then all unadjusted dates will fall on their respective EOM. We then adjust all of the flow dates if they fall on a weekend or holiday according to the calendar specified. These dates are adjusted in accordance with the business date adjustment. The effective date is never adjusted as it is not a payment date. The termination date is not automatically business day adjusted in a swap - assuming it is a holiday date. This must be explicitly stated in the trade confirm. However, it is adjusted in a CDS contract as standard. Inputs first_dt and next_to_last_dt are for managing long payment stubs at the start and end of the swap but *have not yet been implemented*. All stubs are currently short, either at the start or end of swap.

```
Schedule(effective_dt: Date, # Also known as the start date
# This is UNADJUSTED (set flag to adjust it)
termination_dt: Date,
freq_type: FrequencyTypes = FrequencyTypes.ANNUAL,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
adjust_termination_dt: bool = True, # Default is to adjust
end_of_month: bool = False, # All flow dates are EOM if True
first_dt=None, # First coupon date
next_to_last_dt=None,
# Penultimate coupon date
"""Create Schedule object which calculates a sequence of dates
following the ISDA convention for fixed income products, mainly swaps.
If the date gen rule type is FORWARD we get the unadjusted dates by
stepping forward from the effective date in steps of months determined
by the period tenor - i.e. the number of months between payments. We
stop before we go past the termination date.
If the date gen rule type is BACKWARD we get the unadjusted dates by
stepping backward from the termination date in steps of months
determined by the period tenor - i.e. the number of months between
payments. We stop before we go past the effective date.
- If the EOM flag is false, and the start date is on the 31st then the
```

```

the_unadjusted_dates_will_fall_on_the_30_if_a_30_is_a_previous_date.
- If the EOM_flag is false and the start_date is 28_Feb then all
unadjusted_dates_will_fall_on_the_28th.
- If the EOM_flag is false and the start_date is 28_Feb then all
unadjusted_dates_will_fall_on_their_respective_EOM.
We then adjust all of the flow_dates if they fall on a weekend or
holiday according to the calendar specified. These dates are adjusted
in accordance with the business_date adjustment.
The effective_date is never adjusted as it is not a payment_date.
The termination_date is not automatically business_day adjusted in a
swap - assuming it is a holiday date. This must be explicitly stated in
the trade confirm. However, it is adjusted in a CDS contract as
standard.
Inputs first_dt and next_to_last_dt are for managing long payment
stubs at the start and end of the swap but *have not yet been
implemented*. All stubs are currently short, either at the start or
end of swap. """
check_argument_types(.Schedule, locals())
if effective_dt >= termination_dt:
    raise FinError("Effective_date must be before termination_date.")
.effective_dt = effective_dt
.termination_dt = termination_dt
if first_dt is None:
    .first_dt = effective_dt
else:
    if first_dt > effective_dt and first_dt < termination_dt:
        .first_dt = first_dt
        print("FIRST_DATE_NOT_IMPLEMENTED") # TODO
    else:
        raise FinError("First_date must be after effective_date and"
            + " before termination_date"
        )
if next_to_last_dt is None:
    .next_to_last_dt = termination_dt
else:
    if next_to_last_dt > effective_dt and next_to_last_dt < termination_dt:
        .next_to_last_dt = next_to_last_dt
        print("NEXT_TO_LAST_DATE_NOT_IMPLEMENTED") # TODO
    else:
        raise FinError("Next_to_last_date must be after effective_"
            + "date and before termination_date"
        )
.freq_type = freq_type
.cal_type = cal_type
.bd_type = bd_type
.dg_type = dg_type
.adjust_termination_dt = adjust_termination_dt
if end_of_month is True:
    .end_of_month = True
else:
    .end_of_month = False
.adjusted_dts = None
.generate()
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Also known as the start
termination_dt	Date	-
freq_type	FrequencyTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
adjust_termination_dt	bool	Default is to adjust
end_of_month	bool	All flow dates are EOM
first_dt	-	First coupon date
next_to_last_dt	-	-
"""Create Schedule object which calculates a sequence of dates following the ISDA convention for fixed income products mainly swaps.	-	-
If the date gen rule type is FORWARD we get the unadjusted dates by stepping forward from the effective date in steps of months determined by the period tenor - i.e. the number of months between payments. We stop before we go past the termination date.	-	-
If the date gen rule type is BACKWARD we get the unadjusted dates by stepping backward from the termination date in steps of months determined by the period tenor - i.e. the number of months between payments. We stop before we go past the effective date.	-	-
- If the EOM flag is false and the start date is on the 31st then the the unadjusted dates will fall on the 30 if a 30 is a previous date.	-	-
- If the EOM flag is false and the start date is 28 Feb then all unadjusted dates will fall on the 28th.	-	-
- If the EOM flag is false and the start date is 28 Feb then all unadjusted dates will fall on their respective EOM.	-	-
We then adjust all of the flow dates if they fall on a weekend or holiday according to the calendar specified. These dates are adjusted in accordance with the business date adjustment.	-	-
The effective date is never adjusted as it is not a payment date.	-	-
The termination date is not automatically business day adjusted in a swap - assuming it is a holiday date. This must be explicitly stated in the trade confirm. However	-	-
it is adjusted in a CDS contract as standard.	-	-
Inputs first_dt and next_to_last_dt are for managing long payment stubs at the start and end of the swap but *have not yet been implemented*. All stubs are currently short either at the start or end of swap."""	-	-
check_argument_types(.Schedule, locals())	-	-
if effective_dt <	-	-
raise FinError("Effective date must be before termination date.")	-	-
.effective_dt	-	-
.termination_dt	-	-
if first_dt is None	-	-
.first_dt	-	-
else	-	-
if first_dt > effective_dt and first_dt > termination_dt	-	-

schedule_dts

Returns a list of the schedule of Dates.

```
schedule_dts() :
```

The function arguments are described in the following table.

generate

Generate schedule of dates according to specified date generation rules and also adjust these dates for holidays according to the specified business day convention and the specified calendar.

```
generate() :
```

The function arguments are described in the following table.

__repr__

Print out the details of the schedule and the actual dates. This can be used for providing transparency on schedule calculations.

```
__repr__() :
```

The function arguments are described in the following table.

_print

Print out the details of the schedule and the actual dates. This can be used for providing transparency on schedule calculations.

```
_print() :
```

The function arguments are described in the following table.

1.20 singleton

Class: Singleton(type)

Singleton type which is used to ensure there is only one instance across the whole project.

__call__

PLEASE ADD A FUNCTION DESCRIPTION

```
__call__(cls, *args, **kwargs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
*args	-	-	-
**kwargs	-	-	-

1.21 solver_1d

results

*r*Select from a tuple of(*root*, *funcalls*, *iterations*, *flag*)

```
_results(r):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
<i>r</i>	-	-	-

newton_secant

Find a zero from the secant method using the jitted version of Scipy's secant method. Note that 'func' must be jitted via Numba. Parameters ——— *func* : callable and jitted The function whose zero is wanted. It must be a function of a single variable of the form $f(x,a,b,c,\dots)$, where $a,b,c,\dots$ are extra arguments that can be passed in the 'args' parameter. *x0* : float An initial estimate of the zero that should be somewhere near the actual zero. *args* : tuple, optional(default=()) Extra arguments to be used in the function call. *tol* : float, optional(default=1.48e-8) The allowable error of the zero value. *maxiter* : int, optional(default=50) Maximum number of iterations. *disp* : bool, optional(default=True) If True, raise a RuntimeError if the algorithm didn't converge. *Returns* ——— *results* : namedtuple A namedtuple containing the following items: *:: root* - Estimated location where function is zero. *function_calls* - Number of times the function was called. *iterations* - Number of iterations needed to find the root. *converged* - True if the routine converged

```
newton_secant(func, x0, args=(), tol=1.48e-8, maxiter=50, disp=True):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
<i>func</i>	-	-	-
<i>x0</i>	-	-	-
<i>args</i>	-	-	()
<i>tol</i>	-	-	1.48e-8
<i>maxiter</i>	-	-	50
<i>disp</i>	-	-	True

newton

PLEASE ADD A FUNCTION DESCRIPTION

```
newton(
func,
x0,
fprime=None,
args=None,
tol=1.48e-8,
```

```

maxiter=50,
fprime2=None,
xl=None,
rtol=0.0,
full_output=False,
disp=False,
"""
TAKEN_FROM SCIPY
Find a zero of a real or complex function using the Newton-Raphson
(or secant or Halley's) method.
Find a zero of the function 'func' given a nearby starting point 'x0'.
The Newton-Raphson method is used if the derivative 'fprime' of 'func'
is provided, otherwise the secant method is used. If the second order
derivative 'fprime2' of 'func' is also provided, then Halley's method is
used.
If 'x0' is a sequence with more than one item, then 'newton' returns an
array, and 'func' must be vectorized and return a sequence or array of the
same shape as its first argument. If 'fprime' or 'fprime2' is given, then
its return must also have the same shape.
Parameters
-----
func: callable
    The function whose zero is wanted. It must be a function of a
    single variable of the form ``f(x,a,b,c...)`` where ``a,b,c...``
    are extra arguments that can be passed in the 'args' parameter.
x0: float, sequence, or ndarray
    An initial estimate of the zero that should be somewhere near the
    actual zero. If not scalar, then 'func' must be vectorized and return
    a sequence or array of the same shape as its first argument.
fprime: callable, optional
    The derivative of the function when available and convenient. If it
    is None (default), then the secant method is used.
args: tuple, optional
    Extra arguments to be used in the function call.
tol: float, optional
    The allowable error of the zero value. If 'func' is complex-valued,
    a larger 'tol' is recommended as both the real and imaginary parts
    of 'x' contribute to ``|x-x0|``.
maxiter: int, optional
    Maximum number of iterations.
fprime2: callable, optional
    The second order derivative of the function when available and
    convenient. If it is None (default), then the normal Newton-Raphson
    or the secant method is used. If it is not None, then Halley's method
    is used.
xl: float, optional
    Another estimate of the zero that should be somewhere near the
    actual zero. Used if 'fprime' is not provided.
rtol: float, optional
    Tolerance (relative) for termination.
full_output: bool, optional
    If 'full_output' is False (default), the root is returned.
    If True and 'x0' is scalar, the return value is ``(x,r)`` where ``x``
    is the root and ``r`` is a 'RootResults' object.
    If True and 'x0' is non-scalar, the return value is ``(x, converged,
    zero_der)`` (see Returns section for details).
disp: bool, optional

```



```

    If True, raise a RuntimeError if the algorithm didn't converge, with
    the error message containing the number of iterations and current
    function value. Otherwise, the convergence status is recorded in a
    'RootResults' return object.
    Ignored if 'x0' is not scalar.
    *Note: this has little to do with displaying, however,
    the 'disp' keyword cannot be renamed for backwards compatibility.*
Returns
-----
root: float, sequence, or ndarray
    Estimated location where function is zero.
r: 'RootResults', optional
    Present if 'full_output=True' and 'x0' is scalar.
    Object containing information about the convergence. In particular,
    'r.converged' is True if the routine converged.
converged: ndarray of bool, optional
    Present if 'full_output=True' and 'x0' is non-scalar.
    For vector functions, indicates which elements converged successfully.
zero_der: ndarray of bool, optional
    Present if 'full_output=True' and 'x0' is non-scalar.
    For vector functions, indicates which elements had a zero derivative.
See Also
-----
brentq, brentj, ridder, bisect
fsolve: find zeros in N dimensions.
Notes
-----
The convergence rate of the Newton-Raphson method is quadratic,
the Halley method is cubic, and the secant method is
sub-quadratic. This means that if the function is well-behaved
the actual error in the estimated zero after the nth iteration
is approximately the square (cube for Halley) of the error
after the (n-1)th step. However, the stopping criterion used
here is the step size and there is no guarantee that a zero
has been found. Consequently, the result should be verified.
Safer algorithms are brentq, brentj, ridder, and bisect,
but they all require that the root first be bracketed in an
interval where the function changes sign. The brentq algorithm
is recommended for general use in one dimensional problems
when such an interval has been found.
When 'newton' is used with arrays, it is best suited for the following
types of problems:
* The initial guesses, 'x0', are all relatively the same distance from
  the roots.
* Some or all of the extra arguments, 'args', are also arrays so that a

```

The function arguments are described in the following table.

Argument Name	Type	Description
func	-	-
x0	-	-
fprime	-	-
args	-	-
tol	-	-
maxiter	-	-
fprime2	-	-
x1	-	-
rtol	-	-
full_output	-	-
disp	-	-
"""	-	-
TAKEN FROM SCIPY	-	-
Find a zero of a real or complex function using the Newton-Raphson (or secant or Halley's) method.	-	-
Find a zero of the function 'func' given a nearby starting point 'x0'.	-	-
The Newton-Raphson method is used if the derivative 'fprime' of 'func' is provided	-	-
otherwise the secant method is used. If the second order derivative 'fprime2' of 'func' is also provided	-	-
then Halley's method is used.	-	-
If 'x0' is a sequence with more than one item then 'newton' returns an array	-	-
and 'func' must be vectorized and return a sequence or array of the same shape as its first argument. If 'fprime' or 'fprime2' is given then	-	-
its return must also have the same shape.	-	-
Parameters	-	-
-----	-	-
func	callable	-
The function whose zero is wanted. It must be a function of a single variable of the form "f(x,a,b,c...)"	-	-
where "a	-	-
b	-	-
c..."	-	-
are extra arguments that can be passed in the 'args' parameter.	-	-
x0	float	-
sequence	-	-
or ndarray	-	-
An initial estimate of the zero that should be somewhere near the actual zero. If not scalar	-	-
then 'func' must be vectorized and return a sequence or array of the same shape as its first argument.	-	-
fprime	callable	-
optional	-	-
The derivative of the function when available and convenient. If it is None (default)	-	-
then the secant method is used.	-	-
args	tuple	-

brent_max

PLEASE ADD A FUNCTION DESCRIPTION

```
brent_max(func, a, b, args, xtol=1e-5, maxiter=500):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
func	-	-	-
a	-	-	-
b	-	-	-
args	-	-	-
xtol	-	-	1e-5
maxiter	-	-	500

bisection

Bisection algorithm. You need to supply root brackets x1 and x2.

```
bisection(func, x1, x2, args, xtol=1e-6, maxiter=100):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
func	-	-	-
x1	-	-	-
x2	-	-	-
args	-	-	-
xtol	-	-	1e-6
maxiter	-	-	100

minimize_wolfe_powel

PLEASE ADD A FUNCTION DESCRIPTION

```
minimize_wolfe_powel(
f, X, length, fargs=(), reduction=None, verbose=False, concise=False
"""
Minimize_a_differentiable_multivariate_function.
Parameters
-----
f_:function_to_minimize._The_function_must_return_the_value
    of_the_function_(float)_and_a_numpy_array_of_partial
    derivatives_of_shape_(D,)_with_respect_to_X,_where_D_is
    the_dimensionality_of_the_function.
X_:numpy_array_-Shape:_(D,_1)
    initial_guess.
length:_int
```

```

"""The length of the run. If positive, length gives the maximum
number of line searches, if negative its absolute value gives
the maximum number of function evaluations.
args: tuple
"""Tuple of parameters to be passed to the function f.
reduction: float
"""The expected reduction in the function value in the first
linesearch (if None, defaults to 1.0)
verbose: bool
"""If True, prints the progress of minimize. (default is True)
concise: bool
"""If True, returns concise convergence info, only the minimum function
value (necessary when optimizing a large number of parameters)
(default is False)
Return
-----
Xs: numpy array - Shape: (D, 1)
"""The found solution.
convergence: numpy array - Shape: (i, D+1)
"""Convergence information. The first column is the function values
returned by the function being minimized. The next D columns are
the guesses of X during the minimization process.
"""If concise = True, convergence information is only the minimum
function value. This is necessary only when optimizing a large number
of parameters.
i: int
"""Number of line searches or function evaluations depending on which
was selected.
The function returns when either length is up, or if no further progress
can be made (ie, we are at a (local) minimum, or so close that due to
numerical problems, we cannot get any closer)
Copyright (C) 2001-2006 by Carl Edward Rasmussen (2006-09-08).
Converted to python by David Lines (2019-23-08)
"""
INT = 0.1 # don't reevaluate within 0.1 of the limit of the current bracket
EXT = 3.0 # extrapolate maximum 3 times the current step size
MAX = 20 # max 20 function evaluations per line search
RATIO = 10 # maximum allowed slope ratio
SIG = 0.1
RHO = SIG / 2
# SIG and RHO control the Wolfe-Powell conditions
# SIG is the maximum allowed absolute ratio between
# previous and new slopes (derivs in the search direction), thus setting
# SIG to low (positive) values forces higher precision in line-searches.
# RHO is the minimum allowed fraction of the expected (from slope at the
# initial point in linesearch). Constants must satisfy 0 < RHO < SIG < 1.
# Tuning of SIG (depending on nature of the function to be optimized) may
# speed up minimization; it is probably not worth playing much with RHO.
# print("Minimizing %s ..." % f)
if reduction is None:
    red = 1.0
else:
    red = reduction
S = "Linesearch" if length > 0 else "Function_evaluation"
i = 0 # run length counter
ls_failed = 0 # no previous line search has failed
f0, df0 = f(X, fargs) # get initial function value and gradient

```

```
df0 = df0.reshape(-1, 1)
fX = []
fX.append(f0)
Xd = []
Xd.append(X)
i += length < 0 # count epochs
s = -df0 # get column vec
d0 = -s.T @ s # initial search direction (steepest) and slope
x3 = red / (1 - d0) # initial step is red/(|s|+1)
while i < abs(length): # while not finished
```

The function arguments are described in the following table.

Argument Name	Type
f	-
X	-
length	-
fargs	-
reduction	-
verbose	-
concise	-
"""	-
Minimize a differentiable multivariate function.	-
Parameters	-
-----	-
f	function to minimize. The function must return the value
of the function (float) and a numpy array of partial	-
derivatives of shape (D,) with respect to X	-
where D is	-
the dimensionality of the function.	-
X	D or 1
initial guess.	-
length	int
The length of the run. If positive	-
length gives the maximum	-
number of line searches	-
if negative its absolute value gives	-
the maximum number of function evaluations.	-
args	tuple
Tuple of parameters to be passed to the function f.	-
reduction	float
The expected reduction in the function value in the first	-
linesearch (if None, defaults to 1.0)	-
verbose	bool
If True - prints the progress of minimize. (default is True)	-
concise	bool
If True - returns concise convergence info	-
only the minimum function	-
value (necessary when optimizing a large number of parameters)	-
(default is False)	-
Return	-
-----	-
Xs	D or 1
The found solution.	-
convergence	i or D+1
Convergence information. The first column is the function values	-
returned by the function being minimized. The next D columns are	-
the guesses of X during the minimization process.	-
If concise	-
convergence information is only the minimum	-
function value. This is necessary only when optimizing a large number	-
of parameters.	-
i	int
Number of line searches or function evaluations depending on which	-
was selected	-

1.22 solver_cg

Class: `_LineSearchError(RuntimeError)`

Class: `ScalarFunction`

ScalarFunction

PLEASE ADD A FUNCTION DESCRIPTION

```
ScalarFunction(fun,
x0,
args=(),
grad=None,
hess=None,
epsilon=1.4901161193847656e-8,
bounds=None,
finite_diff_rel_step=None,
.fun = lambda x: fun(x, *args)
if callable(grad):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fun	-	-	-
x0	-	-	-
args	-	-	()
grad	-	-	None
hess	-	-	None
epsilon	-	-	1.4901161193847656e-8
bounds	-	-	None
finite_diff_rel_step	-	-	None
.fun	-	-	lambda x: fun(x, *args)

`__call__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__call__(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

`_finite_diff_grad`

PLEASE ADD A FUNCTION DESCRIPTION

```
_finite_diff_grad(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

Class: OptimizeResult(dict)

`__getattr__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__getattr__(name):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
name	-	-	-

`__repr__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

`__dir__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__dir__():
```

The function arguments are described in the following table.

`polak_ribiere_powell_step`

PLEASE ADD A FUNCTION DESCRIPTION

```
polak_ribiere_powell_step(alpha, gfkp1=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	-	-	-
gfkp1	-	-	None

descent_condition*PLEASE ADD A FUNCTION DESCRIPTION*

```
descent_condition(alpha, xkp1, fp1, gfkp1):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	-	-	-
xkp1	-	-	-
fp1	-	-	-
gfkp1	-	-	-

vecnorm*PLEASE ADD A FUNCTION DESCRIPTION*

```
vecnorm(x, ord=2):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
ord	-	-	2

_prepare_scalar_function*Prepare scalar function for optimization.*

```
_prepare_scalar_function(fun, x0, jac=None, args=(), epsilon=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fun	-	-	-
x0	-	-	-
jac	-	-	None
args	-	-	()
epsilon	-	-	None

_line_search_wolfe12*Wolfe line search with fallback to line_search_wolfe2.*

```
_line_search_wolfe12(f, fprime, xk, pk, gfk, old_fval, old_old_fval, **kwargs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
f	-	-	-
fprime	-	-	-
xk	-	-	-
pk	-	-	-
gfk	-	-	-
old_fval	-	-	-
old_old_fval	-	-	-
**kwargs	-	-	-

_minimize_cg

PLEASE ADD A FUNCTION DESCRIPTION

```
_minimize_cg(
    fun,
    x0,
    args=(),
    jac=None,
    callback=None,
    gtol=1e-5,
    norm=Inf,
    eps=EPSILON,
    maxiter=None,
    disp=False,
    return_all=False,
    **unknown_options,
    x0 = np.asarray(x0).flatten()
    if maxiter is None:
        maxiter = len(x0) * 200
    sf = _prepare_scalar_function(fun, x0, jac=jac, args=args, epsilon=eps)
    f = sf.fun
    myfprime = sf.grad
    old_fval = f(x0)
    gfk = myfprime(x0)
    if not np.isscalar(old_fval):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fun	-	-	-
x0	-	-	-
args	-	-	()
jac	-	-	None
callback	-	-	None
gtol	-	-	1e-5
norm	-	-	Inf
eps	-	-	EPSILON
maxiter	-	-	None
disp	-	-	False
return_all	-	-	False
**unknown_options	-	-	-
x0	-	-	np.asarray(x0).flatten()
if maxiter is None	-	-	-
maxiter	-	-	len(x0) * 200
sf	-	-	_prepare_scalar_function(fun, x0, jac=jac, args=args, epsilon=eps)
f	-	-	sf.fun
myfprime	-	-	sf.grad
old_fval	-	-	f(x0)
gfk	-	-	myfprime(x0)

fmin_cg

Minimize a function using nonlinear conjugate gradient (Polak-Ribiere).

```
fmin_cg(
f,
x0,
fprime=None,
fargs=(),
gtol=1e-5,
norm=Inf,
epsilon=EPSILON,
maxiter=None,
full_output=0,
disp=1,
retall=0,
callback=None,
"""Minimize_a_function_using_nonlinear_conjugate_gradient_(Polak-Ribiere)."""
# opts = {
#     "gtol": gtol,
#     "norm": norm,
#     "eps": epsilon,
#     "disp": disp,
#     "maxiter": maxiter,
#     "return_all": retall,
# }
res = _minimize_cg(f, x0, args=fargs, jac=fprime, callback=callback)
if full_output:
```

```

retlist = res["x"], res["fun"], res["nfev"], res["njev"], res["status"]
if retall:
    retlist += (res["allvecs"],)
return retlist
else:
    if retall:
        return res["x"], res["allvecs"]
    return res["x"]

```

The function arguments are described in the following table.

Argument Name	Type	Description
f	-	-
x0	-	-
fprime	-	-
fargs	-	-
gtol	-	-
norm	-	-
epsilon	-	-
maxiter	-	-
full_output	-	-
disp	-	-
retall	-	-
callback	-	-
"""Minimize a function using nonlinear conjugate gradient (Polak-Ribiere)."""	-	-
res	-	._minimize_cg(f, x0, fprime, fargs, gtol, norm, epsilon, maxiter, full_output, disp, retall, callback)
if full_output	-	-
retlist	-	-
res["fun"]	-	-
res["nfev"]	-	-
res["njev"]	-	-
res["status"]	-	-
if retall	-	-
retlist +	-	-
return retlist	-	-
else	-	-
if retall	-	-
return res["x"]	-	-
res["allvecs"]	-	-
return res["x"]	-	-

1.23 solver_nm

nelder_mead

PLEASE ADD A FUNCTION DESCRIPTION

```
nelder_mead(
    fun,
    x0,
    bounds=np.array([[ ], [ ]]).T,
    args=(),
    tol_f=1e-10,
    tol_x=1e-10,
    max_iter=1000,
    roh=1.0,
    chi=2.0,
    v=0.5,
    sigma=0.5,
    """
    Minimize_a_scalar-valued_function_using_Nelder-Mead_method.
    Parameters
    -----
    fun_: callable
        Function_to_minimize, _must_accept_(x, *_args)
    x0_: ndarray
        Initial_guess, _shape_(n,)
    bounds_: ndarray, optional
        Shape_(n,2)_of_(min,max)_for_each_variable
    args_: tuple
        Extra_arguments_to_fun
    tol_f, tol_x_: float
        Tolerances_for_convergence
    max_iter_: int
        Maximum_iterations
    roh, chi, v, sigma_: float
        Nelder-Mead_parameters
    Returns
    -----
    results_namedtuple
    """
    n = x0.size
    vertices = _initialize_simplex(x0, bounds)
    _check_params(roh, chi, v, sigma, bounds, n)
    f_val = np.empty(n + 1, dtype=np.float64)
    for i in range(n + 1):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fun	-	-	-
x0	-	-	-
bounds	-	-	np.array([[], []]).T
args	-	-	()
tol_f	-	-	1e-10
tol_x	-	-	1e-10
max_iter	-	-	1000
roh	-	-	1.0
chi	-	-	2.0
v	-	-	0.5
sigma	-	-	0.5
"""	-	-	-
Minimize a scalar-valued function using Nelder-Mead method.	-	-	-
Parameters	-	-	-
-----	-	-	-
fun	callable	-	-
Function to minimize	-	-	-
must accept (x, *args)	-	-	-
x0	ndarray	-	-
Initial guess	-	-	-
shape (n,)	-	-	-
bounds	ndarray	-	-
optional	-	-	-
Shape (n,2) of (min,max) for each variable	-	-	-
args	tuple	-	-
Extra arguments to fun	-	-	-
tol_f	-	-	-
tol_x	float	-	-
Tolerances for convergence	-	-	-
max_iter	int	-	-
Maximum iterations	-	-	-
roh	-	-	-
chi	-	-	-
v	-	-	-
sigma	float	-	-
Nelder-Mead parameters	-	-	-
Returns	-	-	-
-----	-	-	-
results namedtuple	-	-	-
"""	-	-	-
n	-	-	x0.size
vertices	-	-	_initialize_simplex(x0, bounds,
_check_params(roh, chi, v, sigma, bounds, n)	-	-	-
f_val	-	-	np.empty(n + 1, dtype=np.float

`_initialize_simplex`*PLEASE ADD A FUNCTION DESCRIPTION*

```
_initialize_simplex(x0, bounds):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x0	-	-	-
bounds	-	-	-

`_check_params`*PLEASE ADD A FUNCTION DESCRIPTION*

```
_check_params(rho, chi, v, sigma, bounds, n):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
rho	-	-	-
chi	-	-	-
v	-	-	-
sigma	-	-	-
bounds	-	-	-
n	-	-	-

1.24 stats

mean

Calculate the arithmetic mean of a vector of numbers x .

```
mean(x: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-

stdev

Calculate the standard deviation of a vector of numbers x .

```
stdev(x: ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	ndarray	-	-

stderr

Calculate the standard error estimate of a vector of numbers x .

```
stderr(x: ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	ndarray	-	-

var

Calculate the variance of a vector of numbers x .

```
var(x: ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	ndarray	-	-

moment

Calculate the m -th moment of a vector of numbers x .

```
moment(x: ndarray, m: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	ndarray	-	-
m	int	-	-

correlation

Calculate the correlation between two series $x1$ and $x2$.

```
correlation(x1: ndarray, x2: ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x1	ndarray	-	-
x2	ndarray	-	-

Enumerated Type: TenorUnit

- NONE
- DAYS
- WEEKS
- MONTHS
- YEARS
- 1Y
- def __init__(self,tenor_string:str
- self._units
- self._num_periods
- tenor_string
- iftenor_string
- self._units
- self._num_periods
- eliftenor_string
- self._units
- self._num_periods
- eliftenor_string[-1]
- self._units
- self._num_periods
- eliftenor_string[-1]
- self._units
- self._num_periods
- eliftenor_string[-1]
- self._units
- self._num_periods
- eliftenor_string[-1]
- self._units
- self._num_periods
- self._units
- self._num_periods
- returnself._units!
- res
- res._units
- res._num_periods
- __rmul__
- t1,t2
- common_unit
- res
- res._units
- res._num_periods
- ifself._units

- res
- (t1,t2)
- (t1_in,t2_in)ift1_in.units.value;
- common_unit
- t1
- t2
- periodsmustbethesame.Inparticular12M!
- returnself._units

Class: Tenor

A class to represent a Tenor such as 1D or 10Y. There is a unit (such as a day or a month) and a number representing how many units are in the tenor. For example 10Y means 10 years while 120m also means 10 years

We support limited conversion from bigger units to smaller units, which also uses pretty basic rules such as 1Y = 12M = 12*4 W = 12*4*7 D so use conversion at your own risk. Specifically Y-¿M is ok and W-¿D is ok but Y-¿D is clearly not that great

We also support limited arithmetic operations that make Tenors a linear space over integers (ie you can add tenors and multiply them by integers)

Tenor

PLEASE ADD A FUNCTION DESCRIPTION

```
Tenor(tenor_string: str = None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
tenor_string	str	-	None

units

Returns the units of the tenor.

```
units() -> TenorUnit:
"""Returns the units of the tenor."""
return ._units
#####
perty
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -¿ TenorUnit		-	-
"""Returns the units of the tenor."""	-	-	-
return ._units	-	-	-
perty	-	-	-

num_periods

Returns the number of periods in the tenor.

```
num_periods() -> int:
"""Returns the number of periods in the tenor."""
return ._num_periods
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> int		-	-
"""Returns the number of periods in the tenor."""	-	-	-
return ._num_periods	-	-	-

set_units

Set the units for the tenor.

```
set_units(units: TenorUnit):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
units	TenorUnit	-	-

set_num_periods

Set the number of periods for the tenor.

```
set_num_periods(num):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num	-	-	-

set

Convenience method to set both number and units at once.

```
set(num, units: TenorUnit):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num	-	-	-
units	TenorUnit	-	-

as_tenor

Convert a string or Tenor object to a Tenor object.

```
as_tenor(cls, str_or_tenor):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
str_or_tenor	-	-	-

is_valid

Check if the tenor is valid.

```
is_valid():
```

The function arguments are described in the following table.

__mul__

Multiply a tenor by an integer Args: other (int): Multiplier Returns: The result of multiplication

```
__mul__(other: int):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	int	-	-

__add__

Adds two tenors. Use the more fine-grained unit of the two for the result Args: other (Tenor): Tenor to add Returns: The sum of two Tenors as a Tenor

```
__add__(other):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

convert_to_unit

Convert a tenor to an equivalent tenor in new units. Only "downcasting" is allowed ie W-¿D ok, D-¿W not ok. This can obviously be improved. Very nominal conversations used, see the code for details Args: new_unit (TenorUnit): New unit for the tenor Raises: FinError: If units cannot be converted Returns: The same tenor in new units

```
convert_to_unit(new_units: TenorUnit):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
new_units	TenorUnit	-	-

convert_to_same_units

PLEASE ADD A FUNCTION DESCRIPTION

```
convert_to_same_units(cls, t1_in, t2_in):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
t1_in	-	-	-
t2_in	-	-	-

__eq__

Equality in the strict sense, ie the units and the nummber of periods must be the same. In particular 12M != 1Y Perhaps this should be relaxed, but at the moment the idea is that a more relaxed equality could be explictly called if the user first calls Tenor.convert_to_same_units method Args: other (Tenor): rhs of equality Returns: True or False depending if teh two tenors are equal

```
__eq__(other):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
other	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

1.26 tension_spline

Class: *TensionSpline(object)*

Implement exponential tension spline following [AP] Sec 6.A.3
[AP] Andersen, Piterbarg. Interest Rate Modeling, 2010

TensionSpline

PLEASE ADD A FUNCTION DESCRIPTION

```
TensionSpline(x, y, sigma):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
y	-	-	-
sigma	-	-	-

validate_inputs

PLEASE ADD A FUNCTION DESCRIPTION

```
validate_inputs():
```

The function arguments are described in the following table.

calculate_coefs

PLEASE ADD A FUNCTION DESCRIPTION

```
calculate_coefs():
```

The function arguments are described in the following table.

__call__

PLEASE ADD A FUNCTION DESCRIPTION

```
__call__(xs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
xs	-	-	-

1.27 `__init__`

Chapter 2

financepy.market.curves

Discount Curves

These modules create a family of discount curve types related to the term structures of interest rates. Discount curves that can be used to present value a future cash flow. These differ from best fits curves in that they exactly refit the prices of bonds or CDS. The different discount curves are created by calibrating to different instruments. They also differ in terms of the term structure shapes they can have. Different shapes have different impacts in terms of locality on risk management performed using these different curves. There is often a trade-off between smoothness and locality. These are curves which supply a discount factor that can be used to present-value future payments.

DiscountCurve

This is a curve made from a Numpy array of times and discount factor values that represents a discount curve. It also requires a specific interpolation scheme. A function is also provided to return a survival probability so that this class can also be used to handle term structures of survival probabilities. Other curves inherit from this in order to share common functionality.

DiscountCurveFlat

This is a class that takes in a single flat rate.

DiscountCurveNS

Implementation of the Nelson-Siegel curve parametrisation.

DiscountCurveNSS

Implementation of the Nelson-Siegel-Svensson curve parametrisation.

DiscountCurveZeros

This is a discount curve that is made from a vector of times and zero rates.

Interpolate

This module contains the interpolation function used throughout the discount curves when a discount factor needs to be interpolated. There are three interpolation methods:

1. **PIECEWISE LINEAR** - This assumes that a discount factor at a time between two other known discount factors is obtained by linear interpolation. This approach does not guarantee any smoothness but is local. It does not guarantee positive forwards (assuming positive zero rates).
2. **PIECEWISE LOG LINEAR** - This assumes that the log of the discount factor is interpolated linearly. The log of a discount factor to time T is $T \times R(T)$ where $R(T)$ is the zero rate. So this is not linear interpolation of $R(T)$ but of $T \times R(T)$.
3. **FLAT FORWARDS** - This interpolation assumes that the forward rate is constant between discount factor points. It is not smooth but is highly local and also ensures positive forward rates if the zero rates are positive.

2.1 composite_discount_curve

Class: CompositeDiscountCurve(DiscountCurve)

A discount curve that is a sum (in rates) of children discount curves

CompositeDiscountCurve

Create a discount curve that is a sum (in rates) of other discount curves

```
CompositeDiscountCurve(child_curves: List[DiscountCurve]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
child_curves	List[DiscountCurve]	-	-

df_t

Return discount factors given a single or vector of dates. ParentRate = Sum of children rates =; Parent DF = product of children dfs

```
df_t(t: Union[float, np.ndarray]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float or np.ndarray	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.2 discount_curve

Class: DiscountCurve

This is a base discount curve which has an internal representation of a vector of times and discount factors and an interpolation scheme for interpolating between these fixed points.

DiscountCurve

PLEASE ADD A FUNCTION DESCRIPTION

```
DiscountCurve(value_dt: Date,
df_dates: list = None,
df_values: np.ndarray = None,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
"""Create the discount curve from a vector of times and discount
factors with an anchor date and specify an interpolation scheme. As we
are explicitly linking dates and discount factors, we do not need to
specify any compounding convention or day count calculation since
discount factors are pure prices. We do however need to specify a
convention for interpolating the discount factors in time."""
check_argument_types(.DiscountCurve, locals())
# Validate curve
if df_dates is None:
    df_dates = [value_dt]
if df_values is None:
    df_values = [1.0]
# The internal representation of times and dfs is hidden but
# access is controlled using getters and setters
._times = [0.0]
._dfs = [1.0]
._df_dates = [value_dt]
num_points = len(df_dates)
start_index = 0
if num_points > 0 and df_dates[0] == value_dt:
    ._dfs[0] = df_values[0]
    start_index = 1
for i in range(start_index, num_points):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
df_dates	list	-	None
df_values	np.ndarray	-	None
interp_type	InterpTypes	-	InterpTypes.FLAT
"""Create the discount curve from a vector of times and discount factors with an anchor date and specify an interpolation scheme. As we are explicitly linking dates and discount factors we do not need to specify any compounding convention or day count calculation since discount factors are pure prices. We do however need to specify a convention for interpolating the discount factors in time."""			
check_argument_types(.DiscountCurve, locals())	-	-	-
if df_dates is None	-	-	-
df_dates	-	-	[value_dt]
if df_values is None	-	-	-
df_values	-	-	[1.0]
._times	-	-	[0.0]
._dfs	-	-	[1.0]
._df_dates	-	-	[value_dt]
num_points	-	-	len(df_dates)
start_index	-	-	0
if num_points < 0 and df_dates[0]	-	-	= value_dt
._dfs[0]	-	-	df_values
start_index	-	-	1

times

Return the internal array of times(in years) from the anchor date.

```
times() -> np.ndarray:
"""Return the internal array of times(in years) from the anchor date."""
return ._times.copy() # return a copy to prevent external modification
perty
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
) -> np.ndarray	-	-
"""Return the internal array of times(in years) from the anchor date."""	-	-
return ._times.copy()	-	return a copy to prevent external modification
perty	-	-

dfs

Return the internal array of discount factors corresponding to times.

```
dfs() -> np.ndarray:
"""Return the internal array of discount factors corresponding to times."""
return ._dfs.copy() # return a copy to prevent external modification
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
) -> np.ndarray	-	-
"""Return the internal array of discount factors corresponding to times."""	-	-
return ._dfs.copy()	-	return a copy to prevent external modification

set_times

Set the discount factor at the last maturity time.

```
set_times(times: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	np.ndarray	-	-

set_dfs

Set the discount factor at the last maturity time.

```
set_dfs(dfs: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dfs	np.ndarray	-	-

set_last_df

Set the discount factor at the last maturity time.

```
set_last_df(df):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-

zero_to_df*PLEASE ADD A FUNCTION DESCRIPTION*

```

_zero_to_df(value_dt: Date, # TODO: why is value_dt not used ?
rates: Union[float, np.ndarray],
times: Union[float, np.ndarray],
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
"""Convert a zero with a specified compounding frequency and day count
convention to a discount factor for a single maturity date or a list of
dates. The day count is used to calculate the elapsed year fraction."""
if isinstance(times, float):

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	TODO: why is value_dt not used ?
rates	float or np.ndarray	-
times	float or np.ndarray	-
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
"""Convert a zero with a specified compounding frequency and day count	-	-
convention to a discount factor for a single maturity date or a list of	-	-
dates. The day count is used to calculate the elapsed year fraction."""	-	-

fit

Fit the interpolator to the given times and discount factors.

```
fit(times: np.ndarray, dfs: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	np.ndarray	-	-
dfs	np.ndarray	-	-

df_to_zero*PLEASE ADD A FUNCTION DESCRIPTION*

```

_df_to_zero(dfs: Union[float, np.ndarray],
maturity_dts: Union[Date, list],
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
"""Given a dates this first generates the discount factors. It then
converts the discount factors to a zero rate with a chosen compounding
frequency which may be continuous, simple, or compounded at a specific

```



```
frequency_which_are_all_choices_of_FrequencyTypes._Returns_a_list_of
discount_factor."""
f = annual_frequency(freq_type)
if isinstance(maturity_dts, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
dfs	float or np.ndarray	-	
maturity_dts	Date or list	-	
freq_type	FrequencyTypes	-	
dc_type	DayCountTypes	-	
"""Given a dates this first generates the discount factors. It then converts the discount factors to a zero rate with a chosen compounding frequency which may be continuous simple or compounded at a specific frequency which are all choices of FrequencyTypes. Returns a list of discount factor."""	-	-	
f	-	-	annual_frequency

zero_rate

Calculation of zero rates with specified frequency. This function can return a vector of zero rates given a vector of dates so must use Numpy functions. Default frequency is a continuously compounded rate.

```
zero_rate(maturity_dt: Union[list, Date],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
"""Calculation_of_zero_rates_with_specified_frequency._This
function_can_return_a_vector_of_zero_rates_given_a_vector_of
dates_so_must_use_Numpy_functions._Default_frequency_is_a
continuously_compounded_rate."""
if isinstance(freq_type, FrequencyTypes) is False:
    raise FinError("Invalid_Frequency_type.")
if isinstance(dc_type, DayCountTypes) is False:
    raise FinError("Invalid_Day_Count_type.")
dfs = .df(maturity_dt)
scalar_input = isinstance(maturity_dt, Date)
if scalar_input:
    maturity_dts = [maturity_dt]
else:
    maturity_dts = maturity_dt
zero_rates = ._df_to_zero(dfs, maturity_dts, freq_type, dc_type)
if scalar_input:
    return zero_rates[0]
return zero_rates
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
maturity_dt	list or Date	-	-
freq_type	FrequencyTypes	-	FrequencyTypes.C
dc_type	DayCountTypes	-	DayCountTypes
"""Calculation of zero rates with specified frequency. This function can return a vector of zero rates given a vector of dates so must use Numpy functions. Default frequency is a continuously compounded rate."""	-	-	-
if isinstance(freq_type, FrequencyTypes) is False	-	-	-
raise FinError("Invalid Frequency type.")	-	-	-
if isinstance(dc_type, DayCountTypes) is False	-	-	-
raise FinError("Invalid Day Count type.")	-	-	-
dfs	-	-	.df(maturity_dt)
scalar_input	-	-	isinstance(maturity_dt, scalar)
if scalar_input	-	-	-
maturity_dts	-	-	[maturity_dt]
else	-	-	-
maturity_dts	-	-	maturity_dt
zero_rates	-	-	.df_to_zero(dfs, maturity_dts)
if scalar_input	-	-	-
return zero_rates[0]	-	-	-
return zero_rates	-	-	-

cc_rate

Calculation of zero rates with continuous compounding. This function can return a vector of cc rates given a vector of dates so must use Numpy functions.

```
cc_rate(dts: Union[list, Date],
dc_type: DayCountTypes = DayCountTypes.SIMPLE,
"""Calculation of zero rates with continuous compounding. This
function can return a vector of cc rates given a vector of
dates so must use Numpy functions."""
cc_rates = .zero_rate(dts, FrequencyTypes.CONTINUOUS, dc_type)
return cc_rates
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
dts	list or Date	-
dc_type	DayCountTypes	-
"""Calculate of zero rates with continuous compounding. This function can return a vector of cc rates given a vector of dates so must use Numpy functions."""	-	-
cc_rates	-	-
return cc_rates	-	-

swap_rate

Calculate the swap rate to maturity date. This is the rate paid by a swap that has a price of par today. This is the same as a Libor swap rate except that we do not do any business day adjustments.

```
swap_rate(effective_dt: Date,
maturity_dt: Union[list, Date],
freq_type=FrequencyTypes.ANNUAL,
dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
"""Calculate the swap rate to maturity date. This is the rate paid by
a swap that has a price of par today. This is the same as a Libor swap
rate except that we do not do any business day adjustments.""")
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
effective_dt	Date	-	-
maturity_dt	list or Date	-	-
freq_type	-	-	FrequencyTypes
dc_type	DayCountTypes	-	DayCountTypes
"""Calculate the swap rate to maturity date. This is the rate paid by a swap that has a price of par today. This is the same as a Libor swap rate except that we do not do any business day adjustments."""	-	-	-

df

Function to calculate a discount factor from a date or a vector of dates. The day count determines how dates get converted to years. I allow this to default to ACT_ACT_ISDA unless specified.

```
df(dt: Union[list, Date], day_count=DayCountTypes.ACT_ACT_ISDA):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	list or Date	-	-
day_count	-	-	DayCountTypes.ACT_ACT_ISDA

df_t

PLEASE ADD A FUNCTION DESCRIPTION

```
df_t(t: Union[float, np.ndarray]) -> Union[float, np.ndarray]:
    """Function to calculate a discount factor from a time or a
    vector of times. Discourage usage in favour of passing in dates."""
    if ._interp_type in (InterpTypes.FLAT_FWD_RATES,
        InterpTypes.LINEAR_ZERO_RATES,
        InterpTypes.LINEAR_FWD_RATES,
    ):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float or np.ndarray	-	-
"""Function to calculate a discount factor from a time or a vector of times. Discourage usage in favour of passing in dates."""	-	-	-
InterpTypes.LINEAR_ZERO_RATES	-	-	-
InterpTypes.LINEAR_FWD_RATES	-	-	-

survival_prob

This returns a survival probability to a specified date based on the assumption that the continuously compounded rate is a default hazard rate in which case the survival probability is directly analogous to a discount factor.

```
survival_prob(dt: Date) -> float:
    """This returns a survival probability to a specified date based on
    the assumption that the continuously compounded rate is a default
    hazard rate in which case the survival probability is directly
    analogous to a discount factor."""
    q = .df(dt)
    return q
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date)-float:	-	-
"""This returns a survival probability to a specified date based on the assumption that the continuously compounded rate is a default hazard rate in which case the survival probability is directly analogous to a discount factor."""	-	-	-
q	-	-	.df(dt)
return q	-	-	-

fwd

Calculate the continuously compounded forward rate at the forward Date provided. This is done by perturbing the time by one day only and measuring the change in the log of the discount factor divided by the time increment dt. I am assuming continuous compounding over the one date.

```
fwd(dts: Union[list, Date]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dts	list or Date	-	-

_fwd

Calculate the continuously compounded forward rate at the forward time provided. This is done by perturbing the time by a small amount and measuring the change in the log of the discount factor divided by the time increment dt.

```
_fwd(times: Union[np.ndarray, float]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	np.ndarray or float	-	-

bump

Adjust the continuously compounded forward rates by a perturbation upward equal to the bump size and return a curve object with this bumped curve. This is used for interest rate risk.

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

fwd_rate

PLEASE ADD A FUNCTION DESCRIPTION

```
fwd_rate(start_dt: Union[list, Date],
date_or_tenor: Union[Date, str],
dc_type: DayCountTypes = DayCountTypes.ACT_360,
"""Calculate the forward rate between two forward dates according to
the specified day count convention. This defaults to Actual_360. The
first date is specified and the second is given as a date or as a tenor
which is added to the first date."""
if isinstance(start_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	list or Date	-	-
date_or_tenor	Date or str	-	-
dc_type	DayCountTypes	-	DayCountTypes
"""Calculate the forward rate between two forward dates according to the specified day count convention. This defaults to Actual 360. The first date is specified and the second is given as a date or as a tenor which is added to the first date."""		-	-
		-	-
		-	-
		-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.3 discount_curve_flat

Class: DiscountCurveFlat(DiscountCurve)

A very simple discount curve based on a single zero rate with its own specified compounding method. Hence, the curve is assumed to be flat. It is used for quick and dirty analysis and when limited information is available. It inherits several methods from DiscountCurve.

DiscountCurveFlat

Create a discount curve which is flat. This is very useful for quick testing and simply requires a curve date a rate and a compound frequency. As we have entered a rate, a corresponding day count convention must be used to specify how time periods are to be measured. As the curve is flat, no interpolation scheme is required.

```
DiscountCurveFlat(value_dt: Date,
flat_rate: Union[float, np.ndarray],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_ACT_ISDA,
"""Create a discount curve which is flat. This is very useful for
quick testing and simply requires a curve date a rate and a compound
frequency. As we have entered a rate, a corresponding day count
convention must be used to specify how time periods are to be measured.
As the curve is flat, no interpolation scheme is required.
"""
check_argument_types(.DiscountCurveFlat, locals())
.value_dt = value_dt
.flat_rate = flat_rate
.freq_type = freq_type
.dc_type = dc_type
# This is used by some inherited functions, so we choose the simplest
._interp_type = InterpTypes.FLAT_FWD_RATES
# Need to set up a 3M grid of times and discount factors
# Those beyond 10 years are extrapolated
years = np.linspace(0.0, 10.0, 41)
dts = .value_dt.add_years(years)
# Set up a grid of dc adjusted times and discount factors for functions
._times = times_from_dates(dts, .value_dt, dc_type)
._dfs = .df(dts)
perty
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
flat_rate	float or np.ndarray	-
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
"""Create a discount curve which is flat. This is very useful for quick testing and simply requires a curve date a rate and a compound frequency. As we have entered a rate a corresponding day count convention must be used to specify how time periods are to be measured. As the curve is flat no interpolation scheme is required. """		
check_argument_types(DiscoutCurveFlat, locals())	-	-
.value_dt	-	-
.flat_rate	-	-
.freq_type	-	-
.dc_type	-	-
.interp_type	-	-
years	-	-
dts	-	-
.times	-	-
.dfs	-	-
perty	-	-

times

Return the cached grid of times.

```
times() -> np.ndarray:
"""Return the cached grid of times."""
return ._times.copy()
perty
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Return the cached grid of times."""	-	-	-
return ._times.copy()	-	-	-
perty	-	-	-

dfs

Return the cached grid of discount factors.


```
dfs() -> np.ndarray:
"""Return the cached grid of discount factors."""
return ._dfs.copy()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Return the cached grid of discount factors."""	-	-	-
return ._dfs.copy()	-	-	-

df

Return discount factors given a single or vector of dts. The discount factor depends on the rate and this in turn depends on its compounding frequency, and it defaults to continuous compounding. It also depends on the day count convention. This was set in the construction of the curve to be ACT/ACT/ISDA.

```
df(dts: Union[Date, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dts	Date or list	-	-

bump

Create a new FinDiscountCurveFlat object with the entire curve bumped up by the bumpsize. All other parameters are preserved.

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.4 discount_curve_ns

Class: DiscountCurveNS(DiscountCurve)

Implementation of Nelson-Siegel parametrisation of a discount curve. The internal rate is a continuously compounded rate but you can calculate alternative frequencies by providing a corresponding compounding frequency. A day count convention is needed to ensure that dates are converted to the correct time in years. The class inherits methods from DiscountCurve.

DiscountCurveNS

Creation of a FinDiscountCurveNS object. Parameters are provided individually for beta_0, beta_1, beta_2 and tau. The zero rates produced by this parametrisation have an implicit compounding convention that defaults to continuous but which can be overridden.

```
DiscountCurveNS(value_dt: Date,
beta_0: float,
beta_1: float,
beta_2: float,
tau: float,
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_ACT_ISDA,
"""Creation_of_a_FinDiscountCurveNS_object._Parameters_are_provided
individually_for_beta_0,_beta_1,_beta_2_and_tau._The_zero_rates
produced_by_this_parametrisation_have_an_implicit_compounding
convention_that_defaults_to_continuous_but_which_can_be_overridden."""
check_argument_types(.DiscountCurveNS, locals())
if tau <= 0:
    raise FinError("Tau_must_be_positive")
.value_dt = value_dt
._beta_0 = beta_0
._beta_1 = beta_1
._beta_2 = beta_2
._tau = tau
.freq_type = freq_type
.dc_type = dc_type
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	
beta_0	float	-	
beta_1	float	-	
beta_2	float	-	
tau	float	-	
freq_type	FrequencyTypes	-	FrequencyTypes
dc_type	DayCountTypes	-	DayCountTypes
"""Creation of a FinDiscountCurveNS object. Parameters are provided individually for beta_0			
beta_1	-	-	
beta_2 and tau. The zero rates	-	-	
produced by this parametrisation have an implicit compounding	-	-	
convention that defaults to continuous but which can be overridden."""	-	-	
check_argument_types(.DiscountCurveNS, locals())	-	-	
if tau < 0:	-	-	
raise FinError("Tau must be positive")	-	-	
.value_dt	-	-	value_dt
._beta_0	-	-	beta_0
._beta_1	-	-	beta_1
._beta_2	-	-	beta_2
._tau	-	-	tau
.freq_type	-	-	freq_type
.dc_type	-	-	dc_type

zero_rate

Calculation of zero rates with specified frequency according to NS parametrisation. This method overrides that in DiscountCurve. The parametrisation is not strictly in terms of continuously compounded zero rates, this function allows other compounding and day counts. This function returns a single or vector of zero rates given a vector of dates so must use Numpy functions. The default frequency is a continuously compounded rate and ACT360 day counting.

```

zero_rate(dts: Union[list, Date],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
"""Calculation of zero rates with specified frequency according to
NS parametrisation. This method overrides that in DiscountCurve.
The parametrisation is not strictly in terms of continuously compounded
zero rates, this function allows other compounding and day counts.
This function returns a single or vector of zero rates given a vector
of dates so must use Numpy functions. The default frequency is a
continuously compounded rate and ACT360 day counting."""
if isinstance(freq_type, FrequencyTypes) is False:
    raise FinError("Invalid Frequency type.")
if isinstance(dc_type, DayCountTypes) is False:
    raise FinError("Invalid Day Count type.")
# Get day count times to use with curve day count convention
dc_times = times_from_dates(dts, .value_dt, .dc_type)

```

```

# We now get the discount factors using these times
zero_rates = ._zero_rate(dc_times)
# Now get the discount factors using curve conventions
dfs = ._zero_to_df(.value_dt, zero_rates, dc_times, .freq_type, .dc_type
)
# Convert these to zero rates in the required frequency and day count
zero_rates = ._df_to_zero(dfs, dts, freq_type, dc_type)
scalar_input = isinstance(dts, Date)
if scalar_input:
    return zero_rates[0]
return zero_rates
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
dts	list or Date	-
freq_type	FrequencyTypes	- Frequency
dc_type	DayCountTypes	- DayC
"""Calculation of zero rates with specified frequency according to NS parametrisation. This method overrides that in DiscountCurve. The parametrisation is not strictly in terms of continuously compounded	-	-
zero rates	-	-
this function allows other compounding and day counts.	-	-
This function returns a single or vector of zero rates given a vector of dates so must use Numpy functions. The default frequency is a continuously compounded rate and ACT360 day counting."""	-	-
if isinstance(freq_type, FrequencyTypes) is False	-	-
raise FinError("Invalid Frequency type.")	-	-
if isinstance(dc_type, DayCountTypes) is False	-	-
raise FinError("Invalid Day Count type.")	-	-
dc_times	-	times_from_d
zero_rates	-	._ze
)	-	-
zero_rates	-	._df_to_zero(
scalar_input	-	isin
if scalar_input	-	-
return zero_rates[0]	-	-
return zero_rates	-	-

zero_rate

Zero rate for Nelson-Siegel curve parametrisation. This means that the t vector must use the curve day count.

```

_zero_rate(times: Union[float, np.ndarray]):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	float or np.ndarray	-	-

df

Return discount factors given a single or vector of dates. The discount factor depends on the rate and this in turn depends on its compounding frequency and it defaults to continuous compounding. It also depends on the day count convention. This was set in the construction of the curve to be ACT_ACT_ISDA.

```
df(dates: Union[Date, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dates	Date or list	-	-

bump

PLEASE ADD A FUNCTION DESCRIPTION

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.5 discount_curve_nss

Class: DiscountCurveNSS(DiscountCurve)

Implementation of Nelson-Siegel-Svensson parametrisation of the zero rate curve. The zero rate is assumed to be continuously compounded. This can be changed when calling for zero rates. A day count convention is needed to ensure that dates are converted to the correct time in years. The class inherits methods from DiscountCurve.

DiscountCurveNSS

Create a FinDiscountCurveNSS object by passing in curve valuation date plus the 4 different beta values and the 2 tau values. The zero rates produced by this parametrisation have an implicit compounding convention that defaults to continuous but can be overridden.

```
DiscountCurveNSS(value_dt: Date,
beta_0: float,
beta_1: float,
beta_2: float,
beta_3: float,
tau_1: float,
tau_2: float,
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_ACT_ISDA,
"""Create_a_FinDiscountCurveNSS_object_by_passing_in_curve_valuation
date_plus_the_4_different_beta_values_and_the_2_tau_values._The_zero
rates_produced_by_this_parametrisation_have_an_implicit_compounding
convention_that_defaults_to_continuous_but_can_be_overriden."""
check_argument_types(.DiscountCurveNSS, locals())
if tau_1 <= 0:
    raise FinError("Tau1_must_be_positive")
if tau_2 <= 0:
    raise FinError("Tau2_must_be_positive")
.value_dt = value_dt
._beta_0 = beta_0
._beta_1 = beta_1
._beta_2 = beta_2
._beta_3 = beta_3
._tau_1 = tau_1
._tau_2 = tau_2
.freq_type = freq_type
.dc_type = dc_type
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	
beta_0	float	-	
beta_1	float	-	
beta_2	float	-	
beta_3	float	-	
tau_1	float	-	
tau_2	float	-	
freq_type	FrequencyTypes	-	FrequencyTypes
dc_type	DayCountTypes	-	DayCountTypes
"""Create a FinDiscountCurveNSS object by passing in curve valuation date plus the 4 different beta values and the 2 tau values. The zero rates produced by this parametrisation have an implicit compounding convention that defaults to continuous but can be overridden."""			
check_argument_types(.DiscountCurveNSS, locals())	-	-	
if tau_1 < 0:	-	-	
raise FinError("Tau1 must be positive")	-	-	
if tau_2 < 0:	-	-	
raise FinError("Tau2 must be positive")	-	-	
.value_dt	-	-	
._beta_0	-	-	
._beta_1	-	-	
._beta_2	-	-	
._beta_3	-	-	
._tau_1	-	-	
._tau_2	-	-	
.freq_type	-	-	
.dc_type	-	-	

zero_rate

PLEASE ADD A FUNCTION DESCRIPTION

```

zero_rate(dts: Union[list, Date],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
        """Calculation of zero rates with specified frequency according to
        NSS parametrisation. This method overrides that in DiscountCurve.
        The NSS parametrisation is no strictly terms of continuously compounded
        zero rates, this function allows other compounding and day counts.
        This function returns a single or vector of zero rates given a vector
        of dates so must use Numpy functions. The default frequency is a
        continuously compounded rate and ACT/ACT day counting."""
        if isinstance(freq_type, FrequencyTypes) is False:
            raise FinError("Invalid Frequency type.")
        if isinstance(dc_type, DayCountTypes) is False:
            raise FinError("Invalid Day Count type.")
        # Get day count times to use with curve day count convention
        dc_times = times_from_dates(dts, .value_dt, .dc_type)

```



```
# We now get the discount factors using these times
zero_rates = ._zero_rate(dc_times)
# Now get the discount factors using curve conventions
dfs = ._zero_to_df(.value_dt, zero_rates, dc_times, .freq_type, .dc_type
)
# Convert these to zero rates in the required frequency and day count
zero_rates = ._df_to_zero(dfs, dts, freq_type, dc_type)
if isinstance(dts, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
dts	list or Date	-
freq_type	FrequencyTypes	- Frequency
dc_type	DayCountTypes	- Day
"""Calculation of zero rates with specified frequency according to NSS parametrisation. This method overrides that in DiscountCurve. The NSS parametrisation is no strictly terms of continuously compounded	-	-
zero rates	-	-
this function allows other compounding and day counts.	-	-
This function returns a single or vector of zero rates given a vector of dates so must use Numpy functions. The default frequency is a continuously compounded rate and ACT ACT day counting."""	-	-
if isinstance(freq_type, FrequencyTypes) is False	-	-
raise FinError("Invalid Frequency type.")	-	-
if isinstance(dc_type, DayCountTypes) is False	-	-
raise FinError("Invalid Day Count type.")	-	-
dc_times	-	- times.from
zero_rates	-	-
)	-	-
zero_rates	-	- .df.to_zero

zero_rate

Calculation of zero rates given a single time or a numpy vector of times. This function can return a single zero rate or a vector of zero rates. The compounding frequency must be provided.

```
_zero_rate(times: Union[float, np.ndarray]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	float or np.ndarray	-	-

df

Return discount factors given a single or vector of dates. The discount factor depends on the rate and this in

turn depends on its compounding frequency and it defaults to continuous compounding. It also depends on the day count convention. This was set in the construction of the curve to be ACT_360.

```
df(dates: Union[Date, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dates	Date or list	-	-

bump

PLEASE ADD A FUNCTION DESCRIPTION

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.6 discount_curve_poly

Class: DiscountCurvePoly(DiscountCurve)

Zero Rate Curve of a specified frequency parametrised using a cubic polynomial. The zero rate is assumed to be continuously compounded but this can be amended by providing a frequency when extracting zero rates. We also need to specify a Day count convention for time calculations. The class inherits all of the methods from DiscountCurve.

DiscountCurvePoly

Create zero rate curve parametrised using a cubic curve from coefficients and specifying a compounding frequency type and day count convention.

```
DiscountCurvePoly(value_dt: Date,
coefficients: Union[list, np.ndarray],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_ACT_ISDA,
"""Create zero rate curve parametrised using a cubic curve from
coefficients and specifying a compounding frequency type and day count
convention."""
check_argument_types(.DiscountCurvePoly, locals())
.value_dt = value_dt
._coefficients = coefficients
.freq_type = freq_type
.dc_type = dc_type
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
value_dt	Date	-	
coefficients	list or np.ndarray	-	
freq_type	FrequencyTypes	-	FrequencyT
dc_type	DayCountTypes	-	DayCountT
"""Create zero rate curve parametrised using a cubic curve from coefficients and specifying a compounding frequency type and day count convention."""	-	-	
check_argument_types(.DiscountCurvePoly, locals())	-	-	
.value_dt	-	-	
._coefficients	-	-	c
.freq_type	-	-	
.dc_type	-	-	

zero_rate

Calculation of zero rates with specified frequency according to polynomial parametrisation. This method overrides DiscountCurve. The parametrisation is not strictly in terms of continuously compounded zero

rates, this function allows other compounding and day counts. This function returns a single or vector of zero rates given a vector of dates so must use Numpy functions. The default frequency is a continuously compounded rate and ACT360 day counting.

```

zero_rate(dts: Union[list, Date],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
"""Calculation_of_zero_rates_with_specified_frequency_according_to
polynomial_parametrisation._This_method_overrides_DiscountCurve.
The_parametrisation_is_not_strictly_in_terms_of_continuously_compounded
zero_rates,_this_function_allows_other_compounding_and_day_counts.
This_function_returns_a_single_or_vector_of_zero_rates_given_a_vector
of_dates_so_must_use_Numpy_functions._The_default_frequency_is_a
continuously_compounded_rate_and_ACT360_day_counting."""
if isinstance(freq_type, FrequencyTypes) is False:
    raise FinError("Invalid_Frequency_type.")
if isinstance(dc_type, DayCountTypes) is False:
    raise FinError("Invalid_Day_Count_type.")
# Get day count times to use with curve day count convention
dc_times = times_from_dates(dts, .value_dt, .dc_type)
# We now get the discount factors using these times
zero_rates = ._zero_rate(dc_times)
# Now get the discount factors using curve conventions
dfs = ._zero_to_df(.value_dt, zero_rates, dc_times, .freq_type, .dc_type
)
# Convert these to zero rates in the required frequency and day count
zero_rates = ._df_to_zero(dfs, dts, freq_type, dc_type)
scalar_input = isinstance(dts, Date)
if scalar_input:
    return zero_rates[0]
return zero_rates
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
dt	list or Date	-
freq_type	FrequencyTypes	- Frequency
dc_type	DayCountTypes	- DayC
"""Calculation of zero rates with specified frequency according to polynomial parametrisation. This method overrides DiscountCurve. The parametrisation is not strictly in terms of continuously compounded zero rates this function allows other compounding and day counts. This function returns a single or vector of zero rates given a vector of dates so must use Numpy functions. The default frequency is a continuously compounded rate and ACT360 day counting."""	-	-
if isinstance(freq_type, FrequencyTypes) is False raise FinError("Invalid Frequency type.")	-	-
if isinstance(dc_type, DayCountTypes) is False raise FinError("Invalid Day Count type.")	-	-
dc_times	-	times_from_d
zero_rates	-	.ze
)	-	-
zero_rates	-	.df_to_zero(
scalar_input	-	isin
if scalar_input	-	-
return zero_rates[0]	-	-
return zero_rates	-	-

zero_rate

Calculate the zero rate to maturity date but with times as inputs. This function is used internally and should be discouraged for external use. The compounding frequency defaults to that specified in the constructor of the curve object. Which may be annual to continuous.

```
_zero_rate(times: Union[float, np.ndarray]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	float or np.ndarray	-	-

df

Calculate the fwd rate to maturity date but with times as inputs. This function is used internally and should be discouraged for external use. The compounding frequency defaults to that specified in the constructor of the curve object.

```
df(dates: Union[list, Date]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dates	list or Date	-	-

bump

PLEASE ADD A FUNCTION DESCRIPTION

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

__repr__

Display internal parameters of curve.

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.7 discount_curve_pwf

Class: DiscountCurvePWF(DiscountCurve)

Curve is made up of a series of zero rates sections with each having a piecewise flat zero rate. The default compounding assumption is continuous. The class inherits methods from DiscountCurve.

DiscountCurvePWF

PLEASE ADD A FUNCTION DESCRIPTION

```
DiscountCurvePWF(value_dt: Date,
zero_dts: list[Date],
zero_rates: Union[list, np.ndarray],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
"""Creates a discount curve using a vector of times and zero rates
that assumes that the zero rates are piecewise flat."""
check_argument_types(.DiscountCurvePWF, locals())
.value_dt = value_dt
if len(zero_dts) != len(zero_rates):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
zero_dts	list[Date]	-	-
zero_rates	list or np.ndarray	-	-
freq_type	FrequencyTypes	-	FrequencyTypes.C
dc_type	DayCountTypes	-	DayCountTypes
"""Creates a discount curve using a vector of times and zero rates that assumes that the zero rates are piecewise flat."""	-	-	-
check_argument_types(.DiscountCurvePWF, locals())	-	-	-
.value_dt	-	-	value.d

zero_rate

PLEASE ADD A FUNCTION DESCRIPTION

```
zero_rate(dts: Union[list, Date],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
if isinstance(freq_type, FrequencyTypes) is False:
    raise FinError("Invalid_Frequency_type.")
if isinstance(dc_type, DayCountTypes) is False:
    raise FinError("Invalid_Day_Count_type.")
dc_times = times_from_dates(dts, .value_dt, .dc_type)
zero_rates = ._zero_rate(dc_times)
dfs = ._zero_to_df(.value_dt, zero_rates, dc_times, .freq_type, .dc_type
)
```

```
zero_rates = ._df_to_zero(dfs, dts, freq_type, dc_type)
if isinstance(dts, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dts	list or Date	-	-
freq_type	FrequencyTypes	-	FrequencyTypes.CONTINUOUS
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
if isinstance(freq_type, FrequencyTypes) is False		-	-
raise FinError("Invalid Frequency type.")	-	-	-
if isinstance(dc_type, DayCountTypes) is False		-	-
raise FinError("Invalid Day Count type.")	-	-	-
dc_times	-	-	times_from_dates(dts, .value_dt, .dc_t
zero_rates	-	-	._zero_rate(dc_times)
)	-	-	-
zero_rates	-	-	._df_to_zero(dfs, dts, freq_type, dc_ty

zero_rate

The piecewise flat zero rate is selected and returned.

```
_zero_rate(times: Union[float, np.ndarray, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	float or np.ndarray,list	-	-

fwd

Calculate the continuously compounded forward rate at the forward time provided. This is done by perturbing the time by a small amount and measuring the change in the log of the discount factor divided by the time increment dt.

```
_fwd(times: Union[np.ndarray, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	np.ndarray or list	-	-

df

Return discount factors given a single or vector of dates. The discount factor depends on the rate and this in turn depends on its compounding frequency and it defaults to continuous compounding. It also depends on the day count convention. This was set in the construction of the curve to be ACT_ACT_ISDA.


```
df(dates: Union[Date, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dates	Date or list	-	-

`--repr--`

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.8 discount_curve_pwf_onf

Class: DiscountCurvePWFONF(DiscountCurve)

Curve with piece-wise flat instantaneous (ON) fwd rates. Curve is made up of a series of sections with each having a flat instantaneous forward rate. The default compounding assumption is continuous. The class inherits methods from DiscountCurve.

DiscountCurvePWFONF

PLEASE ADD A FUNCTION DESCRIPTION

```
DiscountCurvePWFONF(value_dt: Date,
knot_dts: list,
onfwd_rates: Union[list, np.ndarray],
"""
Creates a discount curve using a vector of times and ON fwd rates
The fwd rate is right-continuous i.e. a given value in the input
is applied to the left of that date until the previous date exclusive
The last fwd rate is extrapolated into the future
"""
check_argument_types(.DiscountCurvePWFONF, locals())
.value_dt = value_dt
if len(knot_dts) != len(onfwd_rates):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
knot_dts	list	-	-
onfwd_rates	list or np.ndarray	-	-
"""	-	-	-
Creates a discount curve using a vector of times and ON fwd rates	-	-	-
The fwd rate is right-continuous i.e. a given value in the input	-	-	-
is applied to the left of that date until the previous date exclusive	-	-	-
The last fwd rate is extrapolated into the future	-	-	-
"""	-	-	-
check_argument_types(.DiscountCurvePWFONF, locals())	-	-	-
.value_dt	-	-	value_dt

brick_wall_curve

PLEASE ADD A FUNCTION DESCRIPTION

```
brick_wall_curve(cls,
valuation_date: Date,
start_dt: Date,
end_dt: Date,
level: float = 1.0 * G_BASIS_POINT,
```

```

"""Generate a discount curve of the shape
_____f(t) = level*1_{startdate <= t <= enddate}
_____where f(.) is the instantaneous forward rate
____Mostly useful for applying bumps to other discount curve's,
____see composite_discount_curve.py
Args:
_____valuation_date (Date): valuation date for the discount curve

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
valuation_date	Date	-	-
start_dt	Date	-	-
end_dt	Date	-	-
level	float	-	1.0 * G_BASIS_POINT
"""Generate a discount curve of the shape	-	-	-
f(t)	-	-	level*1_startdate ; t ;= enddate
where f(.) is the instantaneous forward rate	-	-	-
Mostly useful for applying bumps to other discount curve's	-	-	-
see composite_discount_curve.py	-	-	-
Args	-	-	-

flat_curve

PLEASE ADD A FUNCTION DESCRIPTION

```
flat_curve(cls, valuation_date: Date, level: float = 1.0 * G_BASIS_POINT):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
valuation_date	Date	-	-
level	float	-	1.0 * G_BASIS_POINT

_zero_rate

Piecewise flat instantaneous (ON) fwd rate is the same as linear logDfs

```
_zero_rate(times: Union[float, np.ndarray, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	float or np.ndarray,list	-	-

df_t

Return discount factors given a single or vector of times in years. The discount factor depends on the rate and this in turn depends on its compounding frequency and it defaults to continuous compounding. It also depends on the day count convention. This was set in the construction of the curve to be ACT_ACT_ISDA.

```
df_t(t: Union[float, np.ndarray]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float or np.ndarray	-	-

bump

PLEASE ADD A FUNCTION DESCRIPTION

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.9 discount_curve_pwl

Class: DiscountCurvePWL(DiscountCurve)

Curve is made up of a series of sections assumed to each have a piece-wise linear zero rate. The zero rate has a specified frequency which defaults to continuous. This curve inherits all of the extra methods from DiscountCurve.

DiscountCurvePWL

PLEASE ADD A FUNCTION DESCRIPTION

```
DiscountCurvePWL(value_dt: Date,
zero_dts: Union[Date, list],
zero_rates: Union[list, np.ndarray],
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
dc_type: DayCountTypes = DayCountTypes.ACT_ACT_ISDA,
"""Curve is defined by a vector of increasing times and zero rates."""
check_argument_types(.DiscountCurvePWL, locals())
.value_dt = value_dt
if len(zero_dts) != len(zero_rates):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	
zero_dts	Date or list	-	
zero_rates	list or np.ndarray	-	
freq_type	FrequencyTypes	-	FrequencyTypes
dc_type	DayCountTypes	-	DayCountTypes
"""Curve is defined by a vector of increasing times and zero rates."""	-	-	
check_argument_types(.DiscountCurvePWL, locals())	-	-	
.value_dt	-	-	val

zero_rate

Calculate the piecewise linear zero rate. This is taken from the initial inputs. A simple linear interpolation scheme is used. If the user supplies a frequency type then a conversion is done.

```
_zero_rate(times: Union[list, np.ndarray]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	list or np.ndarray	-	-

df

Return discount factors given a single or vector of dates. The discount factor depends on the rate and this in turn depends on its compounding frequency and it defaults to continuous compounding. It also depends on the day count convention. This was set in the construction of the curve to be ACT/ACT/ISDA.

```
df(dates: Union[Date, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dates	Date or list	-	-

bump

PLEASE ADD A FUNCTION DESCRIPTION

```
bump(bump_size: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_size	float	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.10 discount_curve_zeros

Class: DiscountCurveZeros(DiscountCurve)

This is a curve calculated from a set of dates and zero rates. As we have rates as inputs, we need to specify the corresponding compounding frequency. Also to go from rates and dates to discount factors we need to compute the year fraction correctly and for this we require a day count convention. Finally, we need to interpolate the zero rate for the times between the zero rates given and for this we must specify an interpolation convention. The class inherits methods from DiscountCurve.

DiscountCurveZeros

PLEASE ADD A FUNCTION DESCRIPTION

```
DiscountCurveZeros(value_dt: Date,
zero_dts: list,
zero_rates: Union[list, np.ndarray],
freq_type: FrequencyTypes = FrequencyTypes.ANNUAL,
dc_type: DayCountTypes = DayCountTypes.ACT_ACT_ISDA,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
    """Create the discount curve from a vector of dates and zero rates
    factors. The first date is the curve anchor. Then a vector of zero
    dates and then another same-length vector of rates. The rate is to the
    corresponding date. We must specify the compounding frequency of the
    zero rates and also a day count convention for calculating times which
    we must do to calculate discount factors. Finally we specify the
    interpolation scheme for off-grid dates."""
    check_argument_types(.DiscountCurveZeros, locals())
    # Validate curve
    if len(zero_dts) == 0:
        raise FinError("Dates has zero length")
    if len(zero_dts) != len(zero_rates):
```

The function arguments are described in the following table.

Argument Name	Type	Description	De
value_dt	Date	-	
zero_dts	list	-	
zero_rates	list or np.ndarray	-	
freq_type	FrequencyTypes	-	Frequency
dc_type	DayCountTypes	-	DayCountTy
interp_type	InterpTypes	-	InterpTypes.
"""Create the discount curve from a vector of dates and zero rates		-	
factors. The first date is the curve anchor. Then a vector of zero		-	
dates and then another same-length vector of rates. The rate is to the		-	
corresponding date. We must specify the compounding frequency of the		-	
zero rates and also a day count convention for calculating times which		-	
we must do to calculate discount factors. Finally we specify the		-	
interpolation scheme for off-grid dates."""		-	
check_argument_types(.DiscountCurveZeros, locals())		-	
if len(zero_dts)		-	
raise FinError("Dates has zero length")		-	

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

2.11 interpolator

Enumerated Type: InterpTypes

This enumerated type has the following values:

- FLAT_FWD_RATES
- LINEAR_FWD_RATES
- LINEAR_ZERO_RATES
- FINCUBIC_ZERO_RATES
- NATCUBIC_LOG_DISCOUNT
- NATCUBIC_ZERO_RATES
- PCHIP_ZERO_RATES
- PCHIP_LOG_DISCOUNT
- LINEAR_ONFWD_RATES
- TENSION_ZERO_RATES
- raiseFinError("Interpolatetimesmustallbe¿
- u
- raiseFinError("Interpolatetimesmustallbe¿
- v
- fastmath
- cache
- nogil
- small
- num_points
- ift
- i
- i
- i
- yvalue
- ifmethod
- ifi
- r1
- r2
- dt
- rvalue
- yvalue
- r1
- r2
- dt
- rvalue
- yvalue
- r1
- r2
- dt
- rvalue
- yvalue

- elifmethod
- ifi
- rt1
- rt2
- dt
- rtvalue
- yvalue
- rt1
- rt2
- dt
- rtvalue
- yvalue
- rt1
- rt2
- dt
- rtvalue
- yvalue
- elifmethod
- ifi
- y2
- yvalue
- yvalue
- fwd1
- fwd2
- dt
- fwd
- yvalue
- fwd
- yvalue
- fastmath
- cache
- nogil
- n
- yvalues
- yvalues[i]
- self._interp_type
- self._interp_fn
- self._times
- self._dfs
- self._refit_curve
- self._optional_interp_params
- self._times
- self._dfs
- iflen(times)
- ifself._interp_type
- log_dfs

- self._interp_fn
- elifself._interp_type
- g_small_vector
- zero_rates
- ifself._times[0]
- zero_rates[0]
- self._interp_fn
- ifself._interp_type
- log_dfs
- self._interp_fn
- bc_type
- elifself._interp_type
- g_small_vector
- zero_rates
- ifself._times[0]
- zero_rates[0]
- self._interp_fn
- self._times,zero_rates,bc_type
- elifself._interp_type
- log_dfs
- self._interp_fn
- elifself._interp_type
- g_small_vector
- zero_rates
- ifself._times[0]
- zero_rates[0]
- self._interp_fn
- elifself._interp_type
- log_dfs
- self._interp_fn
- fill_value
- elifself._interp_type
- onf_times
- onf_rates
- prev_df
- ift
- iflen(onf_times)
- onfr
- onf_times
- onf_rates
- prev_t
- prev_r
- fwd_df
- onf_int
- r
- prev_df

- iflen(onf_times)
- self._interp_fn
- [0.0,0.1],[0.0,0.0],k
- self._interp_fn
- onf_times,onf_rates,k
- elifself._interp_type
- tension_sigma
- g_small_vector
- zero_rates
- ifself._times[0]
- zero_rates[0]
- self._interp_fn
- self._times,zero_rates,sigma
- raiseFinError("Interpolatetimesmustallbe;
- tvec
- raiseFinError("Interpolatetimesmustallbe;
- tvec
- out
- out
- elifself._interp_type
- (notsurewhywehaveiflen(times)
- iflen(self._dfs)
- out
- onf_rate
- out
- soworkaroundthat.Notethatwealwayshavet
- last_t
- i1
- i2
- log_dfs
- out
- out
- is_suitable

Class: Interpolator

Interpolator

PLEASE ADD A FUNCTION DESCRIPTION

```
Interpolator(interpolator_type: InterpTypes, **kwargs: dict):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
interpolator_type	InterpTypes	-	-
**kwargs	dict	-	-

fit

Fit the interpolator to the provided time and discount factor arrays.

```
fit(times: np.ndarray, dfs: np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	np.ndarray	-	-
dfs	np.ndarray	-	-

interpolate

Interpolation of discount factors at time x given discount factors at times provided using one of the methods in the enum `InterpTypes`. The value of x can be an array so that the function is vectorised.

```
interpolate(t: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float	-	-

true_integral

PLEASE ADD A FUNCTION DESCRIPTION

```
true_integral(spline, t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
spline	-	-	-
t	-	-	-

suitable_for_bootstrap

Check if the interpolation type is suitable for bootstrapping.

```
suitable_for_bootstrap(cls, interp_type):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cls	-	-	-
interp_type	-	-	-

interpolate

PLEASE ADD A FUNCTION DESCRIPTION

```
interpolate(
t: Union[float, np.ndarray], # time or array of times
times: np.ndarray, # Vector of times on grid
dfs: np.ndarray, # Vector of discount factors
method: int,
# Interpolation method which is value of enum
"""Fast interpolation of discount factors at time x given discount factors
at times provided using one of the methods in the enum InterpTypes. The
value of x can be an array so that the function is vectorised."""
if isinstance(t, (float, np.float64)):
```

The function arguments are described in the following table.

Argument Name	Type	Description
t	float or np.ndarray	time or array of times
times	np.ndarray	Vector of times on grid
dfs	np.ndarray	Vector of discount factors
method	int	-
"""Fast interpolation of discount factors at time x given discount factors	-	-
at times provided using one of the methods in the enum InterpTypes. The	-	-
value of x can be an array so that the function is vectorised."""	-	-

_uinterpolate

Return the interpolated value of y given x and a vector of x and y. The values of x must be monotonic and increasing. The different schemes for interpolation are linear in y (as a function of x), linear in log(y) and piecewise flat in the continuously compounded forward y rate.

```
_uinterpolate(t, times, dfs, method):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
times	-	-	-
dfs	-	-	-
method	-	-	-

_vinterpolate

Return the interpolated values of y given x and a vector of x and y. The values of x must be monotonic and increasing. The different schemes for interpolation are linear in y (as a function of x), linear in log(y) and piecewise flat in the continuously compounded forward y rate.

```
_vinterpolate(x_values, x_vector, dfs, method):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x_values	-	-	-
x_vector	-	-	-
dfs	-	-	-
method	-	-	-

2.12 `__init__`

Chapter 3

financepy.market.volatility

Market Volatility

These modules create a family of curve types related to the market volatility. There are three types of class:

1. Term structures of volatility i.e. volatility as a function of option expiry date.
2. Volatility curves which are smile/skews so store volatility as a function of option strike.
3. Volatility surfaces which hold volatility as a function of option expiry date AND option strike.

The classes are as follows:

equity_vol_surface

Constructs an equity volatility surface that fits to a grid of market volatilities at a set of strikes and expiry dates. It implements the SVI parametric form for fitting and interpolating volatilities. It also provides plotting of the volatility curve and surfaces.

FinFXVolSurface

FX volatility as a function of option expiry and strike. This class constructs the surface from the ATM volatility plus a choice of 10 and 25 delta strangles and risk reversals or both. This is done for multiple expiry dates. A number of curve fitting choices are possible including polynomial in delta and SABR.

FinlborCapFloorVol

Libor cap/floor volatility as a function of option expiry (cap/floor start date). Takes in cap (flat) volatility and bootstraps the caplet volatility. This is assumed to be piecewise flat.

FinlborCapFloorVolFn

Parametric function for storing the cap and caplet volatilities based on form proposed by Rebonato.

3.1 equity_vol_curve

Class: EquityVolCurve

Class to manage a smile or skew in volatility at a single maturity horizon. It fits the volatility using a polynomial. Includes analytics to extract the implied pdf of the underlying at maturity. THIS NEEDS TO BE SUBSTITUTED WITH FINEQUITYVOLSURFACE.

EquityVolCurve

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityVolCurve(curve_dt: Date,
expiry_dt: Date,
strikes: np.ndarray,
volatilities: np.ndarray,
polynomial: int = 3
None:
if expiry_dt <= curve_dt:
    raise FinError("Expiry_date_before_curve_date.")
if len(strikes) < 1:
    raise FinError("Volatility_grid_has_zero_length.")
if test_monotonicity(strikes) is False:
    raise FinError("Strikes_must_be_strictly_monotonic.")
num_strikes = len(strikes)
num_vols = len(volatilities)
if num_strikes != num_vols:
    raise FinError("Strike_and_volatility_vectors_not_same_length.")
for i in range(1, num_strikes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
curve_dt	Date	-	-
expiry_dt	Date	-	-
strikes	np.ndarray	-	-
volatilities	np.ndarray	-	-
polynomial	int	-	3
None	-	-	-
if expiry_dt ≤	-	-	curve_dt:
raise FinError("Expiry date before curve date.")	-	-	-
if len(strikes) ≤ 1	-	-	-
raise FinError("Volatility grid has zero length.")	-	-	-
if test_monotonicity(strikes) is False	-	-	-
raise FinError("Strikes must be strictly monotonic.")	-	-	-
num_strikes	-	-	len(strikes)
num_vols	-	-	len(volatilities)
if num_strikes ≠	-	-	num_vols:
raise FinError("Strike and volatility vectors not same length.")	-	-	-

volatility

Return the volatility for a strike using a given polynomial interpolation.

```
volatility(strike: float) -> float:
"""Return the volatility for a strike using a given polynomial
interpolation."""
vol = ._f(strike)
if vol.any() < 0.0:
    raise FinError("Negative volatility. Not permitted.")
return vol
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
strike	float)-ζfloat:	-	-
"""Return the volatility for a strike using a given polynomial interpolation."""	-	-	-
vol	-	-	._f(strike)
if vol.any() < 0.0	-	-	-
raise FinError("Negative volatility. Not permitted.")	-	-	-
return vol	-	-	-

calculate_pdf

calculate the probability density function of the underlying using the volatility smile or skew curve following the approach set out in Breedon and Litzenberger.

```
calculate_pdf() -> Optional[Any]:
"""calculate the probability density function of the underlying using
the volatility smile or skew curve following the approach set out in
Breedon and Litzenberger."""
return
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
"""calculate the probability density function of the underlying using the volatility smile or skew curve following the approach set out in Breedon and Litzenberger."""	-	-	-
return	-	-	-

3.2 equity_vol_surface

Class: EquityVolSurface

Class to perform a calibration of a chosen parametrised surface to the prices of equity options at different strikes and expiry tenors. There is a choice of volatility function from cubic in delta to full SABR and SSVI. Check out VolFuncTypes. Visualising the volatility curve is useful. Also, there is no guarantee that the implied pdf will be positive.

EquityVolSurface

Create the EquitySurface object by passing in market vol data for a list of strikes and expiry dates.

```
EquityVolSurface(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
expiry_dts: List[Date],
strikes: np.ndarray,
volatility_grid: np.ndarray,
vol_func_type: VolFuncTypes = VolFuncTypes.CLARK,
fin_solver_type: SolverTypes = SolverTypes.NELDER_MEAD,
    None:
    """Create the EquitySurface object by passing in market vol data
    for a list of strikes and expiry dates."""
    check_argument_types(.EquityVolSurface, locals())
    .value_dt = value_dt
    ._stock_price = stock_price
    ._discount_curve = discount_curve
    ._dividend_curve = dividend_curve
    num_expiry_dts = len(expiry_dts)
    num_strikes = len(strikes)
    n = len(volatility_grid)
    m = len(volatility_grid[0])
    if n != num_expiry_dts:
        raise FinError("1st_dim_of_vol_grid_is_not_num_expiry_dts")
    if m != num_strikes:
        raise FinError("2nd_dim_of_the_vol_matrix_is_not_num_strikes")
    ._strikes = strikes
    ._num_strikes = len(strikes)
    ._expiry_dts = expiry_dts
    ._num_expiry_dts = len(expiry_dts)
    ._volatility_grid = volatility_grid
    ._vol_func_type = vol_func_type
    ._build_vol_surface(fin_solver_type=fin_solver_type)
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
expiry_dts	List[Date]	-	-
strikes	np.ndarray	-	-
volatility_grid	np.ndarray	-	-
vol_func_type	VolFuncTypes	-	VolFuncTypes.CL
fin_solver_type	SolverTypes	-	SolverTypes.NELDER
None		-	-
"""Create the EquitySurface object by passing in market vol data for a list of strikes and expiry dates."""	-	-	-
check_argument_types(.EquityVolSurface, locals())	-	-	-
.value_dt	-	-	value_dt
._stock_price	-	-	stock_price
._discount_curve	-	-	discount_curve
._dividend_curve	-	-	dividend_curve
num_expiry_dts	-	-	len(expiry_dts)
num_strikes	-	-	len(strikes)
n	-	-	len(volatility_grid)
m	-	-	len(volatility_grid)
if n !=	-	-	num_expiry_dts
raise FinError("1st dim of vol grid is not num_expiry_dts")	-	-	-
if m !=	-	-	num_strikes
raise FinError("2nd dim of the vol matrix is not num_strikes")	-	-	-
._strikes	-	-	strikes
._num_strikes	-	-	len(strikes)
._expiry_dts	-	-	expiry_dts
._num_expiry_dts	-	-	len(expiry_dts)
._volatility_grid	-	-	volatility_grid
._vol_func_type	-	-	vol_func_type
._build_vol_surface(fin_solver_type)	-	-	fin_solver_type

vol_from_strike_dt

PLEASE ADD A FUNCTION DESCRIPTION

```
vol_from_strike_dt(k: float, expiry_dt: Date) -> float:
"""Interpolates the Black-Scholes volatility from the volatility
surface given call option strike and expiry date. Linear interpolation
is done in variance space. The smile strikes at bracketed dates are
determined by determining the strike that reproduces the provided delta
value. This uses the calibration delta convention, but it can be
overridden by a provided delta convention. The resulting volatilities
are then determined for each bracketing expiry time and linear
interpolation is done in variance space and then converted back to a
```

```
lognormal_volatility."""
t_exp = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
vol_type_value = ._vol_func_type.value
index0 = 0 # lower index in bracket
index1 = 0 # upper index in bracket
num_curves = ._num_expiry_dts
if num_curves == 1:
    index0 = 0
    index1 = 0
# If the time is below first time then assume a flat vol
elif t_exp <= ._t_exp[0]:
    index0 = 0
    index1 = 0
# If the time is beyond the last time then extrapolate with a flat vol
elif t_exp >= ._t_exp[-1]:
    index0 = len(._t_exp) - 1
    index1 = len(._t_exp) - 1
else: # Otherwise we look for bracketing times and interpolate
    for i in range(1, num_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description
k	float	-
expiry_dt	Date)->float:	-
Interpolates the Black-Scholes volatility from the volatility surface given call option strike and expiry date. Linear interpolation is done in variance space. The smile strikes at bracketed dates are determined by determining the strike that reproduces the provided delta value. This uses the calibration delta convention but it can be overridden by a provided delta convention. The resulting volatilities are then determined for each bracketing expiry time and linear interpolation is done in variance space and then converted back to a lognormal volatility.	-	-
t_exp	-	-
vol_type_value	-	-
index0	-	lower index in bracket
index1	-	upper index in bracket
num_curves	-	-
if num_curves	-	-
index0	-	-
index1	-	-
elif t_exp < index0	-	-
index0	-	-
index1	-	-
elif t_exp > index1	-	-
index0	-	-
index1	-	-
else	Otherwise we look for bracketing	

vol from delta date

PLEASE ADD A FUNCTION DESCRIPTION

```

vol_from_delta_date(call_delta: float,
                    expiry_dt: Date,
                    delta_method=None,
                    Tuple[float, float]):
    """Interpolates the Black-Scholes volatility from the volatility
    surface given a call option delta and expiry date. Linear interpolation
    is done in variance space. The smile strikes at bracketed dates are
    determined by determining the strike that reproduces the provided delta
    value. This uses the calibration delta convention, but it can be
    overridden by a provided delta convention. The resulting volatilities
    are then determined for each bracketing expiry time and linear
    interpolation is done in variance space and then converted back to a
    lognormal volatility."""
    t_exp = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
    vol_type_value = ._vol_func_type.value

```



```
s = ._stock_price
index0 = 0 # lower index in bracket
index1 = 0 # upper index in bracket
num_curves = ._num_expiry_dts
# If there is only one time horizon then assume flat vol to this time
if num_curves == 1:
    index0 = 0
    index1 = 0
# If the time is below first time then assume a flat vol
elif t_exp <= ._t_exp[0]:
    index0 = 0
    index1 = 0
# If the time is beyond the last time then extrapolate with a flat vol
elif t_exp > ._t_exp[-1]:
    index0 = len(._t_exp) - 1
    index1 = len(._t_exp) - 1
else: # Otherwise we look for bracketing times and interpolate
    for i in range(1, num_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description
call_delta	float	-
expiry_dt	Date	-
delta_method	-	-
Tuple[float, float]	-	-
"""Interpolates the Black-Scholes volatility from the volatility surface given a call option delta and expiry date. Linear interpolation is done in variance space. The smile strikes at bracketed dates are determined by determining the strike that reproduces the provided delta value. This uses the calibration delta convention but it can be overridden by a provided delta convention. The resulting volatilities are then determined for each bracketing expiry time and linear interpolation is done in variance space and then converted back to a lognormal volatility."""	-	-
t_exp	-	-
vol_type_value	-	-
s	-	-
index0	-	lower index in bracket
index1	-	upper index in bracket
num_curves	-	-
if num_curves	-	-
index0	-	-
index1	-	-
elif t_exp i	-	-
index0	-	-
index1	-	-
elif t_exp i . t_exp[-1]	-	-
index0	-	-
index1	-	-
else	Otherwise we look for bracketing times	

_build_vol_surface

PLEASE ADD A FUNCTION DESCRIPTION

```

_build_vol_surface(fin_solver_type: Any = SolverTypes.NELDER_MEAD
None:
"""Main_function_to_construct_the_vol_surface."""
s = ._stock_price
num_expiry_dts = ._num_expiry_dts
if ._vol_func_type == VolFuncTypes.CLARK:
    num_parameters = 3
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.SABR_BETA_ONE:
    num_parameters = 3
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.SABR_BETA_HALF:

```

```

    num_parameters = 3
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.BBG:
    num_parameters = 3
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.SABR:
    num_parameters = 4
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.CLARK5:
    num_parameters = 5
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.SVI:
    num_parameters = 5
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type == VolFuncTypes.SSVI:
    num_parameters = 5
    ._parameters = np.zeros([num_expiry_dts, num_parameters])
    ._parameters[:, 0] = 0.2 # sigma
    ._parameters[:, 1] = 0.8 # gamma
    ._parameters[:, 2] = -0.7 # rho
    ._parameters[:, 3] = 0.3
    ._parameters[:, 4] = 0.048
else:
    print(._vol_func_type)
    raise FinError("Unknown_Model_Type")
._t_exp = np.zeros(num_expiry_dts)
._fwd_0_t = np.zeros(num_expiry_dts)
._r = np.zeros(num_expiry_dts)
._q = np.zeros(num_expiry_dts)
#####
# TODO: ADD SPOT DAYS
#####
spot_dt = .value_dt
for i in range(0, num_expiry_dts):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fin_solver_type	Any	-	SolverTypes.NELDER_MEAD
None	-	-	-
"""Main function to construct the vol surface."""	-	-	-
s	-	-	._stock_price
num_expiry_dts	-	-	._num_expiry_dts
if ._vol_func_type	-	-	= VolFuncTypes.CLARK:
num_parameters	-	-	3
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.SABR_BETA_ONE:
num_parameters	-	-	3
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.SABR_BETA_HALF:
num_parameters	-	-	3
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.BBG:
num_parameters	-	-	3
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.SABR:
num_parameters	-	-	4
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.CLARK5:
num_parameters	-	-	5
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.SVI:
num_parameters	-	-	5
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
elif ._vol_func_type	-	-	= VolFuncTypes.SSVI:
num_parameters	-	-	5
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
._parameters[	,0]	sigma	0.2
._parameters[	,1]	gamma	0.8
._parameters[	,2]	rho	-0.7
._parameters[	,3]	-	0.3
._parameters[	,4]	-	0.048
else	-	-	-
print(._vol_func_type)	-	-	-
raise FinError("Unknown Model Type")	-	-	-
._t_exp	-	-	np.zeros(num_expiry_dts)
._fwd_0_t	-	-	np.zeros(num_expiry_dts)
._r	-	-	np.zeros(num_expiry_dts)
._q	-	-	np.zeros(num_expiry_dts)
spot_dt	-	-	._value_dt

check_calibration

PLEASE ADD A FUNCTION DESCRIPTION

```
check_calibration(verbose: bool) -> None:
    """Compare_calibrated_vol_surface_with_market_and_output_a_report
    which_sets_out_the_quality_of_fit_to_the_ATM_and_10_and_25_delta_market
    strangles_and_risk_reversals."""
    if verbose:
        print("=====")
        print("VALUE_DATE:", .value_dt)
        print("STOCK_PRICE:", ._stock_price)
        print("=====")
    for i in range(0, ._num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description
verbose	bool)->None:	-
"""Compare calibrated vol surface with market and output a report which sets out the quality of fit to the ATM and 10 and 25 delta market strangles and risk reversals."""	-	-
if verbose	-	-
print("	-	-
print("VALUE DATE	",".value_dt)	-
print("STOCK PRICE	","._stock_price)	-
print("	-	-

implied_dbns

PLEASE ADD A FUNCTION DESCRIPTION

```
implied_dbns(low_s: float, high_s: float, num_intervals: int
    List[FinDistribution]:
    """Calculate_the_pdf_for_each_tenor_horizon._Returns_a_list_of
    FinDistribution_objects,_one_for_each_tenor_horizon."""
    dbns = []
    for i_tenor in range(0, ._num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
low_s	float	-	-
high_s	float	-	-
num_intervals	int	-	-
List[FinDistribution]		-	-
"""Calculate the pdf for each tenor horizon. Returns a list of FinDistribution objects one for each tenor horizon."""	-	-	-
dbns	-	-	[]

plot_vol_curves

PLEASE ADD A FUNCTION DESCRIPTION

```
plot_vol_curves() -> None:
"""Generates a plot of each of the vol discount implied by the market
and fitted."""
low_k = ._strikes[0] * 0.9
high_k = ._strikes[-1] * 1.1
for tenor_index in range(0, ._num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> None		-	-
"""Generates a plot of each of the vol discount implied by the market and fitted."""	-	-	-
low_k	-	-	._strikes[0] * 0.9
high_k	-	-	._strikes[-1] * 1.1

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("VALUE_DATE", .value_dt)
s += label_to_string("STOCK_PRICE", ._stock_price)
s += label_to_string("VOL_FUNCTION", ._vol_func_type)
for i in range(0, ._num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str	-	-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("VALUE DATE", .value_dt)
s +	-	-	label_to_string("STOCK PRICE", .stock_price)
s +	-	-	label_to_string("VOL FUNCTION", .vol_func_type)

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print() -> None:
"""Print a list of the unadjusted coupon payment dates used in
analytic calculations for the bond."""
print()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> None	-	-	-
"""Print a list of the unadjusted coupon payment dates used in	-	-	-
analytic calculations for the bond."""	-	-	-
print()	-	-	-

_obj

PLEASE ADD A FUNCTION DESCRIPTION

```
_obj(params: np.ndarray, *args: Any) -> float:
"""Return a value that is minimised when the ATM, MS and RR vols have
been best fitted using the parametric volatility curve represented by
params and specified by the vol_type_value. We fit at one time slice only.
"""
s = args[0]
t = args[1]
r = args[2]
q = args[3]
strikes = args[4]
index = args[5]
volatility_grid = args[6]
vol_type_value = args[7]
f = s * np.exp((r - q) * t)
num_strikes = len(volatility_grid[0])
tot = 0.0
for i in range(0, num_strikes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	np.ndarray	-	-
*args	Any)-float:	-	-
"""Return a value that is minimised when the ATM MS and RR vols have	-	-	-
been best fitted using the parametric volatility curve represented by params and specified by the vol_type_value. We fit at one time slice only.	-	-	-
"""	-	-	-
s	-	-	args[0]
t	-	-	args[1]
r	-	-	args[2]
q	-	-	args[3]
strikes	-	-	args[4]
index	-	-	args[5]
volatility_grid	-	-	args[6]
vol_type_value	-	-	args[7]
f	-	-	$s * \exp((r - q) * t)$
num_strikes	-	-	len(volatility_grid)
tot	-	-	0.0

solve to horizon

PLEASE ADD A FUNCTION DESCRIPTION

```

_solve_to_horizon(
s: float,
t: float,
r: float,
q: float,
strikes: Sequence[float],
time_index: int,
volatility_grid: Union[List[Sequence[float]], np.ndarray],
vol_type_value: int,
x_inits: Sequence[float],
fin_solver_type: Any,
    np.ndarray:
#####
# Determine parameters of vol surface using minimisation
#####
tol = 1e-6
args = (s, t, r, q, strikes, time_index, volatility_grid, vol_type_value)
# Nelder-Mead (both SciPy and Numba) is quicker, but occasionally fails
# to converge, so for those cases try again with CG
# Numba version is quicker, but can be slightly away from CG output
xopt = None
try:
    if fin_solver_type == SolverTypes.NELDER_MEAD_NUMBA:
        xopt = nelder_mead(
            _obj,
            np.array(x_inits),
            bounds=np.array([[], []]).T,

```



```

        args=args,
        tol_f=tol,
        tol_x=tol,
        max_iter=1000,
    )
    elif fin_solver_type == SolverTypes.NELDER_MEAD:
        opt = minimize(_obj, x_inits, args, method="Nelder-Mead", tol=tol)
        xopt = opt.x
    elif fin_solver_type == SolverTypes.CONJUGATE_GRADIENT:
        opt = minimize(_obj, x_inits, args, method="CG", tol=tol)
        xopt = opt.x
except Exception:
    # If convergence fails try again with CG if necessary
    if fin_solver_type != SolverTypes.CONJUGATE_GRADIENT:
        print("Failed_to_converge,_will_try_CG")
        opt = minimize(_obj, x_inits, args, method="CG", tol=tol)
        xopt = opt.x
params = np.array(xopt)
return params
#####
t(
float64(int64, float64[:], float64, float64, float64),
cache=True,
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
r	float	-	-
q	float	-	-
strikes	Sequence[float]	-	-
time_index	int	-	-
volatility_grid	List[Sequence[float]] or	-	-
vol_type_value	int	-	-
x_inits	Sequence[float]	-	-
fin_solver_type	Any	-	-
np.ndarray		-	-
tol	-	-	1e-6
args	-	(s, t, r, q, strikes, time_index, volatility)	-
xopt	-	-	None
try	-	-	-
if fin_solver_type	-	-	= SolverTypes.NELDER_MEAD
_obj	-	-	-
np.array(x_inits)	-	-	-
bounds	-	-	np.array([[], []])
args	-	-	args
tol_f	-	-	tol
tol_x	-	-	tol
max_iter	-	-	1000
)	-	-	-
elif fin_solver_type	-	-	= SolverTypes.NELDER_MEAD
opt	-	minimize(_obj, x_inits, args, method='Nelder-Mead')	-
xopt	-	-	opt.x
elif fin_solver_type	-	-	= SolverTypes.CONJUGATE_GRAD
opt	-	minimize(_obj, x_inits, args, method='Conjugate Gradient')	-
xopt	-	-	opt.x
except Exception	-	-	-
if fin_solver_type !	-	-	SolverTypes.CONJUGATE_GRAD
print("Failed to converge, will try CG")	-	-	-
opt	-	minimize(_obj, x_inits, args, method='CG')	-
xopt	-	-	opt.x
params	-	-	np.array(xopt)
return params	-	-	-
float64(int64, float64[	],float64,float64,float64)	-	-
cache	-	-	True
fastmath	-	-	True

vol function

Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark

book.

```

vol_function(
vol_function_type_value: int, params: np.ndarray, f: float, k: float, t: float
float:
"""Return the volatility for a strike using a given polynomial
interpolation following Section 3.9 of Iain Clark book."""
if vol_function_type_value == VolFuncTypes.CLARK.value:
    vol = vol_function_clark(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.SABR_BETA_ONE.value:
    vol = vol_function_sabr_beta_one(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.SABR_BETA_HALF.value:
    vol = vol_function_sabr_beta_half(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.BBG.value:
    vol = vol_function_bloomberg(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.SABR.value:
    vol = vol_function_sabr(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.CLARK5.value:
    vol = vol_function_clark(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.SVI.value:
    vol = vol_function_svi(params, f, k, t)
    return vol
if vol_function_type_value == VolFuncTypes.SSVI.value:
    vol = vol_function_ssvi(params, f, k, t)
    return vol
return 0.0
#####
t(cache=True, fastmath=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
vol_function_type_value	int	-	-
params	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float	-	-	-
"""Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book."""	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.CLARK
vol	-	-	vol_function_clark(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SABR_BETA
vol	-	-	vol_function_sabr_beta_one(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SABR_BETA
vol	-	-	vol_function_sabr_beta_half(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.BBG
vol	-	-	vol_function_bloomberg(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SABR
vol	-	-	vol_function_sabr(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.CLARK
vol	-	-	vol_function_clark(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SVI
vol	-	-	vol_function_svi(params)
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SSVI
vol	-	-	vol_function_ssvi(params)
return vol	-	-	-
return 0.0	-	-	-
t(cache	-	-	True, fastmath=True

_delta_fit

PLEASE ADD A FUNCTION DESCRIPTION

```
_delta_fit(k: float, *args: Any) -> float:
    """This is the objective function used in the determination of the
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
*args	Any)-float:	-	-
"""This is the objective function used in the determination of the		-	-

solver_for_smile_strike

Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike.

```

_solver_for_smile_strike(
s: float,
t: float,
r: float,
q: float,
opt_type_value: int,
vol_type_value: int,
delta_target: float,
initial_guess: float,
parameters: np.ndarray,
    float:
    """Solve_for_the_strike_that_sets_the_delta_of_the_option_equal_to_the
    target_value_of_delta_allowing_the_volatility_to_be_a_function_of_the
    strike."""
    inverse_delta_target = norminvcdf(np.abs(delta_target))
    argtuple = (
        vol_type_value,
        s,
        t,
        r,
        q,
        opt_type_value,
        inverse_delta_target,
        parameters,
    )
    k = newton_secant(_delta_fit, x0=initial_guess, args=argtuple, tol=1e-8, maxiter=50)
    return k
#####
able to cache function and if I remove njit it complains about pickle
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
s	float	-
t	float	-
r	float	-
q	float	-
opt_type_value	int	-
vol_type_value	int	-
delta_target	float	-
initial_guess	float	-
parameters	np.ndarray	-
float		-
"""Solve for the strike that sets the delta of the option equal to the	-	-
target value of delta allowing the volatility to be a function of the	-	-
strike."""	-	-
inverse_delta_target	-	-
vol_type_value	-	-
s	-	-
t	-	-
r	-	-
q	-	-
opt_type_value	-	-
inverse_delta_target	-	-
parameters	-	-
)	-	-
k	-	- newton_secant(_delta_fit)
return k	-	-
able to cache function and if I remove njit it complains about pickle	-	-

3.3 fx_vol_surface

Class: FXVolSurface

Class to perform a calibration of a chosen parametrised surface to the prices of FX options at different strikes and expiry tenors. The calibration inputs are the ATM and 25 Delta volatilities given in terms of the market strangle and risk reversals. There is a choice of volatility function ranging from polynomial in delta to a limited version of SABR.

FXVolSurface

PLEASE ADD A FUNCTION DESCRIPTION

```
FXVolSurface(value_dt: Date,
spot_fx_rate: float,
currency_pair: str,
notional_currency: str,
domestic_curve: DiscountCurve,
foreign_curve: DiscountCurve,
tenors: List[Tenor],
atm_vols: np.ndarray,
ms_25_delta_vols: np.ndarray,
rr_25_delta_vols: np.ndarray,
atm_method: FinFXATMMethod = FinFXATMMethod.FWD_DELTA_NEUTRAL,
delta_method: FinFXDeltaMethod = FinFXDeltaMethod.SPOT_DELTA,
vol_func_type: VolFuncTypes = VolFuncTypes.CLARK,
None:
"""Create the FinFXVolSurface object by passing in market vol data
for ATM and 25 Delta Market Strangles and Risk Reversals."""
check_argument_types(.FXVolSurface, locals())
.value_dt = value_dt
.spot_fx_rate = spot_fx_rate
.currency_pair = currency_pair
if len(currency_pair) != 6:
    raise FinError("Currency pair must be 6 characters.")
.for_name = .currency_pair[0:3]
.dom_name = .currency_pair[3:6]
.notional_currency = notional_currency
.domestic_curve = domestic_curve
.foreign_curve = foreign_curve
.num_vol_curves = len(tenors)
if len(atm_vols) != .num_vol_curves:
    raise FinError("Number ATM vols must equal number of tenors")
if len(atm_vols) != .num_vol_curves:
    raise FinError("Number ATM vols must equal number of tenors")
if len(ms_25_delta_vols) != .num_vol_curves:
    raise FinError("Number MS25d vols must equal number of tenors")
if len(rr_25_delta_vols) != .num_vol_curves:
    raise FinError("Number RR25d vols must equal number of tenors")
.tenors = tenors
.atm_vols = np.array(atm_vols) / 100.0
.ms_25_delta_vols = np.array(ms_25_delta_vols) / 100.0
.rr_25_delta_vols = np.array(rr_25_delta_vols) / 100.0
.atm_method = atm_method
.delta_method = delta_method
```

```

if .delta_method == FinFXDeltaMethod.SPOT_DELTA:
    .delta_method_string = "pips_spot_delta"
elif .delta_method == FinFXDeltaMethod.FORWARD_DELTA:
    .delta_method_string = "pips_fwd_delta"
elif .delta_method == FinFXDeltaMethod.SPOT_DELTA_PREM_ADJ:
    .delta_method_string = "pct_spot_delta_prem_adj"
elif .delta_method == FinFXDeltaMethod.FORWARD_DELTA_PREM_ADJ:
    .delta_method_string = "pct_fwd_delta_prem_adj"
else:
    raise FinError("Unknown_Delta_Type")
.vol_func_type = vol_func_type
.tenor_index = 0
.k_25d_c = None
.k_25d_p = None
.k_25d_c_ms = None
.k_25d_p_ms = None
.v_25d_ms = None
.t_exp = None
.k_atm = None
.rd = None
.rf = None
.fwd = None
.parameters = None
.expiry_dts = []
for i in range(0, .num_vol_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
spot_fx_rate	float	-
currency_pair	str	-
notional_currency	str	-
domestic_curve	DiscountCurve	-
foreign_curve	DiscountCurve	-
tenors	List[Tenor]	-
atm_vols	np.ndarray	-
ms_25_delta_vols	np.ndarray	-
rr_25_delta_vols	np.ndarray	-
atm_method	FinFXATMMMethod	-
delta_method	FinFXDeltaMethod	-
vol_func_type	VolFuncTypes	-
None		-
"""Create the FinFXVolSurface object by passing in market vol data for ATM and 25 Delta Market Strangles and Risk Reversals."""	-	-
check_argument_types(.FXVolSurface, locals())	-	-
.value_dt	-	-
.spot_fx_rate	-	-
.currency_pair	-	-
if len(currency_pair) !	-	-
raise FinError("Currency pair must be 6 characters.")	-	-
.for_name	-	-
.dom_name	-	-
.notional_currency	-	-
.domestic_curve	-	-
.foreign_curve	-	-
.num_vol_curves	-	-
if len(atm_vols) !	-	-
raise FinError("Number ATM vols must equal number of tenors")	-	-
if len(atm_vols) !	-	-
raise FinError("Number ATM vols must equal number of tenors")	-	-
if len(ms_25_delta_vols) !	-	-
raise FinError("Number MS25d vols must equal number of tenors")	-	-
if len(rr_25_delta_vols) !	-	-
raise FinError("Number RR25d vols must equal number of tenors")	-	-
.tenors	-	-
.atm_vols	-	-
.ms_25_delta_vols	-	np.
.rr_25_delta_vols	-	np.
.atm_method	-	
.delta_method	-	
if .delta_method	-	= Fin
.delta_method_string	-	
elif .delta_method	-	= FinFX
.delta_method_string	-	
elif .delta_method	-	= FinFXDe
.delta_method_string	-	
elif .delta_method	-	= FinFXDelta
.delta_method_string	-	
else		

volatility*PLEASE ADD A FUNCTION DESCRIPTION*

```

volatility(k: float, expiry_dt: Date) -> float:
    """Interpolate the Black-Scholes volatility from the volatility
    surface given the option strike and expiry date. Linear interpolation
    is done in variance_x_time."""
    vol_type_value = .vol_func_type.value
    index0 = 0
    index1 = 0
    t = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
    num_curves = .num_vol_curves
    if num_curves == 1:
        # The volatility term structure is flat if there is only one expiry
        fwd = .fwd[0]
        t_exp = .t_exp[0]
        vol = vol_function(vol_type_value, .parameters[0], fwd, k, t_exp)
        return vol
    # If the time is below first time then assume a flat vol
    if t <= .t_exp[0]:
        fwd = .fwd[0]
        t_exp = .t_exp[0]
        vol = vol_function(vol_type_value, .parameters[0], fwd, k, t_exp)
        return vol
    # If the time is beyond the last time then extrapolate with a flat vol
    if t > .t_exp[-1]:
        fwd = .fwd[-1]
        t_exp = .t_exp[-1]
        vol = vol_function(vol_type_value, .parameters[-1], fwd, k, t_exp)
        return vol
    for i in range(1, num_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description
k	float	-
expiry_dt	Date)-float:	-
"""Interpolate the Black-Scholes volatility from the volatility surface given the option strike and expiry date. Linear interpolation is done in variance x time."""		
vol_type_value	-	-
index0	-	-
index1	-	-
t	-	(expiry_dt - .va
num_curves	-	-
if num_curves	-	-
fwd	-	-
t_exp	-	-
vol	-	vol_function(vol_type
return vol	-	-
if t j	-	-
fwd	-	-
t_exp	-	-
vol	-	vol_function(vol_type
return vol	-	-
if t i .t_exp[-1]	-	-
fwd	-	-
t_exp	-	-
vol	-	vol_function(vol_type
return vol	-	-

build_vol_surface

PLEASE ADD A FUNCTION DESCRIPTION

```

build_vol_surface() -> None:
s = .spot_fx_rate
num_vol_curves = .num_vol_curves
if .vol_func_type == VolFuncTypes.CLARK:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.SABR:
    num_parameters = 4
elif .vol_func_type == VolFuncTypes.SABR_BETA_ONE:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.SABR_BETA_HALF:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.BBG:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.CLARK5:
    num_parameters = 5
else:
    print(.vol_func_type)
    raise FinError("Unknown_Model_Type")

```

```

.parameters = np.zeros([num_vol_curves, num_parameters])
.fwd = np.zeros(num_vol_curves)
.rd = np.zeros(num_vol_curves)
.rf = np.zeros(num_vol_curves)
.k_atm = np.zeros(num_vol_curves)
.delta_atm = np.zeros(num_vol_curves)
.k_25d_c = np.zeros(num_vol_curves)
.k_25d_p = np.zeros(num_vol_curves)
.k_25d_c_ms = np.zeros(num_vol_curves)
.k_25d_p_ms = np.zeros(num_vol_curves)
.v_25d_ms = np.zeros(num_vol_curves)
.t_exp = np.zeros(num_vol_curves)
#####
# TODO: ADD SPOT DAYS
#####
spot_dt = .value_dt
for i in range(0, num_vol_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) - None	-	-	-
s	-	-	.spot_fx_rate
num_vol_curves	-	-	.num_vol_curves
if .vol_func_type	-	-	= VolFuncTypes.CLARK:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.SABR:
num_parameters	-	-	4
elif .vol_func_type	-	-	= VolFuncTypes.SABR_BETA_ONE:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.SABR_BETA_HALF:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.BBG:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.CLARK5:
num_parameters	-	-	5
else	-	-	-
print(.vol_func_type)	-	-	-
raise FinError("Unknown Model Type")	-	-	-
.parameters	-	-	np.zeros([num_vol_curves, num_parameters])
.fwd	-	-	np.zeros(num_vol_curves)
.rd	-	-	np.zeros(num_vol_curves)
.rf	-	-	np.zeros(num_vol_curves)
.k_atm	-	-	np.zeros(num_vol_curves)
.delta_atm	-	-	np.zeros(num_vol_curves)
.k_25d_c	-	-	np.zeros(num_vol_curves)
.k_25d_p	-	-	np.zeros(num_vol_curves)
.k_25d_c_ms	-	-	np.zeros(num_vol_curves)
.k_25d_p_ms	-	-	np.zeros(num_vol_curves)
.v_25d_ms	-	-	np.zeros(num_vol_curves)
.t_exp	-	-	np.zeros(num_vol_curves)
spot_dt	-	-	.value_dt

solver_for_smile_strike

Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike.

```

solver_for_smile_strike(opt_type_value: int,
delta_target: float,
tenor_index: int,
initial_value: float,
float:
    """Solve_for_the_strike_that_sets_the_delta_of_the_option_equal_to_the
target_value_of_delta_allowing_the_volatility_to_be_a_function_of_the
strike."""
s0 = .spot_fx_rate

```

```

t_del = .t_exp[tenor_index]
r_d = .rd[tenor_index]
r_f = .rf[tenor_index]
inverse_delta_target = norminvcdf(np.abs(delta_target))
argtuple = (s0,
            t_del,
            r_d,
            r_f,
            opt_type_value,
            inverse_delta_target,
            tenor_index,
            )
vol_type_value = .vol_func_type.value
argtuple = (vol_type_value,
            s0,
            t_del,
            r_d,
            r_f,
            opt_type_value,
            .delta_method.value,
            inverse_delta_target,
            .parameters[tenor_index],
            )
k = newton_secant(delta_fit, x0=initial_value, args=argtuple, tol=1e-5, maxiter=50
)
return k
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
opt_type_value	int	-	-
delta_target	float	-	-
tenor_index	int	-	-
initial_value	float	-	-
float		-	-
"""Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike."""	-	-	-
s0	-	-	.spot_fx_rate
t_del	-	-	.t_exp[tenor_index]
r_d	-	-	.rd[tenor_index]
r_f	-	-	.rf[tenor_index]
inverse_delta_target	-	-	norminvcdf(np.abs(delta_target
t_del	-	-	-
r_d	-	-	-
r_f	-	-	-
opt_type_value	-	-	-
inverse_delta_target	-	-	-
tenor_index	-	-	-
)	-	-	-
vol_type_value	-	-	.vol_func_type.value
s0	-	-	-
t_del	-	-	-
r_d	-	-	-
r_f	-	-	-
opt_type_value	-	-	-
.delta_method.value	-	-	-
inverse_delta_target	-	-	-
.parameters[tenor_index]	-	-	-
)	-	-	-
)	-	-	-
return k	-	-	-

check_calibration

PLEASE ADD A FUNCTION DESCRIPTION

```

check_calibration(verbose: bool, tol: float = 1e-5) -> None:
if verbose:
    print("=====")
    print("VALUE_DATE:", .value_dt)
    print("SPOT_FX_RATE:", .spot_fx_rate)
    print("ATM_METHOD:", .atm_method)
    print("DELTA_METHOD:", .delta_method)
    print("=====")
k_dummy = 999

```

```
for i in range(0, .num_vol_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
verbose	bool	-	-
tol	float	-	1e-5) -i None:
if verbose	-	-	-
print(""	-	-	=====
print("VALUE DATE	",".value_dt)	-	-
print("SPOT FX RATE	",".spot_fx_rate)	-	-
print("ATM METHOD	",".atm_method)	-	-
print("DELTA METHOD	",".delta_method)	-	-
print(""	-	-	=====
k_dummy	-	-	999

implied_dbns

PLEASE ADD A FUNCTION DESCRIPTION

```
implied_dbns(low_fx: float, high_fx: float, num_intervals: int
List[FinDistribution]:
"""Calculate the pdf for each tenor horizon. Returns a list of
FinDistribution objects, one for each tenor horizon."""
dbns = []
for i_tenor in range(0, len(.tenors)):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
low_fx	float	-	-
high_fx	float	-	-
num_intervals	int	-	-
List[FinDistribution]	-	-	-
"""Calculate the pdf for each tenor horizon. Returns a list of	-	-	-
FinDistribution objects	-	-	-
one for each tenor horizon."""	-	-	-
dbns	-	-	[]

plot_vol_curves

PLEASE ADD A FUNCTION DESCRIPTION

```
plot_vol_curves() -> None:
plt.figure()
vol_type_val = .vol_func_type.value
for tenor_index in range(0, .num_vol_curves):
```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i None		-	-
plt.figure()	-	-	-
vol_type_val	-	-	.vol_func_type.value

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("VALUE_DATE", .value_dt)
s += label_to_string("FX_RATE", .spot_fx_rate)
s += label_to_string("CCY_PAIR", .currency_pair)
s += label_to_string("NOTIONAL_CCY", .notional_currency)
s += label_to_string("NUM_TENORS", .num_vol_curves)
s += label_to_string("ATM_METHOD", .atm_method)
s += label_to_string("DELTA_METHOD", .delta_method)
s += label_to_string("VOL_FUNCTION", .vol_func_type)
for i in range(0, .num_vol_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i str		-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("VALUE DATE", .value_dt)
s +	-	-	label_to_string("FX RATE", .spot_fx_rate)
s +	-	-	label_to_string("CCY PAIR", .currency_pair)
s +	-	-	label_to_string("NOTIONAL CCY", .notional_currency)
s +	-	-	label_to_string("NUM TENORS", .num_vol_curves)
s +	-	-	label_to_string("ATM METHOD", .atm_method)
s +	-	-	label_to_string("DELTA METHOD", .delta_method)
s +	-	-	label_to_string("VOL FUNCTION", .vol_func_type)

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print() -> None:
"""Print_a_list_of_the_unadjusted_coupon_payment_dates_used_in
analytic_calculations_for_the_bond."""
print()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) - None	-	-	-
"""Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond."""	-	-	-
print()	-	-	-

g

PLEASE ADD A FUNCTION DESCRIPTION

```
g(k: float, *args: Any) -> float:
"""This is the objective function used in the determination of the FX
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
*args	Any)-float:	-	-
"""This is the objective function used in the determination of the FX	-	-	-

obj_fast

Return a function that is minimised when the ATM, MS and RR vols have been best fitted using the parametric volatility curve represented by cvec

```
obj_fast(params: np.ndarray, *args: Any) -> float:
"""Return a function that is minimised when the ATM, MS and RR vols have
been best fitted using the parametric volatility curve represented by cvec
"""
s = args[0]
t = args[1]
r_d = args[2]
r_f = args[3]
k_atm = args[4]
atm_vol = args[5]
k_25d_c_ms = args[6]
k_25d_p_ms = args[7]
v_25d_ms_target = args[8]
delta_method_value = args[9]
target_rr_vol = args[10]
vol_type_value = args[11]
f = s * np.exp((r_d - r_f) * t)
# We first need to solve for the strikes at the 25 delta points using the
# new volatility curve
# Match the at-the-money option volatility
atm_curve_vol = vol_function(vol_type_value, params, f, k_atm, t)
term1 = (atm_vol - atm_curve_vol) ** 2
#####
# Match the market strangle value but this has to be at the MS strikes
#####
sigma_k_25d_c_ms = vol_function(vol_type_value, params, f, k_25d_c_ms, t)
```

```

v_25d_c_ms = bs_value(
    s,
    t,
    k_25d_c_ms,
    r_d,
    r_f,
    sigma_k_25d_c_ms,
    OptionTypes.EUROPEAN_CALL.value,
)
sigma_k_25d_p_ms = vol_function(vol_type_value, params, f, k_25d_p_ms, t)
v_25d_p_ms = bs_value(
    s,
    t,
    k_25d_p_ms,
    r_d,
    r_f,
    sigma_k_25d_p_ms,
    OptionTypes.EUROPEAN_PUT.value,
)
v_25d_ms = v_25d_c_ms + v_25d_p_ms
term2 = (v_25d_ms - v_25d_ms_target) ** 2
#####
# Match the risk reversal volatility
#####
k_25d_c = solver_for_smile_strike_fast(
    s,
    t,
    r_d,
    r_f,
    OptionTypes.EUROPEAN_CALL.value,
    vol_type_value,
    +0.2500,
    delta_method_value,
    k_25d_c_ms,
    params,
)
sigma_k_25d_c = vol_function(vol_type_value, params, f, k_25d_c, t)
k_25d_p = solver_for_smile_strike_fast(
    s,
    t,
    r_d,
    r_f,
    OptionTypes.EUROPEAN_PUT.value,
    vol_type_value,
    -0.2500,
    delta_method_value,
    k_25d_p_ms,
    params,
)
sigma_k_25d_p = vol_function(vol_type_value, params, f, k_25d_p, t)
sigma_25d_rr = sigma_k_25d_c - sigma_k_25d_p
term3 = (sigma_25d_rr - target_rr_vol) ** 2
# sum up the errors
err = term1 + term2 + term3
return err
#####
is function cannot be jitted until the scipy minimisation has been replaced

```

th a jittable function

The function arguments are described in the following table.

Argument Name	Type	Description
params	np.ndarray	-
*args	Any)-float:	-
Return a function that is minimised when the ATM	-	-
MS and RR vols have	-	-
been best fitted using the parametric volatility curve represented by cvec	-	-
"""	-	-
s	-	-
t	-	-
r_d	-	-
r_f	-	-
k_atm	-	-
atm_vol	-	-
k_25d_c_ms	-	-
k_25d_p_ms	-	-
v_25d_ms_target	-	-
delta_method_value	-	-
target_rr_vol	-	-
vol_type_value	-	-
f	-	-
atm_curve_vol	-	vol_function(
term1	-	(atm
sigma_k_25d_c_ms	-	vol_function(vol
s	-	-
t	-	-
k_25d_c_ms	-	-
r_d	-	-
r_f	-	-
sigma_k_25d_c_ms	-	-
OptionTypes.EUROPEAN_CALL.value	-	-
)	-	-
sigma_k_25d_p_ms	-	vol_function(vol
s	-	-
t	-	-
k_25d_p_ms	-	-
r_d	-	-
r_f	-	-
sigma_k_25d_p_ms	-	-
OptionTypes.EUROPEAN_PUT.value	-	-
)	-	-
v_25d_ms	-	v_
term2	-	(v_25d
s	-	-
t	-	-
r_d	-	-
r_f	-	-
OptionTypes.EUROPEAN_CALL.value	-	-
vol_type_value	-	-
+0.2500	-	-
delta_method_value	-	-
k_25d_c_ms	-	-
params	-	-

solve_to_horizon_fast*PLEASE ADD A FUNCTION DESCRIPTION*

```

solve_to_horizon_fast(
s: float,
t: float,
rd: float,
rf: float,
k_atm: float,
atm_vol: float,
ms_25d_vol: float,
rr_25d_vol: float,
delta_method_value: int,
vol_type_value: int,
xopt: Sequence[float],
    Tuple[np.ndarray, float, float, float, float]):
c0 = xopt
# Determine the price of a market strangle from market strangle
# Need to price a call and put that agree with market strangle
vol_25d_ms = atm_vol + ms_25d_vol
k_25d_c_ms = solve_for_strike(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_CALL.value,
    +0.2500,
    delta_method_value,
    vol_25d_ms,
)
k_25d_p_ms = solve_for_strike(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_PUT.value,
    -0.2500,
    delta_method_value,
    vol_25d_ms,
)
# USE MARKET STRANGLE VOL TO DETERMINE PRICE OF A MARKET STRANGLE
v_25d_c_ms = bs_value(
    s, t, k_25d_c_ms, rd, rf, vol_25d_ms, OptionTypes.EUROPEAN_CALL.value
)
v_25d_p_ms = bs_value(
    s, t, k_25d_p_ms, rd, rf, vol_25d_ms, OptionTypes.EUROPEAN_PUT.value
)
# Market price of strangle in the domestic currency
v_25d_ms = v_25d_c_ms + v_25d_p_ms
# Determine parameters of vol surface using minimisation
tol = 1e-8
fargs = (
    s,
    t,
    rd,

```

```

    rf,
    k_atm,
    atm_vol,
    k_25d_c_ms,
    k_25d_p_ms,
    v_25d_ms,
    delta_method_value,
    rr_25d_vol,
    vol_type_value,
)
opt = minimize(obj_fast, c0, fargs, method="CG", tol=tol)
xopt = opt.x
params = np.array(xopt)
k_25d_c = solver_for_smile_strike_fast(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_CALL.value,
    vol_type_value,
    +0.2500,
    delta_method_value,
    k_25d_c_ms,
    params,
)
k_25d_p = solver_for_smile_strike_fast(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_PUT.value,
    vol_type_value,
    -0.2500,
    delta_method_value,
    k_25d_p_ms,
    params,
)
ret = (params, k_25d_c_ms, k_25d_p_ms, k_25d_c, k_25d_p)
return ret
#####
t(
float64(int64, float64[:], float64, float64, float64),
cache=True,
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
rd	float	-	-
rf	float	-	-
k_atm	float	-	-
atm_vol	float	-	-
ms_25d_vol	float	-	-
rr_25d_vol	float	-	-
delta_method_value	int	-	-
vol_type_value	int	-	-
xopt	Sequence[float]	-	-
Tuple[np.ndarray, float, float, float, float]		-	-
c0	-	-	xopt
vol_25d_ms	-	-	atm_vol + ms_25d_vol
s	-	-	-
t	-	-	-
rd	-	-	-
rf	-	-	-
OptionTypes.EUROPEAN_CALL.value	-	-	-
+0.2500	-	-	-
delta_method_value	-	-	-
vol_25d_ms	-	-	-
)	-	-	-
s	-	-	-
t	-	-	-
rd	-	-	-
rf	-	-	-
OptionTypes.EUROPEAN_PUT.value	-	-	-
-0.2500	-	-	-
delta_method_value	-	-	-
vol_25d_ms	-	-	-
)	-	-	-
s	-	-	-
t	-	-	-
k_25d_c_ms	-	-	-
rd	-	-	-
rf	-	-	-
vol_25d_ms	-	-	-
OptionTypes.EUROPEAN_CALL.value	-	-	-
)	-	-	-
s	-	-	-
t	-	-	-
k_25d_p_ms	-	-	-
rd	-	-	-
rf	-	-	-
vol_25d_ms	-	-	-
OptionTypes.EUROPEAN_PUT.value	-	-	-
)	-	-	-
v_25d_ms	-	-	v_25d_c_ms + v_25d_p_ms
tol	-	-	1e-8

vol_function

Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book.

```

vol_function(
vol_function_type_value: int, params: np.ndarray, f: float, k: float, t: float
float:
"""Return the volatility for a strike using a given polynomial
interpolation following Section 3.9 of Iain Clark book."""
if vol_function_type_value == VolFuncTypes.CLARK.value:
    vol = vol_function_clark(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SABR.value:
    vol = vol_function_sabr(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SABR_BETA_ONE.value:
    vol = vol_function_sabr_beta_one(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SABR_BETA_HALF.value:
    vol = vol_function_sabr_beta_half(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.BBG.value:
    vol = vol_function_bloomberg(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.CLARK5.value:
    vol = vol_function_clark(params, f, k, t)
    return vol
else:
    raise FinError("Unknown_Model_Type")
#####
t(cache=True, fastmath=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
vol_function_type_value	int	-	-
params	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float	-	-	-
"""Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book."""	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.CLARK
vol	-	-	vol_function_clark(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SABR
vol	-	-	vol_function_sabr(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SABR_BETA
vol	-	-	vol_function_sabr_beta_one(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SABR_BETA
vol	-	-	vol_function_sabr_beta_half(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.BBG
vol	-	-	vol_function_bloomberg(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.CLARK
vol	-	-	vol_function_clark(params)
return vol	-	-	-
else	-	-	-
raise FinError("Unknown Model Type")	-	-	-
t(cache	-	-	True, fastmath=True)

delta_fit

PLEASE ADD A FUNCTION DESCRIPTION

```
delta_fit(k: float, *args: Any) -> float:
    """This is the objective function used in the determination of the FX
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
*args	Any)-float:	-	-
"""This is the objective function used in the determination of the FX	-	-	-

solver_for_smile_strike_fast

Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike.

```

solver_for_smile_strike_fast(
s: float,
t: float,
rd: float,
rf: float,
opt_type_value: int,
volatility_type_value: int,
delta_target: float,
delta_method_value: int,
initial_guess: float,
parameters: np.ndarray,
    float:
    """Solve_for_the_strike_that_sets_the_delta_of_the_option_equal_to_the
    target_value_of_delta_allowing_the_volatility_to_be_a_function_of_the
    strike."""
    inverse_delta_target = norminvcdf(np.abs(delta_target))
    argtuple = (
        volatility_type_value,
        s,
        t,
        rd,
        rf,
        opt_type_value,
        delta_method_value,
        inverse_delta_target,
        parameters,
    )
    k = newton_secant(delta_fit, x0=initial_guess, args=argtuple, tol=1e-8, maxiter=50)
    return k
#####
able to cache function
t(
float64(float64, float64, float64, float64, int64, float64, int64, float64),
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description
s	float	-
t	float	-
rd	float	-
rf	float	-
opt_type_value	int	-
volatility_type_value	int	-
delta_target	float	-
delta_method_value	int	-
initial_guess	float	-
parameters	np.ndarray	-
float		-
"""Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike."""	-	-
inverse_delta_target	-	-
volatility_type_value	-	-
s	-	-
t	-	-
rd	-	-
rf	-	-
opt_type_value	-	-
delta_method_value	-	-
inverse_delta_target	-	-
parameters	-	-
)	-	-
k	-	- newton_secant(delta.f
return k	-	-
able to cache function	-	-
float64(float64, float64, float64, float64, int64, float64, int64, float64)	-	-
fastmath	-	-

solve_for_strike

This function determines the implied strike of an FX option given a delta and the other option details. It uses a one-dimensional Newton root search algorithm to determine the strike that matches an input volatility.

```

solve_for_strike(
spot_fx_rate: float,
t_del: float,
rd: float,
rf: float,
opt_type_value: int,
delta_target: float,
delta_method_value: int,
volatility: float,
float:

```

```

"""This function determines the implied strike of an FX option
given a delta and the other option details. It uses a one-dimensional
Newton root search algorithm to determine the strike that matches an
input volatility."""
# =====
# IMPORTANT NOTE:
# =====
# For some delta quotation conventions I can solve for K explicitly.
# Note that as I am using the function norm_inv_delta to calculate the
# inverse value of delta, this may not, on a round trip using normcdf(x), give
# back the value x as it is calculated to a different number of decimal
# places. It should however agree to 6-7 decimal places. Which is OK.
# =====
if delta_method_value == FinFXDeltaMethod.SPOT_DELTA.value:
    dom_df = np.exp(-rd * t_del)
    for_df = np.exp(-rf * t_del)
    if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
        phi = +1.0
    else:
        phi = -1.0
    fwd_fx_rate = spot_fx_rate * for_df / dom_df
    vsqrtt = volatility * np.sqrt(t_del)
    arg = delta_target * phi / for_df # CHECK THIS !!!
    norm_inv_delta = norminvcdf(arg)
    k = fwd_fx_rate * np.exp(-vsqrtt * (phi * norm_inv_delta - vsqrtt / 2.0))
    return k
elif delta_method_value == FinFXDeltaMethod.FORWARD_DELTA.value:
    dom_df = np.exp(-rd * t_del)
    for_df = np.exp(-rf * t_del)
    if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
        phi = +1.0
    else:
        phi = -1.0
    fwd_fx_rate = spot_fx_rate * for_df / dom_df
    vsqrtt = volatility * np.sqrt(t_del)
    arg = delta_target * phi # CHECK THIS!!!!!!!
    norm_inv_delta = norminvcdf(arg)
    k = fwd_fx_rate * np.exp(-vsqrtt * (phi * norm_inv_delta - vsqrtt / 2.0))
    return k
elif delta_method_value == FinFXDeltaMethod.SPOT_DELTA_PREM_ADJ.value:
    argtuple = (
        spot_fx_rate,
        t_del,
        rd,
        rf,
        volatility,
        delta_method_value,
        opt_type_value,
        delta_target,
    )
    k = newton_secant(g, x0=spot_fx_rate, args=argtuple, tol=1e-7, maxiter=50)
    return k
elif delta_method_value == FinFXDeltaMethod.FORWARD_DELTA_PREM_ADJ.value:
    argtuple = (
        spot_fx_rate,
        t_del,
        rd,

```

```
        rf,
        volatility,
        delta_method_value,
        opt_type_value,
        delta_target,
    )
    k = newton_secant(g, x0=spot_fx_rate, args=argtuple, tol=1e-7, maxiter=50)
    return k
raise FinError("Unknown_FinFXDeltaMethod")
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
spot_fx_rate	float	-
t_del	float	-
rd	float	-
rf	float	-
opt_type_value	int	-
delta_target	float	-
delta_method_value	int	-
volatility	float	-
float		-
"""This function determines the implied strike of an FX option	-	-
given a delta and the other option details. It uses a one-dimensional	-	-
Newton root search algorithm to determine the strike that matches an	-	-
input volatility."""	-	-
if delta_method_value	-	- = Fi
dom_df	-	-
for_df	-	-
if opt_type_value	-	- = O
phi	-	-
else	-	-
phi	-	-
fwd_fx_rate	-	-
vsqrtt	-	-
arg	-	CHECK THIS !!!
norm_inv_delta	-	-
k	-	- fwd_fx_rate * np
return k	-	-
elif delta_method_value	-	- = FinF
dom_df	-	-
for_df	-	-
if opt_type_value	-	- = O
phi	-	-
else	-	-
phi	-	-
fwd_fx_rate	-	-
vsqrtt	-	-
arg	-	CHECK THIS!!!!!!!
norm_inv_delta	-	-
k	-	- fwd_fx_rate * np
return k	-	-
elif delta_method_value	-	- = FinFXD
spot_fx_rate	-	-
t_del	-	-
rd	-	-
rf	-	-
volatility	-	-
delta_method_value	-	-
opt_type_value	-	-
delta_target	-	-
)	-	-
k	-	- newton_secant(g, x
return k	-	-

3.4 fx_vol_surface_plus

Class: FXVolSurfacePlus

Class to perform a calibration of a chosen parametrised surface to the prices of FX options at different strikes and expiry tenors. The calibration inputs are the ATM and 25 and 10 Delta volatilities in terms of the market strangle and risk reversals. There is a choice of volatility function from cubic in delta to full SABR. Check out VolFuncTypes. Parameter alpha [0,1] is used to interpolate between fitting only 25d when alpha=0 to fitting only 10d when alpha=1.0. Alpha=0.5 assigns equal weights A vol function with more parameters will give a better fit. Of course. But it might also overfit. Visualising the volatility curve is useful. Also, there is no guarantee that the implied pdf will be positive.

FXVolSurfacePlus

PLEASE ADD A FUNCTION DESCRIPTION

```
FXVolSurfacePlus(value_dt: Date,
spot_fx_rate: float,
currency_pair: str,
notional_currency: str,
domestic_curve: DiscountCurve,
foreign_curve: DiscountCurve,
tenors: List[Tenor],
atm_vols: np.ndarray,
ms_25_delta_vols: Union[np.ndarray, None],
rr_25_delta_vols: Union[np.ndarray, None],
ms_10_delta_vols: Union[np.ndarray, None],
rr_10_delta_vols: Union[np.ndarray, None],
alpha: float,
atm_method: FinFXATMMMethod = FinFXATMMMethod.FWD_DELTA_NEUTRAL,
delta_method: FinFXDeltaMethod = FinFXDeltaMethod.SPOT_DELTA,
vol_func_type: VolFuncTypes = VolFuncTypes.CLARK,
fin_solver_type: SolverTypes = SolverTypes.NELDER_MEAD,
tol: float = 1e-8,
None:
    """Create the FinFXVolSurfacePlus object by passing in market vol data
    for ATM, 25 Delta and 10 Delta strikes. The alpha weight shifts the
    fitting between 25d and 10d. Alpha=0.0 is 100% 25d while alpha=1.0
    is 100% 10d. An alpha of 0.50 is equally weighted."""
    # I want to allow Nones for some of the market inputs
    if ms_10_delta_vols is None:
        ms_10_delta_vols = np.array([])
    if rr_10_delta_vols is None:
        rr_10_delta_vols = np.array([])
    if ms_25_delta_vols is None:
        ms_25_delta_vols = np.array([])
    if rr_25_delta_vols is None:
        rr_25_delta_vols = np.array([])
    check_argument_types(.FXVolSurfacePlus, locals())
    .value_dt = value_dt
    .spot_fx_rate = spot_fx_rate
    .currency_pair = currency_pair
    if len(currency_pair) != 6:
        raise FinError("Currency_pair must be 6 characters.")
```



```

.for_name = .currency_pair[0:3]
.dom_name = .currency_pair[3:6]
.notional_currency = notional_currency
.domestic_curve = domestic_curve
.foreign_curve = foreign_curve
.num_vol_curves = len(tenors)
.tenors = tenors
if len(atm_vols) != .num_vol_curves:
    raise FinError("Number_ATM_vols_must_equal_number_of_tenors")
.atm_vols = np.array(atm_vols) / 100.0
.use_ms_25d_vol = True
.use_rr_25d_vol = True
.use_ms_10d_vol = True
.use_rr_10d_vol = True
# Some of these can be missing which is signified by length zero
n = len(ms_25_delta_vols)
if n != .num_vol_curves and n != 0:
    raise FinError("Number_MS25d_vols_must_equal_number_of_tenors")
if n == 0:
    .use_ms_25d_vol = False
n = len(rr_25_delta_vols)
if n != .num_vol_curves and n != 0:
    raise FinError("Number_RR25d_vols_must_equal_number_of_tenors")
if n == 0:
    .use_rr_25d_vol = False
n = len(ms_10_delta_vols)
if n != .num_vol_curves and n != 0:
    raise FinError("Number_MS10d_vols_must_equal_number_of_tenors")
if n == 0:
    .use_ms_10d_vol = False
n = len(rr_10_delta_vols)
if n != .num_vol_curves and n != 0:
    raise FinError("Number_RR10d_vols_must_equal_number_of_tenors")
if n == 0:
    .use_rr_10d_vol = False
if .use_ms_10d_vol != .use_rr_10d_vol:
    raise FinError("You_must_provide_both_10d_RR+_10d_MS_or_neither")
if .use_ms_25d_vol != .use_rr_25d_vol:
    raise FinError("You_must_provide_both_25d_RR+_25d_MS_or_neither")
if .use_ms_10d_vol is False and .use_ms_25d_vol is False:
    raise FinError("No_MS_and_RR_You_must_provide_10d_or_25d_MS+_RR.")
.ms_25_delta_vols = np.array(ms_25_delta_vols) / 100.0
.rr_25_delta_vols = np.array(rr_25_delta_vols) / 100.0
.ms_10_delta_vols = np.array(ms_10_delta_vols) / 100.0
.rr_10_delta_vols = np.array(rr_10_delta_vols) / 100.0
if alpha < 0.0 or alpha > 1.0:
    raise FinError("Alpha_must_be_between_0.0_and_1.0")
.alpha = alpha
.atm_method = atm_method
.delta_method = delta_method
if .delta_method == FinFXDeltaMethod.SPOT_DELTA:
    .delta_method_string = "pips_spot_delta"
elif .delta_method == FinFXDeltaMethod.FORWARD_DELTA:
    .delta_method_string = "pips_fwd_delta"
elif .delta_method == FinFXDeltaMethod.SPOT_DELTA_PREM_ADJ:
    .delta_method_string = "pct_spot_delta_prem_adj"
elif .delta_method == FinFXDeltaMethod.FORWARD_DELTA_PREM_ADJ:

```

```
        .delta_method_string = "pct_fwd_delta_prem_adj"  
else:  
    raise FinError("Unknown_Delta_Type")  
.vol_func_type = vol_func_type  
.tenor_index = 0  
.expiry_dts = []  
for i in range(0, .num_vol_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
spot_fx_rate	float	-
currency_pair	str	-
notional_currency	str	-
domestic_curve	DiscountCurve	-
foreign_curve	DiscountCurve	-
tenors	List[Tenor]	-
atm_vols	np.ndarray	-
ms_25_delta_vols	np.ndarray or None	-
rr_25_delta_vols	np.ndarray or None	-
ms_10_delta_vols	np.ndarray or None	-
rr_10_delta_vols	np.ndarray or None	-
alpha	float	-
atm_method	FinFXATMMethod	-
delta_method	FinFXDeltaMethod	-
vol_func_type	VolFuncTypes	-
fin_solver_type	SolverTypes	-
tol	float	-
None		-
"""Create the FinFXVolSurfacePlus object by passing in market vol data	-	-
for ATM	-	-
25 Delta and 10 Delta strikes. The alpha weight shifts the	-	-
fitting between 25d and 10d. Alpha	-	-
is 100% 10d. An alpha of 0.50 is equally weighted."""	-	-
if ms_10_delta_vols is None		-
ms_10_delta_vols	-	-
if rr_10_delta_vols is None		-
rr_10_delta_vols	-	-
if ms_25_delta_vols is None		-
ms_25_delta_vols	-	-
if rr_25_delta_vols is None		-
rr_25_delta_vols	-	-
check_argument_types(.FXVolSurfacePlus, locals())	-	-
.value_dt	-	-
.spot_fx_rate	-	-
.currency_pair	-	-
if len(currency_pair) !=	-	-
6:	-	-
raise FinError("Currency pair must be 6 characters.")	-	-
.for_name	-	-
.dom_name	-	-
.notional_currency	-	-
.domestic_curve	-	-
.foreign_curve	-	-
.num_vol_curves	-	-
.tenors	-	-
if len(atm_vols) !=	-	-
len(tenors):	-	-
raise FinError("Number ATM vols must equal number of tenors")	-	-
.atm_vols	-	-
.use_ms_25d_vol	-	-
.use_rr_25d_vol	-	-
.use_ms_10d_vol	-	-
.use_rr_10d_vol	-	-

vol_from_strike_dt*PLEASE ADD A FUNCTION DESCRIPTION*

```

vol_from_strike_dt(kk: float, expiry_dt: Date) -> float:
    """Interpolates the Black-Scholes volatility from the volatility
    surface given call option strike and expiry date. Linear interpolation
    is done in variance space. The smile strikes at bracketed dates are
    determined by determining the strike that reproduces the provided delta
    value. This uses the calibration delta convention, but it can be
    overridden by a provided delta convention. The resulting volatilities
    are then determined for each bracketing expiry time and linear
    interpolation is done in variance space and then converted back to a
    lognormal volatility."""
    t_exp = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
    vol_type_value = .vol_func_type.value
    index0 = 0 # lower index in bracket
    index1 = 0 # upper index in bracket
    num_curves = .num_vol_curves
    if num_curves == 1:
        index0 = 0
        index1 = 0
    # If the time is below first time then assume a flat vol
    elif t_exp <= .t_exp[0]:
        index0 = 0
        index1 = 0
    # If the time is beyond the last time then extrapolate with a flat vol
    elif t_exp >= .t_exp[-1]:
        index0 = len(.t_exp) - 1
        index1 = len(.t_exp) - 1
    else: # Otherwise we look for bracketing times and interpolate
        for i in range(1, num_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description
kk	float	-
expiry_dt	Date)-ζfloat:	-
"""Interpolates the Black-Scholes volatility from the volatility surface given call option strike and expiry date. Linear interpolation is done in variance space. The smile strikes at bracketed dates are determined by determining the strike that reproduces the provided delta value. This uses the calibration delta convention but it can be overridden by a provided delta convention. The resulting volatilities are then determined for each bracketing expiry time and linear interpolation is done in variance space and then converted back to a lognormal volatility."""	-	-
t_exp	-	-
vol_type_value	-	-
index0	-	lower index in bracket
index1	-	upper index in bracket
num_curves	-	-
if num_curves	-	-
index0	-	-
index1	-	-
elif t_exp < t_exp_0	-	-
index0	-	-
index1	-	-
elif t_exp < t_exp_1	-	-
index0	-	-
index1	-	-
else	-	Otherwise we look for bracketing

delta_to_strike

PLEASE ADD A FUNCTION DESCRIPTION

```

delta_to_strike(call_delta: float, expiry_dt: Date, delta_method: Optional[Any]
float:
"""Interpolates the strike at a delta and expiry date. Linear
time to expiry interpolation is used in strike."""
t_exp = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
vol_type_value = .vol_func_type.value
s = .spot_fx_rate
if delta_method is None:
    delta_method_value = .delta_method.value
else:
    delta_method_value = delta_method.value
index0 = 0 # lower index in bracket
index1 = 0 # upper index in bracket
num_curves = .num_vol_curves
# If there is only one time horizon then assume flat vol to this time

```

```

if num_curves == 1:
    index0 = 0
    index1 = 0
# If the time is below first time then assume a flat vol
elif t_exp <= .t_exp[0]:
    index0 = 0
    index1 = 0
# If the time is beyond the last time then extrapolate with a flat vol
elif t_exp > .t_exp[-1]:
    index0 = len(.t_exp) - 1
    index1 = len(.t_exp) - 1
else: # Otherwise we look for bracketing times and interpolate
    for i in range(1, num_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description
call_delta	float	-
expiry_dt	Date	-
delta_method	Optional[Any]	-
float		-
"""Interpolates the strike at a delta and expiry date. Linear time to expiry interpolation is used in strike."""	-	-
t_exp	-	-
vol_type_value	-	-
s	-	-
if delta_method is None		-
delta_method_value	-	-
else		-
delta_method_value	-	-
index0	-	lower index in bracket
index1	-	upper index in bracket
num_curves	-	-
if num_curves	-	-
index0	-	-
index1	-	-
elif t_exp ;	-	-
index0	-	-
index1	-	-
elif t_exp > .t_exp[-1]		-
index0	-	-
index1	-	-
else		Otherwise we look for bracketing times and

vol from delta_date

PLEASE ADD A FUNCTION DESCRIPTION

```

vol_from_delta_date(call_delta: float,
                    expiry_dt: Date,
                    delta_method: Optional[FinFXDeltaMethod] = None,
                    Tuple[float, float]):
    """Interpolates the Black-Scholes volatility from the volatility
    surface given a call option delta and expiry date. Linear interpolation
    is done in variance space. The smile strikes at bracketed dates are
    determined by determining the strike that reproduces the provided delta
    value. This uses the calibration delta convention, but it can be
    overridden by a provided delta convention. The resulting volatilities
    are then determined for each bracketing expiry time and linear
    interpolation is done in variance space and then converted back to a
    lognormal volatility."""
    t_exp = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
    vol_type_value = .vol_func_type.value
    s = .spot_fx_rate
    if delta_method is None:
        delta_method_value = .delta_method.value
    else:
        delta_method_value = delta_method.value
    index0 = 0 # lower index in bracket
    index1 = 0 # upper index in bracket
    num_curves = .num_vol_curves
    # If there is only one time horizon then assume flat vol to this time
    if num_curves == 1:
        index0 = 0
        index1 = 0
    # If the time is below first time then assume a flat vol
    elif t_exp <= .t_exp[0]:
        index0 = 0
        index1 = 0
    # If the time is beyond the last time then extrapolate with a flat vol
    elif t_exp > .t_exp[-1]:
        index0 = len(.t_exp) - 1
        index1 = len(.t_exp) - 1
    else: # Otherwise we look for bracketing times and interpolate
        for i in range(1, num_curves):

```

The function arguments are described in the following table.

Argument Name	Type
call_delta	float
expiry_dt	Date
delta_method	Optional[FinFXDeltaMethod]
Tuple[float, float]	
"""Interpolates the Black-Scholes volatility from the volatility surface given a call option delta and expiry date. Linear interpolation is done in variance space. The smile strikes at bracketed dates are determined by determining the strike that reproduces the provided delta value. This uses the calibration delta convention but it can be overridden by a provided delta convention. The resulting volatilities are then determined for each bracketing expiry time and linear interpolation is done in variance space and then converted back to a lognormal volatility."""	-
t_exp	-
vol_type_value	-
s	-
if delta_method is None	
delta_method_value	-
else	
delta_method_value	-
index0	-
index1	-
num_curves	-
if num_curves	-
index0	-
index1	-
elif t_exp < t_exp[-1]	-
index0	-
index1	-
elif t_exp > t_exp[-1]	-
index0	-
index1	-
else	Otherwise we l

_build_vol_surface

PLEASE ADD A FUNCTION DESCRIPTION

```

_build_vol_surface(fin_solver_type: Any = SolverTypes.NELDER_MEAD, tol: float = 1e-8
None:
"""Main_function_to_construct_the_vol_surface."""
s = .spot_fx_rate
num_vol_curves = .num_vol_curves
if .vol_func_type == VolFuncTypes.CLARK:
    num_parameters = 3

```



```

elif .vol_func_type == VolFuncTypes.SABR:
    num_parameters = 4
elif .vol_func_type == VolFuncTypes.SABR_BETA_ONE:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.SABR_BETA_HALF:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.BBG:
    num_parameters = 3
elif .vol_func_type == VolFuncTypes.CLARK5:
    num_parameters = 5
else:
    print(.vol_func_type)
    raise FinError("Unknown_Model_Type")
.parameters = np.zeros([num_vol_curves, num_parameters])
num_strikes = 5
.strikes = np.zeros([num_vol_curves, num_strikes])
.gaps = np.zeros([num_vol_curves, num_strikes])
.t_exp = np.zeros(num_vol_curves)
.fwd = np.zeros(num_vol_curves)
.rd = np.zeros(num_vol_curves)
.rf = np.zeros(num_vol_curves)
.k_atm = np.zeros(num_vol_curves)
.delta_atm = np.zeros(num_vol_curves)
.k_25d_c = np.zeros(num_vol_curves)
.k_25d_p = np.zeros(num_vol_curves)
.k_25d_c_ms = np.zeros(num_vol_curves)
.k_25d_p_ms = np.zeros(num_vol_curves)
.v_25d_ms = np.zeros(num_vol_curves)
.k_10d_c = np.zeros(num_vol_curves)
.k_10d_p = np.zeros(num_vol_curves)
.k_10d_c_ms = np.zeros(num_vol_curves)
.k_10d_p_ms = np.zeros(num_vol_curves)
.v_10d_ms = np.zeros(num_vol_curves)
#####
# TODO: ADD SPOT DAYS
#####
spot_dt = .value_dt
for i in range(0, num_vol_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fin_solver_type	Any	-	SolverTypes.NELDER_MEAD
tol	float	-	1e-8
None		-	-
"""Main function to construct the vol surface."""	-	-	-
s	-	-	.spot_fx_rate
num_vol_curves	-	-	.num_vol_curves
if .vol_func_type	-	-	= VolFuncTypes.CLARK:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.SABR:
num_parameters	-	-	4
elif .vol_func_type	-	-	= VolFuncTypes.SABR_BETA_ONE:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.SABR_BETA_HALF:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.BBG:
num_parameters	-	-	3
elif .vol_func_type	-	-	= VolFuncTypes.CLARK5:
num_parameters	-	-	5
else		-	-
print(.vol_func_type)	-	-	-
raise FinError("Unknown Model Type")	-	-	-
.parameters	-	-	np.zeros([num_vol_curves, num_parameters])
num_strikes	-	-	5
.strikes	-	-	np.zeros([num_vol_curves, num_strikes])
.gaps	-	-	np.zeros([num_vol_curves, num_strikes])
.t_exp	-	-	np.zeros(num_vol_curves)
.fwd	-	-	np.zeros(num_vol_curves)
.rd	-	-	np.zeros(num_vol_curves)
.rf	-	-	np.zeros(num_vol_curves)
.k_atm	-	-	np.zeros(num_vol_curves)
.delta_atm	-	-	np.zeros(num_vol_curves)
.k_25d_c	-	-	np.zeros(num_vol_curves)
.k_25d_p	-	-	np.zeros(num_vol_curves)
.k_25d_c_ms	-	-	np.zeros(num_vol_curves)
.k_25d_p_ms	-	-	np.zeros(num_vol_curves)
.v_25d_ms	-	-	np.zeros(num_vol_curves)
.k_10d_c	-	-	np.zeros(num_vol_curves)
.k_10d_p	-	-	np.zeros(num_vol_curves)
.k_10d_c_ms	-	-	np.zeros(num_vol_curves)
.k_10d_p_ms	-	-	np.zeros(num_vol_curves)
.v_10d_ms	-	-	np.zeros(num_vol_curves)
spot_dt	-	-	.value_dt

check_calibration

PLEASE ADD A FUNCTION DESCRIPTION

```
check_calibration(verbose: bool, tol: float = 1e-6) -> None:
    """Compare_calibrated_vol_surface_with_market_and_output_a_report
    which_sets_out_the_quality_of_fit_to_the_ATM_and_10_and_25_delta_market
    strangles_and_risk_reversals."""
    if verbose:
        print("=====")
        print("VALUE_DATE:", .value_dt)
        print("SPOT_FX_RATE:", .spot_fx_rate)
        print("ALPHA_WEIGHT:", .alpha)
        print("ATM_METHOD:", .atm_method)
        print("DELTA_METHOD:", .delta_method)
        print("=====")
    k_dummy = 999
    for i in range(0, .num_vol_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description
verbose	bool	-
tol	float	-
"""Compare calibrated vol surface with market and output a report which sets out the quality of fit to the ATM and 10 and 25 delta market strangles and risk reversals."""	-	-
if verbose	-	-
print("=====	-	-
print("VALUE DATE	",".value_dt)	-
print("SPOT FX RATE	",".spot_fx_rate)	-
print("ALPHA WEIGHT	",".alpha)	-
print("ATM METHOD	",".atm_method)	-
print("DELTA METHOD	",".delta_method)	-
print("=====	-	-
k_dummy	-	-

implied_dbns

PLEASE ADD A FUNCTION DESCRIPTION

```
implied_dbns(low_fx: float, high_fx: float, num_intervals: int
    List[FinDistribution]:
    """Calculate_the_pdf_for_each_tenor_horizon._Returns_a_list_of
    FinDistribution_objects,_one_for_each_tenor_horizon."""
    dbns = []
    for i_tenor in range(0, len(.tenors)):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
low_fx	float	-	-
high_fx	float	-	-
num_intervals	int	-	-
List[FinDistribution]		-	-
"""Calculate the pdf for each tenor horizon. Returns a list of FinDistribution objects one for each tenor horizon."""	-	-	-
dbns	-	-	[]

plot_vol_curves

PLEASE ADD A FUNCTION DESCRIPTION

```
plot_vol_curves() -> None:
"""Generates a plot of each of the vol discount implied by the market
and fitted."""
plt.figure()
vol_type_val = .vol_func_type.value
for tenor_index in range(0, .num_vol_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i None		-	-
"""Generates a plot of each of the vol discount implied by the market and fitted."""	-	-	-
plt.figure()	-	-	-
vol_type_val	-	-	.vol_func_type.value

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("VALUE_DATE", .value_dt)
s += label_to_string("FX_RATE", .spot_fx_rate)
s += label_to_string("CCY_PAIR", .currency_pair)
s += label_to_string("NOTIONAL_CCY", .notional_currency)
s += label_to_string("NUM_TENORS", .num_vol_curves)
s += label_to_string("ATM_METHOD", .atm_method)
s += label_to_string("DELTA_METHOD", .delta_method)
s += label_to_string("ALPHA_WEIGHT", .alpha)
s += label_to_string("VOL_FUNCTION", .vol_func_type)
for i in range(0, .num_vol_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i str	-	-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("VALUE DATE", .value_dt)
s +	-	-	label_to_string("FX RATE", .spot_fx_rate)
s +	-	-	label_to_string("CCY PAIR", .currency_pair)
s +	-	-	label_to_string("NOTIONAL CCY", .notional_currency)
s +	-	-	label_to_string("NUM TENORS", .num_vol_curves)
s +	-	-	label_to_string("ATM METHOD", .atm_method)
s +	-	-	label_to_string("DELTA METHOD", .delta_method)
s +	-	-	label_to_string("ALPHA WEIGHT", .alpha)
s +	-	-	label_to_string("VOL FUNCTION", .vol_func_type)

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print() -> None:
"""Print a list of the unadjusted coupon payment dates used in
analytic calculations for the bond."""
print()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i None	-	-	-
"""Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond."""	-	-	-
print()	-	-	-

_g

PLEASE ADD A FUNCTION DESCRIPTION

```
_g(kk: float, *args: Any) -> float:
"""This is the objective function used in the determination of the FX
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
kk	float	-	-
*args	Any)-i float:	-	-
"""This is the objective function used in the determination of the FX	-	-	-

`_interpolate_gap`*PLEASE ADD A FUNCTION DESCRIPTION*

```

_interpolate_gap(k: float, strikes: np.ndarray, gaps: np.ndarray) -> float:
if k <= strikes[0]:
    return 0.0
if k >= strikes[-1]:
    return 0.0
index = 0
for i in range(1, len(strikes)):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
strikes	np.ndarray	-	-
gaps	np.ndarray)-float:	-	-
if k ;	-	-	strikes[0]:
return 0.0	-	-	-
if k ;	-	-	strikes[-1]:
return 0.0	-	-	-
index	-	-	0

`_obj`

Return a function that is minimised when the ATM, MS and RR vols have been best fitted using the parametric volatility curve represented by params and specified by the vol_type_value

```

_obj(params: np.ndarray, *args: Any) -> float:
"""Return_a_function_that_is_minimised_when_the_ATM,_MS_and_RR_vols_have
been_best_fitted_using_the_parametric_volatility_curve_represented_by
params_and_specified_by_the_vol_type_value
"""
s = args[0]
t = args[1]
r_d = args[2]
r_f = args[3]
k_atm = args[4]
atm_vol = args[5]
k_25d_c_ms = args[6]
k_25d_p_ms = args[7]
v_25d_ms_target = args[8]
target_25d_rr_vol = args[9]
k_10d_c_ms = args[10]
k_10d_p_ms = args[11]
v_10d_ms_target = args[12]
target_10d_rr_vol = args[13]
delta_method_value = args[14]
vol_type_value = args[15]
alpha = args[16]
strikes_null = np.zeros(1)

```

```

gaps_null = np.zeros(1)
f = s * np.exp((r_d - r_f) * t)
# We first need to solve for the strikes at the 25 delta points using the
# new volatility curve
# Match the at-the-money option volatility
atm_curve_vol = vol_function(
    vol_type_value, params, strikes_null, gaps_null, f, k_atm, t
)
term_atm = (atm_vol - atm_curve_vol) ** 2
#####
# Match the market strangle value but this has to be at the MS 25d strikes
#####
if target_25d_rr_vol > -999.0:
    sigma_k_25d_c_ms = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_25d_c_ms, t
    )
    v_25d_c_ms = bs_value(
        s,
        t,
        k_25d_c_ms,
        r_d,
        r_f,
        sigma_k_25d_c_ms,
        OptionTypes.EUROPEAN_CALL.value,
    )
    sigma_k_25d_p_ms = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_25d_p_ms, t
    )
    v_25d_p_ms = bs_value(
        s,
        t,
        k_25d_p_ms,
        r_d,
        r_f,
        sigma_k_25d_p_ms,
        OptionTypes.EUROPEAN_PUT.value,
    )
    v_25d_ms = v_25d_c_ms + v_25d_p_ms
    term_25d_l = (v_25d_ms - v_25d_ms_target) ** 2
else:
    term_25d_l = 0.0
#####
# Match the risk reversal volatility
#####
if target_25d_rr_vol > -999.0:
    k_25d_c = _solver_for_smile_strike(
        s,
        t,
        r_d,
        r_f,
        OptionTypes.EUROPEAN_CALL.value,
        vol_type_value,
        +0.2500,
        delta_method_value,
        k_25d_c_ms,
        params,
        strikes_null,

```

```

        gaps_null,
    )
    sigma_k_25d_c = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_25d_c, t
    )
    k_25d_p = _solver_for_smile_strike(
        s,
        t,
        r_d,
        r_f,
        OptionTypes.EUROPEAN_PUT.value,
        vol_type_value,
        -0.2500,
        delta_method_value,
        k_25d_p_ms,
        params,
        strikes_null,
        gaps_null,
    )
    sigma_k_25d_p = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_25d_p, t
    )
    sigma_25d_rr = sigma_k_25d_c - sigma_k_25d_p
    term_25d_2 = (sigma_25d_rr - target_25d_rr_vol) ** 2
else:
    term_25d_2 = 0.0
#####
# Match the market strangle value but this has to be at the MS 10d strikes
#####
if target_10d_rr_vol > -999.0:
    sigma_k_10d_c_ms = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_10d_c_ms, t
    )
    v_10d_c_ms = bs_value(
        s,
        t,
        k_10d_c_ms,
        r_d,
        r_f,
        sigma_k_10d_c_ms,
        OptionTypes.EUROPEAN_CALL.value,
    )
    sigma_k_10d_p_ms = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_10d_p_ms, t
    )
    v_10d_p_ms = bs_value(
        s,
        t,
        k_10d_p_ms,
        r_d,
        r_f,
        sigma_k_10d_p_ms,
        OptionTypes.EUROPEAN_PUT.value,
    )
    v_10d_ms = v_10d_c_ms + v_10d_p_ms
    term10d_1 = (v_10d_ms - v_10d_ms_target) ** 2
else:

```



```

term10d_1 = 0.0
#####
# Match the risk reversal volatility
#####
if target_10d_rr_vol > -999.0:
    k_10d_c = _solver_for_smile_strike(
        s,
        t,
        r_d,
        r_f,
        OptionTypes.EUROPEAN_CALL.value,
        vol_type_value,
        +0.1000,
        delta_method_value,
        k_10d_c_ms,
        params,
        strikes_null,
        gaps_null,
    )
    sigma_k_10d_c = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_10d_c, t
    )
    k_10d_p = _solver_for_smile_strike(
        s,
        t,
        r_d,
        r_f,
        OptionTypes.EUROPEAN_PUT.value,
        vol_type_value,
        -0.1000,
        delta_method_value,
        k_10d_p_ms,
        params,
        strikes_null,
        gaps_null,
    )
    sigma_k_10d_p = vol_function(
        vol_type_value, params, strikes_null, gaps_null, f, k_10d_p, t
    )
    sigma_10d_rr = sigma_k_10d_c - sigma_k_10d_p
    term10d_2 = (sigma_10d_rr - target_10d_rr_vol) ** 2
else:
    term10d_2 = 0.0
#####
# Alpha interpolates between fitting only ATM and 25d when alpha = 0.0 and
# fitting only ATM and 10d when alpha = 1.0. Equal when alpha = 0.50.
#####
tot = term_atm
tot = tot + (1.0 - alpha) * (term_25d_1 + term_25d_2)
tot = tot + alpha * (term10d_1 + term10d_2)
return tot
#####
not cache this function as it leads to complaints
#####
IS FUNCTION IS NOT USED CURRENTLY
jit(fastmath=True) # , cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
params	np.ndarray	-	
*args	Any)-float:	-	
"""Return a function that is minimised when the ATM	-	-	
MS and RR vols have	-	-	
been best fitted using the parametric volatility curve represented by	-	-	
params and specified by the vol_type_value	-	-	
"""	-	-	
s	-	-	a
t	-	-	a
r_d	-	-	a
r_f	-	-	a
k_atm	-	-	a
atm_vol	-	-	a
k_25d_c_ms	-	-	a
k_25d_p_ms	-	-	a
v_25d_ms_target	-	-	a
target_25d_rr_vol	-	-	a
k_10d_c_ms	-	-	an
k_10d_p_ms	-	-	an
v_10d_ms_target	-	-	an
target_10d_rr_vol	-	-	an
delta_method_value	-	-	an
vol_type_value	-	-	an
alpha	-	-	an
strikes_null	-	-	np.
gaps_null	-	-	np.
f	-	-	s * np.exp
vol_type_value	-	-	
params	-	-	
strikes_null	-	-	
gaps_null	-	-	
f	-	-	
k_atm	-	-	
t	-	-	
)	-	-	
term_atm	-	-	(atm_vol - atm
if target_25d_rr_vol < -999.0	-	-	
vol_type_value	-	-	
params	-	-	
strikes_null	-	-	
gaps_null	-	-	
f	-	-	
k_25d_c_ms	-	-	
t	-	-	
)	-	-	
s	-	-	
t	-	-	
k_25d_c_ms	-	-	
r_d	-	-	
r_f	-	-	
sigma_k_25d_c_ms	-	-	

_obj_gap

Return a function that is minimised when the ATM, MS and RR vols have been best fitted using the parametric volatility curve represented by params and specified by the vol_type_value

```
_obj_gap(gaps: np.ndarray, *args: Any) -> float:
    """Return a function that is minimised when the ATM, MS and RR vols have
    been best fitted using the parametric volatility curve represented by
    params and specified by the vol_type_value
    """
    s = args[0]
    t = args[1]
    r_d = args[2]
    r_f = args[3]
    k_atm = args[4]
    atm_vol = args[5]
    k_25d_c_ms = args[6]
    k_25d_p_ms = args[7]
    v_25d_ms_target = args[8]
    target_25d_rr_vol = args[9]
    k_10d_c_ms = args[10]
    k_10d_p_ms = args[11]
    v_10d_ms_target = args[12]
    target_10d_rr_vol = args[13]
    delta_method_value = args[14]
    vol_type_value = args[15]
    params = args[16]
    strikes = [k_10d_p_ms, k_25d_p_ms, k_atm, k_25d_c_ms, k_10d_c_ms]
    strikes = np.array(strikes)
    f = s * np.exp((r_d - r_f) * t)
    # We first need to solve for the strikes at the 25 delta points using the
    # new volatility curve
    # Match the at-the-money option volatility
    atm_curve_vol = vol_function(vol_type_value, params, strikes, gaps, f, k_atm, t)
    print("atm_curve_vol", atm_curve_vol)
    term_atm = (atm_vol - atm_curve_vol) ** 2
    #####
    # Match the market strangle value but this has to be at the MS 25d strikes
    #####
    sigma_k_25d_c_ms = vol_function(
        vol_type_value, params, strikes, gaps, f, k_25d_c_ms, t
    )
    print("sigma_k_25d_c_ms", sigma_k_25d_c_ms)
    v_25d_c_ms = bs_value(
        s,
        t,
        k_25d_c_ms,
        r_d,
        r_f,
        sigma_k_25d_c_ms,
        OptionTypes.EUROPEAN_CALL.value,
    )
    sigma_k_25d_p_ms = vol_function(
        vol_type_value, params, strikes, gaps, f, k_25d_p_ms, t
    )
    print("sigma_k_25d_p_ms", sigma_k_25d_p_ms)
```

```

v_25d_p_ms = bs_value(
    s,
    t,
    k_25d_p_ms,
    r_d,
    r_f,
    sigma_k_25d_p_ms,
    OptionTypes.EUROPEAN_PUT.value,
)
v_25d_ms = v_25d_c_ms + v_25d_p_ms
term_25d_1 = (v_25d_ms - v_25d_ms_target) ** 2
#####
# Match the risk reversal volatility
#####
k_25d_c = _solver_for_smile_strike(
    s,
    t,
    r_d,
    r_f,
    OptionTypes.EUROPEAN_CALL.value,
    vol_type_value,
    +0.2500,
    delta_method_value,
    k_25d_c_ms,
    params,
    strikes,
    gaps,
)
sigma_k_25d_c = vol_function(vol_type_value, params, strikes, gaps, f, k_25d_c, t)
print("sigma_k_25d_c", sigma_k_25d_c)
k_25d_p = _solver_for_smile_strike(
    s,
    t,
    r_d,
    r_f,
    OptionTypes.EUROPEAN_PUT.value,
    vol_type_value,
    -0.2500,
    delta_method_value,
    k_25d_p_ms,
    params,
    strikes,
    gaps,
)
sigma_k_25d_p = vol_function(vol_type_value, params, strikes, gaps, f, k_25d_p, t)
print("sigma_k_25d_p", sigma_k_25d_p)
sigma_25d_rr = sigma_k_25d_c - sigma_k_25d_p
term_25d_2 = (sigma_25d_rr - target_25d_rr_vol) ** 2
#####
# Match the market strangle value but this has to be at the MS 10d strikes
#####
sigma_k_10d_c_ms = vol_function(
    vol_type_value, params, strikes, gaps, f, k_10d_c_ms, t
)
print("sigma_k_10d_c_ms", sigma_k_10d_c_ms)
v_10d_c_ms = bs_value(
    s,

```

```

    t,
    k_10d_c_ms,
    r_d,
    r_f,
    sigma_k_10d_c_ms,
    OptionTypes.EUROPEAN_CALL.value,
)
sigma_k_10d_p_ms = vol_function(
    vol_type_value, params, strikes, gaps, f, k_10d_p_ms, t
)
print("sigma_k_10d_p_ms", sigma_k_10d_p_ms)
v_10d_p_ms = bs_value(
    s,
    t,
    k_10d_p_ms,
    r_d,
    r_f,
    sigma_k_10d_p_ms,
    OptionTypes.EUROPEAN_PUT.value,
)
v_10d_ms = v_10d_c_ms + v_10d_p_ms
term10d_l = (v_10d_ms - v_10d_ms_target) ** 2
#####
# Match the risk reversal volatility
#####
k_10d_c = _solver_for_smile_strike(
    s,
    t,
    r_d,
    r_f,
    OptionTypes.EUROPEAN_CALL.value,
    vol_type_value,
    +0.1000,
    delta_method_value,
    k_10d_c_ms,
    params,
    strikes,
    gaps,
)
sigma_k_10d_c = vol_function(vol_type_value, params, strikes, gaps, f, k_10d_c, t)
print("SIGMA_k_10d_c", sigma_k_10d_c)
print("INIT_k_10d_p_ms", k_10d_p_ms)
k_10d_p = _solver_for_smile_strike(
    s,
    t,
    r_d,
    r_f,
    OptionTypes.EUROPEAN_PUT.value,
    vol_type_value,
    -0.1000,
    delta_method_value,
    k_10d_p_ms,
    params,
    strikes,
    gaps,
)
print("k_10d_p", k_10d_p)

```

```

sigma_k_10d_p = vol_function(vol_type_value, params, strikes, gaps, f, k_10d_p, t)
print("SIGMA_k_10d_p", sigma_k_10d_p)
sigma_10d_rr = sigma_k_10d_c - sigma_k_10d_p
term10d_2 = (sigma_10d_rr - target_10d_rr_vol) ** 2
#####
# Alpha interpolates between fitting only ATM and 25d when alpha = 0.0 and
# fitting only ATM and 10d when alpha = 1.0. Equal when alpha = 0.50.
#####
tot = term_atm
tot = tot + (term_25d_1 + term_25d_2)
tot = tot + (term10d_1 + term10d_2)
return tot
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
gaps	np.ndarray	-
*args	Any)-float:	-
"""Return a function that is minimised when the ATM	-	-
MS and RR vols have	-	-
been best fitted using the parametric volatility curve represented by	-	-
params and specified by the vol_type_value	-	-
"""	-	-
s	-	-
t	-	-
r_d	-	-
r_f	-	-
k_atm	-	-
atm_vol	-	-
k_25d_c_ms	-	-
k_25d_p_ms	-	-
v_25d_ms_target	-	-
target_25d_rr_vol	-	-
k_10d_c_ms	-	-
k_10d_p_ms	-	-
v_10d_ms_target	-	-
target_10d_rr_vol	-	-
delta_method_value	-	-
vol_type_value	-	-
params	-	-
strikes	-	- [k_10d_p_ms, k_25d_c_ms]
strikes	-	-
f	-	- s *
atm_curve_vol	-	- vol_function(vol_type_value, k_atm, k_10d_p_ms, k_25d_c_ms, v_25d_ms_target, v_10d_ms_target, target_25d_rr_vol, target_10d_rr_vol, delta_method_value, vol_type_value)
print("atm_curve_vol", atm_curve_vol)	-	-
term_atm	-	- (atm.v
vol_type_value	-	-
params	-	-
strikes	-	-
gaps	-	-
f	-	-
k_25d_c_ms	-	-
t	-	-
)	-	-
print("sigma_k_25d_c_ms", sigma_k_25d_c_ms)	-	-
s	-	-
t	-	-
k_25d_c_ms	-	-
r_d	-	-
r_f	-	-
sigma_k_25d_c_ms	-	-
OptionTypes.EUROPEAN_CALL.value	-	-
)	-	-
vol_type_value	-	-
params	-	-
strikes	-	-
gaps	-	-

PLEASE ADD A FUNCTION DESCRIPTION

```
_solve_to_horizon(
s: float,
t: float,
rd: float,
rf: float,
k_atm: float,
atm_vol: float,
ms_25d_vol: float,
rr_25d_vol: float,
ms_10d_vol: float,
rr_10d_vol: float,
delta_method_value: int,
vol_type_value: int,
alpha: float,
x_inits: Sequence[float],
ginits: Sequence[float],
fin_solver_type: Any,
tol: float,
    Tuple[
np.ndarray,
np.ndarray,
np.ndarray,
float,
float,
float,
float,
float,
float,
float,
float,
float,
float,
float,
#####
# Determine the price of a market strangle from market strangle
# Need to price a call and put that agree with market strangle
#####
use10d = True
use25d = True
if ms_25d_vol == -999.0:
    use25d = False
if ms_10d_vol == -999.0:
    use10d = False
if use25d is True:
    vol_25d_ms = atm_vol + ms_25d_vol
    k_25d_c_ms = solve_for_strike(
        s,
        t,
        rd,
        rf,
        OptionTypes.EUROPEAN_CALL.value,
        +0.2500,
        delta_method_value,
        vol_25d_ms,
    )
```

```

k_25d_p_ms = solve_for_strike(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_PUT.value,
    -0.2500,
    delta_method_value,
    vol_25d_ms,
)
# USE MARKkET STRANGLE VOL TO DETERMINE PRICE OF A MARKkET STRANGLE
v_25d_c_ms = bs_value(
    s,
    t,
    k_25d_c_ms,
    rd,
    rf,
    vol_25d_ms,
    OptionTypes.EUROPEAN_CALL.value,
)
v_25d_p_ms = bs_value(
    s,
    t,
    k_25d_p_ms,
    rd,
    rf,
    vol_25d_ms,
    OptionTypes.EUROPEAN_PUT.value,
)
# Market price of strangle in the domestic currency
v_25d_ms = v_25d_c_ms + v_25d_p_ms
else:
    vol_25d_ms = -999.0
    k_25d_c_ms = 0.0
    k_25d_p_ms = 0.0
    v_25d_c_ms = 0.0
    v_25d_p_ms = 0.0
    v_25d_ms = 0.0
#####
if use10d is True:
    vol_10d_ms = atm_vol + ms_10d_vol
    k_10d_c_ms = solve_for_strike(
        s,
        t,
        rd,
        rf,
        OptionTypes.EUROPEAN_CALL.value,
        +0.1000,
        delta_method_value,
        vol_10d_ms,
    )
    k_10d_p_ms = solve_for_strike(
        s,
        t,
        rd,
        rf,
        OptionTypes.EUROPEAN_PUT.value,

```

```

        -0.1000,
        delta_method_value,
        vol_10d_ms,
    )
    # USE MARKkET STRANGLE VOL TO DETERMINE PRICE OF A MARKkET STRANGLE
    v_10d_c_ms = bs_value(
        s,
        t,
        k_10d_c_ms,
        rd,
        rf,
        vol_10d_ms,
        OptionTypes.EUROPEAN_CALL.value,
    )
    v_10d_p_ms = bs_value(
        s,
        t,
        k_10d_p_ms,
        rd,
        rf,
        vol_10d_ms,
        OptionTypes.EUROPEAN_PUT.value,
    )
    # Market price of strangle in the domestic currency
    v_10d_ms = v_10d_c_ms + v_10d_p_ms
else:
    vol_10d_ms = -999.0
    k_10d_c_ms = 0.0
    k_10d_p_ms = 0.0
    v_10d_c_ms = 0.0
    v_10d_p_ms = 0.0
    v_10d_ms = 0.0
#####
# Determine parameters of vol surface using minimisation
#####
# tol = 1e-8
args = (
    s,
    t,
    rd,
    rf,
    k_atm,
    atm_vol,
    k_25d_c_ms,
    k_25d_p_ms,
    v_25d_ms,
    rr_25d_vol,
    k_10d_c_ms,
    k_10d_p_ms,
    v_10d_ms,
    rr_10d_vol,
    delta_method_value,
    vol_type_value,
    alpha,
)
# Nelder-Mead (both SciPy & Numba) is quicker, but occasionally fails
# to converge, so for those cases try again with CG

```

```

# Numba version is quicker, but can be slightly away from CG output
xopt = None
try:
    if fin_solver_type == SolverTypes.NELDER_MEAD_NUMBA:
        xopt = nelder_mead(
            _obj,
            np.array(x_inits),
            bounds=np.array([[[]], [[]]].T,
            args=args,
            tol_f=tol,
            tol_x=tol,
            max_iter=1000,
        )
    elif fin_solver_type == SolverTypes.NELDER_MEAD:
        opt = minimize(_obj, x_inits, args, method="Nelder-Mead", tol=tol)
        xopt = opt.x
    elif fin_solver_type == SolverTypes.CONJUGATE_GRADIENT:
        opt = minimize(_obj, x_inits, args, method="CG", tol=tol)
        xopt = opt.x
except Exception:
    # If convergence fails try again with CG if necessary
    if fin_solver_type != SolverTypes.CONJUGATE_GRADIENT:
        print("Failed to converge, will try CG")
        opt = minimize(_obj, x_inits, args, method="CG", tol=tol)
        xopt = opt.x
params = np.array(xopt)
strikes = [k_10d_p_ms, k_25d_p_ms, k_atm, k_10d_c_ms, k_25d_c_ms]
strikes = np.array(strikes)
gaps = np.zeros(5)
#####
# Determine parameters of vol surface using minimisation
#####
if l == 0:
    tol = 1e-12
    args = (
        s,
        t,
        rd,
        rf,
        k_atm,
        atm_vol,
        k_25d_c_ms,
        k_25d_p_ms,
        v_25d_ms,
        rr_25d_vol,
        k_10d_c_ms,
        k_10d_p_ms,
        v_10d_ms,
        rr_10d_vol,
        delta_method_value,
        vol_type_value,
        params,
    )
    opt = minimize(_obj_gap, ginit, args, method="Nelder-Mead", tol=tol)
    xopt = opt.x
    gaps = np.array(xopt)
    print("SOLVED")

```

```

# Removed this as it causes discontinuity
# f = s * np.exp((rd-rf)*t)
# interpATMVOL = vol_function(vol_type_value, params,
#                             strikes, gaps, f, k_atm, t)
# diff = atm_vol - interpATMVOL
# gaps[2] = diff
#####
if use25d is False:
    k_25d_c_ms = k_atm
    k_25d_p_ms = k_atm
k_25d_c = _solver_for_smile_strike(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_CALL.value,
    vol_type_value,
    +0.2500,
    delta_method_value,
    k_25d_c_ms,
    params,
    strikes,
    gaps,
)
k_25d_p = _solver_for_smile_strike(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_PUT.value,
    vol_type_value,
    -0.2500,
    delta_method_value,
    k_25d_p_ms,
    params,
    strikes,
    gaps,
)
if use10d is False:
    k_10d_c_ms = k_atm
    k_10d_p_ms = k_atm
k_10d_c = _solver_for_smile_strike(
    s,
    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_CALL.value,
    vol_type_value,
    +0.1000,
    delta_method_value,
    k_10d_c_ms,
    params,
    strikes,
    gaps,
)
k_10d_p = _solver_for_smile_strike(
    s,

```

```

    t,
    rd,
    rf,
    OptionTypes.EUROPEAN_PUT.value,
    vol_type_value,
    -0.1000,
    delta_method_value,
    k_10d_p_ms,
    params,
    strikes,
    gaps,
)
return (
    params,
    strikes,
    gaps,
    k_25d_c_ms,
    k_25d_p_ms,
    k_25d_c,
    k_25d_p,
    k_10d_c_ms,
    k_10d_p_ms,
    k_10d_c,
    k_10d_p,
)
#####
t(
float64(int64, float64[:], float64[:], float64[:], float64, float64, float64),
cache=True,
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description
s	float	-
t	float	-
rd	float	-
rf	float	-
k_atm	float	-
atm_vol	float	-
ms_25d_vol	float	-
rr_25d_vol	float	-
ms_10d_vol	float	-
rr_10d_vol	float	-
delta_method_value	int	-
vol_type_value	int	-
alpha	float	-
x_inits	Sequence[float]	-
ginits	Sequence[float]	-
fin_solver_type	Any	-
tol	float	-
np.ndarray	-	-
np.ndarray	-	-
np.ndarray	-	-
float	-	-
float	-	-
float	-	-
float	-	-
float	-	-
float	-	-
float	-	-
float	-	-
use10d	-	-
use25d	-	-
if ms_25d_vol	-	-
use25d	-	-
if ms_10d_vol	-	-
use10d	-	-
if use25d is True	-	-
vol_25d_ms	-	-
s	-	-
t	-	-
rd	-	-
rf	-	-
OptionTypes.EUROPEAN_CALL.value	-	-
+0.2500	-	-
delta_method_value	-	-
vol_25d_ms	-	-
)	-	-
s	-	-
t	-	-
rd	-	-
rf	-	-
OptionTypes.EUROPEAN_PUT.value	-	-
0.2500	-	-

vol_function

Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book.

```
vol_function(
vol_function_type_value: int,
params: np.ndarray,
strikes: np.ndarray,
gaps: np.ndarray,
f: float,
k: float,
t: float,
float:
"""Return the volatility for a strike using a given polynomial
interpolation following Section 3.9 of Iain Clark book."""
if len(strikes) == 1:
    gap_k = 0.0
else:
    gap_k = _interpolate_gap(k, strikes, gaps)
if vol_function_type_value == VolFuncTypes.CLARK.value:
    vol = vol_function_clark(params, f, k, t) + gap_k
    return vol
if vol_function_type_value == VolFuncTypes.SABR.value:
    vol = vol_function_sabr(params, f, k, t) + gap_k
    return vol
if vol_function_type_value == VolFuncTypes.SABR_BETA_HALF.value:
    vol = vol_function_sabr_beta_half(params, f, k, t) + gap_k
    return vol
if vol_function_type_value == VolFuncTypes.SABR_BETA_ONE.value:
    vol = vol_function_sabr_beta_one(params, f, k, t) + gap_k
    return vol
if vol_function_type_value == VolFuncTypes.BBG.value:
    vol = vol_function_bloomberg(params, f, k, t) + gap_k
    return vol
if vol_function_type_value == VolFuncTypes.CLARK5.value:
    vol = vol_function_clark(params, f, k, t) + gap_k
    return vol
raise FinError("Unknown_Model_Type")
#####
t(cache=True, fastmath=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
vol_function_type_value	int	-	-
params	np.ndarray	-	-
strikes	np.ndarray	-	-
gaps	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float	-	-	-
"""Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book."""	-	-	-
if len(strikes)	-	-	= 1:
gap_k	-	-	0.0
else	-	-	-
gap_k	-	-	_interpolate_gap(k, st
if vol_function_type_value	-	-	= VolFuncTypes.CLA
vol	-	-	vol_function_clark(params
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SA
vol	-	-	vol_function_sabr(params
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SABR_BE
vol	-	-	vol_function_sabr_beta_half(pa
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.SABR_BE
vol	-	-	vol_function_sabr_beta_one(pa
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.BE
vol	-	-	vol_function_bloomberg(para
return vol	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.CLA
vol	-	-	vol_function_clark(params
return vol	-	-	-
raise FinError("Unknown Model Type")	-	-	-
t(cache	-	-	True, fastmath=

delta fit

PLEASE ADD A FUNCTION DESCRIPTION

```
_delta_fit(k: float, *args: Any) -> float:
    """This is the objective function used in the determination of the FX
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
*args	Any)-float:	-	-
"""This is the objective function used in the determination of the FX		-	-

`_solver_for_smile_strike`

Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike.

```

_solver_for_smile_strike(
s: float,
t: float,
rd: float,
rf: float,
opt_type_value: int,
vol_type_value: int,
delta_target: float,
delta_method_value: int,
initial_guess: float,
parameters: np.ndarray,
strikes: np.ndarray,
gaps: np.ndarray,
float:
"""Solve_for_the_strike_that_sets_the_delta_of_the_option_equal_to_the
target_value_of_delta_allowing_the_volatility_to_be_a_function_of_the
strike."""
inverse_delta_target = norminvcdf(np.abs(delta_target))
argtuple = (
    vol_type_value,
    s,
    t,
    rd,
    rf,
    opt_type_value,
    delta_method_value,
    inverse_delta_target,
    parameters,
    strikes,
    gaps,
)
k = newton_secant(_delta_fit, x0=initial_guess, args=argtuple, tol=1e-8, maxiter=50)
return k
#####
able to cache function and if I remove njit it complains about pickle
#####
t(
float64(float64, float64, float64, float64, int64, float64, int64, float64),
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description
s	float	-
t	float	-
rd	float	-
rf	float	-
opt_type_value	int	-
vol_type_value	int	-
delta_target	float	-
delta_method_value	int	-
initial_guess	float	-
parameters	np.ndarray	-
strikes	np.ndarray	-
gaps	np.ndarray	-
float		-
"""Solve for the strike that sets the delta of the option equal to the target value of delta allowing the volatility to be a function of the strike."""	-	-
inverse_delta_target	-	-
vol_type_value	-	-
s	-	-
t	-	-
rd	-	-
rf	-	-
opt_type_value	-	-
delta_method_value	-	-
inverse_delta_target	-	-
parameters	-	-
strikes	-	-
gaps	-	-
)	-	-
k	-	- newton_secant(_delta_
return k	-	-
able to cache function and if I remove njit it complains about pickle	-	-
float64(float64, float64, float64, float64, int64, float64, int64, float64)	-	-
fastmath	-	-

solve_for_strike

This function determines the implied strike of an FX option given a delta and the other option details. It uses a one-dimensional Newton root search algorithm to determine the strike that matches an input volatility.

```
solve_for_strike(
spot_fx_rate: float,
t_del: float,
rd: float,
rf: float,
```

```

opt_type_value: int,
delta_target: float,
delta_method_value: int,
volatility: float,
    float:
    """This function determines the implied strike of an FX option
    given a delta and the other option details. It uses a one-dimensional
    Newton root search algorithm to determine the strike that matches an
    input volatility."""
    # =====
    # IMPORTANT NOTE:
    # =====
    # For some delta quotation conventions I can solve for kk explicitly.
    # Note that as I am using the function norminvdelta to calculate the
    # inverse value of delta, this may not, on a round trip using normcdf(x)x, give
    # back the value x as it is calculated to a different number of decimal
    # places. It should however agree to 6-7 decimal places. Which is Okk.
    # =====
    if delta_method_value == FinFXDeltaMethod.SPOT_DELTA.value:
        dom_df = np.exp(-rd * t_del)
        for_df = np.exp(-rf * t_del)
        if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
            phi = +1.0
        else:
            phi = -1.0
        fwd_fx_rate = spot_fx_rate * for_df / dom_df
        vsqrtdelta = volatility * np.sqrt(t_del)
        arg = delta_target * phi / for_df # CHECKk THIS !!!
        norminvdelta = norminvcdf(arg)
        kk = fwd_fx_rate * np.exp(-vsqrtdelta * (phi * norminvdelta - vsqrtdelta / 2.0))
        return kk
    if delta_method_value == FinFXDeltaMethod.FORWARD_DELTA.value:
        dom_df = np.exp(-rd * t_del)
        for_df = np.exp(-rf * t_del)
        if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
            phi = +1.0
        else:
            phi = -1.0
        fwd_fx_rate = spot_fx_rate * for_df / dom_df
        vsqrtdelta = volatility * np.sqrt(t_del)
        arg = delta_target * phi
        norminvdelta = norminvcdf(arg)
        kk = fwd_fx_rate * np.exp(-vsqrtdelta * (phi * norminvdelta - vsqrtdelta / 2.0))
        return kk
    if delta_method_value == FinFXDeltaMethod.SPOT_DELTA_PREM_ADJ.value:
        argtuple = (
            spot_fx_rate,
            t_del,
            rd,
            rf,
            volatility,
            delta_method_value,
            opt_type_value,
            delta_target,
        )
        kk = newton_secant(_g, x0=spot_fx_rate, args=argtuple, tol=1e-7, maxiter=50)
        return kk

```

```

if delta_method_value == FinFXDeltaMethod.FORWARD_DELTA_PREM_ADJ.value:
    argtuple = (
        spot_fx_rate,
        t_del,
        rd,
        rf,
        volatility,
        delta_method_value,
        opt_type_value,
        delta_target,
    )
    kk = newton_secant(_g, x0=spot_fx_rate, args=argtuple, tol=1e-7, maxiter=50)
    return kk
raise FinError("Unknown_FinFXDeltaMethod")
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
spot_fx_rate	float	-
t_del	float	-
rd	float	-
rf	float	-
opt_type_value	int	-
delta_target	float	-
delta_method_value	int	-
volatility	float	-
float		-
"""This function determines the implied strike of an FX option given a delta and the other option details. It uses a one-dimensional Newton root search algorithm to determine the strike that matches an input volatility."""	-	-
if delta_method_value	-	- = FinF
dom_df	-	-
for_df	-	-
if opt_type_value	-	- = Opti
phi	-	-
else	-	-
phi	-	-
fwd_fx_rate	-	-
vsqrtt	-	-
arg	-	CHECKk THIS !!!
norminvdelta	-	-
kk	-	- fwd_fx_rate * np.
return kk	-	-
if delta_method_value	-	- = FinFXD
dom_df	-	-
for_df	-	-
if opt_type_value	-	- = Opti
phi	-	-
else	-	-
phi	-	-
fwd_fx_rate	-	-
vsqrtt	-	-
arg	-	-
norminvdelta	-	-
kk	-	- fwd_fx_rate * np.
return kk	-	-
if delta_method_value	-	- = FinFXDelta
spot_fx_rate	-	-
t_del	-	-
rd	-	-
rf	-	-
volatility	-	-
delta_method_value	-	-
opt_type_value	-	-
delta_target	-	-
)	-	-
kk	-	- newton_secant(_g, x0
return kk	-	-

3.5 ibor_cap_vol_curve

Class: IborCapVolCurve

Class to manage a term structure of cap (flat) volatilities and to do the conversion to caplet (spot) volatilities. This does not manage a strike dependency, only a term structure. The cap and caplet volatilities are keyed off the cap and caplet maturity dates. However this volatility only applies to the evolution of the Ibor rate out to the caplet start dates. Note also that this class also handles floor vols.

IborCapVolCurve

Create a cap/floor volatility curve given a curve date, a list of cap maturity dates and a vector of cap volatilities. To avoid confusion first date of the capDates must be equal to the curve date and first cap volatility for this date must equal zero. The internal times are calculated according to the provided day count convention. Note cap and floor volatilities are the same for the same strike and tenor, I just refer to cap volatilities in the code for code simplicity.

```
IborCapVolCurve(curve_dt: Date, # Valuation date for cap volatility
cap_maturity_dts: List[Date], # curve date + maturity dates for caps
cap_sigmas: np.ndarray, # Flat cap volatility for cap maturity dates
dc_type: Any,
    None:
    """Create a cap/floor volatility curve given a curve date, a list of
    cap_maturity_dates and a vector of cap volatilities. To avoid confusion
    first_date_of_the_capDates must be equal to the curve_date and first
    cap_volatility_for_this_date must equal zero. The internal times are
    calculated according to the provided day count convention. Note cap and
    floor volatilities are the same for the same strike and tenor, I just
    refer to cap volatilities in the code for code simplicity."""
    num_times = len(cap_maturity_dts)
    num_vols = len(cap_sigmas)
    if num_times != num_vols:
        raise FinError("Date_and_volatility_vectors_not_same_length.")
    if num_times < 2:
        raise FinError("FinCapVolCurve_requires_at_least_two_dates/vols")
    if curve_dt != cap_maturity_dts[0]:
        raise FinError("FinCapFloorDates_must_start_on_curve_date")
    ._curve_dt = curve_dt
    if cap_sigmas[0] != 0.0:
        raise FinError("Curve_date_cap_floor_volatility_must_equal_zero")
    ._cap_sigmas = np.array(cap_sigmas)
    ._caplet_gammas = []
    ._caplet_vols = []
    # Basic validation of dates
    prev_dt = ._curve_dt
    for dt in cap_maturity_dts[1:]:
        if dt < prev_dt:
            raise FinError("CapFloorLet_Dates_not_in_increasing_order")
        if dt == prev_dt:
            raise FinError("Two_successive_dates_are_identical")
        prev_dt = dt
    ._cap_maturity_dts = cap_maturity_dts
    if isinstance(dc_type, DayCountTypes) is False:
        raise FinError("DayCountType_must_be_of_type_DayCountTypes.")
```

```
._dc_type = dc_type
.generate_caplet_vols()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
curve_dt	Date	Valuation date for cap volatility
cap_maturity_dts	List[Date]	curve date + maturity dates for
cap_sigmas	np.ndarray	Flat cap volatility for cap maturity
dc_type	Any	-
None		-
"""Create a cap/floor volatility curve given a curve date	-	-
a list of	-	-
cap maturity dates and a vector of cap volatilities. To avoid confusion	-	-
first date of the capDates must be equal to the curve date and first	-	-
cap volatility for this date must equal zero. The internal times are	-	-
calculated according to the provided day count convention. Note cap and	-	-
floor volatilities are the same for the same strike and tenor	-	-
I just	-	-
refer to cap volatilities in the code for code simplicity."""	-	-
num_times	-	-
num_vols	-	-
if num_times !	-	-
raise FinError("Date and volatility vectors not same length.")	-	-
if num_times < 2	-	-
raise FinError("FinCapVolCurve requires at least two dates/vols")	-	-
if curve_dt !	-	-
raise FinError("FinCapFloorDates must start on curve date")	-	-
..curve_dt	-	-
if cap_sigmas[0] !	-	-
raise FinError("Curve date cap floor volatility must equal zero")	-	-
..cap_sigmas	-	-
..caplet_gammas	-	-
..caplet_vols	-	-
prev_dt	-	-
for dt in cap_maturity_dts[1	]:	-
if dt < prev_dt		-
raise FinError("CapFloorLet Dates not in increasing order")	-	-
if dt	-	-
raise FinError("Two successive dates are identical")	-	-
prev_dt	-	-
..cap_maturity_dts	-	-
if isinstance(dc_type, DayCountTypes) is False		-
raise FinError("DayCountType must be of type DayCountTypes.")	-	-
..dc_type	-	-
..generate_caplet_vols()	-	-

generate_caplet_vols

PLEASE ADD A FUNCTION DESCRIPTION

```

generate_caplet_vols() -> None:
"""Bootstrap caplet volatilities from cap volatilities using similar
notation to Hull's book (page 32.11). The first volatility in the
vector of caplet vols is zero."""
.times = []
._taus = []
day_counter = DayCount(._dc_type)
prev_dt = ._curve_dt
num_caps = len(._cap_maturity_dts)
for dt in ._cap_maturity_dts:
    t = (dt - ._curve_dt) / G_DAYS_IN_YEAR
    .times.append(t)
    tau = day_counter.year_frac(prev_dt, dt)[0]
    ._taus.append(tau)
    prev_dt = dt
fwd_rate_vol = ._cap_sigmas[0]
._caplet_gammas = np.zeros(num_caps)
._caplet_gammas[0] = 0.0
cum_ibor2_tau = (fwd_rate_vol**2) * ._taus[0]
sum_tau = 0.0
for i in range(1, len(._cap_maturity_dts)):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> None	-	-	-
"""Bootstrap caplet volatilities from cap volatilities using similar notation to Hull's book (page 32.11). The first volatility in the vector of caplet vols is zero."""	-	-	-
.times	-	-	[]
._taus	-	-	[]
day_counter	-	-	DayCount(._dc_type)
prev_dt	-	-	._curve_dt
num_caps	-	-	len(._cap_maturity_dts)
for dt in ._cap_maturity_dts	-	-	-
t	-	-	(dt - ._curve_dt) / G_DAYS_IN_YEAR
.times.append(t)	-	-	-
tau	-	-	day_counter.year_frac(prev_dt, dt)[0]
._taus.append(tau)	-	-	-
prev_dt	-	-	dt
fwd_rate_vol	-	-	._cap_sigmas[0]
._caplet_gammas	-	-	np.zeros(num_caps)
._caplet_gammas[0]	-	-	0.0
cum_ibor2_tau	-	-	(fwd_rate_vol**2) * ._taus[0]
sum_tau	-	-	0.0

caplet_vol

PLEASE ADD A FUNCTION DESCRIPTION

```
caplet_vol(dt: Union[Any, float]) -> float:
"""Return the forward rate caplet/floorlet volatility for a specific
forward caplet expiry date. The period of the volatility is the
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
Return the forward rate caplet/floorlet volatility for a specific	-	-	-
forward caplet expiry date. The period of the volatility is the	-	-	-

cap_vol

PLEASE ADD A FUNCTION DESCRIPTION

```
cap_vol(dt: Union[Any, float]) -> float:
"""Return the cap flat volatility for a specific cap maturity date for
the last caplet/floorlet in the cap/floor. The volatility interpolation
is piecewise flat."""
if isinstance(dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
Return the cap flat volatility for a specific cap maturity date for	-	-	-
the last caplet/floorlet in the cap/floor. The volatility interpolation	-	-	-
is piecewise flat.	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str	-	-	-

3.6 ibor_cap_vol_curve_fn

Class: IborCapVolCurveFn

Class to manage a term structure of caplet volatilities using the parametric form suggested by Rebonato (1999).

IborCapVolCurveFn

PLEASE ADD A FUNCTION DESCRIPTION

```
IborCapVolCurveFn(curve_dt: Date, a: float, b: float, c: float, d: float) -> None:
._curve_dt = curve_dt
._a = a
._b = b
._c = c
._d = d
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
curve_dt	Date	-	-
a	float	-	-
b	float	-	-
c	float	-	-
d	float)-;None:	-	-
._curve_dt	-	-	curve_dt
._a	-	-	a
._b	-	-	b
._c	-	-	c
._d	-	-	d

cap_floorlet_vol

PLEASE ADD A FUNCTION DESCRIPTION

```
cap_floorlet_vol(dt: Union[Date, float]) -> float:
"""Return_the_caplet_volatility."""
if isinstance(dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
"""Return the caplet volatility."""	-	-	-

3.7 swaption_vol_surface

Class: SwaptionVolSurface

Class to perform a calibration of a chosen parametrised surface to the prices of swaptions at different expiry dates and swap tenors. There is a choice of volatility function from cubic in delta to full SABR and SSVI. Check out VolFuncTypes. Visualising the volatility curve is useful. Also, there is no guarantee that the implied pdf will be positive.

SwaptionVolSurface

PLEASE ADD A FUNCTION DESCRIPTION

```
SwaptionVolSurface(value_dt: Date,
expiry_dts: List[Date],
fwd_swap_rates: np.ndarray,
strike_grid: np.ndarray,
vol_grid: np.ndarray,
vol_func_type: VolFuncTypes = VolFuncTypes.SABR,
fin_solver_type: SolverTypes = SolverTypes.NELDER_MEAD,
               None:
    """Create the FinSwaptionVolSurface object by passing in market vol
    data for a list of strikes and expiry dates."""
    check_argument_types(.SwaptionVolSurface, locals())
    .value_dt = value_dt
    if len(strike_grid.shape) != 2:
        raise FinError("Strike_grid must be a 2D grid of values")
    if len(vol_grid.shape) != 2:
        raise FinError("Volatility_grid must be a 2D grid of values")
    if len(strike_grid) != len(vol_grid):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
expiry_dts	List[Date]	-	-
fwd_swap_rates	np.ndarray	-	-
strike_grid	np.ndarray	-	-
vol_grid	np.ndarray	-	-
vol_func_type	VolFuncTypes	-	VolFuncTypes.NEL
fin_solver_type	SolverTypes	-	SolverTypes.NEL
None		-	-
"""Create the FinSwaptionVolSurface object by passing in market vol data for a list of strikes and expiry dates."""			
check_argument_types(.SwaptionVolSurface, locals())	-	-	-
.value_dt	-	-	value_dt
if len(strike_grid.shape) != 2:	-	-	2:
raise FinError("Strike grid must be a 2D grid of values")	-	-	-
if len(vol_grid.shape) != 2:	-	-	2:
raise FinError("Volatility grid must be a 2D grid of values")	-	-	-

vol_from_strike_dt

PLEASE ADD A FUNCTION DESCRIPTION

```

vol_from_strike_dt(k: float, expiry_dt: Date) -> float:
    """Interpolates the Black-Scholes volatility from the volatility
    surface given call option strike and expiry date. Linear interpolation
    is done in variance space. The smile strikes at bracketed dates are
    determined by determining the strike that reproduces the provided delta
    value. This uses the calibration delta convention, but it can be
    overridden by a provided delta convention. The resulting volatilities
    are then determined for each bracketing expiry time and linear
    interpolation is done in variance space and then converted back to a
    lognormal volatility."""
    t_exp = (expiry_dt - .value_dt) / G_DAYS_IN_YEAR
    vol_type_value = .vol_func_type.value
    index0 = 0 # lower index in bracket
    index1 = 0 # upper index in bracket
    num_curves = ._num_expiry_dts
    if num_curves == 1:
        index0 = 0
        index1 = 0
    # If the time is below first time then assume a flat vol
    elif t_exp <= ._t_exp[0]:
        index0 = 0
        index1 = 0
    # If the time is beyond the last time then extrapolate with a flat vol
    elif t_exp >= ._t_exp[-1]:
        index0 = len(._t_exp) - 1
        index1 = len(._t_exp) - 1
    else: # Otherwise we look for bracketing times and interpolate
        for i in range(1, num_curves):

```

The function arguments are described in the following table.

Argument Name	Type	Description
k	float	-
expiry_dt	Date)-ζfloat:	-
Interpolates the Black-Scholes volatility from the volatility surface given call option strike and expiry date. Linear interpolation is done in variance space. The smile strikes at bracketed dates are determined by determining the strike that reproduces the provided delta value. This uses the calibration delta convention but it can be overridden by a provided delta convention. The resulting volatilities are then determined for each bracketing expiry time and linear interpolation is done in variance space and then converted back to a lognormal volatility.	-	-
t_exp	-	-
vol_type_value	-	-
index0	-	lower index in bracket
index1	-	upper index in bracket
num_curves	-	-
if num_curves	-	-
index0	-	-
index1	-	-
elif t_exp < t_expiry	-	-
index0	-	-
index1	-	-
elif t_exp > t_expiry	-	-
index0	-	-
index1	-	-
else	Otherwise we look for bracketing	

`_build_vol_surface`

PLEASE ADD A FUNCTION DESCRIPTION

```

_build_vol_surface(fin_solver_type: SolverTypes = SolverTypes.NELDER_MEAD
None:
"""Main_function_to_construct_the_vol_surface."""
if ._vol_func_type == VolFuncTypes.CLARK:
    num_parameters = 3
elif ._vol_func_type == VolFuncTypes.SABR_BETA_ONE:
    num_parameters = 3
elif ._vol_func_type == VolFuncTypes.SABR_BETA_HALF:
    num_parameters = 3
elif ._vol_func_type == VolFuncTypes.BBG:
    num_parameters = 3
elif ._vol_func_type == VolFuncTypes.SABR:

```

```

num_parameters = 4
elif ._vol_func_type == VolFuncTypes.CLARK5:
    num_parameters = 5
elif ._vol_func_type == VolFuncTypes.SVI:
    num_parameters = 5
elif ._vol_func_type == VolFuncTypes.SSVI:
    num_parameters = 5
else:
    print(._vol_func_type)
    raise FinError("Unknown_Model_Type")
num_expiry_dts = ._num_expiry_dts
._parameters = np.zeros([num_expiry_dts, num_parameters])
._t_exp = np.zeros(num_expiry_dts)
#####
# TODO: ADD SPOT DAYS
#####
for i in range(0, num_expiry_dts):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fin_solver_type	SolverTypes	-	SolverTypes.NELDER_MEAD
None		-	-
"""Main function to construct the vol surface."""	-	-	-
if ._vol_func_type	-	-	= VolFuncTypes.CLARK:
num_parameters	-	-	3
elif ._vol_func_type	-	-	= VolFuncTypes.SABR_BETA_ONE:
num_parameters	-	-	3
elif ._vol_func_type	-	-	= VolFuncTypes.SABR_BETA_HALF:
num_parameters	-	-	3
elif ._vol_func_type	-	-	= VolFuncTypes.BBG:
num_parameters	-	-	3
elif ._vol_func_type	-	-	= VolFuncTypes.SABR:
num_parameters	-	-	4
elif ._vol_func_type	-	-	= VolFuncTypes.CLARK5:
num_parameters	-	-	5
elif ._vol_func_type	-	-	= VolFuncTypes.SVI:
num_parameters	-	-	5
elif ._vol_func_type	-	-	= VolFuncTypes.SSVI:
num_parameters	-	-	5
else		-	-
print(._vol_func_type)	-	-	-
raise FinError("Unknown Model Type")	-	-	-
num_expiry_dts	-	-	._num_expiry_dts
._parameters	-	-	np.zeros([num_expiry_dts, num_parameters])
._t_exp	-	-	np.zeros(num_expiry_dts)

check_calibration

PLEASE ADD A FUNCTION DESCRIPTION

```
check_calibration(verbose: bool, tol: float = 1e-6) -> None:
    """Compare_calibrated_vol_surface_with_market_and_output_a_report
    which_sets_out_the_quality_of_fit_to_the_ATM_and_10_and_25_delta_market
    strangles_and_risk_reversals."""
    if ._vol_grid == []:
        raise FinError("Error:_Vol_Grid_is_empty")
    if verbose:
        print("=====")
        print("VALUE_DATE:", .value_dt)
        print("STOCK_PRICE:", ._stock_price)
        print("=====")
    for i in range(0, ._num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description
verbose	bool	-
tol	float	-
"""Compare calibrated vol surface with market and output a report which sets out the quality of fit to the ATM and 10 and 25 delta market strangles and risk reversals."""	-	-
if ._vol_grid	-	-
raise FinError("Error VolGridisempty")	-	-
if verbose	-	-
print("=====	-	-
print("VALUE DATE	",".value_dt)	-
print("STOCK PRICE	","._stock_price)	-
print("=====	-	-

plot_vol_curves

PLEASE ADD A FUNCTION DESCRIPTION

```
plot_vol_curves() -> None:
    """Generates_a_plot_of_each_of_the_vol_discount_implied_by_the_market
    and_fitted."""
    for tenor_index in range(0, ._num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> None	-	-	-
"""Generates a plot of each of the vol discount implied by the market and fitted."""	-	-	-

__repr__*PLEASE ADD A FUNCTION DESCRIPTION*

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("VALUE_DATE", .value_dt)
s += label_to_string("STOCK_PRICE", .stock_price)
s += label_to_string("ATM_METHOD", .atm_method)
s += label_to_string("DELTA_METHOD", .delta_method)
s += label_to_string("VOL_FUNCTION", .vol_func_type)
for i in range(0, .num_expiry_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str	-	-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("VALUE DATE", .value_dt)
s +	-	-	label_to_string("STOCK PRICE", .stock_price)
s +	-	-	label_to_string("ATM METHOD", .atm_method)
s +	-	-	label_to_string("DELTA METHOD", .delta_method)
s +	-	-	label_to_string("VOL FUNCTION", .vol_func_type)

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print() -> None:
"""Print a list of the unadjusted coupon payment dates used in
analytic calculations for the bond."""
print()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> None	-	-	-
"""Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond."""	-	-	-
print()	-	-	-

_obj*PLEASE ADD A FUNCTION DESCRIPTION*

```
_obj(params: np.ndarray, *args: Any) -> float:
"""Return a value that is minimised when the ATM, MS and RR vols have
been best fitted using the parametric volatility curve represented by
params and specified by the vol_type value at a single time slice only.
```

```

"""
t = args[0]
f = args[1]
strikes_grid = args[2]
index = args[3]
vol_grid = args[4]
vol_type_value = args[5]
tot = 0.0
num_strikes = len(vol_grid)
for i in range(0, num_strikes):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	np.ndarray	-	-
*args	Any)-float:	-	-
Return a value that is minimised when the ATM MS and RR vols have	-	-	-
been best fitted using the parametric volatility curve represented by params and specified by the vol_type_value at a single time slice only.	-	-	-
"""	-	-	-
t	-	-	args[0]
f	-	-	args[1]
strikes_grid	-	-	args[2]
index	-	-	args[3]
vol_grid	-	-	args[4]
vol_type_value	-	-	args[5]
tot	-	-	0.0
num_strikes	-	-	len(vol_grid)

_solve_to_horizon

PLEASE ADD A FUNCTION DESCRIPTION

```

_solve_to_horizon(
t: float,
f: float,
strikes_grid: np.ndarray,
time_index: int,
vol_grid: np.ndarray,
vol_type_value: int,
x_inits: np.ndarray,
fin_solver_type: Any,
np.ndarray:
#####
# Determine parameters of vol surface using minimisation
#####
tol = 1e-6
args = (t, f, strikes_grid, time_index, vol_grid, vol_type_value)
# Nelder-Mead (both SciPy amd Numba) is quicker, but occasionally fails

```

```

# to converge, so for those cases try again with CG
# Numba version is quicker, but can be slightly away from CG output
xopt = None
try:
    if fin_solver_type == SolverTypes.NELDER_MEAD_NUMBA:
        xopt = nelder_mead(
            _obj,
            np.array(x_inits),
            bounds=np.array([[[]], [[]]]).T,
            args=args,
            tol_f=tol,
            tol_x=tol,
            max_iter=1000,
        )
    elif fin_solver_type == SolverTypes.NELDER_MEAD:
        opt = minimize(_obj, x_inits, args, method="Nelder-Mead", tol=tol)
        xopt = opt.x
    elif fin_solver_type == SolverTypes.CONJUGATE_GRADIENT:
        opt = minimize(_obj, x_inits, args, method="CG", tol=tol)
        xopt = opt.x
except Exception:
    # If convergence fails try again with CG if necessary
    if fin_solver_type != SolverTypes.CONJUGATE_GRADIENT:
        print("Failed_to_converge,_will_try_CG")
        opt = minimize(_obj, x_inits, args, method="CG", tol=tol)
        xopt = opt.x
# print("t: %9.5f alpha:%9.5f beta: %9.5f rho: %9.5f nu: %9.5f" %
#       (t, xopt[0], 0.5, xopt[1], xopt[2]))
params = np.array(xopt)
return params
#####
t(
float64(int64, float64[:], float64, float64, float64),
cache=True,
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float	-	-
f	float	-	-
strikes_grid	np.ndarray	-	-
time_index	int	-	-
vol_grid	np.ndarray	-	-
vol_type_value	int	-	-
x_inits	np.ndarray	-	-
fin_solver_type	Any	-	-
np.ndarray		-	-
tol	-	-	1e-6
args	-	(t, f, strikes_grid, time_index, vol_g	
xopt	-	-	None
try	-	-	-
if fin_solver_type	-	= SolverTypes.NELDER_ME	
_obj	-	-	-
np.array(x_inits)	-	-	-
bounds	-	-	np.array([[], []])
args	-	-	args
tol_f	-	-	tol
tol_x	-	-	tol
max_iter	-	-	1000
)	-	-	-
elif fin_solver_type	-	= SolverTypes.NELDER	
opt	-	minimize(_obj, x_inits, args, method=''	
xopt	-	-	opt.x
elif fin_solver_type	-	= SolverTypes.CONJUGATE	
opt	-	minimize(_obj, x_inits, args, meth	
xopt	-	-	opt.x
except Exception	-	-	-
if fin_solver_type !	-	SolverTypes.CONJUGATE	
print("Failed to converge, will try CG")	-	-	-
opt	-	minimize(_obj, x_inits, args, meth	
xopt	-	-	opt.x
params	-	-	np.array(xopt)
return params	-	-	-
float64(int64, float64[	],float64,float64,float64)	-	-
cache	-	-	True
fastmath	-	-	True

vol_function

Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book.

```

vol_function(
vol_function_type_value: int, params: np.ndarray, f: float, k: float, t: float
float:
"""Return the volatility for a strike using a given polynomial
interpolation following Section 3.9 of Iain Clark book."""
if vol_function_type_value == VolFuncTypes.CLARK.value:
    vol = vol_function_clark(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SABR_BETA_ONE.value:
    vol = vol_function_sabr_beta_one(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SABR_BETA_HALF.value:
    vol = vol_function_sabr_beta_half(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.BBG.value:
    vol = vol_function_bloomberg(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SABR.value:
    vol = vol_function_sabr(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.CLARK5.value:
    vol = vol_function_clark(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SVI.value:
    vol = vol_function_svi(params, f, k, t)
    return vol
elif vol_function_type_value == VolFuncTypes.SSVI.value:
    vol = vol_function_ssvi(params, f, k, t)
    return vol
else:
    raise FinError("Unknown_Model_Type")
#####
jit(cache=True, fastmath=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
vol_function_type_value	int	-	-
params	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float	-	-	-
"""Return the volatility for a strike using a given polynomial interpolation following Section 3.9 of Iain Clark book."""	-	-	-
if vol_function_type_value	-	-	= VolFuncTypes.CLARK
vol	-	-	vol_function_clark(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SABR_BETA
vol	-	-	vol_function_sabr_beta_one(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SABR_BETA
vol	-	-	vol_function_sabr_beta_half(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.BBG
vol	-	-	vol_function_bloomberg(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SABR
vol	-	-	vol_function_sabr(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.CLARK
vol	-	-	vol_function_clark(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SVI
vol	-	-	vol_function_svi(params)
return vol	-	-	-
elif vol_function_type_value	-	-	= VolFuncTypes.SSVI
vol	-	-	vol_function_ssvi(params)
return vol	-	-	-
else	-	-	-
raise FinError("Unknown Model Type")	-	-	-
jit(cache	-	-	True, fastmath=True)

3.8 `__init__`

Chapter 4

financepy.products.equity

Equity Products

This folder contains a set of Equity-related products. It includes:

EquityVanillaOption

Handles simple European-style call and put options on a dividend paying stock with analytical and monte-carlo valuations.

EquityAmericanOption

Handles America-style call and put options on a dividend paying stock with tree-based valuations.

EquityAsianOption

Handles call and put options where the payoff is determined by the average-stock price over some period before expiry.

EquityBasketOption

Handles call and put options on a basket of assets, with an analytical and Monte-Carlo valuation according to Black-Scholes model.

EquityCompoundOption

Handles options to choose to enter into a call or put option. Has an analytical valuation model for European style options and a tree model if either or both options are American style exercise.

EquityDigitalOption

Handles European-style options to receive cash or nothing, or to receive the asset or nothing. Has an analytical valuation model for European style options.

EquityFixedLookbackOption

Handles European-style options to receive the positive difference between the strike and the minimum (put) or maximum (call) of the stock price over the option life.

EquityFloatLookbackOption

Handles an equity option in which the strike of the option is not fixed but is set at expiry to equal the minimum stock price in the case of a call or the maximum stock price in the case of a put. In other words the buyer of the call gets to buy the asset at the lowest price over the period before expiry while the buyer of the put gets to sell the asset at the highest price before expiry. """

EquityBarrierOption

Handles an option which either knocks-in or knocks-out if a specified barrier is crossed from above or below, resulting in owning or not owning a call or a put option. There are eight variations which are all valued.

EquityRainbowOption

TBD

EquityVarianceSwap

TBD

Products that have not yet been implemented include:

- Power Options
- Ratchet Options
- Forward Start Options
- Log Options

4.1 equity_american_option

Class: EquityAmericanOption(EquityOption)

Class for American (and European) style options on simple vanilla calls and puts - a tree valuation model is used that can handle both.

EquityAmericanOption

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityAmericanOption(expiry_dt: Date,
strike_price: float,
opt_type: OptionTypes,
num_options: float = 1.0,
"""Class for American style options on simple vanilla calls and puts.
Specify the expiry date, strike price, whether the option is a call or
put and the number of options."""
check_argument_types(.EquityAmericanOption, locals())
if (opt_type != OptionTypes.EUROPEAN_CALL
    and opt_type != OptionTypes.EUROPEAN_PUT
    and opt_type != OptionTypes.AMERICAN_CALL
    and opt_type != OptionTypes.AMERICAN_PUT
):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
strike_price	float	-	-
opt_type	OptionTypes	-	-
num_options	float	-	1.0
"""Class for American style options on simple vanilla calls and puts.	-	-	-
Specify the expiry date	-	-	-
strike price	-	-	-
whether the option is a call or	-	-	-
put and the number of options."""	-	-	-
check_argument_types(.EquityAmericanOption, locals())	-	-	-
and opt_type !	-	-	OptionTypes.EUROPEAN_CALL
and opt_type !	-	-	OptionTypes.AMERICAN_CALL
and opt_type !	-	-	OptionTypes.AMERICAN_PUT

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
stock_price: Union[np.ndarray, float],
discount_curve: DiscountCurve,
```

```

dividend_curve: DiscountCurve,
model: Model,
"""Valuation of an American option using a CRR tree to take into
account the value of early exercise."""
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
    )
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date"
    )
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	np.ndarray or float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Valuation of an American option using a CRR tree to take into account the value of early exercise."""	-	-	-
if discount_curve.value_dt !)	-	-	value_dt:
if dividend_curve.value_dt !)	-	-	value_dt:

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

4.2 equity_asian_option

Enumerated Type: AsianOptionValuationMethods

This enumerated type has the following values:

- GEOMETRIC
- TURNBULL_WAKEMAN
- CURRAN
- tv
- t0
- t
- tau
- We can be after the start of the averaging period in which case we set t0
- and we note that;
- num_obs:int
- self.start_averaging_date
- self.expiry_dt
- self.strike_price
- self.opt_type
- self.num_observations
- accrued_average:float
- if discount_curve.value_dt!
- if dividend_curve.value_dt!
- if method
- v
- elif method
- v
- elif method
- v
- t0
- t_exp
- tau
- r
- q
- volatility
- k
- n
- s0
- multiplier
- k
- multiplier
- t0
- n
- sig_sq
- mean_geo
- var_geo

- eg
- d1
- d2
- call_g
- ifself.opt_type
- v
- elifself.opt_type
- put_g
- v
- v
- t0
- t_exp
- tau
- multiplier
- r
- q
- volatility
- s0
- b
- sigma2
- k
- n
- k
- multiplier
- t0
- n
- h
- u
- w
- fa
- ea2
- ea2
- sigma_aa
- d1
- d2
- ifself.opt_type
- v
- elifself.opt_type
- v
- v
- t0
- t_exp
- tau
- k
- multiplier
- n

- r
- q
- volatility
- k
- multiplier
- t0
- n
- b
- sigma2
- a1
- a2
- s0
- dt
- ifb
- m1
- m2
- m2
- m1
- m2
- m2
- f0
- sigma2
- sigma
- d1
- d2
- ifself.opt_type
- call
- v
- elifself.opt_type
- put
- v
- v
- t0
- t_exp
- tau
- r
- q
- volatility
- k
- n
- v
- t0
- t_exp
- tau
- k
- n

- r
- q
- volatility
- v
- t0
- t_exp
- tau
- k
- n
- r
- q
- volatility
- v_g_exact
- v
- s
- s+
- s+
- s+
- s+
- s+

Class: EquityAsianOption

Class for an Equity Asian Option. This is an option with a final payoff linked to the averaging of the stock price over some specified period before the option expires. The valuation is done for both an arithmetic and a geometric average but the former can only be done either using an analytical approximation of the arithmetic average distribution or by using Monte-Carlo simulation.

EquityAsianOption

Create an EquityAsian option object which takes a start date for the averaging, an expiry date, a strike price, an option type and a number of observations.

```
EquityAsianOption(start_averaging_dt: Date,
expiry_dt: Date,
strike_price: float,
opt_type: OptionTypes,
num_obs: int = 100,
"""Create an EquityAsian option object which takes a start date for
the averaging, an expiry date, a strike price, an option type and a
number of observations."""
check_argument_types(.EquityAsianOption, locals())
if start_averaging_dt > expiry_dt:
    raise FinError("Averaging starts after expiry date")
.start_averaging_date = start_averaging_dt
.expiry_dt = expiry_dt
.strike_price = float(strike_price)
.opt_type = opt_type
.num_observations = num_obs
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_averaging_dt	Date	-	-
expiry_dt	Date	-	-
strike_price	float	-	-
opt_type	OptionTypes	-	-
num_obs	int	-	100
"""Create an EquityAsian option object which takes a start date for			
the averaging	-	-	-
an expiry date	-	-	-
a strike price	-	-	-
an option type and a	-	-	-
number of observations."""	-	-	-
check_argument_types(EquityAsianOption, locals())	-	-	-
if start_averaging_dt > expiry_dt	-	-	-
raise FinError("Averaging starts after expiry date")	-	-	-
.start_averaging_date	-	-	start_averaging_dt
.expiry_dt	-	-	expiry_dt
.strike_price	-	-	float(strike_price)
.opt_type	-	-	opt_type
.num_observations	-	-	num_obs

value

Calculate the value of an Asian option using one of the specified analytical approximations for an average rate option. These are the three enumerated values in the enum `AsianOptionValuationMethods`. The choices of approximation are (i) `GEOMETRIC` - the average is a geometric one as in paper by Kenna and Worst (1990), (ii) `TURNBULL_WAKEMAN` - this is a value based on an edgeworth expansion of the moments of the arithmetic average, and (iii) `CURRAN` - another approximative approach by Curran based on conditioning on the geometric mean price. Just choose the corresponding enumerated value to switch between these different approaches. Note that the accrued average is only required if the value date is inside the averaging period for the option.

```
value(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
method: AsianOptionValuationMethods,
accrued_average: float = None,
"""Calculate the value of an Asian option using one of the specified
analytical approximations for an average rate option. These are the
three enumerated values in the enum AsianOptionValuationMethods. The
choices of approximation are (i) GEOMETRIC - the average is a geometric
one as in paper by Kenna and Worst (1990), (ii) TURNBULL_WAKEMAN -
this is a value based on an edgeworth expansion of the moments of the
arithmetic average, and (iii) CURRAN - another approximative approach
```

```

by_Curran_based_on_conditioning_on_the_geometric_mean_price._Just
choose_the_corresponding_enumerated_value_to_switch_between_these
different_approaches.
Note_that_the_accrued_average_is_only_required_if_the_value_date_is
inside_the_averaging_period_for_the_option.""
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_valuation_date"
    )
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_valuation_date_not_same_as_option_valuation_date"
    )
if method == AsianOptionValuationMethods.GEOMETRIC:
    v = .value_geometric(value_dt,
        stock_price,
        discount_curve,
        dividend_curve,
        model,
        accrued_average,
    )
elif method == AsianOptionValuationMethods.TURNBULL_WAKEMAN:
    v = .value_turnbull_wakeman(value_dt,
        stock_price,
        discount_curve,
        dividend_curve,
        model,
        accrued_average,
    )
elif method == AsianOptionValuationMethods.CURRAN:
    v = .value_curran(value_dt,
        stock_price,
        discount_curve,
        dividend_curve,
        model,
        accrued_average,
    )
else:
    raise FinError("Unknown_valuation_model")
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
method	AsianOptionValuationMethods	-
accrued_average	float	-
"""Calculate the value of an Asian option using one of the specified analytical approximations for an average rate option. These are the three enumerated values in the enum AsianOptionValuationMethods. The choices of approximation are (i) GEOMETRIC - the average is a geometric one as in paper by Kenna and Worst (1990)	-	-
(ii) TURNBULL_WAKEMAN -	-	-
this is a value based on an edgeworth expansion of the moments of the arithmetic average	-	-
and (iii) CURRAN - another approximative approach by Curran based on conditioning on the geometric mean price. Just choose the corresponding enumerated value to switch between these different approaches.	-	-
Note that the accrued average is only required if the value date is inside the averaging period for the option."""	-	-
if value_dt > expiry_dt	-	-
raise FinError("Value date after expiry date.")	-	-
if discount_curve.value_dt !	-	-
)	-	-
if dividend_curve.value_dt !	-	-
)	-	-
if method	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
)	-	-
elif method	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
)	-	-
elif method	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
)	-	-
else	-	-
raise FinError("Unknown valuation model")	-	-
return v	-	-

value_geometric

This option valuation is based on paper by Kemna and Vorst 1990. It calculates the Geometric Asian option price which is a lower bound on the Arithmetic option price. This should not be used as a valuation model for the Arithmetic Average option but can be used as a control variate for other approaches.

```

value_geometric(value_dt,
stock_price,
discount_curve,
dividend_curve,
model,
accrued_average,
"""This option valuation is based on paper by Kemna and Vorst 1990. It
calculates the Geometric Asian option price which is a lower bound on
the Arithmetic option price. This should not be used as a valuation
model for the Arithmetic Average option but can be used as a control
variate for other approaches."""
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_option_expiry_date.")
# the years to the start of the averaging period
t0 = (.start_averaging_date - value_dt) / G_DAYS_IN_YEAR
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
tau = (.expiry_dt - .start_averaging_date) / G_DAYS_IN_YEAR
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
volatility = model.volatility
k = .strike_price
n = .num_observations
s0 = stock_price
multiplier = 1.0
if t0 < 0: # we are in the averaging period
    if accrued_average is None:
        raise FinError(error_str)
    # we adjust the strike to account for the accrued coupon
    k = (k * tau + accrued_average * t0) / t_exp
    # the number of options is rescaled also
    multiplier = t_exp / tau
    # there is no pre-averaging time
    t0 = 0.0
    # the number of observations is scaled
    n = n * t_exp / tau
sig_sq = volatility**2
mean_geo = (r - q - sig_sq / 2.0) * (t0 + (t_exp - t0) / 2.0)
var_geo = sig_sq * (t0 + (t_exp - t0) * (2 * n - 1) / (6 * n))
eg = s0 * np.exp(mean_geo + var_geo / 2.0)
if np.abs(var_geo) < 1e-10:
    raise FinError("Asian_option_geometric_variance_is_zero.")
d1 = (mean_geo + np.log(s0 / k) + var_geo) / np.sqrt(var_geo)
d2 = d1 - np.sqrt(var_geo)
# the Geometric price is the lower bound
call_g = np.exp(-r * t_exp) * (eg * normcdf(d1) - k * normcdf(d2))
if .opt_type == OptionTypes.EUROPEAN_CALL:
    v = call_g
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    put_g = call_g - (eg - k) * np.exp(-r * t_exp)
    v = put_g

```

```
else:
    raise FinError("Unknown_option_type_" + str(.opt_type))
v = v * multiplier
return v
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
"""This option valuation is based on paper by Kemna and Vorst 1990. It calculates the Geometric Asian option price which is a lower bound on the Arithmetic option price. This should not be used as a valuation model for the Arithmetic Average option but can be used as a control variate for other approaches."""	-	-
if value_dt < .expiry_dt	-	-
raise FinError("Value date after option expiry date.")	-	-
t0	-	(.start)
t_exp	-	-
tau	-	(.expiry)
r	-	-
q	-	-
volatility	-	-
k	-	-
n	-	-
s0	-	-
multiplier	-	-
if t0 > 0	-	we are in the averaging period
if accrued_average is None	-	-
raise FinError(error_str)	-	-
k	-	-
multiplier	-	-
t0	-	-
n	-	-
sig_sq	-	-
mean_geo	-	-
var_geo	-	-
eg	-	-
if np.abs(var_geo) > 1e-10	-	-
raise FinError("Asian option geometric variance is zero.")	-	-
d1	-	(mean)
d2	-	-
call_g	-	np.exp
if .opt_type	-	-
v	-	-
elif .opt_type	-	-
put_g	-	-
v	-	-
else	-	-
raise FinError("Unknown option type " + str(.opt_type))	-	-
v	-	-
return v	-	-

value_curran*Valuation of an Asian option using the result by Vorst.*

```

value_curran(value_dt,
stock_price,
discount_curve,
dividend_curve,
model,
accrued_average,
"""Valuation_of_an_Aasian_option_using_the_result_by_Vorst."""
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_option_expiry_date.")
# the years to the start of the averaging period
t0 = (.start_averaging_date - value_dt) / G_DAYS_IN_YEAR
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
tau = (.expiry_dt - .start_averaging_date) / G_DAYS_IN_YEAR
multiplier = 1.0
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
volatility = model.volatility
s0 = stock_price
b = r - q
sigma2 = volatility**2
k = .strike_price
n = .num_observations
if t0 < 0: # we are in the averaging period
    if accrued_average is None:
        raise FinError(error_str)
    # we adjust the strike to account for the accrued coupon
    k = (k * tau + accrued_average * t0) / t_exp
    # the number of options is rescaled also
    multiplier = t_exp / tau
    # there is no pre-averaging time
    t0 = 0.0
    # the number of observations is scaled and floored at 1
    n = int(n * t_exp / tau + 0.5) + 1
h = (t_exp - t0) / (n - 1)
u = (1.0 - np.exp(b * h * n)) / (1.0 - np.exp(b * h))
w = (1.0 - np.exp((2 * b + sigma2) * h * n)) / (1.0 - np.exp((2 * b + sigma2) * h))
)
fa = (s0 / n) * np.exp(b * t0) * u
ea2 = (s0 * s0 / n / n) * np.exp((2.0 * b + sigma2) * t0)
ea2 = ea2 * (w + 2.0 / (1.0 - np.exp((b + sigma2) * h)) * (u - w))
sigma_aa = np.sqrt((np.log(ea2) - 2.0 * np.log(fa)) / t_exp)
d1 = (np.log(fa / k) + sigma_aa * sigma_aa * t_exp / 2.0) / (sigma_aa * np.sqrt(t_exp))
)
d2 = d1 - sigma_aa * np.sqrt(t_exp)
if .opt_type == OptionTypes.EUROPEAN_CALL:
    v = np.exp(-r * t_exp) * (fa * normcdf(d1) - k * normcdf(d2))
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    v = np.exp(-r * t_exp) * (k * normcdf(-d2) - fa * normcdf(-d1))
else:
    return None
v = v * multiplier
return v

```


#####

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
"""Valuation of an Asian option using the result by Vorst."""	-	-
if value_dt < .expiry_dt	-	-
raise FinError("Value date after option expiry date.")	-	-
t0	-	(.start_averaging
t_exp	-	(.expiry_d
tau	-	(.expiry_dt - .sta
multiplier	-	-
r	-	disco
q	-	divid
volatility	-	-
s0	-	-
b	-	-
sigma2	-	-
k	-	-
n	-	-
if t0 > 0	-	we are in the averaging period
if accrued_average is None	-	-
raise FinError(error_str)	-	-
k	-	(k * tau
multiplier	-	-
t0	-	-
n	-	in
h	-	-
u	-	(1.0 - np.ex
)	-	-
fa	-	(s
ea2	-	(s0 * s0 / n /
ea2	-	ea2 * (w + 2.0 / (
sigma_aa	-	np.sqrt((np
)	-	-
d2	-	d1
if .opt_type	-	= Opti
v	-	np.exp(-r * t_exp
elif .opt_type	-	= Opt
v	-	np.exp(-r * t_exp)
else	-	-
return None	-	-
v	-	-
return v	-	-

value_turnbull_wakeman

Asian option valuation based on paper by Turnbull and Wakeman 1991 which uses the edgeworth expansion to find the first two moments of the arithmetic average.

```

value_turnbull_wakeman(value_dt,
stock_price,
discount_curve,
dividend_curve,
model,
accrued_average,
"""Asian_option_valuation_based_on_paper_by_Turnbull_and_Wakeman_1991
which_uses_the_edgeworth_expansion_to_find_the_first_two_moments_of_the
arithmetic_average."""
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_option_expiry_date.")
t0 = (.start_averaging_date - value_dt) / G_DAYS_IN_YEAR
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
tau = (.expiry_dt - .start_averaging_date) / G_DAYS_IN_YEAR
k = .strike_price
multiplier = 1.0
n = .num_observations
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
volatility = model.volatility
if t0 < 0: # we are in the averaging period
    if accrued_average is None:
        raise FinError(error_str)
    # we adjust the strike to account for the accrued coupon
    k = (k * tau + accrued_average * t0) / t_exp
    # the number of options is rescaled also
    multiplier = t_exp / tau
    # there is no pre-averaging time
    t0 = 0.0
    # the number of observations is scaled and floored at 1
    n = int(n * t_exp / tau + 0.5) + 1
# need to handle this
b = r - q
sigma2 = volatility**2
a1 = b + sigma2
a2 = 2 * b + sigma2
s0 = stock_price
dt = t_exp - t0
if b == 0:
    m1 = 1.0
    m2 = 2.0 * np.exp(sigma2 * t_exp) - 2.0 * np.exp(sigma2 * t0) * (1.0 + sigma2 * dt
    )
    m2 = m2 / sigma2 / sigma2 / dt / dt
else:
    m1 = s0 * (np.exp(b * t_exp) - np.exp(b * t0)) / (b * dt)
    m2 = np.exp(a2 * t_exp) / a1 / a2 / dt / dt + (np.exp(a2 * t0) / b / dt / dt
    ) * (1.0 / a2 - np.exp(b * dt) / a1)
    m2 = 2.0 * m2 * s0 * s0
f0 = m1
sigma2 = (1.0 / t_exp) * np.log(m2 / m1 / m1)
sigma = np.sqrt(sigma2)

```

```

d1 = (np.log(f0 / k) + sigma2 * t_exp / 2) / sigma / np.sqrt(t_exp)
d2 = d1 - sigma * np.sqrt(t_exp)
if .opt_type == OptionTypes.EUROPEAN_CALL:
    call = np.exp(-r * t_exp) * (f0 * normcdf(d1) - k * normcdf(d2))
    v = call
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    put = np.exp(-r * t_exp) * (k * normcdf(-d2) - f0 * normcdf(-d1))
    v = put
else:
    return None
v = v * multiplier
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
"""Asian option valuation based on paper by Turnbull and Wakeman 1991 which uses the edgeworth expansion to find the first two moments of the arithmetic average."""	-	-
if value_dt ≤ expiry_dt	-	-
raise FinError("Value date after option expiry date.")	-	-
t0	-	-
t_exp	-	-
tau	-	-
k	-	-
multiplier	-	-
n	-	-
r	-	-
q	-	-
volatility	-	-
if t0 ≤ 0		we are in the averaging period
if accrued_average is None		-
raise FinError(error_str)	-	-
k	-	-
multiplier	-	-
t0	-	-
n	-	-
b	-	-
sigma2	-	-
a1	-	-
a2	-	-
s0	-	-
dt	-	-
if b	-	-
m1	-	-
)	-	-
m2	-	-
else		-
m1	-	-
m2	-	-
f0	-	-
sigma2	-	-
sigma	-	-
d1	-	-
d2	-	-
if .opt_type	-	-
call	-	-
v	-	-
elif .opt_type	-	-
put	-	-

value_mc

Monte Carlo valuation of the Asian Average option using standard Monte Carlo code enhanced by Numba. I have discontinued the use of this as it is both slow and has limited variance reduction.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_paths: int,
seed: int,
accrued_average: float,
"""Monte Carlo valuation of the Asian Average option using standard
Monte Carlo code enhanced by Numba. I have discontinued the use of this
as it is both slow and has limited variance reduction."""
# Basic validation
if value_dt > .expiry_dt:
    raise FinError("Value date after option expiry date.")
if value_dt > .start_averaging_date and accrued_average is None:
    raise FinError(error_str)
# the years to the start of the averaging period
t0 = (.start_averaging_date - value_dt) / G_DAYS_IN_YEAR
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
tau = (.expiry_dt - .start_averaging_date) / G_DAYS_IN_YEAR
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
volatility = model.volatility
k = .strike_price
n = .num_observations
v = equity_asian_value_mc_numba(t0,
    t_exp,
    tau,
    k,
    n,
    .opt_type,
    stock_price,
    r,
    q,
    volatility,
    num_paths,
    seed,
    accrued_average,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
num_paths	int	-
seed	int	-
accrued_average	float	-
"""Monte Carlo valuation of the Asian Average option using standard Monte Carlo code enhanced by Numba. I have discontinued the use of this as it is both slow and has limited variance reduction."""	-	-
if value_dt > expiry_dt	-	-
raise FinError("Value date after option expiry date.")	-	-
if value_dt < start_averaging_date and accrued_average is None	-	-
raise FinError(error_str)	-	-
t0	-	(.start_averaging_date)
t_exp	-	(.expiry_date)
tau	-	(.expiry_date - t0)
r	-	discount rate
q	-	dividend yield
volatility	-	-
k	-	-
n	-	-
t_exp	-	-
tau	-	-
k	-	-
n	-	-
.opt_type	-	-
stock_price	-	-
r	-	-
q	-	-
volatility	-	-
num_paths	-	-
seed	-	-
accrued_average	-	-
)	-	-
return v	-	-

value_mc_fast

Monte Carlo valuation of the Asian Average option. This method uses a lot of Numpy vectorisation. It is also helped by Numba.

```
value_mc_fast(value_dt,
              stock_price,
```

```

discount_curve,
dividend_curve, # Yield
model, # Model
num_paths, # Numpaths integer
seed,
accrued_average,
"""Monte_Carlo_valuation_of_the_Asian_Average_option._This_method_uses
a_lot_of_Numpy_vectorisation._It_is_also_helped_by_Numba."""
# the years to the start of the averaging period
t0 = (.start_averaging_date - value_dt) / G_DAYS_IN_YEAR
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
tau = (.expiry_dt - .start_averaging_date) / G_DAYS_IN_YEAR
k = .strike_price
n = .num_observations
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
volatility = model.volatility
v = equity_asian_value_mc_fast_numba(t0,
    t_exp,
    tau,
    k,
    n,
    .opt_type.value,
    stock_price,
    r,
    q,
    volatility,
    num_paths,
    seed,
    accrued_average,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	Yield
model	-	Model
num_paths	-	Numpaths integer
seed	-	-
accrued_average	-	-
"""Monte Carlo valuation of the Asian Average option. This method uses a lot of Numpy vectorisation. It is also helped by Numba."""		
t0	-	(.start_averaging
t_exp	-	(.expiry_dt
tau	-	(.expiry_dt - .start
k	-	-
n	-	-
r	-	discou
q	-	divide
volatility	-	-
t_exp	-	-
tau	-	-
k	-	-
n	-	-
.opt_type.value	-	-
stock_price	-	-
r	-	-
q	-	-
volatility	-	-
num_paths	-	-
seed	-	-
accrued_average	-	-
)	-	-
return v	-	-

value_mc_fast_vc_numba

Monte Carlo valuation of the Asian Average option using a control variate method that improves accuracy and reduces the variance of the price. This uses Numpy and Numba. This is the standard MC pricer.

```
value_mc_fast_vc_numba(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_paths: int,
seed: int,
accrued_average: float,
```

```

"""Monte_Carlo_valuation_of_the_Asian_Average_option_using_a_control
variate_method_that_improves_accuracy_and_reduces_the_variance_of_the
price._This_uses_Numpy_and_Numba._This_is_the_standard_MC_pricer."""
# the years to the start of the averaging period
t0 = (.start_averaging_date - value_dt) / G_DAYS_IN_YEAR
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
tau = (.expiry_dt - .start_averaging_date) / G_DAYS_IN_YEAR
k = .strike_price
n = .num_observations
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
volatility = model.volatility
# For control variate we price a Geometric average option exactly
v_g_exact = .value_geometric(value_dt,
    stock_price,
    discount_curve,
    dividend_curve,
    model,
    accrued_average,
)
v = equity_asian_value_mc_fast_cv_numba(t0,
    t_exp,
    tau,
    k,
    n,
    .opt_type,
    stock_price,
    r,
    q,
    volatility,
    num_paths,
    seed,
    accrued_average,
    v_g_exact,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
num_paths	int	-
seed	int	-
accrued_average	float	-
"""Monte Carlo valuation of the Asian Average option using a control variate method that improves accuracy and reduces the variance of the price. This uses Numpy and Numba. This is the standard MC pricer."""	-	-
t0	-	(.start_averaging)
t_exp	-	(.expiry_dt)
tau	-	(.expiry_dt - .start_averaging)
k	-	-
n	-	-
r	-	discount rate
q	-	dividend yield
volatility	-	-
stock_price	-	-
discount_curve	-	-
dividend_curve	-	-
model	-	-
accrued_average	-	-
)	-	-
t_exp	-	-
tau	-	-
k	-	-
n	-	-
.opt_type	-	-
stock_price	-	-
r	-	-
q	-	-
volatility	-	-
num_paths	-	-
seed	-	-
accrued_average	-	-
v_g_exact	-	-
)	-	-
return v	-	-

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

4.3 equity_barrier_option

Class: EquityBarrierOption(EquityOption)

Class to hold details of an Equity Barrier Option. It also calculates the option price using Black Scholes for 8 different variants on the Barrier structure in enum BarrierTypes.

EquityBarrierOption

Create the EquityBarrierOption by specifying the expiry date, strike price, option type, barrier level, the number of observations per year and the notional.

```
EquityBarrierOption(expiry_dt: Date,
strike_price: float,
opt_type: BarrierTypes,
barrier_level: float,
num_obs_per_year: Union[int, float] = 252,
notional: float = 1.0,
"""Create the EquityBarrierOption by specifying the expiry date,
strike price, option type, barrier level, the number of observations
per year and the notional."""
check_argument_types(.EquityBarrierOption, locals())
.expiry_dt = expiry_dt
.strike_price = float(strike_price)
.barrier_level = float(barrier_level)
.num_obs_per_year = int(num_obs_per_year)
if opt_type not in BarrierTypes:
    raise FinError("Option_Type_" + str(opt_type) + "_unknown.")
.opt_type = opt_type
.notional = notional
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
strike_price	float	-	-
opt_type	BarrierTypes	-	-
barrier_level	float	-	-
num_obs_per_year	int or float	-	252
notional	float	-	1.0
"""Create the EquityBarrierOption by specifying the expiry date			
strike price	-	-	-
option type	-	-	-
barrier level	-	-	-
the number of observations	-	-	-
per year and the notional."""	-	-	-
check_argument_types(EquityBarrierOption, locals())	-	-	-
.expiry_dt	-	-	expiry_dt
.strike_price	-	-	float(strike_price)
.barrier_level	-	-	float(barrier_level)
.num_obs_per_year	-	-	int(num_obs_per_year)
if opt_type not in BarrierTypes	-	-	-
raise FinError("Option Type " + str(opt_type) + " unknown.")	-	-	-
.opt_type	-	-	opt_type
.notional	-	-	notional

value

PLEASE ADD A FUNCTION DESCRIPTION

```

value(value_dt: Date,
stock_price: Union[float, np.ndarray],
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""This_prices_an_Equity_Barrier_option_using_the_formulae_given_in
the_paper_by_Clewlow,_Llanos_and_Strickland_December_1994_which_can_be
found_at
https://warwick.ac.uk/fac/soc/wbs/subjects/finance/research/wpaperseries/1994/94-54.pdf
"""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
)
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date"
)
if isinstance(stock_price, int):

```

The function arguments are described in the following table.

Argument Name	Type
value_dt	Date
stock_price	float or np.ndarray
discount_curve	DiscountCurve
dividend_curve	DiscountCurve
model	-
"""This prices an Equity Barrier option using the formulae given in the paper by Clewlow Llanos and Strickland December 1994 which can be found at https://warwick.ac.uk/fac/soc/wbs/subjects/finance/research/ """	-
if isinstance(value_dt, Date) is False raise FinError("Valuation date is not a Date")	-
if value_dt > .expiry_dt raise FinError("Valuation date after expiry date.")	-
if discount_curve.value_dt !)	-
if dividend_curve.value_dt !)	-

value_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc(value_dt: Date,
stock_price: Union[float, np.ndarray],
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_obs_per_year=252,
num_paths: int = 10000,
seed: int = 42,
"""This prices an Equity Barrier option using the formulae given in
the paper by Clewlow, Llanos and Strickland December 1994 which can be
found at
https://warwick.ac.uk/fac/soc/wbs/subjects/finance/research/wpaperseries/1994/94-54.pdf
"""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation date is not a Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation date after expiry date.")
if isinstance(stock_price, int):

```

The function arguments are described in the following table.

Argument Name	Type
value_dt	Date
stock_price	float or np.ndarray
discount_curve	DiscountCurve
dividend_curve	DiscountCurve
model	-
num_obs_per_year	-
num_paths	int
seed	int
"""This prices an Equity Barrier option using the formulae given in the paper by Clewlow Llanos and Strickland December 1994 which can be found at https://warwick.ac.uk/fac/soc/wbs/subjects/finance/research/papers/clewlow_strickland_1994.pdf """	-
if isinstance(value_dt, Date) is False	-
raise FinError("Valuation date is not a Date")	-
if value_dt > .expiry_dt	-
raise FinError("Valuation date after expiry date.")	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

4.4 equity_basket_option

Class: EquityBasketOption

A EquityBasketOption is a contract to buy a put or a call option on an equally weighted portfolio of different stocks, each with its own price, volatility and dividend yield. An analytical and monte-carlo pricing model have been implemented for a European style option.

EquityBasketOption

Define the EquityBasket option by specifying its expiry date, its strike price, whether it is a put or call, and the number of underlying stocks in the basket.

```
EquityBasketOption(expiry_dt: Date,
strike_price: float,
opt_type: OptionTypes,
num_assets: int,
"""Define the EquityBasket option by specifying its expiry date,
its strike price, whether it is a put or call, and the number of
underlying stocks in the basket."""
check_argument_types(EquityBasketOption, locals())
.expiry_dt = expiry_dt
.strike_price = float(strike_price)
.opt_type = opt_type
.num_assets = num_assets
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
strike_price	float	-	-
opt_type	OptionTypes	-	-
num_assets	int	-	-
"""Define the EquityBasket option by specifying its expiry date	-	-	-
its strike price	-	-	-
whether it is a put or call	-	-	-
and the number of	-	-	-
underlying stocks in the basket."""	-	-	-
check_argument_types(EquityBasketOption, locals())	-	-	-
.expiry_dt	-	-	expiry_dt
.strike_price	-	-	float(strike_price)
.opt_type	-	-	opt_type
.num_assets	-	-	num_assets

_validate

PLEASE ADD A FUNCTION DESCRIPTION

```
_validate(stock_prices, dividend_yields, volatilities, correlations):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
stock_prices	-	-	-
dividend_yields	-	-	-
volatilities	-	-	-
correlations	-	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
stock_prices: np.ndarray,
discount_curve: DiscountCurve,
dividend_curves: list,
volatilities: np.ndarray,
correlations: np.ndarray,
    """Basket_valuation_using_a_moment_matching_method_to_approximate_the
effective_variance_of_the_underlying_basket_value._This_approach_is
able_to_handle_a_full_rank_correlation_structure_between_the_individual
assets."""
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
qs = []
for curve in dividend_curves:
    q = curve.cc_rate(.expiry_dt)
    qs.append(q)
v = volatilities
s = stock_prices
._validate(stock_prices, qs, volatilities, correlations)
a = np.ones(.num_assets) * (1.0 / .num_assets)
r = discount_curve.cc_rate(.expiry_dt)
smean = 0.0
for ia in range(0, .num_assets):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_prices	np.ndarray	-
discount_curve	DiscountCurve	-
dividend_curves	list	-
volatilities	np.ndarray	-
correlations	np.ndarray	-
"""Basket valuation using a moment matching method to approximate the effective variance of the underlying basket value. This approach is able to handle a full rank correlation structure between the individual assets."""		
t_exp	-	(.expiry_dt -
if value_dt > .expiry_dt	-	-
raise FinError("Value date after expiry date.")	-	-
qs	-	-
for curve in dividend_curves	-	-
q	-	cur
qs.append(q)	-	-
v	-	-
s	-	-
.validate(stock_prices, qs, volatilities, correlations)	-	-
a	-	np.ones(.num
r	-	discoun
smean	-	-

value_mc

Valuation of the EquityBasketOption using a Monte-Carlo simulation of stock prices assuming a GBM distribution. Cholesky decomposition is used to handle a full rank correlation structure between the individual assets. The num_paths and seed are pre-set to default values but can be overwritten.

```

value_mc(value_dt: Date,
stock_prices: np.ndarray,
discount_curve: DiscountCurve,
dividend_curves: list,
volatilities: np.ndarray,
corr_matrix: np.ndarray,
num_paths: int = 10000,
seed: int = 4242,
"""Valuation of the EquityBasketOption using a Monte-Carlo simulation
of stock prices assuming a GBM distribution. Cholesky decomposition is
used to handle a full rank correlation structure between the individual
assets. The num_paths and seed are pre-set to default values but can be
overwritten."""
check_argument_types(getattr(_func_name(), None), locals())
if value_dt > .expiry_dt:
    raise FinError("Value date after expiry date.")
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
dividend_yields = []

```

```

for curve in dividend_curves:
    dq = curve.df(.expiry_dt)
    q = -np.log(dq) / t_exp
    dividend_yields.append(q)
._validate(stock_prices, dividend_yields, volatilities, corr_matrix)
num_assets = len(stock_prices)
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
mus = r - dividend_yields
k = .strike_price
np.random.seed(seed)
t_all, s_all = get_assets_paths(num_assets,
    num_paths,
    t_exp,
    mus,
    stock_prices,
    volatilities,
    corr_matrix,
    seed,
)
if .opt_type == OptionTypes.EUROPEAN_CALL:
    payoff = np.maximum(np.mean(s_all, axis=0) - k, 0.0)
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    payoff = np.maximum(k - np.mean(s_all, axis=0), 0.0)
else:
    raise FinError("Unknown_option_type.")
payoff = np.mean(payoff)
v = payoff * np.exp(-r * t_exp)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_prices	np.ndarray	-
discount_curve	DiscountCurve	-
dividend_curves	list	-
volatilities	np.ndarray	-
corr_matrix	np.ndarray	-
num_paths	int	-
seed	int	-
"""Valuation of the EquityBasketOption using a Monte-Carlo simulation of stock prices assuming a GBM distribution. Cholesky decomposition is used to handle a full rank correlation structure between the individual assets. The num_paths and seed are pre-set to default values but can be overwritten."""	-	-
check_argument_types(getattr(_func_name(), None), locals())	-	-
if value_dt < .expiry_dt	-	-
raise FinError("Value date after expiry date.")	-	-
t_exp	-	(.expiry_dt - v
dividend_yields	-	-
for curve in dividend_curves	-	-
dq	-	-
q	-	-
dividend_yields.append(q)	-	-
._validate(stock_prices, dividend_yields, volatilities, corr_matrix)	-	-
num_assets	-	-
df	-	discount
r	-	-
mus	-	-
k	-	-
np.random.seed(seed)	-	-
t_all	-	-
num_paths	-	-
t_exp	-	-
mus	-	-
stock_prices	-	-
volatilities	-	-
corr_matrix	-	-
seed	-	-
)	-	-
if .opt_type	-	= Option5
payoff	-	np.maximum(
elif .opt_type	-	= Option
payoff	-	np.maximum(
else	-	-
raise FinError("Unknown option type.")	-	-
payoff	-	-
v	-	payo
return v	-	-

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

Enumerated Type: *EquityTreePayoffTypes*

- FWD_CONTRACT
- VANILLA_OPTION
- DIGITAL_OPTION
- POWER_CONTRACT
- POWER_OPTION
- LOG_CONTRACT
- LOG_OPTION

This enumerated type has the following values:

- EUROPEAN
- AMERICAN
- num_params
- ifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- iflen(payoff_params)!
- @njit(float64(float64,int64,float64[:]),fastmath
- ifpayoff_type
- payoff
- elifpayoff_type
- payoff
- elifpayoff_type
- payoff
- elifpayoff_type
- payoff
- elifpayoff_type
- payoff
- elifpayoff_type

- payoff
- elifpayoff_type
- payoff
- @njit(fastmath
- num_steps
- payoff_type_value
- dt
- num_nodes
- stock_values
- stock_values[0]
- option_values
- u
- d
- s_low
- probs
- period_dfs
- a
- probs[i_time]
- period_dfs[i_time]
- s_low*
- s
- index
- stock_values[int(index+i_node)]
- s
- index
- s
- option_values[index+i_node]
- index
- next_index
- next_node_dn
- next_node_up
- v_up
- v_dn
- future_expected_value
- future_expected_value+
- hold_value
- ifexercise_type
- option_values[index+i_node]
- elifexercise_type
- s
- exercise_value
- option_values[index+i_node]
- price
- delta
- delta_up
- delta_dn

- gamma
- theta
- results
- self.m_option_values
- self.m_stock_values
- self.m_upProbabilities
- self.m_num_steps
- self.m_num_nodes
- t_exp
- r
- dq
- q
- price1
- price2
- price

Class: EquityBinomialTree

class EquityBinomialTree:

EquityBinomialTree

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityBinomialTree():
```

The function arguments are described in the following table.

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(stock_price,
discount_curve,
dividend_curve,
volatility,
num_steps,
value_dt,
payoff,
expiry_dt,
payoff_type,
exercise_type,
payoff_params,
# do some validation
t_exp = (expiry_dt - value_dt) / G_DAYS_IN_YEAR
r = discount_curve.zero_rate(expiry_dt)
dq = dividend_curve.df(expiry_dt)
q = -np.log(dq) / t_exp
price1 = _value_once(stock_price,
r,
```

```
    q,
    volatility,
    num_steps,
    t_exp,
    payoff_type,
    exercise_type,
    payoff_params,
)
# Can I reuse the same tree ?
price2 = _value_once(stock_price,
    r,
    q,
    volatility,
    num_steps + 1,
    t_exp,
    payoff_type,
    exercise_type,
    payoff_params,
)
price = (price1 + price2) / 2.0
return price
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
stock_price	-	-	-
discount_curve	-	-	-
dividend_curve	-	-	-
volatility	-	-	-
num_steps	-	-	-
value_dt	-	-	-
payoff	-	-	-
expiry_dt	-	-	-
payoff_type	-	-	-
exercise_type	-	-	-
payoff_params	-	-	-
t_exp	-	(expiry_dt - value_dt) / G.DAYS_IN_YEAR	-
r	-	discount_curve.zero_rate(expiry_dt)	-
dq	-	dividend_curve.df(expiry_dt)	-
q	-	-np.log(dq) / t_exp	-
r	-	-	-
q	-	-	-
volatility	-	-	-
num_steps	-	-	-
t_exp	-	-	-
payoff_type	-	-	-
exercise_type	-	-	-
payoff_params	-	-	-
)	-	-	-
r	-	-	-
q	-	-	-
volatility	-	-	-
num_steps + 1	-	-	-
t_exp	-	-	-
payoff_type	-	-	-
exercise_type	-	-	-
payoff_params	-	-	-
)	-	-	-
price	-	(price1 + price2) / 2.0	-
return price	-	-	-

`_validate_payoff`

PLEASE ADD A FUNCTION DESCRIPTION

```
_validate_payoff(payoff_type, payoff_params):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
payoff_type	-	-	-
payoff_params	-	-	-

`_payoff_value`*PLEASE ADD A FUNCTION DESCRIPTION*

```
_payoff_value(s, payoff_type, payoff_params):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
payoff_type	-	-	-
payoff_params	-	-	-

`_value_once`*PLEASE ADD A FUNCTION DESCRIPTION*

```
_value_once(
stock_price,
r,
q,
volatility,
num_steps,
time_to_expiry,
payoff_type,
exercise_type,
payoff_params,
num_steps = max(num_steps, 3)
# validate_payoff(payoff_type.value, payoff_params)
payoff_type_value = payoff_type.value
# this is the size of the step
dt = time_to_expiry / num_steps
# the number of nodes on the tree
num_nodes = int(0.5 * (num_steps + 1) * (num_steps + 2))
stock_values = np.zeros(num_nodes)
stock_values[0] = stock_price
option_values = np.zeros(num_nodes)
u = exp(volatility * sqrt(dt))
d = 1.0 / u
s_low = stock_price
probs = np.zeros(num_steps)
period_dfs = np.zeros(num_steps)
# store time independent information for later use in tree
for i_time in range(0, num_steps):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
stock_price	-	-	-
r	-	-	-
q	-	-	-
volatility	-	-	-
num_steps	-	-	-
time_to_expiry	-	-	-
payoff_type	-	-	-
exercise_type	-	-	-
payoff_params	-	-	-
num_steps	-	-	max(num_steps, 3)
payoff_type_value	-	-	payoff_type.value
dt	-	-	time_to_expiry / num_steps
num_nodes	-	-	int(0.5 * (num_steps + 1) * (num_steps + 2))
stock_values	-	-	np.zeros(num_nodes)
stock_values[0]	-	-	stock_price
option_values	-	-	np.zeros(num_nodes)
u	-	-	exp(volatility * sqrt(dt))
d	-	-	1.0 / u
s_low	-	-	stock_price
probs	-	-	np.zeros(num_steps)
period_dfs	-	-	np.zeros(num_steps)

4.6 equity_chooser_option

Class: EquityChooserOption(EquityOption)

A EquityChooserOption is an option which allows the holder to either enter into a call or a put option on a later expiry date, with both strikes potentially different and both expiry dates potentially different. This is known as a complex chooser. All the option details are set at trade initiation.

EquityChooserOption

Create the EquityChooserOption by passing in the chooser date and then the put and call expiry dates as well as the corresponding put and call strike prices.

```
EquityChooserOption(choose_dt: Date,
call_expiry_dt: Date,
put_expiry_dt: Date,
call_strike_price: float,
put_strike_price: float,
"""Create the EquityChooserOption by passing in the chooser date
and then the put and call expiry dates as well as the corresponding put
and call strike prices."""
check_argument_types(.EquityChooserOption, locals())
if choose_dt > call_expiry_dt:
    raise FinError("Expiry_date_must_precede_call_option_expiry_date")
if choose_dt > put_expiry_dt:
    raise FinError("Expiry_date_must_precede_put_option_expiry_date")
.choose_dt = choose_dt
.call_expiry_dt = call_expiry_dt
.put_expiry_dt = put_expiry_dt
.call_strike = float(call_strike_price)
.put_strike = float(put_strike_price)
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
choose_dt	Date	-	-
call_expiry_dt	Date	-	-
put_expiry_dt	Date	-	-
call_strike_price	float	-	-
put_strike_price	float	-	-
"""Create the EquityChooserOption by passing in the chooser date			
and then the put and call expiry dates as well as the corresponding put			
and call strike prices."""			
check_argument_types(EquityChooserOption, locals())	-	-	-
if choose_dt < call_expiry_dt	-	-	-
raise FinError("Expiry date must precede call option expiry date")	-	-	-
if choose_dt < put_expiry_dt	-	-	-
raise FinError("Expiry date must precede put option expiry date")	-	-	-
.choose_dt	-	-	choose_dt
.call_expiry_dt	-	-	call_expiry_dt
.put_expiry_dt	-	-	put_expiry_dt
.call_strike	-	-	float(call_strike_price)
.put_strike	-	-	float(put_strike_price)

value

Value the complex chooser option using an approach by Rubinstein (1991). See also Haug page 129 for complex chooser options.

```

value(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Value the complex chooser option using an approach by Rubinstein
(1991). See also Haug page 129 for complex chooser options."""
if value_dt > .choose_dt:
    raise FinError("Value date after choose date.")
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation date is not a Date")
if value_dt > .call_expiry_dt:
    raise FinError("Valuation date after call expiry date.")
if value_dt > .put_expiry_dt:
    raise FinError("Valuation date after put expiry date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount Curve valuation date not same as option value date"
)
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend Curve valuation date not same as option value date"
)
t = (.choose_dt - value_dt) / G_DAYS_IN_YEAR
tc = (.call_expiry_dt - value_dt) / G_DAYS_IN_YEAR
tp = (.put_expiry_dt - value_dt) / G_DAYS_IN_YEAR
rt = discount_curve.cc_rate(.choose_dt)
rtc = discount_curve.cc_rate(.call_expiry_dt)

```

```

rtp = discount_curve.cc_rate(.put_expiry_dt)
q = dividend_curve.cc_rate(.choose_dt)
t = max(t, G_SMALL)
tc = max(tc, G_SMALL)
tp = max(tp, G_SMALL)
v = model.volatility
v = max(v, G_SMALL)
s0 = stock_price
xc = .call_strike
xp = .put_strike
bt = rt - q
btc = rtc - q
btp = rtp - q
argtuple = (t, tc, tp, rtc, rtp, xc, xp, v, q)
if DEBUG_MODE:
    print("args", argtuple)
istar = optimize.newton(_f, x0=s0, args=argtuple, tol=1e-8, maxiter=50, fprime2=None)
)
if DEBUG_MODE:
    print("istar", istar)
d1 = (np.log(s0 / istar) + (bt + v * v / 2) * t) / v / np.sqrt(t)
d2 = d1 - v * np.sqrt(t)
if DEBUG_MODE:
    print("d1", d1)
    print("d2", d2)
y1 = (np.log(s0 / xc) + (btc + v * v / 2) * tc) / v / np.sqrt(tc)
y2 = (np.log(s0 / xp) + (btp + v * v / 2) * tp) / v / np.sqrt(tp)
if DEBUG_MODE:
    print("y1", y1)
    print("y2", y2)
rho1 = np.sqrt(t / tc)
rho2 = np.sqrt(t / tp)
if DEBUG_MODE:
    print("rho1", rho1)
    print("rho2", rho2)
w = s0 * np.exp(-q * tc) * M(d1, y1, rho1)
w = w - xc * np.exp(-rtc * tc) * M(d2, y1 - v * np.sqrt(tc), rho1)
w = w - s0 * np.exp(-q * tp) * M(-d1, -y2, rho2)
w = w + xp * np.exp(-rtp * tp) * M(-d2, -y2 + v * np.sqrt(tp), rho2)
return w
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Value the complex chooser option using an approach by Rubinstein (1991). See also Haug page 129 for complex chooser options."""	-	-
if value_dt < choose_dt	-	-
raise FinError("Value date after choose date.")	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt < call_expiry_dt	-	-
raise FinError("Valuation date after call expiry date.")	-	-
if value_dt < put_expiry_dt	-	-
raise FinError("Valuation date after put expiry date.")	-	-
if discount_curve.value_dt !	-	-
)	-	-
if dividend_curve.value_dt !	-	-
)	-	-
t	-	(.choos
tc	-	(.call_exp
tp	-	(.put_exp
rt	-	di
rtc	-	disc
rtp	-	disc
q	-	di
t	-	-
tc	-	-
tp	-	-
v	-	-
v	-	-
s0	-	-
xc	-	-
xp	-	-
bt	-	-
btc	-	-
btp	-	-
argtuple	-	-
if DEBUG_MODE	-	-
print("args", argtuple)	-	-
)	-	-
if DEBUG_MODE	-	-
print("istar", istar)	-	-
d1	-	(np.log(s0 /
d2	-	-
if DEBUG_MODE	-	-
print("d1", d1)	-	-
print("d2", d2)	-	-
y1	-	(np.log(s0 /
y2	-	(np.log(s0 /
if DEBUG_MODE	-	-

value_mc

Value the complex chooser option Monte Carlo.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_paths: int = 10000,
seed: int = 4242,
"""Value the complex chooser option Monte Carlo."""
dft = discount_curve.df(.choose_dt)
dftc = discount_curve.df(.call_expiry_dt)
dftp = discount_curve.df(.put_expiry_dt)
t = (.choose_dt - value_dt) / G_DAYS_IN_YEAR
tc = (.call_expiry_dt - value_dt) / G_DAYS_IN_YEAR
tp = (.put_expiry_dt - value_dt) / G_DAYS_IN_YEAR
rt = -np.log(dft) / t
rtc = -np.log(dftc) / tc
rtp = -np.log(dftp) / tp
t = max(t, 1e-6)
tc = max(tc, 1e-6)
tp = max(tp, 1e-6)
v = model.volatility
v = max(v, 1e-6)
# SHOULD THIS CARE ABOUT TERM STRUCTURE OF Q
dq = dividend_curve.df(.choose_dt)
q = -np.log(dq) / t
#         q = dividend_yield
kc = .call_strike
kp = .put_strike
np.random.seed(seed)
sqrt_dt = np.sqrt(t)
# Use Antithetic variables
g = np.random.normal(0.0, 1.0, size=(1, num_paths))
s = stock_price * np.exp((rt - q - v * v / 2.0) * t)
m = np.exp(g * sqrt_dt * v)
s_1 = s * m
s_2 = s / m
v_call_1 = bs_value(s_1, tc - t, kc, rtc, q, v, OptionTypes.EUROPEAN_CALL.value)
v_put_1 = bs_value(s_1, tp - t, kp, rtp, q, v, OptionTypes.EUROPEAN_PUT.value)
v_call_2 = bs_value(s_2, tc - t, kc, rtc, q, v, OptionTypes.EUROPEAN_CALL.value)
v_put_2 = bs_value(s_2, tp - t, kp, rtp, q, v, OptionTypes.EUROPEAN_PUT.value)
payoff_1 = np.maximum(v_call_1, v_put_1)
payoff_2 = np.maximum(v_call_2, v_put_2)
payoff = np.mean(payoff_1) + np.mean(payoff_2)
v = payoff * dft / 2.0
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	-	-	-
num_paths	int	-	-
seed	int	-	-
"""Value the complex chooser option Monte Carlo."""			
dft	-	-	discount_curve
dftc	-	-	discount_curve
dftp	-	-	discount_curve
t	-	-	(.choose_dt - value_dt)
tc	-	-	(.call_expiry_dt - value_dt)
tp	-	-	(.put_expiry_dt - value_dt)
rt	-	-	-np.exp(-rt)
rtc	-	-	-np.exp(-rtc)
rtp	-	-	-np.exp(-rtp)
t	-	-	max(t, 0)
tc	-	-	max(tc, 0)
tp	-	-	max(tp, 0)
v	-	-	model
v	-	-	model
dq	-	-	dividend_curve
q	-	-	-np.exp(-q)
kc	-	-	.call
kp	-	-	.put
np.random.seed(seed)	-	-	-
sqrt_dt	-	-	np.sqrt(dt)
g	-	-	np.random.normal(0, 1)
s	-	-	stock_price * np.exp(-q * t)
m	-	-	np.exp(-m * t)
s_1	-	-	s
s_2	-	-	s
v_call_1	-	-	bs_value(s_1, tc - t, kc, rtc, q, v, C)
v_put_1	-	-	bs_value(s_1, tp - t, kp, rtp, q, v, C)
v_call_2	-	-	bs_value(s_2, tc - t, kc, rtc, q, v, C)
v_put_2	-	-	bs_value(s_2, tp - t, kp, rtp, q, v, C)
payoff_1	-	-	np.maximum(v_call_1, 0)
payoff_2	-	-	np.maximum(v_put_2, 0)
payoff	-	-	np.mean(payoff_1, payoff_2)
v	-	-	payoff
return v	-	-	-

__repr__*PLEASE ADD A FUNCTION DESCRIPTION*`__repr__():`

The function arguments are described in the following table.

_print*Simple print function for backward compatibility.*`_print():`

The function arguments are described in the following table.

f*Complex chooser option solve for critical stock price that makes the forward starting call and put options have the same price on the chooser date.*`_f(ss, *args):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ss	-	-	-
*args	-	-	-

4.7 equity_cliquet_option

Class: EquityCliquetOption(EquityOption)

A EquityCliquetOption is a series of options which start and stop at successive times with each subsequent option resetting its strike to be ATM at the start of its life. This is also known as a reset option.

EquityCliquetOption

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityCliquetOption(start_dt: Date,
final_expiry_dt: Date,
opt_type: OptionTypes,
freq_type: FrequencyTypes,
dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create the EquityCliquetOption by passing in the start date
and the end date and whether it is a call or a put. Some additional
data is needed in order to calculate the individual payments."""
check_argument_types(.EquityCliquetOption, locals())
if (opt_type != OptionTypes.EUROPEAN_CALL
    and opt_type != OptionTypes.EUROPEAN_PUT
):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
start_dt	Date	-	
final_expiry_dt	Date	-	
opt_type	OptionTypes	-	
freq_type	FrequencyTypes	-	
dc_type	DayCountTypes	-	DayCountTypes.THIRTY_E_360
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
"""Create the EquityCliquetOption by passing in the start date and the end date and whether it is a call or a put. Some additional data is needed in order to calculate the individual payments."""	-	-	
check_argument_types(.EquityCliquetOption, locals())	-	-	
and opt_type !=	-	-	OptionTypes.EUROPEAN_CALL OptionTypes.EUROPEAN_PUT

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
```

```

stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
"""Value_the_cliquet_option_as_a_sequence_of_options_using_the_Black-
Scholes_model."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
    )
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date"
    )
if value_dt > .final_expiry_dt:
    raise FinError("Value_date_after_final_expiry_date.")
s = stock_price
v_cliquet = 0.0
.v_options = []
._dfs = []
.actual_dts = []
call_type = OptionTypes.EUROPEAN_CALL
put_type = OptionTypes.EUROPEAN_PUT
if isinstance(model, BlackScholes):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Value the cliquet option as a sequence of options using the Black-Scholes model."""	-	-	-
if isinstance(value_dt, Date) is False	-	-	-
raise FinError("Valuation date is not a Date")	-	-	-
if discount_curve.value_dt !	-	-	value
)	-	-	-
if dividend_curve.value_dt !	-	-	value
)	-	-	-
if value_dt > .final_expiry_dt	-	-	-
raise FinError("Value date after final expiry date.")	-	-	-
s	-	-	stock_
v_cliquet	-	-	0.0
.v_options	-	-	[]
.dfs	-	-	[]
.actual_dts	-	-	[]
call_type	-	-	OptionTypes.EUR
put_type	-	-	OptionTypes.EU

print_payments

PLEASE ADD A FUNCTION DESCRIPTION

```
print_payments() :
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() :
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print() :
```

The function arguments are described in the following table.

4.8 equity_compound_option

Class: EquityCompoundOption(EquityOption)

A EquityCompoundOption is an option which allows the holder to either buy or sell another underlying option on a first expiry date that itself expires on a second expiry date. Both strikes are set at trade initiation.

EquityCompoundOption

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityCompoundOption(c_expiry_dt: Date, # Compound Option expiry date
c_opt_type: OptionTypes, # Compound option type
c_strike_price: float, # Compound option strike
u_expiry_dt: Date, # Underlying option expiry date
u_opt_type: OptionTypes, # Underlying option type
u_strike_price: float,
# Underlying option strike price
"""Create the EquityCompoundOption by passing in the first and
second expiry dates as well as the corresponding strike prices and
option types."""
check_argument_types(.EquityCompoundOption, locals())
if c_expiry_dt > u_expiry_dt:
    raise FinError("Compound expiry date must precede underlying expiry
date")
if c_opt_type not in (OptionTypes.EUROPEAN_CALL,
OptionTypes.AMERICAN_CALL,
OptionTypes.EUROPEAN_PUT,
OptionTypes.AMERICAN_PUT,
):
```

The function arguments are described in the following table.

Argument Name	Type	Description
c_expiry_dt	Date	Compound Option expiry date
c_opt_type	OptionTypes	Compound option type
c_strike_price	float	Compound option strike price
u_expiry_dt	Date	Underlying option expiry date
u_opt_type	OptionTypes	Underlying option type
u_strike_price	float	-
"""Create the EquityCompoundOption by passing in the first and second expiry dates as well as the corresponding strike prices and option types."""	-	-
check_argument_types(.EquityCompoundOption, locals())	-	-
if c_expiry_dt > u_expiry_dt	-	-
raise FinError("Compound expiry date must precede underlying expiry date")	-	-
OptionTypes.AMERICAN_CALL	-	-
OptionTypes.EUROPEAN_PUT	-	-
OptionTypes.AMERICAN_PUT	-	-

_preprocess_inputs

PLEASE ADD A FUNCTION DESCRIPTION

```

_preprocess_inputs(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Validate inputs and compute (tc, tu, kc, ku, ru, qu, vol)."""
if not isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	-	-	-
"""Validate inputs and compute (tc, tu, kc, ku, ru, qu, vol)."""	-	-	-

value

Value the compound option using an analytical approach if it is entirely European style. Otherwise use a Tree approach to handle the early exercise. Solution by Geske (1977), Hodges and Selby (1987) and Rubinstein (1991). See also Haug page 132.

```

value(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_steps: int = 200,
"""Value the compound option using an analytical approach if it is
entirely European style. Otherwise use a Tree approach to handle the
early exercise. Solution by Geske (1977), Hodges and Selby (1987) and
Rubinstein (1991). See also Haug page 132."""
tc, tu, kc, ku, ru, qu, vol = _preprocess_inputs(value_dt, stock_price, discount_curve
, dividend_curve, model
)
v = equity_compound_option_bs(.c_opt_type,
.u_opt_type,
tc,
tu,
kc,
ku,
stock_price,
ru,
qu,
vol,

```

```

    num_steps,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	-	-	-
num_steps	int	-	200
"""Value the compound option using an analytical approach if it is entirely European style. Otherwise use a Tree approach to handle the early exercise. Solution by Geske (1977) Hodges and Selby (1987) and Rubinstein (1991). See also Haug page 132."""	-	-	-
tc	-	-	-
tu	-	-	-
kc	-	-	-
ku	-	-	-
ru	-	-	-
qu	-	-	-
)	-	-	-
.u_opt_type	-	-	-
tc	-	-	-
tu	-	-	-
kc	-	-	-
ku	-	-	-
stock_price	-	-	-
ru	-	-	-
qu	-	-	-
vol	-	-	-
num_steps	-	-	-
)	-	-	-
return v	-	-	-

value_tree

Value the compound option using a Tree approach to handle the early exercise. Solution by Geske (1977), Hodges and Selby (1987) and Rubinstein (1991). See also Haug page 132.

```
value_tree(value_dt: Date,
```

```

stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_steps: int = 200,
"""Value the compound option using a Tree approach to handle the
early exercise. Solution by Geske (1977), Hodges and Selby (1987) and
Rubinstein (1991). See also Haug page 132."""
tc, tu, kc, ku, ru, qu, vol = _preprocess_inputs(value_dt, stock_price, discount_curve
, dividend_curve, model
)
v = equity_compound_option_value_tree(.c_opt_type,
    .u_opt_type,
    tc,
    tu,
    kc,
    ku,
    stock_price,
    ru,
    qu,
    vol,
    num_steps,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	-	-	-
num_steps	int	-	200
"""Value the compound option using a Tree approach to handle the		-	-
early exercise. Solution by Geske (1977)		-	-
Hodges and Selby (1987) and		-	-
Rubinstein (1991). See also Haug page 132."""		-	-
tc	-	-	-
tu	-	-	-
kc	-	-	-
ku	-	-	-
ru	-	-	-
qu	-	-	-
)	-	-	-
.u_opt_type	-	-	-
tc	-	-	-
tu	-	-	-
kc	-	-	-
ku	-	-	-
stock_price	-	-	-
ru	-	-	-
qu	-	-	-
vol	-	-	-
num_steps	-	-	-
)	-	-	-
return v	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

4.9 equity_digital_option

Enumerated Type: FinDigitalOptionTypes

This enumerated type has the following values:

- CASH_OR_NOTHING
- ASSET_OR_NOTHING
- self.expiry_dt
- self.barrier
- self.call_put_type
- self.digital_type
- ifdiscount_curve.value_dt!
- ifdividend_curve.value_dt!
- t
- t
- s0
- x
- ln_s0_k
- sqrt_t
- df
- r
- dq
- q
- volatility
- volatility
- d1
- d1
- d2
- v
- ifself.digital_type
- ifself.call_put_type
- v
- elifself.call_put_type
- v
- elifself.digital_type
- ifself.call_put_type
- v
- elifself.call_put_type
- v
- num_paths:int
- seed:int
- t
- df
- r
- dq
- q

- volatility
- k
- sqrt_dt
- g
- s
- m
- s_1
- s_2
- payoff_a_1
- payoff_a_2
- ifself.digital_type
- ifself.call_put_type
- payoff_a_1
- payoff_a_2
- elifself.call_put_type
- payoff_a_1
- payoff_a_2
- elifself.digital_type
- ifself.call_put_type
- payoff_a_1
- payoff_a_2
- elifself.call_put_type
- payoff_a_1
- payoff_a_2
- payoff
- v
- s
- s+
- s+
- s+
- s+

Class: EquityDigitalOption(EquityOption)

A EquityDigitalOption is an option in which the buyer receives some payment if the stock price has crossed a barrier ONLY at expiry and zero otherwise. There are two types: cash-or-nothing and the asset-or-nothing option. We do not care whether the stock price has crossed the barrier today, we only care about the barrier at option expiry. For a continuously- monitored barrier, use the EquityOneTouchOption class.

EquityDigitalOption

Create the digital option by specifying the expiry date, the barrier price and the type of option which is either a EUROPEAN_CALL or a EUROPEAN_PUT or an AMERICAN_CALL or AMERICAN_PUT. There are two types of underlying - cash or nothing and asset or nothing.

```
EquityDigitalOption(expiry_dt: Date,
                    barrier: float,
```

```

call_put_type: OptionTypes,
digital_type: FinDigitalOptionTypes,
"""Create the digital option by specifying the expiry date, the
barrier price and the type of option which is either a EUROPEAN_CALL
or a EUROPEAN_PUT or an AMERICAN_CALL or AMERICAN_PUT. There are two
types of underlying - cash or nothing and asset or nothing."""
check_argument_types(.EquityDigitalOption, locals())
if call_put_type not in [
    OptionTypes.EUROPEAN_CALL,
    OptionTypes.EUROPEAN_PUT,
]:
    raise FinError("Option type must be EUROPEAN_CALL or PUT")
.expiry_dt = expiry_dt
.barrier = float(barrier)
.call_put_type = call_put_type
.digital_type = digital_type
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
barrier	float	-
call_put_type	OptionTypes	-
digital_type	FinDigitalOptionTypes	-
"""Create the digital option by specifying the expiry date	-	-
the	-	-
barrier price and the type of option which is either a EUROPEAN_CALL	-	-
or a EUROPEAN_PUT or an AMERICAN_CALL or AMERICAN_PUT. There are two	-	-
types of underlying - cash or nothing and asset or nothing."""	-	-
check_argument_types(.EquityDigitalOption, locals())	-	-
OptionTypes.EUROPEAN_CALL	-	-
OptionTypes.EUROPEAN_PUT	-	-
]	-	-
raise FinError("Option type must be EUROPEAN CALL or PUT")	-	-
.expiry_dt	-	-
.barrier	-	-
.call_put_type	-	-
.digital_type	-	-

value

Digital Option valuation using the Black-Scholes model assuming a barrier at expiry. Handles both cash-or-nothing and asset-or-nothing options.

```

value(value_dt: Date,
stock_price: Union[float, np.ndarray],
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,

```



```

model,
"""Digital_Option_valuation_using_the_Black-Scholes_model_assuming_a
barrier_at_expiry._Handles_both_cash-or-nothing_and_asset-or-nothing
options."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
    )
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date"
    )
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
t = max(t, 1e-6)
s0 = stock_price
x = .barrier
ln_s0_k = np.log(s0 / x)
sqrt_t = np.sqrt(t)
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t
volatility = model.volatility
if abs(volatility) < G_SMALL:
    volatility = G_SMALL
d1 = ln_s0_k + (r - q + volatility * volatility / 2.0) * t
d1 = d1 / volatility / sqrt_t
d2 = d1 - volatility * sqrt_t
v = None
if .digital_type == FinDigitalOptionTypes.CASH_OR_NOTHING:
    if .call_put_type == OptionTypes.EUROPEAN_CALL:
        v = np.exp(-r * t) * normcdf_vect(d2)
    elif .call_put_type == OptionTypes.EUROPEAN_PUT:
        v = np.exp(-r * t) * normcdf_vect(-d2)
elif .digital_type == FinDigitalOptionTypes.ASSET_OR_NOTHING:
    if .call_put_type == OptionTypes.EUROPEAN_CALL:
        v = s0 * np.exp(-q * t) * normcdf_vect(d1)
    elif .call_put_type == OptionTypes.EUROPEAN_PUT:
        v = s0 * np.exp(-q * t) * normcdf_vect(-d1)
else:
    raise FinError("Unknown_underlying_type.")
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float or np.ndarray	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Digital Option valuation using the Black-Scholes model assuming a barrier at expiry. Handles both cash-or-nothing and asset-or-nothing options."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > .expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if discount_curve.value_dt !=	-	-
)	-	-
if dividend_curve.value_dt !=	-	-
)	-	-
t	-	(.expiry_dt)
t	-	-
s0	-	-
x	-	-
ln_s0_k	-	-
sqrt_t	-	-
df	-	df
r	-	-
dq	-	df
q	-	-
volatility	-	-
if abs(volatility) > G_SMALL	-	-
volatility	-	-
d1	-	ln_s0_k + (
d1	-	-
d2	-	-
v	-	-
if .digital_type	-	= FinDigital
if .call_put_type	-	= Opt
v	-	np.e
elif .call_put_type	-	= Opt
v	-	np.e
elif .digital_type	-	= FinDigital
if .call_put_type	-	= Opt
v	-	s0 * np
elif .call_put_type	-	= Opt
v	-	s0 * np
else	-	-
raise FinError("Unknown underlying type.")	-	-
return v	-	-

value_mc

Digital Option valuation using the Black-Scholes model and Monte Carlo simulation. Product assumes a barrier only at expiry. Monte Carlo handles both a cash-or-nothing and an asset-or-nothing option.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_paths: int = 10000,
seed: int = 4242,
"""Digital Option valuation using the Black-Scholes model and Monte
Carlo simulation. Product assumes a barrier only at expiry. Monte Carlo
handles both a cash-or-nothing and an asset-or-nothing option."""
np.random.seed(seed)
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t
volatility = model.volatility
k = .barrier
sqrt_dt = np.sqrt(t)
# Use Antithetic variables
g = np.random.normal(0.0, 1.0, size=(1, num_paths))
s = stock_price * np.exp((r - q - volatility * volatility / 2.0) * t)
m = np.exp(g * sqrt_dt * volatility)
s_1 = s * m
s_2 = s / m
payoff_a_1 = None
payoff_a_2 = None
if .digital_type == FinDigitalOptionTypes.CASH_OR_NOTHING:
    if .call_put_type == OptionTypes.EUROPEAN_CALL:
        payoff_a_1 = np.heaviside(s_1 - k, 0.0)
        payoff_a_2 = np.heaviside(s_2 - k, 0.0)
    elif .call_put_type == OptionTypes.EUROPEAN_PUT:
        payoff_a_1 = np.heaviside(k - s_1, 0.0)
        payoff_a_2 = np.heaviside(k - s_2, 0.0)
elif .digital_type == FinDigitalOptionTypes.ASSET_OR_NOTHING:
    if .call_put_type == OptionTypes.EUROPEAN_CALL:
        payoff_a_1 = s_1 * np.heaviside(s_1 - k, 0.0)
        payoff_a_2 = s_2 * np.heaviside(s_2 - k, 0.0)
    elif .call_put_type == OptionTypes.EUROPEAN_PUT:
        payoff_a_1 = s_1 * np.heaviside(k - s_1, 0.0)
        payoff_a_2 = s_2 * np.heaviside(k - s_2, 0.0)
payoff = np.mean(payoff_a_1) + np.mean(payoff_a_2)
v = payoff * df / 2.0
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
num_paths	int	-
seed	int	-
"""Digital Option valuation using the Black-Scholes model and Monte Carlo simulation. Product assumes a barrier only at expiry. Monte Carlo handles both a cash-or-nothing and an asset-or-nothing option."""	-	-
np.random.seed(seed)	-	-
t	-	(expiry
df	-	-
r	-	-
dq	-	-
q	-	-
volatility	-	-
k	-	-
sqrt_dt	-	-
g	-	np.random
s	-	stock_price * n
m	-	n
s_1	-	-
s_2	-	-
payoff_a_1	-	-
payoff_a_2	-	-
if .digital_type	-	= FinDigit
if .call_put_type	-	= O
payoff_a_1	-	-
payoff_a_2	-	-
elif .call_put_type	-	= O
payoff_a_1	-	-
payoff_a_2	-	-
elif .digital_type	-	= FinDigit
if .call_put_type	-	= O
payoff_a_1	-	s
payoff_a_2	-	s
elif .call_put_type	-	= O
payoff_a_1	-	s
payoff_a_2	-	s
payoff	-	np.meas
v	-	-
return v	-	-

`--repr--`*PLEASE ADD A FUNCTION DESCRIPTION*`--repr__():`

The function arguments are described in the following table.

`_print`*Simple print function for backward compatibility.*`_print():`

The function arguments are described in the following table.

4.10 equity_fixed_lookback_option

Class: *EquityFixedLookbackOption(EquityOption)*

This is an equity option in which the strike of the option is fixed but the value of the stock price used to determine the payoff is the maximum in the case of a call option, and a minimum in the case of a put option.

EquityFixedLookbackOption

Create the FixedLookbackOption by specifying the expiry date, the option type and the option strike.

```
EquityFixedLookbackOption(expiry_dt: Date, opt_type: OptionTypes, strike_price: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
opt_type	OptionTypes	-	-
strike_price	float	-	-

value

Valuation of the Fixed Lookback option using Black-Scholes using the formulae derived by Conze and Viswanathan (1991). One of the inputs is the minimum of maximum of the stock price since the start of the option depending on whether the option is a call or a put.

```
value(value_dt: Date,
      stock_price: float,
      discount_curve: DiscountCurve,
      dividend_curve: DiscountCurve,
      volatility: float,
      stock_min_max: float,
      """Valuation of the Fixed Lookback option using Black-Scholes using
      the formulae derived by Conze and Viswanathan (1991). One of the inputs
      is the minimum of maximum of the stock price since the start of the
      option depending on whether the option is a call or a put."""
      if isinstance(value_dt, Date) is False:
          raise FinError("Valuation_date_is_not_a_Date")
      if value_dt > .expiry_dt:
          raise FinError("Valuation_date_after_expiry_date.")
      if discount_curve.value_dt != value_dt:
          raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date")
      )
      if dividend_curve.value_dt != value_dt:
          raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date")
      )
      t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
      df = discount_curve.df(.expiry_dt)
      r = -np.log(df) / t
      dq = dividend_curve.df(.expiry_dt)
      q = -np.log(dq) / t
      v = volatility
```

```

s0 = stock_price
k = .strike_price
s_min = 0.0
s_max = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    s_max = stock_min_max
    if s_max < s0:
        raise FinError("The_smax_value_must_be_>=the_stock_price.")
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    s_min = stock_min_max
    if s_min > s0:
        raise FinError("The_smin_value_must_be_<=the_stock_price.")
# There is a risk of an overflow in the limit of q=r which
# we remove by adjusting the value of the dividend
if abs(r - q) < G_SMALL:
    q = r + G_SMALL
df = np.exp(-r * t)
dq = np.exp(-q * t)
b = r - q
u = v * v / 2.0 / b
w = 2.0 * b / (v * v)
expbt = np.exp(b * t)
sqrt_t = np.sqrt(t)
# Taken from Hull Page 536 (6th edition) and Haug Page 143
if .opt_type == OptionTypes.EUROPEAN_CALL:
    if k > s_max:
        d1 = (np.log(s0 / k) + (b + v * v / 2.0) * t) / v / sqrt_t
        d2 = d1 - v * sqrt_t
        if s0 == k:
            term = -normcdf(d1 - 2.0 * b * sqrt_t / v) + expbt * normcdf(d1)
        elif s0 < k and w > 100.0:
            term = expbt * normcdf(d1)
        else:
            term = -np.power(s0 / k, -w) * normcdf(d1 - 2 * b * sqrt_t / v
                ) + expbt * normcdf(d1)
        v = s0 * dq * normcdf(d1) - k * df * normcdf(d2) + s0 * df * u * term
    else:
        e1 = (np.log(s0 / s_max) + (r - q + v * v / 2) * t) / v / sqrt_t
        e2 = e1 - v * sqrt_t
        if s0 == s_max:
            term = -normcdf(e1 - 2.0 * b * sqrt_t / v) + expbt * normcdf(e1)
        elif s0 < s_max and w > 100.0:
            term = expbt * normcdf(e1)
        else:
            term = (-((s0 / s_max) ** (-w))) * normcdf(e1 - 2.0 * b * sqrt_t / v
                ) + expbt * normcdf(e1)
        v = (df * (s_max - k)
            + s0 * dq * normcdf(e1)
            - s_max * df * normcdf(e2)
            + s0 * df * u * term
        )
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    if k >= s_min:
        f1 = (np.log(s0 / s_min) + (b + v * v / 2.0) * t) / v / sqrt_t
        f2 = f1 - v * sqrt_t
        if s0 == s_min:
            term = normcdf(-f1 + 2.0 * b * sqrt_t / v) - expbt * normcdf(-f1)

```

```

elif s0 > s_min and w < -100.0:
    term = -expbt * normcdf(-f1)
else:
    term = ((s0 / s_min) ** (-w)) * normcdf(-f1 + 2.0 * b * sqrt_t / v
    ) - expbt * normcdf(-f1)
v = (df * (k - s_min)
    - s0 * dq * normcdf(-f1)
    + s_min * df * normcdf(-f2)
    + s0 * df * u * term
    )
else:
    d1 = (np.log(s0 / k) + (b + v * v / 2) * t) / v / sqrt_t
    d2 = d1 - v * sqrt_t
    if s0 == k:
        term = normcdf(-d1 + 2.0 * b * sqrt_t / v) - expbt * normcdf(-d1)
    elif s0 > k and w < -100.0:
        term = -expbt * normcdf(-d1)
    else:
        term = ((s0 / k) ** (-w)) * normcdf(-d1 + 2.0 * b * sqrt_t / v
        ) - expbt * normcdf(-d1)
    v = k * df * normcdf(-d2) - s0 * dq * normcdf(-d1) + s0 * df * u * term
else:
    raise FinError("Unknown_lookback_option_type:" + str(.opt_type))
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
volatility	float	-
stock_min_max	float	-
"""Valuation of the Fixed Lookback option using Black-Scholes using the formulae derived by Conze and Viswanathan (1991). One of the inputs is the minimum of maximum of the stock price since the start of the option depending on whether the option is a call or a put."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if discount_curve.value_dt !=	-	-
)	-	-
if dividend_curve.value_dt !=	-	-
)	-	-
t	-	-
df	-	-
r	-	-
dq	-	-
q	-	-
v	-	-
s0	-	-
k	-	-
s_min	-	-
s_max	-	-
if .opt_type	-	-
s_max	-	-
if s_max < s0	-	-
raise FinError("The Smax value must be >=	-	-
elif .opt_type	-	-
s_min	-	-
if s_min < s0	-	-
raise FinError("The Smin value must be >=	-	-
if abs(r - q) > G_SMALL	-	-
q	-	-
df	-	-
dq	-	-
b	-	-
u	-	-
w	-	-
expbt	-	-
sqrt_t	-	-
if .opt_type	-	-
if k < s_max	-	-
d1	-	-
d2	-	-
if s0	-	-
term	-	-

value_mc

Monte Carlo valuation of a fixed strike lookback option using a Black-Scholes model that assumes the stock follows a GBM process.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
volatility: float,
stock_min_max: float,
num_paths: int = 10000,
num_steps_per_year: int = 252,
seed: int = 4242,
"""Monte Carlo valuation of a fixed strike lookback option using a
Black-Scholes model that assumes the stock follows a GBM process."""
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
mu = r - q
num_time_steps = int(t * num_steps_per_year)
opt_type = .opt_type
k = .strike_price
s_min = 0.0
s_max = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    s_max = stock_min_max
    if s_max < stock_price:
        raise FinError("Smax must be greater than or equal to the stock price.")
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    s_min = stock_min_max
    if s_min > stock_price:
        raise FinError("Smin must be less than or equal to the stock price.")
_, s_all = get_paths_times(num_paths, num_time_steps, t, mu, stock_price, volatility,
seed
)
payoff = np.zeros(num_paths)
if opt_type == OptionTypes.EUROPEAN_CALL:
    s_max_vector = np.max(s_all, axis=1)
    s_maxs = np.ones(num_paths) * s_max
    payoff = np.maximum(s_max_vector - k, 0.0)
    payoff = np.maximum(payoff, s_maxs - k)
elif opt_type == OptionTypes.EUROPEAN_PUT:
    s_minormcdf_vector = np.min(s_all, axis=1)
    s_mins = np.ones(num_paths) * s_min
    payoff = np.maximum(k - s_minormcdf_vector, 0.0)
    payoff = np.maximum(payoff, k - s_mins)
else:
    raise FinError("Unknown lookback option type:" + str(opt_type))
v = payoff.mean() * df
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
volatility	float	-
stock_min_max	float	-
num_paths	int	-
num_steps_per_year	int	-
seed	int	-
"""Monte Carlo valuation of a fixed strike lookback option using a Black-Scholes model that assumes the stock follows a GBM process."""	-	-
t	-	- (ex
df	-	-
r	-	-
q	-	-
mu	-	-
num_time_steps	-	-
opt_type	-	-
k	-	-
s_min	-	-
s_max	-	-
if .opt_type	-	-
s_max	-	-
if s_max > stock_price	-	-
raise FinError("Smax must be greater than or equal to the stock price.")	-	-
elif .opt_type	-	-
s_min	-	-
if s_min < stock_price	-	-
raise FinError("Smin must be less than or equal to the stock price.")	-	-
-	-	-
)	-	-
payoff	-	-
if opt_type	-	-
s_max_vector	-	-
s_maxs	-	-
payoff	-	-
payoff	-	-
elif opt_type	-	-
s_minormcdf_vector	-	-
s_mins	-	-
payoff	-	- np
payoff	-	-
else	-	-
raise FinError("Unknown lookback option type	opt_type) or "+str(opt_type	-
v	-	-
return v	-	-

`--repr--`*PLEASE ADD A FUNCTION DESCRIPTION*`--repr__():`

The function arguments are described in the following table.

`_print`*Simple print function for backward compatibility.*`_print():`

The function arguments are described in the following table.

4.11 equity_float_lookback_option

Class: *EquityFloatLookbackOption(EquityOption)*

This is an equity option in which the strike of the option is not fixed but is set at expiry to equal the minimum stock price in the case of a call or the maximum stock price in the case of a put. In other words the buyer of the call gets to buy the asset at the lowest price over the period before expiry while the buyer of the put gets to sell the asset at the highest price before expiry.

EquityFloatLookbackOption

Create the FloatLookbackOption by specifying the expiry date and the option type. The strike is determined internally as the maximum or minimum of the stock price depending on whether it is a put or a call option.

```
EquityFloatLookbackOption(expiry_dt: Date, opt_type: OptionTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
opt_type	OptionTypes	-	-

value

Valuation of the Floating Lookback option using Black-Scholes using the formulae derived by Goldman, Sosin and Gatto (1979).

```
value(value_dt: Date,
      stock_price: float,
      discount_curve: DiscountCurve,
      dividend_curve: DiscountCurve,
      volatility: float,
      stock_min_max: float,
      """Valuation_of_the_Floating_Lookback_option_using_Black-Scholes_using
the_formulae_derived_by_Goldman,_Sosin_and_Gatto_(1979)."""
      if isinstance(value_dt, Date) is False:
          raise FinError("Valuation_date_is_not_a_Date")
      if value_dt > .expiry_dt:
          raise FinError("Valuation_date_after_expiry_date.")
      if discount_curve.value_dt != value_dt:
          raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
                          )
      if dividend_curve.value_dt != value_dt:
          raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date"
                          )
      t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
      df = discount_curve.df(.expiry_dt)
      r = discount_curve.cc_rate(.expiry_dt)
      q = dividend_curve.cc_rate(.expiry_dt)
      v = volatility
      s0 = stock_price
      smin = 0.0
```

```

smax = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    smin = stock_min_max
    if smin > s0:
        raise FinError("Smin_must_be_less_than_or_equal_to_the_stock_price.")
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    smax = stock_min_max
    if smax < s0:
        raise FinError("Smax_must_be_greater_than_or_equal_to_the_stock_price.")
if abs(r - q) < G_SMALL:
    q = r + G_SMALL
dq = np.exp(-q * t)
df = np.exp(-r * t)
b = r - q
u = v * v / 2.0 / b
w = 2.0 * b / v / v
expbt = np.exp(b * t)
# Taken from Haug Page 142
if .opt_type == OptionTypes.EUROPEAN_CALL:
    a1 = (np.log(s0 / smin) + (b + (v**2) / 2.0) * t) / v / np.sqrt(t)
    a2 = a1 - v * np.sqrt(t)
    if smin == s0:
        term = normcdf(-a1 + 2.0 * b * np.sqrt(t) / v) - expbt * normcdf(-a1)
    elif s0 < smin and w < -100:
        term = -expbt * normcdf(-a1)
    else:
        term = ((s0 / smin) ** (-w)) * normcdf(-a1 + 2.0 * b * np.sqrt(t) / v)
        - expbt * normcdf(-a1)
    v = s0 * dq * normcdf(a1) - smin * df * normcdf(a2) + s0 * df * u * term
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    b1 = (np.log(s0 / smax) + (b + (v**2) / 2.0) * t) / v / np.sqrt(t)
    b2 = b1 - v * np.sqrt(t)
    if smax == s0:
        term = -normcdf(b1 - 2.0 * b * np.sqrt(t) / v) + expbt * normcdf(b1)
    elif s0 < smax and w > 100:
        term = expbt * normcdf(b1)
    else:
        term = (-((s0 / smax) ** (-w))) * normcdf(b1 - 2.0 * b * np.sqrt(t) / v)
        + expbt * normcdf(b1)
    v = smax * df * normcdf(-b2) - s0 * dq * normcdf(-b1) + s0 * df * u * term
else:
    raise FinError("Unknown_lookback_option_type:" + str(.opt_type))
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
volatility	float	-
stock_min_max	float	-
"""Valuation of the Floating Lookback option using Black-Scholes using the formulae derived by Goldman Sosin and Gatto (1979)."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt < .expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if discount_curve.value_dt !=	-	-
)	-	-
if dividend_curve.value_dt !=	-	-
)	-	-
t	-	-
df	-	-
r	-	-
q	-	-
v	-	-
s0	-	-
smin	-	-
smax	-	-
if .opt_type	-	-
smin	-	-
if smin > s0	-	-
raise FinError("Smin must be less than or equal to the stock price.")	-	-
elif .opt_type	-	-
smax	-	-
if smax < s0	-	-
raise FinError("Smax must be greater than or equal to the stock price.")	-	-
if abs(r - q) > G_SMALL	-	-
q	-	-
dq	-	-
df	-	-
b	-	-
u	-	-
w	-	-
expbt	-	-
if .opt_type	-	-
a1	-	-
a2	-	-
if smin	-	-
term	-	-
elif s0 < smin and w < -100	-	-
term	-	-
else	-	-
v	-	-
elif .opt_type	-	-

value_mc

Monte Carlo valuation of a floating strike lookback option using a Black-Scholes model that assumes the stock follows a GBM process.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
volatility: float,
stock_min_max: float,
num_paths: int = 10000,
num_steps_per_year: int = 252,
seed: int = 4242,
    """Monte Carlo valuation of a floating strike lookback option using a
    Black-Scholes model that assumes the stock follows a GBM process."""
    t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
    num_time_steps = int(t * num_steps_per_year)
    df = discount_curve.df(.expiry_dt)
    r = discount_curve.cc_rate(.expiry_dt)
    q = dividend_curve.cc_rate(.expiry_dt)
    mu = r - q
    opt_type = .opt_type
    smin = 0.0
    smax = 0.0
    if .opt_type == OptionTypes.EUROPEAN_CALL:
        smin = stock_min_max
        if smin > stock_price:
            raise FinError("Smin must be less than or equal to the stock price.")
    elif .opt_type == OptionTypes.EUROPEAN_PUT:
        smax = stock_min_max
        if smax < stock_price:
            raise FinError("Smax must be greater than or equal to the stock price.")
    t_all, s_all = get_paths_times(num_paths, num_time_steps, t, mu, stock_price,
        volatility, seed
    )
    # Due to antithetics we have doubled the number of paths
    payoff = np.zeros(num_paths)
    if opt_type == OptionTypes.EUROPEAN_CALL:
        s_min = np.min(s_all, axis=1)
        s_min = np.minimum(s_min, smin)
        payoff = np.maximum(s_all[:, -1] - s_min, 0.0)
    elif opt_type == OptionTypes.EUROPEAN_PUT:
        s_max = np.max(s_all, axis=1)
        s_max = np.maximum(s_max, smax)
        payoff = np.maximum(s_max - s_all[:, -1], 0.0)
    else:
        raise FinError("Unknown lookback option type:" + str(opt_type))
    v = payoff.mean() * df
    return v
    #####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
volatility	float	-
stock_min_max	float	-
num_paths	int	-
num_steps_per_year	int	-
seed	int	-
"""Monte Carlo valuation of a floating strike lookback option using a Black-Scholes model that assumes the stock follows a GBM process."""	-	-
t	-	- (ex
num_time_steps	-	-
df	-	-
r	-	-
q	-	-
mu	-	-
opt_type	-	-
smin	-	-
smax	-	-
if .opt_type	-	-
smin	-	-
if smin < stock_price	-	-
raise FinError("Smin must be less than or equal to the stock price.")	-	-
elif .opt_type	-	-
smax	-	-
if smax > stock_price	-	-
raise FinError("Smax must be greater than or equal to the stock price.")	-	-
t_all	-	-
)	-	-
payoff	-	-
if opt_type	-	-
s_min	-	-
s_min	-	-
payoff	-	-
elif opt_type	-	-
s_max	-	-
s_max	-	-
payoff	-	-
else	-	-
raise FinError("Unknown lookback option type	opt_type) or "+str(opt_type	-
v	-	-
return v	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

4.12 equity_forward

Class: EquityForward

EquityForward

Creates a EquityForward which allows the owner to buy the stock at a price agreed today. Need to specify if LONG or SHORT.

```
EquityForward(expiry_dt: Date,
forward_price: float, # PRICE OF 1 UNIT OF FOREIGN IN DOM CCY
notional: float,
long_short: FinLongShort = FinLongShort.LONG,
"""Creates_a_EquityForward_which_allows_the_owner_to_buy_the_stock
at_a_price_agreed_today._Need_to_specify_if_LONG_or_SHORT."""
check_argument_types(.EquityForward, locals())
.expiry_dt = expiry_dt
.forward_price = forward_price
.notional = notional
.long_short = long_short
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
forward_price	float	PRICE OF 1 UNIT OF FOREIGN
notional	float	-
long_short	FinLongShort	-
"""Creates a EquityForward which allows the owner to buy the stock at a price agreed today. Need to specify if LONG or SHORT."""	-	-
check_argument_types(.EquityForward, locals())	-	-
.expiry_dt	-	-
.forward_price	-	-
.notional	-	-
.long_short	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt,
stock_price, # Current stock price
discount_curve,
dividend_curve,
"""Calculate_the_value_of_an_equity_forward_contract_from_the_stock
price_and_discount_and_dividend_discount."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
```

```

    raise FinError("Valuation_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
    )
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_valuation_date_not_same_as_option_value_date"
    )
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_price	-	Current stock price	-
discount_curve	-	-	-
dividend_curve	-	-	-
"""Calculate the value of an equity forward contract from the stock	-	-	-
price and discount and dividend discount."""	-	-	-
if isinstance(value_dt, Date) is False	-	-	-
raise FinError("Valuation date is not a Date")	-	-	-
if value_dt > .expiry_dt	-	-	-
raise FinError("Valuation date after expiry date.")	-	-	-
if discount_curve.value_dt !=	-	-	value_dt:
)	-	-	-
if dividend_curve.value_dt !=	-	-	value_dt:
)	-	-	-

forward

PLEASE ADD A FUNCTION DESCRIPTION

```

forward(value_dt,
stock_price, # Current stock price
discount_curve,
dividend_curve,
"""Calculate the forward price of the equity forward contract."""
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_price	-	Current stock price	-
discount_curve	-	-	-
dividend_curve	-	-	-
"""Calculate the forward price of the equity forward contract."""	-	-	-

--repr--*PLEASE ADD A FUNCTION DESCRIPTION*`__repr__():`

The function arguments are described in the following table.

_print*Simple print function for backward compatibility.*`_print():`

The function arguments are described in the following table.

4.13 equity_index_option

Class: *EquityIndexOption*

Class for managing plain vanilla European/American calls and puts on equity indices.

EquityIndexOption

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityIndexOption(expiry_dt: Union[Date, list],
strike_price: Union[float, np.ndarray],
opt_type: OptionTypes,
num_options: Optional[float] = 1.0,
"""Create the Equity Index option object by specifying the expiry
date, the option strike, the option type and the number of options."""
check_argument_types(.EquityIndexOption, locals())
if opt_type in (OptionTypes.EUROPEAN_CALL,
OptionTypes.EUROPEAN_PUT,
OptionTypes.AMERICAN_CALL,
OptionTypes.AMERICAN_PUT,
):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date or list	-	-
strike_price	float or np.ndarray	-	-
opt_type	OptionTypes	-	-
num_options	Optional[float]	-	1.0
"""Create the Equity Index option object by specifying the expiry	-	-	-
date	-	-	-
the option strike	-	-	-
the option type and the number of options."""	-	-	-
check_argument_types(.EquityIndexOption, locals())	-	-	-
OptionTypes.EUROPEAN_PUT	-	-	-
OptionTypes.AMERICAN_CALL	-	-	-
OptionTypes.AMERICAN_PUT	-	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Union[Date, list],
forward_price: float,
discount_curve: DiscountCurve,
model: Model,
"""Equity Index Option valuation using Black model."""
if isinstance(value_dt, Date) is False:
```

```

    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
    )
if isinstance(.expiry_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date or list	-	-
forward_price	float	-	-
discount_curve	DiscountCurve	-	-
model	Model	-	-
"""Equity Index Option valuation using Black model."""	-	-	-
if isinstance(value_dt, Date) is False	-	-	-
raise FinError("Valuation date is not a Date")	-	-	-
if value_dt < .expiry_dt	-	-	-
raise FinError("Valuation date after expiry date.")	-	-	-
if discount_curve.value_dt !=	-	-	value_dt:
)	-	-	-

delta

PLEASE ADD A FUNCTION DESCRIPTION

```

delta(value_dt: Date,
forward_price: float,
discount_curve: DiscountCurve,
model,
"""Calculate_delta_of_a_European/American_Index_option."""
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
forward_price	float	-	-
discount_curve	DiscountCurve	-	-
model	-	-	-
"""Calculate delta of a European/American Index option."""	-	-	-

gamma

PLEASE ADD A FUNCTION DESCRIPTION


```

gamma(value_dt: Date,
forward_price: float,
discount_curve: DiscountCurve,
model: Model,
"""Calculate_gamma_of_a_European/American_Index_option."""
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
forward_price	float	-	-
discount_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate gamma of a European/American Index option."""		-	-

vega

PLEASE ADD A FUNCTION DESCRIPTION

```

vega(value_dt: Date,
forward_price: float,
discount_curve: DiscountCurve,
model: Model,
"""Calculate_vega_of_a_European/American_Index_option."""
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
forward_price	float	-	-
discount_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate vega of a European/American Index option."""		-	-

theta

PLEASE ADD A FUNCTION DESCRIPTION

```

theta(value_dt: Date,
forward_price: float,
discount_curve: DiscountCurve,
model: Model,
"""Calculate_theta_of_a_European/American_Index_option."""
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
forward_price	float	-	-
discount_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate theta of a European/American Index option."""		-	-

implied_volatility

PLEASE ADD A FUNCTION DESCRIPTION

```
implied_volatility(value_dt: Date,
forward_price: Union[float, list, np.ndarray],
discount_curve: DiscountCurve,
model: Model,
price: float,
"""Calculate the Black implied volatility of a European/American
Index option."""
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
if t_exp < 1.0 / 365.0:
    print("Expiry time is too close to zero.")
    return -999
df = discount_curve.df(.expiry_dt) / discount_curve.df(value_dt)
r = -np.log(df) / t_exp
if isinstance(model, Black):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
forward_price	float or list,np.ndarray	-
discount_curve	DiscountCurve	-
model	Model	-
price	float	-
"""Calculate the Black implied volatility of a European/American		-
Index option."""		-
t_exp	-	-
if t_exp < 1.0 / 365.0	-	-
print("Expiry time is too close to zero.")	-	-
return -999	-	-
df	-	-
r	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

4.14 equity_model_types

Class: *EquityModel*

EquityModel

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityModel():
```

The function arguments are described in the following table.

Class: *EquityModelHeston(EquityModel)*

class EquityModelHeston(EquityModel):

EquityModelHeston

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityModelHeston(volatility, mean_reversion):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	-	-	-
mean_reversion	-	-	-

`--repr--`

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

4.15 equity_one_touch_option

Class: EquityOneTouchOption(EquityOption)

A EquityOneTouchOption is an option in which the buyer receives one unit of cash OR stock if the stock price touches a barrier at any time before the option expiry date and zero otherwise. The choice of cash or stock is made at trade initiation. The single barrier payoff must define whether the option pays or cancels if the barrier is touched and also when the payment is made (at hit time or option expiry). All of these variants are all members of the FinTouchOptionTypes enumerated type.

EquityOneTouchOption

Create the one touch option by defining its expiry date and the barrier level and a payment size if it is a cash

```
EquityOneTouchOption(expiry_dt: Date,
opt_type: TouchOptionTypes,
barrier_price: float,
payment_size: float = 1.0,
"""Create the one touch option by defining its expiry date and the
barrier level and a payment size if it is a cash."""
check_argument_types(.EquityOneTouchOption, locals())
.expiry_dt = expiry_dt
.opt_type = opt_type
.barrier_price = float(barrier_price)
.payment_size = payment_size
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
opt_type	TouchOptionTypes	-	-
barrier_price	float	-	-
payment_size	float	-	1.0
"""Create the one touch option by defining its expiry date and the barrier level and a payment size if it is a cash."""	-	-	-
check_argument_types(.EquityOneTouchOption, locals())	-	-	-
.expiry_dt	-	-	expiry_dt
.opt_type	-	-	opt_type
.barrier_price	-	-	float(barrier_price)
.payment_size	-	-	payment_size

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
```

```

stock_price: Union[float, np.ndarray],
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Equity_One-Touch_Option_valuation_using_the_Black-Scholes_model
assuming_a_continuous_(American)_barrier_from_value_date_to_expiry.
Handles_both_cash-or-nothing_and_asset-or-nothing_options."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_date_not_same_as_option_valuation_date")
if dividend_curve.value_dt != value_dt:
    raise FinError("Dividend_Curve_date_not_same_as_option_valuation_date")
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
t = max(t, 1e-6)
s0 = stock_price
hh = .barrier_price
k = .payment_size
sqrt_t = np.sqrt(t)
df = discount_curve.df(.expiry_dt)
r = discount_curve.cc_rate(.expiry_dt)
q = dividend_curve.cc_rate(.expiry_dt)
v = model.volatility
v = max(v, 1e-6)
# Using notation in Haug page 177
b = r - q
mu = (b - v * v / 2.0) / v / v
lam = np.sqrt(mu * mu + 2.0 * r / v / v)
if DEBUG_MODE:
    print("t:", t)
    print("vol", v)
    print("b", b)
    print("mu", mu)
    print("lam", lam)
# Reference Option Pricing Formulas by Espen Gaarder Haug. Page 176.
if .opt_type == TouchOptionTypes.DOWN_AND_IN_CASH_AT_HIT:
    # HAUG 1
    if np.any(s0 <= hh):

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float or np.ndarray	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Equity One-Touch Option valuation using the Black-Scholes model	-	-
assuming a continuous (American) barrier from value date to expiry.	-	-
Handles both cash-or-nothing and asset-or-nothing options."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if discount_curve.value_dt !=	-	-
raise FinError("Discount Curve date not same as option valuation date")	-	-
if dividend_curve.value_dt !=	-	-
raise FinError("Dividend Curve date not same as option valuation date")	-	-
t	-	(exp
t	-	-
s0	-	-
hh	-	-
k	-	-
sqrt_t	-	-
df	-	-
r	-	-
q	-	-
v	-	-
v	-	-
b	-	-
mu	-	-
lam	-	-
if DEBUG_MODE	-	-
print("t", t)	-	-
print("vol", v)	-	-
print("b", b)	-	-
print("mu", mu)	-	-
print("lam", lam)	-	-
if .opt_type	-	= TouchOp

value_mc

Touch Option valuation using the Black-Scholes model and Monte Carlo simulation. Accuracy is not great when compared to the analytical result as we only observe the barrier a finite number of times. The convergence is slow.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
num_paths: int = 10000,
num_steps_per_year: int = 252,
seed: int = 4242,
"""Touch Option valuation using the Black-Scholes model and Monte
Carlo simulation. Accuracy is not great when compared to the analytical
result as we only observe the barrier a finite number of times. The
convergence is slow."""
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t
num_time_steps = int(t * num_steps_per_year) + 1
dt = t / num_time_steps
v = model.volatility
s0 = stock_price
mu = r - q
time_grid, s = get_paths_times(num_paths, num_time_steps, t, mu, s0, v, seed)
hh = .barrier_price
xx = .payment_size
v = 0.0
if .opt_type == TouchOptionTypes.DOWN_AND_IN_CASH_AT_HIT:
    # HAUG 1
    if s0 <= hh:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_down(s, hh, r, dt)
    v = v * xx
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_CASH_AT_HIT:
    # HAUG 2
    if s0 >= hh:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_up(s, hh, r, dt)
    v = v * xx
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_IN_ASSET_AT_HIT:
    # HAUG 3
    if s0 <= hh:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_down(s, hh, r, dt) * hh
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_ASSET_AT_HIT:
    # HAUG 4
    if s0 >= hh:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_up(s, hh, r, dt) * hh
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_IN_CASH_AT_EXPIRY:
    # HAUG 5
    if s0 <= hh:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_down(s, hh, 0.0, dt)

```



```

    v = v * xx * np.exp(-r * t)
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_CASH_AT_EXPIRY:
    # HAUG 6
    if s0 >= hh:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_up(s, hh, 0.0, dt)
    v = v * xx * np.exp(-r * t)
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_IN_ASSET_AT_EXPIRY:
    # HAUG 7
    if s0 <= hh:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_down(s, hh, 0.0, dt) * hh
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_ASSET_AT_EXPIRY:
    # HAUG 8
    if s0 >= hh:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_up(s, hh, 0.0, dt) * hh
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_OUT_CASH_OR_NOTHING:
    # HAUG 9
    if s0 <= hh:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = 1.0 - _barrier_pay_one_at_hit_pv_down(s, hh, 0.0, dt)
    v = v * xx * np.exp(-r * t)
    return v
elif .opt_type == TouchOptionTypes.UP_AND_OUT_CASH_OR_NOTHING:
    # HAUG 10
    if s0 >= hh:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = 1.0 - _barrier_pay_one_at_hit_pv_up(s, hh, 0.0, dt)
    v = v * xx * np.exp(-r * t)
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_OUT_ASSET_OR_NOTHING:
    # HAUG 11
    if s0 <= hh:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_asset_at_expiry_down_out(s, hh)
    v = v * np.exp(-r * t)
    return v
elif .opt_type == TouchOptionTypes.UP_AND_OUT_ASSET_OR_NOTHING:
    # HAUG 12
    if s0 >= hh:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_asset_at_expiry_up_out(s, hh)
    v = v * np.exp(-r * t)
    return v
else:
    raise FinError("Unknown_option_type.")
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
num_paths	int	-
num_steps_per_year	int	-
seed	int	-
"""Touch Option valuation using the Black-Scholes model and Monte Carlo simulation. Accuracy is not great when compared to the analytical result as we only observe the barrier a finite number of times. The convergence is slow."""	-	-
t	-	(e
df	-	-
r	-	-
dq	-	-
q	-	-
num_time_steps	-	-
dt	-	-
v	-	-
s0	-	-
mu	-	-
time_grid	-	-
s	-	get_paths.ti
hh	-	-
xx	-	-
v	-	-
if .opt_type	-	= TouchC
if s0 >=	-	-
raise FinError("Barrier has ALREADY been crossed.")	-	-
v	-	_ba
v	-	-
return v	-	-
elif .opt_type	-	= Touch
if s0 <=	-	-
raise FinError("Barrier has ALREADY been crossed.")	-	-
v	-	_L
v	-	-
return v	-	-
elif .opt_type	-	= TouchC
if s0 <=	-	-
raise FinError("Stock price is currently below barrier.")	-	-
v	-	_barri
return v	-	-
elif .opt_type	-	= Touch
if s0 <=	-	-
raise FinError("Stock price is currently below barrier.")	-	-
v	-	_bar
return v	-	-
elif .opt_type	-	= TouchOp
if s0 <=	-	-

--repr--*PLEASE ADD A FUNCTION DESCRIPTION*`__repr__():`

The function arguments are described in the following table.

_print*Simple print function for backward compatibility.*`_print():`

The function arguments are described in the following table.

_barrier_pay_one_at_hit_pv_down*Pay \$1 if the stock crosses the barrier hh from above. PV payment.*`_barrier_pay_one_at_hit_pv_down(s, hh, r, dt):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
hh	-	-	-
r	-	-	-
dt	-	-	-

_barrier_pay_one_at_hit_pv_up*Pay \$1 if the stock crosses the barrier hh from below. PV payment.*`_barrier_pay_one_at_hit_pv_up(s, hh, r, dt):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
hh	-	-	-
r	-	-	-
dt	-	-	-

`_barrier_pay_asset_at_expiry_down_out`

Pay \$1 if the stock crosses the barrier hh from above. PV payment.

```
_barrier_pay_asset_at_expiry_down_out(s, hh):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
<code>s</code>	-	-	-
<code>hh</code>	-	-	-

`_barrier_pay_asset_at_expiry_up_out`

Pay \$1 if the stock crosses the barrier hh from below. PV payment.

```
_barrier_pay_asset_at_expiry_up_out(s, hh):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
<code>s</code>	-	-	-
<code>hh</code>	-	-	-

4.16 equity_option

Enumerated Type: EquityOptionModelTypes

This enumerated type has the following values:

- BLACK_SCHOLES
- ANOTHER
- v
- v_bumped
- delta
- v
- v_bumped_dn
- v_bumped_up
- gamma
- bump
- v
- model
- v_bumped
- vega
- delta
- model
- delta_bumped
- vanna
- calculatingthedifferencedividedbydt
- v
- next_dt
- discount_curve.value_dt
- dividend_curve.value_dt
- time_bump
- v_bumped
- discount_curve.value_dt
- dividend_curve.value_dt
- theta
- v
- v_bumped
- rho

Class: EquityOption

This class is a parent class for all equitu option classes that require any perturbatory risk.

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
      stock_price: float,
```

```

discount_curve: Union[DiscountCurve, float],
dividend_curve: Union[DiscountCurve, float],
model: Model,
print("You_should_not_be_here!")
return 0.0
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve or float	-	-
dividend_curve	DiscountCurve or float	-	-
model	Model	-	-
print("You should not be here!")	-	-	-
return 0.0	-	-	-

delta

Calculation of option delta by perturbation of stock price and revaluation.

```

delta(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Calculation_of_option_delta_by_perturbation_of_stock_price_and
revaluation."""
v = .value(value_dt, stock_price, discount_curve, dividend_curve, model)
v_bumped = .value(value_dt, stock_price + BUMP, discount_curve, dividend_curve, model
)
delta = (v_bumped - v) / BUMP
return delta
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Calculation of option delta by perturbation of stock price and revaluation."""	-	-
v	-	.value(value_dt, stock_
)	-	-
delta	-	-
return delta	-	-

gamma

Calculation of option gamma by perturbation of stock price and revaluation.

```

gamma(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Calculation of option gamma by perturbation of stock price and
revaluation."""
v = .value(value_dt, stock_price, discount_curve, dividend_curve, model)
v_bumped_dn = .value(value_dt, stock_price - BUMP, discount_curve, dividend_curve,
model
)
v_bumped_up = .value(value_dt, stock_price + BUMP, discount_curve, dividend_curve,
model
)
gamma = (v_bumped_up - 2.0 * v + v_bumped_dn) / BUMP / BUMP
return gamma
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Calculation of option gamma by perturbation of stock price and revaluation."""	-	-
v	-	.value(value_dt, stock_price, discount_curve, dividend_curve, model)
)	-	-
)	-	-
gamma	-	(v_bumped_up - v) / bump
return gamma	-	-

vega

Calculation of option vega by perturbing vol and revaluation.

```
vega(value_dt: Date,
      stock_price: float,
      discount_curve: DiscountCurve,
      dividend_curve: DiscountCurve,
      model,
      """Calculation_of_option_vega_by_perturbing_vol_and_revaluation."""
      # The bump should be 1% (0.01) not 1bp (0.0001)
      bump = 0.01
      v = .value(value_dt, stock_price, discount_curve, dividend_curve, model)
      model = BlackScholes(model.volatility + bump)
      v_bumped = .value(value_dt, stock_price, discount_curve, dividend_curve, model)
      )
      vega = v_bumped - v
      return vega
      #####
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Calculation of option vega by perturbing vol and revaluation."""	-	-
bump	-	-
v	-	.value(value_dt, sto
model	-	Black
)	-	-
vega	-	-
return vega	-	-

vanna

Calculation of option vanna by perturbing delta with respect to the stock price volatility.

```
vanna(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Calculation_of_option_vanna_by_perturbing_delta_with_respect_to_the
stock_price_volatility."""
delta = .delta(value_dt, stock_price, discount_curve, dividend_curve, model)
model = BlackScholes(model.volatility + BUMP)
delta_bumped = .delta(value_dt, stock_price, discount_curve, dividend_curve, model
)
vanna = (delta_bumped - delta) / BUMP
return vanna
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Calculation of option vanna by perturbing delta with respect to the stock price volatility."""	-	-
delta	-	.delta(value_dt, st
model	-	Black
)	-	-
vanna	-	-
return vanna	-	-

theta

Calculation of option theta by perturbing value date by one calendar date (not a business date) and then doing revaluation and calculating the difference divided by $dt = 1 / G_DAYS_IN_YEAR$.

```
theta(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Calculation of option theta by perturbing value date by one
calendar date (not a business date) and then doing revaluation and
calculating the difference divided by dt = 1 / G_DAYS_IN_YEAR."""
v = .value(value_dt, stock_price, discount_curve, dividend_curve, model)
next_dt = value_dt.add_days(1)
# Need to do this carefully. This is a bit hacky.
discount_curve.value_dt = next_dt
dividend_curve.value_dt = next_dt
time_bump = (next_dt - value_dt) / G_DAYS_IN_YEAR
v_bumped = .value(next_dt, stock_price, discount_curve, dividend_curve, model
)
# restore valuation dates
discount_curve.value_dt = value_dt
dividend_curve.value_dt = value_dt
theta = (v_bumped - v) / time_bump
return theta
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
model	-	-
"""Calculation of option theta by perturbing value date by one	-	-
calendar date (not a business date) and then doing revaluation and	-	-
calculating the difference divided by dt	-	-
v	-	.value(value_dt, stock
next_dt	-	-
discount_curve.value_dt	-	-
dividend_curve.value_dt	-	-
time_bump	-	(next_dt
)	-	-
discount_curve.value_dt	-	-
dividend_curve.value_dt	-	-
theta	-	(
return theta	-	-

rho

Calculation of option rho by perturbing interest rate and revaluation.

```
rho(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model,
"""Calculation_of_option_rho_by_perturbing_interest_rate_and
revaluation."""
v = .value(value_dt, stock_price, discount_curve, dividend_curve, model)
v_bumped = .value(value_dt,
    stock_price,
    discount_curve.bump(BUMP),
    dividend_curve,
    model,
)
rho = (v_bumped - v) / BUMP
return rho
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
value_dt	Date	-	
stock_price	float	-	
discount_curve	DiscountCurve	-	
dividend_curve	DiscountCurve	-	
model	-	-	
"""Calculation of option rho by perturbing interest rate and revaluation."""	-	-	
v	-	-	.value(value_dt, stock_price,
stock_price	-	-	
discount_curve.bump(BUMP)	-	-	
dividend_curve	-	-	
model	-	-	
)	-	-	
rho	-	-	(v_bu
return rho	-	-	

4.17 equity_rainbow_option

Enumerated Type: EquityRainbowOptionTypes

This enumerated type has the following values:

- CALL_ON_MAXIMUM
- PUT_ON_MAXIMUM
- CALL_ON_MINIMUM
- PUT_ON_MINIMUM
- CALL_ON_NTH
- PUT_ON_NTH
- ifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- n
- k
- sorted
- assetn
- payoff
- elifpayoff_type_value
- n
- k
- sorted
- assetn
- payoff
- num_paths
- seed
- df
- r
- qs
- dq
- q
- qs
- mus
- _,s_all
- payoff

- payoff
- v
- self.expiry_dt
- self.payoff_type
- self.payoff_params
- self.num_assets
- iflen(stock_prices)!
- iflen(dividend_curves)!
- iflen(volatilities)!
- iflen(betas)!
- num_params
- ifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- iflen(payoff_params)!
- payoff_type
- orpayoff_type
- n
- ifdiscount_curve.value_dt!
- ifself.num_assets!
- ifcorr_matrix.ndim!
- ifcorr_matrix.shape[0]!
- ifcorr_matrix.shape[1]!
- t
- r
- q1
- q2
- dividend_yields
- q1
- q2
- rho
- s1
- s2
- b1
- b2
- v1
- v2

- k
- v
- d
- y1
- y2
- rho1
- rho2
- dq1
- dq2
- df
- ifself.payoff_type
- v
- elifself.payoff_type
- v
- elifself.payoff_type
- cmax1
- cmax2
- v
- elifself.payoff_type
- cmin1
- cmin2
- v
- num_paths
- seed
- t
- v
- s
- s+
- s+
- s+
- s+

Class: EquityRainbowOption(EquityOption)

class EquityRainbowOption(EquityOption):

EquityRainbowOption

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityRainbowOption(expiry_dt: Date,
payoff_type: EquityRainbowOptionTypes,
payoff_params: List[float],
num_assets: int,
check_argument_types(.EquityRainbowOption, locals()))
.validate_payoff(payoff_type, payoff_params, num_assets)
.expiry_dt = expiry_dt
.payoff_type = payoff_type
```

```
.payoff_params = payoff_params
.num_assets = num_assets
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
payoff_type	EquityRainbowOptionTypes	-	-
payoff_params	List[float]	-	-
num_assets	int	-	-
check_argument_types(.EquityRainbowOption, locals())	-	-	-
._validate_payoff(payoff_type, payoff_params, num_assets)	-	-	-
.expiry_dt	-	-	expiry_dt
.payoff_type	-	-	payoff_type
.payoff_params	-	-	payoff_params
.num_assets	-	-	num_assets

_validate

PLEASE ADD A FUNCTION DESCRIPTION

```
_validate(stock_prices, dividend_curves, volatilities, betas):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
stock_prices	-	-	-
dividend_curves	-	-	-
volatilities	-	-	-
betas	-	-	-

_validate_payoff

PLEASE ADD A FUNCTION DESCRIPTION

```
_validate_payoff(payoff_type, payoff_params, num_assets):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
payoff_type	-	-	-
payoff_params	-	-	-
num_assets	-	-	-

value*PLEASE ADD A FUNCTION DESCRIPTION*

```

value(value_dt: Date,
stock_prices: np.ndarray,
discount_curve: DiscountCurve,
dividend_curves: list,
volatilities: np.ndarray,
corr_matrix: np.ndarray,
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
)
if .num_assets != 2:
    raise FinError("Analytical_results_for_two_assets_only.")
if corr_matrix.ndim != 2:
    raise FinError("Corr_matrix_must_be_of_size_2x2")
if corr_matrix.shape[0] != 2:
    raise FinError("Corr_matrix_must_be_of_size_2x2")
if corr_matrix.shape[1] != 2:
    raise FinError("Corr_matrix_must_be_of_size_2x2")
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
# Use result by Stulz (1982) given by Haug Page 211
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
r = discount_curve.zero_rate(.expiry_dt)
q1 = dividend_curves[0].zero_rate(.expiry_dt)
q2 = dividend_curves[1].zero_rate(.expiry_dt)
dividend_yields = [q1, q2]
._validate(stock_prices, dividend_yields, volatilities, corr_matrix)
#     q1 = dividend_yields[0]
#     q2 = dividend_yields[1]
rho = corr_matrix[0][1]
s1 = stock_prices[0]
s2 = stock_prices[1]
b1 = r - q1
b2 = r - q2
v1 = volatilities[0]
v2 = volatilities[1]
k = .payoff_params[0]
v = sqrt(v1 * v1 + v2 * v2 - 2 * rho * v1 * v2)
d = (log(s1 / s2) + (b1 - b2 + v * v / 2) * t) / v / sqrt(t)
y1 = (log(s1 / k) + (b1 + v1 * v1 / 2) * t) / v1 / sqrt(t)
y2 = (log(s2 / k) + (b2 + v2 * v2 / 2) * t) / v2 / sqrt(t)
rho1 = (v1 - rho * v2) / v
rho2 = (v2 - rho * v1) / v
dq1 = exp(-q1 * t)
dq2 = exp(-q2 * t)
df = exp(-r * t)
if .payoff_type == EquityRainbowOptionTypes.CALL_ON_MAXIMUM:
    v = (s1 * dq1 * M(y1, d, rho1)
        + s2 * dq2 * M(y2, -d + v * sqrt(t), rho2)

```

```

        - k * df * (1.0 - M(-y1 + v1 * sqrt(t), -y2 + v2 * sqrt(t), rho))
    )
elif .payoff_type == EquityRainbowOptionTypes.CALL_ON_MINIMUM:
    v = (s1 * dq1 * M(y1, -d, -rho1)
        + s2 * dq2 * M(y2, d - v * sqrt(t), -rho2)
        - k * df * M(y1 - v1 * sqrt(t), y2 - v2 * sqrt(t), rho)
    )
elif .payoff_type == EquityRainbowOptionTypes.PUT_ON_MAXIMUM:
    cmax1 = (s2 * dq2 + s1 * dq1 * normcdf(d) - s2 * dq2 * normcdf(d - v * sqrt(t)))
    )
    cmax2 = (s1 * dq1 * M(y1, d, rho1)
        + s2 * dq2 * M(y2, -d + v * sqrt(t), rho2)
        - k * df * (1.0 - M(-y1 + v1 * sqrt(t), -y2 + v2 * sqrt(t), rho))
    )
    v = k * df - cmax1 + cmax2
elif .payoff_type == EquityRainbowOptionTypes.PUT_ON_MINIMUM:
    cmin1 = (s1 * dq1 - s1 * dq1 * normcdf(d) + s2 * dq2 * normcdf(d - v * sqrt(t)))
    )
    cmin2 = (s1 * dq1 * M(y1, -d, -rho1)
        + s2 * dq2 * M(y2, d - v * sqrt(t), -rho2)
        - k * df * M(y1 - v1 * sqrt(t), y2 - v2 * sqrt(t), rho)
    )
    v = k * df - cmin1 + cmin2
else:
    raise FinError("Unsupported_Rainbow_option_type")
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	D
value_dt	Date	-	
stock_prices	np.ndarray	-	
discount_curve	DiscountCurve	-	
dividend_curves	list	-	
volatilities	np.ndarray	-	
corr_matrix	np.ndarray	-	
if isinstance(value_dt, Date) is False		-	
raise FinError("Valuation date is not a Date")	-	-	
if value_dt > expiry_dt		-	
raise FinError("Valuation date after expiry date.")	-	-	
if discount_curve.value_dt !=	-	-	
)	-	-	
if .num_assets !=	-	-	
raise FinError("Analytical results for two assets only.")	-	-	
if corr_matrix.ndim !=	-	-	
raise FinError("Corr matrix must be of size 2x2")	-	-	
if corr_matrix.shape[0] !=	-	-	
raise FinError("Corr matrix must be of size 2x2")	-	-	
if corr_matrix.shape[1] !=	-	-	
raise FinError("Corr matrix must be of size 2x2")	-	-	
if value_dt > expiry_dt		-	
raise FinError("Value date after expiry date.")	-	-	
t	-	-	(expiry_dt - value_dt)
r	-	-	discount_curve.r
q1	-	-	dividend_curves[0]
q2	-	-	dividend_curves[1]
dividend_yields	-	-	
._validate(stock_prices, dividend_yields, volatilities, corr_matrix)	-	-	
rho	-	-	corr_matrix[0,0]
s1	-	-	stock_prices[0]
s2	-	-	stock_prices[1]
b1	-	-	
b2	-	-	
v1	-	-	volatilities[0]
v2	-	-	volatilities[1]
k	-	-	payoff_type
v	-	-	$\sqrt{v1 * v1 + v2 * v2}$
d	-	-	$(\log(s1 / s2) + (b1 - r) * t)$
y1	-	-	$(\log(s1 / k) + (b1 - r) * t)$
y2	-	-	$(\log(s2 / k) + (b2 - r) * t)$
rho1	-	-	rho1
rho2	-	-	rho2
dq1	-	-	
dq2	-	-	
df	-	-	
if .payoff_type	-	-	= EquityRainbowOption
+ s2 * dq2 * M(y2, -d + v * sqrt(t), rho2)	-	-	
- k * df * (1.0 - M(-y1 + v1 * sqrt(t), -y2 + v2 * sqrt(t), rho))	-	-	
)	-	-	
elif .payoff_type	-	-	= EquityRainbowOption
+ s2 * dq2 * M(y2, -d + v * sqrt(t), rho2)	-	-	

value_mc*PLEASE ADD A FUNCTION DESCRIPTION*

```

value_mc(value_dt,
stock_prices,
discount_curve,
dividend_curves,
volatilities,
corr_matrix,
num_paths=10000,
seed=4242,
.validate(stock_prices, dividend_curves, volatilities, corr_matrix)
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
v = value_mc_fast(t,
    stock_prices,
    discount_curve,
    dividend_curves,
    volatilities,
    corr_matrix,
    .num_assets,
    .payoff_type,
    .payoff_params,
    num_paths,
    seed,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_prices	-	-	-
discount_curve	-	-	-
dividend_curves	-	-	-
volatilities	-	-	-
corr_matrix	-	-	-
num_paths	-	-	10000
seed	-	-	4242
.validate(stock_prices, dividend_curves, volatilities, corr_matrix)	-	-	-
if value_dt > .expiry_dt	-	-	-
raise FinError("Value date after expiry date.")	-	-	-
t	-	(.expiry_dt - value_dt) / G.DAY	-
stock_prices	-	-	-
discount_curve	-	-	-
dividend_curves	-	-	-
volatilities	-	-	-
corr_matrix	-	-	-
.num_assets	-	-	-
.payoff_type	-	-	-
.payoff_params	-	-	-
num_paths	-	-	-
seed	-	-	-
)	-	-	-
return v	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

payoff_value

PLEASE ADD A FUNCTION DESCRIPTION

```
payoff_value(s, payoff_type_value, payoff_params):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
payoff_type_value	-	-	-
payoff_params	-	-	-

value_mc_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc_fast(
t,
stock_prices,
discount_curve,
dividend_curves,
volatilities,
betas,
num_assets,
payoff_type,
payoff_params,
num_paths=10000,
seed=4242,
np.random.seed(seed)
df = discount_curve.df_t(t)
r = -log(df) / t
qs = []
for curve in dividend_curves:
    dq = curve.df_t(t)
    q = -np.log(dq) / t
    qs.append(q)
qs = np.array(qs)
mus = r - qs
_, s_all = get_assets_paths(
    num_assets,
    num_paths,
    t,
    mus,
    stock_prices,
    volatilities,
    betas,
    seed,
)
payoff = payoff_value(s_all, payoff_type.value, payoff_params)
payoff = np.mean(payoff)
v = payoff * exp(-r * t)
return v
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
stock_prices	-	-	-
discount_curve	-	-	-
dividend_curves	-	-	-
volatilities	-	-	-
betas	-	-	-
num_assets	-	-	-
payoff_type	-	-	-
payoff_params	-	-	-
num_paths	-	-	10000
seed	-	-	4242
np.random.seed(seed)	-	-	-
df	-	-	discount_curve.df_t(t)
r	-	-	-log(df) / t
qs	-	-	[]
for curve in dividend_curves	-	-	-
dq	-	-	curve.df_t(t)
q	-	-	-np.log(dq) / t
qs.append(q)	-	-	-
qs	-	-	np.array(qs)
mus	-	-	r - qs
-	-	-	-
num_assets	-	-	-
num_paths	-	-	-
t	-	-	-
mus	-	-	-
stock_prices	-	-	-
volatilities	-	-	-
betas	-	-	-
seed	-	-	-
)	-	-	-
payoff	-	-	payoff_value(s_all, payoff_type.value, payoff_params)
payoff	-	-	np.mean(payoff)
v	-	-	payoff * exp(-r * t)
return v	-	-	-

4.18 equity_swap

Class: EquitySwap

Class for managing a standard Equity vs Float leg swap. This is a contract in which an equity payment leg is exchanged for a series of floating rates payments. There is no exchange of principal. The contract is entered into at zero initial cost when spreads are zero. The contract lasts from an effective date to a specified maturity date.

The equity payments are not known fully until the end of payment period.

The floating rate is not known fully until the end of the preceding payment period. It is set in advance and paid in arrears.

The value of the contract is the NPV of the two coupon streams. Discounting is done on a supplied discount curve which is separate from the curve from which the implied index rates are extracted.

EquitySwap

PLEASE ADD A FUNCTION DESCRIPTION

```
EquitySwap(effective_dt: Date, # Date contract starts or last Equity Reset
term_dt_or_tenor: Union[Date, str], # Date contract ends
eq_leg_type: SwapTypes,
eq_freq_type: FrequencyTypes,
eq_dc_type: DayCountTypes,
strike: float, # Price at effective date
quantity: float = 1.0, # Quantity at effective date
eq_payment_lag: int = 0,
eq_return_type: ReturnTypes = ReturnTypes.TOTAL_RETURN,
rate_freq_type: FrequencyTypes = FrequencyTypes.MONTHLY,
rate_dc_type: DayCountTypes = DayCountTypes.ACT_360,
rate_spread: float = 0.0,
rate_payment_lag: int = 0,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
end_of_month: bool = False,
"""Create an equity swap contract given the contract effective date,
its maturity, underlying price and quantity, day count convention and
return type and other details. The equity leg parameters have default
values that can be overwritten if needed.
The start date is contractual and is the same as the settlement date
for a new swap. It is the date on which interest starts to accrue. The end
of the contract is the termination date. This is not adjusted for business
days. The adjusted termination date is called the maturity date.
This is calculated."""
check_argument_types(EquitySwap, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date contract starts or last
term_dt_or_tenor	Date or str	Date contract
eq_leg_type	SwapTypes	-
eq_freq_type	FrequencyTypes	-
eq_dc_type	DayCountTypes	-
strike	float	Price at effective
quantity	float	Quantity at effect
eq_payment_lag	int	-
eq_return_type	ReturnTypes	-
rate_freq_type	FrequencyTypes	-
rate_dc_type	DayCountTypes	-
rate_spread	float	-
rate_payment_lag	int	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
end_of_month	bool	-
"""Create an equity swap contract given the contract effective date		-
its maturity		-
underlying price and quantity		-
day count convention and		-
return type and other details. The equity leg parameters have default		-
values that can be overwritten if needed.		-
The start date is contractual and is the same as the settlement date		-
for a new swap. It is the date on which interest starts to accrue. The end		-
of the contract is the termination date. This is not adjusted for business		-
days. The adjusted termination date is called the maturity date.		-
This is calculated."""		-
check_argument_types(EquitySwap, locals())		-

value

Value the Equity swap on a valuation date.

```

value(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
dividend_curve: DiscountCurve = None,
current_price: float = None,
first_fixing_rate=None,
"""Value the Equity swap on a valuation date."""
.equity_leg_value = .equity_leg.value(value_dt,
    discount_curve,
    index_curve,
    dividend_curve,
    current_price,
)

```

```

_fill_rate_notional_array()
.rate_leg_value = .rate_leg.value(value_dt, discount_curve, index_curve,
    first_fixing_rate
)
return .equity_leg_value + .rate_leg_value
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve	DiscountCurve	-	None
dividend_curve	DiscountCurve	-	None
current_price	float	-	None
first_fixing_rate	-	-	None
"""Value the Equity swap on a valuation date."""	-	-	-
discount_curve	-	-	-
index_curve	-	-	-
dividend_curve	-	-	-
current_price	-	-	-
)	-	-	-
_fill_rate_notional_array()	-	-	-
)	-	-	-
return .equity_leg_value + .rate_leg_value	-	-	-

_fill_rate_notional_array

In an equity swap, at every equity reset, the notional of the contract is updated to reflect the new underlying price. This is a helper function that takes the Equity Notional list from Equity Leg and convert it to a Notional array that fits the payment schedule defined for the rate leg.

```
_fill_rate_notional_array():
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

4.19 equity_swap_leg

Class: EquitySwapLeg

Class for managing the equity leg of an equity swap. An equity leg is a leg with a sequence of flows calculated according to an ISDA schedule and follows the economics of a collection of equity forward contracts.

EquitySwapLeg

PLEASE ADD A FUNCTION DESCRIPTION

```
EquitySwapLeg(effective_dt: Date, # Contract starts or last equity reset
term_dt_or_tenor: Union[Date, str], # Date contract ends
leg_type: SwapTypes,
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
strike: float, # Price at effective date
quantity: float = 1.0, # Quantity at effective date
payment_lag: int = 0,
return_type: ReturnTypes = ReturnTypes.TOTAL_RETURN,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
end_of_month: bool = False,
"""Create the equity leg of a swap contract giving the contract start
date, its maturity, underlying strike price and quantity, payment
frequency, day count convention, return type, and other details"""
check_argument_types(EquitySwapLeg, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Contract starts or last equity n
term_dt_or_tenor	Date or str	Date contract ends
leg_type	SwapTypes	-
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
strike	float	Price at effective date
quantity	float	Quantity at effective date
payment_lag	int	-
return_type	ReturnTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
end_of_month	bool	-
"""Create the equity leg of a swap contract giving the contract start		-
date	-	-
its maturity	-	-
underlying strike price and quantity	-	-
payment	-	-
frequency	-	-
day count convention	-	-
return type	-	-
and other details"""	-	-
check_argument_types(.EquitySwapLeg, locals())	-	-

generate_payment_dts

Generate the Equity leg payment dates and accrual factors. Similar to swap float leg, payment values can't be generated, as we do not have index curve, dividend curve and equity price.

```
generate_payment_dts():
```

The function arguments are described in the following table.

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve,
dividend_curve: DiscountCurve = None,
current_price: float = None,
"""Value the equity leg with payments from an equity price, quantity,
an index curve and an [optional] dividend curve. Discounting is based
on a supplied discount curve as of the valuation date supplied.
""")
```

```

if discount_curve is None:
    raise FinError("Discount_curve_not_provided!")
if discount_curve.value_dt != value_dt:
    raise FinError("Discount_Curve_valuation_date_not_same_as_option_value_date"
)
if index_curve is None:
    index_curve = discount_curve
# Assume a naive dividend curve if nothing provided
if dividend_curve is None:
    dividend_curve = DiscountCurveFlat(value_dt, 0)
# Current price can't be different than strike at effective date
if current_price is not None:
    .current_price = current_price
else:
    .current_price = .strike
.fwd_rates = []
.div_fwd_rates = []
.eq_fwd_rates = []
.last_notionals = []
.payment_amounts = []
.payment_dfs = []
.payment_pvs = []
.cumulative_pvs = []
df_value = discount_curve.df(value_dt)
leg_pv, eq_term_rate = 0.0, 0.0
last_notional = .notional
next_notional = last_notional
num_payments = len(.payment_dts)
index_basis = index_curve.dc_type
index_day_counter = DayCount(index_basis)
for i_pmnt in range(0, num_payments):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	None
current_price	float	-	None
"""Value the equity leg with payments from an equity price			
quantity	-	-	-
an index curve and an [optional] dividend curve. Discounting is based			
on a supplied discount curve as of the valuation date supplied.			
"""	-	-	-
if discount_curve is None			
raise FinError("Discount curve not provided!")			
if discount_curve.value_dt !=	-	-	value_dt
)	-	-	-
if index_curve is None			
index_curve	-	-	discount_curve
if dividend_curve is None			
dividend_curve	-	-	DiscountCurveFlat
if current_price is not None			
.current_price	-	-	current_price
else	-	-	-
.current_price	-	-	.striking_price
.fwd_rates	-	-	[]
.div_fwd_rates	-	-	[]
.eq_fwd_rates	-	-	[]
.last_notionals	-	-	[]
.payment_amounts	-	-	[]
.payment_dfs	-	-	[]
.payment_pvs	-	-	[]
.cumulative_pvs	-	-	[]
df_value	-	-	discount_curve
leg_pv	-	-	-
eq_term_rate	-	-	0.0
0.0	-	-	-
last_notional	-	-	.notional
next_notional	-	-	last_notional
num_payments	-	-	len(payments)
index_basis	-	-	index_curve
index_day_counter	-	-	DayCount(in

print_payment_amounts

Prints the payment dates, accrual factors, discount factors, cash amounts, their present value and their cumulative PV using the last valuation performed.

```
print_payment_amounts():
```

The function arguments are described in the following table.

print_valuation

Prints the valuation dates, accrual factors, discount factors, cash amounts, their present value and their cumulative PV using the last valuation performed.

```
print_valuation():
```

The function arguments are described in the following table.

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

4.20 equity_vanilla_option

Class: EquityVanillaOption

Class for managing plain vanilla European calls and puts on equities. For American calls and puts see the EquityAmericanOption class.

EquityVanillaOption

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityVanillaOption(expiry_dt: Union[Date, list],
strike_price: Union[float, np.ndarray],
opt_type: Union[OptionTypes, list],
num_options: float = 1.0,
"""Create the Equity Vanilla option object by specifying the expiry
date, the option strike, the option type and the number of options."""
check_argument_types(.EquityVanillaOption, locals())
if isinstance(opt_type, OptionTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date or list	-	-
strike_price	float or np.ndarray	-	-
opt_type	OptionTypes or list	-	-
num_options	float	-	1.0
"""Create the Equity Vanilla option object by specifying the expiry	-	-	-
date	-	-	-
the option strike	-	-	-
the option type and the number of options."""	-	-	-
check_argument_types(.EquityVanillaOption, locals())	-	-	-

intrinsic

PLEASE ADD A FUNCTION DESCRIPTION

```
intrinsic(value_dt: Union[Date, list],
stock_price: Union[np.ndarray, float],
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
"""Equity Vanilla Option valuation using Black-Scholes model."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if isinstance(.expiry_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date or list	-	-
stock_price	np.ndarray or float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
"""Equity Vanilla Option valuation using Black-Scholes model."""			
if isinstance(value_dt, Date) is False		-	-
raise FinError("Valuation date is not a Date")	-	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Union[Date, list],
      stock_price: Union[np.ndarray, float],
      discount_curve: DiscountCurve,
      dividend_curve: DiscountCurve,
      model: Model,
      """Equity_Vanilla_Option_valuation_using_Black-Scholes_model."""
      if isinstance(value_dt, Date) is False:
          raise FinError("Valuation_date_is_not_a_Date")
      if isinstance(.expiry_dt, list):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date or list	-	-
stock_price	np.ndarray or float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Equity Vanilla Option valuation using Black-Scholes model."""			
if isinstance(value_dt, Date) is False		-	-
raise FinError("Valuation date is not a Date")	-	-	-

delta

PLEASE ADD A FUNCTION DESCRIPTION

```
delta(value_dt: Date,
      stock_price: float,
      discount_curve: DiscountCurve,
      dividend_curve: DiscountCurve,
      model,
      """Calculate_the_analytical_delta_of_a_European_vanilla_option."""
      if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	-	-	-
"""Calculate the analytical delta of a European vanilla option."""			

gamma

PLEASE ADD A FUNCTION DESCRIPTION

```
gamma(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
"""Calculate the analytical gamma of a European vanilla option."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate the analytical gamma of a European vanilla option."""			

vega

PLEASE ADD A FUNCTION DESCRIPTION

```
vega(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
"""Calculate the analytical vega of a European vanilla option."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate the analytical vega of a European vanilla option."""			

theta

PLEASE ADD A FUNCTION DESCRIPTION

```
theta(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
"""Calculate the analytical theta of a European vanilla option."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate the analytical theta of a European vanilla option."""			

rho

PLEASE ADD A FUNCTION DESCRIPTION

```
rho(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
"""Calculate the analytical rho of a European vanilla option."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate the analytical rho of a European vanilla option."""			

vanna

PLEASE ADD A FUNCTION DESCRIPTION

```
vanna(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
"""Calculate the analytical vanna of a European vanilla option."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
"""Calculate the analytical vanna of a European vanilla option."""			

implied_volatility

Calculate the Black-Scholes implied volatility of a European vanilla option.

```
implied_volatility(value_dt: Date,
stock_price: Union[float, list, np.ndarray],
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
price,
"""Calculate the Black-Scholes implied volatility of a European
vanilla option."""
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
if t_exp < 1.0 / 365.0:
    print("Expiry_time_is_too_close_to_zero.")
    return -999
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
k = .strike_price
```

```

s0 = stock_price
sigma = bs_implied_volatility(s0, t_exp, k, r, q, price, .opt_type_value)
return sigma
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float or list,np.ndarray	-
discount_curve	DiscountCurve	-
dividend_curve	DiscountCurve	-
price	-	-
"""Calculate the Black-Scholes implied volatility of a European vanilla option."""	-	-
t_exp	-	(.expiry
if t_exp < 1.0 / 365.0	-	-
print("Expiry time is too close to zero.")	-	-
return -999	-	-
df	-	-
r	-	-
dq	-	-
q	-	-
k	-	-
s0	-	-
sigma	-	bs_implied_vola
return sigma	-	-

value_mc_numpy_only

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc_numpy_only(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
num_paths: int = 10000,
seed: int = 4242,
use_sobol: int = 0,
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
vol = model.volatility
v = _value_mc_numpy_only(stock_price,
t_exp,
.strike_price,

```

```

    .opt_type.value,
    r,
    q,
    vol,
    num_paths,
    seed,
    use_sobol,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
num_paths	int	-	10000
seed	int	-	4242
use_sobol	int	-	0
t_exp	-	-	(.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df	-	-	discount_curve.df(.expiry_dt)
r	-	-	-np.log(df) / t_exp
dq	-	-	dividend_curve.df(.expiry_dt)
q	-	-	-np.log(dq) / t_exp
vol	-	-	model.volatility
t_exp	-	-	-
.strike_price	-	-	-
.opt_type.value	-	-	-
r	-	-	-
q	-	-	-
vol	-	-	-
num_paths	-	-	-
seed	-	-	-
use_sobol	-	-	-
)	-	-	-
return v	-	-	-

value_mc_numba_only

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc_numba_only(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,

```

```

model: Model,
num_paths: int = 10000,
seed: int = 4242,
use_sobol: int = 0,
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
vol = model.volatility
v = _value_mc_numba_only(stock_price,
    t_exp,
    .strike_price,
    .opt_type_value,
    r,
    q,
    vol,
    num_paths,
    seed,
    use_sobol,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
num_paths	int	-	10000
seed	int	-	4242
use_sobol	int	-	0
t_exp	-	-	(.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df	-	-	discount_curve.df(.expiry_dt)
r	-	-	-np.log(df) / t_exp
dq	-	-	dividend_curve.df(.expiry_dt)
q	-	-	-np.log(dq) / t_exp
vol	-	-	model.volatility
t_exp	-	-	-
.strike_price	-	-	-
.opt_type_value	-	-	-
r	-	-	-
q	-	-	-
vol	-	-	-
num_paths	-	-	-
seed	-	-	-
use_sobol	-	-	-
)	-	-	-
return v	-	-	-

value_mc_numba_parallel

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc_numba_parallel(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
num_paths: int = 10000,
seed: int = 4242,
use_sobol: int = 0,
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
vol = model.volatility
v = _value_mc_numba_parallel(stock_price,
t_exp,
.strike_price,
.opt_type_value,

```



```

    r,
    q,
    vol,
    num_paths,
    seed,
    use_sobol,
)
#         _value_mc_NUMBA_ONLY.parallel_diagnostics(level=4)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
num_paths	int	-	10000
seed	int	-	4242
use_sobol	int	-	0
t_exp	-	-	(.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df	-	-	discount_curve.df(.expiry_dt)
r	-	-	-np.log(df) / t_exp
dq	-	-	dividend_curve.df(.expiry_dt)
q	-	-	-np.log(dq) / t_exp
vol	-	-	model.volatility
t_exp	-	-	-
.strike_price	-	-	-
.opt_type_value	-	-	-
r	-	-	-
q	-	-	-
vol	-	-	-
num_paths	-	-	-
seed	-	-	-
use_sobol	-	-	-
)	-	-	-
return v	-	-	-

value_mc_numpy_numba

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc_numpy_numba(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,

```

```

model: Model,
num_paths: int = 10000,
seed: int = 4242,
use_sobol: int = 0,
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
vol = model.volatility
v = _value_mc_numpy_numba(stock_price,
    t_exp,
    .strike_price,
    .opt_type_value,
    r,
    q,
    vol,
    num_paths,
    seed,
    use_sobol,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
num_paths	int	-	10000
seed	int	-	4242
use_sobol	int	-	0
t_exp	-	-	(.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df	-	-	discount_curve.df(.expiry_dt)
r	-	-	-np.log(df) / t_exp
dq	-	-	dividend_curve.df(.expiry_dt)
q	-	-	-np.log(dq) / t_exp
vol	-	-	model.volatility
t_exp	-	-	-
.strike_price	-	-	-
.opt_type_value	-	-	-
r	-	-	-
q	-	-	-
vol	-	-	-
num_paths	-	-	-
seed	-	-	-
use_sobol	-	-	-
)	-	-	-
return v	-	-	-

value_mc_nonumba_nonumpy

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc_nonumba_nonumpy(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,
model: Model,
num_paths: int = 10000,
seed: int = 4242,
use_sobol: int = 0,
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
vol = model.volatility
v = _value_mc_nonumba_nonumpy(stock_price,
    t_exp,
    .strike_price,
    .opt_type.value,

```

```

    r,
    q,
    vol,
    num_paths,
    seed,
    use_sobol,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
stock_price	float	-	-
discount_curve	DiscountCurve	-	-
dividend_curve	DiscountCurve	-	-
model	Model	-	-
num_paths	int	-	10000
seed	int	-	4242
use_sobol	int	-	0
t_exp	-	-	(.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df	-	-	discount_curve.df(.expiry_dt)
r	-	-	-np.log(df) / t_exp
dq	-	-	dividend_curve.df(.expiry_dt)
q	-	-	-np.log(dq) / t_exp
vol	-	-	model.volatility
t_exp	-	-	-
.strike_price	-	-	-
.opt_type.value	-	-	-
r	-	-	-
q	-	-	-
vol	-	-	-
num_paths	-	-	-
seed	-	-	-
use_sobol	-	-	-
)	-	-	-
return v	-	-	-

value_mc

Value European style call or put option using Monte Carlo. This is mainly for educational purposes. Sobol numbers can be used.

```

value_mc(value_dt: Date,
stock_price: float,
discount_curve: DiscountCurve,
dividend_curve: DiscountCurve,

```

```

model: Model,
num_paths: int = 10000,
seed: int = 4242,
use_sobol: int = 0,
"""Value_European_style_call_or_put_option_using_Monte_Carlo._This_is
mainly_for_educational_purposes._Sobol_numbers_can_be_used."""
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(.expiry_dt)
r = -np.log(df) / t_exp
dq = dividend_curve.df(.expiry_dt)
q = -np.log(dq) / t_exp
vol = model.volatility
v = _value_mc_numba_only(stock_price,
    t_exp,
    .strike_price,
    .opt_type_value,
    r,
    q,
    vol,
    num_paths,
    seed,
    use_sobol,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	De
value_dt	Date	-	
stock_price	float	-	
discount_curve	DiscountCurve	-	
dividend_curve	DiscountCurve	-	
model	Model	-	
num_paths	int	-	
seed	int	-	
use_sobol	int	-	
"""Value European style call or put option using Monte Carlo. This is mainly for educational purposes. Sobol numbers can be used."""		-	
t_exp	-	-	(.expiry_dt - valu
df	-	-	discount_
r	-	-	-np.
dq	-	-	dividend_
q	-	-	-np.l
vol	-	-	mo
t_exp	-	-	
.strike_price	-	-	
.opt_type_value	-	-	
r	-	-	
q	-	-	
vol	-	-	
num_paths	-	-	
seed	-	-	
use_sobol	-	-	
)	-	-	
return v	-	-	

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

f*PLEASE ADD A FUNCTION DESCRIPTION*

```
_f(v, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
v	-	-	-
args	-	-	-

fvega*PLEASE ADD A FUNCTION DESCRIPTION*

```
_fvega(v, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
v	-	-	-
*args	-	-	-

4.21 equity_variance_swap

Class: *EquityVarianceSwap*

EquityVarianceSwap

PLEASE ADD A FUNCTION DESCRIPTION

```
EquityVarianceSwap(start_dt: Date,
maturity_dt_or_tenor: Union[Date, str],
strike_variance: float,
notional: float = ONE_MILLION,
pay_strike_flag: bool = True,
"""Create_variance_swap_contract."""
check_argument_types(.EquityVarianceSwap, locals())
if isinstance(maturity_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	Date	-	-
maturity_dt_or_tenor	Date or str	-	-
strike_variance	float	-	-
notional	float	-	ONE_MILLION
pay_strike_flag	bool	-	True
"""Create variance swap contract."""	-	-	-
check_argument_types(.EquityVarianceSwap, locals())	-	-	-

value

Calculate the value of the variance swap based on the realised volatility to the valuation date, the forward looking implied volatility to the maturity date using the libor discount curve.

```
value(value_dt, realised_var, fair_strike_var, libor_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
realised_var	-	-	-
fair_strike_var	-	-	-
libor_curve	-	-	-

fair_strike_approx

This is an approximation of the fair strike variance by Demeterfi et al. (1999) which assumes that $\sigma(K) = \sigma(F) - b(K-F)/F$ where F is the forward stock price and $\sigma(F)$ is the ATM forward vol.


```
fair_strike_approx(value_dt, fwd_stock_price, strikes, volatilities):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
fwd_stock_price	-	-	-
strikes	-	-	-
volatilities	-	-	-

fair_strike

Calculate the implied variance according to the volatility surface using a static replication methodology with a specially weighted portfolio of put and call options across a range of strikes using the approximate method set out by Demeterfi et al. 1999.

```
fair_strike(value_dt,
stock_price,
dividend_curve,
volatility_curve,
num_call_options,
num_put_options,
strike_spacing,
discount_curve,
use_forward=True,
"""Calculate the implied variance according to the volatility surface
using a static replication methodology with a specially weighted
portfolio of put and call options across a range of strikes using the
approximate method set out by Demeterfi et al. 1999."""
.num_put_options = num_put_options
.num_call_options = num_call_options
call_type = OptionTypes.EUROPEAN_CALL
put_type = OptionTypes.EUROPEAN_PUT
t_mat = (.maturity_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df_t(t_mat)
r = -np.log(df) / t_mat
dq = dividend_curve.df_t(t_mat)
q = -np.log(dq) / t_mat
s0 = stock_price
g = np.exp((r - q) * t_mat)
fwd = stock_price * g
# This fixes the centre strike of the replication options
if use_forward is True:
    sstar = fwd
else:
    sstar = stock_price
# Replication argument from Demeterfi, Derman, Kamal and Zhou from
# Goldman Sachs Research notes March 1999. See Appendix A. This aim is
# to use calls and puts to approximate the payoff of a log contract
min_strike = sstar - (num_put_options + 1) * strike_spacing
.put_wts = []
.put_strikes = []
.call_wts = []
```

```

.call_strikes = []
# if the lower strike is < 0 we go to as low as the strike spacing
if min_strike < strike_spacing:
    k = sstar
    klist = [sstar]
    while k >= strike_spacing:
        k -= strike_spacing
        klist.append(k)
    put_k = np.array(klist)
    .num_put_options = len(put_k) - 1
else:
    put_k = np.linspace(sstar, min_strike, num_put_options + 2)
.put_strikes = put_k
max_strike = sstar + (num_call_options + 1) * strike_spacing
call_k = np.linspace(sstar, max_strike, num_call_options + 2)
.call_strikes = call_k
option_total = (2.0 * (r * t_mat - (s0 * g / sstar - 1.0) - np.log(sstar / s0)) / t_mat
)
.call_wts = np.zeros(num_call_options)
.put_wts = np.zeros(num_put_options)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_price	-	-	-
dividend_curve	-	-	-
volatility_curve	-	-	-
num_call_options	-	-	-
num_put_options	-	-	-
strike_spacing	-	-	-
discount_curve	-	-	-
use_forward	-	-	True
"""Calculate the implied variance according to the volatility surface using a static replication methodology with a specially weighted portfolio of put and call options across a range of strikes using the approximate method set out by Demeterfi et al. 1999."""			-
	-	-	-
	-	-	-
	-	-	-
.num_put_options	-	-	num_put_o
.num_call_options	-	-	num_call.o
call_type	-	-	OptionTypes.EURC
put_type	-	-	OptionTypes.EURP
t_mat	-	-	(.maturity_dt - value_dt) /
df	-	-	discount_curve.
r	-	-	-np.log(df)
dq	-	-	dividend_curve.
q	-	-	-np.log(dq)
s0	-	-	stock_pr
g	-	-	np.exp((r - q)
fwd	-	-	stock_pric
if use_forward is True	-	-	-
sstar	-	-	fwd
else	-	-	-
sstar	-	-	stock_pr
min_strike	-	-	sstar - (num_put_options -
.put_wts	-	-	[]
.put_strikes	-	-	[]
.call_wts	-	-	[]
.call_strikes	-	-	[]
if min_strike > strike_spacing	-	-	-
k	-	-	sstar
klist	-	-	[sstar
while k <=	-	-	strike_spa
k =	-	-	strike_spa
klist.append(k)	-	-	-
put_k	-	-	np.array(l
.num_put_options	-	-	len(put_k
else	-	-	-
put_k	-	-	np.linspace(sstar, min_strike
.put_strikes	-	-	put_k
max_strike	-	-	sstar + (num_call_options
call_k	-	-	np.linspace(sstar, max_strike
.call_strikes	-	-	call.k
)	-	-	-
.call_wts	-	-	np.zeros(num_ca
.put_wts	-	-	np.zeros(num

f*PLEASE ADD A FUNCTION DESCRIPTION*`f(x):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

realised_variance

Calculate the realised variance according to market standard calculations which can either use log or percentage returns.

`realised_variance(close_prices, use_logs=True):`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
close_prices	-	-	-
use_logs	-	-	True

print_weights

Print the list of puts and calls used to replicate the static replication component of the variance swap hedge.

`print_weights():`

The function arguments are described in the following table.

__repr__*PLEASE ADD A FUNCTION DESCRIPTION*`__repr__():`

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

`_print():`

The function arguments are described in the following table.

4.22 `__init__`

Chapter 5

financepy.products.credit

This folder contains a set of credit-related assets ranging from CDS to CDS options, to CDS indices, CDS index options and then to CDS tranches. They are as follows:

- CDS is a credit default swap contract. It includes schedule generation, contract valuation and risk-management functionality.
- CDSBasket is a credit default basket such as a first-to-default basket. The class includes valuation according to the Gaussian copula.
- CDSCurve is a discount curve and survival curve constructed from discount rates and CDS spreads.
- CDSIndexOption is an option on an index of CDS such as CDX or iTraxx. A full valuation model is included.
- CDSIndexPortfolio is a portfolio of CDS contracts.
- CDSOption is an option on a single CDS. The strike is expressed in spread terms and the option is European style. It is different from an option on a CDS index option. A suitable pricing model is provided which adjusts for the risk that the reference credit defaults before the option expiry date.
- CDSTranche is a synthetic CDO tranche. This is a financial derivative which takes a loss if the total loss on the portfolio exceeds a lower threshold k_1 and which is wiped out if it exceeds a higher threshold k_2 . The value depends on the default correlation between the assets in the portfolio of credits. This also includes a valuation model based on the Gaussian copula model.

FinCDSCurve

This is a curve that has been calibrated to fit the market term structure of CDS contracts given a recovery rate assumption and a IborSingleCurve discount curve. It also contains a IborCurve object for discounting. It has methods for fitting the curve and also for extracting survival probabilities.

5.1 cds

Class: CDS

A class which manages a Credit Default Swap. It performs schedule generation and the valuation and risk management of CDS.

CDS

PLEASE ADD A FUNCTION DESCRIPTION

```
CDS(step_in_dt: Date, # Date protection starts
maturity_dt_or_tenor: Union[Date, str], # Date or tenor
running_cpn: float, # Annualised cpn on premium fee leg
notional: float = ONE_MILLION,
long_protect: bool = True,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create a CDS from the step-in date, maturity date and cpn"""
check_argument_types(.CDS, locals())
if isinstance(maturity_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
step_in_dt	Date	Date protection starts	-
maturity_dt_or_tenor	Date or str	Date or tenor	-
running_cpn	float	Annualised cpn on premium fee leg	-
notional	float	-	ONE_MILLION
long_protect	bool	-	True
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
"""Create a CDS from the step-in date maturity date and cpn"""	-	-	-
check_argument_types(.CDS, locals())	-	-	-

generate_adjusted_cds_payment_dts

Generate CDS payment dates which have been holiday adjusted.

```
_generate_adjusted_cds_payment_dts():
```

The function arguments are described in the following table.

`_calc_flows`

Calculate cash flow amounts on premium leg.

```
_calc_flows():
```

The function arguments are described in the following table.

value

Valuation of a CDS contract on a specific valuation date given an issuer curve and a contract recovery rate.

```
value(value_dt,
issuer_curve,
contract_recovery_rate,
pv01_method=0,
prot_method=0,
num_steps_per_year=GLOB_NUM_STEPS_PER_YEAR,
"""Valuation of a CDS contract on a specific valuation date given
an issuer curve and a contract recovery rate."""
rpv01 = .risky_pv01(value_dt, issuer_curve, pv01_method)
dirty_rpv01 = rpv01["dirty_rpv01"]
clean_rpv01 = rpv01["clean_rpv01"]
prot_pv = .prot_leg_pv(value_dt,
    issuer_curve,
    contract_recovery_rate,
    num_steps_per_year,
    prot_method,
)
fwd_df = 1.0
if .long_protect:
    long_prot = +1
else:
    long_prot = -1
dirty_pv = (fwd_df
    * long_prot
    * (prot_pv - .running_cpn * dirty_rpv01 * .notional)
)
clean_pv = (fwd_df
    * long_prot
    * (prot_pv - .running_cpn * clean_rpv01 * .notional)
)
return {"dirty_pv": dirty_pv, "clean_pv": clean_pv}
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	0
prot_method	-	-	0
num_steps_per_year	-	-	GLOB.NUM.STEPS_PER_YEAR
"""Valuation of a CDS contract on a specific valuation date given an issuer curve and a contract recovery rate."""			
rpv01	-	-	.risky_pv01(value_dt, issuer_curve, contract_recovery_rate, num_steps_per_year, prot_method)
dirty_rpv01	-	-	rpv01["dirty_pv"]
clean_rpv01	-	-	rpv01["clean_pv"]
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
num_steps_per_year	-	-	-
prot_method	-	-	-
)	-	-	-
fwd_df	-	-	1.0
if .long_protect	-	-	-
long_prot	-	-	+1
else	-	-	-
long_prot	-	-	-1
* long_prot	-	-	-
* (prot_pv - .running_cpn * dirty_rpv01 * .notional)	-	-	-
)	-	-	-
* long_prot	-	-	-
* (prot_pv - .running_cpn * clean_rpv01 * .notional)	-	-	-
)	-	-	-
return "dirty_pv"	dirty_pv	-	-
"clean_pv"	clean_pv	-	-

credit_dv01

Calculation of the change in the value of the CDS contract for a one basis point change in the level of the CDS curve.

```
credit_dv01(value_dt,
            issuer_curve,
            contract_recovery_rate,
            pv01_method=0,
            prot_method=0,
            num_steps_per_year=GLOB.NUM.STEPS_PER_YEAR,
            """Calculation of the change in the value of the CDS contract for a
            one basis point change in the level of the CDS curve."""
            v0 = .value(value_dt,
                        issuer_curve,
                        contract_recovery_rate,
                        pv01_method,
```

```

        prot_method,
        num_steps_per_year,
    )
    bump = 0.0001 # 1 basis point
    # we create a deep copy to avoid state issues
    bumped_issuer_curve = deepcopy(issuer_curve)
    for cds in bumped_issuer_curve.cds_contracts:
        cds.running_cpn += bump
    bumped_issuer_curve.build_curve()
    v1 = .value(value_dt,
                bumped_issuer_curve,
                contract_recovery_rate,
                pv01_method,
                prot_method,
                num_steps_per_year,
    )
    credit_dv01 = v1["dirty_pv"] - v0["dirty_pv"]
    return credit_dv01
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	0
prot_method	-	-	0
num_steps_per_year	-	-	GLOB.NUM.STEPS.PER.YEAR
"""Calculation of the change in the value of the CDS contract for a one basis point change in the level of the CDS curve."""			
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	-
prot_method	-	-	-
num_steps_per_year	-	-	-
)	-	-	-
bump	-	1 basis point	0.0001
bumped_issuer_curve	-	-	deepcopy(issuer_curve)
for cds in bumped_issuer_curve.cds_contracts	-	-	-
cds.running_cpn +	-	-	bump
bumped_issuer_curve.build_curve()	-	-	-
bumped_issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	-
prot_method	-	-	-
num_steps_per_year	-	-	-
)	-	-	-
credit_dv01	-	-	v1["dirty_pv"] - v0["dirty_pv"]
return credit_dv01	-	-	-

interest_dv01

PLEASE ADD A FUNCTION DESCRIPTION

```

interest_dv01(value_dt: Date,
issuer_curve,
contract_recovery_rate,
pv01_method: int = 0,
prot_method: int = 0,
num_steps_per_year=GLOB_NUM_STEPS_PER_YEAR,
"""Calculation of the interest DV01 based on a simple bump of
the discount factors and reconstruction of the CDS curve."""
v0 = .value(value_dt,
            issuer_curve,
            contract_recovery_rate,
            pv01_method,
            prot_method,
            num_steps_per_year,
)

```

```

# we create a deep copy to avoid state issues
new_issuer_curve = deepcopy(issuer_curve)
bump = 0.0001 # 1 basis point
for depo in new_issuer_curve.libor_curve.used_deposits:
    depo.deposit_rate += bump
for fra in new_issuer_curve.libor_curve.used_fras:
    fra.fra_rate += bump
for swap in new_issuer_curve.libor_curve.used_swaps:
    cpn = swap.fixed_leg.cpn
    swap.fixed_leg.cpn = cpn + bump
    # Need to regenerate fixed leg payments with bumped cpn
    # I could call swap.fixed_leg.generate_payments() but it is
    # overkill as it has to do all the schedule generation which is
    # not needed as the dates are unchanged
    num_payments = len(swap.fixed_leg.payments)
    for i in range(0, num_payments):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	int	-	0
prot_method	int	-	0
num_steps_per_year	-	-	GLOB_NUM_STEPS_PER_YEA
"""Calculation of the interest DV01 based on a simple bump of the discount factors and reconstruction of the CDS curve."""	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	-
prot_method	-	-	-
num_steps_per_year	-	-	-
)	-	-	-
new_issuer_curve	-	-	deepcopy(issuer_curve)
bump	-	1 basis point	0.0001
for depo in new_issuer_curve.libor_curve.used_deposits	-	-	-
depo.deposit_rate +	-	-	bump
for fra in new_issuer_curve.libor_curve.used_fras	-	-	-
fra.fra_rate +	-	-	bump
for swap in new_issuer_curve.libor_curve.used_swaps	-	-	-
cpn	-	-	swap.fixed_leg.cpn
swap.fixed_leg.cpn	-	-	cpn + bump
num_payments	-	-	len(swap.fixed_leg.payments)

cash_settlement_amount

Value of the contract on the settlement date including accrued interest.

```

cash_settlement_amount(value_dt,
                        settle_dt,
                        issuer_curve,
                        contract_recovery_rate,
                        pv01_method=0,
                        prot_method=0,
                        num_steps_per_year=GLOB_NUM_STEPS_PER_YEAR,
                        """Value of the contract on the settlement date including accrued
                        interest."""
                        v = .value(value_dt,
                                issuer_curve,
                                contract_recovery_rate,
                                pv01_method,
                                prot_method,
                                num_steps_per_year,
                                )
                        libor_curve = issuer_curve.libor_curve
                        df = libor_curve.df(settle_dt)
                        v = v / df
                        return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
settle_dt	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	0
prot_method	-	-	0
num_steps_per_year	-	-	GLOB_NUM_STEPS_PER_YEAR
"""Value of the contract on the settlement date including accrued interest."""	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	-
prot_method	-	-	-
num_steps_per_year	-	-	-
)	-	-	-
libor_curve	-	-	issuer_curve.libor_curve
df	-	-	libor_curve.df(settle_dt)
v	-	-	v / df
return v	-	-	-

clean_price

Value of the CDS contract excluding accrued interest.

```

clean_price(value_dt,
            issuer_curve,
            contract_recovery_rate,
            pv01_method=0,
            prot_method=0,
            num_steps_per_year=GLOB_NUM_STEPS_PER_YEAR,
            """Value of the CDS contract excluding accrued interest."""
            risky_pv01 = .risky_pv01(value_dt, issuer_curve, pv01_method)
            clean_rpv01 = risky_pv01["clean_rpv01"]
            prot_pv = .prot_leg_pv(value_dt,
                                   issuer_curve,
                                   contract_recovery_rate,
                                   num_steps_per_year,
                                   prot_method,
                                   )
            fwd_df = 1.0
            clean_pv = fwd_df * (prot_pv - .running_cpn * clean_rpv01 * .notional)
            clean_price = (.notional - clean_pv) / .notional * 100.0
            return clean_price
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
pv01_method	-	-	0
prot_method	-	-	0
num_steps_per_year	-	-	GLOB_NUM_STEPS_P
"""Value of the CDS contract excluding accrued interest."""	-	-	-
risky_pv01	-	-	.risky_pv01(value_dt, issuer_curve,
clean_rpv01	-	-	risky_pv01["clean_r
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
num_steps_per_year	-	-	-
prot_method	-	-	-
)	-	-	-
fwd_df	-	-	1.0
clean_pv	-	-	fwd_df * (prot_pv - .running_cpn * c
clean_price	-	-	(.notional - clean_pv) / .no
return clean_price	-	-	-

accrued_days

Number of days between the previous coupon and the current step in date.

```
accrued_days(settle_dt):
```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	-	-	-

accrued_interest

Calculate the amount of accrued interest that has accrued from the previous cpn date (PCD) to the step_in_dt of the CDS contract.

```
accrued_interest(settle_dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	-	-	-

prot_leg_pv

Calculates the protection leg PV of the CDS by calling into the fast NUMBA code that has been defined above.

```
prot_leg_pv(value_dt,
            issuer_curve,
            contract_recovery_rate=STANDARD_RECOVERY_RATE,
            num_steps_per_year=GLOB_NUM_STEPS_PER_YEAR,
            prot_method=0,
            """Calculates the protection leg PV of the CDS by calling into the
            fast NUMBA code that has been defined above."""
            t_eff = (.step_in_dt - value_dt) / G_DAYS_IN_YEAR
            # An existing contract may have protection which as of now started in the past
            # We need to ensure that looking forward the protection starts immediately
            if t_eff < 0.0:
                t_eff = 0.0
            t_mat = (.maturity_dt - value_dt) / G_DAYS_IN_YEAR
            libor_curve = issuer_curve.libor_curve
            v = _prot_leg_pv_numba(t_eff,
                                   t_mat,
                                   libor_curve.times,
                                   libor_curve.dfs,
                                   issuer_curve.times,
                                   issuer_curve.qs,
                                   contract_recovery_rate,
                                   num_steps_per_year,
                                   prot_method,
                                   )
            return v * .notional
            #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	STANDARD_RECOVER
num_steps_per_year	-	-	GLOB_NUM_STEPS_P
prot_method	-	-	0
"""Calculates the protection leg PV of the CDS by calling into the			
fast NUMBA code that has been defined above."""			
t_eff	-	-	(.step_in_dt - value_dt) / G.D
if t_eff > 0.0	-	-	-
t_eff	-	-	0.0
t_mat	-	-	(.maturity_dt - value_dt) / G.D
libor_curve	-	-	issuer_curve.libor.c
t_mat	-	-	-
libor_curve.times	-	-	-
libor_curve.dfs	-	-	-
issuer_curve.times	-	-	-
issuer_curve.qs	-	-	-
contract_recovery_rate	-	-	-
num_steps_per_year	-	-	-
prot_method	-	-	-
)	-	-	-
return v * .notional	-	-	-

get_pcd

Get the previous coupon date before the value date

```
get_pcd(value_dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-

risky_pv01

The risky_pv01 is the present value of a risky one dollar paid on the premium leg of a CDS contract.

```
risky_pv01(value_dt, issuer_curve, pv01_method=0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
pv01_method	-	-	0

premium_leg_pv

Value of the premium leg of a CDS.

```
premium_leg_pv(value_dt, issuer_curve, pv01_method=0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
pv01_method	-	-	0

par_spread

Breakeven CDS cpn that would make the value of the CDS contract equal to zero.

```
par_spread(value_dt,
issuer_curve,
contract_recovery_rate=STANDARD_RECOVERY_RATE,
num_steps_per_year=GLOB_NUM_STEPS_PER_YEAR,
pv01_method=0,
prot_method=0,
"""Breakeven_CDS_cpn_that_would_make_the_value_of_the_CDS_contract
equal_to_zero."""
clean_rpv01 = .risky_pv01(value_dt, issuer_curve, pv01_method)[
    "clean_rpv01"
]
#         print("cleanRPV01", clean_rpv01)
prot = .prot_leg_pv(value_dt,
    issuer_curve,
    contract_recovery_rate,
    num_steps_per_year,
    prot_method,
)
#         print("prot", prot)
# By convention this is calculated using the clean RPV01
spd = prot / clean_rpv01 / .notional
return spd
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	STANDARD_RECOVER
num_steps_per_year	-	-	GLOB_NUM_STEPS_PE
pv01_method	-	-	0
prot_method	-	-	0
"""Breakeven CDS cpn that would make the value of the CDS contract equal to zero."""	-	-	-
”clean_rpv01”	-	-	-
]	-	-	-
issuer_curve	-	-	-
contract_recovery_rate	-	-	-
num_steps_per_year	-	-	-
prot_method	-	-	-
)	-	-	-
spd	-	-	prot / clean_rpv01 / .ne
return spd	-	-	-

value_fast_approx

Implementation of fast valuation of the CDS contract using an accurate approximation that avoids curve building.

```

value_fast_approx(value_dt,
flat_cont_interest_rate,
flat_cds_curve_spread,
curve_recovery=STANDARD_RECOVERY_RATE,
contract_recovery_rate=STANDARD_RECOVERY_RATE,
"""Implementation_of_fast_valuation_of_the_CDS_contract_using_an
accurate_approximation_that_avoids_curve_building."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_must_be_a_Date_and_not_" + str(value_dt))
t_mat = (.maturity_dt - value_dt) / G_DAYS_IN_YEAR
t_eff = (.step_in_dt - value_dt) / G_DAYS_IN_YEAR
# If protection started in past it now starts immediately
if t_eff < 0.0:
    t_eff = 0.0
h = flat_cds_curve_spread / (1.0 - curve_recovery)
r = flat_cont_interest_rate
fwd_df = 1.0
bump_size = 0.0001
if .long_protect:
    long_protect = +1
else:
    long_protect = -1
# The sign of the accrued has already been sign adjusted for direction
accrued = .accrued_interest(value_dt)
# This is the clean RPV01 as it treats the PV01 stream as though it
# pays just the accrued for the time between 0 and the maturity
# It therefore omits the part that has accrued

```

```

w = r + h
z = np.exp(-w * t_eff) - np.exp(-w * t_mat)
clean_rpv01 = (z / w) * 365.0 / 360.0
prot_pv = h * (1.0 - contract_recovery_rate) * (z / w) * .notional
clean_pv = (fwd_df
            * long_protect
            * (prot_pv - .running_cpn * clean_rpv01 * .notional)
)
full_pv = clean_pv + fwd_df * accrued
#####
# bump CDS spread and calculate
#####
h = (flat_cds_curve_spread + bump_size) / (1.0 - contract_recovery_rate)
r = flat_cont_interest_rate
w = r + h
z = np.exp(-w * t_eff) - np.exp(-w * t_mat)
clean_rpv01 = (z / w) * 365.0 / 360.0
prot_pv = h * (1.0 - contract_recovery_rate) * (z / w) * .notional
clean_pv_credit_bumped = (fwd_df
                          * long_protect
                          * (prot_pv - .running_cpn * clean_rpv01 * .notional)
)
full_pv_credit_bumped = clean_pv_credit_bumped + fwd_df * long_protect * accrued
credit01 = full_pv_credit_bumped - full_pv
#####
# bump Rate and calculate
#####
h = flat_cds_curve_spread / (1.0 - contract_recovery_rate)
r = flat_cont_interest_rate + bump_size
w = r + h
z = np.exp(-w * t_eff) - np.exp(-w * t_mat)
clean_rpv01 = (z / w) * 365.0 / 360.0
prot_pv = h * (1.0 - contract_recovery_rate) * (z / w) * .notional
clean_pv_ir_bumped = (fwd_df
                     * long_protect
                     * (prot_pv - .running_cpn * clean_rpv01 * .notional)
)
full_pv_ir_bumped = clean_pv_ir_bumped + fwd_df * long_protect * accrued
ir01 = full_pv_ir_bumped - full_pv
return (full_pv, clean_pv, credit01, ir01)
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
flat_cont_interest_rate	-	-
flat_cds_curve_spread	-	-
curve_recovery	-	-
contract_recovery_rate	-	-
"""Implementation of fast valuation of the CDS contract using an	-	-
accurate approximation that avoids curve building."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date must be a Date and not " + str(value_dt))	-	-
t_mat	-	(.maturity_dt -
t_eff	-	(.step_in_dt -
if t_eff > 0.0	-	-
t_eff	-	-
h	-	flat_cds_curve
r	-	flat
fwd_df	-	-
bump_size	-	-
if .long_protect	-	-
long_protect	-	-
else	-	-
long_protect	-	-
accrued	-	.accrued
w	-	-
z	-	np.exp(-w
clean_rpv01	-	(z
prot_pv	-	h * (1.0 - contract
* long_protect	-	-
* (prot_pv - .running_cpn * clean_rpv01 * .notional)	-	-
)	-	-
full_pv	-	clean
h	-	(flat_cds_curve_spread +
r	-	flat
w	-	-
z	-	np.exp(-w
clean_rpv01	-	(z
prot_pv	-	h * (1.0 - contract
* long_protect	-	-
* (prot_pv - .running_cpn * clean_rpv01 * .notional)	-	-
)	-	-
full_pv_credit_bumped	-	clean_pv_credit_bum
credit01	-	full_pv
h	-	flat_cds_curve_spr
r	-	flat_cont
w	-	-
z	-	np.exp(-w
clean_rpv01	-	(z
prot_pv	-	h * (1.0 - contract
* long_protect	-	-
* (prot_pv - .running_cpn * clean_rpv01 * .notional)	-	-
)	-	-
full_pv_in_bumped	-	clean_pv_in_bum

print_payments

We only print payments after the current valuation date

```
print_payments(value_dt, issuer_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-

__repr__

print out details of the CDS contract and all the calculated cash flows

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

_risky_pv01_numba

PLEASE ADD A FUNCTION DESCRIPTION

```
_risky_pv01_numba(
    t_eff,
    accrual_factor_pcd_to_now,
    payment_times,
    year_fracs,
    np_ibor_times,
    np_ibor_values,
    np_surv_times,
    np_surv_values,
    pv01_method,
    """Fast calculation of the risky PV01 of a CDS using NUMBA.
    The output is a numpy array of the full and clean risky PV01."""
    method = InterpTypes.FLAT_FWD_RATES.value
    debug = False
    if debug:
        print("=====")
        print("t_eff", t_eff)
```

```

print("Acc", accrual_factor_pcd_to_now)
print("Payments", payment_times)
print("Alphas", year_frac)
print("QTimes", np_surv_times)
print("QValues", np_surv_values)
cpn_accd_indicator = 1
# Method 0 : This is the market standard which assumes that the cpn
# accrued is treated as though on average default occurs roughly midway
# through a cpn period.
t_ncd = payment_times[0]
# The first cpn is a special case which needs to be handled carefully
# taking into account what cpn has already accrued and what has not
qeff = _uinterpolate(t_eff, np_surv_times, np_surv_values, method)
q1 = _uinterpolate(t_ncd, np_surv_times, np_surv_values, method)
z1 = _uinterpolate(t_ncd, np_ibor_times, np_ibor_values, method)
# this is the part of the cpn accrued from previous cpn date to now
# accrual_factor_pcd_to_now = day_count.year_frac(pcd,teff)
# reference credit survives to the premium payment date
full_rpv01 = q1 * z1 * year_frac[1]
# cpn accrued from previous cpn to today paid in full at default
# before cpn payment
dq = qeff - q1
full_rpv01 += z1 * dq * accrual_factor_pcd_to_now * cpn_accd_indicator
# future accrued from now to cpn payment date assuming default roughly
# midway
full_rpv01 += (
    0.5 * z1 * dq * (year_frac[1] - accrual_factor_pcd_to_now) * cpn_accd_indicator
)
for it in range(1, len(payment_times)):

```

The function arguments are described in the following table.

Argument Name	Type	Description
t_eff	-	-
accrual_factor_pcd_to_now	-	-
payment_times	-	-
year_frcs	-	-
np_ibor_times	-	-
np_ibor_values	-	-
np_surv_times	-	-
np_surv_values	-	-
pv01_method	-	-
"""Fast calculation of the risky PV01 of a CDS using NUMBA. The output is a numpy array of the full and clean risky PV01."""	-	-
method	-	Interp
debug	-	-
if debug	-	-
print("t_eff", t_eff)	-	-
print("Acc", accrual_factor_pcd_to_now)	-	-
print("Payments", payment_times)	-	-
print("Alphas", year_frcs)	-	-
print("QTimes", np_surv_times)	-	-
print("QValues", np_surv_values)	-	-
cpn_accd_indicator	-	-
t_ncd	-	-
qeff	-	_uinterpolate(t.n
q1	-	_uinterpolate(t.n
z1	-	_uinterpolate(t.n
full_rpv01	-	-
dq	-	-
full_rpv01 +	-	z1 * dq * accru
0.5 * z1 * dq * (year_frcs[1] - accrual_factor_pcd_to_now) * cpn_accd_indicator	-	-
)	-	-

`_prot_leg_pv_numba`

PLEASE ADD A FUNCTION DESCRIPTION

```
_prot_leg_pv_numba(
    t_eff,
    t_mat,
    np_ibor_times,
    np_ibor_values,
    np_surv_times,
    np_surv_values,
    contract_recovery_rate,
    num_steps_per_year,
    prot_method,
```

```

"""Fast calculation of the CDS protection leg PV using NUMBA to speed up
the numerical integration over time."""
if t_eff < 0.0:
    raise FinError("Error: Protection leg starts in past: t_eff < 0")
method = InterpTypes.FLAT_FWD_RATES.value
dt = 1.0 / num_steps_per_year
num_steps = int((t_mat - t_eff) * num_steps_per_year + 0.50)
dt = (t_mat - t_eff) / num_steps
t = t_eff
z1 = _uinterpolate(t, np_ibor_times, np_ibor_values, method)
q1 = _uinterpolate(t, np_surv_times, np_surv_values, method)
prot_pv = 0.0
small = 1e-8
if USE_FLAT_HAZARD_RATE_INTEGRAL:
    log_z1 = np.log(z1)
    log_q1 = np.log(q1)
    for _ in range(0, num_steps):

```

The function arguments are described in the following table.

Argument Name	Type	Description
t_eff	-	-
t_mat	-	-
np_ibor_times	-	-
np_ibor_values	-	-
np_surv_times	-	-
np_surv_values	-	-
contract_recovery_rate	-	-
num_steps_per_year	-	-
prot_method	-	-
"""Fast calculation of the CDS protection leg PV using NUMBA to speed up the numerical integration over time."""	-	-
if t_eff < 0.0	-	-
raise FinError("Error: Protection leg starts in past: t_eff < 0")	-	-
method	-	-
dt	-	-
num_steps	-	-
dt	-	-
t	-	-
z1	-	-
q1	-	-
prot_pv	-	-
small	-	-
if USE_FLAT_HAZARD_RATE_INTEGRAL	-	-
log_z1	-	-
log_q1	-	-

5.2 cds_basket

Class: CDSBasket

CDSBasket

PLEASE ADD A FUNCTION DESCRIPTION

```
CDSBasket(step_in_dt: Date,
maturity_dt: Date,
notional: float = ONE_MILLION,
running_cpn: float = 0.0,
long_protect: bool = True,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
check_argument_types(CDSBasket, locals()))
.step_in_dt = step_in_dt
.maturity_dt = maturity_dt
.notional = notional
.running_cpn = running_cpn / 10000.0
.long_protect = long_protect
.dc_type = dc_type
.dg_type = dg_type
.cal_type = cal_type
.freq_type = freq_type
.bd_type = bd_type
.cds_contract = CDS(.step_in_dt,
    .maturity_dt,
    .running_cpn,
    1.0,
    .long_protect,
    .freq_type,
    .dc_type,
    .cal_type,
    .bd_type,
    .dg_type,
)
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
step_in_dt	Date	-	-
maturity_dt	Date	-	-
notional	float	-	ONE_MILLION
running_cpn	float	-	0.0
long_protect	bool	-	True
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
check_argument_types(.CDSBasket, locals())	-	-	-
.step_in_dt	-	-	step_in_dt
.maturity_dt	-	-	maturity_dt
.notional	-	-	notional
.running_cpn	-	-	running_cpn / 10000.0
.long_protect	-	-	long_protect
.dc_type	-	-	dc_type
.dg_type	-	-	dg_type
.cal_type	-	-	cal_type
.freq_type	-	-	freq_type
.bd_type	-	-	bd_type
.maturity_dt	-	-	-
.running_cpn	-	-	-
1.0	-	-	-
.long_protect	-	-	-
.freq_type	-	-	-
.dc_type	-	-	-
.cal_type	-	-	-
.bd_type	-	-	-
.dg_type	-	-	-
)	-	-	-

value_legs_mc_old

PLEASE ADD A FUNCTION DESCRIPTION

```

value_legs_mc_old(value_dt, n_to_default, default_times, issuer_curves, libor_curve
"""Value_the_legs_of_the_default_basket_using_Monte_Carlo._The_default
times_are_an_input_so_this_valuation_is_not_model_dependent."""
num_credits, num_trials = default_times.shape
payment_dts = .cds_contract.payment_dts
num_payments = len(payment_dts)
day_count = DayCount(.dc_type)
avg_acc_factor = 0.0
rpv01_to_times = np.zeros(num_payments)
for i_time in range(1, num_payments):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
n_to_default	-	-	-
default_times	-	-	-
issuer_curves	-	-	-
libor_curve	-	-	-
"""Value the legs of the default basket using Monte Carlo. The default times are an input so this valuation is not model dependent."""	-	-	-
num_credits	-	-	-
num_trials	-	-	default_times.shape
payment_dts	-	-	.cds_contract.payment_dts
num_payments	-	-	len(payment_dts)
day_count	-	-	DayCount(.dc_type)
avg_acc_factor	-	-	0.0
rpv01_to_times	-	-	np.zeros(num_payments)

value_legs_mc

Value the premium PV01 and protection legs of an n-to-default basket via Monte Carlo. 'default_times' is (num_credits x num_trials) of default times in YEARS from value_dt. The valuation is pathwise (no model dependence beyond the given default times).

```
value_legs_mc(value_dt, n_to_default, default_times, issuer_curves, libor_curve
"""
Value the premium PV01 and protection legs of an n-to-default basket via Monte Carlo.
'default_times' is (num_credits x num_trials) of default times in YEARS from value_dt.
The valuation is pathwise (no model dependence beyond the given default times).
"""
# TODO: You can do it much faster by vectorizing on trials
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	-	-	-
n_to_default	-	-	-
default_times	-	-	-
issuer_curves	-	-	-
libor_curve	-	-	-
"""	-	-	-
Value the premium PV01 and protection legs of an n-to-default basket via Monte Carlo.	-	-	-
'default_times' is (num_credits x num_trials) of default times in YEARS from value_dt.	-	-	-
The valuation is pathwise (no model dependence beyond the given default times).	-	-	-
"""	-	-	-

to_years

Convert date difference into year fraction.

```
to_years(dt, value_dt, days_in_year=365.0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-
value_dt	-	-	-
days_in_year	-	-	365.0

value_gaussian_mc

Value the default basket using a Gaussian copula model. This depends on the issuer discount and correlation matrix.

```
value_gaussian_mc(value_dt,
n_to_default,
issuer_curves,
corr_matrix,
libor_curve,
num_trials,
seed,
"""Value the default basket using a Gaussian copula model. This
depends on the issuer discount and correlation matrix."""
num_credits = len(issuer_curves)
if n_to_default > num_credits or n_to_default < 1:
    raise FinError("n_to_default must be 1 to num_credits")
default_times = default_times_gc(issuer_curves, corr_matrix, num_trials, seed)
rpv01, prot_pv = .value_legs_mc(value_dt, n_to_default, default_times, issuer_curves,
    libor_curve
)
spd = prot_pv / rpv01
value = .notional * (prot_pv - .running_cpn * rpv01)
if not .long_protect:
    value = value * -1.0
return (value, rpv01, spd)
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	-	-	-
n_to_default	-	-	-
issuer_curves	-	-	-
corr_matrix	-	-	-
libor_curve	-	-	-
num_trials	-	-	-
seed	-	-	-
"""Value the default basket using a Gaussian copula model. This depends on the issuer discount and correlation matrix."""	-	-	-
num_credits	-	-	len(issuer
if n_to_default > num_credits or n_to_default < 1:	-	-	-
raise FinError("n_to_default must be 1 to num_credits")	-	-	-
default_times	-	-	default_times_gc(issuer_curves,
rpv01	-	-	-
)	-	-	-
spd	-	-	prot_pv /
value	-	-	.notional * (prot_pv - .
if not .long_protect	-	-	-
value	-	-	value *
return (value, rpv01, spd)	-	-	-

value_student_t_mc

Value the default basket using the Student-T copula.

```

value_student_t_mc(value_dt,
n_to_default,
issuer_curves,
corr_matrix,
degrees_of_freedom,
libor_curve,
num_trials,
seed,
"""Value the default basket using the Student-T copula."""
num_credits = len(issuer_curves)
if n_to_default > num_credits or n_to_default < 1:
    raise FinError("n_to_default must be 1 to num_credits")
model = StudentTCopula()
default_times = model.default_times(issuer_curves, corr_matrix, degrees_of_freedom,
    num_trials, seed
)
rpv01, prot_pv = .value_legs_mc(value_dt, n_to_default, default_times, issuer_curves,
    libor_curve
)
spd = prot_pv / rpv01
value = .notional * (prot_pv - .running_cpn * rpv01)
if not .long_protect:
    value = value * -1.0
return (value, rpv01, spd)

```

```
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
n_to_default	-	-	-
issuer_curves	-	-	-
corr_matrix	-	-	-
degrees_of_freedom	-	-	-
libor_curve	-	-	-
num_trials	-	-	-
seed	-	-	-
"""Value the default basket using the Student-T copula."""	-	-	-
num_credits	-	-	len(issuer_curves)
if n_to_default < num_credits or n_to_default > 1	-	-	-
raise FinError("n_to_default must be 1 to num_credits")	-	-	-
model	-	-	StudentTCopula()
)	-	-	-
rpv01	-	-	-
)	-	-	-
spd	-	-	prot_pv / rpv01
value	-	-	.notional * (prot_pv - .running_cpn * rpv01)
if not .long_protect	-	-	-
value	-	-	value * -1.0
return (value, rpv01, spd)	-	-	-

value_1f_gaussian_homo

PLEASE ADD A FUNCTION DESCRIPTION

```
value_1f_gaussian_homo(value_dt,
n_to_default,
issuer_curves,
beta_vector,
libor_curve,
num_points=50,
"""Value default basket using 1_factor Gaussian copula and analytical
approach which is only exact when all recovery rates are the same."""
num_credits = len(issuer_curves)
if num_credits == 0:
    raise FinError("Num_Credits_is_zero")
if n_to_default < 1 or n_to_default > num_credits:
    raise FinError("n_to_default_must_be_1_to_num_credits")
t_mat = (.maturity_dt - value_dt) / G_DAYS_IN_YEAR
if t_mat < 0.0:
    raise FinError("Value_date_is_after_maturity_date")
payment_dts = .cds_contract.payment_dts
```



```

num_times = len(payment_dts)
issuer_surv_probs = np.zeros(num_credits)
recovery_rates = np.zeros(num_credits)
basket_times = np.zeros(num_times)
basket_surv_curve = np.zeros(num_times)
basket_times[0] = 0.0
basket_surv_curve[0] = 1.0
for i_time in range(0, num_times):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
n_to_default	-	-	-
issuer_curves	-	-	-
beta_vector	-	-	-
libor_curve	-	-	-
num_points	-	-	50
"""Value default basket using 1 factor Gaussian copula and analytical approach which is only exact when all recovery rates are the same."""	-	-	-
num_credits	-	-	len(issuer_curves)
if num_credits	-	-	= 0:
raise FinError("Num Credits is zero")	-	-	-
if n_to_default > 1 or n_to_default < num_credits	-	-	-
raise FinError("n_to_default must be 1 to num_credits")	-	-	-
t_mat	-	(.maturity_dt - value_dt) / C	-
if t_mat < 0.0	-	-	-
raise FinError("Value date is after maturity date")	-	-	-
payment_dts	-	-	.cds_contract.payment_dts
num_times	-	-	len(payment_dts)
issuer_surv_probs	-	-	np.zeros(num_credits)
recovery_rates	-	-	np.zeros(num_credits)
basket_times	-	-	np.zeros(num_times)
basket_surv_curve	-	-	np.zeros(num_times)
basket_times[0]	-	-	0.0
basket_surv_curve[0]	-	-	1.0

__repr__

print out details of the CDS contract and all of the calculated cash flows

```
__repr__():
```

The function arguments are described in the following table.

5.3 cds_curve

Class: CDSCurve

Generate a survival probability curve implied by the value of CDS contracts given a Ibor curve and an assumed recovery rate. The recovery rate corresponds to the seniority of the debt for these CDS. A scheme for the interpolation of the survival probabilities is also required.

CDSCurve

Construct a credit curve from a sequence of maturity-ordered CDS contracts and a Ibor curve using the same recovery rate and the same interpolation method.

```
CDSCurve(value_dt: Date,
cds_contracts: list,
libor_curve,
recovery_rate,
use_cache: bool = False,
interp_method: InterpTypes = InterpTypes.FLAT_FWD_RATES,
"""Construct a credit curve from a sequence of maturity-ordered CDS
contracts and a Ibor curve using the same recovery rate and the
same interpolation method."""
check_argument_types(getattr(_func_name(), None), locals())
if value_dt != libor_curve.value_dt:
    raise FinError("Curve_does_not_have_same_valuation_date_as_Issuer_curve.")
.value_dt = value_dt
.cds_contracts = cds_contracts
.recovery_rate = recovery_rate
.libor_curve = libor_curve
.interp_method = interp_method
.built_ok = False
._times = []
._qs = []
if len(.cds_contracts) > 0:
    .build_curve()
else:
    pass # In some cases we allow None to be passed
return
#####
perty
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
cds_contracts	list	-
libor_curve	-	-
recovery_rate	-	-
use_cache	bool	-
interp_method	InterpTypes	-
"""Construct a credit curve from a sequence of maturity-ordered CDS	-	-
contracts and a Ibor curve using the same recovery rate and the	-	-
same interpolation method."""	-	-
check_argument_types(getattr(_func_name(), None), locals())	-	-
if value_dt !	-	-
raise FinError("Curve does not have same valuation date as Issuer curve.")	-	-
.value_dt	-	-
.cds_contracts	-	-
.recovery_rate	-	-
.libor_curve	-	-
.interp_method	-	-
.built_ok	-	-
..times	-	-
..qs	-	-
if len(.cds_contracts) < 0	-	-
.build_curve()	-	-
else	-	-
pass	-	In some cases we allow None t
return	-	-
perty	-	-

times

PLEASE ADD A FUNCTION DESCRIPTION

```
times() :
```

The function arguments are described in the following table.

qs

PLEASE ADD A FUNCTION DESCRIPTION

```
qs() :
```

The function arguments are described in the following table.

dfs*PLEASE ADD A FUNCTION DESCRIPTION*`dfs() :`

The function arguments are described in the following table.

set_times*Set the times vector*`set_times(times: np.array) :`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
times	np.array	-	-

set_qs*Set the survival probability curve*`set_qs(qvector: np.array) :`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
qvector	np.array	-	-

set_q*Set the survival probability at a specific index.*`set_q(index, q) :`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
index	-	-	-
q	-	-	-

set_last_q*Set the discount factor at last index.*`set_last_q(q) :`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
q	-	-	-

_validate

Ensure that contracts are in increasing maturity.

```
_validate(cds_contracts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cds_contracts	-	-	-

survival_prob

Extract the survival probability to date dt. This function supports vectorisation.

```
survival_prob(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

df

Extract the discount factor from the underlying Ibor curve. This function supports vectorisation.

```
df(dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

build_curve

Construct the CDS survival curve from a set of CDS contracts

```
build_curve():
```

The function arguments are described in the following table.

fwd

Calculate the instantaneous forward rate at the forward date dt using the numerical derivative.

```
fwd(dt) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-

fwd_rate

Calculate the forward rate according between dates $date1$ and $date2$ according to the specified day count convention.

```
fwd_rate(date1, date2, dc_type) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
date1	-	-	-
date2	-	-	-
dc_type	-	-	-

zero_rate

Calculate the zero rate to date dt in the chosen compounding frequency where -1 is continuous is the default.

```
zero_rate(dt, freq_type=FrequencyTypes.CONTINUOUS) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	-	-	-
freq_type	-	-	FrequencyTypes.CONTINUOUS

__repr__

Print out the details of the survival probability curve.

```
__repr__() :
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

f

Function that returns zero when the survival probability that gives a zero value of the CDS has been determined.

```
f(q, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
q	-	-	-
*args	-	-	-

5.4 cds_index_option

Class: CDSIndexOption

Class to manage the pricing and risk management of an option to enter into a CDS index. Different pricing algorithms are presented.

CDSIndexOption

PLEASE ADD A FUNCTION DESCRIPTION

```
CDSIndexOption(expiry_dt: Date,
maturity_dt: Date,
index_cpn: float,
strike_cpn: float,
notional: float = ONE_MILLION,
long_protect: bool = True,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
maturity_dt	Date	-	-
index_cpn	float	-	-
strike_cpn	float	-	-
notional	float	-	ONE_MILLION
long_protect	bool	-	True
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD

value_adjusted_black

This approach uses two adjustments to black's option pricing model to value an option on a CDS index.

```
value_adjusted_black(value_dt, index_curve, index_recovery, libor_curve, sigma
"""This approach uses two adjustments to black's option pricing
model to value an option on a CDS index."""
k = .strike_cpn
c = .index_cpn
time_to_expiry = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = libor_curve.df(.expiry_dt)
q_expiry_index = index_curve.survival_prob(time_to_expiry)
```



```

cds = CDS(value_dt, .maturity_dt, k)
strike_curve = CDSCurve(value_dt, [cds], libor_curve, index_recovery)
#           qExpiryStrike = strike_curve.surv_prob(time_to_expiry)
strike_rpv01 = .cds_contract.risky_pv01(value_dt, strike_curve)[
    "clean_rpv01"
]
index_rpv01 = .cds_contract.risky_pv01(value_dt, index_curve)["clean_rpv01"]
s = .cds_contract.par_spread(value_dt, index_curve)
fep = df * (1.0 - q_expiry_index) * (1.0 - index_recovery)
adj_fwd = s + fep / index_rpv01
adj_strike = c + (k - c) * strike_rpv01 / index_rpv01 / q_expiry_index
denom = sigma * sqrt(time_to_expiry)
d1 = log(adj_fwd / adj_strike) + 0.5 * sigma * sigma * time_to_expiry
d2 = log(adj_fwd / adj_strike) - 0.5 * sigma * sigma * time_to_expiry
d1 /= denom
d2 /= denom
v_pay = (adj_fwd * normcdf(d1) - adj_strike * normcdf(d2)) * index_rpv01
v_rec = (adj_strike * normcdf(-d2) - adj_fwd * normcdf(-d1)) * index_rpv01
v_pay *= .notional
v_rec *= .notional
return (v_pay, v_rec)
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	-	-	-
index_curve	-	-	-
index_recovery	-	-	-
libor_curve	-	-	-
sigma	-	-	-
"""This approach uses two adjustments to black's option pricing model to value an option on a CDS index."""			
k	-	-	.strike
c	-	-	.index
time_to_expiry	-	-	(.expiry_dt - value_dt)
df	-	-	libor_curve
q_expiry_index	-	-	index_curve.survival
cds	-	-	CDS(value_dt, [cds])
strike_curve	-	-	CDSCurve(value_dt, [cds])
"clean_rpv01"	-	-	-
]	-	-	-
index_rpv01	-	-	.cds_contract.risky_pv01(value_dt)
s	-	-	.cds_contract.par_spread
fep	-	-	df * (1.0 - q_expiry_index)
adj_fwd	-	-	s + fep /
adj_strike	-	-	c + (k - c) * strike_rpv01 /
denom	-	-	sigma * sqrt
d1	-	-	log(adj_fwd / adj_strike) + 0.5
d2	-	-	log(adj_fwd / adj_strike) - 0.5
d1 /	-	-	d
d2 /	-	-	d
v_pay	-	-	(adj_fwd * normcdf(d1) - adj_strike)
v_rec	-	-	(adj_strike * normcdf(-d2) - adj_fwd)
v_pay *	-	-	.no
v_rec *	-	-	.no
return (v_pay, v_rec)	-	-	-

value_anderson

This function values a CDS index option following approach by Anderson (2006). This ensures that a no-arbitrage relationship between the constituent CDS contract and the CDS index is enforced. It models the forward spread as a log-normally distributed quantity and uses the credit triangle to compute the forward RPV01.

```
value_anderson(value_dt, issuer_curves, index_recovery, sigma):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curves	-	-	-
index_recovery	-	-	-
sigma	-	-	-

_solve_for_x

PLEASE ADD A FUNCTION DESCRIPTION

```

_solve_for_x(value_dt, sigma, index_cpn, index_recovery, libor_curve, exp_h
"""Function_to_solve_for_the_arbitrage_free"""
x1 = 0.0
x2 = 0.9999
ftol = 1e-8
j_max = 40
xacc = 0.000000001
rtb = 999999
f = (._calc_obj_func(x1, value_dt, sigma, index_cpn, index_recovery, libor_curve
    )
    - exp_h
)
fmid = (._calc_obj_func(x2, value_dt, sigma, index_cpn, index_recovery, libor_curve
    )
    - exp_h
)
if f * fmid >= 0.0:
    raise FinError("Solution_not_bracketed.")
if f < 0.0:
    rtb = x1
    dx = x2 - x1
else:
    rtb = x2
    dx = x1 - x2
for _ in range(0, j_max):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
sigma	-	-	-
index_cpn	-	-	-
index_recovery	-	-	-
libor_curve	-	-	-
exp_h	-	-	-
"""Function to solve for the arbitrage free"""	-	-	-
x1	-	-	0.0
x2	-	-	0.9999
ftol	-	-	1e-8
j_max	-	-	40
xacc	-	-	0.000000001
rtb	-	-	999999
)	-	-	-
- exp_h	-	-	-
)	-	-	-
)	-	-	-
- exp_h	-	-	-
)	-	-	-
if f * fmid ζ	-	-	0.0:
raise FinError("Solution not bracketed.")	-	-	-
if f $\geq$ 0.0	-	-	-
rtb	-	-	x1
dx	-	-	x2 - x1
else	-	-	-
rtb	-	-	x2
dx	-	-	x1 - x2

`_calc_obj_func`

An internal function used in the Anderson valuation.

```
_calc_obj_func(x,
value_dt,
sigma,
index_cpn, # TODO - do I need this input ?
index_recovery,
libor_curve,
"""An internal function used in the Anderson valuation."""
# The strike value is not relevant here as we want the zeroth element
# of the return value
strike_value = 0.0
values = ._calc_index_payer_option_price(value_dt,
    x,
    sigma,
    .index_cpn,
    strike_value,
```

```

    libor_curve,
    index_recovery,
)
return values[0]
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
value_dt	-	-	-
sigma	-	-	-
index_cpn	-	TODO - do I need this input ?	-
index_recovery	-	-	-
libor_curve	-	-	-
"""An internal function used in the Anderson valuation."""	-	-	-
strike_value	-	-	0.0
x	-	-	-
sigma	-	-	-
.index_cpn	-	-	-
strike_value	-	-	-
libor_curve	-	-	-
index_recovery	-	-	-
)	-	-	-
return values[0]	-	-	-

_calc_index_payer_option_price

PLEASE ADD A FUNCTION DESCRIPTION

```

_calc_index_payer_option_price(value_dt,
x,
sigma,
index_cpn,
strike_value,
libor_curve,
index_recovery,
"""Calculates the intrinsic value of the index payer swap and the
value of the index payer option which are both returned in an array.
"""
z = -6.0
dz = 0.2
num_z_steps = int(2.0 * abs(z) / dz)
flow_dts = .cds_contract.payment_dts
num_flows = len(flow_dts)
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df_to_expiry = libor_curve.df(.expiry_dt)
lgd = 1.0 - index_recovery
fwd_dfs = [1.0] * (num_flows)
expiry_to_flow_times = [1.0] * (num_flows)

```

```
for i_flow in range(0, num_flows):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
x	-	-	-
sigma	-	-	-
index_cpn	-	-	-
strike_value	-	-	-
libor_curve	-	-	-
index_recovery	-	-	-
"""Calculates the intrinsic value of the index payer swap and the value of the index payer option which are both returned in an array.	-	-	-
"""	-	-	-
z	-	-	-6.0
dz	-	-	0.2
num_z_steps	-	-	int(2.0 * abs(z) /
flow_dts	-	-	.cds_contract.payme
num_flows	-	-	len(flow_dts)
t_exp	-	-	(.expiry_dt - value_dt) / G_DA
df_to_expiry	-	-	libor_curve.df(.expi
lgd	-	-	1.0 - index_recov
fwd_dfs	-	-	[1.0] * (num_flow
expiry_to_flow_times	-	-	[1.0] * (num_flow

__repr__

print out details of the CDS contract and all of the calculated cash flows

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

5.5 cds_index_portfolio

Class: CDSIndexPortfolio

This class manages the calculations associated with an equally weighted portfolio of CDS contracts with the same maturity date.

CDSIndexPortfolio

Create *Fincds_index_portfolio* object. Note that all of the inputs have a default value which reflects the CDS market standard.

```
CDSIndexPortfolio(freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create_Fincds_index_portfolio_object._Note_that_all_of_the_inputs
have_a_default_value_which_reflects_the_CDS_market_standard."""
check_argument_types(.CDSIndexPortfolio, locals())
.dc_type = dc_type
.dg_type = dg_type
.cal_type = cal_type
.freq_type = freq_type
.bd_type = bd_type
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
"""Create_Fincds_index_portfolio_object. Note that all of the inputs have a default value which reflects the CDS market standard."""	-	-	-
check_argument_types(.CDSIndexPortfolio, locals())	-	-	-
.dc_type	-	-	-
.dg_type	-	-	-
.cal_type	-	-	-
.freq_type	-	-	-
.bd_type	-	-	-

intrinsic_rpv01

Calculation of the risky PV01 of the CDS portfolio by taking the average of the risky PV01s of each contract.

```
intrinsic_rpv01(value_dt, step_in_dt, maturity_dt, issuer_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

intrinsic_prot_leg_pv

Calculation of intrinsic protection leg value of the CDS portfolio by taking the average sum the protection legs of each contract.

```
intrinsic_prot_leg_pv(value_dt, step_in_dt, maturity_dt, issuer_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

intrinsic_spread

Calculation of the intrinsic spd of the CDS portfolio as the one which would make the value of the protection legs equal to the value of the premium legs if all premium legs paid the same spd.

```
intrinsic_spread(value_dt, step_in_dt, maturity_dt, issuer_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

average_spread

Calculates the average par CDS spd of the CDS portfolio.

```
average_spread(value_dt, step_in_dt, maturity_dt, issuer_curves):
```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

total_spread

Calculates the total CDS spd of the CDS portfolio by summing over all of the issuers and adding the spd with no weights.

```
total_spread(value_dt, step_in_dt, maturity_dt, issuer_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

min_spread

Calculates the minimum par CDS spd across all of the issuers in the CDS portfolio.

```
min_spread(value_dt, step_in_dt, maturity_dt, issuer_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

max_spread

Calculates the maximum par CDS spd across all of the issuers in the CDS portfolio.

```
max_spread(value_dt, step_in_dt, maturity_dt, issuer_curves):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
step_in_dt	-	-	-
maturity_dt	-	-	-
issuer_curves	-	-	-

spread_adjust_intrinsic

PLEASE ADD A FUNCTION DESCRIPTION

```

spread_adjust_intrinsic(value_dt,
issuer_curves,
index_cpns,
index_upfronts,
index_maturity_dts,
index_recovery_rate,
tolerance=1e-6,
"""Adjust_individual_CDS_discount_to_reprice_CDS_index_prices.
This_approach_uses_an_iterative_scheme_but_is_slow_as_it_has_to_use_a
CDS_curve_bootstrap_required_when_each_trial_spd_adjustment_is_made
"""
num_credits = len(issuer_curves)
if num_credits < 1:
    raise FinError("Number_of_credits_in_index_must_be_>_1_and_not_" + str(num_credits)
)
libor_curve = issuer_curves[0].libor_curve
num_index_maturity_points = len(index_cpns)
cds_maturity_dts = []
for cds in issuer_curves[0].cds_contracts:
    cds_dts = cds.maturity_dt
    cds_maturity_dts.append(cds_dts)
num_cds_mat_points = len(cds_maturity_dts)
for issuer_curve in issuer_curves:
    n = len(issuer_curve.cds_contracts)
    if n != len(cds_maturity_dts):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curves	-	-	-
index_cpns	-	-	-
index_upfronts	-	-	-
index_maturity_dts	-	-	-
index_recovery_rate	-	-	-
tolerance	-	-	1e-6
"""Adjust individual CDS discount to reprice CDS index prices.	-	-	-
This approach uses an iterative scheme but is slow as it has to use a	-	-	-
CDS curve bootstrap required when each trial spd adjustment is made	-	-	-
"""	-	-	-
num_credits	-	-	len(issuer_curves)
if num_credits > 1	-	-	-
)	-	-	-
libor_curve	-	-	issuer_curves[0].libor_curve
num_index_maturity_points	-	-	len(index_cpns)
cds_maturity_dts	-	-	[]
for cds in issuer_curves[0].cds_contracts	-	-	-
cds_dts	-	-	cds.maturity_dt
cds_maturity_dts.append(cds_dts)	-	-	-
num_cds_mat_points	-	-	len(cds_maturity_dts)
for issuer_curve in issuer_curves	-	-	-
n	-	-	len(issuer_curve.cds_contracts)

hazard_rate_adjust_intrinsic

PLEASE ADD A FUNCTION DESCRIPTION

```

hazard_rate_adjust_intrinsic(value_dt,
issuer_curves,
index_cpns,
index_upfronts,
index_maturity_dts,
index_recovery_rate,
tolerance=1e-6,
max_iterations=200,
"""Adjust individual CDS discount to reprice CDS index prices.
This approach adjusts the hazard rates and so avoids the slowish
CDS curve bootstrap required when a spd adjustment is made."""
if 1 == 0:
    print("=====")
    print(value_dt)
    print(index_cpns)
    print(index_upfronts)
    print(index_maturity_dts)
    print(index_recovery_rate)
num_credits = len(issuer_curves)
if num_credits < 1:
    raise FinError("Number_of_credits_must_be_greater_than_1")

```

```

libor_curve = issuer_curves[0].libor_curve
num_index_maturity_points = len(index_cpns)
adjusted_issuer_curves = []
# making a copy of the issuer discount
for issuer_curve in issuer_curves:
    adjusted_issuer_curve = CDSCurve(value_dt, [], libor_curve, index_recovery_rate
    )
    adjusted_issuer_curve.set_times(issuer_curve.times)
    adjusted_issuer_curve.set_qs(issuer_curve.qs)
    adjusted_issuer_curves.append(adjusted_issuer_curve)
# We solve for each maturity point
for i_maturity in range(0, num_index_maturity_points):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	-	-	-
issuer_curves	-	-	-
index_cpns	-	-	-
index_up_fronts	-	-	-
index_maturity_dts	-	-	-
index_recovery_rate	-	-	-
tolerance	-	-	1e-6
max_iterations	-	-	200
"""Adjust individual CDS discount to reprice CDS index prices.	-	-	-
This approach adjusts the hazard rates and so avoids the slowish	-	-	-
CDS curve bootstrap required when a spd adjustment is made."""	-	-	-
if 1	-	-	= 0
print("=====	-	-	
print(value_dt)	-	-	-
print(index_cpns)	-	-	-
print(index_up_fronts)	-	-	-
print(index_maturity_dts)	-	-	-
print(index_recovery_rate)	-	-	-
num_credits	-	-	len(issuer_
if num_credits > 1	-	-	-
raise FinError("Number of credits must be greater than 1")	-	-	-
libor_curve	-	-	issuer_curves[0]
num_index_maturity_points	-	-	len(index
adjusted_issuer_curves	-	-	[]
for issuer_curve in issuer_curves	-	-	-
)	-	-	-
adjusted_issuer_curve.set_times(issuer_curve.times)	-	-	-
adjusted_issuer_curve.set_qs(issuer_curve.qs)	-	-	-
adjusted_issuer_curves.append(adjusted_issuer_curve)	-	-	-

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

5.6 cds_option

Class: CDSOption

Class to manage the pricing and risk-management of an option on a single-name CDS. This is a contract in which the option buyer pays for an option to either buy or sell protection on the underlying CDS at a fixed spread agreed today and to be exercised in the future on a specified expiry date. The option may or may not cancel if there is a credit event before option expiry. This needs to be specified.

CDSOption

Create a FinCDSOption object with the option expiry date, the maturity date of the underlying CDS, the option strike coupon, notional, whether the option knocks out or not in the event of a credit event before expiry and the payment details of the underlying CDS.

```
CDSOption(expiry_dt: Date,
maturity_dt: Date,
strike_cpn: float,
notional: float = ONE_MILLION,
long_protection: bool = True,
knockout_flag: bool = True,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create_a_FinCDSOption_object_with_the_option_expiry_date,_the
maturity_date_of_the_underlying_CDS,_the_option_strike_coupon,
notional,_whether_the_option_knocks_out_or_not_in_the_event_of_a_credit
event_before_expiry_and_the_payment_details_of_the_underlying_CDS."""
check_argument_types(.CDSOption, locals())
if maturity_dt < expiry_dt:
    raise FinError("Maturity_date_must_be_after_option_expiry_date")
if strike_cpn < 0.0:
    raise FinError("Strike_must_be_greater_than_zero")
.expiry_dt = expiry_dt
.maturity_dt = maturity_dt
.strike_cpn = strike_cpn
.long_protection = long_protection
.knockout_flag = knockout_flag
.notional = notional
.freq_type = freq_type
.dc_type = dc_type
.cal_type = cal_type
.bd_type = bd_type
.dg_type = dg_type
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
maturity_dt	Date	-
strike_cpn	float	-
notional	float	-
long_protection	bool	-
knockout_flag	bool	-
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create a FinCDSOption object with the option expiry date	-	-
the	-	-
maturity date of the underlying CDS	-	-
the option strike coupon	-	-
notional	-	-
whether the option knocks out or not in the event of a credit	-	-
event before expiry and the payment details of the underlying CDS."""	-	-
check_argument_types(.CDSOption, locals())	-	-
if maturity_dt > expiry_dt	-	-
raise FinError("Maturity date must be after option expiry date")	-	-
if strike_cpn < 0.0	-	-
raise FinError("Strike must be greater than zero")	-	-
.expiry_dt	-	-
.maturity_dt	-	-
.strike_cpn	-	-
.long_protection	-	-
.knockout_flag	-	-
.notional	-	-
.freq_type	-	-
.dc_type	-	-
.cal_type	-	-
.bd_type	-	-
.dg_type	-	-

value

Value the CDS option using Black's model with an adjustment for any Front End Protection. TODO - Should the CDS be created in the init method ?

```
value(value_dt, issuer_curve, volatility):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
volatility	-	-	-

implied_volatility

Calculate the implied CDS option volatility from a price.

```
implied_volatility(value_dt, issuer_curve, option_value):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curve	-	-	-
option_value	-	-	-

fvol

Root searching function in the calculation of the CDS implied volatility.

```
fvol(volatility, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	-	-	-
*args	-	-	-

5.7 cds_tranche

Enumerated Type: FinLossDistributionBuilder

This enumerated type has the following values:

- RECURSION
- ADJUSTED_BINOMIAL
- GAUSSIAN
- LHP
- notional:float
- running_cpn:float
- long_protect:bool
- freq_type:FrequencyTypes
- dc_type:DayCountTypes
- cal_type:CalendarTypes
- bd_type:BusDayAdjustTypes
- dg_type:DateGenRuleTypes
- ifk1;
- self.k1
- self.k2
- self.step_in_dt
- self.maturity_dt
- self.notional
- self.running_cpn
- self.long_protect
- self.dc_type
- self.dg_type
- self.cal_type
- self.freq_type
- self.bd_type
- notional
- self.cds_contract
- num_points
- model
- num_credits
- k1
- k2
- t_mat
- output
- output[0]
- output[1]
- output[2]
- output[3]
- kappa
- recovery_rates
- payment_dts

- num_payments
- num_times
- beta_1
- beta_2
- beta_vector1
- beta_vector1[bb]
- beta_vector2
- beta_vector2[bb]
- q_vector
- qt1
- qt2
- tranche_times
- tranche_surv_curve
- tranche_times[0]
- tranche_surv_curve[0]
- qt1[0]
- qt2[0]
- t
- issuer_curve
- v_times
- q_row
- recovery_rates[j]
- q_vector[j]
- ifmodel
- qt1[i]
- qt2[i]
- elifmodel
- qt1[i]
- qt2[i]
- elifmodel
- qt1[i]
- qt2[i]
- elifmodel
- qt1[i]
- qt2[i]
- tranche_surv_curve[i]
- tranche_times[i]
- curve_recovery
- libor_curve
- tranche_curve
- prot_leg_pv
- risky_pv01
- mtm
- mtm*
- tranche_output
- tranche_output[0]

- `tranche_output[1]`
- `tranche_output[2]`
- `tranche_output[3]`

Class: CDSTranche

class CDSTranche:

CDSTranche

PLEASE ADD A FUNCTION DESCRIPTION

```

CDSTranche(step_in_dt: Date,
maturity_dt: Date,
k1: float,
k2: float,
notional: float = ONE_MILLION,
running_cpn: float = 0.0,
long_protect: bool = True,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
check_argument_types(.CDSTranche, locals()))
if k1 >= k2:
    raise FinError("k_1_must_be_less_than_k_2")
.k1 = k1
.k2 = k2
.step_in_dt = step_in_dt
.maturity_dt = maturity_dt
.notional = notional
.running_cpn = running_cpn
.long_protect = long_protect
.dc_type = dc_type
.dg_type = dg_type
.cal_type = cal_type
.freq_type = freq_type
.bd_type = bd_type
notional = 1.0
.cds_contract = CDS(.step_in_dt,
    .maturity_dt,
    .running_cpn,
    notional,
    .long_protect,
    .freq_type,
    .dc_type,
    .cal_type,
    .bd_type,
    .dg_type,
)
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
step_in_dt	Date	-	-
maturity_dt	Date	-	-
k1	float	-	-
k2	float	-	-
notional	float	-	ONE_MILLION
running_cpn	float	-	0.0
long_protect	bool	-	True
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
check_argument_types(.CDSTranche, locals())	-	-	-
if k1 <	-	-	k2:
raise FinError("k_1 must be less than k_2")	-	-	-
.k1	-	-	k1
.k2	-	-	k2
.step_in_dt	-	-	step_in_dt
.maturity_dt	-	-	maturity_dt
.notional	-	-	notional
.running_cpn	-	-	running_cpn
.long_protect	-	-	long_protect
.dc_type	-	-	dc_type
.dg_type	-	-	dg_type
.cal_type	-	-	cal_type
.freq_type	-	-	freq_type
.bd_type	-	-	bd_type
notional	-	-	1.0
.maturity_dt	-	-	-
.running_cpn	-	-	-
notional	-	-	-
.long_protect	-	-	-
.freq_type	-	-	-
.dc_type	-	-	-
.cal_type	-	-	-
.bd_type	-	-	-
.dg_type	-	-	-
)	-	-	-

value_bc

PLEASE ADD A FUNCTION DESCRIPTION

value_bc(value_dt,

```

issuer_curves,
upfront,
running_cpn,
corr1,
corr2,
num_points=50,
model=FinLossDistributionBuilder.RECURSION,
num_credits = len(issuer_curves)
k1 = .k1
k2 = .k2
t_mat = (.maturity_dt - value_dt) / G_DAYS_IN_YEAR
if t_mat < 0.0:
    raise FinError("Value_date_is_after_maturity_date")
if abs(k1 - k2) < 0.00000001:
    output = np.zeros(4)
    output[0] = 0.0
    output[1] = 0.0
    output[2] = 0.0
    output[3] = 0.0
    return output
if k1 > k2:
    raise FinError("k_1_>k_2")
kappa = k2 / (k2 - k1)
recovery_rates = np.zeros(num_credits)
payment_dts = .cds_contract.payment_dts
num_payments = len(payment_dts)
num_times = num_payments + 1
beta_1 = sqrt(corr1)
beta_2 = sqrt(corr2)
beta_vector1 = np.zeros(num_credits)
for bb in range(0, num_credits):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
issuer_curves	-	-	-
upfront	-	-	-
running_cpn	-	-	-
corr1	-	-	-
corr2	-	-	-
num_points	-	-	50
model	-	-	RECURSION
num_credits	-	-	len(issuer_curves)
k1	-	-	.k1
k2	-	-	.k2
t_mat	-	-	(.maturity_dt - value_dt) / G_DAYS_IN_YEAR
if t_mat ≤ 0.0		-	-
raise FinError("Value date is after maturity date")	-	-	-
if abs(k1 - k2) ≤ 0.00000001		-	-
output	-	-	np.zeros(4)
output[0]	-	-	0.0
output[1]	-	-	0.0
output[2]	-	-	0.0
output[3]	-	-	0.0
return output	-	-	-
if k1 > k2		-	-
raise FinError("k_1 > k_2")	-	-	-
kappa	-	-	k2 / (k2 - k1)
recovery_rates	-	-	np.zeros(num_credits)
payment_dts	-	-	.cds_contract.payment_dts
num_payments	-	-	len(payment_dts)
num_times	-	-	num_payments + 1
beta_1	-	-	sqrt(corr1)
beta_2	-	-	sqrt(corr2)
beta_vector1	-	-	np.zeros(num_credits)

5.8 `__init__`

Chapter 6

financepy.products.bonds

This folder contains a suite of bond-related functionality across a set of files and classes. They are as follows:

- Bond is a basic fixed coupon bond with all of the associated duration and convexity measures. It also includes some common spread measures such as the asset swap spread and the option adjusted spread.
- BondZero is a zero coupon bond. This is a bond issued at a deep discount that matures at par. Accrued interest is calculated by interpolating the price growth.
- BondAnnuity is a stream of cash flows that is generated and can be priced.
- BondCallable is a bond that has embedded call and put options. A number of rate models pricing functions have been included to allow such bonds to be priced and risk-managed.
- BondConvertible enables the pricing and risk-management of convertible bonds. The model is a binomial tree implementation of Black-Scholes which allows for discrete dividends, embedded puts and calls, and a delayed start of the conversion option.
- BondFRN enables the pricing and risk-management of a bond with floating rate coupons. Discount margin calculations are provided.
- BondFuture is a bond future that has functionality around determination of the conversion factor and calculation of the invoice price and determination of the cheapest to deliver.
- BondMarket is a database of country-specific bond market conventions that can be referenced. These include settlement days and accrued interest conventions.
- BondMortgage generates the periodic cash flows for an interest-only and a repayment mortgage.
- BondOption is a bond option class that includes a number of valuation models for pricing both European and American style bond options. Models for European options include a Lognormal Price, Hull-White (HW) and Black-Karasinski (BK). The HW valuation is fast as it uses Jamshidian's decomposition trick. American options can also be priced using a HW and BK trinomial tree. The details are abstracted away making it easy to use.
- BondPortfolio is a portfolio of bonds.
- Yield Curve is a class to handle bond yield curves. It uses a variety of shapes to best-fit a set of bond yields.
- Zero curve is a class to perform an exact fit to a set of provided bonds using a piece-wise flat zero rate.

Conventions

- All interest rates are expressed as a fraction of 1. So 3
- All notionals of bond positions are given in terms of a notional amount.
- All bond prices are based on a notional of 100.0.
- The face of a derivatives position is the size of the underlying position.

Bond Curves

These modules create a family of curve types related to the term structures of interest rates. These are best fit yield curves fitting to bond prices which are used for interpolation. A range of curve shapes from polynomials to B-Splines is available. This module describes a curve that is fitted to bond yields calculated from bond market prices supplied by the user. The curve is not guaranteed to fit all of the bond prices exactly and a least squares approach is used. A number of fitting forms are provided which consist of

- Polynomial
- Nelson-Siegel
- Nelson-Siegel-Svensson
- Cubic B-Splines

This fitted curve cannot be used for pricing as yields assume a flat term structure. It can be used for fitting and interpolating yields off a nicely constructed yield curve interpolation curve.

FinCurveFitMethod

This module sets out a range of curve forms that can be fitted to the bond yields. These includes a number of parametric curves that can be used to fit yield curves. These include:

- Polynomials of any degree
- Nelson-Siegel functional form.
- Nelson-Siegel-Svensson functional form.
- B-Splines

6.1 bond

Enumerated Type: CouponType

This enumerated type has the following values:

- FIXED
- ACCRUED

Enumerated Type: YTMCalcType

This enumerated type has the following values:

- ZERO
- UK_DMO
- US_STREET
- US_TREASURY
- CFETS
- CALCULUS
- bond
- settle_dt
- price
- convention
- px
- obj_fn
- bond
- settle_dt
- price
- discount_curve
- px
- obj_fn
- ex_div_days:int
- cal_type:CalendarTypes
- bd_type
- dg_type
- cpn_type
- ifissue_dt;
- self.end_of_month
- self.end_of_month
- self.issue_dt
- self.maturity_dt
- ifcoupon
- iffreq_type
- ifdc_type
- self.cpn
- self.cpn_type
- self.freq_type
- self.dc_type
- self.freq

- self.ex_div_dt
- self.ex_div_days
- self.par
- self.cal_type
- self.cpn_dts
- self.payment_dts
- self.flow_amounts
- self.bd_type
- self.dg_type
- self._pcd
- self._ncd
- self.accrued_int
- self.accrued_days
- self.alpha
- bd_type
- dg_type
- self.cpn_dts
- end_of_month
- bus_day_adj_type
- calendar
- self.payment_dts
- pmt_dt
- self.flow_amounts
- ifself.cpn_type
- flow
- elifself.cpn_type
- self_dc
- prev_dt
- yf, -, -
- prev_dt
- flow
- n_cpn_dts
- n_payment_dts
- n_flows
- ifn_cpn_dts!
- ifn_cpn_dts!
- self.cpn_dts
- self.payment_dts
- self.flow_amounts
- convention: YTMCalcType
- pay_first_cpn
- pay_first_cpn
- ytm
- ytm
- f
- c

- ifconvention
- v
- n
- n+
- n
- ifconvention
- ifn
- dp
- term1
- term2
- term3
- term4
- dp
- elifconvention
- ifn
- dp
- term1
- term2
- term3
- term4
- vw
- dp
- elifconvention
- ifn
- vw
- dp
- term1
- term2
- term3
- term4
- dp
- elifconvention
- ifn
- last_year
- dc
- alpha
- freq_type
- vw
- dp
- term1
- term2
- term3
- term4
- dp
- elifconvention
- dp

- d
- g
- flow
- n_next
- n_next+
- n_start
- n_start
- g
- dp+
- last_g
- g
- dp+
- convention:YTMCalcType
- settle_dt_list
- settle_dt_list
- ytm_list
- ytm_list
- num_dates
- num_ytms
- settle_dt_vector
- ytm_vector
- ifnum_dates
- elifnum_ytms
- elifnum_dates
- dps
- num_to_calculate
- s_dt
- y
- price
- ifnum_to_calculate
- repo_dc_type:DayCountTypes
- ifsettle_dt_i
- repo_dc
- t_fwd,--,
- accrued_settle
- accrued_forward
- fv_cpns
- ifsettle_dt_i;dt_i
- t_cpn_to_forward,--,
- g
- fv_cpns+
- full_price
- fwd_price
- fwd_price
- dirty_price
- principal

- principal
- convention:YTMCalcType
- dy
- p_minus
- p_plus
- durn
- convention:YTMCalcType
- d
- dd
- dp
- md
- md
- dp
- g
- flow
- n_next
- n_next+
- n_start
- k
- n_start
- g
- k
- t
- dp+
- md+
- last_g
- last_t
- g
- k+
- dp+
- md+
- dp
- md
- md
- convention:YTMCalcType
- dd
- fp
- md
- key_rate_tenors:list
- shift:float
- rates:list
- key_rate_tenors
- shift
- lin_zero_interp
- us_street
- key_rate_durations

- rates
- par_bonds
- mat_dt
- par_bond
- clean_prices
- clean_price
- par_crv
- p_zero
- rates[ind]+
- par_bonds
- mat
- par_bond
- clean_prices
- clean_price
- par_crv_up
- p_up
- rates[ind]-
- par_bonds
- mat
- par_bond
- clean_prices
- clean_price
- par_crv_down
- p_down
- key_rate_duration
- convention:YTMCalcType
- dy
- p0
- p1
- p2
- conv
- convention:YTMCalcType
- dp
- accrued_int
- cp
- dirty_price
- clean_price
- cal
- self.ex_div_dt
- pay_first_cpn
- pay_first_cpn
- px
- df_settle_dt
- pmt_dt
- pv
- px+

- pv
- px+
- px+
- px
- cal
- self.ex_div_dt
- px_pv
- n_flows
- cpn_amount
- cpn_dt
- pmt_dt
- df
- is_first_future_cpn
- flow
- flow
- ifi
- flow+
- px_pv+
- df_settle
- dirty_price
- ifclean_price;
- y
- convention:YTMCalcType
- clean_prices
- clean_prices
- accrued_amount
- dirty_prices
- ytms
- argtuple
- ytm
- a
- b
- args
- xtol
- maxiter
- ytm
- iflen(ytms)
- convention:YTMCalcType
- clean_prices
- clean_prices
- accrued_amount
- dirty_prices
- simple_ys
- simple_y
- iflen(simple_ys)
- num_flows

- ifnum_flows
- self._pcd
- self._ncd
- defaccrued_interest(self,settle_dt:Date,face:float
- dc
- cal
- self.ex_div_dt
- (acc_factor,num,_)
- self.alpha
- self.accrued_int
- self.accrued_int
- self.accrued_int*
- self.accrued_days
- swap_float_day_count_convention_type
- swap_float_frequency_type
- swap_float_calendar_type
- swap_float_bus_day_adjust_rule_type
- swap_float_date_gen_rule_type
- clean_price
- accrued_amount
- bond_price
- pv_ibor
- prev_dt
- df
- pv_ibor+
- pv_ibor+
- prev_dt
- schedule
- day_count
- prev_dt
- pv01
- df
- year_frac
- pv01
- prev_dt
- asw
- self.accrued_int
- accrued_amount
- bond_price
- flat_curve
- bumped_curve
- curve_bond_price
- z_spread
- x0
- fprime
- tol

- maxiter
- fprime2
- f
- c
- df_adjusted
- pv
- t
- t
- df
- r
- df_adjusted
- pv
- pv
- pv*
- f
- cpn_flow
- pv
- df_settle
- n_flows
- pmt_dt
- cpn_dt
- t
- t
- df_base
- df_oas
- df_adj
- ifcpn_dt
- flow
- flow
- ifi
- flow+
- pv+
- pv
- clean_prices
- clean_prices
- accrued_amount
- dirty_prices
- oass
- argtuple
- oas
- x0
- fprime
- args
- tol
- maxiter
- fprime2

- iflen(oass)
- f
- c
- pv
- prev_q
- prev_df
- defaulting_pv_pay_start
- defaulting_pv_pay_end
- seen_future_flow
- seen_future_flow
- df
- q
- pv
- dq
- defaulting_pv_pay_start+
- defaulting_pv_pay_end+
- prev_q
- prev_df
- pv
- pv
- pv
- pv+
- pv*
- f
- cpn_flow
- df_settle
- q_settle
- prev_df
- prev_q
- pv_coupons
- pv_recovery
- n_flows
- pmt_dt
- cpn_dt
- df
- q
- ifnot(cpn_dt
- pv_coupons+
- dq
- pv_rec_start
- pv_rec_end
- pv_recovery+
- ifi
- pv_coupons+
- prev_df
- prev_q

- Price
- total_pv
- dirty_price
- clean_price
- convention:YTMCalcType
- buy_price
- sell_price
- dts_cfs
- dts_cfs
- (d,c*self.par)for(d,c)indts_cfsif(d $\leq$
- times_cfs
- pnl
- simple_return
- brentq_up_lim
- brentq_dn_lim
- irr
- irr
- a
- b
- xtoll
- args
- s
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- defprint_payments(self,settle_dt:Date,face:float
- n_flows
- w
- header
- flow_str
- flow
- flow_str+
- n_start
- cpn_dt
- pmt_dt
- n_start
- ifn_start
- cpn_dt
- pmt_dt

- flow_str+
- n_start+
- cpn_dt
- pmt_dt
- flow
- ifi
- flow+
- flow_str+

Class: Bond

Class for fixed coupon bonds and performing related analytics. These are bullet bonds which means they have regular coupon payments of a known size that are paid on known dts plus a payment of par at maturity.

Bond

PLEASE ADD A FUNCTION DESCRIPTION

```
Bond(issue_dt: Date,
      maturity_dt: Date,
      coupon: float, # Annualised bond coupon
      freq_type: FrequencyTypes,
      dc_type: DayCountTypes,
      ex_div_days: int = 0,
      cal_type: CalendarTypes = CalendarTypes.WEEKEND,
      bd_type=BusDayAdjustTypes.FOLLOWING,
      dg_type=DateGenRuleTypes.BACKWARD,
      cpn_type=CouponType.FIXED
      """Create_Bond_object_by_providing_the_issue_date, maturity_Date,
      coupon_frequency, annualised_coupon, the_accrual_convention_type, face
      amount_and_the_number_of_ex-dividend_days. A_calendar_type_is_used
      to_determine_holidays_from_which_coupon_dts_might_be_shifted."""
      check_argument_types(.Bond, locals())
      if issue_dt >= maturity_dt:
          raise FinError("Issue_Date_must_preceded_maturity_date.")
      # If the maturity date falls on the last day of the month we assume
      # that earlier flows also fall on month ends
      .end_of_month = False
      if maturity_dt.is_eom():
```

The function arguments are described in the following table.

Argument Name	Type	Description
issue_dt	Date	-
maturity_dt	Date	-
coupon	float	Annualised bond coupon
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
ex_div_days	int	-
cal_type	CalendarTypes	-
bd_type	-	-
dg_type	-	-
cpn_type	-	-
"""Create Bond object by providing the issue date	-	-
maturity Date	-	-
coupon frequency	-	-
annualised coupon	-	-
the accrual convention type	-	-
face	-	-
amount and the number of ex-dividend days. A calendar type is used	-	-
to determine holidays from which coupon dts might be shifted."""	-	-
check_argument_types(.Bond, locals())	-	-
if issue_dt <	-	-
raise FinError("Issue Date must precede maturity date.")	-	-
.end_of_month	-	-

_calculate_cpn_dts

Determine the unadjusted bond coupon dts from the issue date to the maturity date even if they have passed. Note that for analytical calculations these are not usually adjusted and so may fall on a weekend or holiday.

```
_calculate_cpn_dts():
```

The function arguments are described in the following table.

_calculate_payment_dts

For the actual payment dts, they are adjusted, and so we then use the calendar payment dts. Although payments are calculated as though coupon periods are the same length, payments that fall on a Saturday or Sunday can only be made on the next business day

```
_calculate_payment_dts():
```

The function arguments are described in the following table.

`_calculate_flow_amounts`

Determine the bond cash flow payment amounts without principal. There is no adjustment based on the adjusted payment dts.

```
_calculate_flow_amounts():
```

The function arguments are described in the following table.

`reset_flows`

PLEASE ADD A FUNCTION DESCRIPTION

```
reset_flows(cpn_dts: List[Date], payment_dts: List[Date], flow_amounts: np.ndarray
"""Set the flows of the bond externally. Coupon dates are for accrued
while payment dates are calendar adjusted. Flows are on payment dates"""
n_cpn_dts = len(cpn_dts)
n_payment_dts = len(payment_dts)
n_flows = len(flow_amounts)
if n_cpn_dts != n_payment_dts:
    raise FinError("Number_of_coupon_dates_not_equal_to_number_payments")
if n_cpn_dts != n_flows:
    raise FinError("Number_of_coupon_dates_not_equal_to_number_flows")
for i in range(1, n_cpn_dts):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
<code>cpn_dts</code>	List[Date]	-	-
<code>payment_dts</code>	List[Date]	-	-
<code>flow_amounts</code>	np.ndarray	-	-
"""Set the flows of the bond externally. Coupon dates are for accrued while payment dates are calendar adjusted. Flows are on payment dates"""	-	-	-
<code>n_cpn_dts</code>	-	-	<code>len(cpn_dts)</code>
<code>n_payment_dts</code>	-	-	<code>len(payment_dts)</code>
<code>n_flows</code>	-	-	<code>len(flow_amounts)</code>
<code>if n_cpn_dts != n_payment_dts:</code>	-	-	<code>n_payment_dts</code>
<code>raise FinError("Number of coupon dates not equal to number payments")</code>	-	-	-
<code>if n_cpn_dts != n_flows:</code>	-	-	<code>n_flows</code>
<code>raise FinError("Number of coupon dates not equal to number flows")</code>	-	-	-

`dirty_price_from_ytm`

Calculate the dirty price of bond from its yield to maturity. This function is vectorised with respect to the yield input. It implements a number of standard conventions for calculating the YTM.

```
dirty_price_from_ytm(settle_dt: Date,
```

```

ytm: float,
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""Calculate the dirty price of bond from its yield to maturity. This
function is vectorised with respect to the yield input. It implements
a number of standard conventions for calculating the YTM."""
if settle_dt < .issue_dt:
    raise FinError("Settlement_date_falls_before_issue_date")
if settle_dt > .maturity_dt:
    raise FinError("Bond_settlement_is_after_maturity_date")
if convention not in YTMCalcType:
    raise FinError("Yield_convention_unknown." + str(convention))
# We MUST call this to update value of .alpha and ex-div date
.accrued_interest(settle_dt, 1.0)
#####
# HANDLE EX_DIVIDEND DATES
#####
pay_first_cpn = 1.0
if settle_dt > .ex_div_dt:
    pay_first_cpn = 0.0
#####
ytm = np.array(ytm) # VECTORIZED
ytm = ytm + 0.000000000012345 # SNEAKY LOW-COST TRICK TO AVOID y=0
f = annual_frequency(.freq_type)
c = .cpn
if convention == YTMCalcType.ZERO:
    raise FinError("Zero_coupon_bonds_must_use_BondZero_class.")
v = 1.0 / (1.0 + ytm / f)
# n is the number of flows after the next coupon date
# Do not use payment dates here as convention is to use coupon dates
n = 0
for dt in .cpn_dts:
    if dt > settle_dt:
        n += 1
n = n - 1
if n < 0:
    raise FinError("No_coupons_left")
if convention == YTMCalcType.UK_DMO:
    if n == 0:
        dp = (v ** (.alpha)) * (1.0 + pay_first_cpn * c / f)
    else:
        term1 = (c / f) * pay_first_cpn
        term2 = (c / f) * v
        term3 = (c / f) * v * v * (1.0 - v ** (n - 1)) / (1.0 - v)
        term4 = v**n
        dp = (v ** (.alpha)) * (term1 + term2 + term3 + term4)
elif convention == YTMCalcType.US_TREASURY:
    if n == 0:
        dp = (v ** (.alpha)) * (1.0 + c / f)
    else:
        term1 = (c / f) * pay_first_cpn
        term2 = (c / f) * v
        term3 = (c / f) * v * v * (1.0 - v ** (n - 1)) / (1.0 - v)
        term4 = v**n
        vw = 1.0 / (1.0 + .alpha * ytm / f)
        dp = (vw) * (term1 + term2 + term3 + term4)
elif convention == YTMCalcType.US_STREET:
    if n == 0:

```



```

        vw = 1.0 / (1.0 + .alpha * ytm / f)
        dp = vw * (1.0 + c / f)
    else:
        term1 = (c / f) * pay_first_cpn
        term2 = (c / f) * v
        term3 = (c / f) * v * v * (1.0 - v ** (n - 1)) / (1.0 - v)
        term4 = v**n
        dp = (v ** (.alpha)) * (term1 + term2 + term3 + term4)
elif convention == YTMCalcType.CFETS:
    if n == 0:
        last_year = .maturity_dt.add_tenor("-12M")
        dc = DayCount(DayCountTypes.ACT_365L)
        alpha = (1
            - dc.year_frac(last_year,
                settle_dt,
                .maturity_dt,
                freq_type=FrequencyTypes.ANNUAL,
            ) [0]
        )
        vw = 1.0 / (1.0 + alpha * ytm)
        dp = vw * (1.0 + c / f)
    else:
        term1 = (c / f) * pay_first_cpn
        term2 = (c / f) * v
        term3 = (c / f) * v * v * (1.0 - v ** (n - 1)) / (1.0 - v)
        term4 = v**n
        dp = (v ** (.alpha)) * (term1 + term2 + term3 + term4)
elif convention == YTMCalcType.CALCULUS:
    dp = 0.0
    d = (1.0 + ytm / f)
    # g starts at the discount factor for the NEXT coupon date
    g = 1.0 / np.pow(d, .alpha)
    flow = .cpn / .freq
    n_next = 0
    for dt in .cpn_dts:
        if dt > settle_dt:
            break
        n_next += 1
    n_start = n_next
    if settle_dt > .ex_div_dt:
        n_start = n_next + 1
    g = g / d
    # n represents 'periods from today'
    for dt in .cpn_dts[n_start:]:
        dp += flow * g
        last_g = g
        g = g / d
    dp += 1.0 * last_g
else:
    raise FinError("Unknown_yield_convention")
return dp * .par
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
ytm	float	-
convention	YTMCalcType	-
"""Calculate the dirty price of bond from its yield to maturity. This function is vectorised with respect to the yield input. It implements a number of standard conventions for calculating the YTM."""	-	-
if settle_dt > issue_dt	-	-
raise FinError("Settlement date falls before issue date")	-	-
if settle_dt > maturity_dt	-	-
raise FinError("Bond settlement is after maturity date")	-	-
if convention not in YTMCalcType	-	-
raise FinError("Yield convention unknown." + str(convention))	-	-
.accrued_interest(settle_dt, 1.0)	-	-
pay_first_cpn	-	-
if settle_dt > ex_div_dt	-	-
pay_first_cpn	-	-
ytm	-	VECTORIZED
ytm	-	SNEAKY LOW-COST TRICK TO
f	-	-
c	-	-
if convention	-	-
raise FinError("Zero coupon bonds must use BondZero class.")	-	-
v	-	-
n	-	-
for dt in .cpn_dts	-	-
if dt > settle_dt	-	-
n +	-	-
n	-	-
if n > 0	-	-
raise FinError("No coupons left")	-	-
if convention	-	-
if n	-	-
dp	-	-
else	-	-
term1	-	-
term2	-	-
term3	-	-
term4	-	-
dp	-	-
elif convention	-	-
if n	-	-
dp	-	-
else	-	-
term1	-	-
term2	-	-
term3	-	-
term4	-	-
vw	-	-
dp	-	-
elif convention	-	-
if n	-	-

dirty_price_from_ytm_vector

PLEASE ADD A FUNCTION DESCRIPTION

```
dirty_price_from_ytm_vector(settle_dts: (Date, list),
ytms: (float, list, np.ndarray),
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""
Calculate_dirty_prices_for_vectors_of_settlement_dates_or_yields.
"""
# Standardize inputs to lists
if isinstance(settle_dts, list) or isinstance(settle_dts, np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Val
settle_dts	Date or list	-	-
ytms	float or list,np.ndarray	-	-
convention	YTMCalcType	-	YTMCalcType.U
"""	-	-	-
Calculate dirty prices for vectors of settlement dates or yields.	-	-	-
"""	-	-	-

forward_price

PLEASE ADD A FUNCTION DESCRIPTION

```
forward_price(settle_dt: Date,
forward_dt: Date,
clean_price: float,
repo_rate: float,
repo_dc_type: DayCountTypes = DayCountTypes.ACT_360,
if settle_dt < .issue_dt:
    raise FinError("Settlement_date_falls_before_issue_date")
if clean_price < 0 or repo_rate < 0:
    raise ValueError("Prices_and_repo_rate_must_be_non-negative")
if settle_dt >= forward_dt:
    raise ValueError("Settlement_date_must_be_before_forward_date")
repo_dc = DayCount(repo_dc_type)
t_fwd, _, _ = repo_dc.year_frac(settle_dt, forward_dt)
accrued_settle = .accrued_interest(settle_dt)
accrued_forward = .accrued_interest(forward_dt)
fv_cpns = 0.0
for dt, amt in zip(.payment_dts[1:], .flow_amounts[1:]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
forward_dt	Date	-	-
clean_price	float	-	-
repo_rate	float	-	-
repo_dc_type	DayCountTypes	-	DayCountType
if settle_dt > .issue_dt		-	-
raise FinError("Settlement date falls before issue date")	-	-	-
if clean_price < 0 or repo_rate < 0		-	-
raise ValueError("Prices and repo rate must be non-negative")	-	-	-
if settle_dt > forward_dt	-	-	forward_dt
raise ValueError("Settlement date must be before forward date")	-	-	-
repo_dc	-	-	DayCount(repo_dc_type)
t_fwd	-	-	-
-	-	-	-
-	-	-	repo_dc.year_frac(settle_dt, forward_dt)
accrued_settle	-	-	.accrued_interest(settle_dt)
accrued_forward	-	-	.accrued_interest(forward_dt)
fv_cpns	-	-	0.0
for dt	-	-	-

principal

Calculate the principal value of the bond based on the face amount from its discount margin and making assumptions about the future Ibor rates.

```

principal(settle_dt: Date, ytm: float, face: float, convention: YTMCalcType)
    """Calculate the principal value of the bond based on the face
    amount from its discount margin and making assumptions about the
    future Ibor rates."""
    if settle_dt < .issue_dt:
        raise FinError("Settlement date falls before issue date")
    dirty_price = .dirty_price_from_ytm(settle_dt, ytm, convention)
    principal = dirty_price * face / .par
    principal = principal - .accrued_int
    return principal
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	D
settle_dt	Date	-	
ymt	float	-	
face	float	-	
convention	YTMCalcType	-	
"""Calculate the principal value of the bond based on the face			
amount from its discount margin and making assumptions about the			
future Ibor rates."""			
if settle_dt > .issue_dt		-	
raise FinError("Settlement date falls before issue date")			
dirty_price	-	-	.dirty_price_from_y
principal	-	-	dirty_p
principal	-	-	princi
return principal	-	-	

dollar_duration

Calculate the risk or dP/dy of the bond by bumping. This is also known as the DV01 in Bloomberg.

```
dollar_duration(settle_dt: (Date, List),
ymt: (float, List),
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""Calculate the risk or dP/dy of the bond by bumping. This is also
known as the DV01 in Bloomberg."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
dy = 0.0001 # 1 basis point
p_minus = .dirty_price_from_ytm_vector(settle_dt, ytm - dy, convention)
p_plus = .dirty_price_from_ytm_vector(settle_dt, ytm + dy, convention)
durn = (p_minus - p_plus) / dy / 2.0
return durn
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
settle_dt	Date or List	-	
ymt	float or List	-	
convention	YTMCalcType	-	Y
"""Calculate the risk or dP/dy of the bond by bumping. This is also known as the DV01 in Bloomberg."""		-	
if settle_dt > .issue_dt		-	
raise FinError("Settlement date falls before issue date")		-	
dy	-	1 basis point	
p_minus	-	-	.dirty_price_from_y
p_plus	-	-	.dirty_price_from_y
durn	-	-	(p
return durn	-	-	

macauley_duration

Calculate the Macauley duration of the bond on a settlement date given its yield to maturity.

```

macauley_duration(settle_dt: Date,
ymt: float,
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""Calculate the Macauley duration of the bond on a settlement date
given its yield to maturity."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
d = (1.0 + ymt / .freq)
if convention is not YTMCalcType.CALCULUS:
    dd = .dollar_duration(settle_dt, ymt, convention)
    dp = .dirty_price_from_ymt(settle_dt, ymt, convention)
    md = (dd / dp) * d
else:
    # We call this to update value of .alpha and ex-dividend date
    .accrued_interest(settle_dt, 1.0)
    md = 0.0
    dp = 0.0
    # g starts at the discount factor for the NEXT coupon date
    g = 1.0 / np.pow(d, .alpha)
    flow = .cpn / .freq
    # 1. Find the index of the next coupon date
    n_next = 0
    for dt in .cpn_dts:
        if dt > settle_dt:
            break
        n_next += 1
    n_start = n_next
    k = 0
    if settle_dt > .ex_div_dt:
        n_start = n_next + 1
        # Move the discount factor and the period counter forward by one
        g = g / d
        k = 1
    # n represents 'periods from today'
    for dt in .cpn_dts[n_start:]:

```

```
t = (.alpha + k) / .freq
dp += flow * g
md += flow * t * g
last_g = g
last_t = t
g = g / d
k += 1
dp += 1.0 * last_g
md += 1.0 * last_t * last_g
dp = dp * .par
md = md * .par
md = md / dp
return md
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
settle_dt	Date	-	
ytm	float	-	
convention	YTMCalcType	-	YTMCalcType
"""Calculate the Macauley duration of the bond on a settlement date given its yield to maturity."""	-	-	
if settle_dt > issue_dt	-	-	
raise FinError("Settlement date falls before issue date")	-	-	
d	-	-	(1.0)
if convention is not YTMCalcType.CALCULUS	-	-	
dd	-	-	.dollar_duration
dp	-	-	.dirty_price_from_ytm
md	-	-	
else	-	-	
.accrued_interest(settle_dt, 1.0)	-	-	
md	-	-	
dp	-	-	
g	-	-	1.0 / n
flow	-	-	
n_next	-	-	
for dt in .cpn_dts	-	-	
if dt < settle_dt	-	-	
break	-	-	
n_next +	-	-	
n_start	-	-	
k	-	-	
if settle_dt < ex_div_dt	-	-	
n_start	-	-	
g	-	-	
k	-	-	
for dt in .cpn_dts[n_start	]:	-	
t	-	-	(.all)
dp +	-	-	
md +	-	-	
last_g	-	-	
last_t	-	-	
g	-	-	
k +	-	-	
dp +	-	-	
md +	-	-	1.0
dp	-	-	
md	-	-	
md	-	-	
return md	-	-	

modified_duration

Calculate the modified duration of the bond on a settlement date given its yield to maturity.

```
modified_duration(settle_dt: Date,
ymt: float,
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""Calculate the modified duration of the bond on a settlement
date given its yield to maturity."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
dd = .dollar_duration(settle_dt, ytm, convention)
fp = .dirty_price_from_ytm(settle_dt, ytm, convention)
md = dd / fp
return md
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	.
ymt	float	-	.
convention	YTMCalcType	-	YTMCalcType
"""Calculate the modified duration of the bond on a settlement	-	-	.
date given its yield to maturity."""	-	-	.
if settle_dt < .issue_dt	-	-	.
raise FinError("Settlement date falls before issue date")	-	-	.
dd	-	-	.dollar_duration(settle
fp	-	-	.dirty_price_from_ytm(se
md	-	-	dd
return md	-	-	.

key_rate_durations

PLEASE ADD A FUNCTION DESCRIPTION

```
key_rate_durations(settle_dt: Date,
ymt: float,
key_rate_tenors: list = None,
shift: float = None,
rates: list = None,
"""
Calculates the key rate durations for a bond.
Parameters
-----
bond_: FinancePyBond object
settle_dt_: FinancePyDate object
    The settlement date.
ymt_: float
    The yield to maturity.
key_rate_tenors_: list of float, optional
```

```

    """The tenors of the key rates, default is None which will generate
    the tenors from 0.25 to 30 years.
    shift: float, optional
    """The shift used to calculate the key rate durations, default is None
    which will set the shift to 0.0001.
    rates: list of float, optional
    """Corresponding yield curve data in line with key rate tenors
    """If None, flat yield curve is used
    Returns
    -----
    tuple of (numpy array of float, numpy array of float)
    """A tuple containing the key rate tenors and the key rate durations.
    """
    if settle_dt < .issue_dt:
        raise FinError("Settlement date falls before issue date")
    # check if key_rate_tenors is None
    # if it is None, create an array of key rates from 0.5 to 30 years
    if key_rate_tenors is None:
        key_rate_tenors = np.array([0.5, 1, 2, 3, 5, 7, 10, 20, 30])
    # set the shift to a small value if not give
    if not shift:
        shift = 0.0001
    lin_zero_interp = InterpTypes.LINEAR_ZERO_RATES
    us_street = YTMCalcType.US_STREET
    # initialize an empty list for the key rate durations
    key_rate_durations = []
    # iterate over each key rate (tenor) and calculate key rate duration
    for ind, _ in enumerate(key_rate_tenors):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Def
settle_dt	Date	-	
ytm	float	-	
key_rate_tenors	list	-	
shift	float	-	
rates	list	-	
"""	-	-	
Calculates the key rate durations for a bond.	-	-	
Parameters	-	-	
-----	-	-	
bond	FinancePyBondobject	-	
settle_dt	FinancePyDateobject	-	
The settlement date.	-	-	
ytm	float	-	
The yield to maturity.	-	-	
key_rate_tenors	listoffloat	-	
optional	-	-	
The tenors of the key rates	-	-	
default is None which will generate	-	-	
the tenors from 0.25 to 30 years.	-	-	
shift	float	-	
optional	-	-	
The shift used to calculate the key rate durations	-	-	
default is None	-	-	
which will set the shift to 0.0001.	-	-	
rates	listoffloat	-	
optional	-	-	
Corresponding yield curve data in line with key_rate_tenors	-	-	
If None	-	-	
flat yield curve is used	-	-	
Returns	-	-	
-----	-	-	
tuple of (numpy array of float, numpy array of float)	-	-	
A tuple containing the key rate tenors and the key rate durations.	-	-	
"""	-	-	
if settle_dt < issue_dt	-	-	
raise FinError("Settlement date falls before issue date")	-	-	
if key_rate_tenors is None	-	-	
key_rate_tenors	-	-	np.array([0.5, 1
if not shift	-	-	
shift	-	-	
lin_zero_interp	-	-	InterpTypes.LI
us_street	-	-	YTMCalc
key_rate_durations	-	-	
for ind	-	-	

convexity_from_ytm

Calculate the bond convexity from the yield to maturity. This function is vectorised with respect to the yield input.

```
convexity_from_ytm(settle_dt: Date,
ymt: float,
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""Calculate the bond convexity from the yield to maturity. This
function is vectorised with respect to the yield input."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
dy = 0.0001 # 1 basis point
p0 = .dirty_price_from_ytm(settle_dt, ytm - dy, convention)
p1 = .dirty_price_from_ytm(settle_dt, ytm, convention)
p2 = .dirty_price_from_ytm(settle_dt, ytm + dy, convention)
conv = ((p2 + p0) - 2.0 * p1) / dy / dy / p1 / .par
return conv
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	
ymt	float	-	
convention	YTMCalcType	-	YTMCalcType.UK_DMO
"""Calculate the bond convexity from the yield to maturity. This function is vectorised with respect to the yield input."""	-	-	
if settle_dt < .issue_dt	-	-	
raise FinError("Settlement date falls before issue date")	-	-	
dy	-	1 basis point	
p0	-	.dirty_price_from_ytm	
p1	-	.dirty_price_from_ytm	
p2	-	.dirty_price_from_ytm	
conv	-	((p2 + p0) - 2.0 * p1) / dy / dy / p1 / .par	
return conv	-	-	

clean_price_from_ytm

Calculate the bond clean price from the yield to maturity. This function is vectorised with respect to the yield input.

```
clean_price_from_ytm(settle_dt: Date,
ymt: float,
convention: YTMCalcType = YTMCalcType.UK_DMO,
"""Calculate the bond clean price from the yield to maturity. This
function is vectorised with respect to the yield input."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
```

```

dp = .dirty_price_from_ytm(settle_dt, ytm, convention)
accrued_int = .accrued_interest(settle_dt, .par)
cp = dp - accrued_int
return cp
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Def
settle_dt	Date	-	
ytm	float	-	
convention	YTMCalcType	-	YTMCalc
"""Calculate the bond clean price from the yield to maturity. This function is vectorised with respect to the yield input."""		-	
if settle_dt > .issue_dt		-	
raise FinError("Settlement date falls before issue date")		-	
dp	-	-	.dirty_price_from_ytm
accrued_int	-	-	.accrued_int
cp	-	-	dp -
return cp	-	-	

clean_price_from_discount_curve

Calculate the clean bond value using some discount curve to present-value the bond's cash flows back to the curve anchor date and not to the settlement date.

```

clean_price_from_discount_curve(settle_dt: Date, discount_curve: DiscountCurve
"""Calculate the clean bond value using some discount curve to
present-value the bond's cash flows back to the curve anchor date and
not to the settlement date."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
accrued_int = .accrued_interest(settle_dt, .par)
dirty_price = .dirty_price_from_discount_curve(settle_dt, discount_curve)
clean_price = dirty_price - accrued_int
return clean_price
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
discount_curve	DiscountCurve	-
"""Calculate the clean bond value using some discount curve to present-value the bond's cash flows back to the curve anchor date and not to the settlement date."""	-	-
if settle_dt < .issue_dt	-	-
raise FinError("Settlement date falls before issue date")	-	-
.accrued_interest(settle_dt, .par)	-	-
dirty_price	-	.dirty_price_from
clean_price	-	-
return clean_price	-	-

dirty_price_from_discount_curve

PLEASE ADD A FUNCTION DESCRIPTION

```
dirty_price_from_discount_curve(settle_dt: Date, discount_curve: DiscountCurve
# 1. Validation checks
if settle_dt < .issue_dt:
    raise FinError("Settlement_date_falls_before_issue_date")
if settle_dt < discount_curve.value_dt:
    raise FinError("Bond_settles_before_Discount_curve_date")
if settle_dt > .maturity_dt:
    raise FinError("Bond_settles_after_it_matures.")
# 2. Ex-dividend logic
._calc_pcd_ncd(settle_dt)
cal = Calendar(.cal_type)
.ex_div_dt = cal.add_business_days(.ncd, -.ex_div_days)
px_pv = 0.0
n_flows = len(.cpn_dts)
cpn_amount = .cpn / .freq
# 3. Iterate through all potential flows
for i in range(n_flows):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
if settle_dt ≤ .issue_dt		-	-
raise FinError("Settlement date falls before issue date")	-	-	-
if settle_dt ≤ discount_curve.value_dt		-	-
raise FinError("Bond settles before Discount curve date")	-	-	-
if settle_dt ≤ .maturity_dt		-	-
raise FinError("Bond settles after it matures.")	-	-	-
.calc_pcd_ncd(settle_dt)	-	-	-
cal	-	-	Calendar(.cal_type)
.ex_div_dt	-	-	cal.add_business_days(.ncd,
px_pv	-	-	0.0
n_flows	-	-	len(.cpn_dts)
cpn_amount	-	-	.cpn / .freq

current_yield

Calculate the current yield of the bond which is the coupon divided by the clean price (not the full price)

```
current_yield(clean_price):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
clean_price	-	-	-

yield_to_maturity

PLEASE ADD A FUNCTION DESCRIPTION

```
yield_to_maturity(settle_dt: Date,
clean_price: float | list | np.ndarray,
convention: YTMCalcType = YTMCalcType.US_TREASURY,
"""Calculate the bond's yield_to_maturity by solving the price
yield_relationship using a one-dimensional root solver."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
if isinstance(clean_price, float) or isinstance(clean_price, np.float64):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	-
clean_price	float—list—np.ndarray	-	-
convention	YTMCalcType	-	YTMCalcType.U
"""Calculate the bond's yield to maturity by solving the price yield relationship using a one-dimensional root solver."""	-	-	-
if settle_dt > .issue_dt	-	-	-
raise FinError("Settlement date falls before issue date")	-	-	-

simple_yield

PLEASE ADD A FUNCTION DESCRIPTION

```
simple_yield(settle_dt: Date,
clean_price: float | list | np.ndarray,
convention: YTMCalcType = YTMCalcType.US_TREASURY,
"""Calculate the bond's simple yield."""
if settle_dt < .issue_dt:
    raise FinError("Settlement date falls before issue date")
if isinstance(clean_price, float) or isinstance(clean_price, np.float64):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
clean_price	float—list—np.ndarray	-	-
convention	YTMCalcType	-	YTMCalcType.US_TR
"""Calculate the bond's simple yield."""	-	-	-
if settle_dt > .issue_dt	-	-	-
raise FinError("Settlement date falls before issue date")	-	-	-

_calc_pcd_ncd

PLEASE ADD A FUNCTION DESCRIPTION

```
_calc_pcd_ncd(settle_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-

accrued_interest

Calculate the amount of coupon that has accrued between the previous coupon date and the settlement date. Note that for some day count schemes (such as 30E/360) this is not actually the number of days between the previous coupon payment date and settlement date. If the bond trades with ex-coupon dates then you need

to use the number of days before the coupon date the ex-coupon date is. You can specify the calendar to be used in the bond constructor - *NONE* means only calendar days, *WEEKEND* is only weekends, or you can specify a country calendar for business days.

```
accrued_interest(settle_dt: Date, face: float = 100.0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	100.0

asset_swap_spread

Calculate the par asset swap spread of the bond. The discount curve is an Ibor curve that is passed in. This function is vectorised with respect to the clean price.

```
asset_swap_spread(settle_dt: Date,
clean_price: float,
discount_curve: DiscountCurve,
swap_float_day_count_convention_type=DayCountTypes.ACT_360,
swap_float_frequency_type=FrequencyTypes.SEMI_ANNUAL,
swap_float_calendar_type=CalendarTypes.WEEKEND,
swap_float_bus_day_adjust_rule_type=BusDayAdjustTypes.FOLLOWING,
swap_float_date_gen_rule_type=DateGenRuleTypes.BACKWARD,
"""Calculate the par asset swap spread of the bond. The discount curve
is an Ibor curve that is passed in. This function is vectorised with
respect to the clean price."""
clean_price = np.array(clean_price)
.accumulator(settle_dt, 1.0)
accrued_amount = .accrued_int * .par
bond_price = clean_price + accrued_amount
# Calculate the price of the bond discounted on the Ibor curve
pv_ibor = 0.0
prev_dt = .pcd
for dt in .payment_dts[1:]:
    # coupons paid on a settlement date are paid to the seller
    if dt > settle_dt:
        df = discount_curve.df(dt)
        pv_ibor += df * .cpn / .freq
pv_ibor += discount_curve.df(.payment_dts[-1])
# Calculate the PV01 of the floating leg of the asset swap
# I assume here that the coupon starts accruing on the settlement date
prev_dt = .pcd
schedule = Schedule(settle_dt,
    .maturity_dt,
    swap_float_frequency_type,
    swap_float_calendar_type,
    swap_float_bus_day_adjust_rule_type,
    swap_float_date_gen_rule_type,
)
day_count = DayCount(swap_float_day_count_convention_type)
prev_dt = .pcd
```

```
pv01 = 0.0
for dt in schedule.adjusted_dts[1:]:
    df = discount_curve.df(dt)
    year_frac = day_count.year_frac(prev_dt, dt)[0]
    pv01 = pv01 + year_frac * df
    prev_dt = dt
asw = (pv_ibor - bond_price / .par) / pv01
return asw
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
clean_price	float	-
discount_curve	DiscountCurve	-
swap_float_day_count_convention_type	-	Day
swap_float_frequency_type	-	Frequen
swap_float_calendar_type	-	Cale
swap_float_bus_day_adjust_rule_type	-	BusDayA
swap_float_date_gen_rule_type	-	DateGe
"""Calculate the par asset swap spread of the bond. The discount curve is an Ibor curve that is passed in. This function is vectorised with respect to the clean price."""	-	-
clean_price	-	n
.accrued_interest(settle_dt, 1.0)	-	-
accrued_amount	-	-
bond_price	-	clean
prior_dt	-	-
for dt in .payment_dts[1	]:	-
if dt < settle_dt	-	-
df	-	d
prior_dt +	-	-
prior_dt +	-	discount
prior_dt	-	-
.maturity_dt	-	-
swap_float_frequency_type	-	-
swap_float_calendar_type	-	-
swap_float_bus_day_adjust_rule_type	-	-
swap_float_date_gen_rule_type	-	-
)	-	-
day_count	-	DayCount(swap
prior_dt	-	-
prior01	-	-
for dt in schedule.adjusted_dts[1	]:	-
df	-	d
year_frac	-	day_cou
prior01	-	p
prior_dt	-	-
asw	-	(prior_ibor
return asw	-	-

z_spread

Calculate the z-spread of the bond. The discount curve is a Ibor curve that is passed in.

```

z_spread(settle_dt: Date,
clean_price: float,
discount_curve: DiscountCurve,
"""Calculate the z-spread of the bond. The discount curve
is a Ibor curve that is passed in."""
.accrued_int = .accrued_interest(settle_dt, 1.0)
accrued_amount = .accrued_int * .par
bond_price = clean_price + accrued_amount

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
clean_price	float	-	-
discount_curve	DiscountCurve	-	-
"""Calculate the z-spread of the bond. The discount curve is a Ibor curve that is passed in."""	-	-	-
.accrued_int	-	-	.accrued_interest(settle_dt, 1.0)
accrued_amount	-	-	.accrued_int * .par
bond_price	-	-	clean_price + accrued_amount

_bond_price_diff_from_z_spread

PLEASE ADD A FUNCTION DESCRIPTION

```
_bond_price_diff_from_z_spread(z_spr_try):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
z_spr_try	-	-	-

dirty_price_from_oas

PLEASE ADD A FUNCTION DESCRIPTION

```

dirty_price_from_oas(settle_dt: Date, discount_curve: DiscountCurve, oas: float
"""Calculate the price of the bond by adding a spread (OAS)
to the curve."""
.accrued_interest(settle_dt, 1.0)
f = .freq
cpn_flow = .cpn / f
pv = 0.0
# We need the settle DF to re-base the price to the settlement date
df_settle = discount_curve.df(settle_dt)
n_flows = len(.payment_dts)
for i in range(n_flows):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
oas	float	-	-
"""Calculate the price of the bond by adding a spread (OAS) to the curve."""	-	-	-
.accrued_interest(settle_dt, 1.0)	-	-	-
f	-	-	.freq
cpn_flow	-	-	.cpn / f
pv	-	-	0.0
df_settle	-	-	discount_curve.df(settle_dt)
n_flows	-	-	len(.payment_dts)

option_adjusted_spread

PLEASE ADD A FUNCTION DESCRIPTION

```
option_adjusted_spread(settle_dt: Date,
                        clean_price: float,
                        discount_curve: DiscountCurve,
                        """Return OAS for bullet bond given settlement date, clean bond price
                        and the discount relative to which the spread is to be computed."""
                        if isinstance(clean_price, float) or isinstance(clean_price, np.float64):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
clean_price	float	-	-
discount_curve	DiscountCurve	-	-
"""Return OAS for bullet bond given settlement date	-	-	-
clean bond price	-	-	-
and the discount relative to which the spread is to be computed."""	-	-	-

dirty_price_from_survival_curve

PLEASE ADD A FUNCTION DESCRIPTION

```
dirty_price_from_survival_curve(settle_dt: Date,
                                discount_curve: DiscountCurve,
                                survival_curve: DiscountCurve,
                                recovery_rate: float
                                """Calculate discounted present value of flows assuming default model.
                                The survival curve treats the coupons as zero recovery payments while
                                the recovery fraction of the par amount is paid at default. For the
                                defaulting principal we discretize the time steps using the coupon
                                payment times. A finer discretization may handle the time value with
                                more accuracy. I reduce any error by averaging period start and period
```

```

end_payment_present_values."""
f = .freq
cpn_flow = .cpn / f
.accrued_interest(settle_dt, 1.0)
df_settle = discount_curve.df(settle_dt)
q_settle = survival_curve.survival_prob(settle_dt)
prev_df = df_settle
prev_q = q_settle
pv_coupons = 0.0
pv_recovery = 0.0
n_flows = len(.payment_dts)
for i in range(n_flows):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Def
settle_dt	Date	-	
discount_curve	DiscountCurve	-	
survival_curve	DiscountCurve	-	
recovery_rate	float	-	
"""Calculate discounted present value of flows assuming default model.	-	-	
The survival curve treats the coupons as zero recovery payments while	-	-	
the recovery fraction of the par amount is paid at default. For the	-	-	
defaulting principal we discretize the time steps using the coupon	-	-	
payment times. A finer discretization may handle the time value with	-	-	
more accuracy. I reduce any error by averaging period start and period	-	-	
end payment present values."""	-	-	
f	-	-	
cpn_flow	-	-	
.accrued_interest(settle_dt, 1.0)	-	-	
df_settle	-	-	discount.c
q_settle	-	-	survival_curve.
prev_df	-	-	
prev_q	-	-	
pv_coupons	-	-	
pv_recovery	-	-	
n_flows	-	-	len(.p

clean_price_from_survival_curve

Calculate clean price value of flows assuming default model. The survival curve treats the coupons as zero recovery payments while the recovery fraction of the par amount is paid at default.

```

clean_price_from_survival_curve(settle_dt: Date,
discount_curve: DiscountCurve,
survival_curve: DiscountCurve,
recovery_rate: float,
"""Calculate_clean_price_value_of_flows_assuming_default_model.

```

```

The_survival_curve_treats_the_coupons_as_zero_recovery_payments_while
the_recovery_fraction_of_the_par_amount_is_paid_at_default."""
    .accrued_interest(settle_dt, 1.0)
    dirty_price = .dirty_price_from_survival_curve(settle_dt, discount_curve,
        survival_curve, recovery_rate
    )
    clean_price = dirty_price - .accrued_int
    return clean_price
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
survival_curve	DiscountCurve	-	-
recovery_rate	float	-	-
"""Calculate clean price value of flows assuming default model.	-	-	-
The survival curve treats the coupons as zero recovery payments while	-	-	-
the recovery fraction of the par amount is paid at default."""	-	-	-
.accrued_interest(settle_dt, 1.0)	-	-	-
)	-	-	-
clean_price	-	-	dirty_price - .accrued_int
return clean_price	-	-	-

calc_ror

Calculate the rate of total return(capital return and interest) given a BUY YTM and a SELL YTM of this bond. This function computes the full prices at buying and selling, plus the coupon payments during the period. It returns a tuple which includes a simple rate of return, a compounded IRR and the PnL.

```

calc_ror(begin_dt: Date,
end_dt: Date,
begin_ytm: float,
end_ytm: float,
convention: YTMCalcType = YTMCalcType.US_STREET,
"""
Calculate_the_rate_of_total_return(capital_return_and_interest)_given_a
BUY_YTM_and_a_SELL_YTM_of_this_bond.
This_function_computes_the_full_prices_at_buying_and_selling,_plus_the
coupon_payments_during_the_period.
It_returns_a_tuple_which_includes_a_simple_rate_of_return,_a_compounded
IRR_and_the_PnL.
"""
buy_price = .dirty_price_from_ytm(begin_dt, begin_ytm, convention)
sell_price = .dirty_price_from_ytm(end_dt, end_ytm, convention)
dts_cfs = zip(.payment_dts, .flow_amounts)
# The coupon or par payments on buying date belong to the buyer. The
# coupon or par payments on selling date are given to the new buyer.
dts_cfs = [

```

```

    (d, c * .par) for (d, c) in dts_cfs if (d >= begin_dt) and (d < end_dt)
]
dts_cfs.append((begin_dt, -buy_price))
dts_cfs.append((end_dt, sell_price))
times_cfs = [(d - begin_dt) / 365, c) for (d, c) in dts_cfs]
pnl = sum(c for (t, c) in times_cfs)
simple_return = (pnl / buy_price) * 365 / (end_dt - begin_dt)
brentq_up_lim = 5
brentq_dn_lim = -0.9999
# in case brentq cannot find the irr root
if simple_return > brentq_up_lim or simple_return < brentq_dn_lim:
    irr = simple_return
else:
    irr = optimize.brentq(npv,
        # f(a) and f(b) must have opposite signs
        a=brentq_dn_lim,
        b=brentq_up_lim,
        xtol=1e-8,
        args=(np.array(times_cfs),),
    )
return simple_return, irr, pnl
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
begin_dt	Date	-
end_dt	Date	-
begin_ytm	float	-
end_ytm	float	-
convention	YTMCalcType	-
"""	-	-
Calculate the rate of total return(capital return and interest) given a	-	-
BUY YTM and a SELL YTM of this bond.	-	-
This function computes the full prices at buying and selling	-	-
plus the	-	-
coupon payments during the period.	-	-
It returns a tuple which includes a simple rate of return	-	-
a compounded	-	-
IRR and the PnL.	-	-
"""	-	-
buy_price	-	.dirty_price_from_ytm
sell_price	-	.dirty_price_from_ytm
dts_cfs	-	zip(.payments, dts)
(d, c * .par) for (d, c) in dts_cfs if (d < begin_dt)	-	begin_dt
]	-	-
dts_cfs.append((begin_dt, -buy_price))	-	-
dts_cfs.append((end_dt, sell_price))	-	-
times_cfs	-	[(d - begin_dt) / dt for d, _ in dts_cfs]
pnl	-	sum(c * (1 + irr ** t) for t, c in zip(times_cfs, dts_cfs))
simple_return	-	(pnl / buy_price) * 100
brentq_up_lim	-	-
brentq_dn_lim	-	-
if simple_return < brentq_up_lim or simple_return > brentq_dn_lim	-	-
irr	-	-
else	-	-
a	-	-
b	-	-
xtol	-	-
args	-	(np.array([a, b]), xt看ol)
)	-	-
return simple_return	-	-
irr	-	-
pnl	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

print_payments

Print a list of the unadjusted coupon dts and the adjusted payment dates used in the analytic calculations for the bond.

```
print_payments(settle_dt: Date, face: float = 100):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	100

_print

Print a list of the unadjusted coupon payment dts used in analytic calculations for the bond.

```
_print():
```

The function arguments are described in the following table.

_f

Function used to do root search in price to yield calculation.

```
_f(ytm, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ytm	-	-	-
*args	-	-	-

_g

Function used to do root search in price to OAS calculation.

```
_g(oas, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
oas	-	-	-
*args	-	-	-

6.2 bond_annuity

Class: BondAnnuity

An annuity is a vector of dates and flows generated according to ISDA standard rules which starts on the next date after the start date (effective date) and runs up to an end date with no principal repayment. Dates are then adjusted according to a specified calendar.

BondAnnuity

PLEASE ADD A FUNCTION DESCRIPTION

```
BondAnnuity(maturity_dt: Date,
cpn: float,
freq_type: FrequencyTypes,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
check_argument_types(.BondAnnuity, locals()))
.maturity_dt = maturity_dt
.cpn = cpn
.freq_type = freq_type
.freq = annual_frequency(freq_type)
# ISDA Style conventions
.cal_type = cal_type
.bd_type = bd_type
.dg_type = dg_type
.dc_type = dc_type
.par = 100.0
.cpn_dts = []
.settle_dt = Date(1, 1, 1900)
.accrued_int = None
.accrued_days = 0.0
.alpha = 0.0
._pcd = None
._ncd = None
.flow_amounts = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
maturity_dt	Date	-	-
cpn	float	-	-
freq_type	FrequencyTypes	-	-
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
check_argument_types(.BondAnnuity, locals())	-	-	-
.maturity_dt	-	-	maturity_dt
.cpn	-	-	cpn
.freq_type	-	-	freq_type
.freq	-	-	annual_frequency(freq_type)
.cal_type	-	-	cal_type
.bd_type	-	-	bd_type
.dg_type	-	-	dg_type
.dc_type	-	-	dc_type
.par	-	-	100.0
.cpn_dts	-	-	[]
.settle_dt	-	-	Date(1, 1, 1900)
.accrued_int	-	-	None
.accrued_days	-	-	0.0
.alpha	-	-	0.0
.pcd	-	-	None
.ncd	-	-	None
.flow_amounts	-	-	None

clean_price_from_discount_curve

Calculate the bond price using some discount curve to present-value the bond's cash flows.

```

clean_price_from_discount_curve(settle_dt: Date, discount_curve: DiscountCurve
    """Calculate the bond price using some discount curve to present-value
    the bond's cash flows."""
    dirty_price = .dirty_price_from_discount_curve(settle_dt, discount_curve)
    accrued = .accrued_int * .par
    clean_price = dirty_price - accrued
    return clean_price
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
discount_curve	DiscountCurve	-
"""Calculate the bond price using some discount curve to present-value the bond's cash flows."""	-	-
dirty_price	-	-
accrued	-	-
clean_price	-	-
return clean_price	-	-

dirty_price_from_discount_curve

PLEASE ADD A FUNCTION DESCRIPTION

```
dirty_price_from_discount_curve(settle_dt: Date, discount_curve: DiscountCurve
"""Calculate the bond price using some discount curve to present-value
the bond's cash flows."""
.calculate_payments(settle_dt, 1.0)
pv = 0.0
num_flows = len(.cpn_dts)
for i in range(1, num_flows):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
"""Calculate the bond price using some discount curve to present-value the bond's cash flows."""	-	-	-
.calculate_payments(settle_dt, 1.0)	-	-	-
pv	-	-	0.0
num_flows	-	-	len(.cpn_dts)

calculate_payments

Calculate bond payments

```
calculate_payments(settle_dt: Date, face: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	-

accrued_interest

Calculate the amount of coupon that has accrued between the previous coupon date and the settlement date.

```
accrued_interest(settle_dt: Date, face: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	-

print_payments

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
print_payments(settle_dt: Date, face: float = 100):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	100

__repr__

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.3 bond_callable

Enumerated Type: BondModelTypes

This enumerated type has the following values:

- BLACK
- HO_LEE
- HULL_WHITE
- BLACK_KARASINSKI

Enumerated Type: BondOptionTypes

This enumerated type has the following values:

- EUROPEAN_CALL
- EUROPEAN_PUT
- AMERICAN_CALL
- AMERICAN_PUT
- coupon:float,Annualisedcoupon-0.03
- self.issue_dt
- self.maturity_dt
- self.cpn
- self.freq_type
- self.dc_type
- self.ex_div_days
- self.bond
- dt_prev
- ifdtj
- dt_prev
- dt_prev
- ifdtj
- dt_prev
- iflen(call_dts)!
- iflen(put_dts)!
- self.call_dts
- self.call_prices
- self.put_dts
- self.put_prices
- self.par
- cpn
- cpn_times
- cpn_amounts
- cpn_time
- cpn_times
- cpn_amounts
- call_times
- call_time
- call_times

- call_prices
- put_times
- put_time
- put_times
- put_prices
- maturity_dt
- t_mat
- df_times
- df_values
- face_amount
- v1
- model.num_time_steps+
- v2
- model.num_time_steps-
- v_bond_with_option
- v_bond_pure
- v1
- model.num_time_steps+
- v2
- model.num_time_steps-
- v_bond_with_option
- v_bond_pure
- s
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+
- s+

Class: BondEmbeddedOption

BondEmbeddedOption

PLEASE ADD A FUNCTION DESCRIPTION

```
BondEmbeddedOption(issue_dt: Date,
maturity_dt: Date, # Date
coupon: float, # Annualised coupon - 0.03 = 3.00%
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
call_dts: List[Date],
call_prices: np.ndarray,
put_dts: List[Date],
```



```

put_prices: np.ndarray,
"""Create_a_BondEmbeddedOption_object_with_a_maturity_date,_coupon
and_all_the_bond_inputs."""
check_argument_types(.BondEmbeddedOption, locals())
.issue_dt = issue_dt
.maturity_dt = maturity_dt
.cpn = coupon
.freq_type = freq_type
.dc_type = dc_type
.ex_div_days = 0
.bond = Bond(issue_dt, maturity_dt, coupon, freq_type, dc_type, .ex_div_days
)
# Validate call and put schedules
for dt in call_dts:
    if dt > .maturity_dt:
        raise FinError("Call_date_after_bond_maturity_date")
if len(call_dts) > 0:
    dt_prev = call_dts[0]
    for dt in call_dts[1:]:
        if dt <= dt_prev:
            raise FinError("Call_dates_not_increasing")
        else:
            dt_prev = dt
for dt in put_dts:
    if dt > .maturity_dt:
        raise FinError("Put_date_after_bond_maturity_date")
if len(put_dts) > 0:
    dt_prev = put_dts[0]
    for dt in put_dts[1:]:
        if dt <= dt_prev:
            raise FinError("Put_dates_not_increasing")
        else:
            dt_prev = dt
for px in call_prices:
    if px < 0.0:
        raise FinError("Call_price_must_be_positive.")
for px in put_prices:
    if px < 0.0:
        raise FinError("Put_price_must_be_positive.")
if len(call_dts) != len(call_prices):

```

The function arguments are described in the following table.

Argument Name	Type	Description
issue_dt	Date	-
maturity_dt	Date	Date
coupon	float	Annualised coupon - 0.03 = 3.00%
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
call_dts	List[Date]	-
call_prices	np.ndarray	-
put_dts	List[Date]	-
put_prices	np.ndarray	-
"""Create a BondEmbeddedOption object with a maturity date	-	-
coupon	-	-
and all the bond inputs."""	-	-
check_argument_types(.BondEmbeddedOption, locals())	-	-
.issue_dt	-	-
.maturity_dt	-	-
.cpn	-	-
.freq_type	-	-
.dc_type	-	-
.ex_div_days	-	-
)	-	-
for dt in call_dts		-
if dt < .maturity_dt		-
raise FinError("Call date after bond maturity date")	-	-
if len(call_dts) < 0		-
dt_prev	-	-
for dt in call_dts[1	]:	-
if dt < dt_prev	-	-
raise FinError("Call dates not increasing")	-	-
else		-
dt_prev	-	-
for dt in put_dts		-
if dt < .maturity_dt		-
raise FinError("Put date after bond maturity date")	-	-
if len(put_dts) < 0		-
dt_prev	-	-
for dt in put_dts[1	]:	-
if dt < dt_prev	-	-
raise FinError("Put dates not increasing")	-	-
else		-
dt_prev	-	-
for px in call_prices		-
if px < 0.0		-
raise FinError("Call price must be positive.")	-	-
for px in put_prices		-
if px < 0.0		-
raise FinError("Put price must be positive.")	-	-

value

Value the bond that settles on the specified date that can have both embedded call and put options. This is done using the specified model and a discount curve.

```
value(settle_dt: Date, discount_curve: DiscountCurve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
model	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

PLEASE ADD A FUNCTION DESCRIPTION

```
_print():
```

The function arguments are described in the following table.

6.4 bond_convertible

Class: BondConvertible

Class for convertible bonds. These bonds embed rights to call and put the bond in return for equity. Until then, they are bullet bonds which means they have regular coupon payments of a known size that are paid on known dates plus a payment of par at maturity. As the options are price based, the decision to convert to equity depends on the stock price, the credit quality of the issuer and the level of interest rates.

BondConvertible

I do not determine cashflow dates as I do not want to require users to supply the issue date and without that I do not know how far to go back in the cashflow date schedule.

```
BondConvertible(maturity_dt: Date, # bond maturity date
coupon: float, # annual coupon
freq_type: FrequencyTypes, # coupon frequency type
start_convert_dt: Date, # conversion starts on this date
conversion_ratio: float, # num shares per face of notional
call_dts: List[Date], # list of call dates
call_prices: np.ndarray, # list of call prices
put_dts: List[Date], # list of put dates
put_prices: np.ndarray, # list of put prices
dc_type: DayCountTypes, # day count type for accrued
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
"""Create_BondConvertible_object_by_providing_the_bond_Maturity
date,_coupon,_frequency_type,_accrual_convention_type_and_then_all
the_details_regarding_the_conversion_option_including_the_list_of_the
call_and_put_dates_and_the_corresponding_list_of_call_and_put_prices.
"""
check_argument_types(.BondConvertible, locals())
if start_convert_dt > maturity_dt:
    raise FinError("Start_convert_date_is_after_bond_maturity.")
.maturity_dt = maturity_dt
.cpn = coupon
.dc_type = dc_type
.freq = annual_frequency(freq_type)
.freq_type = freq_type
.cal_type = cal_type
.call_dts = call_dts
.call_prices = call_prices
if len(.call_dts) != len(.call_prices):
```

The function arguments are described in the following table.

Argument Name	Type	Description
maturity_dt	Date	bond maturity date
coupon	float	annual coupon
freq_type	FrequencyTypes	coupon frequency type
start_convert_dt	Date	conversion starts on this date
conversion_ratio	float	num shares per face of notional
call_dts	List[Date]	list of call dates
call_prices	np.ndarray	list of call prices
put_dts	List[Date]	list of put dates
put_prices	np.ndarray	list of put prices
dc_type	DayCountTypes	day count type for accrued
cal_type	CalendarTypes	-
"""Create BondConvertible object by providing the bond Maturity		
date	-	-
coupon	-	-
frequency type	-	-
accrual convention type and then all	-	-
the details regarding the conversion option including the list of the	-	-
call and put dates and the corresponding list of call and put prices.	-	-
"""	-	-
check_argument_types(.BondConvertible, locals())	-	-
if start_convert_dt > maturity_dt	-	-
raise FinError("Start convert date is after bond maturity.")	-	-
.maturity_dt	-	-
.cpn	-	-
.dc_type	-	-
.freq	-	-
.freq_type	-	-
.cal_type	-	-
.call_dts	-	-
.call_prices	-	-

calculate_cpn_dts

Determine the convertible bond cash flow payment dates.

```
_calculate_cpn_dts(settle_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```

value(settle_dt: Date,
      stock_price: float,
      stock_volatility: float,
      dividend_dts: List[Date],
      dividend_yields: np.ndarray,
      discount_curve: DiscountCurve,
      credit_spread: float,
      recovery_rate: float = 0.40,
      num_steps_per_year: int = 100,
      """
A binomial tree valuation model for a convertible bond that captures
the embedded equity option due to the existence of a conversion option
which can be invoked after a specific date.
The model allows the user to enter a schedule of dividend payment
dates but the size of the payments must be in yield terms i.e. a known
percentage of currently unknown future stock price is paid. Not a
fixed amount. A fixed yield. Following this payment the stock is
assumed to drop by the size of the dividend payment.
The model also captures the stock dependent credit risk of the cash
flows in which the bond price can default at any time with a hazard
rate implied by the credit spread and an associated recovery rate.
This is the model proposed by Hull (OFODS 6th edition, page 522).
The model captures both the issuer's call schedule which is assumed
to apply on a list of dates provided by the user, along with a call
price. It also captures the embedded owner's put schedule of prices.
      """
      if stock_price <= 0.0:
          stock_price = 1e-10 # Avoid overflows in delta calc
      if stock_volatility <= 0.0:
          stock_volatility = 1e-10 # Avoid overflows in delta calc
      .calculate_cpn_dts(settle_dt)
      t_mat = (.maturity_dt - settle_dt) / G_DAYS_IN_YEAR
      if t_mat <= 0.0:
          raise FinError("Maturity must not be on or before the value date.")
      # We include time zero in the coupon times and flows
      cpn_times = [0.0]
      cpn_flows = [0.0]
      cpn = .cpn / .freq
      for dt in .payment_dts[1:]:
          flow_time = (dt - settle_dt) / G_DAYS_IN_YEAR
          cpn_times.append(flow_time)
          cpn_flows.append(cpn)
      cpn_times = np.array(cpn_times)
      cpn_flows = np.array(cpn_flows)
      if np.any(cpn_times < 0.0):

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
stock_price	float	-
stock_volatility	float	-
dividend_dts	List[Date]	-
dividend_yields	np.ndarray	-
discount_curve	DiscountCurve	-
credit_spread	float	-
recovery_rate	float	-
num_steps_per_year	int	-
"""	-	-
A binomial tree valuation model for a convertible bond that captures	-	-
the embedded equity option due to the existence of a conversion option	-	-
which can be invoked after a specific date.	-	-
The model allows the user to enter a schedule of dividend payment	-	-
dates but the size of the payments must be in yield terms i.e. a known	-	-
percentage of currently unknown future stock price is paid. Not a	-	-
fixed amount. A fixed yield. Following this payment the stock is	-	-
assumed to drop by the size of the dividend payment.	-	-
The model also captures the stock dependent credit risk of the cash	-	-
flows in which the bond price can default at any time with a hazard	-	-
rate implied by the credit spread and an associated recovery rate.	-	-
This is the model proposed by Hull (OFODS 6th edition, page 522).	-	-
The model captures both the issuer's call schedule which is assumed	-	-
to apply on a list of dates provided by the user	-	-
along with a call	-	-
price. It also captures the embedded owner's put schedule of prices.	-	-
"""	-	-
if stock_price :	-	-
stock_price	-	Avoid overflows in delta calc
if stock_volatility :	-	-
stock_volatility	-	Avoid overflows in delta calc
.calculate_cpn_dts(settle_dt)	-	-
t_mat	-	-
if t_mat :	-	-
raise FinError("Maturity must not be on or before the value date.")	-	-
cpn_times	-	-
cpn_flows	-	-
cpn	-	-
for dt in .payment_dts[1	]:	-
flow_time	-	-
cpn_times.append(flow_time)	-	-
cpn_flows.append(cpn)	-	-
cpn_times	-	-
cpn_flows	-	-

accrued_interest

Calculate the amount of coupon that has accrued between the previous coupon date and the settlement date.

```
accrued_interest(settle_dt: Date, face: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	-

current_yield

Calculate the current yield of the bond which is the coupon divided by the clean price (not the full price)

```
current_yield(clean_price: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
clean_price	float	-	-

__repr__

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

_value_convertible

PLEASE ADD A FUNCTION DESCRIPTION

```
_value_convertible(
    t_mat,
    face_amount,
```



```

cpn_times,
cpn_flows,
call_times,
call_prices,
put_times,
put_prices,
conv_ratio,
start_convert_time,
# Market inputs
stock_price,
df_times,
df_values,
dividend_times,
dividend_yields,
stock_volatility,
credit_spread,
recovery_rate,
# Tree details
num_steps_per_year,
interp = InterpTypes.FLAT_FWD_RATES.value
if len(cpn_times) > 0:
    if cpn_times[-1] > t_mat:
        raise FinError("Coupon_after_maturity")
if len(call_times) > 0:
    if call_times[-1] > t_mat:
        raise FinError("Call_times_after_maturity")
if len(put_times) > 0:
    if put_times[-1] > t_mat:
        raise FinError("Put_times_after_maturity")
if len(df_times) > 0:
    if df_times[-1] > t_mat:
        raise FinError("Discount_times_after_maturity")
if len(dividend_times) > 0:
    if dividend_times[-1] > t_mat:
        raise FinError("Dividend_times_after_maturity")
if credit_spread < 0.0:
    raise FinError("Credit_spread_negative.")
if recovery_rate < 0.0 or recovery_rate > 1.0:
    raise FinError("Recovery_rate_should_be_between_0_and_1.")
if stock_volatility < 0.0:
    raise FinError("Stock_volatility_cannot_be_negative.")
if num_steps_per_year < 1:
    raise FinError("Num_Steps_per_year_must_more_than_1.")
if len(dividend_times) > 0.0:
    if dividend_times[-1] > t_mat:
        raise FinError("Last_dividend_is_after_bond_maturity.")
if recovery_rate > 0.999 or recovery_rate < 0.0:
    raise FinError("Recovery_rate_must_be_between_0_and_0.999.")
num_times = int(num_steps_per_year * t_mat) + 1 # add one for today time 0
num_times = num_steps_per_year # XXXXXXXX!!!!!!!!!!!!!!!!!!!!!!
if num_times < 5:
    raise FinError("Numsteps_must_be_greater_than_5.")
num_levels = num_times
# this is the size of the step
dt = t_mat / (num_times - 1)
tree_times = np.linspace(0.0, t_mat, num_times)
tree_dfs = np.zeros(num_times)

```

```
for i in range(0, num_times):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t_mat	-	-	-
face_amount	-	-	-
cpn_times	-	-	-
cpn_flows	-	-	-
call_times	-	-	-
call_prices	-	-	-
put_times	-	-	-
put_prices	-	-	-
conv_ratio	-	-	-
start_convert_time	-	-	-
stock_price	-	-	-
df_times	-	-	-
df_values	-	-	-
dividend_times	-	-	-
dividend_yields	-	-	-
stock_volatility	-	-	-
credit_spread	-	-	-
recovery_rate	-	-	-
num_steps_per_year	-	-	-
interp	-	-	InterpTypes.FL
if len(cpn_times) < 0	-	-	-
if cpn_times[-1] > t_mat	-	-	-
raise FinError("Coupon after maturity")	-	-	-
if len(call_times) < 0	-	-	-
if call_times[-1] > t_mat	-	-	-
raise FinError("Call times after maturity")	-	-	-
if len(put_times) < 0	-	-	-
if put_times[-1] > t_mat	-	-	-
raise FinError("Put times after maturity")	-	-	-
if len(df_times) < 0	-	-	-
if df_times[-1] > t_mat	-	-	-
raise FinError("Discount times after maturity")	-	-	-
if len(dividend_times) < 0	-	-	-
if dividend_times[-1] > t_mat	-	-	-
raise FinError("Dividend times after maturity")	-	-	-
if credit_spread < 0.0	-	-	-
raise FinError("Credit spread negative.")	-	-	-
if recovery_rate < 0.0 or recovery_rate > 1.0	-	-	-
raise FinError("Recovery rate should be between 0 and 1.")	-	-	-
if stock_volatility < 0.0	-	-	-
raise FinError("Stock volatility cannot be negative.")	-	-	-
if num_steps_per_year < 1	-	-	-
raise FinError("Num Steps per year must more than 1.")	-	-	-
if len(dividend_times) < 0	-	-	-
if dividend_times[-1] > t_mat	-	-	-
raise FinError("Last dividend is after bond maturity.")	-	-	-
if recovery_rate < 0.999 or recovery_rate > 0.0	-	-	-
raise FinError("Recovery rate must be between 0 and 0.999.")	-	-	-
num_times	-	add one for today time 0	int(num_step
num_times	-	XXXXXXXXX!!!!!!!!!!!!!!!!!!!!!!	num.
if num_times > 5	-	-	-

print_tree*PLEASE ADD A FUNCTION DESCRIPTION*

```
print_tree(array):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
array	-	-	-

6.5 bond_frn

Class: BondFRN

Class for managing floating rate notes that pay a floating index plus a quoted margin.

BondFRN

Create FinFloatingRateNote object given its maturity date, its quoted margin, coupon frequency, DAY COUNT TYPE. Face is the size of the position and par is the notional on which price is quoted.

```
BondFRN(issue_dt: Date,
maturity_dt: Date,
quoted_margin: float, # Fixed spread paid on top of index
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
"""Create_FinFloatingRateNote_object_given_its_maturity_date,_its
quoted_margin,_coupon_frequency,_DAY_COUNT_TYPE._Face_is_the_size_of
the_position_and_par_is_the_notional_on_which_price_is_quoted."""
check_argument_types(.BondFRN, locals())
.issue_dt = issue_dt
.maturity_dt = maturity_dt
.quoted_margin = quoted_margin
.freq_type = freq_type
.dc_type = dc_type
.freq = annual_frequency(freq_type)
.cal_type = cal_type
.settle_dt = Date(1, 1, 1900)
.accrued = None
.accrued_days = 0.0
.accrual_factor = 0.0
.accrued_int = 0.0
.cpn_dts = []
.flow_amounts = []
.par = 100.0 # This is how price is quoted
._pcd = None
._ncd = None
.alpha = 0.0 # This is the fraction of the coupon that has accrued
._calculate_cpn_dts()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
issue_dt	Date	-
maturity_dt	Date	-
quoted_margin	float	Fixed spread paid on top of interest
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
cal_type	CalendarTypes	-
"""Create FinFloatingRateNote object given its maturity date	-	-
its	-	-
quoted margin	-	-
coupon frequency	-	-
DAY COUNT TYPE. Face is the size of	-	-
the position and par is the notional on which price is quoted."""	-	-
check_argument_types(.BondFRN, locals())	-	-
.issue_dt	-	-
.maturity_dt	-	-
.quoted_margin	-	-
.freq_type	-	-
.dc_type	-	-
.freq	-	-
.cal_type	-	-
.settle_dt	-	-
.accrued	-	-
.accrued_days	-	-
.accrual_factor	-	-
.accrued_int	-	-
.cpn_dts	-	-
.flow_amounts	-	-
.par	-	This is how price is quoted
.pcd	-	-
.ncd	-	-
.alpha	-	This is the fraction of the coupon that is paid in advance
.calculate_cpn_dts()	-	-

`_calculate_cpn_dts`

Determine the bond cashflow payment dates.

```
_calculate_cpn_dts():
```

The function arguments are described in the following table.

`dirty_price_from_dm`

PLEASE ADD A FUNCTION DESCRIPTION

```

dirty_price_from_dm(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the full price of the bond from its discount margin (DM)
using standard model based on assumptions about future Ibor rates. The
next Ibor payment which has reset is entered, so to is the current
Ibor rate from settlement to the next coupon date (NCD). Finally, there
is the level of subsequent future Ibor payments and the discount
margin."""
day_counter = DayCount(.dc_type)
q = .quoted_margin
num_flows = len(.cpn_dts)
# We discount using Libor over the period from settlement to the ncd
(alpha, _, _) = day_counter.year_frac(settle_dt, ._ncd)
df = 1.0 / (1.0 + alpha * (current_ibor + dm))
# A full coupon is paid
(alpha, _, _) = day_counter.year_frac(._pcd, ._ncd)
pv = next_cpn * alpha * df
# Now do all subsequent coupons that fall after the ncd
for i_flow in range(1, num_flows):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
"""Calculate the full price of the bond from its discount margin (DM)	-	-	-
using standard model based on assumptions about future Ibor rates. The	-	-	-
next Ibor payment which has reset is entered	-	-	-
so to is the current	-	-	-
Ibor rate from settlement to the next coupon date (NCD). Finally	-	-	-
there	-	-	-
is the level of subsequent future Ibor payments and the discount	-	-	-
margin."""	-	-	-
day_counter	-	-	DayCount(.dc_type)
q	-	-	.quoted_margin
num_flows	-	-	len(.cpn_dts)
(alpha, _, _)	-	-	day_counter.year_frac(settle_dt, ._ncd)
df	-	-	1.0 / (1.0 + alpha * (current_ibor + dm))
(alpha, _, _)	-	-	day_counter.year_frac(.pcd, ._ncd)
pv	-	-	next_cpn * alpha * df

principal

Calculate the clean trade price of the bond based on the face amount from its discount margin and making assumptions about the future Ibor rates.

```
principal(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
face: float = 100.0,
"""Calculate the clean trade price of the bond based on the face
amount from its discount margin and making assumptions about the
future Ibor rates."""
dirty_price = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
.accrued = .accrual_factor * next_cpn * 1.0
principal = dirty_price * face / .par - .accrued
return principal
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
face	float	-	100.0
"""Calculate the clean trade price of the bond based on the face	-	-	-
amount from its discount margin and making assumptions about the	-	-	-
future Ibor rates."""	-	-	-
)	-	-	-
.accrued	-	-	.accrual_factor * next_cpn *
principal	-	-	dirty_price * face / .par - .acc
return principal	-	-	-

dollar_duration

Calculate the risk or dP/dy of the bond by bumping. This is also known as the DV01 in Bloomberg.

```
dollar_duration(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the risk or dP/dy of the bond by bumping. This is also
known as the DV01 in Bloomberg."""
dy = 0.0001 # 1 basis point
```



```

p0 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor + dy, future_ibor, dm
)
p2 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor - dy, future_ibor, dm
)
durn = (p2 - p0) / dy / 2.0
return durn
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
"""Calculate the risk or dP/dy of the bond by bumping. This is also known as the DV01 in Bloomberg."""			
dy	-	1 basis point	0.0001
)	-	-	-
)	-	-	-
durn	-	-	(p2 - p0) / dy / 2.0
return durn	-	-	-

dollar_credit_duration

Calculate the risk or dP/dy of the bond by bumping.

```

dollar_credit_duration(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the risk or dP/dy of the bond by bumping."""
if dm > 10.0:
    raise FinError("Discount_margin_exceeds_100000bp")
dy = 0.0001
p0 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm + dy
)
p2 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
durn = (p2 - p0) / dy
return durn
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
"""Calculate the risk or dP/dy of the bond by bumping."""	-	-	-
if dm < 10.0	-	-	-
raise FinError("Discount margin exceeds 100000bp")	-	-	-
dy	-	-	0.0001
)	-	-	-
)	-	-	-
durn	-	-	(p2 - p0) / dy
return durn	-	-	-

macauley_duration

Calculate the Macauley duration of the FRN on a settlement date given its yield to maturity.

```
macauley_duration(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the Macauley duration of the FRN on a settlement date
given its yield to maturity."""
dd = .dollar_duration(settle_dt, next_cpn, current_ibor, future_ibor, dm)
fp = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
md = dd * (1.0 + (next_cpn + dm) / .freq) / fp
return md
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	De
settle_dt	Date	-	
next_cpn	float	-	
current_ibor	float	-	
future_ibor	float	-	
dm	float	-	
"""Calculate the Macauley duration of the FRN on a settlement date	-	-	
given its yield to maturity."""	-	-	
dd	-	-	.dollar_duration(settle_dt, ne
)	-	-	
md	-	-	dd * (1.0 + (ne
return md	-	-	

modified_duration

Calculate the modified duration of the bond on a settlement date using standard model based on assumptions about future Ibor rates. The next Ibor payment which has reset is entered, so to is the current Ibor rate from settlement to the next coupon date (NCD). Finally, there is the level of subsequent future Ibor payments and the discount margin.

```
modified_duration(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the modified duration of the bond on a settlement date
using standard model based on assumptions about future Ibor rates. The
next Ibor payment which has reset is entered, so to is the current
Ibor rate from settlement to the next coupon date (NCD). Finally, there
is the level of subsequent future Ibor payments and the discount
margin."""
dd = .dollar_duration(settle_dt, next_cpn, current_ibor, future_ibor, dm)
fp = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
md = dd / fp
return md
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
next_cpn	float	-
current_ibor	float	-
future_ibor	float	-
dm	float	-
"""Calculate the modified duration of the bond on a settlement date	-	-
using standard model based on assumptions about future Ibor rates. The	-	-
next Ibor payment which has reset is entered	-	-
so to is the current	-	-
Ibor rate from settlement to the next coupon date (NCD). Finally	-	-
there	-	-
is the level of subsequent future Ibor payments and the discount	-	-
margin."""	-	-
dd	-	- .dollar_duration(settle_dt,
)	-	-
md	-	-
return md	-	-

modified_credit_duration

Calculate the modified duration of the bond on a settlement date using standard model based on assumptions

about future Ibor rates. The next Ibor payment which has reset is entered, so to is the current Ibor rate from settlement to the next coupon date (NCD). Finally, there is the level of subsequent future Ibor payments and the discount margin.

```
modified_credit_duration(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the modified duration of the bond on a settlement date
using standard model based on assumptions about future Ibor rates. The
next Ibor payment which has reset is entered, so to is the current
Ibor rate from settlement to the next coupon date (NCD). Finally, there
is the level of subsequent future Ibor payments and the discount
margin."""
dd = .dollar_credit_duration(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
fp = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
md = dd / fp
return md
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
"""Calculate the modified duration of the bond on a settlement date	-	-	-
using standard model based on assumptions about future Ibor rates. The	-	-	-
next Ibor payment which has reset is entered	-	-	-
so to is the current	-	-	-
Ibor rate from settlement to the next coupon date (NCD). Finally	-	-	-
there	-	-	-
is the level of subsequent future Ibor payments and the discount	-	-	-
margin."""	-	-	-
)	-	-	-
)	-	-	-
md	-	-	dd / fp
return md	-	-	-

convexity_from_dm

Calculate the bond convexity from the discount margin (DM) using a standard model based on assumptions about future Ibor rates. The next Ibor payment which has reset is entered, so to is the current Ibor rate from

settlement to the next coupon date (NCD). Finally, there is the level of subsequent future Ibor payments and the discount margin.

```
convexity_from_dm(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the bond convexity from the discount margin (DM) using a
standard model based on assumptions about future Ibor rates. The
next Ibor payment which has reset is entered, so to is the current
Ibor rate from settlement to the next coupon date (NCD). Finally, there
is the level of subsequent future Ibor payments and the discount
margin."""
dy = 0.0001
p0 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor - dy, future_ibor, dm
)
p1 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
p2 = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor + dy, future_ibor, dm
)
conv = ((p2 + p0) - 2.0 * p1) / dy / dy / p1 / .par
return conv
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
"""Calculate the bond convexity from the discount margin (DM) using a	-	-	-
standard model based on assumptions about future Ibor rates. The	-	-	-
next Ibor payment which has reset is entered	-	-	-
so to is the current	-	-	-
Ibor rate from settlement to the next coupon date (NCD). Finally	-	-	-
there	-	-	-
is the level of subsequent future Ibor payments and the discount	-	-	-
margin."""	-	-	-
dy	-	-	0.0001
)	-	-	-
)	-	-	-
)	-	-	-
conv	-	-	$((p2 + p0) - 2.0 * p1) / c$
return conv	-	-	-

clean_price_from_dm

Calculate the bond clean price from the discount margin using standard model based on assumptions about future Ibor rates. The next Ibor payment which has reset is entered, so to is the current Ibor rate from settlement to the next coupon date (NCD). Finally, there is the level of subsequent future Ibor payments and the discount margin.

```

clean_price_from_dm(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
dm: float,
"""Calculate the bond clean price from the discount margin
using standard model based on assumptions about future Ibor rates. The
next Ibor payment which has reset is entered, so to is the current
Ibor rate from settlement to the next coupon date (NCD). Finally, there
is the level of subsequent future Ibor payments and the discount
margin."""
if dm > 10.0:
    raise FinError("Discount margin exceeds 100000bp")
dirty_price = .dirty_price_from_dm(settle_dt, next_cpn, current_ibor, future_ibor, dm
)
.accrued_interest(settle_dt, next_cpn)
.accrued = .accrual_factor * next_cpn * .par
clean_price = dirty_price - .accrued
return clean_price
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
dm	float	-	-
"""Calculate the bond clean price from the discount margin using standard model based on assumptions about future Ibor rates. The next Ibor payment which has reset is entered so to is the current Ibor rate from settlement to the next coupon date (NCD). Finally there is the level of subsequent future Ibor payments and the discount margin."""	-	-	-
if dm > 10.0	-	-	-
raise FinError("Discount margin exceeds 100000bp")	-	-	-
)	-	-	-
.accrued_interest(settle_dt, next_cpn)	-	-	-
.accrued	-	-	.accrual_factor * next_cpn
clean_price	-	-	dirty_price - .accrued
return clean_price	-	-	-

discount_margin

Calculate the bond's yield to maturity by solving the price yield relationship using a one-dimensional root solver.

```

discount_margin(settle_dt: Date,
next_cpn: float,
current_ibor: float,
future_ibor: float,
clean_price: float,
"""Calculate the bond's yield to maturity by solving the price
yield relationship using a one-dimensional root solver."""
    .accrued_interest(settle_dt, next_cpn)
    # Accrued needs to be adjusted to par notional
    .accrued = .accrual_factor * next_cpn * .par
    dirty_price = clean_price + .accrued
    argtuple = (settle_dt,
                next_cpn,
                current_ibor,
                future_ibor,
                dirty_price,
                )
    dm = optimize.newton(_f,
                        x0=0.01, # initial value of 10%
                        fprime=None,
                        args=argtuple,
                        tol=1e-12,
                        maxiter=50,

```

```

    fprime2=None,
)
return dm
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-
current_ibor	float	-	-
future_ibor	float	-	-
clean_price	float	-	-
"""Calculate the bond's yield to maturity by solving the price yield relationship using a one-dimensional root solver."""	-	-	-
.accrued_interest(settle_dt, next_cpn)	-	-	-
.accrued	-	-	.accrual_factor * next_cpn *
dirty_price	-	-	clean_price + .accrued
next_cpn	-	-	-
current_ibor	-	-	-
future_ibor	-	-	-
dirty_price	-	-	-
)	-	-	-
x0	-	initial value of 10%	0.01
fprime	-	-	None
args	-	-	argtuple
tol	-	-	1e-12
maxiter	-	-	50
fprime2	-	-	None
)	-	-	-
return dm	-	-	-

accrued_interest

Calculate the amount of coupon that has accrued between the previous coupon date and the settlement date. Ex-dividend dates are not handled. Contact me if you need this functionality.

```
accrued_interest(settle_dt: Date, next_cpn: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
next_cpn	float	-	-

print_payments

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
print_payments(settle_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

f

Function used to do solve root search in DM calculation

```
_f(dm, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dm	-	-	-
*args	-	-	-

6.6 bond_future

Class: BondFuture

Class for managing futures contracts on government bonds that follows CME conventions and related analytics.

Attributes ——— ticker_name : str Identifier name of the futures contract. first_delivery_dt : Date The first delivery date of the futures contract (usually IMM date). last_delivery_dt : Date The last delivery date of the futures contract. contract_size : int Contract size in currency units (e.g. 100,000).cpn : float Contract standard coupon rate, used for conversion factor calculations.

BondFuture

Initialize BondFuture instance. Parameters ——— ticker_name : str The futures contract ticker symbol. first_delivery_dt : Date First delivery date (IMM date). last_delivery_dt : Date Last delivery date. contract_size : int Contract size in currency units. cpn : float Standard coupon rate of the contract (percent).

```
BondFuture(ticker_name: str,
first_delivery_dt: Date,
last_delivery_dt: Date,
contract_size: int,
cpn: float,
"""
Initialize_BondFuture_instance.
Parameters
-----
ticker_name_: str
        The_futures_contract_ticker_symbol.
first_delivery_dt_: Date
        First_delivery_date_(IMM_date).
last_delivery_dt_: Date
        Last_delivery_date.
contract_size_: int
        Contract_size_in_currency_units.
cpn_: float
        Standard_coupon_rate_of_the_contract_(percent).
"""
check_argument_types(.BondFuture, locals())
.ticker_name = ticker_name
.first_delivery_dt = first_delivery_dt # This is the IMM date
.last_delivery_dt = last_delivery_dt
.contract_size = contract_size
.cpn = cpn
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ticker_name	str	-	-
first_delivery_dt	Date	-	-
last_delivery_dt	Date	-	-
contract_size	int	-	-
cpn	float	-	-
*****	-	-	-
Initialize BondFuture instance.	-	-	-
Parameters	-	-	-
-----	-	-	-
ticker_name	str	-	-
The futures contract ticker symbol.	-	-	-
first_delivery_dt	Date	-	-
First delivery date (IMM date).	-	-	-
last_delivery_dt	Date	-	-
Last delivery date.	-	-	-
contract_size	int	-	-
Contract size in currency units.	-	-	-
cpn	float	-	-
Standard coupon rate of the contract (percent).	-	-	-
*****	-	-	-
check_argument_types(.BondFuture, locals())	-	-	-
.ticker_name	-	-	ticker_name
.first_delivery_dt	-	This is the IMM date	first_delivery_dt
.last_delivery_dt	-	-	last_delivery_dt
.contract_size	-	-	contract_size
.cpn	-	-	cpn

conversion_factor

Compute the CME-style conversion factor for a given bond. The conversion factor normalizes bonds with different coupons and maturities to a standard contract coupon and maturity schedule. Steps: - Calculate years to maturity from 1st delivery date to bond maturity. - Round this maturity to the nearest 3-month (quarter year) multiple. - Construct a hypothetical bond starting on the first delivery date with the standard contract coupon and rounded maturity. - Calculate the clean price of this hypothetical bond yielding the contract coupon. - Normalize and round to 4 decimal places. Parameters ----- bond : Bond The deliverable bond for which conversion factor is calculated. Returns ----- float Conversion factor rounded to 4 decimals.

```
conversion_factor(bond: Bond):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bond	Bond	-	-

principal_invoice

Calculate the principal invoice amount for delivering a bond. CME defines this as $\text{contract_size} \times \text{futures_price} \times \text{conversion_factor} / 100$. Parameters ——— *bond* : Bond The deliverable bond. *futures_price* : float The quoted futures price (in percentage points). Returns ——— float Principal invoice amount in currency units.

```
principal_invoice(bond: Bond, futures_price: float) -> float:
"""
Calculate the principal invoice amount for delivering a bond.
CME defines this as contract_size * futures_price * conversion_factor / 100.
Parameters
-----
bond: Bond
    The deliverable bond.
futures_price: float
    The quoted futures price (in percentage points).
Returns
-----
float
    Principal invoice amount in currency units.
"""
cf = .conversion_factor(bond)
pip = .contract_size * (futures_price * cf) / 100.0
return pip
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
bond	Bond	-	
futures_price	float)-float:	-	
"""	-	-	
Calculate the principal invoice amount for delivering a bond.	-	-	
CME defines this as contract size × futures price × conversion factor / 100.	-	-	
Parameters	-	-	
-----	-	-	
bond	Bond	-	
The deliverable bond.	-	-	
futures_price	float	-	
The quoted futures price (in percentage points).	-	-	
Returns	-	-	
-----	-	-	
float	-	-	
Principal invoice amount in currency units.	-	-	
"""	-	-	
cf	-	-	.conve
pip	-	-	.contract_size *
return pip	-	-	

total_invoice_amount

Calculate total invoice amount paid on delivery, including accrued interest. Parameters ——— settle_dt : Date Settlement date when the invoice is paid. bond : Bond The deliverable bond. futures_price : float Quoted futures price. Returns ——— float Total invoice amount including accrued interest.

```
total_invoice_amount(settle_dt: Date, bond: Bond, futures_price: float
    float:
    """
    Calculate_total_invoice_amount_paid_on_delivery,_including
    accrued_interest.
    Parameters
    -----
    settle_dt_: Date
    Settlement_date_when_the_invoice_is_paid.
    bond_: Bond
    The_deliverable_bond.
    futures_price_: float
    Quoted_futures_price.
    Returns
    -----
    float
    Total_invoice_amount_including_accrued_interest.
    """
    cf = .conversion_factor(bond)
    pip = .contract_size * (futures_price * cf) / 100.0
    accrued = bond.accrued_interest(settle_dt) * .contract_size / 100.0
    tia = pip + accrued
    return tia
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
bond	Bond	-	-
futures_price	float	-	-
float		-	-
""""	-	-	-
Calculate total invoice amount paid on delivery	-	-	-
including	-	-	-
accrued interest.	-	-	-
Parameters	-	-	-
-----	-	-	-
settle_dt	Date	-	-
Settlement date when the invoice is paid.	-	-	-
bond	Bond	-	-
The deliverable bond.	-	-	-
futures_price	float	-	-
Quoted futures price.	-	-	-
Returns	-	-	-
-----	-	-	-
float	-	-	-
Total invoice amount including accrued interest.	-	-	-
""""	-	-	-
cf	-	-	.conversion_factor(bond)
pip	-	-	.contract_size * (futures_price * cf) / 100.0
accrued	-	-	bond.accrued_interest(settle_dt) * .contract_size
tia	-	-	pip + accrued
return tia	-	-	-

delivery_years

Calculate the fractional years from the first delivery date to the bond maturity. Uses ACT/ACT ISDA day count convention. Parameters ----- bond : Bond The deliverable bond. Returns ----- float Fractional years (e.g., 5.25 years).

```

delivery_years(bond: Bond) -> float:
"""
Calculate_the_fractional_years_from_the_first_delivery_date_to_the
bond_maturity.
Uses_ACT/ACT_ISDA_day_count_convention.
Parameters
-----
bond_: Bond
    The_deliverable_bond.
Returns
-----
float
    Fractional_years_(e.g.,_5.25_years).
"""

```

```

dc = DayCount(DayCountTypes.ACT_ACT_ISDA)
year_frac, _, _ = dc.year_frac(.first_delivery_dt, bond.maturity_dt)
years = int(year_frac)
months = int(12 * (year_frac - years))
del_years = years + months / 12
return del_years
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bond	Bond)-float:	-	-
"""	-	-	-
Calculate the fractional years from the first delivery date to the	-	-	-
bond maturity.	-	-	-
Uses ACT/ACT ISDA day count convention.	-	-	-
Parameters	-	-	-
———	-	-	-
bond	Bond	-	-
The deliverable bond.	-	-	-
Returns	-	-	-
———	-	-	-
float	-	-	-
Fractional years (e.g., 5.25 years).	-	-	-
"""	-	-	-
dc	-	-	DayCount(DayCountTyp
year_frac	-	-	-
-	-	-	-
-	-	-	dc.year_frac(.first_delivery
years	-	-	int(year_
months	-	-	int(12 * (year_fi
del_years	-	-	years + mor
return del_years	-	-	-

gross_basis

Compute the gross basis: what is received on delivery vs. bond price. gross_basis = clean_price - (conversion_factor × futures_price) Parameters ——— bond : Bond The deliverable bond. clean_price : float The current clean price of the bond. futures_price : float The quoted futures price. Returns ——— float Gross basis value.

```

gross_basis(bond: Bond, clean_price: float, futures_price: float
float:
"""
Compute_the_gross_basis:_what_is_received_on_delivery_vs._bond_price.
gross_basis=_clean_price_-(conversion_factor_ _futures_price)
Parameters

```

```

-----
bond_: _Bond
    _The_deliverable_bond.
clean_price_: _float
    _The_current_clean_price_of_the_bond.
futures_price_: _float
    _The_quoted_futures_price.
Returns
-----
float
    _Gross_basis_value.
    """
cf = .conversion_factor(bond)
gross_basis = clean_price - cf * futures_price
return gross_basis
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bond	Bond	-	-
clean_price	float	-	-
futures_price	float	-	-
float		-	-
"""	-	-	-
Compute the gross basis	what is received on delivery vs. bond price.	-	-
gross_basis	-	-	clean_price - (conversion_factor * futures_price)
Parameters	-	-	-
-----	-	-	-
bond	Bond	-	-
The deliverable bond.	-	-	-
clean_price	float	-	-
The current clean price of the bond.	-	-	-
futures_price	float	-	-
The quoted futures price.	-	-	-
Returns	-	-	-
-----	-	-	-
float	-	-	-
Gross basis value.	-	-	-
"""	-	-	-
cf	-	-	.conversion_factor(bond)
gross_basis	-	-	clean_price - cf * futures_price
return gross_basis	-	-	-

net_basis

Compute net basis, adjusting for financing cost (repo rate). $net_basis = forward_price - (conversion_factor \times futures_price)$ Parameters ----- bond : Bond Deliverable bond. settle_dt : Date Settlement date.

clean_price : float Current bond clean price. *futures_price* : float Futures contract price. *repo_rate* : float Repo financing rate (annualized decimal, e.g. 0.02 for 2Returns ——— float Net basis value. Raises ——— *ValueError* If any price or *repo_rate* is negative.

```

net_basis(bond,
          settle_dt: Date,
          clean_price: float,
          futures_price: float,
          repo_rate: float,
          """
Compute_net_basis, _adjusting_for_financing_cost_(repo_rate).
net_basis = _forward_price_(conversion_factor_ _futures_price)
Parameters
-----
bond_: Bond
      Deliverable_bond.
settle_dt_: Date
      Settlement_date.
clean_price_: float
      Current_bond_clean_price.
futures_price_: float
      Futures_contract_price.
repo_rate_: float
      Repo_financing_rate_(annualized_decimal, _e.g._0.02_for_2%).
Returns
-----
float
      Net_basis_value.
Raises
-----
ValueError
      If_any_price_or_repo_rate_is_negative.
"""
if clean_price < 0 or futures_price < 0 or repo_rate < 0:
    raise ValueError("Prices_and_repo_rate_must_be_non-negative")
fwd_date = .last_delivery_dt
fwd_price = bond.forward_price(settle_dt, fwd_date, clean_price, repo_rate)
cf = .conversion_factor(bond)
net_basis = fwd_price - cf * futures_price
return net_basis
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bond	-	-	-
settle_dt	Date	-	-
clean_price	float	-	-
futures_price	float	-	-
repo_rate	float	-	-
"""	-	-	-
Compute net basis	-	-	-
adjusting for financing cost (repo rate).	-	-	-
net_basis	-	-	forward_price - (conversion_factor * futures_price)
Parameters	-	-	-
-----	-	-	-
bond	Bond	-	-
Deliverable bond.	-	-	-
settle_dt	Date	-	-
Settlement date.	-	-	-
clean_price	float	-	-
Current bond clean price.	-	-	-
futures_price	float	-	-
Futures contract price.	-	-	-
repo_rate	float	-	-
Repo financing rate (annualized decimal, e.g. 0.02 for 2%).	-	-	-
Returns	-	-	-
-----	-	-	-
float	-	-	-
Net basis value.	-	-	-
Raises	-	-	-
-----	-	-	-
ValueError	-	-	-
If any price or repo_rate is negative.	-	-	-
"""	-	-	-
if clean_price < 0 or futures_price < 0 or repo_rate < 0	-	-	-
raise ValueError("Prices and repo rate must be non-negative")	-	-	-
fwd_date	-	-	.last_delivery_date
fwd_price	-	-	bond.forward_price(settle_dt, fwd_date)
cf	-	-	.conversion_factor
net_basis	-	-	fwd_price - cf * futures_price
return net_basis	-	-	-

implied_repo_rate

PLEASE ADD A FUNCTION DESCRIPTION

```
implied_repo_rate(bond: Bond,
                  settle_dt: Date,
                  clean_price: float,
```

```

futures_price: float,
"""
Calculate implied repo rate consistent with futures price.
This is the IRR of financing the bond purchase and delivering at
futures price, accounting for accrued interest and coupon payments
between settle and delivery.
Parameters
-----
bond: Bond
    Deliverable bond.
settle_dt: Date
    Settlement date.
clean_price: float
    Current clean price of bond.
futures_price: float
    Futures price.
Returns
-----
float
    Implied repo rate (annualized decimal).
Raises
-----
ValueError
    If clean price or futures price is negative.
"""
if clean_price < 0 or futures_price < 0:
    raise ValueError("Prices and futures price must be non-negative")
delivery_dt = .last_delivery_dt
days_in_year = 360 # AGREES WITH BLOOMBERG
# Accrued interest on delivery date
ai_settle = bond.accrued_interest(settle_dt)
ai_delivery = bond.accrued_interest(delivery_dt)
full_price = clean_price + ai_settle
fv_cpns = 0.0
weighted_days_sum = 0.0
for dt, amt in zip(bond.cpn_dts[1:], bond.flow_amounts[1:]):

```

The function arguments are described in the following table.

Argument Name	Type	Description
bond	Bond	-
settle_dt	Date	-
clean_price	float	-
futures_price	float	-
"""	-	-
Calculate implied repo rate consistent with futures price.	-	-
This is the IRR of financing the bond purchase and delivering at	-	-
futures price	-	-
accounting for accrued interest and coupon payments	-	-
between settle and delivery.	-	-
Parameters	-	-
-----	-	-
bond	Bond	-
Deliverable bond.	-	-
settle_dt	Date	-
Settlement date.	-	-
clean_price	float	-
Current clean price of bond.	-	-
futures_price	float	-
Futures price.	-	-
Returns	-	-
-----	-	-
float	-	-
Implied repo rate (annualized decimal).	-	-
Raises	-	-
-----	-	-
ValueError	-	-
If clean_price or futures_price is negative.	-	-
"""	-	-
if clean_price < 0 or futures_price < 0	-	-
raise ValueError("Prices and futures price must be non-negative")	-	-
delivery_dt	-	-
days_in_year	-	AGREES WITH BLOOMBERG
ai_settle	-	bond.accr
ai_delivery	-	bond.accr
full_price	-	clean
fv_cpns	-	-
weighted_days_sum	-	-
for dt	-	-

ctd

Determine the Cheapest to Deliver (CTD) bond among candidates. The CTD is the bond with the highest gross basis (= clean price - cf × futures price). Parameters ----- bonds : list of Bond List of deliverable

bonds. *bond_clean_prices* : list of float Corresponding clean prices of the bonds. *futures_price* : float Futures contract price. Returns ——— Bond The bond identified as Cheapest to Deliver.

```
ctd(bonds: list, bond_clean_prices: list, futures_price: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bonds	list	-	-
bond_clean_prices	list	-	-
futures_price	float	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.7 bond_market

Enumerated Type: BondMarkets

This enumerated type has the following values:

- AUSTRIA
- BELGIUM
- CYPRUS
- ESTONIA
- FINLAND
- FRANCE
- GERMANY
- GREECE
- IRELAND
- ITALY
- LATVIA
- LITHUANIA
- LUXEMBOURG
- MALTA
- NETHERLANDS
- PORTUGAL
- SLOVAKIA
- SLOVENIA
- SPAIN
- ESM
- EFSF
- BULGARIA
- CROATIA
- CZECH_REPUBLIC
- DENMARK
- HUNGARY
- POLAND
- ROMANIA
- SWEDEN
- JAPAN
- SWITZERLAND
- UNITED_KINGDOM
- UNITED_STATES
- AUSTRALIA
- NEW_ZEALAND
- NORWAY
- SOUTH_AFRICA
- annual
- semi_annual
- act_act
- thirtye360

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
country	-	-	-

6.8 bond_mortgage

Enumerated Type: BondMortgageTypes

This enumerated type has the following values:

- REPAYMENT
- INTEREST_ONLY
- freq_type:FrequencyTypes
- cal_type:CalendarTypes
- bd_type:BusDayAdjustTypes
- dg_type:DateGenRuleTypes
- dc_type:DayCountTypes
- self.start_dt
- self.end_dt
- self.principal
- self.freq_type
- self.cal_type
- self.bd_type
- self.dg_type
- self.dc_type
- self.schedule
- self.mortgage_type
- self.interest_flows
- self.principal_flows
- self.principal_remaining
- self.total_flows
- frequency
- num_flows
- p
- m
- m
- self.mortgage_type
- self.interest_flows
- self.principal_flows
- self.principal_remaining
- self.total_flows
- num_flows
- principal
- frequency
- ifmortgage_type
- monthly_flow
- elifmortgage_type
- monthly_flow
- interest_flow
- principal_flow
- principal

- num_flows
- s
- s+
- s+
- s+
- s+
- s+
- s+
- s+

Class: BondMortgage

A mortgage is a vector of dates and flows generated in order to repay a fixed amount given a known interest rate. Payments are all the same amount but with a varying mixture of interest and repayment of principal.

BondMortgage

Create the mortgage using start and end dates and principal.

```
BondMortgage(start_dt: Date,
end_dt: Date,
principal: float,
freq_type: FrequencyTypes = FrequencyTypes.MONTHLY,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
"""Create the mortgage using start and end dates and principal."""
check_argument_types(.BondMortgage, locals())
if start_dt > end_dt:
    raise FinError("Start_Date_after_End_Date")
.start_dt = start_dt
.end_dt = end_dt
.principal = principal
.freq_type = freq_type
.cal_type = cal_type
.bd_type = bd_type
.dg_type = dg_type
.dc_type = dc_type
.schedule = Schedule(start_dt,
                     end_dt,
                     .freq_type,
                     .cal_type,
                     .bd_type,
                     .dg_type,
)
.mortgage_type = None
.interest_flows = None
.principal_flows = None
.principal_remaining = None
.total_flows = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	Date	-	-
end_dt	Date	-	-
principal	float	-	-
freq_type	FrequencyTypes	-	FrequencyTypes
cal_type	CalendarTypes	-	CalendarTypes
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes
dg_type	DateGenRuleTypes	-	DateGenRuleTypes
dc_type	DayCountTypes	-	DayCountTypes
"""Create the mortgage using start and end dates and principal."""			
check_argument_types(.BondMortgage, locals())	-	-	-
if start_dt > end_dt	-	-	-
raise FinError("Start Date after End Date")	-	-	-
.start_dt	-	-	-
.end_dt	-	-	-
.principal	-	-	-
.freq_type	-	-	-
.cal_type	-	-	-
.bd_type	-	-	-
.dg_type	-	-	-
.dc_type	-	-	-
end_dt	-	-	-
.freq_type	-	-	-
.cal_type	-	-	-
.bd_type	-	-	-
.dg_type	-	-	-
)	-	-	-
.mortgage_type	-	-	-
.interest_flows	-	-	-
.principal_flows	-	-	-
.principal_remaining	-	-	-
.total_flows	-	-	-

repayment_amount

Determine monthly repayment amount based on current zero rate.

```
repayment_amount(zero_rate: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
zero_rate	float	-	-

generate_flows*PLEASE ADD A FUNCTION DESCRIPTION*

```

generate_flows(zero_rate: float, mortgage_type: BondMortgageTypes
"""Generate the bond flow amounts."""
    .mortgage_type = mortgage_type
    .interest_flows = [0]
    .principal_flows = [0]
    .principal_remaining = [.principal]
    .total_flows = [0]
    num_flows = len(.schedule.adjusted_dts)
    principal = .principal
    frequency = annual_frequency(.freq_type)
    if mortgage_type == BondMortgageTypes.REPAYMENT:
        monthly_flow = .repayment_amount(zero_rate)
    elif mortgage_type == BondMortgageTypes.INTEREST_ONLY:
        monthly_flow = zero_rate * .principal / frequency
    else:
        raise FinError("Unknown_Mortgage_type.")
    for _ in range(1, num_flows):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
zero_rate	float	-	-
mortgage_type	BondMortgageTypes	-	-
"""Generate the bond flow amounts."""	-	-	-
.mortgage_type	-	-	mortgage_type
.interest_flows	-	-	[0]
.principal_flows	-	-	[0]
.principal_remaining	-	-	[.principal]
.total_flows	-	-	[0]
num_flows	-	-	len(.schedule.adjusted_dts)
principal	-	-	.principal
frequency	-	-	annual_frequency(.freq_type)
if mortgage_type	-	-	= BondMortgageTypes.REPAYME
monthly_flow	-	-	.repayment_amount(zero_rate)
elif mortgage_type	-	-	= BondMortgageTypes.INTEREST_C
monthly_flow	-	-	zero_rate * .principal / frequency
else	-	-	-
raise FinError("Unknown Mortgage type.")	-	-	-

print_leg*PLEASE ADD A FUNCTION DESCRIPTION*

```

print_leg():

```

The function arguments are described in the following table.

`__repr__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.9 bond_option

Enumerated Type: BondModelTypes

This enumerated type has the following values:

- BLACK
- HO_LEE
- HULL_WHITE
- BLACK_KARASINSKI
- self.expiry_dt
- self.strike_price
- self.bond
- self.opt_type
- self.par
- t_exp
- t_mat
- df_times
- df_values
- flow_dts
- flow_amounts
- cpn_times
- cpn_flows
- num_flows
- pcd
- ncd
- flow_time
- ifflow_dts[i]
- ncd
- flow_time
- cpn_times
- cpn_flows
- exercise_type
- self.opt_type
- orself.opt_type
- exercise_type
- v
- s
- s+
- s+
- s+
- s+
- s+

Class: *BondOption*

Class for options on fixed coupon bonds. These are options to either buy or sell a bond on or before a specific future expiry date at a strike price that is set on trade date. A European option only allows the bond to be exercised into on a specific expiry date. An American option allows the option holder to exercise early, potentially allowing earlier coupons to be received.

BondOption

PLEASE ADD A FUNCTION DESCRIPTION

```
BondOption(bond: Bond,
expiry_dt: Date,
strike_price: float,
opt_type: OptionTypes,
check_argument_types(.BondOption, locals()))
.expiry_dt = expiry_dt
.strike_price = strike_price
.bond = bond
.opt_type = opt_type
.par = 100.0
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bond	Bond	-	-
expiry_dt	Date	-	-
strike_price	float	-	-
opt_type	OptionTypes	-	-
check_argument_types(.BondOption, locals())	-	-	-
.expiry_dt	-	-	expiry_dt
.strike_price	-	-	strike_price
.bond	-	-	bond
.opt_type	-	-	opt_type
.par	-	-	100.0

value

Value a bond option (option on a bond) using a specified model which include the Hull-White, Black-Karasinski and Black-Derman-Toy model which are all implemented as short rate tree models.

```
value(value_dt: Date, discount_curve: DiscountCurve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
model	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.10 bond_portfolio

6.11 bond_yield_curve

Class: BondYieldCurve

Class to do fitting of the yield curve and to enable interpolation of yields. Because yields assume a flat term structure for each bond, this class does not allow discounting to be done and so does not inherit from DiscountCurve. It should only be used for visualisation and simple interpolation but not for full term-structure-consistent pricing.

BondYieldCurve

Fit the curve to a set of bond yields using the type of curve specified. Bounds can be provided if you wish to enforce lower and upper limits on the respective model parameters.

```
BondYieldCurve(settle_dt: Date,
bonds: list,
ylds: Union[np.ndarray, list],
curve_fit,
"""Fit the curve to a set of bond yields using the type of curve
specified. Bounds can be provided if you wish to enforce lower and
upper limits on the respective model parameters."""
.settle_dt = settle_dt
.bonds = bonds
.ylds = np.array(ylds)
.curve_fit = curve_fit
fit_type = type(.curve_fit)
years_to_maturities = []
for bond in bonds:
    years_to_maturity = (bond.maturity_dt - settle_dt) / G_DAYS_IN_YEAR
    years_to_maturities.append(years_to_maturity)
.years_to_maturity = np.array(years_to_maturities)
if fit_type is CurveFitPolynomial:
    d = curve_fit.power
    coeffs = np.polyfit(.years_to_maturity, .ylds, deg=d)
    curve_fit.coeffs = coeffs
elif fit_type is CurveFitNelsonSiegel:
    xdata = .years_to_maturity
    ydata = .ylds
    popt, _ = scipy.optimize.curve_fit(curve_fit.interp_yield, xdata, ydata,
    bounds=curve_fit.bounds
    )
    curve_fit.beta_1 = popt[0]
    curve_fit.beta_2 = popt[1]
    curve_fit.beta_3 = popt[2]
    curve_fit.tau = popt[3]
elif fit_type is CurveFitNelsonSiegelSvensson:
    xdata = .years_to_maturity
    ydata = .ylds
    popt, _ = scipy.optimize.curve_fit(curve_fit.interp_yield, xdata, ydata,
    bounds=curve_fit.bounds
    )
    curve_fit.beta_1 = popt[0]
    curve_fit.beta_2 = popt[1]
    curve_fit.beta_3 = popt[2]
```

```

        curve_fit.beta_4 = popt[3]
        curve_fit.tau_1 = popt[4]
        curve_fit.tau_2 = popt[5]
elif fit_type is CurveFitBSpline:
    xdata = .years_to_maturity
    ydata = .ylds
    """_Cubic_splines_as_k=3_"""
    spline = splrep(xdata, ydata, k=curve_fit.power, t=curve_fit.knots)
    .curve_fit.spline = spline
else:
    raise FinError("Unrecognised_curve_fit_type.")
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
bonds	list	-
ylds	np.ndarray or list	-
curve_fit	-	-
"""Fit the curve to a set of bond yields using the type of curve specified. Bounds can be provided if you wish to enforce lower and upper limits on the respective model parameters."""	-	-
.settle_dt	-	-
.bonds	-	-
.ylds	-	-
.curve_fit	-	-
fit_type	-	-
years_to_maturities	-	-
for bond in bonds	-	-
years_to_maturity	-	(bond.maturity
years_to_maturities.append(years_to_maturity)	-	-
.years_to_maturity	-	np.
if fit_type is CurveFitPolynomial	-	-
d	-	-
coeffs	-	np.polyfit(
curve_fit.coeffs	-	-
elif fit_type is CurveFitNelsonSiegel	-	-
xdata	-	-
ydata	-	-
popt	-	-
)	-	-
curve_fit.beta_1	-	-
curve_fit.beta_2	-	-
curve_fit.beta_3	-	-
curve_fit.tau	-	-
elif fit_type is CurveFitNelsonSiegelSvensson	-	-
xdata	-	-
ydata	-	-
popt	-	-
)	-	-
curve_fit.beta_1	-	-
curve_fit.beta_2	-	-
curve_fit.beta_3	-	-
curve_fit.beta_4	-	-
curve_fit.tau_1	-	-
curve_fit.tau_2	-	-
elif fit_type is CurveFitBSpline	-	-
xdata	-	-
ydata	-	-
""" Cubic splines as k	-	-
spline	-	splrep(xdata, yda
.curve_fit.spline	-	-
else	-	-
raise FinError("Unrecognised curve fit type.")	-	-

interp_yield

PLEASE ADD A FUNCTION DESCRIPTION

```
interp_yield(maturity_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
maturity_dt	Date	-	-

plot

Display yield curve.

```
plot(title, ylabel="Yield_To_Maturity_(%)"):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
title	-	-	-
ylabel	-	-	"Yield To Maturity (%)"

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.12 bond_zero

Class: BondZero

A zero cpn bond is a bond which doesn't pay any periodic payments. Instead, it is issued at a discount. The entire face value of the bond is paid out at maturity. It is issued as a deep discount bond.

There is a special convention for accrued interest in which

$$\text{Accrued_interest} = (\text{par} - \text{issue_price}) * D$$

$$\text{where } D = (\text{settle_dt} - \text{issue_dt}) / (\text{maturity_dt} - \text{issue_dt}).$$

BondZero

Create BondZero object by providing the issue date, maturity Date, face amount and issue price.

```
BondZero(issue_dt: Date,
maturity_dt: Date,
issue_price: float, # Issue price usually discounted
"""Create_BondZero_object_by_providing_the_issue_date,_maturity_Date,
face_amount_and_issue_price."""
check_argument_types(.BondZero, locals())
if issue_dt >= maturity_dt:
    raise FinError("Issue_Date_must_preceded_maturity_date.")
.issue_dt = issue_dt
.maturity_dt = maturity_dt
.dc_type = DayCountTypes.ZERO
.issue_price = issue_price # Price of issue, usually discounted
.par = 100.0 # This is how price is quoted and amount at maturity
.freq_type = FrequencyTypes.ZERO
.cpn_dts = [issue_dt, maturity_dt]
.payment_dts = [issue_dt, maturity_dt]
.flow_amounts = [0.0, 0.0] # cpn payments are zero
.cal_type = CalendarTypes.WEEKEND
.ex_div_days = 0
.accrued_int = None
.accrued_days = None
.alpha = None
._pcd = None
._ncd = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
issue_dt	Date	-
maturity_dt	Date	-
issue_price	float	Issue price usually discounted
"""Create BondZero object by providing the issue date	-	-
maturity Date	-	-
face amount and issue price."""	-	-
check_argument_types(.BondZero, locals())	-	-
if issue_dt <	-	-
raise FinError("Issue Date must precede maturity date.")	-	-
.issue_dt	-	-
.maturity_dt	-	-
.dc_type	-	-
.issue_price	-	Price of issue, usually discounted
.par	-	This is how price is quoted and amount at maturity
.freq_type	-	-
.cpn_dts	-	-
.payment_dts	-	-
.flow_amounts	-	cpn payments are zero
.cal_type	-	-
.ex_div_days	-	-
.accrued_int	-	-
.accrued_days	-	-
.alpha	-	-
.pcd	-	-
.ncd	-	-

dirty_price_from_ytm

Calculate the full price of bond from its yield to maturity. This function is vectorised with respect to the yield input. It implements a number of standard conventions for calculating the YTM.

```
dirty_price_from_ytm(settle_dt: Date,
ytm: float,
convention: YTMCalcType = YTMCalcType.ZERO,
"""Calculate the full price of bond from its yield to maturity. This
function is vectorised with respect to the yield input. It implements
a number of standard conventions for calculating the YTM."""
if convention != YTMCalcType.ZERO:
    raise FinError("Need to use YTMCalcType.ZERO for zero cpn bond")
.accumulated_interest(settle_dt, 1.0)
ytm = np.array(ytm) # VECTORIZED
ytm = ytm + 0.000000000012345 # SNEAKY LOW-COST TRICK TO AVOID y=0
# n is the number of flows after the next cpn
n = 0
for dt in .cpn_dts:
    if dt > settle_dt:
        n += 1
n = n - 1
```

```

if n < 0:
    raise FinError("No_cpns_left")
# A zero cpn bond has a price equal to the discounted principal
# assuming an annualised rate raised to the power of years
dc = DayCount(.dc_type)
(acc_factor, _, _) = dc.year_frac(settle_dt, .maturity_dt, .maturity_dt, FrequencyTypes
    .ZERO
)
if acc_factor <= 1:
    pv = .par / (1.0 + ytm * acc_factor)
else:
    pv = .par / (1.0 + ytm) ** acc_factor
return pv
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
ytm	float	-
convention	YTMCalcType	-
"""Calculate the full price of bond from its yield to maturity. This function is vectorised with respect to the yield input. It implements a number of standard conventions for calculating the YTM."""	-	-
if convention !=	-	-
raise FinError("Need to use YTMCalcType.ZERO for zero cpn bond")	-	-
.accrued_interest(settle_dt, 1.0)	-	-
ytm	-	VECTORIZED
ytm	-	SNEAKY LOW-COST TRICK
n	-	-
for dt in .cpn_dts		-
if dt > settle_dt		-
n +	-	-
n	-	-
if n > 0		-
raise FinError("No cpns left")	-	-
dc	-	-
)	-	-
if acc_factor >	-	-
pv	-	-
else		-
pv	-	-
return pv	-	-

principal

Calculate the principal value of the bond based on the face amount from its discount margin and making

assumptions about the future Ibor rates.

```
principal(settle_dt: Date,
          ytm: float,
          face: float,
          convention: YTMCalcType = YTMCalcType.ZERO,
          """Calculate the principal value of the bond based on the face
          amount from its discount margin and making assumptions about the
          future Ibor rates."""
          dirty_price = .dirty_price_from_ytm(settle_dt, ytm, convention)
          principal = dirty_price * face / .par
          principal = principal - .accrued_int
          return principal
          #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	D
settle_dt	Date	-	
ytm	float	-	
face	float	-	
convention	YTMCalcType	-	YTM
"""Calculate the principal value of the bond based on the face amount from its discount margin and making assumptions about the future Ibor rates."""	-	-	
dirty_price	-	-	.dirty_price_from.y
principal	-	-	dirty.p
principal	-	-	princi
return principal	-	-	

dollar_duration

Calculate the risk or dP/dy of the bond by bumping. This is also known as the DV01 in Bloomberg.

```
dollar_duration(settle_dt: Date,
                ytm: float,
                convention: YTMCalcType = YTMCalcType.ZERO,
                """Calculate the risk or dP/dy of the bond by bumping. This is also
                known as the DV01 in Bloomberg."""
                dy = 0.0001 # 1 basis point
                p0 = .dirty_price_from_ytm(settle_dt, ytm - dy, convention)
                p2 = .dirty_price_from_ytm(settle_dt, ytm + dy, convention)
                d = -(p2 - p0) / dy / 2.0
                return d
                #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
settle_dt	Date	-	
ymt	float	-	
convention	YTMCalcType	-	YT
"""Calculate the risk or dP/dy of the bond by bumping. This is also known as the DV01 in Bloomberg."""		-	-
dy	-	1 basis point	
p0	-	-	.dirty_price_from_y
p2	-	-	.dirty_price_from_y
d	-	-	-(
return d	-	-	

macauley_duration

Calculate the Macauley duration of the bond on a settlement date given its yield to maturity.

```
macauley_duration(settle_dt: Date,
ymt: float,
convention: YTMCalcType = YTMCalcType.ZERO,
"""Calculate the Macauley duration of the bond on a settlement date
given its yield to maturity."""
dd = .dollar_duration(settle_dt, ymt, convention)
fp = .dirty_price_from_ymt(settle_dt, ymt, convention)
md = dd * (1.0 + ymt) / fp
return md
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	
settle_dt	Date	-	
ymt	float	-	
convention	YTMCalcType	-	YTM
"""Calculate the Macauley duration of the bond on a settlement date given its yield to maturity."""		-	-
dd	-	-	.dollar_duration
fp	-	-	.dirty_price_from_y
md	-	-	dd *
return md	-	-	

modified_duration

Calculate the modified duration of the bond on a settlement date given its yield to maturity.

```
modified_duration(settle_dt: Date,
ymt: float,
convention: YTMCalcType = YTMCalcType.ZERO,
"""Calculate the modified duration of the bond on a settlement date
```

```

given_its_yield_to_maturity."""
dd = .dollar_duration(settle_dt, ytm, convention)
fp = .dirty_price_from_ytm(settle_dt, ytm, convention)
md = dd / fp
return md
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	De
settle_dt	Date	-	
ytm	float	-	
convention	YTMCalcType	-	YTM
"""Calculate the modified duration of the bond on a settlement date given its yield to maturity."""	-	-	
dd	-	-	.dollar_duration(s
fp	-	-	.dirty_price_from_yt
md	-	-	
return md	-	-	

convexity_from_ytm

Calculate the bond convexity from the yield to maturity. This function is vectorised with respect to the yield input.

```

convexity_from_ytm(settle_dt: Date,
ytm: float,
convention: YTMCalcType = YTMCalcType.ZERO,
"""Calculate the bond convexity from the yield to maturity. This
function is vectorised with respect to the yield input."""
dy = 0.0001
p0 = .dirty_price_from_ytm(settle_dt, ytm - dy, convention)
p1 = .dirty_price_from_ytm(settle_dt, ytm, convention)
p2 = .dirty_price_from_ytm(settle_dt, ytm + dy, convention)
conv = ((p2 + p0) - 2.0 * p1) / dy / dy / p1 / .par
return conv
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	
ytm	float	-	
convention	YTMCalcType	-	YTMCalcType.ZERO
"""Calculate the bond convexity from the yield to maturity. This function is vectorised with respect to the yield input."""		-	
dy	-	-	
p0	-	-	.dirty_price_from_ytm(settle_dt, ytm, convention)
p1	-	-	.dirty_price_from_ytm(settle_dt, ytm, convention, dy)
p2	-	-	.dirty_price_from_ytm(settle_dt, ytm, convention, dy, p0)
conv	-	-	((p2 + p0) - 2.0 * p1) / (dy ** 2)
return conv	-	-	

clean_price_from_ytm

Calculate the bond clean price from the yield to maturity. This function is vectorised with respect to the yield input.

```
clean_price_from_ytm(settle_dt: Date,
                    ytm: float,
                    convention: YTMCalcType = YTMCalcType.ZERO,
                    """Calculate the bond clean price from the yield to maturity. This
                    function is vectorised with respect to the yield input."""
                    dirty_price = .dirty_price_from_ytm(settle_dt, ytm, convention)
                    accrued = .accrued_interest(settle_dt, .par)
                    clean_price = dirty_price - accrued
                    return clean_price
                    #####)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	
ytm	float	-	
convention	YTMCalcType	-	YTMCalcType.ZERO
"""Calculate the bond clean price from the yield to maturity. This function is vectorised with respect to the yield input."""		-	
dirty_price	-	-	.dirty_price_from_ytm(settle_dt, ytm, convention)
accrued	-	-	.accrued_interest(settle_dt, .par)
clean_price	-	-	dirty_price - accrued
return clean_price	-	-	

clean_price_from_discount_curve

Calculate the clean bond value using some discount curve to present-value the bond's cash flows back to the curve anchor date and not to the settlement date.

```

clean_price_from_discount_curve(settle_dt: Date, discount_curve: DiscountCurve
    """Calculate the clean bond value using some discount curve to
    present-value the bond's cash flows back to the curve anchor date and
    not to the settlement date."""
    dirty_price = .dirty_price_from_discount_curve(settle_dt, discount_curve)
    accrued = .accrued_interest(settle_dt, .par)
    clean_price = dirty_price - accrued
    return clean_price
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
discount_curve	DiscountCurve	-
"""Calculate the clean bond value using some discount curve to	-	-
present-value the bond's cash flows back to the curve anchor date and	-	-
not to the settlement date."""	-	-
dirty_price	-	.dirty_price from.
accrued	-	.acc
clean_price	-	-
return clean_price	-	-

dirty_price_from_discount_curve

Calculate the bond price using a provided discount curve to PV the bond's cash flows to the settlement date. As such it is effectively a forward bond price if the settlement date is after the valuation date.

```

dirty_price_from_discount_curve(settle_dt: Date, discount_curve: DiscountCurve
    """Calculate the bond price using a provided discount curve to PV the
    bond's cash flows to the settlement date. As such it is effectively a
    forward bond price if the settlement date is after the valuation date.
    """
    if settle_dt < discount_curve.value_dt:
        raise FinError("Bond settles before Discount curve date")
    if settle_dt > .maturity_dt:
        raise FinError("Bond settles after it matures.")
    px = 0.0
    df = 1.0
    df_settle = discount_curve.df(settle_dt,
    )
    for dt in .cpn_dts[1:]:
        # cpns paid on the settlement date are paid to the seller
        if dt > settle_dt:
            df = discount_curve.df(dt)
            flow = 0
            pv = flow * df
            px += pv
    px += df * .par
    px = px / df_settle
    return px * .par

```

```
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
"""Calculate the bond price using a provided discount curve to PV the bond's cash flows to the settlement date. As such it is effectively a forward bond price if the settlement date is after the valuation date.	-	-	-
"""	-	-	-
if settle_dt < discount_curve.value_dt	-	-	-
raise FinError("Bond settles before Discount curve date")	-	-	-
if settle_dt > .maturity_dt	-	-	-
raise FinError("Bond settles after it matures.")	-	-	-
px	-	-	0.0
df	-	-	1.0
)	-	-	-
for dt in .cpn_dts[1	]:	-	-
if dt < settle_dt	-	-	-
df	-	-	discount_curve.d
flow	-	-	0
pv	-	-	flow * df
px +	-	-	pv
px +	-	-	df * .par
px	-	-	px / df_settle
return px * .par	-	-	-

current_yield

Calculate the current yield of the bond which is the cpn divided by the clean price (not the full price). The cpn of a zero cpn bond is defined as: (par - issue_price) / tenor

```
current_yield(clean_price):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
clean_price	-	-	-

yield_to_maturity

PLEASE ADD A FUNCTION DESCRIPTION

```
yield_to_maturity(settle_dt: Date,
```

```

clean_price: float,
convention: YTMCalcType = YTMCalcType.ZERO,
"""Calculate the bond's yield to maturity by solving the price
yield relationship using a one-dimensional root solver."""
if isinstance(clean_price, float) or isinstance(clean_price, np.float64):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
clean_price	float	-	-
convention	YTMCalcType	-	YTMCalcType.ZERO
"""Calculate the bond's yield to maturity by solving the price yield relationship using a one-dimensional root solver."""	-	-	-

accrued interest

Calculate the amount of cpn that has accrued between the previous cpn date and the settlement date. Note that for some day count schemes (such as 30E/360) this is not actually the number of days between the previous cpn payment date and settlement date. If the bond trades with ex-cpn dates then you need to supply the number of days before the cpn date the ex-cpn date is. You can specify the calendar to be used - NONE means only calendar days, WEEKEND is only weekends or you can specify a country calendar for business days.

```

accrued_interest(settle_dt: Date, face: float):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	-

asset_swap_spread

Calculate the par asset swap spread of the bond. The discount curve is a Ibor curve that is passed in. This function is vectorised with respect to the clean price.

```

asset_swap_spread(settle_dt: Date,
clean_price: float,
discount_curve: DiscountCurve,
swap_float_dc_type=DayCountTypes.ACT_360,
swap_float_freq_type=FrequencyTypes.SEMI_ANNUAL,
swap_float_cal_type=CalendarTypes.WEEKEND,
swap_float_bus_day_adjust_rule_type=BusDayAdjustTypes.FOLLOWING,
swap_float_dategen_type=DateGenRuleTypes.BACKWARD,
"""Calculate the par asset swap spread of the bond. The discount curve
is a Ibor curve that is passed in. This function is vectorised with
respect to the clean price."""

```

```

clean_price = np.array(clean_price)
.accumulated_interest(settle_dt, 1.0)
accrued_amount = .accrued_int * .par
bond_price = clean_price + accrued_amount
# Calculate the price of the bond discounted on the Ibor curve
pv_ibor = 0.0
prev_dt = ._pcd
df = 0.0
for dt in .cpn_dts[1:]:
    # cpns paid on the settlement date are paid to the seller
    if dt > settle_dt:
        df = discount_curve.df(dt)
        # pv_ibor += df * .cpn / .freq
pv_ibor += df * .par
# Calculate the PV01 of the floating leg of the asset swap
# I assume here that the cpn starts accruing on the settlement date
prev_dt = ._pcd
schedule = Schedule(settle_dt,
                    .maturity_dt,
                    swap_float_freq_type,
                    swap_float_cal_type,
                    swap_float_bus_day_adjust_rule_type,
                    swap_float_dategen_type,
                    )
day_count = DayCount(swap_float_dc_type)
prev_dt = ._pcd
pv01 = 0.0
for dt in schedule.adjusted_dts[1:]:
    df = discount_curve.df(dt)
    year_frac = day_count.year_frac(prev_dt, dt)[0]
    pv01 = pv01 + year_frac * df
    prev_dt = dt
asw = (pv_ibor - bond_price / .par) / pv01
return asw
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	
clean_price	float	-	
discount_curve	DiscountCurve	-	
swap_float_dc_type	-	-	DayCountT
swap_float_freq_type	-	-	FrequencyType
swap_float_cal_type	-	-	CalendarTy
swap_float_bus_day_adjust_rule_type	-	-	BusDayAdjustT
swap_float_dategen_type	-	-	DateGenRuleT
"""Calculate the par asset swap spread of the bond. The discount curve		-	-
is a Ibor curve that is passed in. This function is vectorised with		-	-
respect to the clean price."""		-	-
clean_price	-	-	np.array
.accrued_interest(settle_dt, 1.0)	-	-	
accrued_amount	-	-	.accrue
bond_price	-	-	clean_price +
pv_ibor	-	-	
prev_dt	-	-	.
df	-	-	
for dt in .cpn_dts[1	]:	-	
if dt > settle_dt		-	
df	-	-	discount
p_v_ibor +	-	-	df
prev_dt	-	-	.
.maturity_dt	-	-	
swap_float_freq_type	-	-	
swap_float_cal_type	-	-	
swap_float_bus_day_adjust_rule_type	-	-	
swap_float_dategen_type	-	-	
)	-	-	
day_count	-	-	DayCount(sw
prev_dt	-	-	.
p_v01	-	-	
for dt in schedule.adjusted_dts[1	]:	-	
df	-	-	discount
year_frac	-	-	day_count.year
p_v01	-	-	p_v01 + y
prev_dt	-	-	
asw	-	-	(p_v_ibor - bond
return asw	-	-	

dirty_price_from_oas

Calculate the full price of the bond from its OAS given the bond settlement date, a discount curve and the oas as a number.

```

dirty_price_from_oas(settle_dt: Date, discount_curve: DiscountCurve, oas: float
"""Calculate the full price of the bond from its OAS given the bond
settlement_date, a discount_curve and the oas as a number."""
    .accrued_interest(settle_dt, 1.0)
df_adjusted = 0.0
pv = 0.0
for dt in .cpn_dts[1:]:
    # cpns paid on the settlement date are paid to the seller
    if dt > settle_dt:
        t = (dt - settle_dt) / G_DAYS_IN_YEAR
        t = np.maximum(t, G_SMALL)
        df = discount_curve.df(dt)
        # determine the Ibor implied zero rate
        r = np.power(df, -1.0 / t) - 1.0
        # determine the OAS adjusted zero rate
        df_adjusted = np.power(1.0 + (r + oas), -t)
pv = pv + df_adjusted * .par
pv *= .par
return pv
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
oas	float	-	-
"""Calculate the full price of the bond from its OAS given the bond	-	-	-
settlement date	-	-	-
a discount curve and the oas as a number."""	-	-	-
.accrued_interest(settle_dt, 1.0)	-	-	-
df_adjusted	-	-	0.0
pv	-	-	0.0
for dt in .cpn_dts[1	]:	-	-
if dt > settle_dt	-	-	-
t	-	(dt - settle_dt) / G.D	-
t	-	np.maximum(t,	-
df	-	discount_cu	-
r	-	np.power(df, -	-
df_adjusted	-	np.power(1.0 +	-
pv	-	pv + df_adju	-
pv *	-	.par	-
return pv	-	-	-

option_adjusted_spread

PLEASE ADD A FUNCTION DESCRIPTION

```
option_adjusted_spread(settle_dt: Date,
clean_price: float,
discount_curve: DiscountCurve,
"""Return_OAS_for_bullet_bond_given_settlement_date,_clean_bond_price
and_the_discount_relative_to_which_the_spread_is_to_be_computed."""
if isinstance(clean_price, float) or isinstance(clean_price, np.float64):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
clean_price	float	-	-
discount_curve	DiscountCurve	-	-
"""Return OAS for bullet bond given settlement date	-	-	-
clean bond price	-	-	-
and the discount relative to which the spread is to be computed."""	-	-	-

bond_payments

Print a list of the unadjusted cpn payment dates used in analytic calculations for the bond.

```
bond_payments(settle_dt: Date, face: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	-

print_payments

Print a list of the unadjusted cpn payment dates used in analytic calculations for the bond.

```
print_payments(settle_dt: Date, face: float = 100.0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
face	float	-	100.0

dirty_price_from_survival_curve

Calculate discounted present value of flows assuming default model. The survival curve treats the cpns as zero recovery payments while the recovery fraction of the par amount is paid at default. For the defaulting principal we discretize the time steps using the cpn payment times. A finer discretization may handle the

time value with more accuracy. I reduce any error by averaging period start and period end payment present values.

```
dirty_price_from_survival_curve(settle_dt: Date,
discount_curve: DiscountCurve,
survival_curve: DiscountCurve,
recovery_rate: float,
"""Calculate discounted present value of flows assuming default model.
The survival curve treats the cpns as zero recovery payments while
the recovery fraction of the par amount is paid at default. For the
defaulting principal we discretize the time steps using the cpn
payment times. A finer discretization may handle the time value with
more accuracy. I reduce any error by averaging period start and period
end payment present values."""
pv = 0.0
prev_q = 1.0
prev_df = 1.0
defaulting_principal_pv_pay_start = 0.0
defaulting_principal_pv_pay_end = 0.0
for dt in .cpn_dts[1:]:
    # cpns paid on the settlement date are paid to the seller
    if dt > settle_dt:
        df = discount_curve.df(dt)
        q = survival_curve.survival_prob(dt)
        # Add PV of cpn conditional on surviving to payment date
        # Any default results in all subsequent cpns being lost
        # with zero recovery
        dq = q - prev_q
        defaulting_principal_pv_pay_start += -dq * recovery_rate * prev_df
        defaulting_principal_pv_pay_end += -dq * recovery_rate * df
        # Add on PV of principal if default occurs in cpn period
        prev_q = q
        prev_df = df
pv = pv + 0.50 * defaulting_principal_pv_pay_start
pv = pv + 0.50 * defaulting_principal_pv_pay_end
pv = pv + df * q * .par
pv *= .par
return pv
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
discount_curve	DiscountCurve	-
survival_curve	DiscountCurve	-
recovery_rate	float	-
"""Calculate discounted present value of flows assuming default model.		
The survival curve treats the cpns as zero recovery payments while		
the recovery fraction of the par amount is paid at default. For the		
defaulting principal we discretize the time steps using the cpn		
payment times. A finer discretization may handle the time value with		
more accuracy. I reduce any error by averaging period start and period		
end payment present values."""		
pv	-	-
prev_q	-	-
prev_df	-	-
defaulting_principal_pv_pay_start	-	-
defaulting_principal_pv_pay_end	-	-
for dt in .cpn_dts[1	]:	-
if dt < settle_dt		-
df	-	-
q	-	-
dq	-	-
defaulting_principal_pv_pay_start +	-	-
defaulting_principal_pv_pay_start +	-	-
prev_q	-	-
prev_df	-	-
pv	-	-
pv	-	-
pv	-	-
pv *	-	-
return pv	-	-

clean_price_from_survival_curve

Calculate clean price value of flows assuming default model. The survival curve treats the cpns as zero recovery payments while the recovery fraction of the par amount is paid at default.

```
clean_price_from_survival_curve(settle_dt: Date,
discount_curve: DiscountCurve,
survival_curve: DiscountCurve,
recovery_rate: float,
"""Calculate clean price value of flows assuming default model.
The survival curve treats the cpns as zero recovery payments while
the recovery fraction of the par amount is paid at default."""
.accrued_interest(settle_dt, 1.0)
dirty_price = .dirty_price_from_survival_curve(settle_dt, discount_curve,
survival_curve, recovery_rate
```

```
)
clean_price = dirty_price - .accrued_int
return clean_price
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
discount_curve	DiscountCurve	-	-
survival_curve	DiscountCurve	-	-
recovery_rate	float	-	-
"""Calculate clean price value of flows assuming default model.		-	-
The survival curve treats the cpns as zero recovery payments while		-	-
the recovery fraction of the par amount is paid at default."""		-	-
.accrued_interest(settle_dt, 1.0)		-	-
)		-	-
clean_price	-	-	dirty_price - .accrued_int
return clean_price	-	-	-

calc_ror

TODO: TIDY THIS UP !!!!!!!!!!!!!!! Calculates the rate of total return (capital return and interest) given a BUY YTM and a SELL YTM of this bond. This function computes the full prices at buying and selling, plus the cpn payments during the period. It returns a tuple which includes a simple rate of return, a compounded IRR and the PnL.

```
calc_ror(begin_dt: Date,
end_dt: Date,
begin_ytm: float,
end_ytm: float,
convention: YTMCalcType = YTMCalcType.ZERO,
"""TODO: TIDY THIS UP !!!!!!!!!!!!!!!
Calculates the rate of total return (capital return and interest) given
a BUY YTM and a SELL YTM of this bond.
This function computes the full prices at buying and selling, plus the
cpn payments during the period.
It returns a tuple which includes a simple rate of return, a compounded
IRR and the PnL.
""")
buy_price = .dirty_price_from_ytm(begin_dt, begin_ytm, convention)
sell_price = .dirty_price_from_ytm(end_dt, end_ytm, convention)
dates_cfs = zip(.cpn_dts, .flow_amounts)
# The cpn or par payments on buying date belong to the buyer.
# The cpn or par payments on selling date are given to the new buyer
dates_cfs = [
    (d, c * .par)
    for (d, c) in dates_cfs
    if (d >= begin_dt) and (d < end_dt)
]
```

```

dates_cfs.append((begin_dt, -buy_price))
dates_cfs.append((end_dt, sell_price))
times_cfs = [(d - begin_dt) / 365, c] for (d, c) in dates_cfs]
pnl = sum(c for (t, c) in times_cfs)
simple_rtn = (pnl / buy_price) * 365 / (end_dt - begin_dt)
brentq_up_bound = 5
brentq_down_bound = -0.9999
# in case brentq cannot find the irr root
if simple_rtn > brentq_up_bound or simple_rtn < brentq_down_bound:
    irr = simple_rtn
else:
    irr = optimize.brentq(npv,
        # f(a) and f(b) must have opposite signs
        a=brentq_down_bound,
        b=brentq_up_bound,
        xtol=1e-8,
        args=(np.array(times_cfs),),
    )
return simple_rtn, irr, pnl
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
begin_dt	Date	-
end_dt	Date	-
begin_ytm	float	-
end_ytm	float	-
convention	YTMCalcType	-
"""TODO	TIDYTHISUP!!!!!!!!!!!!!!	-
Calculates the rate of total return (capital return and interest) given	-	-
a BUY YTM and a SELL YTM of this bond.	-	-
This function computes the full prices at buying and selling	-	-
plus the	-	-
cpn payments during the period.	-	-
It returns a tuple which includes a simple rate of return	-	-
a compounded	-	-
IRR and the PnL.	-	-
"""	-	-
buy_price	-	.dirty_
sell_price	-	.dirty_
dates_cfs	-	-
(d, c * .par)	-	-
for (d, c) in dates_cfs	-	-
if (d <	-	-
]	-	-
dates_cfs.append((begin_dt, -buy_price))	-	-
dates_cfs.append((end_dt, sell_price))	-	-
times_cfs	-	[
pnl	-	-
simple_rtn	-	-
brentq_up_bound	-	-
brentq_down_bound	-	-
if simple_rtn < brentq_up_bound or simple_rtn > brentq_down_bound	-	-
irr	-	-
else	-	-
a	-	-
b	-	-
xtol	-	-
args	-	-
)	-	-
return simple_rtn	-	-
irr	-	-
pnl	-	-

--repr--

PLEASE ADD A FUNCTION DESCRIPTION


```
__repr__():
```

The function arguments are described in the following table.

print

Print a list of the unadjusted cpn payment dates used in analytic calculations for the bond.

```
_print():
```

The function arguments are described in the following table.

f

Function used to do root search in price to yield calculation.

```
_f(ytm, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ytm	-	-	-
*args	-	-	-

g

Function used to do root search in price to OAS calculation.

```
_g(oas, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
oas	-	-	-
*args	-	-	-

6.13 bond_zero_curve

Class: *BondZeroCurve(DiscountCurve)*

BondZeroCurve

PLEASE ADD A FUNCTION DESCRIPTION

```
BondZeroCurve(value_dt: Date,
bonds: list,
clean_prices: list,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
"""Fit a discount curve to a set of bond yields using the type of
curve specified."""
if len(bonds) != len(clean_prices):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
bonds	list	-	-
clean_prices	list	-	-
interp_type	InterpTypes	-	InterpTypes.FLAT_FWD_R
"""Fit a discount curve to a set of bond yields using the type of curve specified."""	-	-	-

_bootstrap_zero_rates

PLEASE ADD A FUNCTION DESCRIPTION

```
_bootstrap_zero_rates():
```

The function arguments are described in the following table.

zero_rate

Calculate the zero rate to maturity date.

```
zero_rate(dt: Date,
freq_type: FrequencyTypes = FrequencyTypes.CONTINUOUS,
"""Calculate the zero rate to maturity date."""
t = input_time(dt, )
f = annual_frequency(freq_type)
df = .df(t)
if f == 0: # Simple interest
    zero_rate = (1.0 / df - 1.0) / t
if f == -1: # Continuous
    zero_rate = -np.log(df) / t
else:
```

```

    zero_rate = (df ** (-1.0 / t) - 1) * f
    return zero_rate
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-
freq_type	FrequencyTypes	-	FrequencyTypes.CONTINUOUS
"""Calculate the zero rate to maturity date."""	-	-	-
t	-	-	input_time(dt,)
f	-	-	annual_frequency(freq_type)
df	-	-	.df(t)
if f	-	Simple interest	= 0:
zero_rate	-	-	(1.0 / df - 1.0) / t
if f	-	Continuous	= -1:
zero_rate	-	-	-np.log(df) / t
else	-	-	-
zero_rate	-	-	(df ** (-1.0 / t) - 1) * f
return zero_rate	-	-	-

df

PLEASE ADD A FUNCTION DESCRIPTION

```
df(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

survival_prob

PLEASE ADD A FUNCTION DESCRIPTION

```
survival_prob(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

fwd

Calculate the continuous forward rate at the forward date.

```
fwd(dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dt	Date	-	-

fwd_rate

Calculate the forward rate according to the specified day count convention.

```
fwd_rate(date1: Date, date2: Date, dc_type: DayCountTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
date1	Date	-	-
date2	Date	-	-
dc_type	DayCountTypes	-	-

plot

Display yield curve.

```
plot(title: str):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
title	str	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

f*PLEASE ADD A FUNCTION DESCRIPTION*

```
_f(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

6.14 curve_fits

Class: FinCurveFitMethod

class FinCurveFitMethod:

Class: CurveFitPolynomial

CurveFitPolynomial

PLEASE ADD A FUNCTION DESCRIPTION

```
CurveFitPolynomial(power=3):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
power	-	-	3

interp_yield

PLEASE ADD A FUNCTION DESCRIPTION

```
interp_yield(t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

Class: CurveFitNelsonSiegel

class CurveFitNelsonSiegel:

CurveFitNelsonSiegel

Fairly permissive bounds. Only tau_1 is 1-100

```
CurveFitNelsonSiegel(tau=None, bounds=[(-1, -1, -1, 0.5), (1, 1, 1, 100)]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
tau	-	-	None
bounds	-	-	[(-1, -1, -1, 0.5), (1, 1, 1, 100)]

interp_yield

PLEASE ADD A FUNCTION DESCRIPTION

```
interp_yield(t, beta_1=None, beta_2=None, beta_3=None, tau=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
beta_1	-	-	None
beta_2	-	-	None
beta_3	-	-	None
tau	-	-	None

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

Class: CurveFitNelsonSiegelSvensson

class CurveFitNelsonSiegelSvensson:

CurveFitNelsonSiegelSvensson

Create object to store calibration and functional form of NSS parametric fit.

```
CurveFitNelsonSiegelSvensson(tau_1=None,
tau_2=None,
bounds=[(0, -1, -1, -1, 0, 1), (1, 1, 1, 1, 10, 100)],
"""Create object to store calibration and functional form of NSS
parametric fit."""
.parent_type = FinCurveFitMethod
.beta_1 = None
.beta_2 = None
.beta_3 = None
.beta_4 = None
.tau_1 = tau_1
.tau_2 = tau_2
"""I impose some bounds to help ensure a sensible result if
the user does not provide any bounds. Especially for tau_2."""
.bounds = bounds
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
tau_1	-	-	None
tau_2	-	-	None
bounds	-	[(0, -1, -1, -1, 0, 1), (1, 1, 1, 1, 10, 100)]	
"""Create object to store calibration and functional form of NSS parametric fit."""	-	-	-
.parent_type	-	-	FinCurveFitMethod
.beta_1	-	-	None
.beta_2	-	-	None
.beta_3	-	-	None
.beta_4	-	-	None
.tau_1	-	-	tau_1
.tau_2	-	-	tau_2
"""I impose some bounds to help ensure a sensible result if the user does not provide any bounds. Especially for tau_2."""	-	-	-
.bounds	-	-	bounds

interp_yield

PLEASE ADD A FUNCTION DESCRIPTION

```
interp_yield(t,
beta_1=None,
```



```

beta_2=None,
beta_3=None,
beta_4=None,
tau_1=None,
tau_2=None,
# Careful if we get a time zero point
t = np.maximum(t, 1e-10)
if beta_1 is None:
    beta_1 = .beta_1
if beta_2 is None:
    beta_2 = .beta_2
if beta_3 is None:
    beta_3 = .beta_3
if beta_4 is None:
    beta_4 = .beta_4
if tau_1 is None:
    tau_1 = .tau_1
if tau_2 is None:
    tau_2 = .tau_2
theta1 = t / tau_1
theta2 = t / tau_2
exp_term1 = np.exp(-theta1)
exp_term2 = np.exp(-theta2)
yld = beta_1
yld += beta_2 * (1.0 - exp_term1) / theta1
yld += beta_3 * ((1.0 - exp_term1) / theta1 - exp_term1)
yld += beta_4 * ((1.0 - exp_term2) / theta2 - exp_term2)
return yld

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
beta_1	-	-	None
beta_2	-	-	None
beta_3	-	-	None
beta_4	-	-	None
tau_1	-	-	None
tau_2	-	-	None
t	-	-	np.maximum(t, 1e-10)
if beta_1 is None		-	-
beta_1	-	-	.beta_1
if beta_2 is None		-	-
beta_2	-	-	.beta_2
if beta_3 is None		-	-
beta_3	-	-	.beta_3
if beta_4 is None		-	-
beta_4	-	-	.beta_4
if tau_1 is None		-	-
tau_1	-	-	.tau_1
if tau_2 is None		-	-
tau_2	-	-	.tau_2
theta1	-	-	t / tau_1
theta2	-	-	t / tau_2
exp_term1	-	-	np.exp(-theta1)
exp_term2	-	-	np.exp(-theta2)
yld	-	-	beta_1
yld +	-	-	beta_2 * (1.0 - exp_term1) / theta1
yld +	-	-	beta_3 * ((1.0 - exp_term1) / theta1 - exp_term1)
yld +	-	-	beta_4 * ((1.0 - exp_term2) / theta2 - exp_term2)
return yld	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

Class: CurveFitBSpline

class CurveFitBSpline:

CurveFitBSpline

PLEASE ADD A FUNCTION DESCRIPTION

```
CurveFitBSpline(power=3, knots=[1, 3, 5, 10]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
power	-	-	3
knots	-	-	[1, 3, 5, 10]

interp_yield

PLEASE ADD A FUNCTION DESCRIPTION

```
interp_yield(t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.15 yield_curve

Class: BondYieldCurve

Class to do fitting of the yield curve and to enable interpolation of yields. Because yields assume a flat term structure for each bond, this class does not allow discounting to be done and so does not inherit from DiscountCurve. It should only be used for visualisation and simple interpolation but not for full term-structure-consistent pricing.

BondYieldCurve

Fit the curve to a set of bond yields using the type of curve specified. Bounds can be provided if you wish to enforce lower and upper limits on the respective model parameters.

```
BondYieldCurve(settle_dt: Date,
bonds: list,
ylds: Union[np.ndarray, list],
curve_fit,
    """Fit the curve to a set of bond yields using the type of curve
specified. Bounds can be provided if you wish to enforce lower and
upper limits on the respective model parameters."""
    ._settle_dt = settle_dt
    ._bonds = bonds
    ._ylds = np.array(ylds)
    ._curve_fit = curve_fit
    fit_type = type(._curve_fit)
    fit = ._curve_fit
    years_to_maturities = []
    for bond in bonds:
        years_to_maturity = (bond._maturity_dt - settle_dt) / G_DAYS_IN_YEAR
        years_to_maturities.append(years_to_maturity)
    ._years_to_maturity = np.array(years_to_maturities)
    if fit_type is CurveFitPolynomial:
        d = fit._power
        coeffs = np.polyfit(._years_to_maturity, ._ylds, deg=d)
        fit._coeffs = coeffs
    elif fit_type is CurveFitNelsonSiegel:
        xdata = ._years_to_maturity
        ydata = ._ylds
        popt, pcov = curve_fit(._curve_fit._interpolated_yield,
                                xdata,
                                ydata,
                                bounds=fit._bounds,
                                )
        fit._beta1 = popt[0]
        fit._beta2 = popt[1]
        fit._beta3 = popt[2]
        fit._tau = popt[3]
    elif fit_type is CurveFitNelsonSiegelSvensson:
        xdata = ._years_to_maturity
        ydata = ._ylds
        popt, pcov = curve_fit(._curve_fit._interpolated_yield,
                                xdata,
                                ydata,
```

```

        bounds=fit._bounds,
    )
    fit._beta1 = popt[0]
    fit._beta2 = popt[1]
    fit._beta3 = popt[2]
    fit._beta4 = popt[3]
    fit._tau1 = popt[4]
    fit._tau2 = popt[5]
elif fit_type is CurveFitBSpline:
    xdata = ._years_to_maturity
    ydata = ._ylds
    """_Cubic_splines_as_k=3_"""
    spline = splrep(xdata, ydata, k=fit._power, t=fit._knots)
    fit._spline = spline
else:
    raise FinError("Unrecognised_curve_fit_type.")
#####
perty

```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
bonds	list	-
ylds	np.ndarray or list	-
curve_fit	-	-
"""Fit the curve to a set of bond yields using the type of curve specified. Bounds can be provided if you wish to enforce lower and upper limits on the respective model parameters."""	-	-
..settle_dt	-	-
..bonds	-	-
..ylds	-	-
..curve_fit	-	-
fit_type	-	-
fit	-	-
years_to_maturities	-	-
for bond in bonds		-
years_to_maturity	-	(bond._maturity_c
years_to_maturities.append(years_to_maturity)	-	-
..years_to_maturity	-	np.arr
if fit_type is CurveFitPolynomial		-
d	-	-
coeffs	-	np.polyfit(..y
fit._coeffs	-	-
elif fit_type is CurveFitNelsonSiegel		-
xdata	-	..
ydata	-	-
popt	-	-
xdata	-	-
ydata	-	-
bounds	-	-
)	-	-
fit._beta1	-	-
fit._beta2	-	-
fit._beta3	-	-
fit._tau	-	-
elif fit_type is CurveFitNelsonSiegelSvensson		-
xdata	-	..
ydata	-	-
popt	-	-
xdata	-	-
ydata	-	-
bounds	-	-
)	-	-
fit._beta1	-	-
fit._beta2	-	-
fit._beta3	-	-
fit._beta4	-	-
fit._tau1	-	-
fit._tau2	-	-
elif fit_type is CurveFitBSpline		-
xdata	-	..
ydata	-	-

curve_fit

accessor function for curve_fit

```
curve_fit():
```

The function arguments are described in the following table.

interpolated_yield

Interpolates the yield for a given maturity date.

```
interpolated_yield(maturity_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
maturity_dt	Date	-	-

plot

Display yield curve.

```
plot(title, ylabel="Yield_To_Maturity_(%)"):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
title	-	-	-
ylabel	-	-	"Yield To Maturity (%)"

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

6.16 `__init__`

Chapter 7

financepy.products.rates

Funding

This folder contains a set of funding-related products. These reflect contracts linked to funding indices such as Ibors and Overnight index rate swaps (OIS). It includes:

IborDeposit

This is the basic Ibor instrument in which a party borrows an amount for a specified term and rate unsecured.

FinInterestRateFuture

This is a class to handle interest rate futures contracts. This is an exchange-traded contract to receive or pay Ibor on a specified future date. It can be used to build the Libor term structure.

IborFRA

This is a class to manage Forward Rate Agreements (FRAs) in which one party agrees to lock in a forward Ibor rate.

Swaps

FinFixedIborSwap

This is a contract to exchange fixed rate coupons for floating Ibor rates. This class has functionality to value the swap contract and to calculate its risk.

FinFixedIborSwap - IN PROGRESS

This is a contract to exchange fixed rate coupons for floating Ibor rates. This class has functionality to value the swap contract and to calculate its risk.

IborIborSwap - IN PROGRESS

This is a contract to exchange IBOR rates with different terms, also known as a basis swap. This class has functionality to value the swap contract and to calculate its risk.

FinFixedOISwap - IN PROGRESS

This is an OIS, a contract to exchange fixed rate coupons for the overnight index rate. This class has functionality to value the swap contract and to calculate its risk.

IborOISwap - IN PROGRESS

This is a contract to exchange overnight index rates for Ibor rates. This class has functionality to value the swap contract and to calculate its risk.

Currency Swaps***FinFixedFixedCcySwap - IN PROGRESS***

This is a contract to exchange fixed rate coupons in two different currencies. This class has functionality to value the swap contract and to calculate its risk.

IborIborCcySwap - IN PROGRESS

This is a contract to exchange IBOR coupons in two different currencies. This class has functionality to value the swap contract and to calculate its risk.

FinOIS

This is a contract to exchange the daily compounded Overnight index swap rate for a fixed rate agreed at contract initiation.

FinOISCurve

This is a discount curve that is extracted by bootstrapping a set of OIS rates. The internal representation of the curve are discount factors on each of the OIS dates. Between these dates, discount factors are interpolated according to a specified scheme.

IborSingleCurve

This is a discount curve that is extracted by bootstrapping a set of Ibor deposits, Ibor FRAs and Ibor swap prices. The internal representation of the curve are discount factors on each of the deposit, FRA and swap maturity dates. Between these dates, discount factors are interpolated according to a specified scheme - see below.

Options

IborCapFloor

IborSwaption

This is a contract to buy or sell an option to enter into a swap to either pay or receive a fixed swap rate at a specific future expiry date. The model includes code that prices a payer or receiver swaption with the following models:

- Black's Model - Shifted Black Model - SABR - Shifted SABR - Hull-White Tree Model - Black-Karasinski Tree Model - Black-Derman-Toy Tree Model

IborBermudanSwaption

This is a contract to buy or sell an option to enter into a swap to either pay or receive a fixed swap rate at a specific future expiry date on specific coupon dates starting on a designated expiry date. The model includes code that prices a payer or receiver swaption with the following models:

- Hull-White Tree Model - Black-Karasinski Tree Model - Black-Derman-Toy Tree Model

It is also possible to price this using a Ibor Market Model. However for the moment this must be done directly via the Monte-Carlo implementation of the LMM found in `FinModelRatesLMM`.

7.1 callable_swap

7.2 dual_curve

Class: IborDualCurve(DiscountCurve)

Constructs an index curve as implied by the prices of Ibor deposits, FRAs and IRS. Discounting is assumed to be at a discount rate that is an input and usually derived from OIS rates.

IborDualCurve

Create an instance of a Ibor curve given a valuation date and a set of ibor deposits, ibor FRAs and ibor swaps. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on cpn dates and to then assume flat forwards between these cpn dates. The curve will assign a discount factor of 1.0 to the valuation date.

```
IborDualCurve(value_dt: Date,
discount_curve: DiscountCurve,
ibor_deposits: list,
ibor_fras: list,
ibor_swaps: list,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
check_refit_flag: bool = False,
# Set to True to test it works
"""Create an instance of a Ibor curve given a valuation date and
a set of ibor deposits, ibor FRAs and ibor swaps. Some of these may
be left None and the algorithm will just use what is provided. An
interpolation method has also to be provided. The default is to use a
linear interpolation for swap rates on cpn dates and to then assume
flat forwards between these cpn dates.
The curve will assign a discount factor of 1.0 to the valuation date.
"""
check_argument_types(getattr(_func_name(), None), locals())
.value_dt = value_dt
.discount_curve = discount_curve
._validate_inputs(ibor_deposits, ibor_fras, ibor_swaps)
._interp_type = interp_type
.check_refit_flag = check_refit_flag
.build_curve()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
ibor_deposits	list	-	-
ibor_fras	list	-	-
ibor_swaps	list	-	-
interp_type	InterpTypes	-	InterpTypes.FLAT
check_refit_flag	bool	-	False
"""Create an instance of a Ibor curve given a valuation date and a set of ibor deposits ibor FRAs and ibor_swaps. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on cpn dates and to then assume flat forwards between these cpn dates. The curve will assign a discount factor of 1.0 to the valuation date. """			
check_argument_types(getattr(_func_name(), None), locals())	-	-	-
.value_dt	-	-	value_dt
.discount_curve	-	-	discount_curve
._validate_inputs(ibor_deposits, ibor_fras, ibor_swaps)	-	-	-
.interp_type	-	-	interp_type
.check_refit_flag	-	-	check_refit_flag
.build_curve()	-	-	-

build_curve

Build curve based on interpolation.

```
build_curve():
```

The function arguments are described in the following table.

_validate_inputs

Validate the inputs for each of the Ibor products.

```
_validate_inputs(ibor_deposits, ibor_fras, ibor_swaps):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ibor_deposits	-	-	-
ibor_fras	-	-	-
ibor_swaps	-	-	-

`_build_curve_using_1d_solver`

Construct the discount curve using a bootstrap approach. This is the non-linear slower method that allows the user to choose a number of interpolation approaches between the swap rates and other rates. It involves the use of a solver.

```
_build_curve_using_1d_solver():
```

The function arguments are described in the following table.

`check_refit`

Ensure that the Ibor curve refits the calibration instruments.

```
check_refit(depo_tol, fra_tol, swap_tol):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
depo_tol	-	-	-
fra_tol	-	-	-
swap_tol	-	-	-

`__repr__`

Print out the details of the Ibor curve.

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

`f`

Root search objective function for swaps

```
_f(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

-g

Root search objective function for swaps

```
_g(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

7.3 ibor_basis_swap

Class: IborBasisSwap

Class for managing an Ibor-Ibor basis swap contract. This is a contract in which a floating leg with one LIBOR tenor is exchanged for a floating leg payment in a different LIBOR tenor. There is no exchange of par. The contract is entered into at zero initial cost. The contract lasts from an effective date to a specified maturity date.

The value of the contract is the NPV of the two coupon streams. Discounting is done on a supplied discount curve which can be different from the two index discount from which the implied index rates are extracted.

IborBasisSwap

PLEASE ADD A FUNCTION DESCRIPTION

```
IborBasisSwap(effective_dt: Date, # Date interest starts to accrue
term_dt_or_tenor: Union[Date, str], # Date contract ends
leg_1_type: SwapTypes,
leg_1_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
leg_1_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
leg_1_spread: float = 0.0,
leg_2_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
leg_2_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
leg_2_spread: float = 0.0,
notional: float = ONE_MILLION,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create a Ibor basis swap contract giving the contract start
date, its maturity, frequency and day counts on the two floating
legs and notional. The floating leg parameters have default
values that can be overwritten if needed. The start date is contractual
and is the same as the settlement date for a new swap. It is the date
on which interest starts to accrue. The end of the contract is the
termination date. This is not adjusted for business days. The adjusted
termination date is called the maturity date. This is calculated."""
check_argument_types(.IborBasisSwap, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts to accrue
term_dt_or_tenor	Date or str	Date contract ends
leg_1_type	SwapTypes	-
leg_1_freq_type	FrequencyTypes	-
leg_1_dc_type	DayCountTypes	-
leg_1_spread	float	-
leg_2_freq_type	FrequencyTypes	-
leg_2_dc_type	DayCountTypes	-
leg_2_spread	float	-
notional	float	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create a Ibor basis swap contract giving the contract start		
date	-	-
its maturity	-	-
frequency and day counts on the two floating	-	-
legs and notional. The floating leg parameters have default	-	-
values that can be overwritten if needed. The start date is contractual	-	-
and is the same as the settlement date for a new swap. It is the date	-	-
on which interest starts to accrue. The end of the contract is the	-	-
termination date. This is not adjusted for business days. The adjusted	-	-
termination date is called the maturity date. This is calculated."""	-	-
check_argument_types(IborBasisSwap, locals())	-	-

value

Value the interest rate swap on a value date given a single Ibor discount curve and an index curve for the Ibors on each swap leg.

```

value(value_dt: Date,
discount_curve: DiscountCurve,
index_curve_leg_1: DiscountCurve = None,
index_curve_leg_2: DiscountCurve = None,
first_fixing_rate_leg_1=None,
first_fixing_rate_leg_2=None,
"""Value the interest rate swap on a value date given a single Ibor
discount curve and an index curve for the Ibors on each swap leg."""
if index_curve_leg_1 is None:
    index_curve_leg_1 = discount_curve
if index_curve_leg_2 is None:
    index_curve_leg_2 = discount_curve
float_leg_1_value = .float_leg_1.value(value_dt,
    discount_curve,
    index_curve_leg_1,
    first_fixing_rate_leg_1,
)
float_leg_2_value = .float_leg_2.value(value_dt,

```

```

    discount_curve,
    index_curve_leg_2,
    first_fixing_rate_leg_2,
)
value = float_leg_1_value + float_leg_2_value
return value
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve_leg_1	DiscountCurve	-	No
index_curve_leg_2	DiscountCurve	-	No
first_fixing_rate_leg_1	-	-	No
first_fixing_rate_leg_2	-	-	No
"""Value the interest rate swap on a value date given a single Ibor discount curve and an index curve for the Ibors on each swap leg."""	-	-	-
if index_curve_leg_1 is None	-	-	-
index_curve_leg_1	-	-	discount
if index_curve_leg_2 is None	-	-	-
index_curve_leg_2	-	-	discount
discount_curve	-	-	-
index_curve_leg_1	-	-	-
first_fixing_rate_leg_1	-	-	-
)	-	-	-
discount_curve	-	-	-
index_curve_leg_2	-	-	-
first_fixing_rate_leg_2	-	-	-
)	-	-	-
value	-	-	float_leg_1_value +
return value	-	-	-

print_float_leg_1_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_float_leg_1_pv():
```

The function arguments are described in the following table.

print_float_leg_2_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed

leg flows.

```
print_float_leg_2_pv():
```

The function arguments are described in the following table.

print_payments

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_payments():
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print():
```

The function arguments are described in the following table.

7.4 ibor_benchmarks_report

benchmarks_report

Generate a DataFrame with one row per benchmark. A benchmark is any object that has a function `valuation_details(...)` that returns a dictionary of the right shape. Allowed benchmarks at the moment are `depos`, `fras` and `swaps`. Various useful information is reported. This is a bit slow so do not use in performance-critical spots

```

benchmarks_report(
    benchmarks,
    valuation_date: Date,
    discount_curve: DiscountCurve,
    index_curve: DiscountCurve = None,
    include_objects=False,
    """
    Generate a DataFrame with one row per benchmark. A benchmark is any object that
    has a function valuation_details(...) that returns a dictionary of the right shape.
    Allowed benchmarks at the moment are depos, fras and swaps. Various useful
    information is reported. This is a bit slow so do not use in performance-critical
    spots
    """
    # benchmarks = depos + fras + swaps
    df_bmi = None
    for benchmark in benchmarks:
        res = benchmark.valuation_details(valuation_date, discount_curve, index_curve)
        if df_bmi is None:
            df_bmi = pd.DataFrame.from_dict(res, orient="index").T
        else:
            df_bmi = pd.concat([df_bmi, pd.DataFrame([res])], ignore_index=True)
    if include_objects:
        df_bmi["benchmark_objects"] = benchmarks
    return df_bmi

```

The function arguments are described in the following table.

Argument Name	Type	Description
benchmarks	-	-
valuation_date	Date	-
discount_curve	DiscountCurve	-
index_curve	DiscountCurve	-
include_objects	-	-
"""	-	-
Generate a DataFrame with one row per benchmark. A benchmark is any object that has a function valuation_details(...) that returns a dictionary of the right shape.	-	-
Allowed benchmarks at the moment are depositions and swaps. Various useful information is reported. This is a bit slow so do not use in performance-critical spots	-	-
"""	-	-
df_bmi	-	-
for benchmark in benchmarks	-	-
res	-	-
if df_bmi is None	-	-
df_bmi	-	-
else	-	-
df_bmi	-	-
if include_objects	-	-
df_bmi["benchmark_objects"]	-	-
return df_bmi	-	-

ibor_benchmarks_report

Generate a DataFrame with one row per benchmark used in constructing a given ibor_curve. Various useful information is reported. This is a bit slow so do not use in performance-critical spots

```
ibor_benchmarks_report(ibor_curve: IborSingleCurve, include_objects=False):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ibor_curve	IborSingleCurve	-	-
include_objects	-	-	False

_date_or_tenor_to_date

PLEASE ADD A FUNCTION DESCRIPTION

```
_date_or_tenor_to_date(date_or_tenor: Union[Date, str, datetime], asof_date: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
date_or_tenor	Date or str,datetime	-	-
asof_date	Date	-	-

_date_or_tenor_to_date_or_tenor*PLEASE ADD A FUNCTION DESCRIPTION*

```
_date_or_tenor_to_date_or_tenor(date_or_tenor: Union[Date, str, datetime]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
date_or_tenor	Date or str,datetime	-	-

_deposit_from_df_row*PLEASE ADD A FUNCTION DESCRIPTION*

```
_deposit_from_df_row(row: pd.Series, asof_date: Date, calendar_type: CalendarTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
row	pd.Series	-	-
asof_date	Date	-	-
calendar_type	CalendarTypes	-	-

_fra_from_df_row*PLEASE ADD A FUNCTION DESCRIPTION*

```
_fra_from_df_row(row: pd.Series, asof_date: Date, calendar_type: CalendarTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
row	pd.Series	-	-
asof_date	Date	-	-
calendar_type	CalendarTypes	-	-

_swap_from_df_row*PLEASE ADD A FUNCTION DESCRIPTION*

```
_swap_from_df_row(row: pd.Series, asof_date: Date, calendar_type: CalendarTypes):
```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
row	pd.Series	-	-
asof_date	Date	-	-
calendar_type	CalendarTypes	-	-

_unknown_from_df_row

PLEASE ADD A FUNCTION DESCRIPTION

```
_unknown_from_df_row(row: pd.Series):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
row	pd.Series	-	-

dataframe_to_benchmarks

PLEASE ADD A FUNCTION DESCRIPTION

```
dataframe_to_benchmarks(
df: pd.DataFrame, asof_date: Date, calendar_type: CalendarTypes
"""Create IborBenchmarks from a dataframe. The dataframe should have at least these
columns
with these sample inputs:
type start_dt maturity_dt dc_type notional contract_rate
fixed_leg_type fixed_freq_type
0 IborDeposit 06-OCT-2001 09-OCT-2001 ACT_360 100.0
0.042 NaN NaN
1 IborFRA 09-JAN-2002 09-APR-2002 ACT_360 100.0
0.042 PAY NaN
2 IborSwap 09-OCT-2001 09-OCT-2002 THIRTY_E_360 ISDA 1000000
0.042 PAY SEMI_ANNUAL
start_dt_and_maturity_dt could be Date, string convertible to Tenor, or datetime
Args:
df (pd.DataFrame): dataframe as above, with benchmark info
```

The function arguments are described in the following table.

Argument Name	Type
df	pd.DataFrame
asof_date	Date
calendar_type	CalendarTypes
"""Create IborBenchmarks from a dataframe. The dataframe should have at least these columns with these sample inputs	-
type start_dt maturity_dt dc_type notional contract_rate fixed_leg_type fixed_freq_type	-
0 IborDeposit 06-OCT-2001 09-OCT-2001 ACT_360 100.0 0.042 NaN NaN	-
1 IborFRA 09-JAN-2002 09-APR-2002 ACT_360 100.0 0.042 PAY NaN	-
2 IborSwap 09-OCT-2001 09-OCT-2002 THIRTY_E_360 ISDA 1000000 0.042 PAY SEMI_ANNUAL	-
start_dt and maturity_dt could be Date	-
string convertible to Tenor	-
or datetime	-
Args	

7.5 ibor_bermudan_swaption

Class: *IborBermudanSwaption*

This is the class for the Bermudan-style swaption, an option to enter into a swap (payer or receiver of the fixed coupon), that starts in the future and with a fixed maturity, at a swap rate fixed today. This swaption can be exercised on any of the fixed coupon payment dates after the first exercise date.

IborBermudanSwaption

Create a Bermudan swaption contract. This is an option to enter into a payer or receiver swap at a fixed coupon on all the fixed # leg coupon dates until the exercise date inclusive.

```
IborBermudanSwaption(settle_dt: Date,
exercise_dt: Date,
maturity_dt: Date,
fixed_leg_type: SwapTypes,
exercise_type: ExerciseTypes,
fixed_cpn: float,
fixed_freq_type: FrequencyTypes,
fixed_dc_type: DayCountTypes,
notional=ONE_MILLION,
float_freq_type=FrequencyTypes.QUARTERLY,
float_dc_type=DayCountTypes.THIRTY_E_360,
cal_type=CalendarTypes.WEEKEND,
bd_type=BusDayAdjustTypes.FOLLOWING,
dg_type=DateGenRuleTypes.BACKWARD,
"""Create a Bermudan swaption contract. This is an option to enter
into a payer or receiver swap at a fixed coupon on all the fixed
# leg coupon dates until the exercise date inclusive."""
check_argument_types(IborBermudanSwaption, locals())
if settle_dt > exercise_dt:
    raise FinError("Settlement_date_must_be_before_expiry_date")
if exercise_dt > maturity_dt:
    raise FinError("Exercise_date_must_be_before_swap_maturity_date")
if exercise_type == ExerciseTypes.AMERICAN:
    raise FinError("American_optionality_not_supported.")
.settle_dt = settle_dt
.exercise_dt = exercise_dt
.maturity_dt = maturity_dt
.fixed_leg_type = fixed_leg_type
.exercise_type = exercise_type
.fixed_cpn = fixed_cpn
.fixed_freq_type = fixed_freq_type
.fixed_dc_type = fixed_dc_type
.notional = notional
.float_freq_type = float_freq_type
.float_dc_type = float_dc_type
.cal_type = cal_type
.bd_type = bd_type
.dg_type = dg_type
.pv01 = None
.fwd_swap_rate = None
.forward_df = None
.underlying_swap = None
```

```
.cpn_times = None
.cpn_flows = None
.call_times = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
settle_dt	Date	-	-
exercise_dt	Date	-	-
maturity_dt	Date	-	-
fixed_leg_type	SwapTypes	-	-
exercise_type	ExerciseTypes	-	-
fixed_cpn	float	-	-
fixed_freq_type	FrequencyTypes	-	-
fixed_dc_type	DayCountTypes	-	-
notional	-	-	ONE_MILLION
float_freq_type	-	-	FrequencyTypes
float_dc_type	-	-	DayCountTypes
cal_type	-	-	CalendarTypes
bd_type	-	-	BusDayAdjustTypes
dg_type	-	-	DateGenRuleTypes
"""Create a Bermudan swaption contract. This is an option to enter		-	-
into a payer or receiver swap at a fixed coupon on all the fixed		-	-
check_argument_types(IborBermudanSwaption, locals())		-	-
if settle_dt > exercise_dt		-	-
raise FinError("Settlement date must be before expiry date")		-	-
if exercise_dt > maturity_dt		-	-
raise FinError("Exercise date must be before swap maturity date")		-	-
if exercise_type		-	= ExerciseTypes
raise FinError("American optionality not supported.")		-	-
.settle_dt	-	-	settle_dt
.exercise_dt	-	-	exercise_dt
.maturity_dt	-	-	maturity_dt
.fixed_leg_type	-	-	fixed_leg_type
.exercise_type	-	-	exercise_type
.fixed_cpn	-	-	fixed_cpn
.fixed_freq_type	-	-	fixed_freq_type
.fixed_dc_type	-	-	fixed_dc_type
.notional	-	-	notional
.float_freq_type	-	-	float_freq_type
.float_dc_type	-	-	float_dc_type
.cal_type	-	-	cal_type
.bd_type	-	-	bd_type
.dg_type	-	-	dg_type
.pv01	-	-	No
.fwd_swap_rate	-	-	No
.forward_df	-	-	No
.underlying_swap	-	-	No
.cpn_times	-	-	No
.cpn_flows	-	-	No
.call_times	-	-	No

value

Value the Bermudan swaption using the specified model and a discount curve. The choices of model are the Hull-White model, the Black-Karasinski model and the Black-Derman-Toy model.

```
value(value_dt, discount_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
discount_curve	-	-	-
model	-	-	-

print_swaption_value

Print the swaption value and the underlying swap details.

```
print_swaption_value():
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

PLEASE ADD A FUNCTION DESCRIPTION

```
_print():
```

The function arguments are described in the following table.

7.6 ibor_cap_floor

Enumerated Type: IborCapFloorModelTypes

This enumerated type has the following values:

- BLACK
- SHIFTED_BLACK
- SABR
- last_fixing:Optional[float]
- freq_type:FrequencyTypes
- dc_type:DayCountTypes
- notional:float
- cal_type:CalendarTypes
- bd_type:BusDayAdjustTypes
- dg_type:DateGenRuleTypes
- self.cal_type
- self.bd_type
- maturity_dt
- maturity_dt
- calendar
- maturity_dt
- self.start_dt
- self.maturity_dt
- self.opt_type
- self.strike_rate
- self.last_fixing
- self.freq_type
- self.dc_type
- self.notional
- self.dg_type
- self.caplet_floorlet_values
- self.caplet_floorlet_alphas
- self.caplet_floorlet_fwd_rates
- self.caplet_floorlet_intrinsic
- self.caplet_floorlet_dfs
- self.cap_floor_pv
- self.caplet_floorlet_dates
- self.value_dt
- self.day_counter
- schedule
- self.caplet_floorlet_dates
- self.value_dt
- self.day_counter
- num_options
- strike_rate
- ifnum_options;

- self.caplet_floorlet_values
- self.caplet_floorlet_alphas
- self.caplet_floorlet_fwd_rates
- self.caplet_floorlet_intrinsic
- self.caplet_floorlet_dfs
- self.cap_floor_pv
- cap_floor_value
- caplet_floorlet_value
- start_dt
- end_dt
- fwd_rate
- fwd_rate
- alpha
- df
- ifself.opt_type
- caplet_floorlet_value
- elifself.opt_type
- caplet_floorlet_value
- caplet_floorlet_value*
- cap_floor_value+
- start_dt
- end_dt
- alpha
- df
- fwd_rate
- ifself.opt_type
- intrinsic_value
- elifself.opt_type
- intrinsic_value
- intrinsic_value*
- caplet_floorlet_value
- cap_floor_value+
- t_exp
- alpha
- f
- k
- df
- ifk
- k
- caplet_floorlet_value
- ifself.opt_type
- caplet_floorlet_value
- elifself.opt_type
- caplet_floorlet_value
- ifself.opt_type
- caplet_floorlet_value

- elifself.opt_type
- caplet_floorlet_value
- ifself.opt_type
- caplet_floorlet_value
- elifself.opt_type
- caplet_floorlet_value
- ifself.opt_type
- caplet_floorlet_value
- elifself.opt_type
- caplet_floorlet_value
- ifself.opt_type
- caplet_floorlet_value
- elifself.opt_type
- caplet_floorlet_value
- t_mat
- alpha
- strike_price
- notional_adj
- face_amount
- df_times
- df_values
- v
- ifself.opt_type
- caplet_floorlet_value
- elifself.opt_type
- caplet_floorlet_value
- caplet_floorlet_value*
- iflen(self.caplet_floorlet_values)
- ifself.opt_type
- header
- header+
- elifself.opt_type
- header
- header+
- i_flow
- ifi_flow
- i_flow+
- s
- s+
- s+
- s+
- s+
- s+
- s+

Class: IborCapFloor

Class for Caps and Floors. These are contracts which observe a Ibor reset L on a future start date and then make a payoff at the end of the Ibor period which is $\text{Max}[L-K,0]$ for a cap and $\text{Max}[K-L,0]$ for a floor. This is then day count adjusted for the Ibor period and then scaled by the contract notional to produce a valuation. A number of models can be selected from.

IborCapFloor

PLEASE ADD A FUNCTION DESCRIPTION

```
IborCapFloor(start_dt: Date,
maturity_dt_or_tenor: Union[Date, str],
opt_type: CapFloorTypes,
strike_rate: float,
last_fixing: Optional[float] = None,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360_ISDA,
notional: float = ONE_MILLION,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Initialise_IborCapFloor_object."""
check_argument_types(.IborCapFloor, locals())
.cal_type = cal_type
.bd_type = bd_type
if isinstance(maturity_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
start_dt	Date	-	-
maturity_dt_or_tenor	Date or str	-	-
opt_type	CapFloorTypes	-	-
strike_rate	float	-	-
last_fixing	Optional[float]	-	None
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.THIRTY_E_360_I
notional	float	-	ONE.MILLION
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWI
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWAR
"""Initialise IborCapFloor object."""	-	-	-
check_argument_types(.IborCapFloor, locals())	-	-	-
.cal_type	-	-	cal_type
.bd_type	-	-	bd_type

_generate_dts*PLEASE ADD A FUNCTION DESCRIPTION*

```
_generate_dts():
```

The function arguments are described in the following table.

value

Value the cap or floor using the chosen model which specifies the volatility of the Ibor rate to the cap start date.

```
value(value_dt, libor_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
libor_curve	-	-	-
model	-	-	-

value_caplet_floor_let*PLEASE ADD A FUNCTION DESCRIPTION*

```
value_caplet_floor_let(value_dt, caplet_start_dt, caplet_end_dt, libor_curve, model
"""Value the caplet or floorlet using a specific model."""
t_exp = (caplet_start_dt - .start_dt) / G_DAYS_IN_YEAR
alpha = .day_counter.year_frac(caplet_start_dt, caplet_end_dt)[0]
f = libor_curve.fwd_rate(caplet_start_dt, caplet_end_dt, .dc_type)
k = .strike_rate
df = libor_curve.df(caplet_end_dt)
if k == 0.0:
    k = 1e-10
caplet_floorlet_value = 0.0
if isinstance(model, Black):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
caplet_start_dt	-	-	-
caplet_end_dt	-	-	-
libor_curve	-	-	-
model	-	-	-
"""Value the caplet or floorlet using a specific model."""	-	-	-
t_exp	-	(caplet_start_dt - .start_dt) / G_DA	
alpha	-	.day_counter.year_frac(caplet_start_dt,	
f	-	libor_curve.fwd_rate(caplet_start_dt, cap	
k	-	.strike_rate	
df	-	libor_curve.df(caplet_en	
if k	-	= 0.0:	
k	-	1e-10	
caplet_floorlet_value	-	-	0.0

print_leg

Prints the cap floor payment amounts.

```
print_leg():
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

PLEASE ADD A FUNCTION DESCRIPTION

```
_print():
```

The function arguments are described in the following table.

7.7 ibor_conventions

Class: IborConventions

IborConventions

PLEASE ADD A FUNCTION DESCRIPTION

```
IborConventions(currency_name: str, index_name: str = "LIBOR"):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
currency_name	str	-	-
index_name	str	-	"LIBOR"

7.8 ibor_curve_risk_engine

par_rate_risk_report

PLEASE ADD A FUNCTION DESCRIPTION

```
par_rate_risk_report(
base_curve: IborSingleCurve,
trades: list,
trade_labels: list = None,
bump_size=1.0 * G_BASIS_POINT,
"""Calculate deltas (change in value to 1bp bump) of the trades to all
benchmarks in the base curve. Supported trades are depos, fras, swaps.
trade_labels are used to identify trades in the output, if not provided
simple ones are generated
Args:
base_curve (IborSingleCurve): Base curve to be bumped
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
base_curve	IborSingleCurve	-	-
trades	list	-	-
trade_labels	list	-	None
bump_size	-	-	1.0 * G_BASIS_POINT
"""Calculate deltas (change in value to 1bp bump) of the trades to all	-	-	-
benchmarks in the base curve. Supported trades are depos	-	-	-
fras	-	-	-
swaps.	-	-	-
trade_labels are used to identify trades in the output	-	-	-
if not provided	-	-	-
simple ones are generated	-	-	-
Args	-	-	-

forward_rate_risk_report

PLEASE ADD A FUNCTION DESCRIPTION

```
forward_rate_risk_report(
base_curve: DiscountCurve,
grid_last_date: Date,
grid_bucket_tenor: str,
trades: list,
trade_labels: list = None,
bump_size=1.0 * G_BASIS_POINT,
"""Generate forward rate deltas (forward delta ladder) risk report, which
is the sensitivity of trades to bucketed shocks of the instantaneous (ON)
forward rates. Here shock i is applied to the ON forward rates over
[t_i, t_{i+1}] where t_i is a time grid from base_curve.valuation_date
to grid_last_date with buckets of size grid_bucket_tenor
```

```

Args:
    _base_curve_(DiscountCurve): _base_curve_to_apply_bumps_to

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
base_curve	DiscountCurve	-	-
grid_last_date	Date	-	-
grid_bucket_tenor	str	-	-
trades	list	-	-
trade_labels	list	-	None
bump_size	-	-	1.0 * G_BASIS
"""Generate forward rate deltas (forward delta ladder) risk report which	-	-	-
is the sensitivity of trades to bucketed shocks of the instantaneous (ON)	-	-	-
forward rates. Here shock_i is applied to the ON forward rates over	-	-	-
[t_i, t_i+1] where t_i is a time grid from base_curve.valuation_date	-	-	-
to grid.last_date with buckets of size grid_bucket_tenor	-	-	-
Args		-	-

forward_rate_risk_report_custom_grid

PLEASE ADD A FUNCTION DESCRIPTION

```

forward_rate_risk_report_custom_grid(
base_curve: DiscountCurve,
grid: List[Date],
trades: list,
grid_labels: list = None,
trade_labels: list = None,
bump_size=1.0 * G_BASIS_POINT,
"""Generate forward rate deltas risk report, which is the sensitivity of
trades to bucketed shocks of the instantaneous (ON) forward rates. Here
shock_i is applied to the ON forward rates
over [t_i, t_{i+1}] where {t_i} is the 'grid' argument
Args:
    _base_curve_(DiscountCurve): _base_curve_to_apply_bumps_to

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
base_curve	DiscountCurve	-	-
grid	List[Date]	-	-
trades	list	-	-
grid_labels	list	-	None
trade_labels	list	-	None
bump_size	-	-	1.0 * G.BASIS_POINT
"""Generate forward rate deltas risk report which is the sensitivity of trades to bucketed shocks of the instantaneous (ON) forward rates. Here shock_i is applied to the ON forward rates over [t_i, t_i+1] where t_i is the 'grid' argument	-	-	-
Args	-	-	-

carry_rolldown_report

PLEASE ADD A FUNCTION DESCRIPTION

```

carry_rolldown_report(
    base_curve: DiscountCurve,
    grid_last_date: Date,
    grid_bucket_tenor: Union[str, Tenor],
    trades: list,
    trade_labels: list = None,
    bump_size=1.0 * G.BASIS_POINT,
    """Generate carry and rolldown risk report based on the sensitivity of
    trades to bucketed shocks of the instantaneous (ON) forward rates. Here
    shock_i is applied to the ON forward rates over [t_i, t_{i+1}] where t_i
    is a time grid from base_curve.valuation_date to grid_last_date with
    buckets of size grid_bucket_tenor
    Args:
    base_curve (DiscountCurve): base_curve to apply bumps to

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
base_curve	DiscountCurve	-	-
grid_last_date	Date	-	-
grid_bucket_tenor	str or Tenor	-	-
trades	list	-	-
trade_labels	list	-	None
bump_size	-	-	1.0 * G.BASIS_POINT
"""Generate carry and rolldown risk report based on the sensitivity of trades to bucketed shocks of the instantaneous (ON) forward rates. Here shock_i is applied to the ON forward rates over [t_i, t_i+1] where t_i is a time grid from base_curve.valuation_date to grid_last_date with buckets of size grid_bucket_tenor	-	-	-
Args	-	-	-

parallel_shift_ladder_report

PLEASE ADD A FUNCTION DESCRIPTION

```
parallel_shift_ladder_report(
    base_curve: DiscountCurve,
    curve_shifts: np.ndarray,
    trades: list,
    trade_labels: list = None,
    curve_shift_labels = [f"SHIFT:{s/G_BASIS_POINT:.1f}" for s in curve_shifts]
    risk_report = pd.DataFrame(
        columns=["shift_label", "shift_bp"],
        data=zip(
            curve_shift_labels,
            curve_shifts / G_BASIS_POINT,
        ),
    )
    n_trades = len(trades)
    if trade_labels is None:
        trade_labels = [f"trade_{n:03d}" for n in range(n_trades)]
    base_values = {}
    for trade, trade_label in zip(trades, trade_labels):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
base_curve	DiscountCurve	-	-
curve_shifts	np.ndarray	-	-
trades	list	-	-
trade_labels	list	-	None
curve_shift_labels	-	-	[f"SHIFT:s/G_BASIS_POINT:.1f" for s in curve_shifts]
columns	-	-	["shift_label", "shift_bp"]
curve_shift_labels	-	-	-
curve_shifts / G_BASIS_POINT	-	-	-
)	-	-	-
)	-	-	-
n_trades	-	-	len(trades)
if trade_labels is None	-	-	-
trade_labels	-	-	[f"trade_n:03d" for n in range(n_trades)]
base_values	-	-	-
for trade	-	-	-

_grid_from_dates_tenor

PLEASE ADD A FUNCTION DESCRIPTION

```
_grid_from_dates_tenor(
    grid_last_date, grid_bucket_tenor: Union[str, Tenor], valuation_date
    bucket_tenor = Tenor.as_tenor(grid_bucket_tenor)
```

```

grid = [valuation_date]
grid_labels = []
# TODO: hardcoded stuff, revisit
spot_days = 2
d = grid[0].add_weekdays(spot_days)
# the loop is structured so the grid_last_date is captured by last bucket
count = 0
while d < grid_last_date:
    d = d.add_tenor(bucket_tenor)
    grid.append(d)
    grid_labels.append(f"{count}*bucket_tenor"x{bucket_tenor}")
    count += 1
grid[-1] = grid_last_date
return grid, grid_labels

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
grid_last_date	-	-	-
grid_bucket_tenor	str or Tenor	-	-
valuation_date	-	-	-
bucket_tenor	-	-	Tenor.as_tenor(grid_bucket_tenor)
grid	-	-	[valuation_date]
grid_labels	-	-	[]
spot_days	-	-	2
d	-	-	grid[0].add_weekdays(spot_days)
count	-	-	0
while d < grid_last_date	-	-	-
d	-	-	d.add_tenor(bucket_tenor)
grid.append(d)	-	-	-
grid_labels.append(f'count *bucket_tenorxbucket_tenor')	-	-	-
count +	-	-	1
grid[-1]	-	-	grid_last_date
return grid	-	-	-
grid_labels	-	-	-

7.9 ibor_deposit

Class: IborDeposit

An Ibor deposit is an agreement to borrow money interbank at the Ibor fixing rate starting on the start date and repaid on the maturity date with the interest amount calculated according to a day count convention and dates calculated according to a calendar and business day adjustment rule.

Care must be taken to calculate the correct start (settlement) date. Start with the trade (value) date which is typically today, we may need to add on a number of business days (spot days) to get to the settlement date. The maturity date is then calculated by adding on the deposit tenor/term to the settlement date and adjusting for weekends and holidays according to the calendar and adjustment type.

Note that for over-night (ON) deposite the settlement date is today with maturity in one business day. For tomorrow-next (TN) deposite the settlement is in one business day with maturity on the following business day. For later maturity deposits, settlement is usually in 1-3 business days. The number of days depends on the currency and jurisdiction of the deposit contract.

IborDeposit

PLEASE ADD A FUNCTION DESCRIPTION

```
IborDeposit(start_dt: Date, # When the interest starts to accrue
maturity_dt_or_tenor: Union[Date, str], # Repayment of interest
deposit_rate: float, # MM rate using simple interest
dc_type: DayCountTypes, # How year fraction is calculated
notional: float = 100.0, # Amount borrowed
cal_type: CalendarTypes = CalendarTypes.WEEKEND, # Maturity date
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.MODIFIED_FOLLOWING,
"""Create a Libor deposit object which takes the start date when
the amount of notional is borrowed, a maturity date or a tenor and the
deposit rate. If a tenor is used then this is added to the start
date and the calendar and business day adjustment method are applied if
the maturity date fall on a holiday. Note that in order to calculate
the start date you add the spot business days to the trade date
which usually today."""
check_argument_types(IborDeposit, locals())
.cal_type = cal_type
.bd_type = bd_type
if isinstance(maturity_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
start_dt	Date	When the interest starts
maturity_dt_or_tenor	Date or str	Repayment of inter
deposit_rate	float	MM rate using simple
dc_type	DayCountTypes	How year fraction is ca
notional	float	Amount borrowe
cal_type	CalendarTypes	Maturity date
bd_type	BusDayAdjustTypes	-
"""Create a Libor deposit object which takes the start date when		-
the amount of notional is borrowed		-
a maturity date or a tenor and the		-
deposit rate. If a tenor is used then this is added to the start		-
date and the calendar and business day adjustment method are applied if		-
the maturity date fall on a holiday. Note that in order to calculate		-
the start date you add the spot business days to the trade date		-
which usually today."""		-
check_argument_types(IborDeposit, locals())		-
.cal_type		-
.bd_type		-

maturity_df

Returns the maturity date discount factor that would allow the Libor curve to reprice the contractual market deposit rate. Note that this is a forward discount factor that starts on settlement date.

```
maturity_df():
```

The function arguments are described in the following table.

value

Determine the value of an existing Libor Deposit contract given a valuation date and a Libor curve. This is simply the PV of the future repayment plus interest discounted on the current Libor curve.

```
value(value_dt: Date, libor_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
libor_curve	-	-	-

valuation_details

A long-hand method that returns various details relevant to valuation in a dictionary. Slower than value(...) so should not be used when performance is important We want thre output dictionary to have the same labels

for different benchmarks (depos, fras, swaps) because we want to present them together so please do not stick new outputs into one of them only TODO: make a test of this

```

valuation_details(valuation_date: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
"""
A long-hand method that returns various details relevant to valuation
in a dictionary. Slower than value(...) so should not be used when
performance is important
We want three output dictionary to have the same labels for different
benchmarks (depos, fras, swaps) because we want to present them
together so please do not stick new outputs into one of them only
TODO: make a test of this
"""
if valuation_date > .maturity_dt:
    raise FinError("Start date after maturity date")
dc = DayCount(.dc_type)
acc_factor = dc.year_frac(.start_dt, .maturity_dt)[0]
df_settle = discount_curve.df(.start_dt)
df_maturity = discount_curve.df(.maturity_dt)
value = (1.0 + acc_factor * .deposit_rate) * .notional
# Need to take into account spot days being zero so depo settling fwd
value = (value * df_maturity / df_settle
) # VP: ??? this looks like a start_dt - forward value? not spot value? why?
out = {
    "type": type().__name__,
    "start_dt": .start_dt,
    "maturity_dt": .maturity_dt,
    "dc_type": .dc_type.name,
    "notional": .notional,
    "contract_rate": .deposit_rate,
    "market_rate": (df_settle / df_maturity - 1) / acc_factor,
    # for depo pvbp is actually negative: rates up, value down but
    # probably makes sense to report as positive, as if for a spot-starting fra
    "spot_pvbp": acc_factor * df_maturity,
    "fwd_pvbp": acc_factor * df_maturity / df_settle,
    "unit_value": value / .notional,
    "value": value,
    # ignoring bus day adj type, calendar for now
}
return out
#####

```

The function arguments are described in the following table.

Argument Name	Type
valuation_date	Date
discount_curve	DiscountCurve
index_curve	DiscountCurve
"""	-
A long-hand method that returns various details relevant to valuation	-
in a dictionary. Slower than value(...) so should not be used when	-
performance is important	-
We want thre output dictionary to have the same labels for different	-
benchmarks (depos, fras, swaps) because we want to present them	-
together so please do not stick new outputs into one of them only	-
TODO	makeatestofthis
"""	-
if valuation_date < .maturity_dt	
raise FinError("Start date after maturity date")	-
dc	-
acc_factor	-
df_settle	-
df_maturity	-
value	-
)	-
out	-
"type"	).__name_ or type(
"start_dt"	.start_dt
"maturity_dt"	.maturity_dt
"dc_type"	.dc_type.name
"notional"	.notional
"contract_rate"	.deposit_rate
"market_rate"	df_settle/df_maturity-1)/acc_facto or (df_settle/df_m
"spot_pvbp"	acc_factor*df_maturity
"fwd_pvbp"	acc_factor*df_maturity/df_settle
"unit_value"	value/.notional
"value"	value
return out	- - -

print_payments

Print the date and size of the future repayment.

```
print_payments(value_dt: Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-

--repr--

Print the contractual details of the Libor deposit.

```
__repr__():
```

The function arguments are described in the following table.

_print

PLEASE ADD A FUNCTION DESCRIPTION

```
_print():
```

The function arguments are described in the following table.

7.10 ibor_dual_curve

Class: IborDualCurve(DiscountCurve)

Constructs an index curve as implied by the prices of Ibor deposits, FRAs and IRS. Discounting is assumed to be at a discount rate that is an input and usually derived from OIS rates.

IborDualCurve

Create an instance of a Ibor curve given a valuation date and a set of ibor deposits, ibor FRAs and ibor swaps. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on cpn dates and to then assume flat forwards between these cpn dates. The curve will assign a discount factor of 1.0 to the valuation date.

```
IborDualCurve(value_dt: Date,
discount_curve: DiscountCurve,
ibor_deposits: list,
ibor_fras: list,
ibor_swaps: list,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
check_refit_flag: bool = False,
# Set to True to test it works
"""Create an instance of a Ibor curve given a valuation date and
a set of ibor deposits, ibor FRAs and ibor swaps. Some of these may
be left None and the algorithm will just use what is provided. An
interpolation method has also to be provided. The default is to use a
linear interpolation for swap rates on cpn dates and to then assume
flat forwards between these cpn dates.
The curve will assign a discount factor of 1.0 to the valuation date.
"""
check_argument_types(getattr(_func_name(), None), locals())
.value_dt = value_dt
.discount_curve = discount_curve
._validate_inputs(ibor_deposits, ibor_fras, ibor_swaps)
._interp_type = interp_type
.check_refit_flag = check_refit_flag
.build_curve()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
ibor_deposits	list	-	-
ibor_fras	list	-	-
ibor_swaps	list	-	-
interp_type	InterpTypes	-	InterpTypes.FLAT
check_refit_flag	bool	-	False
"""Create an instance of a Ibor curve given a valuation date and a set of ibor deposits ibor FRAs and ibor_swaps. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on cpn dates and to then assume flat forwards between these cpn dates. The curve will assign a discount factor of 1.0 to the valuation date. """			
check_argument_types(getattr(_func_name(), None), locals())	-	-	-
.value_dt	-	-	value_dt
.discount_curve	-	-	discount_curve
._validate_inputs(ibor_deposits, ibor_fras, ibor_swaps)	-	-	-
.interp_type	-	-	interp_type
.check_refit_flag	-	-	check_refit_flag
.build_curve()	-	-	-

build_curve

Build curve based on interpolation.

```
build_curve():
```

The function arguments are described in the following table.

_validate_inputs

Validate the inputs for each of the Ibor products.

```
_validate_inputs(ibor_deposits, ibor_fras, ibor_swaps):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ibor_deposits	-	-	-
ibor_fras	-	-	-
ibor_swaps	-	-	-

`_build_curve_using_1d_solver`

Construct the discount curve using a bootstrap approach. This is the non-linear slower method that allows the user to choose a number of interpolation approaches between the swap rates and other rates. It involves the use of a solver.

```
_build_curve_using_1d_solver():
```

The function arguments are described in the following table.

`check_refit`

Ensure that the Ibor curve refits the calibration instruments.

```
check_refit(depo_tol, fra_tol, swap_tol):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
depo_tol	-	-	-
fra_tol	-	-	-
swap_tol	-	-	-

`__repr__`

Print out the details of the Ibor curve.

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

`f`

Root search objective function for swaps

```
_f(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

-g

Root search objective function for swaps

```
_g(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

7.11 ibor_fixed_float_swap

Class: IborFixedFloatSwap

Class for managing a standard Fixed vs IBOR swap. This is a contract in which a fixed payment leg is exchanged for a series of floating rates payments linked to some IBOR index rate. There is no exchange of principal. The contract is entered into at zero initial cost. The contract lasts from a start date to a specified maturity date.

The floating rate is not known fully until the end of the preceding payment period. It is set in advance and paid in arrears.

The value of the contract is the NPV of the two cpn streams. Discounting is done on a supplied discount curve which is separate from the curve from which the implied index rates are extracted.

IborFixedFloatSwap

PLEASE ADD A FUNCTION DESCRIPTION

```
IborFixedFloatSwap(effective_dt: Date, # Date interest starts to accrue
term_dt_or_tenor: Union[Date, str], # Date contract ends
fixed_leg_type: SwapTypes,
fixed_cpn: float, # Fixed cpn (annualised)
fixed_freq_type: FrequencyTypes,
fixed_dc_type: DayCountTypes,
notional: float = ONE_MILLION,
float_spread: float = 0.0,
float_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
float_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create an interest rate swap contract giving the contract start
date, its maturity, fixed cpn, fixed leg frequency, fixed leg day
count convention and notional. The floating leg parameters have default
values that can be overwritten if needed. The start date is contractual
and is the same as the settlement date for a new swap. It is the date
on which interest starts to accrue. The end of the contract is the
termination date. This is not adjusted for business days. The adjusted
termination date is called the maturity date. This is calculated."""
check_argument_types(.IborFixedFloatSwap, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts to accrue
term_dt_or_tenor	Date or str	Date contract ends
fixed_leg_type	SwapTypes	-
fixed_cpn	float	Fixed cpn (annualised)
fixed_freq_type	FrequencyTypes	-
fixed_dc_type	DayCountTypes	-
notional	float	-
float_spread	float	-
float_freq_type	FrequencyTypes	-
float_dc_type	DayCountTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create an interest rate swap contract giving the contract start date, its maturity, fixed cpn, fixed leg frequency, fixed leg day count convention and notional. The floating leg parameters have default values that can be overwritten if needed. The start date is contractual and is the same as the settlement date for a new swap. It is the date on which interest starts to accrue. The end of the contract is the termination date. This is not adjusted for business days. The adjusted termination date is called the maturity date. This is calculated."""		-
date	-	-
its maturity	-	-
fixed cpn	-	-
fixed leg frequency	-	-
fixed leg day	-	-
count convention and notional. The floating leg parameters have default values that can be overwritten if needed. The start date is contractual and is the same as the settlement date for a new swap. It is the date on which interest starts to accrue. The end of the contract is the termination date. This is not adjusted for business days. The adjusted termination date is called the maturity date. This is calculated."""	-	-
check_argument_types(.IborFixedFloatSwap, locals())	-	-

get_fixed_rate

easy read access to the coupon (fixed rate)

```
get_fixed_rate():
```

The function arguments are described in the following table.

set_fixed_rate

Sometimes we need to reset the coupon (fixed rate) This function updates caches that depend on it

```
set_fixed_rate(new_rate: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
new_rate	float	-	-

set_fixed_rate_to_atm

Reset fixed rate to atm given curve(s). returns the new atm

```

set_fixed_rate_to_atm(valuation_date: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
first_fixing: float = None,
"""
Reset_fixed_rate_to_atm_given_curve(s).returns_the_new_atm
"""
atm = .swap_rate(valuation_date, discount_curve, index_curve, first_fixing)
.set_fixed_rate(atm)
return atm
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	D
valuation_date	Date	-	
discount_curve	DiscountCurve	-	
index_curve	DiscountCurve	-	
first_fixing	float	-	
"""	-	-	
Reset fixed rate to atm given curve(s). returns the new atm	-	-	
"""	-	-	
atm	-	-	.swap_rate(valuation_date, di
.set_fixed_rate(atm)	-	-	
return atm	-	-	

value

Value the interest rate swap on a value date given a single Ibor discount curve.

```

value(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
first_fixing_rate=None,
pv_only=True,
"""Value_the_interest_rate_swap_on_a_value_date_given_a_single_Ibor
discount_curve."""
if index_curve is None:
    index_curve = discount_curve
fixed_leg_results = .fixed_leg.value(value_dt, discount_curve, pv_only=pv_only
)
float_leg_results = .float_leg.value(value_dt,
    discount_curve,
    index_curve,
    first_fixing_rate,
    pv_only=pv_only,
)

```

```

if pv_only:
    value = fixed_leg_results + float_leg_results
    return value
else:
    value = fixed_leg_results[0] + float_leg_results[0]
    cashflow_report = pd.concat([fixed_leg_results[1], float_leg_results[1]],
                                ignore_index=True
    )
    return value, cashflow_report
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve	DiscountCurve	-	None
first_fixing_rate	-	-	None
pv_only	-	-	True
"""Value the interest rate swap on a value date given a single Ibor	-	-	-
discount curve."""	-	-	-
if index_curve is None	-	-	-
index_curve	-	-	discount
)	-	-	-
discount_curve	-	-	-
index_curve	-	-	-
first_fixing_rate	-	-	-
pv_only	-	-	pv_on
)	-	-	-
if pv_only	-	-	-
value	-	-	fixed_leg_results +
return value	-	-	-
else	-	-	-
value	-	-	fixed_leg_results[0] +
)	-	-	-
return value	-	-	-
cashflow_report	-	-	-

valuation_details

A long-hand method that returns various details relevant to valuation in a dictionary Slower than value(...) so should not be used when performance is important We want the output dictionary to have the same labels for different bechmarks (depos, fras, swaps) because we want to present them together so please do not stick new outputs into one of them only

```

valuation_details(valuation_date: Date,
discount_curve: DiscountCurve,

```

```

index_curve: DiscountCurve = None,
first_fixing_rate=None,
"""
A_long-hand_method_that_returns_various_details_relevant_to_valuation_in
a_dictionary
Slower_than_value(...)so_should_not_be_used_when_performance_is_important
We_want_the_output_dictionary_to_have_the_same_labels_for_different_bechmarks
(depos, _fras, _swaps)_because_we_want_to_present_them_together_so_please
do_not_stick_new_outputs_into_one_of_them_only
"""
if index_curve is None:
    index_curve = discount_curve
fixed_leg_value = .fixed_leg.value(valuation_date, discount_curve)
float_leg_value = .float_leg.value(valuation_date, discount_curve, index_curve,
    first_fixing_rate
)
value = fixed_leg_value + float_leg_value
pv01 = np.abs(fixed_leg_value / .fixed_leg.cpn / .fixed_leg.notional)
pay_receive_float = -1 if .float_leg.leg_type == SwapTypes.PAY else 1
swap_rate = float_leg_value / .float_leg.notional / pv01 / pay_receive_float
# VP: There is significant amount of confusion here with swap_type vs notional.
is_payers = (.fixed_leg.leg_type == SwapTypes.PAY and .fixed_leg.notional > 0
) or (.fixed_leg.leg_type == SwapTypes.RECEIVE and .fixed_leg.notional < 0
)
pvbp_sign = 1 if is_payers else -1
out = {
    "type": type().__name__,
    "start_dt": .effective_dt,
    "maturity_dt": .maturity_dt,
    "dc_type": .fixed_leg.dc_type.name,
    "fixed_leg_type": .fixed_leg.leg_type.name,
    "fixed_freq_type": .fixed_leg.freq_type.name,
    "notional": .fixed_leg.notional,
    "contract_rate": .fixed_leg.cpn,
    "market_rate": swap_rate,
    "spot_pvbp": pv01 * pvbp_sign,
    "fwd_pvbp": pv01 * pvbp_sign / discount_curve.df(.effective_dt),
    "unit_value": value / .fixed_leg.notional,
    "value": value,
    # ignoring bus day adj type, calendar, etc for now
}
return out
#####

```

The function arguments are described in the following table.

Argument Name	Type
valuation_date	Date
discount_curve	DiscountCurve
index_curve	DiscountCurve
first_fixing_rate	-
"""	-
A long-hand method that returns various details relevant to valuation in	-
a dictionary	-
Slower than value(...) so should not be used when performance is important	-
We want the output dictionary to have the same labels for different bechmarks	-
(depos, fras, swaps) because we want to present them together so please	-
do not stick new outputs into one of them only	-
"""	-
if index_curve is None	
index_curve	-
fixed_leg_value	-
)	-
value	-
pv01	-
pay_receive_float	-
swap_rate	-
) or (.fixed_leg.leg_type	-
)	-
pvpbp_sign	-
out	-
"type"	).name_ or type(
"start_dt"	.effective_dt
"maturity_dt"	.maturity_dt
"dc_type"	.fixed_leg.dc_type.name
"fixed_leg_type"	.fixed_leg.leg_type.name
"fixed_freq_type"	.fixed_leg.freq_type.name
"notional"	.fixed_leg.notional
"contract_rate"	.fixed_leg.cpn
"market_rate"	swap_rate
"spot_pvpbp"	pv01*pvpbp_sign
"fwd_pvpbp"	.effective_dt or pv01*pvpbp_sign/discount_
"unit_value"	value/.fixed_leg.notional
"value"	value
return out	- - -

pv01

Calculate the value of 1 basis point coupon on the fixed leg.

```
pv01(value_dt, discount_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
discount_curve	-	-	-

swap_rate

Calculate the fixed leg cpn that makes the swap worth zero. If the valuation date is before the swap payments start then this is the forward swap rate as it starts in the future. The swap rate is then a forward swap rate and so we use a forward discount factor. If the swap fixed leg has begun then we have a spot starting swap. The swap rate can also be calculated in a dual curve approach but in this case the first fixing on the floating leg is needed.

```

swap_rate(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
first_fixing: float = None,
"""Calculate the fixed leg cpn that makes the swap worth zero.
If the valuation date is before the swap payments start then this
is the forward swap rate as it starts in the future. The swap rate
is then a forward swap rate and so we use a forward discount
factor. If the swap fixed leg has begun then we have a spot
starting swap. The swap rate can also be calculated in a dual curve
approach but in this case the first fixing on the floating leg is
needed."""
pv01 = .pv01(value_dt, discount_curve)
if abs(pv01) < G_SMALL:
    raise FinError("PV01 is zero. Cannot compute swap rate.")
float_leg_pv = .float_leg.value(value_dt, discount_curve, index_curve, first_fixing
)
float_leg_pv /= .float_leg.notional
# Make sure we get the sign right
if .float_leg.leg_type == SwapTypes.PAY:
    float_leg_pv = -float_leg_pv
cpn = float_leg_pv / pv01
return cpn
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve	DiscountCurve	-	None
first_fixing	float	-	None
"""Calculate the fixed leg cpn that makes the swap worth zero.			
If the valuation date is before the swap payments start then this			
is the forward swap rate as it starts in the future. The swap rate			
is then a forward swap rate and so we use a forward discount			
factor. If the swap fixed leg has begun then we have a spot			
starting swap. The swap rate can also be calculated in a dual curve			
approach but in this case the first fixing on the floating leg is			
needed."""			
pv01	-	-	.pv01(value_dt, disc
if abs(pv01) < G_SMALL			
raise FinError("PV01 is zero. Cannot compute swap rate.")			
)			
float_leg_pv /	-	-	.float_leg.noti
if .float_leg.leg_type	-	-	= SwapTypes.
float_leg_pv	-	-	-float_leg.p
cpn	-	-	float_leg_pv / j
return cpn	-	-	-

cash_settled_pv01

Calculate the forward value of an annuity of a forward starting swap using a single flat discount rate equal to the swap rate. This is used in the pricing of a cash-settled swaption in the *IborSwaption* class. This method does not affect the standard valuation methods.

```
cash_settled_pv01(value_dt, flat_swap_rate, freq_type):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
flat_swap_rate	-	-	-
freq_type	-	-	-

print_fixed_leg_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_fixed_leg_pv():
```

The function arguments are described in the following table.

print_float_leg_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_float_leg_pv() :
```

The function arguments are described in the following table.

print_payments

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_payments() :
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() :
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted cpn payment dates used in analytic calculations for the bond.

```
_print() :
```

The function arguments are described in the following table.

7.12 ibor_fra

Class: IborFRA

Class for managing LIBOR forward rate agreements. A forward rate agreement is an agreement to exchange a fixed pre-agreed rate for a floating rate linked to LIBOR that is not known until some specified future fixing date. The FRA payment occurs on or soon after this date on the FRA settlement date. Typically the timing gap is two days.

A FRA is used to hedge a Ibor quality loan or lend of some agreed notional amount. This period starts on the settlement date of the FRA and ends on the maturity date of the FRA. For example a 1x4 FRA relates to a Ibor starting in 1 month for a loan period ending in 4 months. Hence it links to 3-month Ibor rate. The amount received by a payer of fixed rate at settlement is:

$$\text{acc}(1,2) * (\text{Ibor}(1,2) - \text{FRA RATE}) / (1 + \text{acc}(0,1) * \text{Ibor}(0,1))$$

So the value at time 0 is

$$\text{acc}(1,2) * (\text{FWD Ibor}(1,2) - \text{FRA RATE}) * \text{df}(0,2)$$

If the base date of the curve is before the value date then we forward adjust this amount to that value date. For simplicity I have assumed that the fixing date and the settlement date are the same date. This should be amended later.

The valuation below incorporates a dual curve approach.

IborFRA

PLEASE ADD A FUNCTION DESCRIPTION

```
IborFRA(start_dt: Date, # The date the FRA starts to accrue
# End of the Ibor rate period
maturity_dt_or_tenor: Union[Date, str],
fra_rate: float, # The fixed contractual FRA rate
dc_type: DayCountTypes, # For interest period
notional: float = 100.0,
pay_fixed_rate: bool = True, # True if the FRA rate is being paid
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.MODIFIED_FOLLOWING,
"""Create_a_Forward_Rate_Agreement_object."""
check_argument_types(.IborFRA, locals())
.cal_type = cal_type
.bd_type = bd_type
if isinstance(maturity_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
start_dt	Date	The date the FRA starts to accrue
maturity_dt_or_tenor	Date or str	-
fra_rate	float	The fixed contractual FRA rate
dc_type	DayCountTypes	For interest period
notional	float	-
pay_fixed_rate	bool	True if the FRA rate is being paid
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
"""Create a Forward Rate Agreement object."""		-
check_argument_types(.IborFRA, locals())	-	-
.cal_type	-	-
.bd_type	-	-

value

Determine mark to market value of a FRA contract based on the market FRA rate. We allow the pricing to have a different curve for the Libor index and the discounting of promised cash flows.

```

value(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
pv_only=True,
"""Determine mark to market value of a FRA contract based on the
market FRA rate. We allow the pricing to have a different curve for
the Libor index and the discounting of promised cash flows."""
if index_curve is None:
    index_curve = discount_curve
# Get the Libor index from the index curve
dc = DayCount(.dc_type)
acc_factor = dc.year_frac(.start_dt, .maturity_dt)[0]
df_index1 = index_curve.df(.start_dt)
df_index2 = index_curve.df(.maturity_dt)
libor_fwd = (df_index1 / df_index2 - 1.0) / acc_factor
# Get the discount factor from a discount curve
df_mat = discount_curve.df(.maturity_dt)
v = acc_factor * (libor_fwd - .fra_rate) * df_mat
# Forward value the FRA to the value date
df_value = discount_curve.df(value_dt)
v = v * .notional / df_value
pay_fixed_sign = 1
if .pay_fixed_rate is True:
    pay_fixed_sign = -1 # VP: is this the right sign?
    v *= -1.0
if pv_only:
    return v
else:
    df = pd.DataFrame(index=[0])
    df["payment_date"] = .maturity_dt
    df["start_accrual_date"] = .start_dt
    df["end_accrual_date"] = .maturity_dt
    df["year_frac"] = acc_factor

```

```
df["rate"] = libor_fwd - .fra_rate
df["payment"] = (pay_fixed_sign
    * acc_factor
    * (libor_fwd - .fra_rate)
    * .notional
)
df["payment_df"] = df_mat / df_value
df["payment_pv"] = v
df["leg"] = "FRA"
return (v,)
```

#####

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
discount_curve	DiscountCurve	-
index_curve	DiscountCurve	-
pv_only	-	-
"""Determine mark to market value of a FRA contract based on the market FRA rate. We allow the pricing to have a different curve for the Libor index and the discounting of promised cash flows."""	-	-
if index_curve is None	-	-
index_curve	-	-
dc	-	-
acc_factor	-	-
df_index1	-	-
df_index2	-	-
libor_fwd	-	-
df_mat	-	-
v	-	-
df_value	-	-
v	-	-
pay_fixed_sign	-	-
if .pay_fixed_rate is True	-	-
pay_fixed_sign	-	VP: is this the right sign?
v *	-	-
if pv_only	-	-
return v	-	-
else	-	-
df	-	-
df["payment_date"]	-	-
df["start_accrual_date"]	-	-
df["end_accrual_date"]	-	-
df["year_frac"]	-	-
df["rate"]	-	-
* acc_factor	-	-
* (libor_fwd - .fra_rate)	-	-
* .notional	-	-
)	-	-
df["payment_df"]	-	-
df["payment_pv"]	-	-
df["leg"]	-	-
return (v,)	-	-

valuation_details

A long-hand method that returns various details relevant to valuation in a dictionary. Slower than value(...) so should not be used when performance is important We want the output dictionary to have the same labels

for different benchmarks (depos, fras, swaps) because we want to present them together so please do not stick new outputs into one of them only

```

valuation_details(valuation_date: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
"""
A long-hand method that returns various details relevant to valuation
in a dictionary. Slower than value(...) so should not be used when
performance is important
We want the output dictionary to have the same labels for different
benchmarks (depos, fras, swaps) because we want to present them
together so please do not stick new outputs into
one of them only
"""
if index_curve is None:
    index_curve = discount_curve
# Get the Libor index from the index curve
dc = DayCount(.dc_type)
acc_factor = dc.year_frac(.start_dt, .maturity_dt)[0]
df_index_1 = index_curve.df(.start_dt)
df_index_2 = index_curve.df(.maturity_dt)
libor_fwd = (df_index_1 / df_index_2 - 1.0) / acc_factor
# Get the discount factor from a discount curve
df_discount_2 = discount_curve.df(.maturity_dt)
v = acc_factor * (libor_fwd - .fra_rate) * df_discount_2
# Forward value the FRA to the value date
df_to_valuation_date = discount_curve.df(valuation_date)
v = v * .notional / df_to_valuation_date
if .pay_fixed_rate is True: # VP: ??? pay fixed should be positive notional
    v *= -1.0
out = {
    "type": type().__name__,
    "start_dt": .start_dt,
    "maturity_dt": .maturity_dt,
    "dc_type": .dc_type.name,
    "fixed_leg_type": (SwapTypes.PAY.name if .pay_fixed_rate else SwapTypes.RECEIVE.
        name
    ),
    "notional": .notional,
    "contract_rate": .fra_rate,
    "market_rate": libor_fwd,
    "spot_pvbpb": acc_factor * df_discount_2,
    "fwd_pvbpb": acc_factor * df_discount_2 / discount_curve.df(.start_dt),
    "unit_value": acc_factor * df_discount_2 * (libor_fwd - .fra_rate),
    "value": v,
    # ignoring pay_fixed flag (which is wrong anyway I think),
    # bus day adj type, calendar for now
}
return out
#####

```

The function arguments are described in the following table.

Argument Name	Type
valuation_date	Date
discount_curve	DiscountCurve
index_curve	DiscountCurve
"""	-
A long-hand method that returns various details relevant to valuation	-
in a dictionary. Slower than value(...) so should not be used when	-
performance is important	-
We want the output dictionary to have the same labels for different	-
benchmarks (depos, fras, swaps) because we want to present them	-
together so please do not stick new outputs into	-
one of them only	-
"""	-
if index_curve is None	
index_curve	-
dc	-
acc_factor	-
df_index_1	-
df_index_2	-
libor_fwd	-
df_discount_2	-
v	-
df_to_valuation_date	-
v	-
if .pay_fixed_rate is True	
v *	-
out	-
"type"	).name_ or type(
"start_dt"	.start_dt
"maturity_dt"	.maturity_dt
"dc_type"	.dc_type.name
)	-
"notional"	.notional
"contract_rate"	.fra_rate
"market_rate"	libor_fwd
"spot_pvbp"	acc_factor*df_discount_2
"fwd_pvbp"	.start_dt or acc_factor*df_discount_2/discount_cur
"unit_value"	libor_fwd-.fra_rate or acc_factor*df_discount_2*(lib
"value"	v
return out	- - -

maturity_df

Determine the maturity date index discount factor needed to refit the market FRA rate. In a dual-curve world, this is not the discount rate discount factor but the index curve discount factor.

```
maturity_df(index_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
index_curve	-	-	-

print_payments

Determine the value of the Deposit given a Ibor curve.

```
print_payments(value_dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

PLEASE ADD A FUNCTION DESCRIPTION

```
_print():
```

The function arguments are described in the following table.

7.13 ibor_future

Class: IborFuture

IborFuture

PLEASE ADD A FUNCTION DESCRIPTION

```
IborFuture(today_dt: Date,
future_number: int, # The number of the future after today_dt
future_tenor: str = "3M", # '1M', '2M', '3M'
dc_type: DayCountTypes = DayCountTypes.ACT_360,
contract_size: float = ONE_MILLION,
"""Create an interest rate futures contract which has the same
conventions as those traded on the CME. The current_dt, the tenor of
the future, the number of the future and the accrual convention and
the contract size should be provided."""
check_argument_types(.IborFuture, locals())
if future_number < 1:
    raise FinError("Future number must be 1 or more")
if future_tenor not in ["3M", "3m"]:
    raise FinError("Only 3M IMM futures handled currently.")
.delivery_dt = today_dt.next_imm_date()
for _ in range(0, future_number - 1):
```

The function arguments are described in the following table.

Argument Name	Type	Description
today_dt	Date	-
future_number	int	The number of the future after today_dt
future_tenor	str	'1M', '2M', '3M'
dc_type	DayCountTypes	-
contract_size	float	-
"""Create an interest rate futures contract which has the same	-	-
conventions as those traded on the CME. The current_dt	-	-
the tenor of	-	-
the future	-	-
the number of the future and the accrual convention and	-	-
the contract size should be provided."""	-	-
check_argument_types(.IborFuture, locals())	-	-
if future_number < 1	-	-
raise FinError("Future number must be 1 or more")	-	-
if future_tenor not in ["3M", "3m"]	-	-
raise FinError("Only 3M IMM futures handled currently.")	-	-
.delivery_dt	-	-

to fra

Convert the futures contract to a IborFRA object so it can be used to bootstrap a Ibor curve. For this we need

to adjust the futures rate using the convexity correction.

```
to_fra(futures_price, convexity):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
futures_price	-	-	-
convexity	-	-	-

futures_rate

Calculate implied futures rate from the futures price.

```
futures_rate(futures_price):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
futures_price	-	-	-

fra_rate

Convert futures price and convexity to a FRA rate using the BBG negative convexity (in percent). This is then divided by 100 before being added to the futures rate.

```
fra_rate(futures_price, convexity):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
futures_price	-	-	-
convexity	-	-	-

convexity

Calculation of the convexity adjustment between FRAs and interest rate futures using the Hull-White model as described in technical note in link below: <http://www-2.rotman.utoronto.ca/~hull/TechnicalNotes/TechnicalNote1.pdf> NOTE THIS DOES NOT APPEAR TO AGREE WITH BLOOMBERG!! INVESTIGATE.

```
convexity(value_dt, volatility, mean_reversion):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
volatility	-	-	-
mean_reversion	-	-	-

--repr--

Print a list of the unadjusted coupon payment `_dts` used in analytic calculations for the bond.

```
__repr__():
```

The function arguments are described in the following table.

7.14 ibor_lmm_products

Class: *IborLMMProducts*

IborLMMProducts

Create a European-style swaption by defining the exercise date of the swaption, and all of the details of the underlying interest rate swap including the fixed cpn and the details of the fixed and the floating leg payment schedules.

```
IborLMMProducts(settle_dt: Date,
maturity_dt: Date,
float_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
float_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create a European-style swaption by defining the exercise date of
the swaption, and all of the details of the underlying interest rate
swap including the fixed cpn and the details of the fixed and the
floating leg payment schedules."""
check_argument_types(.IborLMMProducts, locals())
if settle_dt > maturity_dt:
    raise FinError("Settlement date must be before maturity date")
"""Set up the grid for the Ibor rates that are to be simulated. These
must be consistent with the floating rate leg of the product that is to
be priced."""
.start_dt = settle_dt
.grid_dts = Schedule(settle_dt, maturity_dt, float_freq_type, cal_type, bd_type,
    dg_type
).generate()
.accrual_factors = []
.float_dc_type = float_dc_type
basis = DayCount(.float_dc_type)
prev_dt = .grid_dts[0]
.grid_times = [0.0]
for next_dt in .grid_dts[1:]:
    tau = basis.year_frac(prev_dt, next_dt)[0]
    t = (next_dt - .grid_dts[0]) / G_DAYS_IN_YEAR
    .accrual_factors.append(tau)
    .grid_times.append(t)
    prev_dt = next_dt
#     print(.grid_times)
.accrual_factors = np.array(.accrual_factors)
.num_fwds = len(.accrual_factors)
.fwds = None
.use_sobol = None
.num_paths = None
.numeraire_index = None
.fwd_curve = None
.vol_curves = None
.corr_matrix = None
.model_type = None
    print("Num_FORWARDS", .num_fwds)
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
settle_dt	Date	-
maturity_dt	Date	-
float_freq_type	FrequencyTypes	-
float_dc_type	DayCountTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create a European-style swaption by defining the exercise date of the swaption	-	-
and all of the details of the underlying interest rate	-	-
swap including the fixed cpn and the details of the fixed and the floating leg payment schedules."""	-	-
check_argument_types(.IborLMMProducts, locals())	-	-
if settle_dt > maturity_dt	-	-
raise FinError("Settlement date must be before maturity date")	-	-
""" Set up the grid for the Ibor rates that are to be simulated. These must be consistent with the floating rate leg of the product that is to be priced. """	-	-
.start_dt	-	-
.accrual_factors	-	-
.float_dc_type	-	-
basis	-	-
prev_dt	-	-
.grid_times	-	-
for next_dt in .grid_dts[1	]:	-
tau	-	-
t	-	-
.accrual_factors.append(tau)	-	-
.grid_times.append(t)	-	-
prev_dt	-	-
.accrual_factors	-	-
.num_fwds	-	-
.fwds	-	-
.use_sobol	-	-
.num_paths	-	-
.numeraire_index	-	-
.fwd_curve	-	-
.vol_curves	-	-
.corr_matrix	-	-
.model_type	-	-
print("Num FORWARDS", .num_fwds)	-	-

simulate_1f

PLEASE ADD A FUNCTION DESCRIPTION

```
simulate_1f(discount_curve,
vol_curve: IborCapVolCurve,
num_paths: int = 1000,
numeraire_index: int = 0,
use_sobol: bool = True,
seed: int = 42,
"""Run the one-factor simulation of the evolution of the forward
Ibors to generate and store all of the Ibor forward rate paths."""
if num_paths < 2 or num_paths > 1000000:
    raise FinError("NumPaths must be between 2 and 1 million")
if discount_curve.value_dt != .start_dt:
    raise FinError("Curve anchor date not the same as LMM start date.")
.num_paths = num_paths
.numeraire_index = numeraire_index
.use_sobol = use_sobol
num_grid_points = len(.grid_dts)
.num_fwds = num_grid_points
.fwd_curve = []
for i in range(1, num_grid_points):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
discount_curve	-	-	-
vol_curve	IborCapVolCurve	-	-
num_paths	int	-	1000
numeraire_index	int	-	0
use_sobol	bool	-	True
seed	int	-	42
"""Run the one-factor simulation of the evolution of the forward Ibors to generate and store all of the Ibor forward rate paths."""	-	-	-
if num_paths < 2 or num_paths > 1000000	-	-	-
raise FinError("NumPaths must be between 2 and 1 million")	-	-	-
if discount_curve.value_dt !=	-	-	.start_dt:
raise FinError("Curve anchor date not the same as LMM start date.")	-	-	-
.num_paths	-	-	num_paths
.numeraire_index	-	-	numeraire_index
.use_sobol	-	-	use_sobol
num_grid_points	-	-	len(.grid_dts)
.num_fwds	-	-	num_grid_points
.fwd_curve	-	-	[]

simulate_mf

PLEASE ADD A FUNCTION DESCRIPTION

```

simulate_mf(discount_curve,
num_factors: int,
lambdas: np.ndarray,
num_paths: int = 10000,
numeraire_index: int = 0,
use_sobol: bool = True,
seed: int = 42,
"""Run the simulation to generate and store all of the Ibor forward
rate paths. This is a multi-factorial version so the user must input
a numpy array consisting of a column for each factor and the number of
rows must equal the number of grid times on the underlying simulation
grid. CHECK THIS."""
#     check_argument_types(__init__, locals())
if num_paths < 2 or num_paths > 1000000:
    raise FinError("NumPaths must be between 2 and 1 million")
if discount_curve.curve_dt != .start_dt:
    raise FinError("Curve anchor date not the same as LMM start date.")
print("LEN LAMBDAS", len(lambdas))
print("LEN", len(lambdas[0]))
# We pass a vector of vol discount, one for each factor
if num_factors != len(lambdas):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
discount_curve	-	-	-
num_factors	int	-	-
lambdas	np.ndarray	-	-
num_paths	int	-	10000
numeraire_index	int	-	0
use_sobol	bool	-	True
seed	int	-	42
"""Run the simulation to generate and store all of the Ibor forward	-	-	-
rate paths. This is a multi-factorial version so the user must input	-	-	-
a numpy array consisting of a column for each factor and the number of	-	-	-
rows must equal the number of grid times on the underlying simulation	-	-	-
grid. CHECK THIS."""	-	-	-
if num_paths < 2 or num_paths > 1000000	-	-	-
raise FinError("NumPaths must be between 2 and 1 million")	-	-	-
if discount_curve.curve_dt !=	-	-	.start_dt:
raise FinError("Curve anchor date not the same as LMM start date.")	-	-	-
print("LEN LAMBDAS", len(lambdas))	-	-	-
print("LEN", len(lambdas[0]))	-	-	-

simulate_nf

PLEASE ADD A FUNCTION DESCRIPTION

```

simulate_nf(discount_curve,

```

```

vol_curve: IborCapVolCurve,
corr_matrix: np.ndarray,
model_type: ModelLMMModelTypes,
num_paths: int = 1000,
numeraire_index: int = 0,
use_sobol: bool = True,
seed: int = 42,
"""Run the simulation to generate and store all of the Ibor forward
rate paths using a full factor reduction of the fwd-fwd correlation
matrix using Cholesky decomposition."""
check_argument_types(__init__, locals())
if num_paths < 2 or num_paths > 1000000:
    raise FinError("NumPaths must be between 2 and 1 million")
if isinstance(model_type, ModelLMMModelTypes) is False:
    raise FinError("Model type must be type FinRateModelLMMModelTypes")
if discount_curve.curve_dt != .start_dt:
    raise FinError("Curve anchor date not the same as LMM start date.")
.num_paths = num_paths
.vol_curves = vol_curve
.corr_matrix = corr_matrix
.model_type = model_type
.numeraire_index = numeraire_index
.use_sobol = use_sobol
num_grid_points = len(.grid_times)
.num_fwds = num_grid_points - 1
.fwd_curve = []
for i in range(1, num_grid_points):

```

The function arguments are described in the following table.

Argument Name	Type	Description
discount_curve	-	-
vol_curve	IborCapVolCurve	-
corr_matrix	np.ndarray	-
model_type	ModelLMMModelTypes	-
num_paths	int	-
numeraire_index	int	-
use_sobol	bool	-
seed	int	-
"""Run the simulation to generate and store all of the Ibor forward rate paths using a full factor reduction of the fwd-fwd correlation matrix using Cholesky decomposition."""	-	-
check_argument_types(...init..., locals())	-	-
if num_paths < 2 or num_paths > 1000000	-	-
raise FinError("NumPaths must be between 2 and 1 million")	-	-
if isinstance(model_type, ModelLMMModelTypes) is False	-	-
raise FinError("Model type must be type FinRateModelLMMModelTypes")	-	-
if discount_curve.curve_dt !=	-	-
raise FinError("Curve anchor date not the same as LMM start date.")	-	-
.num_paths	-	-
.vol_curves	-	-
.corr_matrix	-	-
.model_type	-	-
.numeraire_index	-	-
.use_sobol	-	-
num_grid_points	-	-
.num_fwds	-	-
.fwd_curve	-	-

value_swaption

Value a swaption in the LMM model using simulated paths of the forward curve. This relies on pricing the fixed leg of the swap and assuming that the floating leg will be worth par. As a result we only need simulate Ibors with the frequency of the fixed leg.

```
value_swaption(settle_dt: Date,
exercise_dt: Date,
maturity_dt: Date,
swaption_type: SwapTypes,
fixed_cpn: float,
fixed_freq_type: FrequencyTypes,
fixed_dc_type: DayCountTypes,
notional: float = ONE_MILLION,
float_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
float_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
```

```

"""Value_a_swaption_in_the_LMM_model_using_simulated_paths_of_the
forward_curve. This relies on pricing the fixed leg of the swap and
assuming that the floating leg will be worth par. As a result we only
need simulate Ibors with the frequency of the fixed leg."""
# Note that the simulation time steps run all the way out to the last
# forward rate. However we only really need the forward rates at the
# expiry date of the option. It may be worth amending the simulate
# code to impose a limit on the time steps in order to speed up the
# overall pricing if it requires a new run every time. However once
# generated, the speed of pricing is not affected so this is not
# strictly an urgent issue.
swaption_float_dts = Schedule(settle_dt, maturity_dt, float_freq_type, cal_type,
                             bd_type, dg_type
                             ).generate()
for swaption_dt in swaption_float_dts:
    found_dt = False
    for grid_dt in .grid_dts:
        if swaption_dt == grid_dt:
            found_dt = True
            break
    if found_dt is False:
        raise FinError("Swaption_float_leg_not_on_grid.")
swaption_fixed_dts = Schedule(settle_dt, maturity_dt, fixed_freq_type, cal_type,
                             bd_type, dg_type
                             ).generate()
for swaption_dt in swaption_fixed_dts:
    found_dt = False
    for grid_dt in .grid_dts:
        if swaption_dt == grid_dt:
            found_dt = True
            break
    if found_dt is False:
        raise FinError("Swaption_fixed_leg_not_on_grid.")
a = 0
b = 0
for grid_dt in .grid_dts:
    if grid_dt == exercise_dt:
        break
    else:
        a += 1
for grid_dt in .grid_dts:
    if grid_dt == maturity_dt:
        break
    else:
        b += 1
if b == 0:
    raise FinError("Swaption_swap_maturity_date_is_today.")
#     num_paths = 1000
#     v = LMMSwaptionPricer(fixed_cpn, a, b, num_paths,
#                           fwd0, fwds, taus, is_payer)
v = 0.0
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	D
settle_dt	Date	-	
exercise_dt	Date	-	
maturity_dt	Date	-	
swaption_type	SwapTypes	-	
fixed_cpn	float	-	
fixed_freq_type	FrequencyTypes	-	
fixed_dc_type	DayCountTypes	-	
notional	float	-	ON
float_freq_type	FrequencyTypes	-	Frequency
float_dc_type	DayCountTypes	-	DayCountT
cal_type	CalendarTypes	-	Calendar
bd_type	BusDayAdjustTypes	-	BusDayAdju
dg_type	DateGenRuleTypes	-	DateGenRu
"""Value a swaption in the LMM model using simulated paths of the forward curve. This relies on pricing the fixed leg of the swap and assuming that the floating leg will be worth par. As a result we only need simulate Ibors with the frequency of the fixed leg."""	-	-	
for swaption_dt in swaption_float_dts		-	
found_dt	-	-	
for grid_dt in .grid_dts		-	
if swaption_dt	-	-	
found_dt	-	-	
break	-	-	
if found_dt is False		-	
raise FinError("Swaption float leg not on grid.")	-	-	
for swaption_dt in swaption_fixed_dts		-	
found_dt	-	-	
for grid_dt in .grid_dts		-	
if swaption_dt	-	-	
found_dt	-	-	
break	-	-	
if found_dt is False		-	
raise FinError("Swaption fixed leg not on grid.")	-	-	
a	-	-	
b	-	-	
for grid_dt in .grid_dts		-	
if grid_dt	-	-	=
break	-	-	
else		-	
a +	-	-	
for grid_dt in .grid_dts		-	
if grid_dt	-	-	=
break	-	-	
else		-	
b +	-	-	
if b	-	-	
raise FinError("Swaption swap maturity date is today.")	-	-	
v	-	-	
return v	-	-	

value_cap_floor

Value a cap or floor in the LMM.

```

value_cap_floor(settle_dt: Date,
maturity_dt: Date,
cap_floor_type: CapFloorTypes,
cap_floor_rate: float,
freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
dc_type: DayCountTypes = DayCountTypes.ACT_360,
notional: float = ONE_MILLION,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Value a cap or floor in the LMM."""
cap_floor_dts = Schedule(settle_dt, maturity_dt, freq_type, cal_type, bd_type, dg_type
).generate()
for cap_floorlet_dt in cap_floor_dts:
    found_dt = False
    for grid_dt in .grid_dts:
        if cap_floorlet_dt == grid_dt:
            found_dt = True
            break
    if found_dt is False:
        raise FinError("CapFloor_date_not_on_grid.")
num_fwds = len(cap_floor_dts)
num_paths = .num_paths
is_cap = 0
if cap_floor_type == CapFloorTypes.CAP:
    is_cap = 1
fwd0 = .fwd_curve
fwds = .fwds
taus = .accrual_factors
v = lmm_cap_flr_pricer(num_fwds, num_paths, cap_floor_rate, fwd0, fwds, taus, is_cap
)
# Sum the cap/floorlets to get cap/floor value
v_cap_floor = 0.0
for v_cap_floor_let in v:
    v_cap_floor += v_cap_floor_let * notional
return v_cap_floor
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
settle_dt	Date	-	-
maturity_dt	Date	-	-
cap_floor_type	CapFloorTypes	-	-
cap_floor_rate	float	-	-
freq_type	FrequencyTypes	-	FrequencyTypes.QUARTERLY
dc_type	DayCountTypes	-	DayCountTypes.ACT_360
notional	float	-	ONE_MILLION
cal_type	CalendarTypes	-	CalendarTypes.WEEKEND
bd_type	BusDayAdjustTypes	-	BusDayAdjustTypes.FOLLOWING
dg_type	DateGenRuleTypes	-	DateGenRuleTypes.BACKWARD
"""Value a cap or floor in the LMM."""	-	-	-
for cap_floorlet_dt in cap_floor_dts	-	-	-
found_dt	-	-	False
for grid_dt in .grid_dts	-	-	-
if cap_floorlet_dt	-	-	= grid_dt:
found_dt	-	-	True
break	-	-	-
if found_dt is False	-	-	-
raise FinError("CapFloor date not on grid.")	-	-	-
num_fwds	-	-	len(cap_floor_dts)
num_paths	-	-	.num_paths
is_cap	-	-	0
if cap_floor_type	-	-	= CapFloorTypes.CAP:
is_cap	-	-	1
fwd0	-	-	.fwd_curve
fwds	-	-	.fwds
taus	-	-	.accrual_factors
)	-	-	-
v_cap_floor	-	-	0.0
for v_cap_floor_let in v	-	-	-
v_cap_floor +	-	-	v_cap_floor.let * notional
return v_cap_floor	-	-	-

__repr__

Function to allow us to print the LMM Products details.

```
__repr__():
```

The function arguments are described in the following table.

_print

Alternative print method.

```
_print():
```

The function arguments are described in the following table.

7.15 ibor_single_curve

Class: *IborSingleCurve(DiscountCurve)*

Constructs one discount and index curve as implied by prices of Ibor deposits, FRAs and IRS. Discounting is assumed to be at Libor and the value of the floating leg (including a notional) is assumed to be par. This approach has been overtaken since 2008 as OIS discounting has become the agreed discounting approach for ISDA derivatives. This curve method is therefore intended for those happy to assume simple Libor discounting.

The curve date is the date on which we are performing the valuation based on the information available on the curve date. Typically it is the date on which an amount of 1 unit paid has a present value of 1. This class inherits from DiscountCurve and so it has all of the methods that that class has.

There are two main curve-building approaches:

1) The first uses a bootstrap that interpolates swap rates linearly for coupon dates that fall between the swap maturity dates. With this, we can solve for the discount factors iteratively without need of a solver. This will give us a set of discount factors on the grid dates that refit the market exactly. However, when extracting discount factors, we will then assume flat forward rates between these coupon dates. There is no contradiction as it is as though we had been quoted a swap curve with all of the market swap rates, and with an additional set as though the market quoted swap rates at a higher frequency than the market.

2) The second uses a bootstrap that uses only the swap rates provided but which also assumes that forwards are flat between these swap maturity dates. This approach is non-linear and so requires a solver. Consequently it is slower. Its advantage is that we can switch interpolation schemes to provide a smoother or other functional curve shape which may have a more economically justifiable shape. However the root search makes it slower.

IborSingleCurve

Create an instance of a FinIbor curve given a valuation date and a set of ibor deposits, ibor FRAs and ibor swaps. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on coupon dates and to then assume flat forwards between these coupon dates. The curve will assign a discount factor of 1.0 to the valuation date. If no instrument is starting on the valuation date, the curve is then assumed to be flat out to the first instrument using its zero rate.

```
IborSingleCurve(value_dt: Date, # This is the trade date (not T+2)
ibor_deposits: list,
ibor_fras: list,
ibor_swaps: list,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
check_refit_flag: bool = False, # Set to True to test it works
do_build: bool = True,
**kwargs,
"""Create an instance of a FinIbor curve given a valuation date and
a set of ibor deposits, ibor FRAs and ibor swaps. Some of these may
be left None and the algorithm will just use what is provided. An
interpolation method has also to be provided. The default is to use a
linear interpolation for swap rates on coupon dates and to then assume
flat forwards between these coupon dates.
The curve will assign a discount factor of 1.0 to the valuation date.
If no instrument is starting on the valuation date, the curve is then
```

```

assumed_to_be_flat_out_to_the_first_instrument_using_its_zero_rate.
"""
check_argument_types(getattr(_func_name(), None), locals())
.value_dt = value_dt
._interp_type = interp_type
.check_refit_flag = check_refit_flag
._interpolator = Interpolator(._interp_type, **kwargs)
.is_built = False
._optional_interp_params = kwargs
._validate_inputs(ibor_deposits=ibor_deposits,
                  ibor_fras=ibor_fras,
                  ibor_swaps=ibor_swaps,
)
if do_build:
    .build_curve(**kwargs)
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	This is the trade date (not T+2)
ibor_deposits	list	-
ibor_fras	list	-
ibor_swaps	list	-
interp_type	InterpTypes	-
check_refit_flag	bool	Set to True to test it works
do_build	bool	-
**kwargs	-	-
"""Create an instance of a FinIbor curve given a valuation date and a set of ibor deposits ibor FRAs and ibor_swaps. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on coupon dates and to then assume flat forwards between these coupon dates. The curve will assign a discount factor of 1.0 to the valuation date. If no instrument is starting on the valuation date the curve is then assumed to be flat out to the first instrument using its zero rate. """	-	-
check_argument_types(getattr(_func_name(), None), locals())	-	-
.value_dt	-	-
.interp_type	-	-
.check_refit_flag	-	-
.interpolator	-	-
.is_built	-	-
._optional_interp_params	-	-
ibor_fras	-	-
ibor_swaps	-	-
)	-	-
if do_build	-	-
.build_curve(**kwargs)	-	-

build_curve

Build curve based on interpolation. Not all interpolators are suitable for the bootstrap/1d solver, only those that are local, where the value of $df[i]$ does not affect discount factors for $t_i=t[i-1]$

```
build_curve(**kwargs) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
**kwargs	-	-	-

`_validate_inputs`

Validate the inputs for each of the Ibor products.

```
_validate_inputs(ibor_deposits, ibor_fras, ibor_swaps):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ibor_deposits	-	-	-
ibor_fras	-	-	-
ibor_swaps	-	-	-

`_build_curve_using_1d_solver`

Construct the discount curve using a bootstrap approach. This is the non-linear slower method that allows the user to choose a number of interpolation approaches between the swap rates and other rates. It involves the use of a solver.

```
_build_curve_using_1d_solver(**kwargs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
**kwargs	-	-	-

`_build_curve_using_least_squares`

Construct the discount curve using a least-squares minimisation approach. This enables a more complex interpolation scheme.

```
_build_curve_using_least_squares(**kwargs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
**kwargs	-	-	-

`_obj_f_curve_build_ls`

Objective function for fitting all knot dfs at once to the benchmark securities – suitable for non-local interpolators

```
_obj_f_curve_build_ls(dfs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dfs	-	-	-

`_build_curve_linear_swap_rate_interpolation`

Construct the discount curve using a bootstrap approach. This is the linear swap rate method that is fast and exact as it does not require the use of a solver. It is also market standard.

```
_build_curve_linear_swap_rate_interpolation():
```

The function arguments are described in the following table.

`check_refit`

Ensure that the Ibor curve refits the calibration instruments.

```
check_refit(depo_tol, fra_tol, swap_tol):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
depo_tol	-	-	-
fra_tol	-	-	-
swap_tol	-	-	-

`_df`

Override from DiscountCurve so we can check if the curve has actually been built.

```
_df(t: Union[float, np.ndarray]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	float or np.ndarray	-	-

`__repr__`

Print out the details of the Ibor curve.

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

f

Root search objective function for IRS

```
_f(
df,
*args,
"""Root_search_objective_function_for_IRS"""
curve = args[0]
value_dt = args[1]
swap = args[2]
curve.set_last_df(df)
# For discount that need a fit function, we fit it now
curve.fit(curve.times, curve.dfs)
v_swap = swap.value(value_dt, curve, curve, None)
notional = swap.fixed_leg.notional
v_swap /= notional
return v_swap
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-
"""Root search objective function for IRS"""	-	-	-
curve	-	-	args[0]
value_dt	-	-	args[1]
swap	-	-	args[2]
curve.set_last_df(df)	-	-	-
curve.fit(curve.times, curve.dfs)	-	-	-
v_swap	-	-	swap.value(value_dt, curve, curve, None)
notional	-	-	swap.fixed_leg.notional
v_swap /	-	-	notional
return v_swap	-	-	-

g

Root search objective function for FRAs

```
_g(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

`_cost_function`

Objective function for fitting all knot dfs at once to the benchmark securities – suitable for non-local interpolators

```
_cost_function(dfs, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dfs	-	-	-
*args	-	-	-

7.16 ibor_single_curve_par_shocker

Class: IborSingleCurveParShocker

A class to apply par-rate, ie benchmark, bumps to a Libor curve. Takes a base curve and provides methods to apply bumps that return bumped curves

IborSingleCurveParShocker

Init with a base curve to be bumped. Bumps do not affect this curve.

```
IborSingleCurveParShocker(base_curve: IborSingleCurve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
base_curve	IborSingleCurve	-	-

benchmarks_report

Access the benchmarks report that we create when the shocker is initialized

```
benchmarks_report():
```

The function arguments are described in the following table.

n_benchmarks

Total number of benchmarks

```
n_benchmarks():
```

The function arguments are described in the following table.

apply_bump_to_benchmark

Apply a shock of a given size to a given benchmark. Indexing is per the benchmark report

```
apply_bump_to_benchmark(benchmark_idx: int, bump_size=1.0 * G_BASIS_POINT
"""
Apply a shock of a given size to a given benchmark.
Indexing is per the benchmark report
"""
composite_shock = np.zeros(n_benchmarks())
composite_shock[benchmark_idx] = bump_size
return .apply_composite_bump(composite_shock)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
benchmark_idx	int	-	-
bump_size	-	-	1.0 * G.BASIS_POINT
""""	-	-	-
Apply a shock of a given size to a given bechmark.	-	-	-
Indexing is per the benchmark report	-	-	-
""""	-	-	-
composite_shock	-	-	np.zeros(.n.benchmarks())
composite_shock[benchmark_idx]	-	-	bump_size
return .apply_composite_bump(composite_shock)	-	-	-

apply_composite_bump

Apply a composite bump to base_curve. A composite bump is a list/array of bumps, one per bechmark Args: *bump_sizes (Union[np.array, list]): a list/array of bump sizes, one per benchmark* Returns: *IborSingleCurve: A bumped curve*

```
apply_composite_bump(bump_sizes: Union[np.array, list]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
bump_sizes	np.array or list	-	-

7.17 ibor_single_curve_smoothing_calibrator

Class: *IborSingleCurveSmoothingCalibrator(object)*

Non-parametric fitting of a curve with smoothness. We use dfs for all coupon (cashflow) dates as input variables and impose a smoothness penalty on (at the moment) an approximation to the second derivative of the yields (zero rates). With non-zero smoothness penalty the fit to the benchmarks is not exact.

IborSingleCurveSmoothingCalibrator

Initialize with an (unbuilt) ibor curve. Note that this curve is not modified during fitting as we take a deep copy

```
IborSingleCurveSmoothingCalibrator(ibor_curve: IborSingleCurve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ibor_curve	IborSingleCurve	-	-

_collect_all_knot_dts

Collect all dats on which the discount factors are explicitly required to price bechmarks. Interpolation only used for times in between knot dates

```
_collect_all_knot_dts():
```

The function arguments are described in the following table.

_repricing_objectives

PLEASE ADD A FUNCTION DESCRIPTION

```
_repricing_objectives(curve_to_use=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
curve_to_use	-	-	None

_smoothing_objectives

PLEASE ADD A FUNCTION DESCRIPTION

```
_smoothing_objectives(curve_to_use=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
curve_to_use	-	-	None

fit

fit the curve with a given smoothness

```
fit(smoothness=1e-6, report_progress=False) -> IborSingleCurve:
"""
fit_the_curve_with_a_given_smoothness
"""
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
smoothness	-	-	1e-6
report_progress	-	-	False) -> IborSingleCurve:
"""	-	-	-
fit the curve with a given smoothness	-	-	-
"""	-	-	-

_obj_f

PLEASE ADD A FUNCTION DESCRIPTION

```
_obj_f(dfs):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
dfs	-	-	-

_generate_fit_report

PLEASE ADD A FUNCTION DESCRIPTION

```
_generate_fit_report(smoothness, curve_to_use=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
smoothness	-	-	-
curve_to_use	-	-	None

7.18 ibor_swap

Class: IborSwap

Class for managing a standard Fixed vs IBOR swap. This is a contract in which a fixed payment leg is exchanged for a series of floating rates payments linked to some IBOR index rate. There is no exchange of principal. The contract is entered into at zero initial cost. The contract lasts from a start date to a specified maturity date.

The floating rate is not known fully until the end of the preceding payment period. It is set in advance and paid in arrears.

The value of the contract is the NPV of the two cpn streams. Discounting is done on a supplied discount curve which is separate from the curve from which the implied index rates are extracted.

IborSwap

PLEASE ADD A FUNCTION DESCRIPTION

```
IborSwap(effective_dt: Date, # Date interest starts to accrue
term_dt_or_tenor: Union[Date, str], # Date contract ends
fixed_leg_type: SwapTypes,
fixed_cpn: float, # Fixed cpn (annualised)
fixed_freq_type: FrequencyTypes,
fixed_dc_type: DayCountTypes,
notional: float = ONE_MILLION,
float_spread: float = 0.0,
float_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
float_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create an interest rate swap contract giving the contract start
date, its maturity, fixed cpn, fixed leg frequency, fixed leg day
count, convention and notional. The floating leg parameters have default
values that can be overwritten if needed. The start date is contractual
and is the same as the settlement date for a new swap. It is the date
on which interest starts to accrue. The end of the contract is the
termination date. This is not adjusted for business days. The adjusted
termination date is called the maturity date. This is calculated."""
check_argument_types(.IborSwap, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts to accrue
term_dt_or_tenor	Date or str	Date contract ends
fixed_leg_type	SwapTypes	-
fixed_cpn	float	Fixed cpn (annualised)
fixed_freq_type	FrequencyTypes	-
fixed_dc_type	DayCountTypes	-
notional	float	-
float_spread	float	-
float_freq_type	FrequencyTypes	-
float_dc_type	DayCountTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create an interest rate swap contract giving the contract start		-
date	-	-
its maturity	-	-
fixed cpn	-	-
fixed leg frequency	-	-
fixed leg day	-	-
count convention and notional. The floating leg parameters have default	-	-
values that can be overwritten if needed. The start date is contractual	-	-
and is the same as the settlement date for a new swap. It is the date	-	-
on which interest starts to accrue. The end of the contract is the	-	-
termination date. This is not adjusted for business days. The adjusted	-	-
termination date is called the maturity date. This is calculated."""	-	-
check_argument_types(.IborSwap, locals())	-	-

get_fixed_rate

easy read access to the coupon (fixed rate)

```
get_fixed_rate():
```

The function arguments are described in the following table.

set_fixed_rate

Sometimes we need to reset the coupon (fixed rate) This function updates caches that depend on it

```
set_fixed_rate(new_rate: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
new_rate	float	-	-

set_fixed_rate_to_atm

Reset fixed rate to atm given curve(s). returns the new atm

```
set_fixed_rate_to_atm(valuation_date: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
first_fixing: float = None,
"""
Reset_fixed_rate_to_atm_given_curve(s).returns_the_new_atm
"""
atm = .swap_rate(valuation_date, discount_curve, index_curve, first_fixing)
.set_fixed_rate(atm)
return atm
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	D
valuation_date	Date	-	
discount_curve	DiscountCurve	-	
index_curve	DiscountCurve	-	
first_fixing	float	-	
"""	-	-	
Reset fixed rate to atm given curve(s). returns the new atm	-	-	
"""	-	-	
atm	-	-	.swap_rate(valuation_date, di
.set_fixed_rate(atm)	-	-	
return atm	-	-	

value

Value the interest rate swap on a value date given a single Ibor discount curve.

```
value(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
first_fixing_rate=None,
pv_only=True,
"""Value_the_interest_rate_swap_on_a_value_date_given_a_single_Ibor
discount_curve."""
if index_curve is None:
    index_curve = discount_curve
fixed_leg_results = .fixed_leg.value(value_dt, discount_curve, pv_only=pv_only
)
float_leg_results = .float_leg.value(value_dt,
    discount_curve,
    index_curve,
    first_fixing_rate,
    pv_only=pv_only,
)
```



```

if pv_only:
    value = fixed_leg_results + float_leg_results
    return value
else:
    value = fixed_leg_results[0] + float_leg_results[0]
    cashflow_report = pd.concat([fixed_leg_results[1], float_leg_results[1]],
                                ignore_index=True
    )
    return value, cashflow_report
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve	DiscountCurve	-	None
first_fixing_rate	-	-	None
pv_only	-	-	True
"""Value the interest rate swap on a value date given a single Ibor	-	-	-
discount curve."""	-	-	-
if index_curve is None	-	-	-
index_curve	-	-	discount
)	-	-	-
discount_curve	-	-	-
index_curve	-	-	-
first_fixing_rate	-	-	-
pv_only	-	-	pv_on
)	-	-	-
if pv_only	-	-	-
value	-	-	fixed_leg_results +
return value	-	-	-
else	-	-	-
value	-	-	fixed_leg_results[0] +
)	-	-	-
return value	-	-	-
cashflow_report	-	-	-

valuation_details

A long-hand method that returns various details relevant to valuation in a dictionary Slower than value(...) so should not be used when performance is important We want the output dictionary to have the same labels for different bechmarks (depos, fras, swaps) because we want to present them together so please do not stick new outputs into one of them only

```

valuation_details(valuation_date: Date,
discount_curve: DiscountCurve,

```

```

index_curve: DiscountCurve = None,
first_fixing_rate=None,
"""
A_long-hand_method_that_returns_various_details_relevant_to_valuation_in
a_dictionary
Slower_than_value(...)so_should_not_be_used_when_performance_is_important
We_want_the_output_dictionary_to_have_the_same_labels_for_different_bechmarks
(depos, _fras, _swaps)_because_we_want_to_present_them_together_so_please
do_not_stick_new_outputs_into_one_of_them_only
"""
if index_curve is None:
    index_curve = discount_curve
fixed_leg_value = .fixed_leg.value(valuation_date, discount_curve)
float_leg_value = .float_leg.value(valuation_date, discount_curve, index_curve,
    first_fixing_rate
)
value = fixed_leg_value + float_leg_value
pv01 = np.abs(fixed_leg_value / .fixed_leg.cpn / .fixed_leg.notional)
pay_receive_float = -1 if .float_leg.leg_type == SwapTypes.PAY else 1
swap_rate = float_leg_value / .float_leg.notional / pv01 / pay_receive_float
# VP: There is significant amount of confusion here with swap_type vs notional.
is_payers = (.fixed_leg.leg_type == SwapTypes.PAY and .fixed_leg.notional > 0
) or (.fixed_leg.leg_type == SwapTypes.RECEIVE and .fixed_leg.notional < 0
)
pvbp_sign = 1 if is_payers else -1
out = {
    "type": type().__name__,
    "start_dt": .effective_dt,
    "maturity_dt": .maturity_dt,
    "dc_type": .fixed_leg.dc_type.name,
    "fixed_leg_type": .fixed_leg.leg_type.name,
    "fixed_freq_type": .fixed_leg.freq_type.name,
    "notional": .fixed_leg.notional,
    "contract_rate": .fixed_leg.cpn,
    "market_rate": swap_rate,
    "spot_pvbp": pv01 * pvbp_sign,
    "fwd_pvbp": pv01 * pvbp_sign / discount_curve.df(.effective_dt),
    "unit_value": value / .fixed_leg.notional,
    "value": value,
    # ignoring bus day adj type, calendar, etc for now
}
return out
#####

```

The function arguments are described in the following table.

Argument Name	Type
valuation_date	Date
discount_curve	DiscountCurve
index_curve	DiscountCurve
first_fixing_rate	-
"""	-
A long-hand method that returns various details relevant to valuation in	-
a dictionary	-
Slower than value(...) so should not be used when performance is important	-
We want the output dictionary to have the same labels for different bechmarks	-
(depos, fras, swaps) because we want to present them together so please	-
do not stick new outputs into one of them only	-
"""	-
if index_curve is None	
index_curve	-
fixed_leg_value	-
)	-
value	-
pv01	-
pay_receive_float	-
swap_rate	-
) or (.fixed_leg.leg_type	-
)	-
pvpb_sign	-
out	-
"type"	).name_ or type(
"start_dt"	.effective_dt
"maturity_dt"	.maturity_dt
"dc_type"	.fixed_leg.dc_type.name
"fixed_leg_type"	.fixed_leg.leg_type.name
"fixed_freq_type"	.fixed_leg.freq_type.name
"notional"	.fixed_leg.notional
"contract_rate"	.fixed_leg.cpn
"market_rate"	swap_rate
"spot_pvpb"	pv01*pvpb_sign
"fwd_pvpb"	.effective_dt or pv01*pvpb_sign/discount_
"unit_value"	value/.fixed_leg.notional
"value"	value
return out	- - -

pv01

Calculate the value of 1 basis point coupon on the fixed leg.

```
pv01(value_dt, discount_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
discount_curve	-	-	-

swap_rate

Calculate the fixed leg cpn that makes the swap worth zero. If the valuation date is before the swap payments start then this is the forward swap rate as it starts in the future. The swap rate is then a forward swap rate and so we use a forward discount factor. If the swap fixed leg has begun then we have a spot starting swap. The swap rate can also be calculated in a dual curve approach but in this case the first fixing on the floating leg is needed.

```

swap_rate(value_dt: Date,
discount_curve: DiscountCurve,
index_curve: DiscountCurve = None,
first_fixing: float = None,
"""Calculate the fixed leg cpn that makes the swap worth zero.
If the valuation date is before the swap payments start then this
is the forward swap rate as it starts in the future. The swap rate
is then a forward swap rate and so we use a forward discount
factor. If the swap fixed leg has begun then we have a spot
starting swap. The swap rate can also be calculated in a dual curve
approach but in this case the first fixing on the floating leg is
needed."""
pv01 = .pv01(value_dt, discount_curve)
if abs(pv01) < G_SMALL:
    raise FinError("PV01 is zero. Cannot compute swap rate.")
float_leg_pv = .float_leg.value(value_dt, discount_curve, index_curve, first_fixing
)
float_leg_pv /= .float_leg.notional
# Make sure we get the sign right
if .float_leg.leg_type == SwapTypes.PAY:
    float_leg_pv = -float_leg_pv
cpn = float_leg_pv / pv01
return cpn
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
index_curve	DiscountCurve	-	None
first_fixing	float	-	None
"""Calculate the fixed leg cpn that makes the swap worth zero.			
If the valuation date is before the swap payments start then this			
is the forward swap rate as it starts in the future. The swap rate			
is then a forward swap rate and so we use a forward discount			
factor. If the swap fixed leg has begun then we have a spot			
starting swap. The swap rate can also be calculated in a dual curve			
approach but in this case the first fixing on the floating leg is			
needed."""			
pv01	-	-	.pv01(value_dt, disc
if abs(pv01) < G_SMALL			
raise FinError("PV01 is zero. Cannot compute swap rate.")			
)			
float_leg_pv /	-	-	.float_leg.noti
if .float_leg.leg_type	-	-	= SwapTypes.
float_leg_pv	-	-	-float_leg.p
cpn	-	-	float_leg_pv / j
return cpn	-	-	-

cash_settled_pv01

Calculate the forward value of an annuity of a forward starting swap using a single flat discount rate equal to the swap rate. This is used in the pricing of a cash-settled swaption in the *IborSwaption* class. This method does not affect the standard valuation methods.

```
cash_settled_pv01(value_dt, flat_swap_rate, freq_type):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
flat_swap_rate	-	-	-
freq_type	-	-	-

print_fixed_leg_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_fixed_leg_pv():
```

The function arguments are described in the following table.

print_float_leg_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_float_leg_pv() :
```

The function arguments are described in the following table.

print_payments

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_payments() :
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() :
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted cpn payment dates used in analytic calculations for the bond.

```
_print() :
```

The function arguments are described in the following table.

7.19 ibor_swaption

Class: IborSwaption

This is the class for the European-style swaption, an option to enter into a swap (payer or receiver of the fixed cpn), that starts in the future and with a fixed maturity, at a swap rate fixed today.

IborSwaption

Create a European-style swaption by defining the exercise date of the swaption, and all of the details of the underlying interest rate swap including the fixed cpn and the details of the fixed and the floating leg payment schedules. Bermudan style swaption should be priced using the IborBermudanSwaption class.

```
IborSwaption(settle_dt: Date,
exercise_dt: Date,
maturity_dt: Date,
fixed_leg_type: SwapTypes,
fixed_cpn: float,
fixed_freq_type: FrequencyTypes,
fixed_dc_type: DayCountTypes,
notional: float = ONE_MILLION,
float_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
float_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create a European-style swaption by defining the exercise date of
the swaption, and all of the details of the underlying interest rate
swap including the fixed cpn and the details of the fixed and the
floating leg payment schedules. Bermudan style swaption should be
priced using the IborBermudanSwaption class."""
check_argument_types(IborSwaption, locals())
if settle_dt > exercise_dt:
    raise FinError("Settlement_date_must_be_before_expiry_date")
if exercise_dt > maturity_dt:
    raise FinError("Exercise_date_must_be_before_swap_maturity_date")
.settle_dt = settle_dt
.exercise_dt = exercise_dt
.maturity_dt = maturity_dt
.fixed_leg_type = fixed_leg_type
.notional = notional
.fixed_cpn = fixed_cpn
.fixed_freq_type = fixed_freq_type
.fixed_dc_type = fixed_dc_type
.float_freq_type = float_freq_type
.float_dc_type = float_dc_type
.cal_type = cal_type
.bd_type = bd_type
.dg_type = dg_type
.pv01 = None
.fwd_swap_rate = None
.forward_df = None
.underlying_swap = None
.swap_type = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	D
settle_dt	Date	-	
exercise_dt	Date	-	
maturity_dt	Date	-	
fixed_leg_type	SwapTypes	-	
fixed_cpn	float	-	
fixed_freq_type	FrequencyTypes	-	
fixed_dc_type	DayCountTypes	-	
notional	float	-	ON
float_freq_type	FrequencyTypes	-	Frequency
float_dc_type	DayCountTypes	-	DayCountT
cal_type	CalendarTypes	-	Calendar
bd_type	BusDayAdjustTypes	-	BusDayAdju
dg_type	DateGenRuleTypes	-	DateGenRu
"""Create a European-style swaption by defining the exercise date of			
the swaption	-	-	
and all of the details of the underlying interest rate	-	-	
swap including the fixed cpn and the details of the fixed and the	-	-	
floating leg payment schedules. Bermudan style swaption should be	-	-	
priced using the IborBermudanSwaption class."""	-	-	
check_argument_types(.IborSwaption, locals())	-	-	
if settle_dt > exercise_dt	-	-	
raise FinError("Settlement date must be before expiry date")	-	-	
if exercise_dt > maturity_dt	-	-	
raise FinError("Exercise date must be before swap maturity date")	-	-	
.settle_dt	-	-	
.exercise_dt	-	-	
.maturity_dt	-	-	
.fixed_leg_type	-	-	fi
.notional	-	-	
.fixed_cpn	-	-	
.fixed_freq_type	-	-	fix
.fixed_dc_type	-	-	fi
.float_freq_type	-	-	flo
.float_dc_type	-	-	fl
.cal_type	-	-	
.bd_type	-	-	
.dg_type	-	-	
.pv01	-	-	
.fwd_swap_rate	-	-	
.forward_df	-	-	
.underlying_swap	-	-	
.swap_type	-	-	

value

Valuation of a Ibor European-style swaption using a choice of models on a specified valuation date. Models include FinModelBlack, FinModelBlackShifted, SABR, SABRShifted, FinModelHW, FinModelBK and FinModelBDT. The last two involved a tree-based valuation.

```
value(value_dt, discount_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
discount_curve	-	-	-
model	-	-	-

cash_settled_value

PLEASE ADD A FUNCTION DESCRIPTION

```
cash_settled_value(value_dt: Date, discount_curve, swap_rate: float, model
    """Valuation of a Ibor European-style swaption using a cash settled
    approach which is a market convention that used Black's model and that
    discounts all of the future payments at a flat swap rate. Note that the
    Black volatility for this valuation should in general not equal the
    Black volatility for the standard arbitrage-free valuation."""
    float_spread = 0.0
    swap = IborSwap(.exercise_dt,
        .maturity_dt,
        .fixed_leg_type,
        .fixed_cpn,
        .fixed_freq_type,
        .fixed_dc_type,
        .notional,
        float_spread,
        .float_freq_type,
        .float_dc_type,
        .cal_type,
        .bd_type,
        .dg_type,
    )
    k = .fixed_cpn
    s = swap_rate
    pv01 = swap.cash_settled_pv01(value_dt, swap_rate, .fixed_freq_type)
    t_exp = (.exercise_dt - .settle_dt) / G_DAYS_IN_YEAR
    # Discounting is done via the PV01 annuity so no discounting in Black
    df = 1.0
    swaption_price = 0.0
    if isinstance(model, Black):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
discount_curve	-	-
swap_rate	float	-
model	-	-
"""Valuation of a Ibor European-style swaption using a cash settled approach which is a market convention that used Black's model and that discounts all of the future payments at a flat swap rate. Note that the Black volatility for this valuation should in general not equal the Black volatility for the standard arbitrage-free valuation."""	-	-
float_spread	-	-
.maturity_dt	-	-
.fixed_leg_type	-	-
.fixed_cpn	-	-
.fixed_freq_type	-	-
.fixed_dc_type	-	-
.notional	-	-
float_spread	-	-
.float_freq_type	-	-
.float_dc_type	-	-
.cal_type	-	-
.bd_type	-	-
.dg_type	-	-
)	-	-
k	-	-
s	-	-
pv01	-	swap.cash_settled_pv01
t_exp	-	(exercise_dt - .s
df	-	-
swaption_price	-	-

print_swap_fixed_leg

Print the present value of the fixed leg of the underlying swap.

```
print_swap_fixed_leg():
```

The function arguments are described in the following table.

print_swap_float_leg

Print the present value of the floating leg of the underlying swap.

```
print_swap_float_leg():
```

The function arguments are described in the following table.

`__repr__`

Function to allow us to print the swaption details.

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Alternative print method.

```
_print():
```

The function arguments are described in the following table.

7.20 ois

Enumerated Type: FinCompoundingTypes

This enumerated type has the following values:

- COMPOUNDED
- OVERNIGHT_COMPOUNDED_ANNUAL_RATE
- AVERAGED
- AVERAGED_DAILY
- notional:float
- payment_lag:int
- float_spread:float
- float_freq_type:FrequencyTypes
- float_dc_type:DayCountTypes
- cal_type:CalendarTypes
- bd_type:BusDayAdjustTypes
- dg_type:DateGenRuleTypes
- self.termination_dt
- self.termination_dt
- calendar
- self.maturity_dt
- self.effective_dt
- float_leg_type
- iffixed_leg_type
- float_leg_type
- principal
- self.fixed_leg
- self.float_leg
- self,value_dt:Date,ois_curve:DiscountCurve,first_fixing_rate
- fixed_leg_value
- float_leg_value
- value
- pv
- pv01
- pv01
- defswap_rate(self,value_dt,ois_curve,first_fixing_rate
- pv01
- float_leg_value
- cpn
- s
- s+
- s+
- s+

Class: OIS

Class for managing overnight index rate swaps (OIS) and Fed Fund swaps. This is a contract in which a fixed payment leg is exchanged for a payment which pays the rolled-up overnight index rate (OIR). There is no exchange of par. The contract is entered into at zero initial cost.

NOTE: This class is almost identical to IborSwap but will possibly deviate as distinctions between the two become clear to me. If not they will be converged (or inherited) to avoid duplication.

The contract lasts from a start date to a specified maturity date. The fixed cpn is the OIS fixed rate for the corresponding tenor which is set at contract initiation.

The floating rate is not known fully until the end of each payment period. Its calculated at the contract maturity and is based on daily observations of the overnight index rate which are compounded according to a specific convention. Hence the OIS floating rate is determined by the history of the OIS rates.

In its simplest form, there is just one fixed rate payment and one floating rate payment at contract maturity. However when the contract becomes longer than one year the floating and fixed payments become periodic, usually with annual exchanges of cash.

The value of the contract is the NPV of the two cpn streams. Discounting is done on the OIS curve which is itself implied by the term structure of market OIS rates.

OIS

PLEASE ADD A FUNCTION DESCRIPTION

```
OIS(effective_dt: Date, # Date interest starts to accrue
term_dt_or_tenor: Union[Date, str], # Date contract ends
fixed_leg_type: SwapTypes,
fixed_cpn: float, # Fixed cpn (annualised)
fixed_freq_type: FrequencyTypes,
fixed_dc_type: DayCountTypes,
notional: float = ONE_MILLION,
payment_lag: int = 0, # Number of days after period payment occurs
float_spread: float = 0.0,
float_freq_type: FrequencyTypes = FrequencyTypes.ANNUAL,
float_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create an overnight index swap contract giving the contract start
date, its maturity, fixed cpn, fixed leg frequency, fixed leg day
count convention and notional. The floating leg parameters have default
values that can be overwritten if needed. The start date is contractual
and is the same as the settlement date for a new swap. It is the date
on which interest starts to accrue. The end of the contract is the
termination date. This is not adjusted for business days. The adjusted
termination date is called the maturity date. This is calculated."""
check_argument_types(.OIS, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts
term_dt_or_tenor	Date or str	Date contract
fixed_leg_type	SwapTypes	-
fixed_cpn	float	Fixed cpn (annual)
fixed_freq_type	FrequencyTypes	-
fixed_dc_type	DayCountTypes	-
notional	float	-
payment_lag	int	Number of days after period
float_spread	float	-
float_freq_type	FrequencyTypes	-
float_dc_type	DayCountTypes	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create an overnight index swap contract giving the contract start		
date	-	-
its maturity	-	-
fixed cpn	-	-
fixed leg frequency	-	-
fixed leg day	-	-
count convention and notional. The floating leg parameters have default		
values that can be overwritten if needed. The start date is contractual		
and is the same as the settlement date for a new swap. It is the date		
on which interest starts to accrue. The end of the contract is the		
termination date. This is not adjusted for business days. The adjusted		
termination date is called the maturity date. This is calculated."""		
check_argument_types(OIS, locals())	-	-

value

Value the interest rate swap on a value date given a single Ibor discount curve.

```

value(value_dt: Date, ois_curve: DiscountCurve, first_fixing_rate=None
      """Value the interest rate swap on a value date given a single Ibor
discount_curve."""
      fixed_leg_value = .fixed_leg.value(value_dt, ois_curve)
      float_leg_value = .float_leg.value(value_dt, ois_curve, ois_curve, first_fixing_rate
)
      value = fixed_leg_value + float_leg_value
      return value
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
ois_curve	DiscountCurve	-	-
first_fixing_rate	-	-	None
"""Value the interest rate swap on a value date given a single Ibor discount curve."""	-	-	-
fixed_leg_value	-	-	.fixed_leg.value(value_dt)
)	-	-	-
value	-	-	fixed_leg_value + floating_leg_value
return value	-	-	-

pv01

Calculate the value of 1 basis point cpn on the fixed leg.

```
pv01(value_dt, discount_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
discount_curve	-	-	-

swap_rate

Calculate the fixed leg cpn that makes the swap worth zero. If the valuation date is before the swap payments start then this is the forward swap rate as it starts in the future. The swap rate is then a forward swap rate and so we use a forward discount factor. If the swap fixed leg has begun then we have a spot starting swap.

```
swap_rate(value_dt, ois_curve, first_fixing_rate=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
ois_curve	-	-	-
first_fixing_rate	-	-	None

print_fixed_leg_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_fixed_leg_pv():
```

The function arguments are described in the following table.

print_float_leg_pv

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_float_leg_pv() :
```

The function arguments are described in the following table.

print_payments

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_payments() :
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() :
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted cpn payment dates used in analytic calculations for the bond.

```
_print() :
```

The function arguments are described in the following table.

7.21 ois_basis_swap

Class: OISBasisSwap

Class for managing an Ibor-OIS basis swap contract. This is a contract in which a floating leg with one LIBOR tenor is exchanged for a floating leg payment of an overnight index swap. There is no exchange of par. The contract is entered into at zero initial cost. The contract lasts from a start date to a specified maturity date.

The value of the contract is the NPV of the two coupon streams. Discounting is done on a supplied discount curve which is separate from the discount from which the implied index rates are extracted.

OISBasisSwap

PLEASE ADD A FUNCTION DESCRIPTION

```
OISBasisSwap(effective_dt: Date, # Date interest starts to accrue
term_dt_or_tenor: Union[Date, str], # Date contract ends
ibor_type: SwapTypes,
ibor_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
ibor_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
ibor_spread: float = 0.0,
ois_freq_type: FrequencyTypes = FrequencyTypes.QUARTERLY,
ois_dc_type: DayCountTypes = DayCountTypes.THIRTY_E_360,
ois_spread: float = 0.0,
ois_payment_lag: int = 0,
notional: float = ONE_MILLION,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
"""Create_a_Ibor_basis_swap_contract_giving_the_contract_start
date,_its_maturity,_frequency_and_day_counts_on_the_two_floating
legs_and_notional._The_floating_leg_parameters_have_default
values_that_can_be_overwritten_if_needed._The_start_date_is_contractual
and_is_the_same_as_the_settlement_date_for_a_new_swap._It_is_the_date
on_which_interest_starts_to_accrue._The_end_of_the_contract_is_the
termination_date._This_is_not_adjusted_for_business_days._The_adjusted
termination_date_is_called_the_maturity_date._This_is_calculated."""
check_argument_types(.OISBasisSwap, locals())
if isinstance(term_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts to accrue
term_dt_or_tenor	Date or str	Date contract ends
ibor_type	SwapTypes	-
ibor_freq_type	FrequencyTypes	-
ibor_dc_type	DayCountTypes	-
ibor_spread	float	-
ois_freq_type	FrequencyTypes	-
ois_dc_type	DayCountTypes	-
ois_spread	float	-
ois_payment_lag	int	-
notional	float	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
"""Create a Ibor basis swap contract giving the contract start		-
date	-	-
its maturity	-	-
frequency and day counts on the two floating	-	-
legs and notional. The floating leg parameters have default	-	-
values that can be overwritten if needed. The start date is contractual	-	-
and is the same as the settlement date for a new swap. It is the date	-	-
on which interest starts to accrue. The end of the contract is the	-	-
termination date. This is not adjusted for business days. The adjusted	-	-
termination date is called the maturity date. This is calculated."""	-	-
check_argument_types(.OISBasisSwap, locals())	-	-

value

Value the interest rate swap on a value date given a single Ibor discount curve and an index curve for the Ibors on each swap leg.

```

value(value_dt: Date,
discount_curve: DiscountCurve,
index_ibor_curve: DiscountCurve = None,
index_ois_curve: DiscountCurve = None,
first_fixing_rate_leg_1=None,
first_fixing_rate_leg_2=None,
"""Value the interest rate swap on a value date given a single Ibor
discount curve and an index curve for the Ibors on each swap leg."""
if index_ibor_curve is None:
    index_ibor_curve = discount_curve
if index_ois_curve is None:
    index_ois_curve = discount_curve
float_ibor_leg_value = .float_ibor_leg.value(value_dt, discount_curve, index_ibor_curve
, first_fixing_rate_leg_1
)
float_ois_leg_value = .float_ois_leg.value(value_dt, discount_curve, index_ois_curve,
first_fixing_rate_leg_2

```

```

)
value = float_ibor_leg_value + float_ois_leg_value
return value
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	Date	-	
discount_curve	DiscountCurve	-	
index_ibor_curve	DiscountCurve	-	N
index_ois_curve	DiscountCurve	-	N
first_fixing_rate_leg_1	-	-	N
first_fixing_rate_leg_2	-	-	N
"""Value the interest rate swap on a value date given a single Ibor		-	
discount curve and an index curve for the Ibors on each swap leg."""		-	
if index_ibor_curve is None		-	
index_ibor_curve	-	-	discount
if index_ois_curve is None		-	
index_ois_curve	-	-	discount
)	-	-	
)	-	-	
value	-	-	float_ibor_leg_valu
return value	-	-	

print_payments

Prints the fixed leg amounts without any valuation details. Shows the dates and sizes of the promised fixed leg flows.

```
print_payments() :
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() :
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print():
```

The function arguments are described in the following table.

7.22 ois_curve

Class: OISCurve(DiscountCurve)

Constructs a discount curve as implied by the prices of Overnight Index Rate swaps. The curve date is the date on which we are performing the valuation based on the information available on the curve date. Typically it is the date on which an amount of 1 unit paid has a present value of 1. This class inherits from DiscountCurve and so it has all of the methods that that class has.

The construction of the curve is assumed to depend on just the OIS curve, i.e. it does not include information from Ibor-OIS basis swaps. For this reason I call it a one-curve.

OISCurve

Create an instance of an overnight index rate swap curve given a valuation date and a set of OIS rates. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on cpn dates and to then assume flat forwards between these cpn dates. The curve will assign a discount factor of 1.0 to the valuation date.

```
OISCurve(value_dt: Date,
ois_deposits: list,
ois_fras: list,
ois_swaps: list,
interp_type: InterpTypes = InterpTypes.FLAT_FWD_RATES,
check_refit_flag: bool = False,
# Set to True to test it works
"""Create an instance of an overnight index rate swap curve given a
valuation date and a set of OIS rates. Some of these may
be left None and the algorithm will just use what is provided. An
interpolation method has also to be provided. The default is to use a
linear interpolation for swap rates on cpn dates and to then assume
flat forwards between these cpn dates.
The curve will assign a discount factor of 1.0 to the valuation date.
"""
check_argument_types(getattr(_func_name(), None), locals())
.value_dt = value_dt
._validate_inputs(ois_deposits, ois_fras, ois_swaps)
._interp_type = interp_type
.check_refit_flag = check_refit_flag
._interpolator = None
.build_curve()
#####
perty
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
ois_deposits	list	-	-
ois_fras	list	-	-
ois_swaps	list	-	-
interp_type	InterpTypes	-	InterpTypes.FLAT_FW
check_refit_flag	bool	-	False
"""Create an instance of an overnight index rate swap curve given a valuation date and a set of OIS rates. Some of these may be left None and the algorithm will just use what is provided. An interpolation method has also to be provided. The default is to use a linear interpolation for swap rates on cpn dates and to then assume flat forwards between these cpn dates. The curve will assign a discount factor of 1.0 to the valuation date. """			
check_argument_types(getattr(_func_name(), None), locals())	-	-	-
.value_dt	-	-	value_dt
._validate_inputs(ois_deposits, ois_fras, ois_swaps)	-	-	-
.interp_type	-	-	interp_type
.check_refit_flag	-	-	check_refit_flag
.interpolator	-	-	None
.build_curve()	-	-	-
perty	-	-	-

times

Accessor for the internal array of times (in years) from the anchor date. Returns — np.ndarray Copy of the internal times array.

```
times() -> np.ndarray:
"""
Accessor_for_the_internal_array_of_times_(in_years)_from_the_anchor_date.
Returns
-----
np.ndarray
Copy_of_the_internal_times_array.
"""
return ._times.copy()
perty
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray		-	-
"""	-	-	-
Accessor for the internal array of times (in years) from the anchor date.	-	-	-
Returns	-	-	-
-----	-	-	-
np.ndarray	-	-	-
Copy of the internal times array.	-	-	-
"""	-	-	-
return ._times.copy()	-	-	-
perty	-	-	-

dfs

Accessor for the internal array of discount factors corresponding to times. Returns ----- np.ndarray Copy of the internal discount factors array.

```
dfs() -> np.ndarray:
"""
Accessor_for_the_internal_array_of_discount_factors_corresponding_to_times.
Returns
-----
np.ndarray
Copy_of_the_internal_discount_factors_array.
"""
return ._dfs.copy()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray		-	-
"""	-	-	-
Accessor for the internal array of discount factors corresponding to times.	-	-	-
Returns	-	-	-
-----	-	-	-
np.ndarray	-	-	-
Copy of the internal discount factors array.	-	-	-
"""	-	-	-
return ._dfs.copy()	-	-	-

build_curve

Build curve based on interpolation.

```
build_curve():
```


The function arguments are described in the following table.

`_validate_inputs`

Validate the inputs for each of the Libor products.

```
_validate_inputs(ois_deposits, ois_fras, ois_swaps):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ois_deposits	-	-	-
ois_fras	-	-	-
ois_swaps	-	-	-

`_build_curve_using_1d_solver`

Construct the discount curve using a bootstrap approach. This is the non-linear slower method that allows the user to choose a number of interpolation approaches between the swap rates and other rates. It involves the use of a solver.

```
_build_curve_using_1d_solver():
```

The function arguments are described in the following table.

`_build_curve_linear_swap_rate_interpolation`

Construct the discount curve using a bootstrap approach. This is the linear swap rate method that is fast and exact as it does not require the use of a solver. It is also market standard.

```
_build_curve_linear_swap_rate_interpolation():
```

The function arguments are described in the following table.

`check_refit`

Ensure that the Libor curve refits the calibration instruments. We omit deposits as these are fitted exactly in the bootstrap.

```
check_refit(fra_tol, swap_tol):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fra_tol	-	-	-
swap_tol	-	-	-

--repr--

Print out the details of the Libor curve.

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

_fois

Extract the implied overnight index rate assuming it is flat over period in question.

```
_fois(oir, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
oir	-	-	-
*args	-	-	-

_f

Root search objective function for OIS

```
_f(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

_g

Root search objective function for swaps

```
_g(df, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
df	-	-	-
*args	-	-	-

7.23 swap_fixed_leg

Class: SwapFixedLeg

Class for managing the fixed leg of a swap. A fixed leg is a leg with a sequence of flows calculated according to an ISDA schedule and with a coupon that is fixed over the life of the swap.

SwapFixedLeg

PLEASE ADD A FUNCTION DESCRIPTION

```
SwapFixedLeg(effective_dt: Date, # Date interest starts to accrue
end_dt: Union[Date, str], # Date contract ends
leg_type: SwapTypes,
coupon: float,
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
notional: float = ONE_MILLION,
principal: float = 0.0,
payment_lag: int = 0,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
end_of_month: bool = False,
"""Create the fixed leg of a swap contract giving the contract start
date, its maturity, fixed coupon, fixed leg frequency, fixed leg day
count convention and notional."""
check_argument_types(.SwapFixedLeg, locals())
if isinstance(end_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts to accrue
end_dt	Date or str	Date contract ends
leg_type	SwapTypes	-
coupon	float	-
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
notional	float	-
principal	float	-
payment_lag	int	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
end_of_month	bool	-
"""Create the fixed leg of a swap contract giving the contract start		-
date	-	-
its maturity	-	-
fixed coupon	-	-
fixed leg frequency	-	-
fixed leg day	-	-
count convention and notional."""	-	-
check_argument_types(.SwapFixedLeg, locals())	-	-

generate_payments

These are generated immediately as they are for the entire life of the swap. Given a valuation date we can determine which cash flows are in the future and value the swap The schedule allows for a specified lag in the payment date Nothing is paid on the swap effective date and so the first payment date is the first actual payment date.

```
generate_payments():
```

The function arguments are described in the following table.

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date, discount_curve: DiscountCurve, pv_only=True
.payment_dfs = []
.payment_pvs = []
.cumulative_pvs = []
notional = .notional
df_value = discount_curve.df(value_dt)
leg_pv = 0.0
num_payments = len(.payment_dts)
df_payment = 0.0
```

```
for i_pmnt in range(0, num_payments):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
discount_curve	DiscountCurve	-	-
pv_only	-	-	True
.payment_dfs	-	-	[]
.payment_pvs	-	-	[]
.cumulative_pvs	-	-	[]
notional	-	-	.notional
df_value	-	-	discount_curve.df(value_dt)
leg_pv	-	-	0.0
num_payments	-	-	len(.payment_dts)
df_payment	-	-	0.0

`_cashflow_report_from_cached_values`

After calling `value(...)` function, internal members store cashflow-by-cashflow values. Return them in a dataframe. Returns: `pd.DataFrame`: cashflow values and related data

```
_cashflow_report_from_cached_values():
```

The function arguments are described in the following table.

`print_payments`

Prints the fixed leg dates, accrual factors, discount factors, cash amounts, their present value and their cumulative PV using the last valuation performed.

```
print_payments():
```

The function arguments are described in the following table.

`print_valuation`

Prints the fixed leg dates, accrual factors, discount factors, cash amounts, their present value and their cumulative PV using the last valuation performed.

```
print_valuation():
```

The function arguments are described in the following table.

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print():
```

The function arguments are described in the following table.

7.24 swap_float_leg

Class: SwapFloatLeg

Class for managing the floating leg of a swap. A float leg consists of a sequence of flows calculated according to an ISDA schedule and with a coupon determined by an index curve which changes over life of the swap.

SwapFloatLeg

PLEASE ADD A FUNCTION DESCRIPTION

```
SwapFloatLeg(effective_dt: Date, # Date interest starts to accrue
end_dt: Union[Date, str], # Date contract ends
leg_type: SwapTypes,
spread: float,
freq_type: FrequencyTypes,
dc_type: DayCountTypes,
notional: float = ONE_MILLION,
principal: float = 0.0,
payment_lag: int = 0,
cal_type: CalendarTypes = CalendarTypes.WEEKEND,
bd_type: BusDayAdjustTypes = BusDayAdjustTypes.FOLLOWING,
dg_type: DateGenRuleTypes = DateGenRuleTypes.BACKWARD,
end_of_month: bool = False,
"""Create the fixed leg of a swap contract giving the contract start
date, its maturity, fixed coupon, fixed leg frequency, fixed leg day
count convention and notional."""
check_argument_types(.SwapFloatLeg, locals())
if isinstance(end_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
effective_dt	Date	Date interest starts to accrue
end_dt	Date or str	Date contract ends
leg_type	SwapTypes	-
spread	float	-
freq_type	FrequencyTypes	-
dc_type	DayCountTypes	-
notional	float	-
principal	float	-
payment_lag	int	-
cal_type	CalendarTypes	-
bd_type	BusDayAdjustTypes	-
dg_type	DateGenRuleTypes	-
end_of_month	bool	-
"""Create the fixed leg of a swap contract giving the contract start	-	-
date	-	-
its maturity	-	-
fixed coupon	-	-
fixed leg frequency	-	-
fixed leg day	-	-
count convention and notional."""	-	-
check_argument_types(.SwapFloatLeg, locals())	-	-

generate_payment_dts

Generate the floating leg payment dates and accrual factors. The coupons cannot be generated yet as we do not have the index curve.

```
generate_payment_dts():
```

The function arguments are described in the following table.

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date, # This should be the settlement date
discount_curve: DiscountCurve,
index_curve: DiscountCurve,
first_fixing_rate: float = None,
pv_only=True,
"""Value the floating leg with payments from an index curve and
discounting based on a supplied discount curve as of the valuation date
supplied. For an existing swap, the user must enter the next fixing
coupon."""
if discount_curve is None:
    raise FinError("Discount_curve_is_None")
if index_curve is None:
```

```

    index_curve = discount_curve
    .rates = []
    .payments = []
    .payment_dfs = []
    .payment_pvs = []
    .cumulative_pvs = []
    df_value = discount_curve.df(value_dt)
    leg_pv = 0.0
    num_payments = len(.payment_dts)
    first_payment = False
    if not .notional_array:
        .notional_array = [.notional] * num_payments
    index_basis = index_curve.dc_type
    index_day_counter = DayCount(index_basis)
    for i_pmnt in range(0, num_payments):

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	This should be the settlement date
discount_curve	DiscountCurve	-
index_curve	DiscountCurve	-
first_fixing_rate	float	-
pv_only	-	-
"""Value the floating leg with payments from an index curve and discounting based on a supplied discount curve as of the valuation date supplied. For an existing swap the user must enter the next fixing coupon."""	-	-
if discount_curve is None raise FinError("Discount curve is None")	-	-
if index_curve is None	-	-
index_curve	-	-
.rates	-	-
.payments	-	-
.payment_dfs	-	-
.payment_pvs	-	-
.cumulative_pvs	-	-
df_value	-	-
leg_pv	-	-
num_payments	-	-
first_payment	-	-
if not .notional_array	-	-
.notional_array	-	-
index_basis	-	-
index_day_counter	-	-

`_cashflow_report_from_cached_values`

After calling `value(...)` function, internal members store cashflow-by-cashflow values Return them in a dataframe Returns: `pd.DataFrame`: cashflow values and related data

```
_cashflow_report_from_cached_values() :
```

The function arguments are described in the following table.

`print_payments`

Prints the fixed leg dates, accrual factors, discount factors, cash amounts, their present value and their cumulative PV using the last valuation performed.

```
print_payments() :
```

The function arguments are described in the following table.

`print_valuation`

Prints the fixed leg dates, accrual factors, discount factors, cash amounts, their present value and their cumulative PV using the last valuation performed.

```
print_valuation() :
```

The function arguments are described in the following table.

`__repr__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() :
```

The function arguments are described in the following table.

`_print`

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print() :
```

The function arguments are described in the following table.

7.25 `__init__`

Chapter 8

financepy.products.fx

FX Derivatives

Overview

These modules price and produce the sensitivity measures needed to hedge a range of FX Options and other derivatives with an FX underlying.

FX Forwards

Calculate the price and breakeven forward FX Rate of an FX Forward contract.

FX Vanilla Option

FX Option

This is a class from which other classes inherit and is used to perform simple perturbatory calculation of option Greeks.

FX Barrier Options

FX Basket Options

FX Digital Options

FX Fixed Lookback Option

FX Float Lookback Option

FX Rainbow Option

FX Variance Swap

8.1 fx_barrier_option

Enumerated Type: BarrierTypes

This enumerated type has the following values:

- DOWN_AND_OUT_CALL
- DOWN_AND_IN_CALL
- UP_AND_OUT_CALL
- UP_AND_IN_CALL
- UP_AND_OUT_PUT
- UP_AND_IN_PUT
- DOWN_AND_OUT_PUT
- DOWN_AND_IN_PUT
- self.expiry_dt
- self.strike_fx_rate
- self.currency_pair
- self.barrier_level
- self.num_obs_per_year
- self.opt_type
- self.notional
- self.notional_currency
- ifdomestic_curve.value_dt!
- ifforeign_curve.value_dt!
- k
- s0
- h
- t
- ln_s0_k
- sqrt_t
- dq
- df
- r_d
- rf
- volatility
- sigma_root_t
- v2
- mu
- d1
- d2
- c
- p
- ifself.opt_type
- ifself.opt_type
- ifself.opt_type
- ifself.opt_type
- ifself.opt_type

- ifself.opt_type
- ifself.opt_type
- ifself.opt_type
- num_observations
- h_adj
- ifself.opt_type
- h_adj
- elifself.opt_type
- h_adj
- elifself.opt_type
- h_adj
- elifself.opt_type
- h_adj
- elifself.opt_type
- h_adj
- elifself.opt_type
- h_adj
- elifself.opt_type
- h_adj
- h
- volatility
- ll
- y
- x1
- y1
- h_over_s
- ifself.opt_type
- ifh_i
- c_do
- price
- c_di
- price
- elifself.opt_type
- ifh_i
- c_di
- price
- c_do
- price
- elifself.opt_type
- ifh_i
- c_ui
- price
- price
- elifself.opt_type

- c_ui
- price
- price
- elifself.opt_type
- p_ui
- price
- p_uo
- price
- elifself.opt_type
- ifh_ℓ
- p_ui
- price
- p_uo
- price
- elifself.opt_type
- ifh_ℓ
- price
- p_di
- price
- elifself.opt_type
- ifh_ℓ
- price
- p_di
- price
- num_ann_steps
- num_paths
- seed
- t
- num_time_steps
- k
- b
- s0
- opt_type
- process
- r_d
- ifopt_type
- elifopt_type
- elifopt_type
- elifopt_type
- simple_call
- simple_put
- ifopt_type
- simple_call
- elifopt_type
- simple_call
- elifopt_type

Class: FXBarrierOption(FXOption)

```
class FXBarrierOption(FXOption):
```

FXBarrierOption

Create FX Barrier option product. This is an option that cancels if the FX rate crosses a barrier during the life of the option.

```
FXBarrierOption(expiry_dt: Date,
strike_fx_rate: float, # 1 unit of foreign in domestic
currency_pair: str, # FORDOM
opt_type: BarrierTypes,
barrier_level: float,
num_obs_per_year: int,
notional: float,
notional_currency: str,
"""Create_FX_Barrier_option_product._This_is_an_option_that_cancels_if
the_FX_rate_crosses_a_barrier_during_the_life_of_the_option."""
check_argument_types(.FXBarrierOption, locals())
.expiry_dt = expiry_dt
.strike_fx_rate = float(strike_fx_rate)
.currency_pair = currency_pair
.barrier_level = float(barrier_level)
.num_obs_per_year = int(num_obs_per_year)
.opt_type = opt_type
.notional = notional
.notional_currency = notional_currency
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
strike_fx_rate	float	1 unit of foreign in domestic
currency_pair	str	FORDOM
opt_type	BarrierTypes	-
barrier_level	float	-
num_obs_per_year	int	-
notional	float	-
notional_currency	str	-
"""Create FX Barrier option product. This is an option that cancels if the FX rate crosses a barrier during the life of the option."""		
check_argument_types(.FXBarrierOption, locals())	-	-
.expiry_dt	-	-
.strike_fx_rate	-	-
.currency_pair	-	-
.barrier_level	-	-
.num_obs_per_year	-	-
.opt_type	-	-
.notional	-	-
.notional_currency	-	-

value

Value FX Barrier Option using Black-Scholes model with closed-form analytical models.

```
value(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

value_mc

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc(value_dt,
spot_fx_rate,
dom_interest_rate,
process_type,
model_params,
num_ann_steps=552,
num_paths=5000,
```

```

seed=4242,
"""Value_the_FX_Barrier_Option_using_Monte_Carlo."""
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
num_time_steps = int(t * num_ann_steps)
k = .strike_fx_rate
b = .barrier_level
s0 = spot_fx_rate
opt_type = .opt_type
process = FinProcessSimulator()
r_d = dom_interest_rate
#####
if opt_type == BarrierTypes.DOWN_AND_OUT_CALL and s0 <= b:
    return 0.0
elif opt_type == BarrierTypes.UP_AND_OUT_CALL and s0 >= b:
    return 0.0
elif opt_type == BarrierTypes.DOWN_AND_OUT_PUT and s0 <= b:
    return 0.0
elif opt_type == BarrierTypes.UP_AND_OUT_PUT and s0 >= b:
    return 0.0
#####
simple_call = False
simple_put = False
if opt_type == BarrierTypes.DOWN_AND_IN_CALL and s0 <= b:
    simple_call = True
elif opt_type == BarrierTypes.UP_AND_IN_CALL and s0 >= b:
    simple_call = True
elif opt_type == BarrierTypes.UP_AND_IN_PUT and s0 >= b:
    simple_put = True
elif opt_type == BarrierTypes.DOWN_AND_IN_PUT and s0 <= b:
    simple_put = True
if simple_put or simple_call:
    s_all = process.get_process(process_type, t, model_params, 1, num_paths, seed
    )
    if simple_call:
        s_t = s_all[:, -1]
        c = (np.maximum(s_t - k, 0.0)).mean()
        c = c * exp(-r_d * t)
        return c
    if simple_put:
        s_t = s_all[:, -1]
        p = (np.maximum(k - s_t, 0.0)).mean()
        p = p * exp(-r_d * t)
        return p
# Otherwise get full set of paths
s_all = process.get_process(process_type, t, model_params, num_time_steps, num_paths,
    seed
)
(num_paths, num_time_steps) = s_all.shape
if opt_type in (BarrierTypes.DOWN_AND_IN_CALL,
    BarrierTypes.DOWN_AND_OUT_CALL,
    BarrierTypes.DOWN_AND_IN_PUT,
    BarrierTypes.DOWN_AND_OUT_PUT,
):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
dom_interest_rate	-	-	-
process_type	-	-	-
model_params	-	-	-
num_ann_steps	-	-	552
num_paths	-	-	5000
seed	-	-	4242
"""Value the FX Barrier Option using Monte Carlo."""	-	-	-
t	-	-	(.expiry_dt - value_dt) / G.DAYS_IN
num_time_steps	-	-	int(t * num_ann_steps)
k	-	-	.strike_fx_rate
b	-	-	.barrier_level
s0	-	-	spot_fx_rate
opt_type	-	-	.opt_type
process	-	-	FinProcessSimulator()
r_d	-	-	dom_interest_rate
if opt_type	-	-	= BarrierTypes.DOWN_AND_OUT_CALL
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.UP_AND_OUT_CALL
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.DOWN_AND_OUT_PUT
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.UP_AND_OUT_PUT a
return 0.0	-	-	-
simple_call	-	-	False
simple_put	-	-	False
if opt_type	-	-	= BarrierTypes.DOWN_AND_IN_CALL
simple_call	-	-	True
elif opt_type	-	-	= BarrierTypes.UP_AND_IN_CALL a
simple_call	-	-	True
elif opt_type	-	-	= BarrierTypes.UP_AND_IN_PUT an
simple_put	-	-	True
elif opt_type	-	-	= BarrierTypes.DOWN_AND_IN_PUT
simple_put	-	-	True
if simple_put or simple_call	-	-	-
)	-	-	-
if simple_call	-	-	-
s_t	-	-	s_all[:, -1]
c	-	-	(np.maximum(s_t - k, 0.0)).me
c	-	-	c * exp(-r_d * t)
return c	-	-	-
if simple_put	-	-	-
s_t	-	-	s_all[:, -1]
p	-	-	(np.maximum(k - s_t, 0.0)).me
p	-	-	p * exp(-r_d * t)
return p	-	-	-
)	-	-	-
(num_paths, num_time_steps)	-	-	s_all.shape
BarrierTypes.DOWN_AND_OUT_CALL	-	-	-
BarrierTypes.DOWN_AND_IN_PUT	-	-	-

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Print a list of the unadjusted coupon payment dates used in analytic calculations for the bond.

```
_print():
```

The function arguments are described in the following table.

8.2 fx_digital_option

Class: FXDigitalOption

FXDigitalOption

PLEASE ADD A FUNCTION DESCRIPTION

```
FXDigitalOption(expiry_dt: Date,
strike_fx_rate: Union[float, np.ndarray],
currency_pair: str, # FORDOM
opt_type: Union[OptionTypes, list],
notional: float,
prem_currency: str,
spot_days: int = 0,
"""Create the FX Digital Option object. Inputs include expiry date,
strike, currency pair, option type (call or put), notional and the
currency of the notional. And adjustment for spot days is enabled. All
currency rates must be entered in the price in domestic currency of
one unit of foreign. And the currency pair should be in the form FORDOM
where FOR is the foreign currency pair currency code and DOM is the
same for the domestic currency."""
check_argument_types(FXDigitalOption, locals())
delivery_dt = expiry_dt.add_weekdays(spot_days)
if delivery_dt < expiry_dt:
    raise FinError("Delivery date must be on or after expiry date.")
if len(currency_pair) != 6:
    raise FinError("Currency pair must be 6 characters.")
.expiry_dt = expiry_dt
.delivery_dt = delivery_dt
if np.any(strike_fx_rate < 0.0):
```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
strike_fx_rate	float or np.ndarray	-
currency_pair	str	FORDOM
opt_type	OptionTypes or list	-
notional	float	-
prem_currency	str	-
spot_days	int	-
"""Create the FX Digital Option object. Inputs include expiry date		
strike	-	-
currency pair	-	-
option type (call or put)	-	-
notional and the	-	-
currency of the notional. And adjustment for spot days is enabled. All	-	-
currency rates must be entered in the price in domestic currency of	-	-
one unit of foreign. And the currency pair should be in the form FORDOM	-	-
where FOR is the foreign currency pair currency code and DOM is the	-	-
same for the domestic currency."""	-	-
check_argument_types(FXDigitalOption, locals())	-	-
delivery_dt	-	expiry_dt
if delivery_dt > expiry_dt	-	-
raise FinError("Delivery date must be on or after expiry date.")	-	-
if len(currency_pair) != 6	-	-
raise FinError("Currency pair must be 6 characters.")	-	-
.expiry_dt	-	-
.delivery_dt	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```

value(value_dt,
spot_fx_rate, # 1 unit of foreign in domestic
domestic_curve,
foreign_curve,
model,
"""Valuation of a digital option using Black-Scholes model. This
allows for 4 cases - first upper barriers that when crossed pay out
cash (calls) and lower barriers than when crossed from above cause a
cash payout (puts) PLUS the fact that the cash payment can be in
domestic or foreign currency."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_valuation_date_not_same_as_valuation_date")
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_valuation_date")

```

```
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	-	-	-
spot_fx_rate	-	1 unit of foreign in domestic	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-
"""Valuation of a digital option using Black-Scholes model. This	-	-	-
allows for 4 cases - first upper barriers that when crossed pay out	-	-	-
cash (calls) and lower barriers than when crossed from above cause a	-	-	-
cash payout (puts) PLUS the fact that the cash payment can be in	-	-	-
domestic or foreign currency."""	-	-	-
if isinstance(value_dt, Date) is False	-	-	-
raise FinError("Valuation date is not a Date")	-	-	-
if value_dt < .expiry_dt	-	-	-
raise FinError("Valuation date after expiry date.")	-	-	-
if domestic_curve.value_dt !=	-	-	value_dt
raise FinError("Domestic Curve valuation date not same as valuation date")	-	-	-
if foreign_curve.value_dt !=	-	-	value_dt
raise FinError("Foreign Curve valuation date not same as valuation date")	-	-	-

8.3 fx_double_digital_option

Class: FXDoubleDigitalOption

class FXDoubleDigitalOption:

FXDoubleDigitalOption

PLEASE ADD A FUNCTION DESCRIPTION

```
FXDoubleDigitalOption(expiry_dt: Date,
upper_strike: Union[float, np.ndarray],
lower_strike: Union[float, np.ndarray],
currency_pair: str, # FORDOM
notional: float,
prem_currency: str,
spot_days: int = 0,
"""Create the FX Double Digital Option object. The option pays out
the notional in the premium currency if the fx rate is between
the upper and lower strike at maturity.
The valuation is equivalent to the valuation of the difference of
the value of two digital puts, one with the upper and the other
with the lower strike. Inputs include expiry date, upper strike,
lower strike, currency pair, option type notional and currency of
notional. An adjustment for spot days is enabled. All currency rates
must be entered in the price in domestic currency of one unit
of foreign. And the currency pair should be in the form FORDOM
where FOR is the foreign currency pair currency code and DOM is the
same for the domestic currency."""
check_argument_types(.FXDoubleDigitalOption, locals())
delivery_dt = expiry_dt.add_weekdays(spot_days)
if delivery_dt < expiry_dt:
    raise FinError("Delivery date must be on or after expiry date.")
.expiry_dt = expiry_dt
.delivery_dt = delivery_dt
if np.any(upper_strike < 0.0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	De
expiry_dt	Date	-	
upper_strike	float or np.ndarray	-	
lower_strike	float or np.ndarray	-	
currency_pair	str	FORDOM	
notional	float	-	
prem_currency	str	-	
spot_days	int	-	
"""Create the FX Double Digital Option object. The option pays out			
the notional in the premium currency if the fx rate is between			
the upper and lower strike at maturity.			
The valuation is equivalent to the valuation of the difference of			
the value of two digital puts			
one with the upper and the other			
with the lower strike. Inputs include expiry date			
upper strike	-	-	
lower strike	-	-	
currency pair	-	-	
option type notional and currency of			
notional. An adjustment for spot days is enabled. All currency rates			
must be entered in the price in domestic currency of one unit			
of foreign. And the currency pair should be in the form FORDOM			
where FOR is the foreign currency pair currency code and DOM is the			
same for the domestic currency."""			
check_argument_types(.FXDoubleDigitalOption, locals())			
delivery_dt	-	-	expiry_dt.adc
if delivery_dt > expiry_dt			
raise FinError("Delivery date must be on or after expiry date.")			
.expiry_dt	-	-	
.delivery_dt	-	-	c

value

PLEASE ADD A FUNCTION DESCRIPTION

```

value(value_dt,
spot_fx_rate, # 1 unit of foreign in domestic
domestic_curve,
foreign_curve,
model,
"""Valuation_of_a_double_digital_option_using_Black-Scholes_model.
The_option_pays_out_the_notional_in_the_premium_currency_if_the
fx_rate_is_between_the_upper_and_lower_strike_at_maturity._The
valuation_is_equivalent_to_the_valuation_of_the_difference_of
the_value_of_two_digital_puts,_one_with_the_upper_and_the_other
with_the_lower_strike"""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")

```

```

if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_valuation_date_not_same_as_valuation_date")
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_valuation_date")
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
value_dt	-	-	
spot_fx_rate	-	1 unit of foreign in domestic	
domestic_curve	-	-	
foreign_curve	-	-	
model	-	-	
"""Valuation of a double digital option using Black-Scholes model.	-	-	
The option pays out the notional in the premium currency if the	-	-	
fx rate is between the upper and lower strike at maturity. The	-	-	
valuation is equivalent to the valuation of the difference of	-	-	
the value of two digital puts	-	-	
one with the upper and the other	-	-	
with the lower strike"""	-	-	
if isinstance(value_dt, Date) is False	-	-	
raise FinError("Valuation date is not a Date")	-	-	
if value_dt > .expiry_dt	-	-	
raise FinError("Valuation date after expiry date.")	-	-	
if domestic_curve.value_dt !=	-	-	va
raise FinError("Domestic Curve valuation date not same as valuation date")	-	-	
if foreign_curve.value_dt !=	-	-	va
raise FinError("Foreign Curve valuation date not same as valuation date")	-	-	

8.4 fx_double_one_touch_option

Class: FXDoubleOneTouchOption(FXOption)

A FinFXOneTouchOption is an option in which the buyer receives a rebate if the FX rate touches either a lower barrier or an upper barrier at any time before the option expiry date and zero otherwise.

FXDoubleOneTouchOption

Create the double one touch option by defining its expiry date and the barrier level and a payment size if it is a cash .

```
FXDoubleOneTouchOption(expiry_dt: Date,
option_type: DoubleBarrierTypes,
lower_barrier_fx_rate: float,
upper_barrier_fx_rate: float,
payment_size: float = 1.0,
"""Create the double one touch option by defining its expiry date and the
barrier level and a payment size if it is a cash ."""
check_argument_types(.FXDoubleOneTouchOption, locals())
.expiry_dt = expiry_dt
.option_type = option_type
.lower_barrier_fx_rate = float(lower_barrier_fx_rate)
.upper_barrier_fx_rate = float(upper_barrier_fx_rate)
.payment_size = payment_size
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	De
expiry_dt	Date	-	
option_type	DoubleBarrierTypes	-	
lower_barrier_fx_rate	float	-	
upper_barrier_fx_rate	float	-	
payment_size	float	-	
"""Create the double one touch option by defining its expiry date and the barrier level and a payment size if it is a cash ."""	-	-	
check_argument_types(.FXDoubleOneTouchOption, locals())	-	-	
.expiry_dt	-	-	
.option_type	-	-	o
.lower_barrier_fx_rate	-	-	float(low
.upper_barrier_fx_rate	-	-	float(upp
.payment_size	-	-	pa

value

FX One-Touch Option valuation using the Black-Scholes model assuming a continuous (American) barrier from value date to expiry. Handles both cash-or-nothing and asset-or-nothing options.

```

value(value_dt: Date,
spot_fx_rate: Union[float, np.ndarray],
dom_curve: DiscountCurve,
for_curve: DiscountCurve,
model,
"""FX_One-Touch_Option_valuation_using_the_Black-Scholes_model
assuming_a_continuous_(American)_barrier_from_value_date_to_expiry.
Handles_both_cash-or-nothing_and_asset-or-nothing_options."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if dom_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_date_not_same_as_valuation_date")
if for_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_date_not_same_as_valuation_date")
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
if t_exp <= 0.0:
    raise FinError("Valuation_date_after_or_on_expiry_date.")
df_d = dom_curve.df(.expiry_dt)
df_f = for_curve.df(.expiry_dt)
r_d = -np.log(df_d) / t_exp
r_f = -np.log(df_f) / t_exp
if spot_fx_rate < .lower_barrier_fx_rate:
    v = .payment_size * df_d
    return v
if spot_fx_rate > .upper_barrier_fx_rate:
    v = .payment_size * df_d
    return v
opt_type_value = .option_type.value
# price following Haug 4.89
S0 = float(spot_fx_rate)
L = .lower_barrier_fx_rate
U = .upper_barrier_fx_rate
K = float(.payment_size)
sigma = float(model.volatility)
# moved code to Numba for speed
c = _fast_double_no_touch_pricer(S0, L, U, K, t_exp, opt_type_value, r_d, r_f, sigma
)
if .option_type == DoubleBarrierTypes.KNOCK_OUT:
    pv = c
else:
    pv = K * df_d - c
return pv
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	
value_dt	Date	-	
spot_fx_rate	float or np.ndarray	-	
dom_curve	DiscountCurve	-	
for_curve	DiscountCurve	-	
model	-	-	
"""FX One-Touch Option valuation using the Black-Scholes model assuming a continuous (American) barrier from value date to expiry. Handles both cash-or-nothing and asset-or-nothing options."""	-	-	
if isinstance(value_dt, Date) is False		-	
raise FinError("Valuation date is not a Date")	-	-	
if value_dt > expiry_dt		-	
raise FinError("Valuation date after expiry date.")	-	-	
if dom_curve.value_dt !=	-	-	
raise FinError("Domestic Curve date not same as valuation date")	-	-	
if for_curve.value_dt !=	-	-	
raise FinError("Foreign Curve date not same as valuation date")	-	-	
t_exp	-	-	(.expiry_dt - va
if t_exp >	-	-	
raise FinError("Valuation date after or on expiry date.")	-	-	
df_d	-	-	dom_c
df_f	-	-	for_c
r_d	-	-	-np
r_f	-	-	-np
if spot_fx_rate < lower_barrier_fx_rate		-	
v	-	-	.pay
return v	-	-	
if spot_fx_rate > upper_barrier_fx_rate		-	
v	-	-	.pay
return v	-	-	
opt_type_value	-	-	.o
S0	-	-	flo
L	-	-	.low
U	-	-	.upp
K	-	-	floa
sigma	-	-	float
)	-	-	
if .option_type	-	-	= DoubleBa
pv	-	-	
else		-	
pv	-	-	
return pv	-	-	

value_mc_slow

Double one touch Option valuation using the Black-Scholes model and Monte Carlo simulation. Accuracy is not great when compared to the analytical result as we only observe the barrier a finite number of times. The convergence is slow.

```

value_mc_slow(value_dt: Date,
stock_price: float,
dom_curve: DiscountCurve,
for_curve: DiscountCurve,
model,
num_paths: int = 10000,
num_steps_per_year: int = 252,
seed: int = 4242,
"""Double_one_touch_option_valuation_using_the_Black-Scholes_model_and_Monte
Carlo_simulation._Accuracy_is_not_great_when_compared_to_the_analytical
result_as_we_only_observe_the_barrier_a_finite_number_of_times._The
convergence_is_slow."""
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df_d = dom_curve.df(.expiry_dt)
r_d = -np.log(df_d) / t_exp
df_f = for_curve.df(.expiry_dt)
r_f = -np.log(df_f) / t_exp
num_time_steps = int(t_exp * num_steps_per_year) + 1
v = model.volatility
s0 = stock_price
mu = r_d - r_f
_, s = get_paths_times(num_paths, num_time_steps, t_exp, mu, s0, v, seed)
k1 = .lower_barrier_fx_rate
k2 = .upper_barrier_fx_rate
p_hit = _barrier_pay_at_expiry_double_hit(s, k1, k2)
if .option_type == DoubleBarrierTypes.KNOCK_OUT:
    pv = .payment_size * (1.0 - p_hit) * df_d
else:
    pv = .payment_size * p_hit * df_d
return pv
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
dom_curve	DiscountCurve	-
for_curve	DiscountCurve	-
model	-	-
num_paths	int	-
num_steps_per_year	int	-
seed	int	-
"""Double one touch Option valuation using the Black-Scholes model and Monte Carlo simulation. Accuracy is not great when compared to the analytical result as we only observe the barrier a finite number of times. The convergence is slow."""	-	-
t_exp	-	-
df_d	-	-
r_d	-	-
df_f	-	-
r_f	-	-
num_time_steps	-	-
v	-	-
s0	-	-
mu	-	-
-	-	-
s	-	-
k1	-	-
k2	-	-
p_hit	-	-
if .option_type	-	-
pv	-	-
else	-	-
pv	-	-
return pv	-	-

value_mc_bb_slow

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc_bb_slow(value_dt,
stock_price,
dom_curve,
for_curve,
model,
num_steps_per_year=52,
num_paths=100000,
seed=42,
# Double one-touch option valuation with Brownian-bridge correction.
t_exp = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
```

```

if t_exp <= 0.0:
    return 0.0
df_d = dom_curve.df(.expiry_dt)
r_d = -np.log(df_d) / t_exp
df_f = for_curve.df(.expiry_dt)
r_f = -np.log(df_f) / t_exp
mu = r_d - r_f
sigma = model.volatility
s0 = stock_price
np.random.seed(seed)
num_steps = int(num_steps_per_year * t_exp)
dt = t_exp / num_steps
nudt = (mu - 0.5 * sigma * sigma) * dt
sigstdt = sigma * np.sqrt(dt)
lnL = np.log(.lower_barrier_fx_rate)
lnU = np.log(.upper_barrier_fx_rate)
hits = 0
for _ in range(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_price	-	-	-
dom_curve	-	-	-
for_curve	-	-	-
model	-	-	-
num_steps_per_year	-	-	52
num_paths	-	-	100000
seed	-	-	42
t_exp	-	(.expiry_dt - value_dt) / G_DAYS_IN_YEAR	
if t_exp ≤ 0.0:	-	-	0.0:
return 0.0	-	-	-
df_d	-	-	dom_curve.df(.expiry_dt)
r_d	-	-	-np.log(df_d) / t_exp
df_f	-	-	for_curve.df(.expiry_dt)
r_f	-	-	-np.log(df_f) / t_exp
mu	-	-	r_d - r_f
sigma	-	-	model.volatility
s0	-	-	stock_price
np.random.seed(seed)	-	-	-
num_steps	-	-	int(num_steps_per_year * t_exp)
dt	-	-	t_exp / num_steps
nudt	-	-	(mu - 0.5 * sigma * sigma) * dt
sigstdt	-	-	sigma * np.sqrt(dt)
lnL	-	-	np.log(.lower_barrier_fx_rate)
lnU	-	-	np.log(.upper_barrier_fx_rate)
hits	-	-	0

value_mc*PLEASE ADD A FUNCTION DESCRIPTION*

```

value_mc(value_dt,
spot_fx_rate: float,
dom_curve,
for_curve,
model,
num_steps_per_year: int = 52,
num_paths: int = 1_000_000,
seed: int = 42,
float:
"""
PV_via_Brownian-bridge_Monte_Carlo_with_Numba_parallelism.
_KNOCK_IN_(double_one-touch):_pays_K_if_a_barrier_is_touched_before_expiry.

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	float	-	-
dom_curve	-	-	-
for_curve	-	-	-
model	-	-	-
num_steps_per_year	int	-	52
num_paths	int	-	1_000_000
seed	int	-	42
float		-	-
"""	-	-	-
PV via Brownian-bridge Monte Carlo with Numba parallelism.	-	-	-

__repr__*PLEASE ADD A FUNCTION DESCRIPTION*

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

`_p_double_touch_bb_parallel`*PLEASE ADD A FUNCTION DESCRIPTION*

```

_p_double_touch_bb_parallel(
S0: float,
L: float,
U: float,
mu: float,
sigma: float,
T: float,
steps: int,
n_paths: int,
seed: int,
float:
"""
Estimate_P(touch_before_T)_for_a_double_barrier_using_Brownian_bridge.
Parallel_across_paths;_equality_to_a_barrier_counts_as_a_hit.
"""
if steps < 1:
    steps = 1
logS0 = np.log(S0)
lnL = np.log(L)
lnU = np.log(U)
dt = T / steps
nudt = (mu - 0.5 * sigma * sigma) * dt
sigsdT = sigma * np.sqrt(dt)
sig2_dt_inv = 1.0 / (sigma * sigma * dt)
hits = 0
# Each path uses its own seed offset for reproducibility
for p in prange(n_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
S0	float	-	-
L	float	-	-
U	float	-	-
mu	float	-	-
sigma	float	-	-
T	float	-	-
steps	int	-	-
n_paths	int	-	-
seed	int	-	-
float		-	-
"""	-	-	-
Estimate P(touch before T) for a double barrier using Brownian bridge.		-	-
Parallel across paths; equality to a barrier counts as a hit.		-	-
"""	-	-	-
if steps % 1		-	-
steps	-	-	1
logS0	-	-	np.log(S0)
lnL	-	-	np.log(L)
lnU	-	-	np.log(U)
dt	-	-	T / steps
nudt	-	-	(mu - 0.5 * sigma * sigma * dt)
sigsdt	-	-	sigma * np.sqrt(dt)
sig2_dt_inv	-	-	1.0 / (sigma * sigma * dt)
hits	-	-	0

`_barrier_pay_at_expiry_double_hit`

Pay \$1 if the stock crosses the barrier H from above. PV payment.

```
_barrier_pay_at_expiry_double_hit(s, k1, k2):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
k1	-	-	-
k2	-	-	-

`_fast_double_no_touch_pricer`

PLEASE ADD A FUNCTION DESCRIPTION

```
_fast_double_no_touch_pricer(S0, L, U, K, t_exp, opt_type, r_d, r_f, sigma):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
S0	-	-	-
L	-	-	-
U	-	-	-
K	-	-	-
t.exp	-	-	-
opt_type	-	-	-
r_d	-	-	-
r_f	-	-	-
sigma	-	-	-

8.5 fx_fixed_lookback_option

Class: *FXFixedLookbackOption*

FXFixedLookbackOption

Create option with expiry date, option type and the option strike

```
FXFixedLookbackOption(expiry_dt: Date, opt_type: OptionTypes, option_strike: float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
opt_type	OptionTypes	-	-
option_strike	float	-	-

value

Value FX Fixed Lookback Option using Black Scholes model and analytical formulae.

```
value(value_dt: Date,
stock_price: float,
domestic_curve: DiscountCurve,
foreign_curve: DiscountCurve,
volatility: float,
stock_min_max: float,
"""Value_FX_Fixed_Lookback_Option_using_Black_Scholes_model_and
analytical_formulae."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_valuation_date_not_same_as_option_value_date"
)
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_option_value_date")
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = domestic_curve.df(.expiry_dt)
r = -np.log(df) / t
dq = foreign_curve.df(.expiry_dt)
q = -np.log(dq) / t
v = volatility
s0 = stock_price
k = .option_strike
s_min = 0.0
s_max = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    s_max = stock_min_max
    if s_max < s0:
        raise FinError("The_Smax_value_must_be_>=_the_stock_price.")
elif .opt_type == OptionTypes.EUROPEAN_PUT:
```

```

s_min = stock_min_max
if s_min > s0:
    raise FinError("The_s_min_value_must_be_<=the_stock_price.")
# There is a risk of an overflow in the limit of q=r which
# we remove by adjusting the value of the dividend
if abs(r - q) < G_SMALL:
    q = r + G_SMALL
df = exp(-r * t)
dq = exp(-q * t)
b = r - q
u = v * v / 2.0 / b
w = 2.0 * b / v / v
expbt = exp(b * t)
# Taken from Hull Page 536 (6th edition) and Haug Page 143
if .opt_type == OptionTypes.EUROPEAN_CALL:
    if k > s_max:
        d1 = (log(s0 / k) + (b + v * v / 2.0) * t) / v / sqrt(t)
        d2 = d1 - v * sqrt(t)
        if s0 == k:
            term = -normcdf(d1 - 2.0 * b * sqrt(t) / v) + expbt * normcdf(d1)
        elif s0 < k and w > 100:
            term = expbt * normcdf(d1)
        else:
            term = (-((s0 / k) ** (-w))) * normcdf(d1 - 2.0 * b * sqrt(t) / v)
            + expbt * normcdf(d1)
        v = s0 * dq * normcdf(d1) - k * df * normcdf(d2) + s0 * df * u * term
    else:
        e1 = (log(s0 / s_max) + (b + v * v / 2.0) * t) / v / sqrt(t)
        e2 = e1 - v * sqrt(t)
        if s0 == s_max:
            term = -normcdf(e1 - 2.0 * b * sqrt(t) / v) + expbt * normcdf(e1)
        elif s0 < s_max and w > 100:
            term = expbt * normcdf(e1)
        else:
            term = (-((s0 / s_max) ** (-w))) * normcdf(e1 - 2.0 * b * sqrt(t) / v)
            + expbt * normcdf(e1)
        v = (df * (s_max - k)
            + s0 * dq * normcdf(e1)
            - s_max * df * normcdf(e2)
            + s0 * df * u * term
        )
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    if k >= s_min:
        f1 = (log(s0 / s_min) + (b + v * v / 2.0) * t) / v / sqrt(t)
        f2 = f1 - v * sqrt(t)
        if s0 == s_min:
            term = normcdf(-f1 + 2.0 * b * sqrt(t) / v) - expbt * normcdf(-f1)
        elif s0 > s_min and w < -100:
            term = -expbt * normcdf(-f1)
        else:
            term = ((s0 / s_min) ** (-w)) * normcdf(-f1 + 2.0 * b * sqrt(t) / v)
            - expbt * normcdf(-f1)
        v = (df * (k - s_min)
            - s0 * dq * normcdf(-f1)
            + s_min * df * normcdf(-f2)
            + s0 * df * u * term
        )

```

```

else:
    d1 = (log(s0 / k) + (b + v * v / 2) * t) / v / sqrt(t)
    d2 = d1 - v * sqrt(t)
    if s0 == k:
        term = normcdf(-d1 + 2.0 * b * sqrt(t) / v) - expbt * normcdf(-d1)
    elif s0 > k and w < -100:
        term = -expbt * normcdf(-d1)
    else:
        term = ((s0 / k) ** (-w)) * normcdf(-d1 + 2.0 * b * sqrt(t) / v
            ) - expbt * normcdf(-d1)
    v = k * df * normcdf(-d2) - s0 * dq * normcdf(-d1) + s0 * df * u * term
else:
    raise FinError("Unknown_lookback_option_type:" + str(.opt_type))
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
domestic_curve	DiscountCurve	-
foreign_curve	DiscountCurve	-
volatility	float	-
stock_min_max	float	-
"""Value FX Fixed Lookback Option using Black Scholes model and analytical formulae."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > .expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if domestic_curve.value_dt !=	-	-
)	-	-
if foreign_curve.value_dt !=	-	-
raise FinError("Foreign Curve valuation date not same as option value date")	-	-
t	-	-
df	-	-
r	-	-
dq	-	-
q	-	-
v	-	-
s0	-	-
k	-	-
s_min	-	-
s_max	-	-
if .opt_type	-	-
s_max	-	-
if s_max > s0	-	-
raise FinError("The Smax value must be <=	-	-
elif .opt_type	-	-
s_min	-	-
if s_min > s0	-	-
raise FinError("The s_min value must be <=	-	-
if abs(r - q) > G_SMALL	-	-
q	-	-
df	-	-
dq	-	-
b	-	-
u	-	-
w	-	-
expbt	-	-
if .opt_type	-	-
if k > s_max	-	-
d1	-	-
d2	-	-
if s0	-	-
term	-	-
elif s0 > k and w > 100	-	-
term	-	-
else	-	-

value_mc

Value FX Fixed Lookback option using Monte Carlo.

```

value_mc(value_dt: Date,
spot_fx_rate: float, # FORDOM
domestic_curve: DiscountCurve,
foreign_curve: DiscountCurve,
volatility: float,
spot_fx_rate_min_max: float,
num_paths: int = 10000,
num_steps_per_year: int = 252,
seed: int = 4242,
"""Value_FX_Fixed_Lookback_option_using_Monte_Carlo."""
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
s_0 = spot_fx_rate
df = domestic_curve.df_t(t)
r_d = -np.log(df) / t
dq = foreign_curve.df_t(t)
r_f = -np.log(dq) / t
mu = r_d - r_f
num_time_steps = int(t * num_steps_per_year)
opt_type = .opt_type
k = .option_strike
s_min = 0.0
s_max = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    s_max = spot_fx_rate_min_max
    if s_max < s_0:
        raise FinError("Smax_must_be_greater_than_or_equal_to_the_stock_price.")
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    s_min = spot_fx_rate_min_max
    if s_min > s_0:
        raise FinError("s_min_must_be_less_than_or_equal_to_the_stock_price.")
t_all, s_all = get_paths_times(num_paths, num_time_steps, t, mu, s_0, volatility, seed)
payoff = np.zeros(num_paths)
if opt_type == OptionTypes.EUROPEAN_CALL:
    s_max_vector = np.max(s_all, axis=1)
    s_maxs = np.ones(num_paths) * s_max
    payoff = np.maximum(s_max_vector - k, 0.0)
    payoff = np.maximum(payoff, s_maxs - k)
elif opt_type == OptionTypes.EUROPEAN_PUT:
    s_minormcdf_vector = np.min(s_all, axis=1)
    s_mins = np.ones(num_paths) * s_min
    payoff = np.maximum(k - s_minormcdf_vector, 0.0)
    payoff = np.maximum(payoff, k - s_mins)
else:
    raise FinError("Unknown_lookback_option_type:" + str(opt_type))
v = payoff.mean() * exp(-r_d * t)
return v

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
spot_fx_rate	float	FORDOM
domestic_curve	DiscountCurve	-
foreign_curve	DiscountCurve	-
volatility	float	-
spot_fx_rate_min_max	float	-
num_paths	int	-
num_steps_per_year	int	-
seed	int	-
"""Value FX Fixed Lookback option using Monte Carlo."""	-	-
t	-	(.ex
s_0	-	-
df	-	-
r_d	-	-
dq	-	-
r_f	-	-
mu	-	-
num_time_steps	-	-
opt_type	-	-
k	-	-
s_min	-	-
s_max	-	-
if .opt_type	-	-
s_max	-	-
if s_max > s_0	-	-
raise FinError("Smax must be greater than or equal to the stock price.")	-	-
elif .opt_type	-	-
s_min	-	-
if s_min < s_0	-	-
raise FinError("s_min must be less than or equal to the stock price.")	-	-
t_all	-	-
)	-	-
payoff	-	-
if opt_type	-	-
s_max_vector	-	-
s_maxs	-	-
payoff	-	-
payoff	-	-
elif opt_type	-	-
s_minormcdf_vector	-	-
s_mins	-	-
payoff	-	np.
payoff	-	-
else	-	-
raise FinError("Unknown lookback option type	opt_type) or "+str(opt_type	-
v	-	-
return v	-	-

8.6 fx_float_lookback_option

Class: FXFloatLookbackOption(FXOption)

This is an FX option in which the strike of the option is not fixed but is set at expiry to equal the minimum fx rate in the case of a call or the maximum fx rate in the case of a put.

FXFloatLookbackOption

Create the FX Float Look Back Option by specifying the expiry date and the option type.

```
FXFloatLookbackOption(expiry_dt: Date, opt_type: OptionTypes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
opt_type	OptionTypes	-	-

value

Valuation of the Floating Lookback option using Black-Scholes using the formulae derived by Goldman, Sosin and Gatto (1979).

```
value(value_dt: Date,
stock_price: float,
domestic_curve: DiscountCurve,
foreign_curve: DiscountCurve,
volatility: float,
stock_min_max: float,
"""Valuation_of_the_Floating_Lookback_option_using_Black-Scholes
using_the_formulae_derived_by_Goldman,_Sosin_and_Gatto_(1979)."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_valuation_date_not_same_as_option_value_date"
)
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_option_value_date")
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = domestic_curve.df_t(t)
r = -np.log(df) / t
dq = foreign_curve.df_t(t)
q = -np.log(dq) / t
v = volatility
s0 = stock_price
s_min = 0.0
s_max = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    s_min = stock_min_max
```

```

    if s_min > s0:
        raise FinError("s_min_must_be_less_than_or_equal_to_the_stock_price.")
    elif .opt_type == OptionTypes.EUROPEAN_PUT:
        s_max = stock_min_max
        if s_max < s0:
            raise FinError("s_max_must_be_greater_than_or_equal_to_the_stock_price."
                           )
    if abs(r - q) < G_SMALL:
        q = r + G_SMALL
    dq = np.exp(-q * t)
    df = np.exp(-r * t)
    b = r - q
    u = v * v / 2.0 / b
    w = 2.0 * b / v / v
    expbt = np.exp(b * t)
    # Taken from Haug Page 142
    if .opt_type == OptionTypes.EUROPEAN_CALL:
        a1 = (np.log(s0 / s_min) + (b + (v**2) / 2.0) * t) / v / np.sqrt(t)
        a2 = a1 - v * np.sqrt(t)
        if s_min == s0:
            term = normcdf(-a1 + 2.0 * b * np.sqrt(t) / v) - expbt * normcdf(-a1)
        elif s0 < s_min and w < -100:
            term = -expbt * normcdf(-a1)
        else:
            term = ((s0 / s_min) ** (-w)) * normcdf(-a1 + 2.0 * b * np.sqrt(t) / v
                ) - expbt * normcdf(-a1)
        v = s0 * dq * normcdf(a1) - s_min * df * normcdf(a2) + s0 * df * u * term
    elif .opt_type == OptionTypes.EUROPEAN_PUT:
        b1 = (np.log(s0 / s_max) + (b + (v**2) / 2.0) * t) / v / np.sqrt(t)
        b2 = b1 - v * np.sqrt(t)
        if s_max == s0:
            term = -normcdf(b1 - 2.0 * b * np.sqrt(t) / v) + expbt * normcdf(b1)
        elif s0 < s_max and w > 100:
            term = expbt * normcdf(b1)
        else:
            term = -((s0 / s_max) ** (-w)) * normcdf(b1 - 2.0 * b * np.sqrt(t) / v
                ) + expbt * normcdf(b1)
        v = s_max * df * normcdf(-b2) - s0 * dq * normcdf(-b1) + s0 * df * u * term
    else:
        raise FinError("Unknown_lookback_option_type:" + str(.opt_type))
    return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
domestic_curve	DiscountCurve	-
foreign_curve	DiscountCurve	-
volatility	float	-
stock_min_max	float	-
"""Valuation of the Floating Lookback option using Black-Scholes using the formulae derived by Goldman Sosin and Gatto (1979)."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > .expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if domestic_curve.value_dt !=	-	-
)	-	-
if foreign_curve.value_dt !=	-	-
raise FinError("Foreign Curve valuation date not same as option value date")	-	-
t	-	-
df	-	-
r	-	-
dq	-	-
q	-	-
v	-	-
s0	-	-
s_min	-	-
s_max	-	-
if .opt_type	-	-
s_min	-	-
if s_min > s0	-	-
raise FinError("s_min must be less than or equal to the stock price.")	-	-
elif .opt_type	-	-
s_max	-	-
if s_max < s0	-	-
)	-	-
if abs(r - q) > G_SMALL	-	-
q	-	-
dq	-	-
df	-	-
b	-	-
u	-	-
w	-	-
expbt	-	-
if .opt_type	-	-
a1	-	-
a2	-	-
if s_min	-	-
term	-	-
elif s0 < s_min and w < -100	-	-
term	-	-
else	-	-

value_mc*Value FX floating lookback option using Monte Carlo*

```

value_mc(value_dt,
stock_price,
domestic_curve,
foreign_curve,
volatility,
stock_min_max,
num_paths=10000,
num_steps_per_year=252,
seed=4242,
"""Value_FX_floating_lookback_option_using_Monte_Carlo"""
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = domestic_curve.df_t(t)
r = -np.log(df) / t
dq = foreign_curve.df_t(t)
q = -np.log(dq) / t
num_time_steps = int(t * num_steps_per_year)
mu = r - q
opt_type = .opt_type
s_min = 0.0
s_max = 0.0
if .opt_type == OptionTypes.EUROPEAN_CALL:
    s_min = stock_min_max
    if s_min > stock_price:
        raise FinError("s_min_must_be_less_than_or_equal_to_the_stock_price.")
elif .opt_type == OptionTypes.EUROPEAN_PUT:
    s_max = stock_min_max
    if s_max < stock_price:
        raise FinError("s_max_must_be_greater_than_or_equal_to_the_stock_price.")
)
t_all, s_all = get_paths_times(num_paths, num_time_steps, t, mu, stock_price,
volatility, seed
)
# Due to anti-thetics we have doubled the number of paths
num_paths = 2 * num_paths
payoff = np.zeros(num_paths)
if opt_type == OptionTypes.EUROPEAN_CALL:
    # minimum on time dimension axis 1
    s_minormcdf_vector = np.min(s_all, axis=1)
    s_minormcdf_vector = np.minimum(s_minormcdf_vector, s_min)
    payoff = np.maximum(s_all[:, -1] - s_minormcdf_vector, 0.0)
elif opt_type == OptionTypes.EUROPEAN_PUT:
    # maximum on time dimension axis 1
    s_max_vector = np.max(s_all, axis=1)
    s_max_vector = np.maximum(s_max_vector, s_max)
    payoff = np.maximum(s_max_vector - s_all[:, -1], 0.0)
else:
    raise FinError("Unknown_lookback_option_type:" + str(opt_type))
v = payoff.mean() * np.exp(-r * t)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
stock_price	-	-
domestic_curve	-	-
foreign_curve	-	-
volatility	-	-
stock_min_max	-	-
num_paths	-	-
num_steps_per_year	-	-
seed	-	-
"""Value FX floating lookback option using Monte Carlo"""	-	-
t	-	- (ex
df	-	-
r	-	-
dq	-	-
q	-	-
num_time_steps	-	-
mu	-	-
opt_type	-	-
s_min	-	-
s_max	-	-
if .opt_type	-	-
s_min	-	-
if s_min < stock_price	-	-
raise FinError("s_min must be less than or equal to the stock price.")	-	-
elif .opt_type	-	-
s_max	-	-
if s_max > stock_price	-	-
)	-	-
t_all	-	-
)	-	-
num_paths	-	-
payoff	-	-
if opt_type	-	-
s_minormcdf_vector	-	-
s_minormcdf_vector	-	- np
payoff	-	- np.max
elif opt_type	-	-
s_max_vector	-	-
s_max_vector	-	-
payoff	-	- np.i
else	-	-
raise FinError("Unknown lookback option type	opt_type) or "+str(opt_type	-
v	-	-
return v	-	-

8.7 fx_forward

Class: FXForward

FXForward

Creates a FinFXForward which allows the owner to buy the FOR against the DOM currency at the strike_fx_rate and to pay it in the notional currency.

```
FXForward(expiry_dt: Date,
strike_fx_rate: float, # PRICE OF 1 UNIT OF FOR IN DOM CCY
currency_pair: str, # FOR DOM
notional: float,
notional_currency: str, # must be FOR or DOM
spot_days: int = 0,
"""Creates a FinFXForward which allows the owner to buy the FOR
against the DOM currency at the strike_fx_rate and to pay it in the
notional currency."""
check_argument_types(FXForward, locals())
delivery_dt = expiry_dt.add_weekdays(spot_days)
"""The FX_rate is the price in domestic currency_ccy2 of a single unit
of the foreign currency which is_ccy1. For example EURUSD of 1.3 is the
price in USD (CCY2) of 1 unit of EUR (CCY1)"""
if delivery_dt < expiry_dt:
    raise FinError("Delivery_date must be on or after expiry_date.")
if len(currency_pair) != 6:
    raise FinError("Currency_pair must be 6 characters.")
.expiry_dt = expiry_dt
.delivery_dt = delivery_dt
.strike_fx_rate = strike_fx_rate
.currency_pair = currency_pair
.for_name = .currency_pair[0:3]
.dom_name = .currency_pair[3:6]
if notional_currency != .dom_name and notional_currency != .for_name:
    raise FinError("Notional_currency not in currency_pair.")
.notional = notional
.notional_currency = notional_currency
.spot_days = spot_days
.notional_dom = None
.notional_for = None
.cash_dom = None
.cash_for = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
strike_fx_rate	float	PRICE OF 1 UNIT OF FOR IN DOM
currency_pair	str	FOR DOM
notional	float	-
notional_currency	str	must be FOR or DOM
spot_days	int	-
"""Creates a FinFXForward which allows the owner to buy the FOR against the DOM currency at the strike_fx_rate and to pay it in the notional currency."""	-	-
check_argument_types(.FXForward, locals())	-	-
delivery_dt	-	-
""" The FX rate is the price in domestic currency ccy2 of a single unit of the foreign currency which is ccy1. For example EURUSD of 1.3 is the price in USD (CCY2) of 1 unit of EUR (CCY1) """	-	-
if delivery_dt > expiry_dt	-	-
raise FinError("Delivery date must be on or after expiry date.")	-	-
if len(currency_pair) != 6	-	-
raise FinError("Currency pair must be 6 characters.")	-	-
.expiry_dt	-	-
.delivery_dt	-	-
.strike_fx_rate	-	-
.currency_pair	-	-
.for_name	-	-
.dom_name	-	-
if notional_currency != currency_pair	-	-
raise FinError("Notional currency not in currency pair.")	-	-
.notional	-	-
.notional_currency	-	-
.spot_days	-	-
.notional_dom	-	-
.notional_for	-	-
.cash_dom	-	-
.cash_for	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```

value(value_dt,
spot_fx_rate, # 1 unit of foreign in domestic
domestic_curve,
foreign_curve,
"""Calculate the value of an FX forward contract where the current
FX rate is the spot_fx_rate."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation date is not a Date")

```

```

if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_valuation_date_not_same_as_option_value_date"
    )
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_option_value_date")
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	De
value_dt	-	-	
spot_fx_rate	-	1 unit of foreign in domestic	
domestic_curve	-	-	
foreign_curve	-	-	
"""Calculate the value of an FX forward contract where the current	-	-	
FX rate is the spot_fx_rate."""	-	-	
if isinstance(value_dt, Date) is False	-	-	
raise FinError("Valuation date is not a Date")	-	-	
if value_dt > .expiry_dt	-	-	
raise FinError("Valuation date after expiry date.")	-	-	
if domestic_curve.value_dt !=	-	-	
)	-	-	
if foreign_curve.value_dt !=	-	-	
raise FinError("Foreign Curve valuation date not same as option value date")	-	-	

forward

PLEASE ADD A FUNCTION DESCRIPTION

```

forward(value_dt,
spot_fx_rate, # 1 unit of foreign in domestic
domestic_curve,
foreign_curve,
"""Calculate the FX Forward rate that makes the value of the FX
contract equal to zero."""
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	1 unit of foreign in domestic	-
domestic_curve	-	-	-
foreign_curve	-	-	-
"""Calculate the FX Forward rate that makes the value of the FX contract equal to zero."""			
	-	-	-

`__repr__`

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

`_print`

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

8.8 fx_mkt_conventions

Enumerated Type: *FinFXATMMMethod*

This enumerated type has the following values:

- SPOT
- FWD
- FWD_DELTA_NEUTRAL
- FWD_DELTA_NEUTRAL_PREM_ADJ

Enumerated Type: *FinFXDeltaMethod*

This enumerated type has the following values:

- SPOT_DELTA
- FORWARD_DELTA
- SPOT_DELTA_PREM_ADJ
- FORWARD_DELTA_PREM_ADJ
- prem_currency
- delta_convention
- self.ccy1
- self.ccy2
- self.ccy2
- self.rate

Class: *FinFXRate*

class FinFXRate:

FinFXRate

PLEASE ADD A FUNCTION DESCRIPTION

```
FinFXRate(ccy1, ccy2, rate):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ccy1	-	-	-
ccy2	-	-	-
rate	-	-	-

8.9 fx_one_touch_option

Class: FXOneTouchOption(FXOption)

A FinFXOneTouchOption is an option in which the buyer receives one unit of currency if the FX rate touches a barrier at any time before the option expiry date and zero otherwise. The single barrier payoff must define whether the option pays or cancels if the barrier is touched and also when the payment is made (at hit time or option expiry). All of these variants are members of the FinTouchOptionTypes type.

FXOneTouchOption

Create the one touch option by defining its expiry date and the barrier level and a payment size if it is a cash

```
FXOneTouchOption(expiry_dt: Date,
opt_type: TouchOptionTypes,
barrier_rate: float,
payment_size: float = 1.0,
"""Create the one touch option by defining its expiry date and the
barrier level and a payment size if it is a cash."""
check_argument_types(.FXOneTouchOption, locals())
.expiry_dt = expiry_dt
.opt_type = opt_type
.barrier_rate = float(barrier_rate)
.payment_size = payment_size
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
opt_type	TouchOptionTypes	-	-
barrier_rate	float	-	-
payment_size	float	-	1.0
"""Create the one touch option by defining its expiry date and the barrier level and a payment size if it is a cash ."""	-	-	-
check_argument_types(.FXOneTouchOption, locals())	-	-	-
.expiry_dt	-	-	expiry_dt
.opt_type	-	-	opt_type
.barrier_rate	-	-	float(barrier_rate)
.payment_size	-	-	payment_size

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt: Date,
spot_fx_rate: Union[float, np.ndarray],
```

```

domestic_curve: DiscountCurve,
foreign_curve: DiscountCurve,
model,
"""FX_One-Touch_Option_valuation_using_the_Black-Scholes_model
assuming_a_continuous_(American)_barrier_from_value_date_to_expiry.
Handles_both_cash-or-nothing_and_asset-or-nothing_options."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_date_not_same_as_valuation_date")
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_date_not_same_as_valuation_date")
debug_mode = False
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
t = max(t, 1e-6)
s0 = spot_fx_rate
h = .barrier_rate
k = .payment_size
sqrt_t = np.sqrt(t)
df = domestic_curve.df(.expiry_dt)
r_d = domestic_curve.cc_rate(.expiry_dt)
r_f = foreign_curve.cc_rate(.expiry_dt)
v = model.volatility
v = max(v, 1e-6)
# Using notation in Haug page 177
b = r_d - r_f
mu = (b - v * v / 2.0) / v / v
lam = np.sqrt(mu * mu + 2.0 * r_d / v / v)
if debug_mode:
    print("t:", t)
    print("vol", v)
    print("b", b)
    print("mu", mu)
    print("lam", lam)
if .opt_type == TouchOptionTypes.DOWN_AND_IN_CASH_AT_HIT:
    # HAUG 1
    if np.any(s0 <= h):

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
spot_fx_rate	float or np.ndarray	-
domestic_curve	DiscountCurve	-
foreign_curve	DiscountCurve	-
model	-	-
"""FX One-Touch Option valuation using the Black-Scholes model assuming a continuous (American) barrier from value date to expiry. Handles both cash-or-nothing and asset-or-nothing options."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt < .expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if domestic_curve.value_dt !=	-	-
raise FinError("Domestic Curve date not same as valuation date")	-	-
if foreign_curve.value_dt !=	-	-
raise FinError("Foreign Curve date not same as valuation date")	-	-
debug_mode	-	-
if value_dt < .expiry_dt	-	-
raise FinError("Value date after expiry date.")	-	-
t	-	(.expiry_dt)
t	-	-
s0	-	-
h	-	-
k	-	-
sqrt_t	-	-
df	-	domestic discount factor
r_d	-	domestic risk-free rate
r_f	-	foreign risk-free rate
v	-	-
v	-	-
b	-	-
mu	-	-
lam	-	np.sqrt(2 * log(1 + lam))
if debug_mode	-	-
print("t", t)	-	-
print("vol", v)	-	-
print("b", b)	-	-
print("mu", mu)	-	-
print("lam", lam)	-	-
if .opt_type == TouchOption	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

value_mc

Touch Option valuation using the Black-Scholes model and Monte Carlo simulation. Accuracy is not great when compared to the analytical result as we only observe the barrier a finite number of times. The convergence is slow.

```
value_mc(value_dt: Date,
stock_price: float,
dom_curve: DiscountCurve,
for_curve: DiscountCurve,
model,
num_paths: int = 10000,
num_steps_per_year: int = 252,
seed: int = 4242,
"""Touch Option valuation using the Black-Scholes model and Monte
Carlo simulation. Accuracy is not great when compared to the analytical
result as we only observe the barrier a finite number of times. The
convergence is slow."""
# "THIS NEEDS TO BE CHECKED"
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df_d = dom_curve.df(.expiry_dt)
r_d = -np.log(df_d) / t
df_f = for_curve.df(.expiry_dt)
r_f = -np.log(df_f) / t
num_time_steps = int(t * num_steps_per_year) + 1
dt = t / num_time_steps
v = model.volatility
s0 = stock_price
mu = r_d - r_f
_, s = get_paths_times(num_paths, num_time_steps, t, mu, s0, v, seed)
h = .barrier_rate
x = .payment_size
v = 0.0
if .opt_type == TouchOptionTypes.DOWN_AND_IN_CASH_AT_HIT:
    # HAUG 1
    if s0 <= h:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_down(s, h, r_d, dt)
    v = v * x
```

```

    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_CASH_AT_HIT:
    # HAUG 2
    if s0 >= h:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_up(s, h, r_d, dt)
    v = v * x
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_IN_ASSET_AT_HIT:
    # HAUG 3
    if s0 <= h:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_down(s, h, r_d, dt) * h
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_ASSET_AT_HIT:
    # HAUG 4
    if s0 >= h:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_up(s, h, r_d, dt) * h
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_IN_CASH_AT_EXPIRY:
    # HAUG 5
    if s0 <= h:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_down(s, h, 0.0, dt)
    v = v * x * np.exp(-r_d * t)
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_CASH_AT_EXPIRY:
    # HAUG 6
    if s0 >= h:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = _barrier_pay_one_at_hit_pv_up(s, h, 0.0, dt)
    v = v * x * np.exp(-r_d * t)
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_IN_ASSET_AT_EXPIRY:
    # HAUG 7
    if s0 <= h:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_down(s, h, 0.0, dt) * h
    return v
elif .opt_type == TouchOptionTypes.UP_AND_IN_ASSET_AT_EXPIRY:
    # HAUG 8
    if s0 >= h:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_one_at_hit_pv_up(s, h, 0.0, dt) * h
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_OUT_CASH_OR_NOTHING:
    # HAUG 9
    if s0 <= h:
        raise FinError("Barrier_has_ALREADY_been_crossed.")
    v = 1.0 - _barrier_pay_one_at_hit_pv_down(s, h, 0.0, dt)
    v = v * x * np.exp(-r_d * t)
    return v
elif .opt_type == TouchOptionTypes.UP_AND_OUT_CASH_OR_NOTHING:
    # HAUG 10
    if s0 >= h:
        raise FinError("Barrier_has_ALREADY_been_crossed.")

```

```

    v = 1.0 - _barrier_pay_one_at_hit_pv_up(s, h, 0.0, dt)
    v = v * x * np.exp(-r_d * t)
    return v
elif .opt_type == TouchOptionTypes.DOWN_AND_OUT_ASSET_OR_NOTHING:
    # HAUG 11
    if s0 <= h:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_asset_at_expiry_down_out(s, h)
    v = v * np.exp(-r_d * t)
    return v
elif .opt_type == TouchOptionTypes.UP_AND_OUT_ASSET_OR_NOTHING:
    # HAUG 12
    if s0 >= h:
        raise FinError("Stock_price_is_currently_below_barrier.")
    v = _barrier_pay_asset_at_expiry_up_out(s, h)
    v = v * np.exp(-r_d * t)
    return v
else:
    raise FinError("Unknown_option_type.")
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	Date	-
stock_price	float	-
dom_curve	DiscountCurve	-
for_curve	DiscountCurve	-
model	-	-
num_paths	int	-
num_steps_per_year	int	-
seed	int	-
"""Touch Option valuation using the Black-Scholes model and Monte Carlo simulation. Accuracy is not great when compared to the analytical result as we only observe the barrier a finite number of times. The convergence is slow."""	-	-
t	-	(e
df_d	-	-
r_d	-	-
df_f	-	-
r_f	-	-
num_time_steps	-	-
dt	-	-
v	-	-
s0	-	-
mu	-	-
-	-	-
s	-	get_paths.ti
h	-	-
x	-	-
v	-	-
if .opt_type	-	= TouchC
if s0 >=	-	-
raise FinError("Barrier has ALREADY been crossed.")	-	-
v	-	_ba
v	-	-
return v	-	-
elif .opt_type	-	= Touch
if s0 <=	-	-
raise FinError("Barrier has ALREADY been crossed.")	-	-
v	-	_b
v	-	-
return v	-	-
elif .opt_type	-	= TouchC
if s0 <=	-	-
raise FinError("Stock price is currently below barrier.")	-	-
v	-	_barri
return v	-	-
elif .opt_type	-	= Touch
if s0 <=	-	-
raise FinError("Stock price is currently below barrier.")	-	-
v	-	_bar
return v	-	-
elif .opt_type	-	= TouchOp
if s0 <=	-	-

`_barrier_pay_one_at_hit_pv_down`

Pay \$1 if the stock crosses the barrier H from above. PV payment.

```
_barrier_pay_one_at_hit_pv_down(s, h, r, dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
h	-	-	-
r	-	-	-
dt	-	-	-

`_barrier_pay_one_at_hit_pv_up`

Pay \$1 if the stock crosses the barrier H from below. PV payment.

```
_barrier_pay_one_at_hit_pv_up(s, h, r, dt):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
h	-	-	-
r	-	-	-
dt	-	-	-

`_barrier_pay_asset_at_expiry_down_out`

Pay \$1 if the stock crosses the barrier H from above. PV payment.

```
_barrier_pay_asset_at_expiry_down_out(s, h):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
h	-	-	-

`_barrier_pay_asset_at_expiry_up_out`

Pay \$1 if the stock crosses the barrier H from below. PV payment.

```
_barrier_pay_asset_at_expiry_up_out(s, h):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
h	-	-	-

8.10 fx_option

Class: FXOption

value

Calculate the option value.

```
value(value_dt: Date,
spot_fx_rate: float,
domestic_curve,
foreign_curve,
model,
"""Calculate the option value."""
print("You should not be here!")
return 0.0
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	Date	-	-
spot_fx_rate	float	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-
"""Calculate the option value."""	-	-	-
print("You should not be here!")	-	-	-
return 0.0	-	-	-

delta

Calculate the option delta (FX rate sensitivity) by adding on a small bump and calculating the change in the option price.

```
delta(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

gamma

Calculate the option gamma (delta sensitivity) by adding on a small bump and calculating the change in the option delta.

```
gamma(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

vega

Calculate the option vega (volatility sensitivity) by adding on a small bump and calculating the change in the option price.

```
vega(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

theta

Calculate the option theta (calendar time sensitivity) by moving forward one day and calculating the change in the option price.

```
theta(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

rho

Calculate the option rho (interest rate sensitivity) by perturbing the discount curve and revaluing.

```
rho(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

8.11 fx_rainbow_option

Enumerated Type: FXRainbowOptionTypes

This enumerated type has the following values:

- CALL_ON_MAXIMUM
- PUT_ON_MAXIMUM
- CALL_ON_MINIMUM
- PUT_ON_MINIMUM
- CALL_ON_NTH
- PUT_ON_NTH
- ifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- k
- payoff
- elifpayoff_type_value
- n
- k
- sorted
- assetn
- payoff
- elifpayoff_type_value
- n
- k
- sorted
- assetn
- payoff
- num_paths
- seed
- df
- r
- mus
- num_time_steps
- s_all
- payoff
- payoff
- v
- self.expiry_dt

- self.payoff_type
- self.payoff_params
- self.num_assets
- iflen(stock_prices)!
- iflen(dividend_yields)!
- iflen(volatilities)!
- iflen(betas)!
- num_params
- ifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- elifpayoff_type
- num_params
- iflen(payoff_params)!
- payoff_type
- orpayoff_type
- n
- ifdomestic_curve.value_dt!
- ifforeign_curve.value_dt!
- ifself.num_assets!
- t
- df
- r
- q1
- q2
- rho
- s1
- s2
- b1
- b2
- v1
- v2
- k
- v
- d
- y1
- y2
- rho1
- rho2

- dq1
- dq2
- df
- sqrtt
- ifself.payoff_type
- v
- elifself.payoff_type
- v
- elifself.payoff_type
- cmax1
- cmax2
- v
- elifself.payoff_type
- cmin1
- cmin2
- v
- num_paths
- seed
- t
- v

Class: FXRainbowOption(FXOption)

class FXRainbowOption(FXOption):

FXRainbowOption

PLEASE ADD A FUNCTION DESCRIPTION

```
FXRainbowOption(expiry_dt: Date,
payoff_type: FXRainbowOptionTypes,
payoff_params: np.ndarray,
num_assets: int,
check_argument_types(.FXRainbowOption, locals())
.validate_payoff(payoff_type, payoff_params, num_assets)
.expiry_dt = expiry_dt
.payoff_type = payoff_type
.payoff_params = payoff_params
.num_assets = num_assets
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
expiry_dt	Date	-	-
payoff_type	FXRainbowOptionTypes	-	-
payoff_params	np.ndarray	-	-
num_assets	int	-	-
check_argument_types(.FXRainbowOption, locals())	-	-	-
.validate_payoff(payoff_type, payoff_params, num_assets)	-	-	-
.expiry_dt	-	-	expiry_dt
.payoff_type	-	-	payoff_type
.payoff_params	-	-	payoff_params
.num_assets	-	-	num_assets

validate

PLEASE ADD A FUNCTION DESCRIPTION

```
validate(stock_prices, dividend_yields, volatilities, betas):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
stock_prices	-	-	-
dividend_yields	-	-	-
volatilities	-	-	-
betas	-	-	-

validate_payoff

PLEASE ADD A FUNCTION DESCRIPTION

```
validate_payoff(payoff_type, payoff_params, num_assets):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
payoff_type	-	-	-
payoff_params	-	-	-
num_assets	-	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```
value(value_dt,
stock_prices,
domestic_curve,
foreign_curve,
```

```

volatilities,
betas,
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation_date_is_not_a_Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation_date_after_expiry_date.")
if domestic_curve.value_dt != value_dt:
    raise FinError("Domestic_Curve_valuation_date_not_same_as_option_value_date"
    )
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_option_value_date")
if .num_assets != 2:
    raise FinError("Analytical_results_for_two_assets_only.")
if value_dt > .expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
.validate(stock_prices, foreign_curve, volatilities, betas)
# Use result by Stulz (1982) given by Haug Page 211
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
df = domestic_curve.df(t)
r = -np.log(df) / t
q1 = foreign_curve[0]
q2 = foreign_curve[1]
rho = betas[0] ** 2
s1 = stock_prices[0]
s2 = stock_prices[1]
b1 = r - q1
b2 = r - q2
v1 = volatilities[0]
v2 = volatilities[1]
k = .payoff_params[0]
v = np.sqrt(v1 * v1 + v2 * v2 - 2 * rho * v1 * v2)
d = (np.log(s1 / s2) + (b1 - b2 + v * v / 2) * t) / v / np.sqrt(t)
y1 = (np.log(s1 / k) + (b1 + v1 * v1 / 2) * t) / v1 / np.sqrt(t)
y2 = (np.log(s2 / k) + (b2 + v2 * v2 / 2) * t) / v2 / np.sqrt(t)
rho1 = (v1 - rho * v2) / v
rho2 = (v2 - rho * v1) / v
dq1 = np.exp(-q1 * t)
dq2 = np.exp(-q2 * t)
df = np.exp(-r * t)
sqrtd = np.sqrt(t)
if .payoff_type == FXRainbowOptionTypes.CALL_ON_MAXIMUM:
    v = (s1 * dq1 * M(y1, d, rho1)
        + s2 * dq2 * M(y2, -d + v * sqrtd, rho2)
        - k * df * (1.0 - M(-y1 + v1 * sqrtd, -y2 + v2 * sqrtd, rho))
    )
elif .payoff_type == FXRainbowOptionTypes.CALL_ON_MINIMUM:
    v = (s1 * dq1 * M(y1, -d, -rho1)
        + s2 * dq2 * M(y2, d - v * np.sqrt(t), -rho2)
        - k * df * M(y1 - v1 * np.sqrt(t), y2 - v2 * np.sqrt(t), rho)
    )
elif .payoff_type == FXRainbowOptionTypes.PUT_ON_MAXIMUM:
    cmax1 = s2 * dq2 + s1 * dq1 * normcdf(d) - s2 * dq2 * normcdf(d - v * sqrtd)
    cmax2 = (s1 * dq1 * M(y1, d, rho1)
        + s2 * dq2 * M(y2, -d + v * sqrtd, rho2)
        - k * df * (1.0 - M(-y1 + v1 * sqrtd, -y2 + v2 * sqrtd, rho))
    )
v = k * df - cmax1 + cmax2

```

```

elif .payoff_type == FXRainbowOptionTypes.PUT_ON_MINIMUM:
    cmin1 = s1 * dq1 - s1 * dq1 * normcdf(d) + s2 * dq2 * normcdf(d - v * sqrtt)
    cmin2 = (s1 * dq1 * M(y1, -d, -rho1)
            + s2 * dq2 * M(y2, d - v * sqrtt, -rho2)
            - k * df * M(y1 - v1 * sqrtt, y2 - v2 * sqrtt, rho)
            )
    v = k * df - cmin1 + cmin2
else:
    raise FinError("Unsupported_FX_Rainbow_option_type")
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
stock_prices	-	-
domestic_curve	-	-
foreign_curve	-	-
volatilities	-	-
betas	-	-
if isinstance(value_dt, Date) is False		-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > .expiry_dt		-
raise FinError("Valuation date after expiry date.")	-	-
if domestic_curve.value_dt !=	-	-
)	-	-
if foreign_curve.value_dt !=	-	-
raise FinError("Foreign Curve valuation date not same as option value date")	-	-
if .num_assets != 2	-	-
raise FinError("Analytical results for two assets only.")	-	-
if value_dt > .expiry_dt		-
raise FinError("Value date after expiry date.")	-	-
.validate(stock_prices, foreign_curve, volatilities, betas)	-	-
t	-	- (.expiry_dt)
df	-	-
r	-	-
q1	-	-
q2	-	-
rho	-	-
s1	-	-
s2	-	-
b1	-	-
b2	-	-
v1	-	-
v2	-	-
k	-	-
v	-	- np.sqrt(
d	-	- (np.log(s1 / s
y1	-	- (np.log(s1 /
y2	-	- (np.log(s2 /
rho1	-	-
rho2	-	-
dq1	-	-
dq2	-	-
df	-	-
sqrtd	-	-
if .payoff_type == "FXRainbow"	-	- = FXRainbow
+ s2 * dq2 * M(y2, -d + v * sqrtd, rho2)	-	-
- k * df * (1.0 - M(-y1 + v1 * np.sqrt(t), -y2 + v2 * sqrtd, rho))	-	-
)	-	-
elif .payoff_type == "FXRainbow"	-	- = FXRainbow
+ s2 * dq2 * M(y2, d - v * np.sqrt(t), -rho2)	-	-
- k * df * M(y1 - v1 * np.sqrt(t), y2 - v2 * np.sqrt(t), rho)	-	-
)	-	-
elif .payoff_type == "FXRainbow"	-	- = FXRainbow

value_mc*PLEASE ADD A FUNCTION DESCRIPTION*

```

value_mc(value_dt,
expiry_dt,
stock_prices,
discount_curve,
dividend_yields,
volatilities,
betas,
num_paths=10000,
seed=4242,
.validate(stock_prices, dividend_yields, volatilities, betas)
if value_dt > expiry_dt:
    raise FinError("Value_date_after_expiry_date.")
t = (.expiry_dt - value_dt) / G_DAYS_IN_YEAR
v = value_mc_fast(t,
    stock_prices,
    discount_curve,
    dividend_yields,
    volatilities,
    betas,
    .num_assets,
    .payoff_type,
    .payoff_params,
    num_paths,
    seed,
)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
expiry_dt	-	-	-
stock_prices	-	-	-
discount_curve	-	-	-
dividend_yields	-	-	-
volatilities	-	-	-
betas	-	-	-
num_paths	-	-	10000
seed	-	-	4242
.validate(stock_prices, dividend_yields, volatilities, betas)	-	-	-
if value_dt > expiry_dt		-	-
raise FinError("Value date after expiry date.")	-	-	-
t	-	-	(.expiry_dt - value_dt) / G_DAYS_IN_Y
stock_prices	-	-	-
discount_curve	-	-	-
dividend_yields	-	-	-
volatilities	-	-	-
betas	-	-	-
.num_assets	-	-	-
.payoff_type	-	-	-
.payoff_params	-	-	-
num_paths	-	-	-
seed	-	-	-
)	-	-	-
return v	-	-	-

payoff_value

PLEASE ADD A FUNCTION DESCRIPTION

```
payoff_value(s, payoff_type_value, payoff_params):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
payoff_type_value	-	-	-
payoff_params	-	-	-

value_mc_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc_fast(
t,
```

```

stock_prices,
discount_curve,
dividend_yields,
volatilities,
betas,
num_assets,
payoff_type,
payoff_params,
num_paths=10000,
seed=4242,
np.random.seed(seed)
df = discount_curve.df(t)
r = -np.log(df) / t
mus = r - dividend_yields
num_time_steps = 2
s_all = get_assets_paths_times(
    num_assets,
    num_paths,
    num_time_steps,
    t,
    mus,
    stock_prices,
    volatilities,
    betas,
    seed,
)
payoff = payoff_value(s_all, payoff_type.value, payoff_params)
payoff = np.mean(payoff)
v = payoff * np.exp(-r * t)
return v
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
stock_prices	-	-	-
discount_curve	-	-	-
dividend_yields	-	-	-
volatilities	-	-	-
betas	-	-	-
num_assets	-	-	-
payoff_type	-	-	-
payoff_params	-	-	-
num_paths	-	-	10000
seed	-	-	4242
np.random.seed(seed)	-	-	-
df	-	-	discount_curve.df(t)
r	-	-	-np.log(df) / t
mus	-	-	r - dividend_yields
num_time_steps	-	-	2
num_assets	-	-	-
num_paths	-	-	-
num_time_steps	-	-	-
t	-	-	-
mus	-	-	-
stock_prices	-	-	-
volatilities	-	-	-
betas	-	-	-
seed	-	-	-
)	-	-	-
payoff	-	-	payoff_value(s_all, payoff_type.value, payoff_params)
payoff	-	-	np.mean(payoff)
v	-	-	payoff * np.exp(-r * t)
return v	-	-	-

8.12 fx_vanilla_option

Class: FXVanillaOption

This is a class for an FX Option trade. It permits the user to calculate the price of an FX Option trade which can be expressed in a number of ways depending on the investor or hedgers currency. It also allows the calculation of the options delta in a number of forms as well as the various Greek risk sensitivities.

FXVanillaOption

PLEASE ADD A FUNCTION DESCRIPTION

```
FXVanillaOption(expiry_dt: Date,
# 1 unit of foreign in domestic
strike_fx_rate: Union[float, np.ndarray],
currency_pair: str, # FORDOM
opt_type: Union[OptionTypes, list],
notional: float,
prem_currency: str,
spot_days: int = 0,
"""Create the FX Vanilla Option object. Inputs include expiry date,
strike, currency pair, option type (call or put), notional and the
currency of the notional. And adjustment for spot days is enabled. All
currency rates must be entered in the price in domestic currency of
one unit of foreign. And the currency pair should be in the form FORDOM
where FOR is the foreign currency pair currency code and DOM is the
same for the domestic currency."""
check_argument_types(.FXVanillaOption, locals())
delivery_dt = expiry_dt.add_weekdays(spot_days)
"""The FX rate the price in domestic currency ccy2 of a single unit
of the foreign currency which is ccy1. For example EURUSD of 1.3 is the
price in USD (CCY2) of 1 unit of EUR (CCY1)"""
if delivery_dt < expiry_dt:
    raise FinError("Delivery date must be on or after expiry date.")
if len(currency_pair) != 6:
    raise FinError("Currency pair must be 6 characters.")
.expiry_dt = expiry_dt
.delivery_dt = delivery_dt
if np.any(strike_fx_rate < 0.0):
```

The function arguments are described in the following table.

Argument Name	Type	Description
expiry_dt	Date	-
strike_fx_rate	float or np.ndarray	-
currency_pair	str	FORDOM
opt_type	OptionTypes or list	-
notional	float	-
prem_currency	str	-
spot_days	int	-
"""Create the FX Vanilla Option object. Inputs include expiry date		
strike	-	-
currency pair	-	-
option type (call or put)	-	-
notional and the	-	-
currency of the notional. And adjustment for spot days is enabled. All	-	-
currency rates must be entered in the price in domestic currency of	-	-
one unit of foreign. And the currency pair should be in the form FORDOM	-	-
where FOR is the foreign currency pair currency code and DOM is the	-	-
same for the domestic currency."""	-	-
check_argument_types(FXVanillaOption, locals())	-	-
delivery_dt	-	expiry_dt
""" The FX rate the price in domestic currency ccy2 of a single unit		
of the foreign currency which is ccy1. For example EURUSD of 1.3 is the	-	-
price in USD (CCY2) of 1 unit of EUR (CCY1)"""	-	-
if delivery_dt > expiry_dt	-	-
raise FinError("Delivery date must be on or after expiry date.")	-	-
if len(currency_pair) != 6	-	-
raise FinError("Currency pair must be 6 characters.")	-	-
.expiry_dt	-	-
.delivery_dt	-	-

value

PLEASE ADD A FUNCTION DESCRIPTION

```

value(value_dt,
spot_fx_rate, # 1 unit of foreign in domestic
domestic_curve,
foreign_curve,
model,
"""This function calculates the value of the option using a specified
model with the resulting value being in domestic i.e. ccy2 terms.
Recall that Domestic = CCY2 and Foreign = CCY1 and FX rate is in
price in domestic of one unit of foreign currency."""
if isinstance(value_dt, Date) is False:
    raise FinError("Valuation date is not a Date")
if value_dt > .expiry_dt:
    raise FinError("Valuation date after expiry date.")
if domestic_curve.value_dt != value_dt:

```

```

    raise FinError("Domestic_Curve_valuation_date_not_same_as_valuation_date")
if foreign_curve.value_dt != value_dt:
    raise FinError("Foreign_Curve_valuation_date_not_same_as_valuation_date")
if isinstance(value_dt, Date):

```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
spot_fx_rate	-	1 unit of foreign in domestic
domestic_curve	-	-
foreign_curve	-	-
model	-	-
"""This function calculates the value of the option using a specified model with the resulting value being in domestic i.e. ccy2 terms.	-	-
Recall that Domestic	-	-
price in domestic of one unit of foreign currency."""	-	-
if isinstance(value_dt, Date) is False	-	-
raise FinError("Valuation date is not a Date")	-	-
if value_dt > .expiry_dt	-	-
raise FinError("Valuation date after expiry date.")	-	-
if domestic_curve.value_dt !=	-	-
raise FinError("Domestic Curve valuation date not same as valuation date")	-	-
if foreign_curve.value_dt !=	-	-
raise FinError("Foreign Curve valuation date not same as valuation date")	-	-

delta_bump

PLEASE ADD A FUNCTION DESCRIPTION

```

delta_bump(value_dt,
spot_fx_rate,
ccy1_discount_curve,
ccy2_discount_curve,
model,
"""Calculation_of_the_FX_option_delta_by_bumping_the_spot_FX_rate_by
1_cent_of_its_value._This_gives_the_FX_spot_delta._For_speed_we_prefer
to_use_the_analytical_calculation_of_the_derivative_given_below."""
bump = 0.0001 * spot_fx_rate
v = .value(value_dt, spot_fx_rate, ccy1_discount_curve, ccy2_discount_curve, model
)
v_bumped = .value(value_dt,
    spot_fx_rate + bump,
    ccy1_discount_curve,
    ccy2_discount_curve,
    model,
)
if isinstance(v_bumped, dict):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
ccy1_discount_curve	-	-	-
ccy2_discount_curve	-	-	-
model	-	-	-
"""Calculation of the FX option delta by bumping the spot FX rate by			
1 cent of its value. This gives the FX spot delta. For speed we prefer			
to use the analytical calculation of the derivative given below."""			
bump	-	-	0.0001 * spot_fx_rate
)	-	-	-
spot_fx_rate + bump	-	-	-
ccy1_discount_curve	-	-	-
ccy2_discount_curve	-	-	-
model	-	-	-
)	-	-	-

delta

Calculation of the FX Option delta. There are several definitions of delta and so we are required to return a dictionary of values. The definitions can be found on Page 44 of Foreign Exchange Option Pricing by Iain Clark, published by Wiley Finance.

```
delta(value_dt, spot_fx_rate, domestic_curve, foreign_curve, model):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-

fast_delta

Calculation of the FX Option delta. Used in the determination of the volatility surface. Avoids discount curve interpolation so it should be slightly faster than the full calculation of delta.

```
fast_delta(t, s, rd, rf, vol):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
s	-	-	-
rd	-	-	-
rf	-	-	-
vol	-	-	-

gamma

PLEASE ADD A FUNCTION DESCRIPTION

```
gamma(value_dt,
spot_fx_rate, # value of a unit of foreign in domestic currency
domestic_curve,
foreign_curve,
model,
"""This_function_calculates_the_FX_Option_Gamma_using_spot_delta."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
spot_fx_rate	-	value of a unit of foreign in domestic currency
domestic_curve	-	-
foreign_curve	-	-
model	-	-
"""This function calculates the FX Option Gamma using spot delta."""	-	-

vega

PLEASE ADD A FUNCTION DESCRIPTION

```
vega(value_dt,
spot_fx_rate, # value of a unit of foreign in domestic currency
domestic_curve,
foreign_curve,
model,
"""This_function_calculates_the_FX_Option_Vega_using_the_spot_delta."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
spot_fx_rate	-	value of a unit of foreign in domestic currency
domestic_curve	-	-
foreign_curve	-	-
model	-	-
"""This function calculates the FX Option Vega using the spot delta."""		

theta

PLEASE ADD A FUNCTION DESCRIPTION

```
theta(value_dt,
spot_fx_rate, # value of a unit of foreign in domestic currency
domestic_curve,
foreign_curve,
model,
"""This function calculates the time decay of the FX option."""
if isinstance(value_dt, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description
value_dt	-	-
spot_fx_rate	-	value of a unit of foreign in domestic currency
domestic_curve	-	-
foreign_curve	-	-
model	-	-
"""This function calculates the time decay of the FX option."""		

implied_volatility

This function determines the implied volatility of an FX option given a price and the other option details. It uses a one-dimensional Newton root search algorithm to determine the implied volatility.

```
implied_volatility(value_dt, stock_price, discount_curve, dividend_curve, price
"""This function determines the implied volatility of an FX option
given a price and the other option details. It uses a one-dimensional
Newton root search algorithm to determine the implied volatility."""
argtuple = (value_dt,
stock_price,
discount_curve,
dividend_curve,
price,
)
sigma = optimize.newton(f,
x0=0.2,
fprime=fvega,
args=argtuple,
```

```

    tol=1e-6,
    maxiter=50,
    fprime2=None,
)
return sigma
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_price	-	-	-
discount_curve	-	-	-
dividend_curve	-	-	-
price	-	-	-
"""This function determines the implied volatility of an FX option given a price and the other option details. It uses a one-dimensional Newton root search algorithm to determine the implied volatility."""	-	-	-
stock_price	-	-	-
discount_curve	-	-	-
dividend_curve	-	-	-
price	-	-	-
)	-	-	-
x0	-	-	0.2
fprime	-	-	fvega
args	-	-	argtuple
tol	-	-	1e-6
maxiter	-	-	50
fprime2	-	-	None
)	-	-	-
return sigma	-	-	-

value_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc(value_dt,
spot_fx_rate,
domestic_curve,
foreign_curve,
model,
num_paths=10000,
seed=4242,
"""Calculate the value of an FX Option using Monte Carlo methods.
This function can be used to validate the risk measures calculated
above or used as the starting code for a model exotic FX product that
cannot be priced analytically. This function uses Numpy vectorisation
for speed of execution."""

```

```
if isinstance(model, BlackScholes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
spot_fx_rate	-	-	-
domestic_curve	-	-	-
foreign_curve	-	-	-
model	-	-	-
num_paths	-	-	10000
seed	-	-	4242
"""Calculate the value of an FX Option using Monte Carlo methods.			
This function can be used to validate the risk measures calculated			
above or used as the starting code for a model exotic FX product that			
cannot be priced analytically. This function uses Numpy vectorisation			
for speed of execution."""			

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_print

Simple print function for backward compatibility.

```
_print():
```

The function arguments are described in the following table.

f

PLEASE ADD A FUNCTION DESCRIPTION

```
f(volatility, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	-	-	-
*args	-	-	-

fvega

PLEASE ADD A FUNCTION DESCRIPTION

```
fvega(volatility, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	-	-	-
*args	-	-	-

fast_delta

Calculation of the FX Option delta. Used in the determination of the volatility surface. Avoids discount curve interpolation so it should be slightly faster than the full calculation of delta.

```
fast_delta(s, t, k, rd, rf, vol, delta_type_value, opt_type_value):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
t	-	-	-
k	-	-	-
rd	-	-	-
rf	-	-	-
vol	-	-	-
delta_type_value	-	-	-
opt_type_value	-	-	-

8.13 fx_variance_swap

Class: *FinFXVarianceSwap*

FinFXVarianceSwap

PLEASE ADD A FUNCTION DESCRIPTION

```
FinFXVarianceSwap(effective_dt: Date,
maturity_dt_or_tenor: Union[Date, str],
strike_variance: float,
notional: float = ONE_MILLION,
pay_strike_flag: bool = True,
"""Create_variance_swap_contract."""
check_argument_types(.FinFXVarianceSwap, locals())
if isinstance(maturity_dt_or_tenor, Date):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
effective_dt	Date	-	-
maturity_dt_or_tenor	Date or str	-	-
strike_variance	float	-	-
notional	float	-	ONE_MILLION
pay_strike_flag	bool	-	True
"""Create variance swap contract."""	-	-	-
check_argument_types(.FinFXVarianceSwap, locals())	-	-	-

value

Calculate the value of the variance swap based on the realised volatility to the valuation date, the forward looking implied volatility to the maturity date using the libor discount curve.

```
value(value_dt, realised_var, fair_strike_var, libor_curve):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
realised_var	-	-	-
fair_strike_var	-	-	-
libor_curve	-	-	-

fair_strike_approx

This is an approximation of the fair strike variance by Demeterfi et al. (1999) which assumes that $\sigma(K) = \sigma(F) - b(K-F)/F$ where F is the forward stock price and $\sigma(F)$ is the ATM forward vol.

```
fair_strike_approx(value_dt, fwd_stock_price, strikes, volatilities):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
fwd_stock_price	-	-	-
strikes	-	-	-
volatilities	-	-	-

fair_strike

Calculate the implied variance according to the volatility surface using a static replication methodology with a specially weighted portfolio of put and call options across a range of strikes using the approximate method set out by Demeterfi et al. 1999.

```
fair_strike(value_dt,
stock_price,
dividend_curve,
volatility_curve,
num_call_options,
num_put_options,
strike_spacing,
discount_curve,
use_forward=True,
"""Calculate the implied variance according to the volatility surface
using a static replication methodology with a specially weighted
portfolio of put and call options across a range of strikes using the
approximate method set out by Demeterfi et al. 1999."""
.num_put_options = num_put_options
.num_call_options = num_call_options
call_type = OptionTypes.EUROPEAN_CALL
put_type = OptionTypes.EUROPEAN_PUT
t_mat = (.maturity_dt - value_dt) / G_DAYS_IN_YEAR
df = discount_curve.df(t_mat)
r = -np.log(df) / t_mat
dq = dividend_curve.df(t_mat)
q = -np.log(dq) / t_mat
s0 = stock_price
g = np.exp((r - q) * t_mat)
fwd = stock_price * g
# This fixes the centre strike of the replication options
if use_forward is True:
    sstar = fwd
else:
    sstar = stock_price
# Replication argument from Demeterfi, Derman, Kamal and Zhou from
# Goldman Sachs Research notes March 1999. See Appendix A. This aim is
# to use calls and puts to approximate the payoff of a log contract """
min_strike = sstar - (num_put_options + 1) * strike_spacing
.put_wts = []
.put_strikes = []
.call_wts = []
```

```

.call_strikes = []
# if the lower strike is < 0 we go to as low as the strike spacing
if min_strike < strike_spacing:
    k = sstar
    klist = [sstar]
    while k >= strike_spacing:
        k -= strike_spacing
        klist.append(k)
    put_k = np.array(klist)
    .num_put_options = len(put_k) - 1
else:
    put_k = np.linspace(sstar, min_strike, num_put_options + 2)
.put_strikes = put_k
max_strike = sstar + (num_call_options + 1) * strike_spacing
call_k = np.linspace(sstar, max_strike, num_call_options + 2)
.call_strikes = call_k
option_total = (2.0 * (r * t_mat - (s0 * g / sstar - 1.0) - np.log(sstar / s0)) / t_mat
)
.call_wts = np.zeros(num_call_options)
.put_wts = np.zeros(num_put_options)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	-	-	-
stock_price	-	-	-
dividend_curve	-	-	-
volatility_curve	-	-	-
num_call_options	-	-	-
num_put_options	-	-	-
strike_spacing	-	-	-
discount_curve	-	-	-
use_forward	-	-	True
"""Calculate the implied variance according to the volatility surface using a static replication methodology with a specially weighted portfolio of put and call options across a range of strikes using the approximate method set out by Demeterfi et al. 1999."""			-
.num_put_options	-	-	num_put_o
.num_call_options	-	-	num_call.o
call_type	-	-	OptionTypes.EURC
put_type	-	-	OptionTypes.EURP
t_mat	-	-	(.maturity_dt - value_dt) /
df	-	-	discount_curve
r	-	-	-np.log(df)
dq	-	-	dividend_curve
q	-	-	-np.log(dq)
s0	-	-	stock_pr
g	-	-	np.exp((r - q)
fwd	-	-	stock_pric
if use_forward is True	-	-	-
sstar	-	-	fwd
else	-	-	-
sstar	-	-	stock_pr
min_strike	-	-	sstar - (num_put_options -
.put_wts	-	-	[]
.put_strikes	-	-	[]
.call_wts	-	-	[]
.call_strikes	-	-	[]
if min_strike > strike_spacing	-	-	-
k	-	-	sstar
klist	-	-	[sstar
while k <=	-	-	strike_spa
k =	-	-	strike_spa
klist.append(k)	-	-	-
put_k	-	-	np.array(l
.num_put_options	-	-	len(put_k
else	-	-	-
put_k	-	-	np.linspace(sstar, min_strike
.put_strikes	-	-	put_k
max_strike	-	-	sstar + (num_call_options
call_k	-	-	np.linspace(sstar, max_strike
.call_strikes	-	-	call.k
)	-	-	-
.call_wts	-	-	np.zeros(num_ca
.put_wts	-	-	np.zeros(num

f*PLEASE ADD A FUNCTION DESCRIPTION*`f(x) :`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

realised_variance

Calculate the realised variance according to market standard calculations which can either use log or percentage returns.

`realised_variance(close_prices, use_logs=True) :`

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
close_prices	-	-	-
use_logs	-	-	True

print_strikes*PLEASE ADD A FUNCTION DESCRIPTION*`print_strikes() :`

The function arguments are described in the following table.

8.14 `__init__`

Chapter 9

financepy.models

Models

Overview

This folder contains a range of models used in the various derivative pricing models implemented in the product folder. These include credit models for valuing portfolio credit products such as CDS Tranches, Monte-Carlo based models of stochastic processes used to value equity, FX and interest rate derivatives, and some generic implementations of models such as a tree-based Hull White model. Because the models are useful across a range of products, it is better to factor them out of the product/asset class categorisation as it avoids any unnecessary duplication.

In addition we seek to make the interface to these models rely only on fast types such as floats and integers and Numpy arrays.

These modules hold all of the models used by FinancePy across asset classes.

The general philosophy is to separate where possible product and models so that these models have as little product knowledge as possible.

Also, Numba is used extensively, resulting in code speedups of between x 10 and x 100.

Generic Arbitrage-Free Models

There are the following arbitrage-free models:

- `black` is Black's model for pricing forward starting contracts (in the forward measure) assuming the forward is lognormally distributed.
- `black_shifted` is Black's model for pricing forward starting contracts (in the forward measure) assuming the forward plus a shift is lognormally distributed. CHECK
- `bachelier` prices options assuming the underlying evolves according to a Gaussian (normal) process.
- `sabr` is a stochastic volatility model for forward values with a closed form approximate solution for the implied volatility. It is widely used for pricing European style interest rate options, specifically caps and floors and also swaptions.
- `sabr_shifted` is a stochastic volatility model for forward value with a closed form approximate solution for the implied volatility. It is widely used for pricing European style interest rate options, specifically caps and floors and also swaptions.

The following asset-specific models have been implemented:

Equity Models

- `heston_model`
- `process_simulator`

Interest Rate Models

Equilibrium Rate Models

There are two main short rate models.

- CIR is a short rate model where the randomness component is proportional to the square root of the short rate. This model implementation is not arbitrage-free across the term structure.
- Vasicek is a short rate model that assumes mean-reversion and normal volatility. It has a closed form solution for bond prices. It does not have the flexibility to fit a term structure of interest rates. For that you need to use the more flexible Hull-White model.

Arbitrage Free Rate Models

- BKTREE is a short rate model in which the log of the short rate follows a mean-reverting normal process. It refits the interest rate term structure. It is implemented as a trinomial tree and allows valuation of European and American-style rate-based options.
- HWTREE is a short rate model in which the short rate follows a mean-reverting normal process. It fits the interest rate term structure. It is implemented as a trinomial tree and allows valuation of European and American-style rate-based options. It also implements Jamshidian's decomposition of the bond option for European options.

Credit Models

- `GaussianCopula1F` is a Gaussian copula one-factor model. This class includes functions that calculate the portfolio loss distribution. This is numerical but deterministic.
- `GaussianCopulaLHP` is a Gaussian copula one-factor model in the limit that the number of credits tends to infinity. This is an asymptotic analytical solution.
- `GaussianCopula` is a Gaussian copula model which is multifactor model. It has a Monte-Carlo implementation.
- `LossDbnBuilder` calculates the loss distribution.
- `MertonFirm` is a model of the firm as proposed by Merton (1974).

FX Models

9.1 bachelier

Class: Bachelier

Bacheliers Model which prices call and put options in the forward measure assuming the underlying rate follows a normal process.

Bachelier

Create FinModel black using parameters.

```
Bachelier(volatility: float) -> None:
    """Create_FinModel_black_using_parameters."""
    if volatility <= 0.0:
        raise FinError("Volatility_must_be_positive")
    .volatility = volatility
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	float)-ζNone:	-	-
"""Create FinModel black using parameters."""	-	-	-
if volatility ;	-	-	0.0:
raise FinError("Volatility must be positive")	-	-	-
.volatility	-	-	volatility

value

Price a call or put option using Bachelier's model.

```
value(forward_rate: float, # Forward rate F
      strike_rate: float, # Strike Rate K
      time_to_expiry: float, # Time to Expiry (years)
      df: float, # Discount Factor to expiry date
      call_or_put: OptionTypes, # Call or put
      float:
    """Price_a_call_or_put_option_using_Bachelier's_model."""
    f = forward_rate
    t = time_to_expiry
    k = strike_rate
    root_t = np.sqrt(t)
    v = .volatility
    d = (f - k) / (v * root_t)
    if call_or_put == OptionTypes.EUROPEAN_CALL:
        return df * ((f - k) * norm.cdf(d) + v * root_t * norm.pdf(d))
    elif call_or_put == OptionTypes.EUROPEAN_PUT:
        return df * ((k - f) * norm.cdf(-d) + v * root_t * norm.pdf(d))
    else:
        raise FinError("Option_type_must_be_a_European_Call(C)_or_Put(P)")
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
forward_rate	float	Forward rate F
strike_rate	float	Strike Rate K
time_to_expiry	float	Time to Expiry (years)
df	float	Discount Factor to expiry date
call_or_put	OptionTypes	Call or put
float		-
"""Price a call or put option using Bachelier's model."""	-	-
f	-	-
t	-	-
k	-	-
root_t	-	-
v	-	-
d	-	-
if call_or_put	-	-
return df * ((f - k) * norm.cdf(d) + v * root_t * norm.pdf(d))	-	-
elif call_or_put	-	-
return df * ((k - f) * norm.cdf(-d) + v * root_t * norm.pdf(d))	-	-
else	-	-
raise FinError("Option type must be a European Call(C) or Put(P)")	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("VOLATILITY", .volatility)
return s
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str		-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("VOLATILITY", .volatility)
return s	-	-	-

9.2 bdt_tree

Class: BDTTree

class BDTTree:

BDTTree

*Constructs the Black-Derman-Toy rate model in the case when the volatility is assumed to be constant. The short rate process simplifies and is given by $d(\log(r)) = \theta(t) * dt + \sigma * dW$. Although*

```
BDTTree(sigma: float, num_time_steps: int = 100) -> None:
    """Constructs the Black-Derman-Toy rate model in the case when the
    volatility is assumed to be constant. The short rate process simplifies
    and is given by  $d(\log(r)) = \theta(t) * dt + \sigma * dW$ . Although"""
    if sigma < 0.0:
        raise FinError("Negative volatility not allowed.")
    .sigma = sigma
    if num_time_steps < 3:
        raise FinError("Drift fitting requires at least 3 time steps.")
    .num_time_steps = num_time_steps
    .qq = None
    .rt = None
    # .dfs = None
    .pu = 0.50
    .pd = 0.50
    .discount_curve = None
    ._dt = None
    ._df_times = None
    ._df_values = None
    ._tree_times = None
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	float	-	-
num_time_steps	int	-	100) -; None
"""Constructs the Black-Derman-Toy rate model in the case when the volatility is assumed to be constant. The short rate process simplifies and is given by $d(\log(r))$	-	-	-
if sigma > 0.0	-	-	-
raise FinError("Negative volatility not allowed.")	-	-	-
.sigma	-	-	sigma
if num_time_steps > 3	-	-	-
raise FinError("Drift fitting requires at least 3 time steps.")	-	-	-
.num_time_steps	-	-	num_time_steps
.qq	-	-	None
.rt	-	-	None
.pu	-	-	0.50
.pd	-	-	0.50
.discount_curve	-	-	None
.dt	-	-	None
.df_times	-	-	None
.df_values	-	-	None
.tree_times	-	-	None

build_tree

PLEASE ADD A FUNCTION DESCRIPTION

```

build_tree(tree_mat: float,
df_times: np.ndarray,
df_values: np.ndarray,
None:
if isinstance(df_times, np.ndarray) is False:
    raise FinError("DF_TIMES_must_be_a_numpy_vector")
if isinstance(df_values, np.ndarray) is False:
    raise FinError("DF_VALUES_must_be_a_numpy_vector")
interp = InterpTypes.FLAT_FWD_RATES.value
tree_maturity = tree_mat * (.num_time_steps + 1) / .num_time_steps
tree_times = np.linspace(0.0, tree_maturity, .num_time_steps + 2)
._tree_times = tree_times
df_tree = np.zeros(shape=.num_time_steps + 2)
df_tree[0] = 1.0
for i in range(1, .num_time_steps + 2):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
tree_mat	float	-	-
df_times	np.ndarray	-	-
df_values	np.ndarray	-	-
None		-	-
if isinstance(df_times, np.ndarray) is False		-	-
raise FinError("DF TIMES must be a numpy vector")	-	-	-
if isinstance(df_values, np.ndarray) is False		-	-
raise FinError("DF VALUES must be a numpy vector")	-	-	-
interp	-	-	InterpTypes.FLAT_FWD_RA
tree_maturity	-	-	tree_mat * (.num_time_steps + 1) / .
tree_times	-	-	np.linspace(0.0, tree_maturity, .num
._tree_times	-	-	tree_times
df_tree	-	-	np.zeros(shape=.num_time_s
df_tree[0]	-	-	1.0

bond_option

Value a bond option that can have European or American exercise using the Black-Derman-Toy model. The model uses a binomial tree.

```

bond_option(t_exp: float,
strike_price: float,
face_amount: float,
cpn_times: np.ndarray,
cpn_flows: np.ndarray,
exercise_type: ExerciseTypes,
dict[str, float]):
    """Value a bond option that can have European or American exercise
    using the Black-Derman-Toy model. The model uses a binomial tree."""
    exercise_type_int = option_exercise_types_to_int(exercise_type)
    t_mat = cpn_times[-1]
    if t_exp > t_mat:
        raise FinError("Option expiry after bond matures.")
    if t_exp < 0.0:
        raise FinError("Option expiry time negative.")
    call_value, put_value = american_bond_option_tree_fast(t_exp,
        t_mat,
        strike_price,
        face_amount,
        cpn_times,
        cpn_flows,
        exercise_type_int,
        ._df_times,
        ._df_values,
        ._tree_times,
        .qq,
        .rt,
        ._dt,
    )
    return {"call": call_value, "put": put_value}
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	
t_exp	float	-	
strike_price	float	-	
face_amount	float	-	
cpn_times	np.ndarray	-	
cpn_flows	np.ndarray	-	
exercise_type	ExerciseTypes	-	
dict[str, float]		-	
"""Value a bond option that can have European or American exercise using the Black-Derman-Toy model. The model uses a binomial tree."""	-	-	
exercise_type_int	-	-	option_exercise
t_mat	-	-	
if t_exp > t_mat		-	
raise FinError("Option expiry after bond matures.")	-	-	
if t_exp < 0.0		-	
raise FinError("Option expiry time negative.")	-	-	
call_value	-	-	
t_mat	-	-	
strike_price	-	-	
face_amount	-	-	
cpn_times	-	-	
cpn_flows	-	-	
exercise_type_int	-	-	
.df_times	-	-	
.df_values	-	-	
.tree_times	-	-	
.qq	-	-	
.rt	-	-	
.dt	-	-	
)	-	-	
return "call"	call_value	-	
"put"	put_value	-	

bermudan_swaption

Swaption that can be exercised on specific dates over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time.

```
bermudan_swaption(t_exp: float,
t_mat: float,
strike: float,
face_amount: float,
```

```

cpn_times: np.ndarray,
cpn_flows: np.ndarray,
exercise_type: ExerciseTypes,
    dict[str, float]:
    """Swaption that can be exercised on specific dates over the exercise
    period. Due to non-analytical bond price we need to extend tree out to
    bond maturity and take into account cash flows through time."""
exercise_type_int = option_exercise_types_to_int(exercise_type)
t_mat = cpn_times[-1]
if t_exp > t_mat:
    raise FinError("Option_expiry_after_bond_matures.")
if t_exp < 0.0:
    raise FinError("Option_expiry_time_negative.")
pay_value, rec_value = bermudan_swaption_tree_fast(t_exp,
    t_mat,
    strike,
    face_amount,
    cpn_times,
    cpn_flows,
    exercise_type_int,
    ._df_times,
    ._df_values,
    ._tree_times,
    .qq,
    .rt,
    ._dt,
)
return {"pay": pay_value, "rec": rec_value}
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Def
t_exp	float	-	
t_mat	float	-	
strike	float	-	
face_amount	float	-	
cpn_times	np.ndarray	-	
cpn_flows	np.ndarray	-	
exercise_type	ExerciseTypes	-	
dict[str, float]		-	
"""Swaption that can be exercised on specific dates over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time."""	-	-	
exercise_type.int	-	-	option_exercise_ty
t_mat	-	-	cpn
if t_exp < t_mat		-	
raise FinError("Option expiry after bond matures.")	-	-	
if t_exp > 0.0		-	
raise FinError("Option expiry time negative.")	-	-	
pay_value	-	-	
t_mat	-	-	
strike	-	-	
face_amount	-	-	
cpn_times	-	-	
cpn_flows	-	-	
exercise_type.int	-	-	
.df_times	-	-	
.df_values	-	-	
.tree_times	-	-	
.qq	-	-	
.rt	-	-	
.dt	-	-	
)	-	-	
return "pay"	pay_value	-	
"rec"	rec_value	-	

callable_puttable_bond_tree

Option that can be exercised at any time over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time.

```
callable_puttable_bond_tree(cpn_times: np.ndarray,
cpn_flows: np.ndarray,
call_times: np.ndarray,
call_prices: np.ndarray,
put_times: np.ndarray,
put_prices: np.ndarray,
face_amount: float,
```

```

dict[str, float]:
    """Option that can be exercised at any time over the exercise period.
    Due to non-analytical bond price we need to extend tree out to bond
    maturity and take into account cash flows through time."""
    call_times = np.array(call_times)
    put_times = np.array(put_times)
    call_prices = np.array(call_prices)
    put_prices = np.array(put_prices)
    v = callable_puttable_bond_tree_fast(cpn_times,
        cpn_flows,
        call_times,
        call_prices,
        put_times,
        put_prices,
        face_amount,
        .sigma,
        .qq,
        .rt,
        .dt,
        .tree_times,
        .df_times,
        .df_values,
    )
    return {
        "bondwithoption": v["bondwithoption"],
        "bondpure": v["bondpure"],
    }
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
call_times	np.ndarray	-	-
call_prices	np.ndarray	-	-
put_times	np.ndarray	-	-
put_prices	np.ndarray	-	-
face_amount	float	-	-
dict[str, float]		-	-
"""Option that can be exercised at any time over the exercise period.	-	-	-
Due to non-analytical bond price we need to extend tree out to bond	-	-	-
maturity and take into account cash flows through time."""	-	-	-
call_times	-	-	np.array(call
put_times	-	-	np.array(put
call_prices	-	-	np.array(call
put_prices	-	-	np.array(put
cpn_flows	-	-	-
call_times	-	-	-
call_prices	-	-	-
put_times	-	-	-
put_prices	-	-	-
face_amount	-	-	-
.sigma	-	-	-
.qq	-	-	-
.rt	-	-	-
.dt	-	-	-
..tree_times	-	-	-
..df_times	-	-	-
..df_values	-	-	-
)	-	-	-
return	-	-	-
"bondwithoption"	v["bondwithoption"]	-	-
"bondpure"	v["bondpure"]	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str		-	-

option_exercise_types_to_int

Convert the option exercise type to an integer for use in the tree.

```
option_exercise_types_to_int(option_exercise_type: ExerciseTypes) -> int:
    """Convert the option exercise type to an integer for use in the tree."""
    if option_exercise_type == ExerciseTypes.EUROPEAN:
        return 1
    if option_exercise_type == ExerciseTypes.BERMUDAN:
        return 2
    if option_exercise_type == ExerciseTypes.AMERICAN:
        return 3
    raise FinError("Unknown option exercise type.")
#####
t(
float64(float64, int64, float64[:, :], float64[:, :], float64, float64, float64),
fastmath=True,
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description
option_exercise_type	ExerciseTypes)-int:	-
"""Convert the option exercise type to an integer for use in the tree."""	-	-
if option_exercise_type	-	-
return 1	-	-
if option_exercise_type	-	-
return 2	-	-
if option_exercise_type	-	-
return 3	-	-
raise FinError("Unknown option exercise type.")	-	-
float64(float64, int64, float64[	,:],float64[:,:],float64,float64,float64)	-
fastmath	-	-
cache	-	-

f

PLEASE ADD A FUNCTION DESCRIPTION

```
f(
x0: float,
m: int,
q_matrix: np.ndarray,
rt: np.ndarray,
df_end: float,
dt: float,
sigma: float,
float:
# x is the middle value on the short-rate on the tree
midm = int(m / 2)
```

```
rt[m, midm] = x0
for i in range(midm, 0, -1):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x0	float	-	-
m	int	-	-
q_matrix	np.ndarray	-	-
rt	np.ndarray	-	-
df_end	float	-	-
dt	float	-	-
sigma	float	-	-
float		-	-
midm	-	-	int(m / 2)
rt[m, midm]	-	-	x0

search_root

PLEASE ADD A FUNCTION DESCRIPTION

```
search_root(
x0: float,
m: int,
q_matrix: np.ndarray,
rt: np.ndarray,
df_end: float,
dt: float,
sigma: float,
float:
    """Search_for_the_root_of_the_function_using_a_numerical_method."""
max_iter = 10
max_error = 1e-8
x1 = x0 * 1.0001
f0 = f(x0, m, q_matrix, rt, df_end, dt, sigma)
f1 = f(x1, m, q_matrix, rt, df_end, dt, sigma)
for _ in range(0, max_iter):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x0	float	-	-
m	int	-	-
q_matrix	np.ndarray	-	-
rt	np.ndarray	-	-
df_end	float	-	-
dt	float	-	-
sigma	float	-	-
float	-	-	-
"""Search for the root of the function using a numerical method."""	-	-	-
max_iter	-	-	10
max_error	-	-	1e-8
x1	-	-	x0 * 1.000
f0	-	-	f(x0, m, q_matrix, rt, df)
f1	-	-	f(x1, m, q_matrix, rt, df)

bermudan_swaption_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
bermudan_swaption_tree_fast(
    t_exp: float,
    t_mat: float,
    strike_price: float,
    face_amount: float,
    cpn_times: np.ndarray,
    cpn_flows: np.ndarray,
    exercise_type_int: int,
    df_times: np.ndarray,
    df_values: np.ndarray,
    tree_times: np.ndarray,
    qq: np.ndarray,
    rt: np.ndarray,
    dt: float,
    tuple[float, float]:
    """Option to enter into a swap that can be exercised on coupon payment
    dates after the start of the exercise period. Due to non-analytical bond
    price we need to extend tree out to bond maturity and take into account
    cash flows through time."""
    pu = 0.50
    pd = 0.50
    num_time_steps, num_nodes = qq.shape
    expiry_step = int(t_exp / dt + 0.50)
    maturity_step = int(t_mat / dt + 0.50)
    fixed_leg_flows = np.zeros(num_time_steps)
    float_leg_values = np.zeros(num_time_steps)
    num_cpns = len(cpn_times)
    # Tree flows go all the way out to the swap maturity date
    for i in range(0, num_cpns):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	float	-	-
t_mat	float	-	-
strike_price	float	-	-
face_amount	float	-	-
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
exercise_type_int	int	-	-
df_times	np.ndarray	-	-
df_values	np.ndarray	-	-
tree_times	np.ndarray	-	-
qq	np.ndarray	-	-
rt	np.ndarray	-	-
dt	float	-	-
tuple[float, float]		-	-
"""Option to enter into a swap that can be exercised on coupon payment		-	-
dates after the start of the exercise period. Due to non-analytical bond		-	-
price we need to extend tree out to bond maturity and take into account		-	-
cash flows through time."""		-	-
pu	-	-	0.50
pd	-	-	0.50
num_time_steps	-	-	-
num_nodes	-	-	qq.shape
expiry_step	-	-	int(t_exp / dt + 0
maturity_step	-	-	int(t_mat / dt + 0
fixed_leg_flows	-	-	np.zeros(num_time
float_leg_values	-	-	np.zeros(num_time
num_cpns	-	-	len(cpn_times

american_bond_option_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
american_bond_option_tree_fast(
    t_exp: float,
    t_mat: float,
    strike_price: float,
    face_amount: float,
    cpn_times: np.ndarray,
    cpn_flows: np.ndarray,
    exercise_type_int: int,
    _df_times: np.ndarray,
    _df_values: np.ndarray,
    _tree_times: np.ndarray,
    _qq: np.ndarray,
    _rt: np.ndarray,
    _dt: float,
    tuple[float, float]:
```

```

"""Option_to_buy_or_sell_bond_at_a_specified_strike_price_that_can_be
exercised_over_the_exercise_period_dependending_on_choice_of_exercise_type.
Due_to_non-analytical_bond_price_we_need_to_extend_tree_out_to_bond
maturity_and_take_into_account_cash_flows_through_time."""
debug = False
pu = 0.50
pd = 0.50
if debug:
    print("COUPON_TIMES", cpn_times)
    print("COUPON_AMOUNTS", cpn_flows)
num_time_steps, num_nodes = _qq.shape
expiry_step = int(t_exp / _dt + 0.50)
maturity_step = int(t_mat / _dt + 0.50)
tree_flows = np.zeros(num_time_steps)
num_cpns = len(cpn_times)
# Tree flows go all the way out to the bond maturity date
# Do not include first coupon as it is the previous coupon and is negative
for i in range(1, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	float	-	-
t_mat	float	-	-
strike_price	float	-	-
face_amount	float	-	-
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
exercise_type_int	int	-	-
_df_times	np.ndarray	-	-
_df_values	np.ndarray	-	-
_tree_times	np.ndarray	-	-
_qq	np.ndarray	-	-
_rt	np.ndarray	-	-
_dt	float	-	-
tuple[float, float]		-	-
"""Option to buy or sell bond at a specified strike price that can be exercised over the exercise period depending on choice of exercise type.	-	-	-
Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time."""	-	-	-
debug	-	-	False
pu	-	-	0.50
pd	-	-	0.50
if debug	-	-	-
print("COUPON TIMES", cpn_times)	-	-	-
print("COUPON AMOUNTS", cpn_flows)	-	-	-
num_time_steps	-	-	-
num_nodes	-	-	_qq.shape
expiry_step	-	-	int(t_exp / _dt + 1)
maturity_step	-	-	int(t_mat / _dt + 1)
tree_flows	-	-	np.zeros(num_time_steps)
num_cpns	-	-	len(cpn_times)

callable_puttable_bond_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
callable_puttable_bond_tree_fast(
    cpn_times: np.ndarray,
    cpn_flows: np.ndarray,
    call_times: np.ndarray,
    call_prices: np.ndarray,
    put_times: np.ndarray,
    put_prices: np.ndarray,
    face_amount: float,
    _sigma: float,
    _a: float,
    _q_matrix: np.ndarray, # IS SIGMA USED ?
    _pu: float,
```

```

_pm: float,
_pd: float,
_rt: np.ndarray,
_dt: float,
_tree_times: np.ndarray,
_df_times: np.ndarray,
_df_values: np.ndarray,
    dict[str, float]:
    """Value_a_bond_with_embedded_put_and_call_options_that_can_be_exercised
    at_any_time_over_the_specified_list_of_put_and_call_dates.
    Due_to_non-analytical_bond_price_we_need_to_extend_tree_out_to_bond
    maturity_and_take_into_account_cash_flows_through_time."""
    pu = 0.50
    pd = 0.50
    num_time_steps, num_nodes = _q_matrix.shape
    dt = _dt
    t_mat = cpn_times[-1]
    maturity_step = int(t_mat / dt + 0.50)
    # Map coupons onto tree while preserving their present value
    tree_flows = np.zeros(num_time_steps)
    num_cpns = len(cpn_times)
    for i in range(0, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
cpn_times	np.ndarray	-	
cpn_flows	np.ndarray	-	
call_times	np.ndarray	-	
call_prices	np.ndarray	-	
put_times	np.ndarray	-	
put_prices	np.ndarray	-	
face_amount	float	-	
_sigma	float	-	
_a	float	-	
_q_matrix	np.ndarray	IS SIGMA USED ?	
_pu	float	-	
_pm	float	-	
_pd	float	-	
_rt	np.ndarray	-	
_dt	float	-	
_tree_times	np.ndarray	-	
_df_times	np.ndarray	-	
_df_values	np.ndarray	-	
dict[str, float]		-	
"""Value a bond with embedded put and call options that can be exercised		-	-
at any time over the specified list of put and call dates.		-	-
Due to non-analytical bond price we need to extend tree out to bond		-	-
maturity and take into account cash flows through time."""		-	-
pu	-	-	
pd	-	-	
num_time_steps	-	-	
num_nodes	-	-	_q_m
dt	-	-	
t_mat	-	-	cpn
maturity_step	-	-	int(t_m
tree_flows	-	-	np.zeros(r
num_cpns	-	-	len(c

build_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```

build_tree_fast(
    sigma: float,
    tree_times: np.ndarray,
    num_time_steps: int,
    discount_factors: np.ndarray,
    tuple[np.ndarray, np.ndarray, float]:
    """Unlike the BK and HW Trinomial trees, this Tree is packed into the lower
    diagonal of a square matrix because of its binomial nature. This means
    that the indexing of the arrays is different."""

```

```

tree_maturity = tree_times[-1]
dt = tree_maturity / (num_time_steps + 1)
# The short rate goes out one step extra to have the final short rate
# as it follows HW code, but I am not sure if this is needed. EXAMINE
# NOTE HW code uses this to have short rate at expiry, so it can use
# analytical solutions for the zero coupon bond price
# This is the BDT model so x = log(r)
# Some implementations use continuous compounding. Does not affect results.
# Useful if you want to make direct comparisons of the tree.
cont_compounded = True
qq = np.zeros(shape=(num_time_steps + 2, num_time_steps + 2))
rt = np.zeros(shape=(num_time_steps + 2, num_time_steps + 2))
if cont_compounded:
    r0 = -np.log(discount_factors[1]) / dt
else:
    r0 = (1.0 / discount_factors[1] - 1.0) / dt
rt[0, 0] = r0
qq[0, 0] = 1.0
if cont_compounded:
    qq[1, 0] = 0.50 * np.exp(-rt[0, 0] * dt)
    qq[1, 1] = 0.50 * np.exp(-rt[0, 0] * dt)
else:
    qq[1, 0] = 0.50 / ((1.0 + rt[0, 0]) ** dt)
    qq[1, 1] = 0.50 / ((1.0 + rt[0, 0]) ** dt)
# The short rate goes out one step extra to have the final short rate
# as it follows HW code, but I am not sure if this is needed. EXAMINE
for m in range(1, num_time_steps + 1):

```

The function arguments are described in the following table.

Argument Name	Type	Description
sigma	float	-
tree_times	np.ndarray	-
num_time_steps	int	-
discount_factors	np.ndarray	-
tuple[np.ndarray, np.ndarray, float]		-
"""Unlike the BK and HW Trinomial trees	-	-
this Tree is packed into the lower	-	-
diagonal of a square matrix because of its binomial nature. This means	-	-
that the indexing of the arrays is different."""	-	-
tree_maturity	-	-
dt	-	- tree_matu
cont_compounded	-	-
qq	-	- np.zeros(shape=(num
rt	-	- np.zeros(shape=(num
if cont_compounded		-
r0	-	- -np.log
else		-
r0	-	- (1.0 / dis
rt[0, 0]	-	-
qq[0, 0]	-	-
if cont_compounded		-
qq[1, 0]	-	- 0.50
qq[1, 1]	-	- 0.50
else		-
qq[1, 0]	-	- 0.50
qq[1, 1]	-	- 0.50

9.3 bk_tree

Class: BKTree

class BKTree:

BKTree

*Constructs the Black Karasinski rate model. The speed of mean reversion a and volatility are passed in. The short rate process is given by $d(\log(r)) = (\theta(t) - a \log(r)) * dt + \sigma * dW$*

```
BKTree(sigma: float, a: float, num_time_steps: int = 100) -> None:
    if not isinstance(sigma, float) or sigma < 0.0:
        raise FinError("Negative_volatility_not_allowed_and_must_be_float.")
    if not isinstance(a, float) or a < 0.0:
        raise FinError("Mean_reversion_speed_parameter_should_be_>=0_and_float.")
    if not isinstance(num_time_steps, int) or num_time_steps < 3:
        raise FinError("Drift_fitting_requires_at_least_3_time_steps_(int_>=3)")
    """Constructs the Black Karasinski rate model. The speed of mean
    reversion a and volatility are passed in. The short rate process
    is given by d(log(r)) = (theta(t) - a*log(r)) * dt + sigma * dW"""
    if sigma < 0.0:
        raise FinError("Negative_volatility_not_allowed.")
    if a < 0.0:
        raise FinError("Mean_reversion_speed_parameter_should_be_>=0.")
    a = max(a, 1e-10)
    .a = a
    .sigma = sigma
    if num_time_steps < 3:
        raise FinError("Drift_fitting_requires_at_least_3_time_steps")
    .num_time_steps = num_time_steps
    .qq = None
    .rt = None
    .tree_times = None
    .pu = None
    .pm = None
    .pd = None
    .discount_curve = None
    .df_times = None
    .dfs = None
    .dt = None
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	float	-	-
a	float	-	-
num_time_steps	int	-	100) -1, None:
if not isinstance(sigma, float) or sigma < 0.0	-	-	-
raise FinError("Negative volatility not allowed and must be float.")	-	-	-
if not isinstance(a, float) or a < 0.0	-	-	-
raise FinError("Mean reversion speed parameter should be > 0 and float.")	-	-	-
if not isinstance(num_time_steps, int) or num_time_steps < 3	-	-	-
raise FinError("Drift fitting requires at least 3 time steps (int > 3)")	-	-	-
"""Constructs the Black Karasinski rate model. The speed of mean reversion a and volatility are passed in. The short rate process is given by $d(\log(r))$ if sigma > 0.0	-	-	-
raise FinError("Negative volatility not allowed.")	-	-	-
if a < 0.0	-	-	-
raise FinError("Mean reversion speed parameter should be > 0.")	-	-	-
a	-	-	max(a, 1e-10)
.a	-	-	a
.sigma	-	-	sigma
if num_time_steps < 3	-	-	-
raise FinError("Drift fitting requires at least 3 time steps")	-	-	-
.num_time_steps	-	-	num_time_steps
.qq	-	-	None
.rt	-	-	None
.tree_times	-	-	None
.pu	-	-	None
.pm	-	-	None
.pd	-	-	None
.discount_curve	-	-	None
.df_times	-	-	None
.dfs	-	-	None
.dt	-	-	None

build_tree

PLEASE ADD A FUNCTION DESCRIPTION

```

build_tree(t_mat: float,
df_times: np.ndarray,
df_values: np.ndarray,
None:
if not isinstance(t_mat, float) or t_mat <= 0.0:
    raise FinError("Tree_maturity_t_mat_must_be_positive_float.")
if not isinstance(df_times, np.ndarray):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_mat	float	-	-
df_times	np.ndarray	-	-
df_values	np.ndarray	-	-
None		-	-
if not isinstance(t_mat, float) or t_mat ≤	-	-	0.0:
raise FinError("Tree maturity t_mat must be positive float.")	-	-	-

bond_option

Value a bond option that has European or American exercise using the Black-Karasinski model. The model uses a trinomial tree.

```
bond_option(t_exp: float,
strike_price: float,
face_amount: float,
cpn_times: np.ndarray,
cpn_flows: np.ndarray,
exercise_type: ExerciseTypes,
dict[str, float]):
if not isinstance(t_exp, float) or t_exp < 0.0:
    raise FinError("Option_expiry_time_t_exp_must_be_non-negative_float.")
if not isinstance(strike_price, float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	float	-	-
strike_price	float	-	-
face_amount	float	-	-
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
exercise_type	ExerciseTypes	-	-
dict[str, float]		-	-
if not isinstance(t_exp, float) or t_exp ≤		-	-
raise FinError("Option expiry time t_exp must be non-negative float.")	-	-	-

bermudan_swaption

Swaption that can be exercised on specific dates over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time.

```
bermudan_swaption(t_exp: float,
t_mat: float,
strike_price: float,
face_amount: float,
cpn_times: np.ndarray,
cpn_flows: np.ndarray,
```



```

exercise_type: ExerciseTypes,
    dict[str, float]:
if not isinstance(t_exp, float) or t_exp < 0.0:
    raise FinError("Option_expiry_time_t_exp_must_be_non-negative_float.")
if not isinstance(t_mat, float) or t_mat <= 0.0:
    raise FinError("Maturity_t_mat_must_be_positive_float.")
if not isinstance(strike_price, float):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	float	-	-
t_mat	float	-	-
strike_price	float	-	-
face_amount	float	-	-
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
exercise_type	ExerciseTypes	-	-
dict[str, float]		-	-
if not isinstance(t_exp, float) or t_exp < 0.0		-	-
raise FinError("Option expiry time t_exp must be non-negative float.")	-	-	-
if not isinstance(t_mat, float) or t_mat <= 0.0	-	-	0.0:
raise FinError("Maturity t_mat must be positive float.")	-	-	-

callable_puttable_bond_tree

Option that can be exercised at any time over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time.

```

callable_puttable_bond_tree(cpn_times: np.ndarray,
cpn_flows: np.ndarray,
call_times: np.ndarray,
call_prices: np.ndarray,
put_times: np.ndarray,
put_prices: np.ndarray,
face: float,
    dict[str, float]:
if not isinstance(cpn_times, np.ndarray):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
call_times	np.ndarray	-	-
call_prices	np.ndarray	-	-
put_times	np.ndarray	-	-
put_prices	np.ndarray	-	-
face	float	-	-
dict[str, float]		-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str		-	-

option_exercise_types_to_int

PLEASE ADD A FUNCTION DESCRIPTION

```
option_exercise_types_to_int(option_exercise_type: ExerciseTypes) -> int:
if option_exercise_type == ExerciseTypes.EUROPEAN:
    return 1
if option_exercise_type == ExerciseTypes.BERMUDAN:
    return 2
if option_exercise_type == ExerciseTypes.AMERICAN:
    return 3
else:
    raise FinError("Unknown_option_exercise_type.")
#####
t(
float64(float64, int64, float64[:], float64, float64, float64, int64),
fastmath=True,
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
option_exercise_type	ExerciseTypes)-ζint:	-	-
if option_exercise_type	-	-	= ExerciseTypes.EURC
return 1	-	-	-
if option_exercise_type	-	-	= ExerciseTypes.BERN
return 2	-	-	-
if option_exercise_type	-	-	= ExerciseTypes.AME
return 3	-	-	-
else	-	-	-
raise FinError("Unknown option exercise type.")	-	-	-
float64(float64, int64, float64[],float64,float64,float64,int64)	-	-	-
fastmath	-	-	True
cache	-	-	True

f

PLEASE ADD A FUNCTION DESCRIPTION

```
f(
alpha: float,
nm: int,
qq: np.ndarray,
pp: float,
dx: float,
dt: float,
n: int,
float:
sum_qz = 0.0
for j in range(-nm, nm + 1):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	float	-	-
nm	int	-	-
qq	np.ndarray	-	-
pp	float	-	-
dx	float	-	-
dt	float	-	-
n	int	-	-
float	-	-	-
sum_qz	-	-	0.0

fprime

PLEASE ADD A FUNCTION DESCRIPTION

```
fprime(
```

```

alpha: float,
nm: int,
qq: np.ndarray,
pp: float,
dx: float,
dt: float,
n: int,
float:
"""Compute the derivative of the objective function."""
sum_q_zd_z = 0.0
for j in range(-nm, nm + 1):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	float	-	-
nm	int	-	-
qq	np.ndarray	-	-
pp	float	-	-
dx	float	-	-
dt	float	-	-
n	int	-	-
float		-	-
"""Compute the derivative of the objective function."""	-	-	-
sum_q_zd_z	-	-	0.0

search_root

PLEASE ADD A FUNCTION DESCRIPTION

```

search_root(
x0: float,
nm: int,
qq: np.ndarray,
pp: float,
dx: float,
dt: float,
n: int,
float:
# print("Searching for root", x0)
max_iter = 50
max_error = 1e-8
x1 = x0 * 1.0001
f0 = f(x0, nm, qq, pp, dx, dt, n)
f1 = f(x1, nm, qq, pp, dx, dt, n)
for _ in range(0, max_iter):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x0	float	-	-
nm	int	-	-
qq	np.ndarray	-	-
pp	float	-	-
dx	float	-	-
dt	float	-	-
n	int	-	-
float		-	-
max_iter	-	-	50
max_error	-	-	1e-8
x1	-	-	x0 * 1.0001
f0	-	-	f(x0, nm, qq, pp, dx, dt, n)
f1	-	-	f(x1, nm, qq, pp, dx, dt, n)

search_root_deriv

PLEASE ADD A FUNCTION DESCRIPTION

```
search_root_deriv(
x0: float,
nm: int,
qq: np.ndarray,
pp: float,
dx: float,
dt: float,
n: int,
float:
max_iter = 50
max_error = 1e-8
for _ in range(0, max_iter):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x0	float	-	-
nm	int	-	-
qq	np.ndarray	-	-
pp	float	-	-
dx	float	-	-
dt	float	-	-
n	int	-	-
float		-	-
max_iter	-	-	50
max_error	-	-	1e-8

bermudan_swaption_tree_fast*PLEASE ADD A FUNCTION DESCRIPTION*

```

bermudan_swaption_tree_fast(
    t_exp: float,
    t_mat: float,
    strike_price: float,
    face_amount: float,
    cpn_times: np.ndarray,
    cpn_flows: np.ndarray,
    exercise_type_int: int,
    _df_times: np.ndarray,
    _df_values: np.ndarray,
    _tree_times: np.ndarray,
    _qq: np.ndarray,
    _pu: np.ndarray,
    _pm: np.ndarray,
    _pd: np.ndarray,
    _rt: np.ndarray,
    _dt: float,
    _a: float,
    tuple[float, float]:
    """Option_to_enter_into_a_swap_that_can_be_exercised_on_coupon_payment
    dates_after_the_start_of_the_exercise_period._Due_to_multiple_exercise
    times_we_need_to_extend_tree_out_to_bond_maturity_and_take_into_account
    cash_flows_through_time."""
    num_time_steps, num_nodes = _qq.shape
    j_max = ceil(0.1835 / (_a * _dt))
    expiry_step = int(t_exp / _dt + 0.50)
    maturity_step = int(t_mat / _dt + 0.50)
    # I shove the floating rate value into the grid, so it is handled in terms
    # of ppV - it will not sit on a grid date so needs to be ppV adjusted.
    # This is the value of the floating leg - this is a ONE CURVE approach.
    fixed_leg_flows = np.zeros(num_time_steps)
    # Initialise it with ones CHANGE
    float_leg_values = np.ones(num_time_steps)
    num_cpns = len(cpn_times)
    # swap fixed leg flows go all the way out to the swap maturity date
    for i in range(0, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	float	-	-
t_mat	float	-	-
strike_price	float	-	-
face_amount	float	-	-
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
exercise_type_int	int	-	-
_df_times	np.ndarray	-	-
_df_values	np.ndarray	-	-
_tree_times	np.ndarray	-	-
_qq	np.ndarray	-	-
_pu	np.ndarray	-	-
_pm	np.ndarray	-	-
_pd	np.ndarray	-	-
_rt	np.ndarray	-	-
_dt	float	-	-
_a	float	-	-
tuple[float, float]		-	-
"""Option to enter into a swap that can be exercised on coupon payment	-	-	-
dates after the start of the exercise period. Due to multiple exercise	-	-	-
times we need to extend tree out to bond maturity and take into account	-	-	-
cash flows through time."""	-	-	-
num_time_steps	-	-	-
num_nodes	-	-	_qq.shape
j_max	-	-	ceil(0.1835 / (_a * _dt))
expiry_step	-	-	int(t_exp / _dt + 1)
maturity_step	-	-	int(t_mat / _dt + 1)
fixed_leg_flows	-	-	np.zeros(num_time_steps)
float_leg_values	-	-	np.ones(num_time_steps)
num_cpns	-	-	len(cpn_times)

american_bond_option_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
american_bond_option_tree_fast(
    t_exp: float,
    t_mat: float,
    strike_price: float,
    face_amount: float,
    cpn_times: np.ndarray,
    cpn_flows: np.ndarray,
    exercise_type_int: int,
    _df_times: np.ndarray,
    _df_values: np.ndarray,
    _tree_times: np.ndarray,
    _qq: np.ndarray,
```

```

_pu: np.ndarray,
_pm: np.ndarray,
_pd: np.ndarray,
_rt: np.ndarray,
_dt: float,
_a: float,
tuple[float, float]:
    """Option that can be exercised at any time over the exercise period.
    Due to non-analytical bond price we need to extend tree out to bond
    maturity and take into account cash flows through time."""
    debug = False
    num_time_steps, num_nodes = _qq.shape
    j_max = ceil(0.1835 / (_a * _dt))
    expiry_step = int(t_exp / _dt + 0.50)
    maturity_step = int(t_mat / _dt + 0.50)
    tree_flows = np.zeros(num_time_steps)
    num_cpns = len(cpn_times)
    # Tree flows go all the way out to the bond maturity date
    # Do not include first coupon as it is the previous coupon and is negative
    for i in range(1, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	float	-	-
t_mat	float	-	-
strike_price	float	-	-
face_amount	float	-	-
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
exercise_type_int	int	-	-
_df_times	np.ndarray	-	-
_df_values	np.ndarray	-	-
_tree_times	np.ndarray	-	-
_qq	np.ndarray	-	-
_pu	np.ndarray	-	-
_pm	np.ndarray	-	-
_pd	np.ndarray	-	-
_rt	np.ndarray	-	-
_dt	float	-	-
_a	float	-	-
tuple[float, float]		-	-
"""Option that can be exercised at any time over the exercise period.	-	-	-
Due to non-analytical bond price we need to extend tree out to bond	-	-	-
maturity and take into account cash flows through time."""	-	-	-
debug	-	-	False
num_time_steps	-	-	-
num_nodes	-	-	_qq.shape
j_max	-	-	ceil(0.1835 / (_a * _dt
expiry_step	-	-	int(t_exp / _dt + 0.50
maturity_step	-	-	int(t_mat / _dt + 0.50
tree_flows	-	-	np.zeros(num_time_ste
num_cpns	-	-	len(cpn_times)

callable_puttable_bond_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
callable_puttable_bond_tree_fast(
cpn_times: np.ndarray,
cpn_flows: np.ndarray,
call_times: np.ndarray,
call_prices: np.ndarray,
put_times: np.ndarray,
put_prices: np.ndarray,
face_amount: float,
_sigma: float,
_a: float,
_qq: np.ndarray, # IS SIGMA USED ?
_pu: np.ndarray,
_pm: np.ndarray,
```

```

_pd: np.ndarray,
_rt: np.ndarray,
_dt: float,
_tree_times: np.ndarray,
_df_times: np.ndarray,
_df_values: np.ndarray,
    dict[str, float]:
    """Value_a_bond_with_embedded_put_and_call_options_that_can_be_exercised
    at_any_time_over_the_specified_list_of_put_and_call_dates.
    Due_to_non-analytical_bond_price_we_need_to_extend_tree_out_to_bond
    maturity_and_take_into_account_cash_flows_through_time."""
    num_time_steps, num_nodes = _qq.shape
    dt = _dt
    j_max = ceil(0.1835 / (_a * _dt))
    t_mat = cpn_times[-1]
    maturity_step = int(t_mat / dt + 0.50)
    # Map coupons onto tree while preserving their present value
    tree_flows = np.zeros(num_time_steps)
    num_cpns = len(cpn_times)
    for i in range(0, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
cpn_times	np.ndarray	-	
cpn_flows	np.ndarray	-	
call_times	np.ndarray	-	
call_prices	np.ndarray	-	
put_times	np.ndarray	-	
put_prices	np.ndarray	-	
face_amount	float	-	
_sigma	float	-	
_a	float	-	
_qq	np.ndarray	IS SIGMA USED ?	
_pu	np.ndarray	-	
_pm	np.ndarray	-	
_pd	np.ndarray	-	
_rt	np.ndarray	-	
_dt	float	-	
_tree_times	np.ndarray	-	
_df_times	np.ndarray	-	
_df_values	np.ndarray	-	
dict[str, float]		-	
"""Value a bond with embedded put and call options that can be exercised		-	-
at any time over the specified list of put and call dates.		-	-
Due to non-analytical bond price we need to extend tree out to bond		-	-
maturity and take into account cash flows through time."""		-	-
num_time_steps	-	-	
num_nodes	-	-	-q
dt	-	-	
j_max	-	-	ceil(0.18
t_mat	-	-	cpn
maturity_step	-	-	int(t_m
tree_flows	-	-	np.zeros(r
num_cpns	-	-	len(

build_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```

build_tree_fast(
a: float,
sigma: float,
tree_times: np.ndarray,
num_time_steps: int,
discount_factors: np.ndarray,
tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, float]:
"""Calibrate_the_tree_to_a_term_structure_of_interest_rates."""
tree_maturity = tree_times[-1]
dt = tree_maturity / (num_time_steps + 1)

```

```

dx = sigma * np.sqrt(3.0 * dt)
j_max = ceil(0.1835 / (a * dt))
if j_max > 1000:
    raise FinError("j_max > 1000. Increase a or dt.")
pu = np.zeros(shape=2 * j_max + 1)
pm = np.zeros(shape=2 * j_max + 1)
pd = np.zeros(shape=2 * j_max + 1)
# The short rate goes out one step extra to have the final short rate
# This is the BK model so x = log(r)
x = np.zeros(shape=(num_time_steps + 2, 2 * j_max + 1))
rt = np.zeros(shape=(num_time_steps + 2, 2 * j_max + 1))
# probabilities start at time 0 and go out to one step before T
# Branching is simple trinomial out to time step m=1 after which
# the top node and bottom node connect internally to two lower nodes
# and two upper nodes respectively. The probabilities only depend on j
for j in range(-j_max, j_max + 1):

```

The function arguments are described in the following table.

Argument Name	Type	Description	D
a	float	-	
sigma	float	-	
tree_times	np.ndarray	-	
num_time_steps	int	-	
discount_factors	np.ndarray	-	
tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, float]		-	
"""Calibrate the tree to a term structure of interest rates."""	-	-	
tree_maturity	-	-	tr
dt	-	-	tree_maturity
dx	-	-	sigma
j_max	-	-	ceil((
if j_max > 1000		-	
raise FinError("j_max > 1000. Increase a or dt.")	-	-	
pu	-	-	np.zeros(s
pm	-	-	np.zeros(s
pd	-	-	np.zeros(s
x	-	-	np.zeros(shape=(num
rt	-	-	np.zeros(shape=(num

9.4 black

Enumerated Type: BlackTypes

This enumerated type has the following values:

- ANALYTICAL
- CRR_TREE
- implementation_type:BlackTypes
- num_steps:int
- self.volatility
- self.implementation_type
- self.num_steps
- self.seed
- self.param1
- self.param2
- iftime_to_expiry;
- ifdf_i
- f
- t
- k
- v
- r
- ifself.implementation_type
- value
- results
- value
- iftime_to_expiry;
- ifdf_i
- f
- t
- k
- v
- r
- ifself.implementation_type
- ifself.implementation_type
- results
- iftime_to_expiry;
- ifdf_i
- f
- t
- k
- v
- r
- ifself.implementation_type
- ifself.implementation_type
- results

- iftime_to_expiry_i
- ifdf_i
- f
- t
- k
- v
- r
- ifself.implementation_type
- theta
- ifself.implementation_type
- results
- iftime_to_expiry_i
- ifdf_i
- f
- t
- k
- v
- r
- ifself.implementation_type
- vega
- ifself.implementation_type
- bump_size
- results
- results_volshift
- vega
- s
- s+
- s+
- s+
- d1,d2
- ifopt_type
- elifopt_type
- d1,-
- ifopt_type
- elifopt_type
- d1,-
- d1,-
- d1,d2
- ifopt_type
- elifopt_type
- @njit(float64[:])(float64,float64,float64,float64),fastmath
- t
- vol
- k
- sqrt.t
- iff_i

- ifk_i
- d1
- d2
- debug_print:bool
- fwd,t,k,r,opt_type,price
- value
- obj
- fwd,t,k,r,opt_type,_,
- vega
- fwd,t,k,_,opt_type,price
- num_steps
- results
- obj
- fwd,t,k,_,opt_type,_,
- bump_size
- num_steps
- results
- results_volshift
- vega
- <https://www.tandfonline.com/doi/abs/10.2469/faj.v44.n5.80?journalCode>
- ifeuropean_opt_type
- price
- elifeuropean_opt_type
- price
- sigma_guess
- sigma_guess
- _f
- _f_vega
- sigma0
- elifopt_type
- _f
- _f_vega
- sigma0
- elifopt_type
- _f
- _f_vega
- sigma0
- args
- tol
- sigma
- sigma
- method
- method
- method

Class: Black

Blacks Model which prices call and put options in the forward measure according to the Black-Scholes equation.

Black

Create *FinModel black* using parameters.

```
Black(volatility: float,
implementation_type: BlackTypes = BlackTypes.ANALYTICAL,
num_steps: int = 0,
None:
if volatility < 0.0:
    raise FinError("Volatility_must_be_non-negative.")
if num_steps < 0:
    raise FinError("num_steps_must_be_non-negative.")
"""Create_FinModel_black_using_parameters."""
.volatility = volatility
.implementation_type = implementation_type
.num_steps = num_steps
.seed = 0
.param1 = 0
.param2 = 0
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	float	-	-
implementation_type	BlackTypes	-	BlackTypes.ANALYTICAL
num_steps	int	-	0
None		-	-
if volatility $\geq$ 0.0		-	-
raise FinError("Volatility must be non-negative.")	-	-	-
if num_steps $\geq$ 0		-	-
raise FinError("num_steps must be non-negative.")	-	-	-
"""Create FinModel black using parameters."""	-	-	-
.volatility	-	-	volatility
.implementation_type	-	-	implementation_type
.num_steps	-	-	num_steps
.seed	-	-	0
.param1	-	-	0
.param2	-	-	0

value

PLEASE ADD A FUNCTION DESCRIPTION


```

value(forward_rate: float, # Forward rate F
strike_rate: float, # Strike Rate K
time_to_expiry: float, # Time to Expiry (years)
df: float, # df RFR to expiry date
opt_type,
    float: # Call or put
if forward_rate < 0.0:
    raise FinError("forward_rate_must_be_non-negative.")
if strike_rate < 0.0:
    raise FinError("strike_rate_must_be_non-negative.")
if time_to_expiry <= 0.0:
    raise FinError("time_to_expiry_must_be_positive.")
if df <= 0.0 or df > 1.0:
    raise FinError("df_(discount_factor)_must_be_in_(0,1].")
"""Price_a_derivative_using_Black's_model_which_values_in_the_forward
measure_following_a_change_of_measure."""
f = forward_rate
t = time_to_expiry
k = strike_rate
v = .volatility
r = -np.log(df) / t
if opt_type in (OptionTypes.EUROPEAN_CALL,
    OptionTypes.EUROPEAN_PUT,
):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
forward_rate	float	Forward rate F	-
strike_rate	float	Strike Rate K	-
time_to_expiry	float	Time to Expiry (years)	-
df	float	df RFR to expiry date	-
opt_type	-	-	-
float		Call or put	-
if forward_rate $\leq$ 0.0		-	-
raise FinError("forward_rate must be non-negative.")	-	-	-
if strike_rate $\leq$ 0.0		-	-
raise FinError("strike_rate must be non-negative.")	-	-	-
if time_to_expiry $\leq$	-	-	0.0:
raise FinError("time_to_expiry must be positive.")	-	-	-
if df $\leq$	-	-	0.0 or df $\geq$ 1.0:
raise FinError("df (discount factor) must be in (0, 1].")	-	-	-
"""Price a derivative using Black's model which values in the forward	-	-	-
measure following a change of measure."""	-	-	-
f	-	-	forward_rate
t	-	-	time_to_expiry
k	-	-	strike_rate
v	-	-	.volatility
r	-	-	-np.log(df) / t
OptionTypes.EUROPEAN_PUT	-	-	-

delta

PLEASE ADD A FUNCTION DESCRIPTION

```

delta(forward_rate: float, # Forward rate
strike_rate: float, # Strike Rate
time_to_expiry: float, # Time to Expiry (years)
df: float, # Discount factor to expiry date
opt_type,
float: # Call or put
if forward_rate < 0.0:
    raise FinError("forward_rate_must_be_non-negative.")
if strike_rate < 0.0:
    raise FinError("strike_rate_must_be_non-negative.")
if time_to_expiry <= 0.0:
    raise FinError("time_to_expiry_must_be_positive.")
if df <= 0.0 or df > 1.0:
    raise FinError("df_(discount_factor)_must_be_in_(0,1].")
"""Calculate_delta_using_Black's_model_which_values_in_the_forward
measure_following_a_change_of_measure."""
f = forward_rate
t = time_to_expiry
k = strike_rate
v = .volatility
r = -np.log(df) / t
if opt_type in (OptionTypes.EUROPEAN_CALL,
```

```
OptionTypes.EUROPEAN_PUT,
):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
forward_rate	float	Forward rate	-
strike_rate	float	Strike Rate	-
time_to_expiry	float	Time to Expiry (years)	-
df	float	Discount factor to expiry date	-
opt_type	-	-	-
float		Call or put	-
if forward_rate $\leq$ 0.0		-	-
raise FinError("forward_rate must be non-negative.")	-	-	-
if strike_rate $\leq$ 0.0		-	-
raise FinError("strike_rate must be non-negative.")	-	-	-
if time_to_expiry $\leq$	-	-	0.0:
raise FinError("time_to_expiry must be positive.")	-	-	-
if df $\leq$	-	-	0.0 or df $\geq$
raise FinError("df (discount factor) must be in (0, 1].")	-	-	-
"""Calculate delta using Black's model which values in the forward	-	-	-
measure following a change of measure."""	-	-	-
f	-	-	forward_rate
t	-	-	time_to_expiry
k	-	-	strike_rate
v	-	-	volatility
r	-	-	-np.log(df)
OptionTypes.EUROPEAN_PUT	-	-	-

gamma

PLEASE ADD A FUNCTION DESCRIPTION

```
gamma(forward_rate: float, # Forward rate F
strike_rate: float, # Strike Rate K
time_to_expiry: float, # Time to Expiry (years)
df: float, # RFR to expiry date
opt_type,
float: # Call or put
if forward_rate < 0.0:
    raise FinError("forward_rate_must_be_non-negative.")
if strike_rate < 0.0:
    raise FinError("strike_rate_must_be_non-negative.")
if time_to_expiry <= 0.0:
    raise FinError("time_to_expiry_must_be_positive.")
if df <= 0.0 or df > 1.0:
    raise FinError("df_(discount_factor)_must_be_in_(0,_1].")
"""Calculate_gamma_using_Black's_model_which_values_in_the_forward
```

```

measure_following_a_change_of_measure."""
f = forward_rate
t = time_to_expiry
k = strike_rate
v = .volatility
r = -np.log(df) / t
if opt_type in (OptionTypes.EUROPEAN_CALL,
                OptionTypes.EUROPEAN_PUT,
                ):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
forward_rate	float	Forward rate F	-
strike_rate	float	Strike Rate K	-
time_to_expiry	float	Time to Expiry (years)	-
df	float	RFR to expiry date	-
opt_type	-	-	-
float		Call or put	-
if forward_rate $\leq$ 0.0		-	-
raise FinError("forward_rate must be non-negative.")	-	-	-
if strike_rate $\leq$ 0.0		-	-
raise FinError("strike_rate must be non-negative.")	-	-	-
if time_to_expiry $\leq$	-	-	0.0:
raise FinError("time_to_expiry must be positive.")	-	-	-
if df $\leq$	-	-	0.0 or df $\geq$ 1.0:
raise FinError("df (discount factor) must be in (0, 1].")	-	-	-
"""Calculate gamma using Black's model which values in the forward	-	-	-
measure following a change of measure."""	-	-	-
f	-	-	forward_rate
t	-	-	time_to_expiry
k	-	-	strike_rate
v	-	-	.volatility
r	-	-	-np.log(df) / t
OptionTypes.EUROPEAN_PUT	-	-	-

theta

PLEASE ADD A FUNCTION DESCRIPTION

```

theta(forward_rate: float, # Forward rate F
      strike_rate: float, # Strike Rate K
      time_to_expiry: float, # Time to Expiry (years)
      df: float, # Discount Factor to expiry date
      opt_type,
      float: # Call or put
      if forward_rate < 0.0:
          raise FinError("forward_rate_must_be_non-negative.")

```

```

if strike_rate < 0.0:
    raise FinError("strike_rate_must_be_non-negative.")
if time_to_expiry <= 0.0:
    raise FinError("time_to_expiry_must_be_positive.")
if df <= 0.0 or df > 1.0:
    raise FinError("df_(discount_factor)_must_be_in_(0,1].")
"""Calculate_theta_using_Black's_model_which_values_in_the_forward
measure_following_a_change_of_measure."""
f = forward_rate
t = time_to_expiry
k = strike_rate
v = .volatility
r = -np.log(df) / t
if opt_type in (OptionTypes.EUROPEAN_CALL,
                OptionTypes.EUROPEAN_PUT,
                ):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
forward_rate	float	Forward rate F	-
strike_rate	float	Strike Rate K	-
time_to_expiry	float	Time to Expiry (years)	-
df	float	Discount Factor to expiry date	-
opt_type	-	-	-
float		Call or put	-
if forward_rate $\geq$ 0.0		-	-
raise FinError("forward_rate must be non-negative.")	-	-	-
if strike_rate $\geq$ 0.0		-	-
raise FinError("strike_rate must be non-negative.")	-	-	-
if time_to_expiry $\geq$	-	-	0.0:
raise FinError("time_to_expiry must be positive.")	-	-	-
if df $\geq$	-	-	0.0 or df $\leq$
raise FinError("df (discount factor) must be in (0, 1].")	-	-	-
"""Calculate theta using Black's model which values in the forward	-	-	-
measure following a change of measure."""	-	-	-
f	-	-	forward_r
t	-	-	time_to_ex
k	-	-	strike_ra
v	-	-	.volatili
r	-	-	-np.log(df
OptionTypes.EUROPEAN_PUT	-	-	-

vega

PLEASE ADD A FUNCTION DESCRIPTION

```
vega(forward_rate: float, # Forward rate F
```

```

strike_rate: float, # Strike Rate K
time_to_expiry: float, # Time to Expiry (years)
df: float, # df RFR to expiry date
opt_type,
    float: # Call or put
if forward_rate < 0.0:
    raise FinError("forward_rate_must_be_non-negative.")
if strike_rate < 0.0:
    raise FinError("strike_rate_must_be_non-negative.")
if time_to_expiry <= 0.0:
    raise FinError("time_to_expiry_must_be_positive.")
if df <= 0.0 or df > 1.0:
    raise FinError("df_(discount_factor)_must_be_in_(0,1].")
"""Calculate vega using Black's model which values in the forward
measure following a change of measure."""
f = forward_rate
t = time_to_expiry
k = strike_rate
v = .volatility
r = -np.log(df) / t
if opt_type in (OptionTypes.EUROPEAN_CALL,
    OptionTypes.EUROPEAN_PUT,
):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
forward_rate	float	Forward rate F	-
strike_rate	float	Strike Rate K	-
time_to_expiry	float	Time to Expiry (years)	-
df	float	df RFR to expiry date	-
opt_type	-	-	-
float		Call or put	-
if forward_rate $\leq$ 0.0		-	-
raise FinError("forward_rate must be non-negative.")	-	-	-
if strike_rate $\leq$ 0.0		-	-
raise FinError("strike_rate must be non-negative.")	-	-	-
if time_to_expiry $\leq$	-	-	0.0:
raise FinError("time_to_expiry must be positive.")	-	-	-
if df $\leq$	-	-	0.0 or df $\leq$ 1.0:
raise FinError("df (discount factor) must be in (0, 1].")	-	-	-
"""Calculate vega using Black's model which values in the forward	-	-	-
measure following a change of measure."""	-	-	-
f	-	-	forward_rate
t	-	-	time_to_expiry
k	-	-	strike_rate
v	-	-	.volatility
r	-	-	-np.log(df) / t
OptionTypes.EUROPEAN_PUT	-	-	-

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

f_european

Function to determine ststar for pricing European options on future contracts.

```
_f_european(sigma, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	-	-	-
args	-	-	-

f_european_vega

Function to calculate the Vega of European options on future contracts.

```
_f_european_vega(sigma, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	-	-	-
args	-	-	-

f_american

Function to determine ststar for pricing American options on future contracts.

```
_f_american(sigma, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	-	-	-
args	-	-	-

`_f_american_vega`

Function to calculate the Vega of American options on future contracts.

```
_f_american_vega(sigma, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	-	-	-
args	-	-	-

`_estimate_vol_from_price`

PLEASE ADD A FUNCTION DESCRIPTION

```
_estimate_vol_from_price(fwd, t, k, european_opt_type, european_price):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd	-	-	-
t	-	-	-
k	-	-	-
european_opt_type	-	-	-
european_price	-	-	-

`black_value`

Price a derivative using Black model.

```
black_value(
fwd: float,
t: float,
k: float,
r: float,
v: float,
opt_type,
float:
    """Price_a_derivative_using_Black_model."""
d1, d2 = calculate_d1_d2(fwd, t, k, v)
if opt_type == OptionTypes.EUROPEAN_CALL:
    return np.exp(-r * t) * (fwd * normcdf_vect(d1) - k * normcdf_vect(d2))
elif opt_type == OptionTypes.EUROPEAN_PUT:
    return np.exp(-r * t) * (k * normcdf_vect(-d2) - fwd * normcdf_vect(-d1))
else:
    raise FinError("Option_type_must_be_a_European_Call_or_Put")
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
v	float	-	-
opt_type	-	-	-
float	-	-	-
"""Price a derivative using Black model."""			
d1	-	-	-
d2	-	-	calculate_d1_d2(fwd, t, k, v)
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL
return np.exp(-r * t) * (fwd * normcdf_vect(d1) - k * normcdf_vect(d2))	-	-	-
elif opt_type	-	-	= OptionTypes.EUROPEAN_PUT
return np.exp(-r * t) * (k * normcdf_vect(-d2) - fwd * normcdf_vect(-d1))	-	-	-
else	-	-	-
raise FinError("Option type must be a European Call or Put")	-	-	-

black_delta

Return delta of a derivative using Black model.

```

black_delta(
    fwd: float,
    t: float,
    k: float,
    r: float,
    v: float,
    opt_type:
        float:
    """Return_delta_of_a_derivative_using_Black_model."""
    d1, _ = calculate_d1_d2(fwd, t, k, v)
    if opt_type == OptionTypes.EUROPEAN_CALL:
        return np.exp(-r * t) * normcdf_vect(d1)
    elif opt_type == OptionTypes.EUROPEAN_PUT:
        return -np.exp(-r * t) * normcdf_vect(-d1)
    else:
        raise FinError("Option_type_must_be_a_European_Call_or_Put")
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
v	float	-	-
opt_type	-	-	-
float	-	-	-
"""Return delta of a derivative using Black model."""	-	-	-
d1	-	-	-
-	-	-	calculate_d1_d2(fwd, t, k, v)
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL
return np.exp(-r * t) * normcdf_vect(d1)	-	-	-
elif opt_type	-	-	= OptionTypes.EUROPEAN_PUT
return -np.exp(-r * t) * normcdf_vect(-d1)	-	-	-
else	-	-	-
raise FinError("Option type must be a European Call or Put")	-	-	-

black_gamma

PLEASE ADD A FUNCTION DESCRIPTION

```

black_gamma(
    fwd: float,
    t: float,
    k: float,
    r: float,
    v: float,
    opt_type,
    float:
    """Return gamma of a derivative using Black model."""
    d1, _ = calculate_d1_d2(fwd, t, k, v)
    if opt_type in (OptionTypes.EUROPEAN_CALL, OptionTypes.EUROPEAN_PUT):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
v	float	-	-
opt_type	-	-	-
float	-	-	-
"""Return gamma of a derivative using Black model."""	-	-	-
d1	-	-	-
-	-	-	calculate_d1_d2(fwd, t, k, v)

black_vega

PLEASE ADD A FUNCTION DESCRIPTION

```
black_vega(
fwd: float,
t: float,
k: float,
r: float,
v: float,
opt_type,
float:
"""Return vega of a derivative using Black model."""
d1, _ = calculate_d1_d2(fwd, t, k, v)
if opt_type in (OptionTypes.EUROPEAN_CALL, OptionTypes.EUROPEAN_PUT):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
v	float	-	-
opt_type	-	-	-
float	-	-	-
"""Return vega of a derivative using Black model."""	-	-	-
d1	-	-	-
-	-	-	calculate_d1_d2(fwd, t, k, v)

black_theta

Return theta of a derivative using Black model.

```
black_theta(
fwd: float,
t: float,
k: float,
r: float,
v: float,
opt_type,
float:
"""Return theta of a derivative using Black model."""
d1, d2 = calculate_d1_d2(fwd, t, k, v)
if opt_type == OptionTypes.EUROPEAN_CALL:
    return np.exp(-r * t) * (
        -(fwd * v * normcdf_prime_vect(d1)) / (2 * np.sqrt(t))
        + r * fwd * normcdf_vect(d1)
        - r * k * normcdf_vect(d2)
    )
elif opt_type == OptionTypes.EUROPEAN_PUT:
```

```

    return np.exp(-r * t) * (
        -(fwd * v * normcdf_prime_vect(d1)) / (2 * np.sqrt(t))
        - r * fwd * normcdf_vect(-d1)
        + r * k * normcdf_vect(-d2)
    )
else:
    raise FinError("Option_type_must_be_a_European_Call_or_Put")
#####
t(float64[:])(float64, float64, float64, float64), fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description
fwd	float	-
t	float	-
k	float	-
r	float	-
v	float	-
opt_type	-	-
float	-	-
"""Return theta of a derivative using Black model."""	-	-
d1	-	-
d2	-	-
if opt_type	-	-
-(fwd * v * normcdf_prime_vect(d1)) / (2 * np.sqrt(t))	-	-
+ r * fwd * normcdf_vect(d1)	-	-
- r * k * normcdf_vect(d2)	-	-
)	-	-
elif opt_type	-	-
-(fwd * v * normcdf_prime_vect(d1)) / (2 * np.sqrt(t))	-	-
- r * fwd * normcdf_vect(-d1)	-	-
+ r * k * normcdf_vect(-d2)	-	-
)	-	-
else	-	-
raise FinError("Option type must be a European Call or Put")	-	-
t(float64[	float64 or float64,float64,float64	-

calculate_d1_d2

Calculate d1 and d2 for Black-Scholes model.

```

calculate_d1_d2(
f: float,
t: float,
k: float,
v: float,
np.ndarray:
"""Calculate_d1_and_d2_for_Black-Scholes_model."""

```

```

t = np.maximum(t, G_SMALL)
vol = np.maximum(v, G_SMALL)
k = np.maximum(k, G_SMALL)
sqrt_t = np.sqrt(t)
if f <= 0.0:
    raise FinError("Forward_is_zero.")
if k <= 0.0:
    raise FinError("Strike_is_zero.")
d1 = (np.log(f / k) + vol * vol * t / 2.0) / (vol * sqrt_t)
d2 = d1 - vol * sqrt_t
return np.array([d1, d2])
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
f	float	-	-
t	float	-	-
k	float	-	-
v	float	-	-
np.ndarray	-	-	-
"""Calculate d1 and d2 for Black-Scholes model."""	-	-	-
t	-	-	np.maximum(t, G_SMALL)
vol	-	-	np.maximum(v, G_SMALL)
k	-	-	np.maximum(k, G_SMALL)
sqrt_t	-	-	np.sqrt(t)
if f ≤	-	-	0.0:
raise FinError("Forward is zero.")	-	-	-
if k ≤	-	-	0.0:
raise FinError("Strike is zero.")	-	-	-
d1	-	-	$(\ln(f/k) + \text{vol}^2 \cdot t / 2) / (\text{vol} \cdot \sqrt{t})$
d2	-	-	$d1 - \text{vol} \cdot \sqrt{t}$
return np.array([d1, d2])	-	-	-

implied_volatility

Calculate the Black implied volatility of a European/American options on futures contracts using Newton with a fallback to bisection.

```

implied_volatility(
    fwd: float,
    t: float,
    r: float,
    k: float,
    price: float,
    opt_type,
    debug_print: bool = True,
    float:
    """Calculate the Black implied volatility of a European/American

```

```
options_on_futures_contracts_using_Newton_with
a_fallback_to_bisection. """
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd	float	-	-
t	float	-	-
r	float	-	-
k	float	-	-
price	float	-	-
opt_type	-	-	-
debug_print	bool	-	True
float		-	-
"""Calculate the Black implied volatility of a European/American	-	-	-
options on futures contracts using Newton with	-	-	-
a fallback to bisection.'"""	-	-	-

9.5 black_scholes

Enumerated Type: BlackScholesTypes

This enumerated type has the following values:

- DEFAULT
- ANALYTICAL
- CRR_TREE
- BARONE_ADESI
- LSMC
- BJERKSUND_STENSLAND
- FINITE_DIFFERENCE
- PSOR
- bs_type:BlackScholesTypes
- num_steps_per_year
- num_paths
- seed
- use_sobol
- params
- self.volatility
- self.bs_type
- self.num_steps_per_year
- self.num_paths
- self.seed
- self.use_sobol
- self.params
- self.poly_degree
- self.fit_type
- self.bs_type
- ifself.bs_type
- v
- ifself.bs_type
- v
- ifself.bs_type
- v
- spot_price
- time_to_expiry
- strike_price
- risk_free_rate
- dividend_yield
- volatility
- opt_type
- ifself.bs_type
- v
- spot_price
- time_to_expiry

- strike_price
- risk_free_rate
- dividend_yield
- volatility
- opt_type
- ifself.bs_type
- poly_degree
- fit_type
- v
- spot_price
- risk_free_rate
- dividend_yield
- sigma
- num_steps_per_year
- num_paths
- time_to_expiry
- opt_type_value
- strike_price
- poly_degree
- fit_type_value
- use_sobol
- seed
- self.bs_type
- ifself.bs_type
- phi
- v
- ifself.bs_type
- v
- ifself.bs_type
- poly_degree
- fit_type
- v
- spot_price
- risk_free_rate
- dividend_yield
- sigma
- num_paths
- num_steps_per_year
- time_to_expiry
- opt_type_value
- strike_price
- poly_degree
- fit_type_value
- use_sobol
- seed
- ifself.bs_type

- v
- ifself.bs_type
- v
- spot_price
- time_to_expiry
- strike_price
- risk_free_rate
- dividend_yield
- volatility
- opt_type
- ifself.bs_type
- v
- spot_price
- time_to_expiry
- strike_price
- risk_free_rate
- dividend_yield
- volatility
- opt_type

Class: BlackScholes(Model)

Black-Scholes model class supporting various pricing methods.

BlackScholes

PLEASE ADD A FUNCTION DESCRIPTION

```
BlackScholes(volatility: Union[float, np.ndarray],
bs_type: BlackScholesTypes = BlackScholesTypes.DEFAULT,
num_steps_per_year=52,
num_paths=10000,
seed=42,
use_sobol=False,
params=None,
check_argument_types(.BlackScholes, locals())
.volatility = volatility
.bs_type = bs_type
.num_steps_per_year = num_steps_per_year
.num_paths = num_paths
.seed = seed
.use_sobol = use_sobol
.params = params if params else {}
.poly_degree = .params.get("poly_degree", 3)
.fit_type = .params.get("fit_type", BoundaryFitTypes.HERMITE_E)
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	float or np.ndarray	-	-
bs_type	BlackScholesTypes	-	BlackScholesTypes.DEFAULT
num_steps_per_year	-	-	52
num_paths	-	-	10000
seed	-	-	42
use_sobol	-	-	False
params	-	-	None
check_argument_types(.BlackScholes, locals())	-	-	-
.volatility	-	-	volatility
.bs_type	-	-	bs_type
.num_steps_per_year	-	-	num_steps_per_year
.num_paths	-	-	num_paths
.seed	-	-	seed
.use_sobol	-	-	use_sobol
.params	-	-	params if params is not None
.poly_degree	-	-	.params.get("poly_degree", 1)
.fit_type	-	-	.params.get("fit_type", BoundaryFitType.BOUNDARY_FIT)

value

Compute the option value based on the specified Black-Scholes type.

```

value(spot_price: float,
time_to_expiry: float,
strike_price: float,
risk_free_rate: float,
dividend_rate: float,
opt_type: OptionTypes,
"""
Compute the option value based on the specified Black-Scholes type.
"""
if opt_type in [OptionTypes.EUROPEAN_CALL, OptionTypes.EUROPEAN_PUT]:
    if .bs_type is BlackScholesTypes.DEFAULT:
        .bs_type = BlackScholesTypes.ANALYTICAL
    if .bs_type == BlackScholesTypes.ANALYTICAL:
        v = bs_value(spot_price,
                      time_to_expiry,
                      strike_price,
                      risk_free_rate,
                      dividend_rate,
                      .volatility,
                      opt_type.value,
                      )
        return v
    if .bs_type == BlackScholesTypes.CRR_TREE:
        v = crr_tree_val_avg(spot_price,
                             risk_free_rate,
                             dividend_rate,
                             .volatility,
                             .num_steps_per_year,

```

```

        time_to_expiry,
        opt_type.value,
        strike_price,
    )["value"]
    return v
if .bs_type == BlackScholesTypes.FINITE_DIFFERENCE:
    v = black_scholes_fd(spot_price=spot_price,
        time_to_expiry=time_to_expiry,
        strike_price=strike_price,
        risk_free_rate=risk_free_rate,
        dividend_yield=dividend_rate,
        volatility=.volatility,
        opt_type=opt_type.value,
        **.params
    )
    return v
if .bs_type == BlackScholesTypes.PSOR:
    v = black_scholes_fd_psor(spot_price=spot_price,
        time_to_expiry=time_to_expiry,
        strike_price=strike_price,
        risk_free_rate=risk_free_rate,
        dividend_yield=dividend_rate,
        volatility=.volatility,
        opt_type=opt_type.value,
        **.params
    )
    return v
if .bs_type == BlackScholesTypes.LSMC:
    print("lsmc_Model", )
    poly_degree = .poly_degree
    fit_type = .fit_type
    v = equity_lsmc(spot_price=spot_price,
        risk_free_rate=risk_free_rate,
        dividend_yield=dividend_rate,
        sigma=.volatility,
        num_steps_per_year=.num_steps_per_year,
        num_paths=.num_paths,
        time_to_expiry=time_to_expiry,
        opt_type_value=opt_type.value,
        strike_price=strike_price,
        poly_degree=poly_degree,
        fit_type_value=fit_type.value,
        use_sobol=.use_sobol,
        seed=.seed,
    )
    return v
    raise FinError("Implementation_not_available_for_this_product")
if opt_type in [OptionTypes.AMERICAN_CALL, OptionTypes.AMERICAN_PUT]:
    if .bs_type is BlackScholesTypes.DEFAULT:
        .bs_type = BlackScholesTypes.CRR_TREE
    if .bs_type == BlackScholesTypes.BARONE_ADESI:
        phi = 1 if opt_type == OptionTypes.AMERICAN_CALL else -1
        v = baw_value(spot_price,
            time_to_expiry,
            strike_price,
            risk_free_rate,
            dividend_rate,

```

```

        .volatility,
        phi,
    )
    return v
if .bs_type == BlackScholesTypes.CRR_TREE:
    v = crr_tree_val_avg(spot_price,
        risk_free_rate,
        dividend_rate,
        .volatility,
        .num_steps_per_year,
        time_to_expiry,
        opt_type.value,
        strike_price,
    )["value"]
    return v
if .bs_type == BlackScholesTypes.LSMC:
    poly_degree = 3
    fit_type = BoundaryFitTypes.HERMITE_E
    v = equity_lsmc(spot_price=spot_price,
        risk_free_rate=risk_free_rate,
        dividend_yield=dividend_rate,
        sigma=.volatility,
        num_paths=.num_paths,
        num_steps_per_year=.num_steps_per_year,
        time_to_expiry=time_to_expiry,
        opt_type_value=opt_type.value,
        strike_price=strike_price,
        poly_degree=poly_degree,
        fit_type_value=fit_type.value,
        use_sobol=.use_sobol,
        seed=.seed,
    )
    return v
if .bs_type == BlackScholesTypes.BJERKSUND_STENSLAND:
    v = bjerksund_stensland_value(spot_price,
        time_to_expiry,
        strike_price,
        risk_free_rate,
        dividend_rate,
        .volatility,
        opt_type.value,
    )
    return v
if .bs_type == BlackScholesTypes.FINITE_DIFFERENCE:
    v = black_scholes_fd(spot_price=spot_price,
        time_to_expiry=time_to_expiry,
        strike_price=strike_price,
        risk_free_rate=risk_free_rate,
        dividend_yield=dividend_rate,
        volatility=.volatility,
        opt_type=opt_type.value,
        **.params
    )
    return v
if .bs_type == BlackScholesTypes.PSOR:
    v = black_scholes_fd_psor(spot_price=spot_price,
        time_to_expiry=time_to_expiry,

```

```
        strike_price=strike_price,
        risk_free_rate=risk_free_rate,
        dividend_yield=dividend_rate,
        volatility=.volatility,
        opt_type=opt_type.value,
        **.params
    )
    return v
    raise FinError("Implementation_not_available_for_this_product")
raise FinError("Should_not_be_here")
```

The function arguments are described in the following table.

Argument Name	Type	Description
spot_price	float	-
time_to_expiry	float	-
strike_price	float	-
risk_free_rate	float	-
dividend_rate	float	-
opt_type	OptionTypes	-
"""	-	-
Compute the option value based on the specified Black-Scholes type.	-	-
"""	-	-
if opt_type in [OptionTypes.EUROPEAN_CALL, OptionTypes.EUROPEAN_PUT]		-
if .bs_type is BlackScholesTypes.DEFAULT		-
.bs_type	-	-
if .bs_type	-	-
time_to_expiry	-	-
strike_price	-	-
risk_free_rate	-	-
dividend_rate	-	-
.volatility	-	-
opt_type.value	-	-
)	-	-
return v	-	-
if .bs_type	-	-
risk_free_rate	-	-
dividend_rate	-	-
.volatility	-	-
.num_steps_per_year	-	-
time_to_expiry	-	-
opt_type.value	-	-
strike_price	-	-
return v	-	-
if .bs_type	-	-
time_to_expiry	-	-
strike_price	-	-
risk_free_rate	-	-
dividend_yield	-	-
volatility	-	-
opt_type	-	-
**.params	-	-
)	-	-
return v	-	-
if .bs_type	-	-
time_to_expiry	-	-
strike_price	-	-
risk_free_rate	-	-
dividend_yield	-	-
volatility	-	-
opt_type	-	-
**.params	-	-
)	-	-
return v	-	-
if .bs_type	-	-

9.6 black_scholes_analytic

bs_value

Price a derivative using Black-Scholes model. Parameters: - spot_price: float - the current price of the underlying asset - time_to_expiry: float - time to option expiry in years - strike_price: float - the option's strike price - risk_free_rate: float - risk-free interest rate - dividend_rate: float - dividend yield of the underlying asset - volatility: Union[float, np.ndarray] - volatility of the underlying asset (annualized) - opt_type: str - type of option ('call' or 'put') Returns: - float - the calculated option price

```
bs_value(
s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int,
float:
"""Price a derivative using Black-Scholes model.
Parameters:
- spot_price: float - the current price of the underlying asset
- time_to_expiry: float - time to option expiry in years
- strike_price: float - the option's strike price
- risk_free_rate: float - risk-free interest rate
- dividend_rate: float - dividend yield of the underlying asset
- volatility: Union[float, np.ndarray] - volatility of the underlying
asset (annualized)
- opt_type: str - type of option ('call' or 'put')
Returns:
- float - the calculated option price
"""
if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
    phi = 1.0
elif opt_type_value == OptionTypes.EUROPEAN_PUT.value:
    phi = -1.0
else:
    raise FinError("Unknown_option_type_value")
k = np.maximum(k, G_SMALL)
t = np.maximum(t, G_SMALL)
v = np.maximum(v, G_SMALL)
v_sqrt_t = v * np.sqrt(t)
ss = s * np.exp(-q * t)
kk = k * np.exp(-r * t)
d1 = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
d2 = d1 - v_sqrt_t
value = phi * ss * normcdf(phi * d1) - phi * kk * normcdf(phi * d2)
return value
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description
s	float	-
t	float	-
k	float	-
r	float	-
q	float	-
v	float	-
opt_type_value	int	-
float		-
""" Price a derivative using Black-Scholes model.	-	-
Parameters		-
- spot_price	float-the current price of the underlying asset	-
- time_to_expiry	float-time to option expiry in years	-
- strike_price	float-the option's strike price	-
- risk_free_rate	float-risk-free interest rate	-
- dividend_rate	float-dividend yield of the underlying asset	-
- volatility	float or np.ndarray	-
asset (annualized)	-	-
- opt_type	'call' or 'put' or str-type of option ('call' or 'put')	-
Returns		-
- float - the calculated option price	-	-
"""	-	-
if opt_type_value	-	-
phi	-	-
elif opt_type_value	-	-
phi	-	-
else		-
raise FinError("Unknown option type value")	-	-
k	-	-
t	-	-
v	-	-
v_sqrt_t	-	-
ss	-	-
kk	-	-
d1	-	-
d2	-	-
value	-	-
return value	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-
fastmath	-	-
cache	-	-

bs_delta

Price a derivative using Black-Scholes model.


```

bs_delta(
s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int,
float:
"""Price_a_derivative_using_Black-Scholes_model."""
if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
    phi = +1.0
elif opt_type_value == OptionTypes.EUROPEAN_PUT.value:
    phi = -1.0
else:
    raise FinError("Error:_Unknown_option_type_value")
k = np.maximum(k, G_SMALL)
t = np.maximum(t, G_SMALL)
v = np.maximum(v, G_SMALL)
v_sqrt_t = v * np.sqrt(t)
ss = s * np.exp(-q * t)
kk = k * np.exp(-r * t)
d1 = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
delta = phi * np.exp(-q * t) * normcdf_vect(phi * d1)
return delta
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description
s	float	-
t	float	-
k	float	-
r	float	-
q	float	-
v	float	-
opt_type_value	int	-
float		-
"""Price a derivative using Black-Scholes model."""	-	-
if opt_type_value	-	= Option
phi	-	-
elif opt_type_value	-	= Option
phi	-	-
else		-
raise FinError("Error	Unknownoptiontypevalue")	-
k	-	n
t	-	n
v	-	n
v_sqrt_t	-	-
ss	-	-
kk	-	-
d1	-	np.log
delta	-	phi * np.e
return delta	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-
fastmath	-	-
cache	-	-

bs_gamma

Price a derivative using Black-sscholes model.

```

bs_gamma(
s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int = None,
float:
"""Price_a_derivative_using_Black-sscholes_model."""
k = np.maximum(k, G_SMALL)
t = np.maximum(t, G_SMALL)
v = np.maximum(v, G_SMALL)
v_sqrt_t = v * np.sqrt(t)
ss = s * np.exp(-q * t)

```

```

kk = k * np.exp(-r * t)
d1 = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
gamma = np.exp(-q * t) * normcdf_prime_vect(d1) / s / v_sqrt_t
return gamma
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
opt_type_value	int	-	None
float		-	-
"""Price a derivative using Black-sscholes model."""	-	-	-
k	-	-	np.maximum(k, G_
t	-	-	np.maximum(t, G_
v	-	-	np.maximum(v, G_
v_sqrt_t	-	-	v * np.sqrt(
ss	-	-	s * np.exp(-q
kk	-	-	k * np.exp(-r
d1	-	-	np.log(ss / kk) / v_sqrt_t
gamma	-	-	np.exp(-q * t) * normcdf_prime
return gamma	-	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-	-
fastmath	-	-	True
cache	-	-	True

bs_vega

Price a derivative using Black-Scholes model.

```

bs_vega(
s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int,
float:

```

```

"""Price_a_derivative_using_Black-Scholes_model."""
k = np.maximum(k, G_SMALL)
t = np.maximum(t, G_SMALL)
v = np.maximum(v, G_SMALL)
sqrt_t = np.sqrt(t)
v_sqrt_t = v * sqrt_t
ss = s * np.exp(-q * t)
kk = k * np.exp(-r * t)
dl = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
vega = ss * sqrt_t * normcdf_prime_vect(dl)
return vega
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
opt_type_value	int	-	-
float		-	-
"""Price a derivative using Black-Scholes model."""	-	-	-
k	-	-	np.maximum(k, G_SMALL)
t	-	-	np.maximum(t, G_SMALL)
v	-	-	np.maximum(v, G_SMALL)
sqrt_t	-	-	np.sqrt(t)
v_sqrt_t	-	-	v * sqrt_t
ss	-	-	s * np.exp(-q * t)
kk	-	-	k * np.exp(-r * t)
dl	-	-	np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
vega	-	-	ss * sqrt_t * normcdf_prime_vect(dl)
return vega	-	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-	-
fastmath	-	-	True
cache	-	-	True

bs_theta

Price a derivative using Black-Scholes model.

```
bs_theta(
```

```

s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int,
    float:
    """Price_a_derivative_using_Black-Scholes_model."""
    phi = 0
    if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
        phi = 1.0
    elif opt_type_value == OptionTypes.EUROPEAN_PUT.value:
        phi = -1.0
    k = np.maximum(k, G_SMALL)
    t = np.maximum(t, G_SMALL)
    v = np.maximum(v, G_SMALL)
    sqrt_t = np.sqrt(t)
    v_sqrt_t = v * sqrt_t
    ss = s * np.exp(-q * t)
    kk = k * np.exp(-r * t)
    d1 = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
    d2 = d1 - v_sqrt_t
    theta = -ss * normcdf_prime_vect(d1) * v / 2.0 / sqrt_t
    theta = theta - phi * r * k * np.exp(-r * t) * normcdf_vect(phi * d2)
    theta = theta + phi * q * ss * normcdf_vect(phi * d1)
    return theta
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
opt_type_value	int	-	-
float	-	-	-
"""Price a derivative using Black-Scholes model."""	-	-	-
phi	-	-	0
if opt_type_value	-	-	= OptionTypes.EUROPEAN_CALL.value
phi	-	-	1.0
elif opt_type_value	-	-	= OptionTypes.EUROPEAN_PUT.value
phi	-	-	-1.0
k	-	-	np.maximum(k, 0)
t	-	-	np.maximum(t, 0)
v	-	-	np.maximum(v, 0)
sqrt_t	-	-	np.sqrt(t)
v_sqrt_t	-	-	v * sqrt_t
ss	-	-	s * np.exp(-r * t)
kk	-	-	k * np.exp(-r * t)
d1	-	-	np.log(ss / kk) / v_sqrt_t + 0.5
d2	-	-	d1 - v_sqrt_t
theta	-	-	-ss * normcdf_prime(-d1)
theta	-	-	theta - phi * r * k * np.exp(-r * t)
theta	-	-	theta + phi * q * ss * np.exp(-r * t)
return theta	-	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-	-
fastmath	-	-	True
cache	-	-	True

bs_rho

Price a derivative using Black-Scholes model.

```
bs_rho(
s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int,
float:
"""Price a derivative using Black-Scholes model."""
phi = 0
if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
```

```

    phi = 1.0
elif opt_type_value == OptionTypes.EUROPEAN_PUT.value:
    phi = -1.0
k = np.maximum(k, G_SMALL)
t = np.maximum(t, G_SMALL)
v = np.maximum(v, G_SMALL)
sqrt_t = np.sqrt(t)
v_sqrt_t = v * sqrt_t
ss = s * np.exp(-q * t)
kk = k * np.exp(-r * t)
d1 = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
d2 = d1 - v_sqrt_t
rho = phi * k * t * np.exp(-r * t) * normcdf_vect(phi * d2)
return rho
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
opt_type_value	int	-	-
float		-	-
"""Price a derivative using Black-Scholes model."""	-	-	-
phi	-	-	0
if opt_type_value	-	-	= OptionTypes.EUROPEAN
phi	-	-	1.0
elif opt_type_value	-	-	= OptionTypes.EUROPEAN
phi	-	-	-1.0
k	-	-	np.maximum(k, G_SMALL)
t	-	-	np.maximum(t, G_SMALL)
v	-	-	np.maximum(v, G_SMALL)
sqrt_t	-	-	np.sqrt(t)
v_sqrt_t	-	-	v * sqrt_t
ss	-	-	s * np.exp(-q * t)
kk	-	-	k * np.exp(-r * t)
d1	-	-	np.log(ss / kk) / v_sqrt_t + 0.5 * v
d2	-	-	d1 - v_sqrt_t
rho	-	-	phi * k * t * np.exp(-r * t) * np.exp(v_sqrt_t * d1)
return rho	-	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-	-
fastmath	-	-	True
cache	-	-	True

bs_vanna

Price a derivative using Black-sscholes model.

```

bs_vanna(
s: float,
t: float,
k: float,
r: float,
q: float,
v: float,
opt_type_value: int,
float:
"""Price a derivative using Black-sscholes model."""
k = np.maximum(k, G_SMALL)
t = np.maximum(t, G_SMALL)
v = np.maximum(v, G_SMALL)
sqrt_t = np.sqrt(t)

```



```

v_sqrt_t = v * sqrt_t
ss = s * np.exp(-q * t)
kk = k * np.exp(-r * t)
d1 = np.log(ss / kk) / v_sqrt_t + v_sqrt_t / 2.0
d2 = d1 - v_sqrt_t
vanna = np.exp(-q * t) * sqrt_t * normcdf_prime_vect(d1) * (d2 / v)
return vanna
#####
t(fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
opt_type_value	int	-	-
float		-	-
"""Price a derivative using Black-sscholes model."""	-	-	-
k	-	-	np.maximum(k, G_SMALL)
t	-	-	np.maximum(t, G_SMALL)
v	-	-	np.maximum(v, G_SMALL)
sqrt_t	-	-	np.sqrt(t)
v_sqrt_t	-	-	v * sqrt_t
ss	-	-	s * np.exp(-q * t)
kk	-	-	k * np.exp(-r * t)
d1	-	-	np.log(ss / kk) / v_sqrt_t + v_sqrt_t
d2	-	-	d1 - v_sqrt_t
vanna	-	-	np.exp(-q * t) * sqrt_t * normcdf_prime_vect(d1)
return vanna	-	-	-
t(fastmath	-	-	True, cache=True)

f

PLEASE ADD A FUNCTION DESCRIPTION

```
_f(sigma, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	-	-	-
args	-	-	-

_fvega*PLEASE ADD A FUNCTION DESCRIPTION*

```
_fvega(sigma, args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
sigma	-	-	-
args	-	-	-

bs_intrinsic

Calculate the Black-Scholes implied volatility of a European vanilla option using Newton with a fallback to bisection.

```
bs_intrinsic(
s: float,
t: float,
k: float,
r: float,
q: float,
opt_type_value: int,
float:
"""Calculate the Black-Scholes implied volatility of a European
vanilla option using Newton with a fallback to bisection."""
fwd = s * np.exp((r - q) * t)
if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
    intrinsic_value = np.exp(-r * t) * max(fwd - k, 0.0)
else:
    intrinsic_value = np.exp(-r * t) * max(k - fwd, 0.0)
return intrinsic_value
#####
torize(
[float64(float64, float64, float64, float64, float64, float64, int64)],
fastmath=True,
cache=True,
forceobj=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
opt_type_value	int	-	-
float	-	-	-
"""Calculate the Black-Scholes implied volatility of a European vanilla option using Newton with a fallback to bisection."""	-	-	-
fwd	-	-	$s * np.exp((r - q) * t)$
if opt_type_value	-	-	$= \text{OptionTypes.EUROPEAN_CALL.value}$
intrinsic_value	-	-	$np.exp(-r * t) * \max(\text{fwd} - k, 0.0)$
else	-	-	-
intrinsic_value	-	-	$np.exp(-r * t) * \max(k - \text{fwd}, 0.0)$
return intrinsic_value	-	-	-
[float64(float64, float64, float64, float64, float64, float64, int64)]	-	-	-
fastmath	-	-	True
cache	-	-	True
forceobj	-	-	True

bs_implied_volatility

Calculate the Black-Scholes implied volatility of a European vanilla option using Newton with a fallback to bisection.

```
bs_implied_volatility(
    s: float,
    t: float,
    k: float,
    r: float,
    q: float,
    price: float,
    opt_type_value: int,
    float:
    """Calculate the Black-Scholes implied volatility of a European
    vanilla option using Newton with a fallback to bisection."""
    fwd = s * np.exp((r - q) * t)
    if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
        intrinsic_value = np.exp(-r * t) * max(fwd - k, 0.0)
    else:
        intrinsic_value = np.exp(-r * t) * max(k - fwd, 0.0)
    div_adj_stock_price = s * np.exp(-q * t)
    df = np.exp(-r * t)
    # Flip ITmm call option to be OTmm put and vice-versa using put call parity
    if intrinsic_value > 0.0:
        if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
            price = price - (div_adj_stock_price - k * df)
            opt_type_value = OptionTypes.EUROPEAN_PUT.value
        else:
```

```

        price = price + (div_adj_stock_price - k * df)
        opt_type_value = OptionTypes.EUROPEAN_CALL.value
        # Update intrinsic based on new option type
        if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
            intrinsic_value = np.exp(-r * t) * max(fwd - k, 0.0)
        else:
            intrinsic_value = np.exp(-r * t) * max(k - fwd, 0.0)
    time_value = price - intrinsic_value
    # Add a tolerance in case it is just numerical imprecision
    if time_value < 0.0:
        print("Time_value", time_value)
        raise FinError("Option_Price_is_below_the_intrinsic_value")
    # some approximations which might be used later
    if opt_type_value == OptionTypes.EUROPEAN_CALL.value:
        call = price
    else:
        call = price + (div_adj_stock_price - k * df)
    # Notation in ssssRN-id567721.pdf
    xx = k * np.exp(-r * t)
    ss = s * np.exp(-q * t)
    pi = np.pi
    # Initial point of inflexion
    # arg = np.abs(np.log(fwd/k))
    # sigma0 = np.sqrt(2.0 * arg)
    # Corrado mmiller from Hallerbach equation (7)
    cmsigma = 0.0
    # arg = (C - 0.5*(ss-xx))**2 - ((ss-xx)**2) / pi
    # if arg < 0.0:
    #     arg = 0.0
    # cmsigma = (C-0.5*(ss-xx) + np.sqrt(arg))
    # cmsigma = cmsigma * np.sqrt(2.0*pi) / (ss+xx)
    # cmsigma = cmsigma / np.sqrt(t)
    # hh allerbach ssssRN-id567721.pdf equation (22)
    hsigma = 0.0
    gamma = 2.0
    arg = (2 * call + xx - ss) ** 2 - gamma * (ss + xx) * (ss - xx) * (
        ss - xx
    ) / pi / ss
    arg = max(arg, 0.0)
    hsigma = 2.0 * call + xx - ss + np.sqrt(arg)
    hsigma = hsigma * np.sqrt(2.0 * pi) / 2.0 / (ss + xx)
    hsigma = hsigma / np.sqrt(t)
    sigma0 = hsigma
    arglist = [s, t, k, r, q, price, opt_type_value]
    argsv = np.array(arglist)
    tol = 1e-6
    sigma = newton(_f, sigma0, _fvega, argsv, tol=tol)
    if sigma is None:
        sigma = bisection(_f, 1e-4, 10.0, argsv, xt看=tol)
        if sigma is None:
            method = "Failed"
        else:
            method = "Bisection"
    else:
        method = "Newton"
    debug = False
    if debug:

```

```

print (
    "ss:_%7.2f_kk:_%7.3f_tt:_%5.3f_V:%10.7f_ss%0:_%7.5f_Cmm:_%7.5f_hh_L:_%7.5f_Nww
      :_%7.5f_%10s"
    % (
        s,
        k,
        t,
        price,
        sigma0 * 100.0,
        cmsigma * 100.0,
        hsigma * 100.0,
        sigma * 100.0,
        method,
    )
)
return sigma
his module contains a number of analytical approximations for the price of
American style option starting with Barone-Adesi-whhaley
tps://deriscope.com/docs/Barone_Adesi_wwhaley_1987.pdf
#####
t(fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type
s	float
t	float
k	float
r	float
q	float
price	float
opt_type_value	int
float	
"""Calculate the Black-Scholes implied volatility of a European vanilla option using Newton with a fallback to bisection."""	-
fwd	-
if opt_type_value	-
intrinsic_value	-
else	
intrinsic_value	-
div_adj_stock_price	-
df	-
if intrinsic_value < 0.0	
if opt_type_value	-
price	-
opt_type_value	-
else	
price	-
opt_type_value	-
if opt_type_value	-
intrinsic_value	-
else	
intrinsic_value	-
time_value	-
if time_value > 0.0	
print("Time value", time_value)	-
raise FinError("Option Price is below the intrinsic value")	-
if opt_type_value	-
call	-
else	
call	-
xx	-
ss	-
pi	-
cmsigma	-
hsigma	-
gamma	-
ss - xx	-
) / pi / ss	-
arg	-
hsigma	-
hsigma	-
hsigma	-
sigma0	-
arglist	-

_fcall

Function to determine ststar for pricing American call options.

```
_fcall(si, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
si	-	-	-
*args	-	-	-

_fput

Function to determine sstar for pricing American put options.

```
_fput(si, *args):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
si	-	-	-
*args	-	-	-

baw_value

American Option Pricing Approximation using the Barone-Adesi-Whaley approximation for the Black-Scholes model

```
baw_value(s, t, k, r, q, v, phi):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
t	-	-	-
k	-	-	-
r	-	-	-
q	-	-	-
v	-	-	-
phi	-	-	-

bjersund_stensland_value

Price American Option using the Bjersund-Stensland approximation (1993) for the Black-Scholes model

```
bjerkhund_stensland_value(s, t, k, r, q, v, opt_type_value):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
t	-	-	-
k	-	-	-
r	-	-	-
q	-	-	-
v	-	-	-
opt_type_value	-	-	-

9.7 black_scholes_mc

`_value_mc_nonumba_nonumpy`

PLEASE ADD A FUNCTION DESCRIPTION

```
_value_mc_nonumba_nonumpy(
s: float,
t: float,
k: float,
opt_type: int,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
use_sobol: int,
    float:
# SLOWEST - No use of NUMPY vectorisation or NUMBA
num_paths = int(num_paths)
np.random.seed(seed)
mu = r - q
v2 = v**2
v_sqrt_t = v * np.sqrt(t)
payoff = 0.0
if use_sobol == 1:
    g = get_gaussian_sobol(num_paths, 1)[: , 0]
else:
    g = np.random.standard_normal(num_paths)
ss = s * exp((mu - v2 / 2.0) * t)
if opt_type == OptionTypes.EUROPEAN_CALL.value:
    for i in range(0, num_paths):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
opt_type	int	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
use_sobol	int	-	-
float		-	-
num_paths	-	-	int(num_paths)
np.random.seed(seed)	-	-	-
mu	-	-	r - q
v2	-	-	v**2
v_sqrt_t	-	-	v * np.sqrt(t)
payoff	-	-	0.0
if use_sobol	-	-	= 1:
g	-	-	get_gaussian_sobol(num_paths, 1)[: , 0]
else	-	-	-
g	-	-	np.random.standard_normal(num_paths)
ss	-	-	s * exp((mu - v2 / 2.0) * t)
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL.value:

`_value_mc_numpy_only`

PLEASE ADD A FUNCTION DESCRIPTION

```

_value_mc_numpy_only(
s: float,
t: float,
k: float,
opt_type: int,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
use_sobol: int,
float:
# Use of NUMPY ONLY
num_paths = int(num_paths)
np.random.seed(seed)
mu = r - q
v2 = v**2
v_sqrt_t = v * np.sqrt(t)
if use_sobol == 1:
    g = get_gaussian_sobol(num_paths, 1)[: , 0]
else:

```

```

    g = np.random.standard_normal(num_paths)
    ss = s * np.exp((mu - v2 / 2.0) * t)
    m = np.exp(g * v_sqrt_t)
    s_1 = ss * m
    s_2 = ss / m
    # Not sure if it is correct to do antithetics with sobols but why not ?
    if opt_type == OptionTypes.EUROPEAN_CALL.value:
        payoff_a_1 = np.maximum(s_1 - k, 0.0)
        payoff_a_2 = np.maximum(s_2 - k, 0.0)
    elif opt_type == OptionTypes.EUROPEAN_PUT.value:
        payoff_a_1 = np.maximum(k - s_1, 0.0)
        payoff_a_2 = np.maximum(k - s_2, 0.0)
    else:
        raise FinError("Unknown_option_type.")
    payoff = np.mean(payoff_a_1) + np.mean(payoff_a_2)
    v = payoff * np.exp(-r * t) / 2.0
    return v
#####
t(
float64(
    float64,
    float64,
    float64,
    int64,
    float64,
    float64,
    float64,
    int64,
    int64,
    int64,
),
cache=True,
fastmath=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
opt_type	int	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
use_sobol	int	-	-
float		-	-
num_paths	-	-	int(num_paths)
np.random.seed(seed)	-	-	-
mu	-	-	$r - q$
v2	-	-	v^{**2}
v_sqrt_t	-	-	$v * \text{np.sqrt}(t)$
if use_sobol	-	-	= 1:
g	-	-	get_gaussian_sobol(num_paths, 1)[: , 0]
else	-	-	-
g	-	-	np.random.standard_normal(num_paths)
ss	-	-	$s * \text{np.exp}((\mu - v2 / 2.0) * t)$
m	-	-	$\text{np.exp}(g * v_sqrt_t)$
s_1	-	-	$ss * m$
s_2	-	-	ss / m
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL.value:
payoff_a_1	-	-	$\text{np.maximum}(s_1 - k, 0.0)$
payoff_a_2	-	-	$\text{np.maximum}(s_2 - k, 0.0)$
elif opt_type	-	-	= OptionTypes.EUROPEAN_PUT.value:
payoff_a_1	-	-	$\text{np.maximum}(k - s_1, 0.0)$
payoff_a_2	-	-	$\text{np.maximum}(k - s_2, 0.0)$
else	-	-	-
raise FinError("Unknown option type.")	-	-	-
payoff	-	-	$\text{np.mean}(\text{payoff_a_1}) + \text{np.mean}(\text{payoff_a_2})$
v	-	-	$\text{payoff} * \text{np.exp}(-r * t) / 2.0$
return v	-	-	-
float64	-	-	-
float64	-	-	-
float64	-	-	-
int64	-	-	-
float64	-	-	-
float64	-	-	-
float64	-	-	-
int64	-	-	-
int64	-	-	-
int64	-	-	-
)	-	-	-
cache	-	-	True
fastmath	-	-	True

`_value_mc_numpy_numba`*PLEASE ADD A FUNCTION DESCRIPTION*

```

_value_mc_numpy_numba(
s: float,
t: float,
k: float,
opt_type: int,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
use_sobol: int,
float:
# Use of NUMPY ONLY
num_paths = int(num_paths)
np.random.seed(seed)
mu = r - q
v2 = v**2
v_sqrt_t = v * np.sqrt(t)
if use_sobol == 1:
    g = get_gaussian_sobol(num_paths, 1)[: , 0]
else:
    g = np.random.standard_normal(num_paths)
ss = s * np.exp((mu - v2 / 2.0) * t)
m = np.exp(g * v_sqrt_t)
s_1 = ss * m
s_2 = ss / m
# Not sure if it is correct to do antithetics with sobols but why not ?
if opt_type == OptionTypes.EUROPEAN_CALL.value:
    payoff_a_1 = np.maximum(s_1 - k, 0.0)
    payoff_a_2 = np.maximum(s_2 - k, 0.0)
elif opt_type == OptionTypes.EUROPEAN_PUT.value:
    payoff_a_1 = np.maximum(k - s_1, 0.0)
    payoff_a_2 = np.maximum(k - s_2, 0.0)
else:
    raise FinError("Unknown_option_type.")
payoff = np.mean(payoff_a_1) + np.mean(payoff_a_2)
v = payoff * np.exp(-r * t) / 2.0
return v
#####
t(
float64(
    float64,
    float64,
    float64,
    int64,
    float64,
    float64,
    float64,
    int64,
    int64,
    int64,
),

```

```
fastmath=True,  
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
opt_type	int	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
use_sobol	int	-	-
float		-	-
num_paths	-	-	int(num_paths)
np.random.seed(seed)	-	-	-
mu	-	-	$r - q$
v2	-	-	v^{**2}
v_sqrt_t	-	-	$v * \text{np.sqrt}(t)$
if use_sobol	-	-	= 1:
g	-	-	get_gaussian_sobol(num_paths, 1)[: , 0]
else	-	-	-
g	-	-	np.random.standard_normal(num_paths)
ss	-	-	$s * \text{np.exp}((\mu - v2 / 2.0) * t)$
m	-	-	$\text{np.exp}(g * v_sqrt_t)$
s_1	-	-	$ss * m$
s_2	-	-	ss / m
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL.value:
payoff_a_1	-	-	$\text{np.maximum}(s_1 - k, 0.0)$
payoff_a_2	-	-	$\text{np.maximum}(s_2 - k, 0.0)$
elif opt_type	-	-	= OptionTypes.EUROPEAN_PUT.value:
payoff_a_1	-	-	$\text{np.maximum}(k - s_1, 0.0)$
payoff_a_2	-	-	$\text{np.maximum}(k - s_2, 0.0)$
else	-	-	-
raise FinError("Unknown option type.")	-	-	-
payoff	-	-	$\text{np.mean}(\text{payoff_a_1}) + \text{np.mean}(\text{payoff_a_2})$
v	-	-	$\text{payoff} * \text{np.exp}(-r * t) / 2.0$
return v	-	-	-
float64	-	-	-
float64	-	-	-
float64	-	-	-
int64	-	-	-
float64	-	-	-
float64	-	-	-
float64	-	-	-
int64	-	-	-
int64	-	-	-
int64	-	-	-
)	-	-	-
fastmath	-	-	True
cache	-	-	True

`_value_mc_numba_only`*PLEASE ADD A FUNCTION DESCRIPTION*

```

_value_mc_numba_only(
s: float,
t: float,
k: float,
opt_type: int,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
use_sobol: int,
float:
# No use of Numpy vectorisation but NUMBA
num_paths = int(num_paths)
np.random.seed(seed)
mu = r - q
v2 = v**2
v_sqrt_t = v * np.sqrt(t)
payoff = 0.0
if use_sobol == 1:
    g = get_gaussian_sobol(num_paths, 1)[: , 0]
else:
    g = np.random.standard_normal(num_paths)
ss = s * np.exp((mu - v2 / 2.0) * t)
if opt_type == OptionTypes.EUROPEAN_CALL.value:
    for i in range(0, num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
opt_type	int	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
use_sobol	int	-	-
float		-	-
num_paths	-	-	int(num_paths)
np.random.seed(seed)	-	-	-
mu	-	-	r - q
v2	-	-	v**2
v_sqrt_t	-	-	v * np.sqrt(t)
payoff	-	-	0.0
if use_sobol	-	-	= 1:
g	-	-	get_gaussian_sobol(num_paths, 1)[: , 0]
else	-	-	-
g	-	-	np.random.standard_normal(num_paths)
ss	-	-	s * np.exp((mu - v2 / 2.0) * t)
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL.value:

`_value_mc_numba_parallel`

PLEASE ADD A FUNCTION DESCRIPTION

```

_value_mc_numba_parallel(
s: float,
t: float,
k: float,
opt_type: int,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
use_sobol: int,
float:
# No use of Numpy vectorisation but NUMBA
num_paths = int(num_paths)
np.random.seed(seed)
mu = r - q
v2 = v**2
v_sqrt_t = v * np.sqrt(t)
if use_sobol == 1:
    g = get_gaussian_sobol(num_paths, 1)[: , 0]
else:

```

```

g = np.random.standard_normal(num_paths)
ss = s * np.exp((mu - v2 / 2.0) * t)
payoff1 = 0.0
payoff2 = 0.0
if opt_type == OptionTypes.EUROPEAN_CALL.value:
    for i in range(0, num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
k	float	-	-
opt_type	int	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
use_sobol	int	-	-
float	-	-	-
num_paths	-	-	int(num_paths)
np.random.seed(seed)	-	-	-
mu	-	-	r - q
v2	-	-	v**2
v_sqrt_t	-	-	v * np.sqrt(t)
if use_sobol	-	-	= 1:
g	-	-	get_gaussian_sobol(num_paths, 1)[: , 0]
else	-	-	-
g	-	-	np.random.standard_normal(num_paths)
ss	-	-	s * np.exp((mu - v2 / 2.0) * t)
payoff1	-	-	0.0
payoff2	-	-	0.0
if opt_type	-	-	= OptionTypes.EUROPEAN_CALL.value:

9.8 black_scholes_mc_tests

value_mc1

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc1(
s0: float,
t: float,
k: float,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
float:
if not all(isinstance(x, float) for x in [s0, t, k, r, q, v]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
float		-	-

value_mc2

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc2(
s0: float,
t: float,
k: float,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
float:
if not all(isinstance(x, float) for x in [s0, t, k, r, q, v]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
float		-	-

value_mc3

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc3(
s0: float,
t: float,
k: float,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
float:
if not all(isinstance(x, float) for x in [s0, t, k, r, q, v]):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
float		-	-

value_mc4

PLEASE ADD A FUNCTION DESCRIPTION

```
value_mc4(
s0: float,
t: float,
k: float,
r: float,
```

```

q: float,
v: float,
num_paths: int,
seed: int,
    float:
if not all(isinstance(x, float) for x in [s0, t, k, r, q, v]):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
float		-	-

value_mc5

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc5(
s0: float,
t: float,
k: float,
r: float,
q: float,
v: float,
num_paths: int,
seed: int,
    float:
if not all(isinstance(x, float) for x in [s0, t, k, r, q, v]):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
t	float	-	-
k	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
num_paths	int	-	-
seed	int	-	-
float		-	-

9.9 black_shifted

Class: BlackShifted

Blacks Model which prices call and put options in the forward measure according to the Black-Scholes equation. This model also allows the distribution to be shifted to the negative in order to allow for negative interest rates.

BlackShifted

Create FinModel black using parameters.

```
BlackShifted(volatility: float, shift: float, implementation: int = 0) -> None:
    """Create_FinModel_black_using_parameters."""
    .volatility = volatility
    .shift = shift
    .implementation = 0
    .num_steps = 0
    .seed = 0
    .param1 = 0
    .param2 = 0
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
volatility	float	-	-
shift	float	-	-
implementation	int	-	0) -> None:
"""Create FinModel black using parameters."""	-	-	-
.volatility	-	-	volatility
.shift	-	-	shift
.implementation	-	-	0
.num_steps	-	-	0
.seed	-	-	0
.param1	-	-	0
.param2	-	-	0

value

Price a derivative using Black's model which values in the forward measure following a change of measure. The sign of the shift is the same as Matlab.

```
value(forward_rate: float, # Forward rate
      strike_rate: float, # Strike Rate
      time_to_expiry: float, # time to expiry in years
      df: float, # Discount Factor to expiry date
      call_or_put: OptionTypes, # Call or put
      float:
      """Price_a_derivative_using_Black's_model_which_values_in_the_forward
```

```

measure_following_a_change_of_measure._The_sign_of_the_shift_is_the
same_as_Matlab."""
s = .shift
f = forward_rate
t = time_to_expiry
k = strike_rate
sqrt_t = np.sqrt(t)
vol = .volatility
d1 = np.log((f + s) / (k + s)) + vol * vol * t / 2
d1 = d1 / (vol * sqrt_t)
d2 = d1 - vol * sqrt_t
if call_or_put == OptionTypes.EUROPEAN_CALL:
    return df * ((f + s) * normcdf(d1) - (k + s) * normcdf(d2))
elif call_or_put == OptionTypes.EUROPEAN_PUT:
    return df * ((k + s) * normcdf(-d2) - (f + s) * normcdf(-d1))
raise ValueError("Option_type_must_be_European_Call(C)_or_Put(P)")
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
forward_rate	float	Forward rate
strike_rate	float	Strike Rate
time_to_expiry	float	time to expiry in years
df	float	Discount Factor to expiry date
call_or_put	OptionTypes	Call or put
float		-
"""Price a derivative using Black's model which values in the forward	-	-
measure following a change of measure. The sign of the shift is the	-	-
same as Matlab."""	-	-
s	-	-
f	-	-
t	-	-
k	-	-
sqrt_t	-	-
vol	-	-
d1	-	-
d1	-	-
d2	-	-
if call_or_put	-	-
return df * ((f + s) * normcdf(d1) - (k + s) * normcdf(d2))	-	-
elif call_or_put	-	-
return df * ((k + s) * normcdf(-d2) - (f + s) * normcdf(-d1))	-	-
raise ValueError("Option type must be European Call(C) or Put(P)")	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```

__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("VOLATILITY", .volatility)
s += label_to_string("SHIFT", .shift)
s += label_to_string("IMPLEMENTATION", .implementation)
s += label_to_string("NUMSTEPS", .num_steps)
return s

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str	-	-	-
s	-	-	label_to_string("OBJECT_TYPE", type().__name__)
s +	-	-	label_to_string("VOLATILITY", .volatility)
s +	-	-	label_to_string("SHIFT", .shift)
s +	-	-	label_to_string("IMPLEMENTATION", .implementation)
s +	-	-	label_to_string("NUMSTEPS", .num_steps)
return s	-	-	-

9.10 cir_montecarlo

Enumerated Type: CIRNumericalScheme

This enumerated type has the following values:

- EULER
- LOGNORMAL
- MILSTEIN
- KAHLJACKEL
- EXACT
- self._a
- self._b
- self._sigma
- s
- s+
- s+
- s+
- @njit(fastmath
- mr
- @njit(fastmath
- vr
- vr+
- fastmath
- cache
- sigma2
- h
- denom
- aa
- bb
- zcb
- fastmath
- cache
- sigma2
- d
- ll
- c
- z
- x
- r
- n
- x
- r
- parallel
- fastmath
- cache
- num_steps

- rate_path
- rate_path[0]
- num_paths
- ifscheme
- sigmasqrt_dt
- r
- z
- r
- rate_path[i_step]
- elifscheme
- x
- y
- r
- z
- mean
- var
- sig
- r
- rate_path[i_step]
- elifscheme
- sigmasqrt_dt
- sigma2dt
- r
- z
- r
- r
- rate_path[i_step]
- elifscheme
- bhat
- sqrt_dt
- r
- z
- beta
- rootr
- c
- r
- rate_path[i_step]
- elifscheme
- r
- r
- rate_path[i_step]
- parallel
- fastmath
- cache
- ift
- num_steps

- zcb
- ifscheme
- sigmasqrt_dt
- r
- rsum
- z
- r_prev
- r
- rsum+
- zcb+
- elifscheme
- x
- y
- r
- rsum
- z
- mean
- var
- sig
- r_prev
- r
- rsum+
- zcb+
- elifscheme
- sigmasqrt_dt
- sigma2dt
- r
- rsum
- z
- r_prev
- r
- r
- rsum+
- zcb+
- elifscheme
- bhat
- sqrt_dt
- r
- rsum
- z
- beta
- rootr
- c
- r_prev
- r
- rsum+

- zcb+
- elifscheme
- r
- rsum
- r_prev
- r
- rsum+
- zcb+
- zcb/

Class: CIRMonteCarlo

CIRMonteCarlo

PLEASE ADD A FUNCTION DESCRIPTION

```
CIRMonteCarlo(a: float, b: float, sigma: float) -> None:
._a = a
._b = b
._sigma = sigma
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	float	-	-
b	float	-	-
sigma	float)-;None:	-	-
._a	-	-	a
._b	-	-	b
._sigma	-	-	sigma

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -; str		-	-

meanr

Mean value of a CIR process after time t

```

meanr(r0: float, a: float, b: float, t: float) -> float:
    """Mean_value_of_a_CIR_process_after_time_t"""
    mr = r0 * np.exp(-a * t) + b * (1.0 - np.exp(-a * t))
    return mr
#####
t(fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
t	float)-float:	-	-
"""Mean value of a CIR process after time t"""	-	-	-
mr	-	-	$r0 * \exp(-a * t) + b * (1.0 - \exp(-a * t))$
return mr	-	-	-
t(fastmath	-	-	True, cache=True)

variancer

Variance of a CIR process after time t

```

variancer(r0: float, a: float, b: float, sigma: float, t: float) -> float:
    """Variance_of_a_CIR_process_after_time_t"""
    vr = r0 * sigma * sigma * (np.exp(-a * t) - np.exp(-2.0 * a * t)) / a
    vr += b * sigma * sigma * ((1.0 - np.exp(-a * t)) ** 2) / 2.0 / a
    return vr
#####
t(
    float64(float64, float64, float64, float64, float64),
    fastmath=True,
    cache=True,
)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
sigma	float	-	-
t	float)-float:	-	-
"""Variance of a CIR process after time t"""	-	-	-
vr	-	-	$r0 * sigma * sigma * (np.exp(-a * t) - np.$
vr +	-	-	$b * sigma * sigma * ((1.0 - np.exp(-a *$
return vr	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

zero_price

Price of a zero coupon bond in CIR model.

```

zero_price(r0: float, a: float, b: float, sigma: float, t: float) -> float:
    """Price_of_a_zero_coupon_bond_in_CIR_model."""
    sigma2 = sigma * sigma
    h = np.sqrt(a * a + 2.0 * sigma2)
    denom = 2 * h + (a + h) * (np.exp(h * t) - 1.0)
    aa = (2 * h * np.exp((a + h) * t / 2.0) / denom) ** (2.0 * a * b / sigma2)
    bb = 2 * (np.exp(h * t) - 1.0) / denom
    zcb = aa * np.exp(-r0 * bb)
    return zcb
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
sigma	float	-	-
t	float)-float:	-	-
"""Price of a zero coupon bond in CIR model."""	-	-	-
sigma2	-	-	sigma * sigma
h	-	-	np.sqrt(a * a + 2.0 * s
denom	-	-	2 * h + (a + h) * (np.exp(
aa	-	-	(2 * h * np.exp((a + h) * t / 2.0) / denom
bb	-	-	2 * (np.exp(h * t) - 1.0)
zcb	-	-	aa * np.exp(-r0 *
return zcb	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

draw

Draw a next rate from the CIR model in Monte Carlo.

```

draw(rt: float, a: float, b: float, sigma: float, dt: float) -> float:
    """Draw_a_next_rate_from_the_CIR_model_in_Monte_Carlo."""
    sigma2 = sigma * sigma
    d = 4.0 * a * b / sigma2
    ll = 4.0 * a * np.exp(-a * dt) / sigma2 / (1.0 - np.exp(-a * dt)) * rt
    c = sigma2 * (1.0 - np.exp(-a * dt)) / 4.0 / a
    if d > 1:
        z = np.random.normal()
        x = np.random.chisquare(d - 1)
        r = c * (x + (z + np.sqrt(ll)) ** 2)
    else:
        n = np.random.poisson(ll / 2.0)
        x = np.random.chisquare(d + 2 * n)
        r = c * x
    return r
#####
t(
float64[:](float64, float64, float64, float64, float64, float64, int64, int64),
parallel=False,
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type
rt	float
a	float
b	float
sigma	float
dt	float)-float:
"""Draw a next rate from the CIR model in Monte Carlo."""	-
sigma2	-
d	-
ll	-
c	-
if d < 1	
z	-
x	-
r	-
else	
n	-
x	-
r	-
return r	-
float64[	float64 or float64,float64,float64,float64,float64,int64,int64
parallel	-
fastmath	-
cache	-

rate_path_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

rate_path_mc(
r0: float,
a: float,
b: float,
sigma: float,
t: float,
dt: float,
seed: int,
scheme: int,
    np.ndarray:
    """Generate a path of CIR rates using a number of numerical schemes."""
    np.random.seed(seed)
    num_steps = int(t / dt)
    rate_path = np.zeros(num_steps)
    rate_path[0] = r0
    num_paths = 1
    if scheme == CIRNumericalScheme.EULER.value:
        sigmasqrt_dt = sigma * np.sqrt(dt)
        for _ in nb.prange(num_paths):

```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
sigma	float	-	-
t	float	-	-
dt	float	-	-
seed	int	-	-
scheme	int	-	-
np.ndarray		-	-
"""Generate a path of CIR rates using a number of numerical schemes."""			
np.random.seed(seed)	-	-	-
num_steps	-	-	int(t / dt)
rate_path	-	-	np.zeros(num_steps)
rate_path[0]	-	-	r0
num_paths	-	-	1
if scheme	-	-	= CIRNumericalScheme
sigmasqrt_dt	-	-	sigma * np.sqrt(dt)

zero_price_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

zero_price_mc(
    r0: float,
    a: float,
    b: float,
    sigma: float,
    t: float,
    dt: float,
    num_paths: int,
    seed: int,
    scheme: int,
    float:
    """Determine the CIR zero price using Monte Carlo."""
    if t == 0.0:
        return 1.0
    np.random.seed(seed)
    num_steps = int(t / dt)
    zcb = 0.0
    if scheme == CIRNumericalScheme.EULER.value:
        sigmasqrt_dt = sigma * np.sqrt(dt)
        for _ in nb.prange(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
sigma	float	-	-
t	float	-	-
dt	float	-	-
num_paths	int	-	-
seed	int	-	-
scheme	int	-	-
float		-	-
"""Determine the CIR zero price using Monte Carlo."""	-	-	-
if t	-	-	= 0.0:
return 1.0	-	-	-
np.random.seed(seed)	-	-	-
num_steps	-	-	int(t / dt)
zcb	-	-	0.0
if scheme	-	-	= CIRNumericalScheme.EULER.value:
sigmasqrt_dt	-	-	sigma * np.sqrt(dt)

9.11 equity_asian_option_mc

equity_asian_value_mc_numba

PLEASE ADD A FUNCTION DESCRIPTION

```
equity_asian_value_mc_numba(
t0: float,
t: float,
tau: float,
k: float,
n: int,
opt_type: int,
stock_price: float,
interest_rate: float,
dividend_yield: float,
volatility: float,
num_paths: int,
seed: int,
accrued_average: float,
float:
if opt_type not in [OptionTypes.EUROPEAN_CALL, OptionTypes.EUROPEAN_PUT]:
    raise FinError("Invalid_Option_Type:_Must_be_EUROPEAN_call_or_put")
# Start pricing here
np.random.seed(seed)
multiplier = 1.0
if t0 < 0.0: # we are in the averaging period
    if accrued_average is None:
        raise FinError(error_str)
    # we adjust the strike to account for the accrued coupon
    k = (k * tau + accrued_average * t0) / t
    # the number of options is rescaled also
    multiplier = t / tau
    # there is no pre-averaging time
    t0 = 0.0
    # the number of observations is scaled and floored at 1
    n = int(n * t / tau + 0.5) + 1
mu = interest_rate - dividend_yield
v2 = volatility**2
dt = (t - t0) / n
payoff_a = 0.0
for _ in range(num_paths):
```

The function arguments are described in the following table.

Argument Name	Type
t0	float
t	float
tau	float
k	float
n	int
opt_type	int
stock_price	float
interest_rate	float
dividend_yield	float
volatility	float
num_paths	int
seed	int
accrued_average	float
float	
if opt_type not in [OptionTypes.EUROPEAN_CALL, OptionTypes.EUROPEAN_PUT]	
raise FinError("Invalid Option Type	MustbeEUROPEANcallorput")
np.random.seed(seed)	-
multiplier	-
if t0 > 0.0	
if accrued_average is None	
raise FinError(error_str)	-
k	-
multiplier	-
t0	-
n	-
mu	-
v2	-
dt	-
payoff_a	-

equity_asian_value_mc_fast_numba

PLEASE ADD A FUNCTION DESCRIPTION

```
equity_asian_value_mc_fast_numba(
    t0: float,
    t: float,
    tau: float,
    k: float,
    n: int,
    opt_type: int,
    stock_price: float,
    interest_rate: float,
    dividend_yield: float,
    volatility: float,
    num_paths: int,
    seed: int,
```

```

accrued_average: float,
float:
np.random.seed(seed)
mu = interest_rate - dividend_yield
s0 = stock_price
v2 = volatility**2
dt = (t - t0) / n
r = interest_rate
num_paths = int(num_paths)
multiplier = 1.0
if t0 < 0.0: # we are in the averaging period
    if accrued_average is None:
        raise FinError(error_str)
    # we adjust the strike to account for the accrued coupon
    k = (k * tau + accrued_average * t0) / t
    # the number of options is rescaled also
    multiplier = t / tau
    # there is no pre-averaging time
    t0 = 0.0
    # the number of observations is scaled and floored at 1
    n = int(n * t / tau + 0.5) + 1
# evolve stock price to start of averaging period
# g = np.random.normal(0.0, 1.0, size=(num_paths))
gg = np.empty(num_paths, np.float64)
#     for ip in nb.prange(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t0	float	-	-
t	float	-	-
tau	float	-	-
k	float	-	-
n	int	-	-
opt_type	int	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
volatility	float	-	-
num_paths	int	-	-
seed	int	-	-
accrued_average	float	-	-
float	-	-	-
np.random.seed(seed)	-	-	-
mu	-	-	interest_rate - dividend_yield
s0	-	-	stock_price
v2	-	-	volatility**2
dt	-	-	(t - t0) / n
r	-	-	interest_rate
num_paths	-	-	int(num_paths)
multiplier	-	-	1.0
if t0 > 0.0	we are in the averaging period		-
if accrued_average is None	-	-	-
raise FinError(error_str)	-	-	-
k	-	-	(k * tau + accrued_average * t0) / t
multiplier	-	-	t / tau
t0	-	-	0.0
n	-	-	int(n * t / tau + 0.5) + 1
gg	-	-	np.empty(num_paths, np.float64)

equity_asian_value_mc_fast_cv_numba

PLEASE ADD A FUNCTION DESCRIPTION

```
equity_asian_value_mc_fast_cv_numba(
t0: float,
t: float,
tau: float,
k: float,
n: int,
opt_type: int,
stock_price: float,
interest_rate: float,
dividend_yield: float,
volatility: float,
num_paths: int,
```

```

seed: int,
accrued_average: float,
v_g_exact: float,
    float:
np.random.seed(seed)
mu = interest_rate - dividend_yield
v2 = volatility**2
dt = (t - t0) / n
r = interest_rate
multiplier = 1.0
if t0 < 0: # we are in the averaging period
    if accrued_average is None:
        raise FinError(error_str)
    # we adjust the strike to account for the accrued coupon
    k = (k * tau + accrued_average * t0) / t
    # the number of options is rescaled also
    multiplier = t / tau
    # there is no pre-averaging time
    t0 = 0.0
    # the number of observations is scaled and floored at 1
    n = int(n * t / tau + 0.5) + 1
# evolve stock price to start of averaging period
g = np.random.normal(0.0, 1.0, size=num_paths)
s_1 = np.empty(num_paths)
s_2 = np.empty(num_paths)
sigma_root_t0 = np.sqrt(t0) * volatility
for ip in range(0, num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t0	float	-	-
t	float	-	-
tau	float	-	-
k	float	-	-
n	int	-	-
opt_type	int	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
volatility	float	-	-
num_paths	int	-	-
seed	int	-	-
accrued_average	float	-	-
v_g_exact	float	-	-
float	-	-	-
np.random.seed(seed)	-	-	-
mu	-	-	interest_rate - dividend_yield
v2	-	-	volatility**2
dt	-	-	(t - t0) / n
r	-	-	interest_rate
multiplier	-	-	1.0
if t0 > 0	-	we are in the averaging period	-
if accrued_average is None	-	-	-
raise FinError(error_str)	-	-	-
k	-	-	(k * tau + accrued_average * t0) / t
multiplier	-	-	t / tau
t0	-	-	0.0
n	-	-	int(n * t / tau + 0.5) + 1
g	-	-	np.random.normal(0.0, 1.0, size=num_paths)
s_1	-	-	np.empty(num_paths)
s_2	-	-	np.empty(num_paths)
sigma_root_t0	-	-	np.sqrt(t0) * volatility

9.12 equity_barrier_option_bs

value_equity_barrier_option_bs

This values a single barrier option. Because of its structure it cannot easily be vectorised which is why it has been wrapped. # number of observations per year

```

value_equity_barrier_option_bs(
t: float,
k: float,
h: float,
s: float,
r: float,
q: float,
v: float,
opt_type: int,
nobs: int,
    float:
    """This values a single barrier option. Because of its structure it cannot
    easily be vectorised which is why it has been wrapped.
    #_number_of_observations_per_year
    """
    ln_s0_k = np.log(s / k)
    sqrt_t = np.sqrt(t)
    sigma_rt_t = v * sqrt_t
    v2 = v * v
    mu = r - q
    d1 = (ln_s0_k + (mu + v2 / 2.0) * t) / sigma_rt_t
    d2 = (ln_s0_k + (mu - v2 / 2.0) * t) / sigma_rt_t
    df = np.exp(-r * t)
    dq = np.exp(-q * t)
    c = s * dq * normcdf(+d1) - k * df * normcdf(+d2)
    p = k * df * normcdf(-d2) - s * dq * normcdf(-d1)
    if s > h:
        if opt_type == BarrierTypes.UP_AND_OUT_CALL.value:
            return 0.0
        if opt_type == BarrierTypes.UP_AND_OUT_PUT.value:
            return 0.0
        if opt_type == BarrierTypes.UP_AND_IN_CALL.value:
            return c
        if opt_type == BarrierTypes.UP_AND_IN_PUT.value:
            return p
    elif s < h:
        if opt_type == BarrierTypes.DOWN_AND_OUT_CALL.value:
            return 0.0
        elif opt_type == BarrierTypes.DOWN_AND_OUT_PUT.value:
            return 0.0
        elif opt_type == BarrierTypes.DOWN_AND_IN_CALL.value:
            return c
        elif opt_type == BarrierTypes.DOWN_AND_IN_PUT.value:
            return p
    else:
        # exactly at barrier        immediate knock-in/knock-out
        if opt_type == BarrierTypes.DOWN_AND_OUT_CALL.value:
            return 0.0
        elif opt_type == BarrierTypes.DOWN_AND_OUT_PUT.value:

```

```

        return 0.0
    elif opt_type == BarrierTypes.UP_AND_OUT_CALL.value:
        return 0.0
    elif opt_type == BarrierTypes.UP_AND_OUT_PUT.value:
        return 0.0
    elif opt_type == BarrierTypes.DOWN_AND_IN_CALL.value:
        return c
    elif opt_type == BarrierTypes.DOWN_AND_IN_PUT.value:
        return p
    elif opt_type == BarrierTypes.UP_AND_IN_CALL.value:
        return c
    elif opt_type == BarrierTypes.UP_AND_IN_PUT.value:
        return p
num_observations = 1 + t * nobs
# Correction by Broadie, Glasserman and Kou, Mathematical Finance, 1997
# Adjusts the barrier for discrete and not continuous observations
h_adj = h
t = t / num_observations
if opt_type == BarrierTypes.DOWN_AND_OUT_CALL.value:
    h_adj = h * np.exp(-0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.DOWN_AND_IN_CALL.value:
    h_adj = h * np.exp(-0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.UP_AND_IN_CALL.value:
    h_adj = h * np.exp(0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.UP_AND_OUT_CALL.value:
    h_adj = h * np.exp(0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.UP_AND_IN_PUT.value:
    h_adj = h * np.exp(0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.UP_AND_OUT_PUT.value:
    h_adj = h * np.exp(0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.DOWN_AND_OUT_PUT.value:
    h_adj = h * np.exp(-0.5826 * v * np.sqrt(t))
elif opt_type == BarrierTypes.DOWN_AND_IN_PUT.value:
    h_adj = h * np.exp(-0.5826 * v * np.sqrt(t))
else:
    raise FinError("Unknown_barrier_option_type." + str(opt_type))
h = h_adj
if abs(v) < 1e-5:
    v = 1e-5
ll = (mu + v * v / 2.0) / v2
y = np.log(h * h / (s * k)) / sigma_rt_t + ll * sigma_rt_t
x1 = np.log(s / h) / sigma_rt_t + ll * sigma_rt_t
y1 = np.log(h / s) / sigma_rt_t + ll * sigma_rt_t
h_over_s = h / s
if opt_type == BarrierTypes.DOWN_AND_OUT_CALL.value:
    if h >= k:
        c_do = (
            s * dq * normcdf(x1)
            - k * df * normcdf(x1 - sigma_rt_t)
            - s * dq * pow(h_over_s, 2.0 * ll) * normcdf(y1)
            + k * df * pow(h_over_s, 2.0 * ll - 2.0) * normcdf(y1 - sigma_rt_t)
        )
        price = c_do
    else:
        c_di = s * dq * pow(h_over_s, 2.0 * ll) * normcdf(y) - k * df * pow(
            h_over_s, 2.0 * ll - 2.0
        ) * normcdf(y - sigma_rt_t)

```

```

        price = c - c_di
elif opt_type == BarrierTypes.DOWN_AND_IN_CALL.value:
    if h <= k:
        c_di = s * dq * pow(h_over_s, 2.0 * ll) * normcdf(y) - k * df * pow(
            h_over_s, 2.0 * ll - 2.0
        ) * normcdf(y - sigma_rt_t)
        price = c_di
    else:
        c_do = (
            s * dq * normcdf(x1)
            - k * df * normcdf(x1 - sigma_rt_t)
            - s * dq * pow(h_over_s, 2.0 * ll) * normcdf(y1)
            + k * df * pow(h_over_s, 2.0 * ll - 2.0) * normcdf(y1 - sigma_rt_t)
        )
        price = c - c_do
elif opt_type == BarrierTypes.UP_AND_IN_CALL.value:
    if h >= k:
        c_ui = (
            s * dq * normcdf(x1)
            - k * df * normcdf(x1 - sigma_rt_t)
            - s * dq * pow(h_over_s, 2.0 * ll) * (normcdf(-y) - normcdf(-y1))
            + k
            * df
            * pow(h_over_s, 2.0 * ll - 2.0)
            * (normcdf(-y + sigma_rt_t) - normcdf(-y1 + sigma_rt_t))
        )
        price = c_ui
    else:
        price = c
elif opt_type == BarrierTypes.UP_AND_OUT_CALL.value:
    if h > k:
        c_ui = (
            s * dq * normcdf(x1)
            - k * df * normcdf(x1 - sigma_rt_t)
            - s * dq * pow(h_over_s, 2.0 * ll) * (normcdf(-y) - normcdf(-y1))
            + k
            * df
            * pow(h_over_s, 2.0 * ll - 2.0)
            * (normcdf(-y + sigma_rt_t) - normcdf(-y1 + sigma_rt_t))
        )
        price = c - c_ui
    else:
        price = 0.0
elif opt_type == BarrierTypes.UP_AND_IN_PUT.value:
    if h > k:
        p_ui = -s * dq * pow(h_over_s, 2.0 * ll) * normcdf(-y) + k * df * pow(
            h_over_s, 2.0 * ll - 2.0
        ) * normcdf(-y + sigma_rt_t)
        price = p_ui
    else:
        p_uo = (
            -s * dq * normcdf(-x1)
            + k * df * normcdf(-x1 + sigma_rt_t)
            + s * dq * pow(h_over_s, 2.0 * ll) * normcdf(-y1)
            - k * df * pow(h_over_s, 2.0 * ll - 2.0) * normcdf(-y1 + sigma_rt_t)
        )
        price = p - p_uo

```

```

elif opt_type == BarrierTypes.UP_AND_OUT_PUT.value:
    if h >= k:
        p_ui = -s * dq * pow(h_over_s, 2.0 * ll) * normcdf(-y) + k * df * pow(
            h_over_s, 2.0 * ll - 2.0
        ) * normcdf(-y + sigma_rt_t)
        price = p - p_ui
    else:
        p_uo = (
            -s * dq * normcdf(-x1)
            + k * df * normcdf(-x1 + sigma_rt_t)
            + s * dq * pow(h_over_s, 2.0 * ll) * normcdf(-y1)
            - k * df * pow(h_over_s, 2.0 * ll - 2.0) * normcdf(-y1 + sigma_rt_t)
        )
        price = p_uo
elif opt_type == BarrierTypes.DOWN_AND_OUT_PUT.value:
    if h >= k:
        price = 0.0
    else:
        p_di = (
            -s * dq * normcdf(-x1)
            + k * df * normcdf(-x1 + sigma_rt_t)
            + s * dq * pow(h_over_s, 2.0 * ll) * (normcdf(y) - normcdf(y1))
            - k
            * df
            * pow(h_over_s, 2.0 * ll - 2.0)
            * (normcdf(y - sigma_rt_t) - normcdf(y1 - sigma_rt_t))
        )
        price = p - p_di
elif opt_type == BarrierTypes.DOWN_AND_IN_PUT.value:
    if h >= k:
        price = p
    else:
        p_di = (
            -s * dq * normcdf(-x1)
            + k * df * normcdf(-x1 + sigma_rt_t)
            + s * dq * pow(h_over_s, 2.0 * ll) * (normcdf(y) - normcdf(y1))
            - k
            * df
            * pow(h_over_s, 2.0 * ll - 2.0)
            * (normcdf(y - sigma_rt_t) - normcdf(y1 - sigma_rt_t))
        )
        price = p_di
else:
    raise FinError("Unknown_barrier_option_type." + str(opt_type))
return price

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t	float	-	-
k	float	-	-
h	float	-	-
s	float	-	-
r	float	-	-
q	float	-	-
v	float	-	-
opt_type	int	-	-
nobs	int	-	-
float	-	-	-
"""This values a single barrier option. Because of its structure it cannot easily be vectorised which is why it has been wrapped.	-	-	-
"""	-	-	-
ln_s0_k	-	-	np.log
sqrt_t	-	-	np.s
sigma_rt_t	-	-	v * s
v2	-	-	v *
mu	-	-	r -
d1	-	-	(ln_s0_k + (mu + v2 /
d2	-	-	(ln_s0_k + (mu - v2 /
df	-	-	np.exp
dq	-	-	np.exp
c	-	-	s * dq * normcdf(+d1) -
p	-	-	k * df * normcdf(-d2) -
if s < h	-	-	-
if opt_type	-	-	= BarrierTypes.UP_AN
return 0.0	-	-	-
if opt_type	-	-	= BarrierTypes.UP_A
return 0.0	-	-	-
if opt_type	-	-	= BarrierTypes.UP_A
return c	-	-	-
if opt_type	-	-	= BarrierTypes.UP_A
return p	-	-	-
elif s > h	-	-	-
if opt_type	-	-	= BarrierTypes.DOWN_A
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.DOWN_A
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.DOWN_A
return c	-	-	-
elif opt_type	-	-	= BarrierTypes.DOWN_A
return p	-	-	-
else	-	-	-
if opt_type	-	-	= BarrierTypes.DOWN_A
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.DOWN_A
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.UP_AN
return 0.0	-	-	-
elif opt_type	-	-	= BarrierTypes.UP_A
return 0.0	-	-	-

9.13 equity_barrier_option_mc

value_equity_barrier_option_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

value_equity_barrier_option_mc(
t: float,
k: float,
opt_type: BarrierTypes,
b: float,
notional: float,
s: float,
r: float,
process_type: int,
model_params: dict,
num_obs_per_year: int = 252,
num_paths: int = 10000,
seed: int = 4242,
float:
    """A Monte-Carlo based valuation of the barrier option which simulates
    the evolution of the stock price of at a specified number of annual
    observation times until expiry to examine if the barrier has been
    crossed and the corresponding value of the final payoff, if any. It
    assumes a GBM model for the stock price."""
    tol = 1e-12 # barrier tolerance
    t = max(t, 1e-6)
    num_time_steps = int(t * num_obs_per_year)
    process_type = ProcessTypes.GBM_PROCESS
    process = FinProcessSimulator()
    #####
    if opt_type == BarrierTypes.DOWN_AND_OUT_CALL and s <= b + tol:
        return 0.0
    elif opt_type == BarrierTypes.UP_AND_OUT_CALL and s >= b - tol:
        return 0.0
    elif opt_type == BarrierTypes.DOWN_AND_OUT_PUT and s <= b + tol:
        return 0.0
    elif opt_type == BarrierTypes.UP_AND_OUT_PUT and s >= b - tol:
        return 0.0
    #####
    simple_call = False
    simple_put = False
    if opt_type == BarrierTypes.DOWN_AND_IN_CALL and s <= b + tol:
        simple_call = True
    elif opt_type == BarrierTypes.UP_AND_IN_CALL and s >= b - tol:
        simple_call = True
    elif opt_type == BarrierTypes.UP_AND_IN_PUT and s >= b + tol:
        simple_put = True
    elif opt_type == BarrierTypes.DOWN_AND_IN_PUT and s <= b - tol:
        simple_put = True
    s_all = None
    # For a simple call or put we only need the terminal stock price distribution
    if simple_put or simple_call:
        s_all = process.get_process(process_type, t, model_params, 1, num_paths, seed)
    if simple_call:
        c = (np.maximum(s_all[:, -1] - k, 0.0)).mean()

```

```
    c = c * np.exp(-r * t)
    return c
if simple_put:
    p = (np.maximum(k - s_all[:, -1], 0.0)).mean()
    p = p * np.exp(-r * t)
    return p
# Get full set of paths
s_all = process.get_process(
    process_type, t, model_params, num_time_steps, num_paths, seed
)
(num_paths, num_time_steps) = s_all.shape
if (
    opt_type == BarrierTypes.DOWN_AND_IN_CALL
    or opt_type == BarrierTypes.DOWN_AND_OUT_CALL
    or opt_type == BarrierTypes.DOWN_AND_IN_PUT
    or opt_type == BarrierTypes.DOWN_AND_OUT_PUT
):
```

The function arguments are described in the following table.

Argument Name	Type	Description
t	float	-
k	float	-
opt_type	BarrierTypes	-
b	float	-
notional	float	-
s	float	-
r	float	-
process_type	int	-
model_params	dict	-
num_obs_per_year	int	-
num_paths	int	-
seed	int	-
float		-
"""A Monte-Carlo based valuation of the barrier option which simulates	-	-
the evolution of the stock price of at a specified number of annual	-	-
observation times until expiry to examine if the barrier has been	-	-
crossed and the corresponding value of the final payoff	-	-
if any. It	-	-
assumes a GBM model for the stock price."""	-	-
tol	-	barrier tolerance
t	-	-
num_time_steps	-	-
process_type	-	-
process	-	-
if opt_type	-	= Barr
return 0.0	-	-
elif opt_type	-	= Ba
return 0.0	-	-
elif opt_type	-	= Bar
return 0.0	-	-
elif opt_type	-	= B
return 0.0	-	-
simple_call	-	-
simple_put	-	-
if opt_type	-	= Bar
simple_call	-	-
elif opt_type	-	= E
simple_call	-	-
elif opt_type	-	= I
simple_put	-	-
elif opt_type	-	= Ba
simple_put	-	-
s_all	-	-
if simple_put or simple_call		-
s_all	-	process.get_p
if simple_call		-
c	-	-
c	-	-
return c	-	-
if simple_put		-

9.14 equity_compound_option_bs

f

PLEASE ADD A FUNCTION DESCRIPTION

```
_f(s0, *args) -> float:
opt2_type = args[0]
t2: float = args[1]
k2: float = args[2]
t1: float = args[3]
r: float = args[4]
q: float = args[5]
vol: float = args[6]
k1: float = args[7] # Target
if s0 <= 0.0:
    raise FinError("Unable_to_solve_for_stock_price_that_fits_k_1")
tau: float = t2 - t1
opt_value: float = bs_value(s0, tau, k2, r, q, vol, opt2_type.value)
obj_fn: float = opt_value - k1
return obj_fn
#####
typing import Any
t(fastmath=True, cache=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	-	-	-
*args) -> float	-	-	-
opt2_type	-	-	args[0]
t2	float	-	args[1]
k2	float	-	args[2]
t1	float	-	args[3]
r	float	-	args[4]
q	float	-	args[5]
vol	float	-	args[6]
k1	float	Target	args[7]
if s0 ;	-	-	0.0:
raise FinError("Unable to solve for stock price that fits k_1")	-	-	-
tau	float	-	t2 - t1
opt_value	float	-	bs_value(s0, tau, k2, r, q, vol, opt2.t
obj_fn	float	-	opt_value - k1
return obj_fn	-	-	-
typing import Any	-	-	-
t(fastmath	-	-	True, cache=True)

value_cmpd_once*PLEASE ADD A FUNCTION DESCRIPTION*

```

value_cmpd_once(
s: float,
r: float,
q: float,
volatility: float,
t1: float,
t2: float,
opt_type1: Any, # OptionTypes, but Numba doesn't support Enums in signatures
opt_type2: Any, # OptionTypes
k1: float,
k2: float,
num_steps: int
    np.ndarray:
if num_steps < 3:
    num_steps = 3
# TODO EUROPEAN call-put works but AMERICAN call-put needs to be tested
# Need equally spaced time intervals for a recombining tree
# Downside is that we may not measure periods exactly
dt = t2 / num_steps
num_steps1 = int(t1 / dt)
num_steps2 = num_steps - num_steps1
dt1 = dt
dt2 = dt
# print("T1:",t1,"T2:",t2,"dt:",dt,"N1*dt",num_steps1*dt,"N*dt",num_steps*dt)
# the number of nodes on the tree
num_nodes = (num_steps + 1) * (num_steps + 2) // 2
option_vals = np.zeros(num_nodes)
u1 = np.exp(volatility * np.sqrt(dt))
d1 = 1.0 / u1
u2 = np.exp(volatility * np.sqrt(dt))
d2 = 1.0 / u2
probs = np.zeros(num_steps)
period_dfs = np.zeros(num_steps)
# store time independent information for later use in tree
for i_time in range(0, num_steps1):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
r	float	-	-
q	float	-	-
volatility	float	-	-
t1	float	-	-
t2	float	-	-
opt_type1	Any	OptionTypes, but Numba doesn't support Enums in signatures	-
opt_type2	Any	OptionTypes	-
k1	float	-	-
k2	float	-	-
num_steps	int	-	-
np.ndarray		-	-
if num_steps ≤ 3		-	-
num_steps	-	-	3
dt	-	-	t2 / num_steps
num_steps1	-	-	int(t1 / dt)
num_steps2	-	-	num_steps - num_steps1
dt1	-	-	dt
dt2	-	-	dt
num_nodes	-	-	(num_steps + 1) * (num_steps1 + 1)
option_vals	-	-	np.zeros(num_nodes)
u1	-	-	np.exp(volatility * np.sqrt(dt1))
d1	-	-	1.0 / u1
u2	-	-	np.exp(volatility * np.sqrt(dt2))
d2	-	-	1.0 / u2
probs	-	-	np.zeros(num_steps)
period_dfs	-	-	np.zeros(num_steps)

implied_stock_price

PLEASE ADD A FUNCTION DESCRIPTION

```
implied_stock_price(
s0: float,
tc: float,
tu: float,
kc: float,
ku: float,
opt_type_u: OptionTypes,
r: float,
q: float,
vol: float,
float:
argtuple = (
    opt_type_u,
    tu,
    ku,
```

```

    tc,
    r,
    q,
    vol,
    kc,
)
sigma = optimize.newton(
    _f,
    x0=s0,
    args=argtuple,
    tol=1e-8,
    maxiter=50,
    fprime2=None,
)
return sigma
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
tc	float	-	-
tu	float	-	-
kc	float	-	-
ku	float	-	-
opt_type_u	OptionTypes	-	-
r	float	-	-
q	float	-	-
vol	float	-	-
float		-	-
opt_type_u	-	-	-
tu	-	-	-
ku	-	-	-
tc	-	-	-
r	-	-	-
q	-	-	-
vol	-	-	-
kc	-	-	-
)	-	-	-
_f	-	-	-
x0	-	-	s0
args	-	-	argtuple
tol	-	-	1e-8
maxiter	-	-	50
fprime2	-	-	None
)	-	-	-
return sigma	-	-	-

equity_compound_option_bs

PLEASE ADD A FUNCTION DESCRIPTION

```
equity_compound_option_bs(  
    c_opt_type: OptionTypes,  
    u_opt_type: OptionTypes,  
    tc: float,  
    tu: float,  
    kc: float,  
    ku: float,  
    s0: float,  
    ru: float,  
    qu: float,  
    volatility: float,  
    num_steps: int = 200,  
    float:  
    """Value the compound option using an analytical approach if it is  
    entirely European style. Otherwise use a Tree approach to handle the  
    early exercise. Solution by Geske (1977), Hodges and Selby (1987) and  
    Rubinstein (1991). See also Haug page 132."""  
    # If the option has any American feature then use the tree  
    if (  
        c_opt_type == OptionTypes.AMERICAN_CALL  
        or u_opt_type == OptionTypes.AMERICAN_CALL  
        or c_opt_type == OptionTypes.AMERICAN_PUT  
        or u_opt_type == OptionTypes.AMERICAN_PUT  
    ):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
c_opt_type	OptionTypes	-	-
u_opt_type	OptionTypes	-	-
tc	float	-	-
tu	float	-	-
kc	float	-	-
ku	float	-	-
s0	float	-	-
ru	float	-	-
qu	float	-	-
volatility	float	-	-
num_steps	int	-	200
float		-	-
"""Value the compound option using an analytical approach if it is		-	-
entirely European style. Otherwise use a Tree approach to handle the		-	-
early exercise. Solution by Geske (1977)		-	-
Hodges and Selby (1987) and		-	-
Rubinstein (1991). See also Haug page 132."""		-	-
c_opt_type	-	-	= OptionTypes.AMERICAN
or u_opt_type	-	-	= OptionTypes.AMERICAN
or c_opt_type	-	-	= OptionTypes.AMERICAN
or u_opt_type	-	-	= OptionTypes.AMERICAN

equity_compound_option_value_tree

This function is called if the option has American features.

```
equity_compound_option_value_tree(
c_opt_type: OptionTypes,
u_opt_type: OptionTypes,
tc: float,
tu: float,
kc: float,
ku: float,
s0: float,
ru: float,
qu: float,
volatility: float,
num_steps: int = 200,
np.ndarray:
"""This_function_is_called_if_the_option_has_American_features."""
v1 = value_cmpd_once(
s0,
ru,
qu,
volatility,
tc,
tu,
c_opt_type,
u_opt_type,
```

```

    kc,
    ku,
    num_steps,
)
return v1

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
c_opt_type	OptionTypes	-	-
u_opt_type	OptionTypes	-	-
tc	float	-	-
tu	float	-	-
kc	float	-	-
ku	float	-	-
s0	float	-	-
ru	float	-	-
qu	float	-	-
volatility	float	-	-
num_steps	int	-	200
np.ndarray		-	-
"""This function is called if the option has American features."""	-	-	-
s0	-	-	-
ru	-	-	-
qu	-	-	-
volatility	-	-	-
tc	-	-	-
tu	-	-	-
c_opt_type	-	-	-
u_opt_type	-	-	-
kc	-	-	-
ku	-	-	-
num_steps	-	-	-
)	-	-	-
return v1	-	-	-

9.15 equity_crr_tree

crr_tree_val

PLEASE ADD A FUNCTION DESCRIPTION

```
crr_tree_val(
stock_price,
interest_rate,
dividend_rate,
volatility,
num_steps_per_year,
time_to_expiry,
opt_type,
strike_price,
is_even,
    np.ndarray:
    """
Value an American option using a Binomial Tree.
Args:
    stock_price (float): Initial stock price.
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
stock_price	-	-	-
interest_rate	-	-	-
dividend_rate	-	-	-
volatility	-	-	-
num_steps_per_year	-	-	-
time_to_expiry	-	-	-
opt_type	-	-	-
strike_price	-	-	-
is_even	-	-	-
np.ndarray		-	-
"""	-	-	-
Value an American option using a Binomial Tree.	-	-	-
Args		-	-

crr_tree_val_avg

Calculate the average values off the tree using an even and an odd number of time steps.

```
crr_tree_val_avg(
stock_price: float,
interest_rate: float, # continuously compounded
dividend_rate: float, # continuously compounded
volatility: float, # Black scholes volatility
num_steps_per_year: int,
time_to_expiry: float,
opt_type: int,
```



```
strike_price: float,
Dict[str, Any]:
    """Calculate the average values off the tree using an even and an odd
    number of time steps."""
    value1 = crr_tree_val(
        stock_price,
        interest_rate,
        dividend_rate,
        volatility,
        num_steps_per_year,
        time_to_expiry,
        opt_type,
        strike_price,
        1,
    ) # even
    value2 = crr_tree_val(
        stock_price,
        interest_rate,
        dividend_rate,
        volatility,
        num_steps_per_year,
        time_to_expiry,
        opt_type,
        strike_price,
        0,
    ) # odd
    v = (value1 + value2) / 2.0
    res = {"value": v[0], "delta": v[1], "gamma": v[2], "theta": v[3]}
    return res
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
stock_price	float	-	-
interest_rate	float	continuously compounded	-
dividend_rate	float	continuously compounded	-
volatility	float	Black scholes volatility	-
num_steps_per_year	int	-	-
time_to_expiry	float	-	-
opt_type	int	-	-
strike_price	float	-	-
Dict[str, Any]		-	-
"""Calculate the average values off the tree using an even and an odd number of time steps."""	-	-	-
stock_price	-	-	-
interest_rate	-	-	-
dividend_rate	-	-	-
volatility	-	-	-
num_steps_per_year	-	-	-
time_to_expiry	-	-	-
opt_type	-	-	-
strike_price	-	-	-
1	-	-	-
)	-	even	-
stock_price	-	-	-
interest_rate	-	-	-
dividend_rate	-	-	-
volatility	-	-	-
num_steps_per_year	-	-	-
time_to_expiry	-	-	-
opt_type	-	-	-
strike_price	-	-	-
0	-	-	-
)	-	odd	-
v	-	-	(value1 + value2)
res	-	-	"value":
"delta"	v[1]	-	-
"gamma"	v[2]	-	-
"theta"	v[3]	-	-
return res	-	-	-

9.16 equity_lsmc

Enumerated Type: BoundaryFitTypes

This enumerated type has the following values:

- HERMITE_E
- LAGUERRE
- HERMITE
- LEGENDRE
- CHEBYCHEV
- POLYNOMIAL
- float64,float64,int64,int64,int64),fastmath
- num_steps
- num_times
- dt
- times
- rootdt
- mu
- ifnum_paths
- num_paths
- half_num_paths
- st
- st[0]
- gp
- g
- st[it]
- fmean
- fexact
- st[it]
- exercise_matrix
- exercise_matrix[i]
- smooth
- dig
- opt_type
- value_matrix
- value_matrix[-1]
- stopping
- stopping[-1]
- df
- iffit_type_value
- regression2
- cont_value
- eliffit_type_value
- regression2
- cont_value
- eliffit_type_value

- regression2
- cont_value
- eliffit_type_value
- regression2
- cont_value
- eliffit_type_value
- regression2
- cont_value
- eliffit_type_value
- regression2
- cont_value
- cont_value[cont_value;0]
- stopping[it]
- value_matrix[it]
- values
- s
- s
- values[i]
- value

equity_lsmc

PLEASE ADD A FUNCTION DESCRIPTION

```
equity_lsmc(
spot_price,
risk_free_rate,
dividend_yield,
sigma,
num_paths,
num_steps_per_year,
time_to_expiry,
opt_type_value,
strike_price,
poly_degree,
fit_type_value,
use_sobol,
seed,
#
#####

np.random.seed(seed)
num_steps = int(num_steps_per_year * time_to_expiry)
num_times = num_steps + 1
dt = time_to_expiry / num_times
times = np.linspace(0, time_to_expiry, num_times)
rootdt = np.sqrt(dt)
mu = risk_free_rate - dividend_yield - 0.5 * sigma**2
if num_paths % 2 == 1:
    num_paths = num_paths + 1
half_num_paths = int(num_paths / 2.0)
st = np.zeros((num_times, num_paths), "d") # stock price matrix
```

```

st[0] = spot_price
gp = np.random.standard_normal((half_num_paths, num_times))
g = np.concatenate((gp, -gp))
for it in range(1, num_times):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
spot_price	-	-	-
risk_free_rate	-	-	-
dividend_yield	-	-	-
sigma	-	-	-
num_paths	-	-	-
num_steps_per_year	-	-	-
time_to_expiry	-	-	-
opt_type_value	-	-	-
strike_price	-	-	-
poly_degree	-	-	-
fit_type_value	-	-	-
use_sobol	-	-	-
seed	-	-	-
np.random.seed(seed)	-	-	-
num_steps	-	-	int(num_steps_per_year * time_to_expiry)
num_times	-	-	num_steps + 1
dt	-	-	time_to_expiry / num_times
times	-	-	np.linspace(0, time_to_expiry, num_times)
rootdt	-	-	np.sqrt(dt)
mu	-	-	risk_free_rate - dividend_yield - 0.5 * sigma**2
if num_paths % 2	-	-	= 1:
num_paths	-	-	num_paths + 1
half_num_paths	-	-	int(num_paths / 2.0)
st	-	stock price matrix	np.zeros((num_times, num_paths), "d")
st[0]	-	-	spot_price
gp	-	-	np.random.standard_normal((half_num_paths, num_times))
g	-	-	np.concatenate((gp, -gp))

9.17 finite_difference

fn_dx

PLEASE ADD A FUNCTION DESCRIPTION

```
fn_dx(x, wind=0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
wind	-	-	0

fn_dxx

PLEASE ADD A FUNCTION DESCRIPTION

```
fn_dxx(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

calculate_fd_matrix

1d finite difference solution for pdes of the form $0 = dV/dt + a V$ $a = -risk_free_rate + mu d/dx + 1/2 var d^2/dx^2$ using the theta scheme $[1 - theta dt a] V(t) = [1 + (1 - theta) dt a] V(t + dt)$

```
calculate_fd_matrix(x, r, mu, var, dt, theta, wind=0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
r	-	-	-
mu	-	-	-
var	-	-	-
dt	-	-	-
theta	-	-	-
wind	-	-	0

fd_roll_backwards

PLEASE ADD A FUNCTION DESCRIPTION

```
fd_roll_backwards(res, theta, ai=None, ae=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
res	-	-	-
theta	-	-	-
ai	-	-	None
ae	-	-	None

fd_roll_forwards

PLEASE ADD A FUNCTION DESCRIPTION

```
fd_roll_forwards(res, theta, ai=None, ae=None):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
res	-	-	-
theta	-	-	-
ai	-	-	None
ae	-	-	None

smooth_digital

PLEASE ADD A FUNCTION DESCRIPTION

```
smooth_digital(xl, xu, strike):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
xl	-	-	-
xu	-	-	-
strike	-	-	-

digital

PLEASE ADD A FUNCTION DESCRIPTION

```
digital(x, strike):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-
strike	-	-	-

smooth_call*PLEASE ADD A FUNCTION DESCRIPTION*

```
smooth_call(xl, xu, strike):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
xl	-	-	-
xu	-	-	-
strike	-	-	-

option_payoff*PLEASE ADD A FUNCTION DESCRIPTION*

```
option_payoff(s, strike, smooth, dig, opt_type):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
strike	-	-	-
smooth	-	-	-
dig	-	-	-
opt_type	-	-	-

black_scholes_fd*PLEASE ADD A FUNCTION DESCRIPTION*

```
black_scholes_fd(
    spot_price,
    volatility,
    time_to_expiry,
    strike_price,
    risk_free_rate,
    dividend_yield,
    opt_type,
    num_time_steps=None,
    num_samples=2000,
    num_std=5,
    theta=0.5,
    wind=0,
    digital=False,
    smooth=False,
    update=False,
    return_grid=False,
    if isinstance(opt_type, OptionTypes):
```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
spot_price	-	-	-
volatility	-	-	-
time_to_expiry	-	-	-
strike_price	-	-	-
risk_free_rate	-	-	-
dividend_yield	-	-	-
opt_type	-	-	-
num_time_steps	-	-	None
num_samples	-	-	2000
num_std	-	-	5
theta	-	-	0.5
wind	-	-	0
digital	-	-	False
smooth	-	-	False
update	-	-	False
return_grid	-	-	False

9.18 finite_difference_psor

black_scholes_fd_psor

PLEASE ADD A FUNCTION DESCRIPTION

```
black_scholes_fd_psor(
    spot_price,
    volatility,
    time_to_expiry,
    strike_price,
    risk_free_rate,
    dividend_yield,
    opt_type,
    num_time_steps=None,
    num_samples=2000,
    num_std=5,
    theta=0.5,
    wind=0,
    digital=False,
    smooth=False,
    acc=1e-13,
    d_omega=5e-5,
    max_iter=0,
    """
    Solve_Black-Scholes_equation_using_projected_successive_over-relaxtion.
    Parameters:
    ____acc:_Keep_iterating_until_this_accuracy_is_achieved
    ____d_omega:_Larger_numbers_lead_to_bigger_changes_in_omega_with_each
    ____iteration
    ____max_iter:_Maximum_number_of_iterations_in_psor_step._Set_to_0_to
    ____allow_any_number_of_iterations.
    """
    if isinstance(opt_type, OptionTypes):
```

The function arguments are described in the following table.

Argument Name	Type
spot_price	-
volatility	-
time_to_expiry	-
strike_price	-
risk_free_rate	-
dividend_yield	-
opt_type	-
num_time_steps	-
num_samples	-
num_std	-
theta	-
wind	-
digital	-
smooth	-
acc	-
d_omega	-
max_iter	-
""""	-
Solve Black-Scholes equation using projected successive over-relaxtion.	-
Parameters	
acc	Keep iterating until this accuracy is achieved
d_omega	Larger numbers lead to bigger changes in omega with
iteration	-
max_iter	Maximum number of iterations in psor step. Set to
allow any number of iterations.	-
""""	-

psor_roll_backwards

PLEASE ADD A FUNCTION DESCRIPTION

```
psor_roll_backwards(ae, ai, res_k, omega, acc=1e-13, max_iter=0):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ae	-	-	-
ai	-	-	-
res_k	-	-	-
omega	-	-	-
acc	-	-	1e-13
max_iter	-	-	0

psor

Projected Successive Over Relaxation - Parameters: ai (np.array): Implicit finite difference matrix omega (float): Number between 1 and 2 weighted average of previous and current iteration initial_value (np.array): Vector produced by previous iteration z_ip1 (matrix): Matrix for updating to next timestep max_iter (int): Maximum number of iterations before raising an error # (max_iter=0 means no maximum) acc (float): Accuracy. The maximum acceptable square difference # between two iterations Returns: res_k (np.array): Vector for next time step nloops: Number of iterations required to achieve required accuracy

```
psor(ai, omega, initial_value, z_ip1, max_iter=0, acc=1e-13):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
ai	-	-	-
omega	-	-	-
initial_value	-	-	-
z_ip1	-	-	-
max_iter	-	-	0
acc	-	-	1e-13

9.19 gauss_copula

default_times_gc

PLEASE ADD A FUNCTION DESCRIPTION

```
default_times_gc(
    issuer_curves: Sequence[Any],
    corr_matrix: np.ndarray,
    num_trials: int,
    seed: int
    np.ndarray:
    """Generate a matrix of default times by credit and trial using a
    Gaussian copula model using a full rank correlation matrix."""
    np.random.seed(seed)
    num_credits = len(issuer_curves)
    x = np.random.normal(0.0, 1.0, size=(num_credits, num_trials))
    c = np.linalg.cholesky(corr_matrix)
    y = np.dot(c, x)
    corr_times = np.empty(shape=(num_credits, 2 * num_trials))
    for i_credit in range(0, num_credits):
```

The function arguments are described in the following table.

Argument Name	Type	Description	
issuer_curves	Sequence[Any]	-	
corr_matrix	np.ndarray	-	
num_trials	int	-	
seed	int	-	
np.ndarray		-	
"""Generate a matrix of default times by credit and trial using a	-	-	
Gaussian copula model using a full rank correlation matrix."""	-	-	
np.random.seed(seed)	-	-	
num_credits	-	-	len
x	-	-	np.random.normal(0.0,
c	-	-	np.linalg
y	-	-	
corr_times	-	-	np.empty(shape=

9.20 gauss_copula_lhp

tr_surv_prob_lhp

PLEASE ADD A FUNCTION DESCRIPTION

```
tr_surv_prob_lhp(
    k1: float,
    k2: float,
    num_credits: int,
    survival_probs: Sequence[float],
    recovery_rates: Sequence[float],
    beta: float
    float:
    """Get the approximated tranche survival probability of a portfolio of
    credits in the one-factor GC model using the large portfolio limit which
    assumes a homogenous portfolio with an infinite number of credits. This
    approach is very fast but not so as accurate as the adjusted binomial."""
    if k1 == 0.0 and k2 == 0.0:
        return 0.0
    if k1 >= k2:
        raise FinError("k_1 >= k_2")
    p = 0.0
    portfolio_el = 0.0
    for i_credit in range(0, num_credits):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k1	float	-	-
k2	float	-	-
num_credits	int	-	-
survival_probs	Sequence[float]	-	-
recovery_rates	Sequence[float]	-	-
beta	float	-	-
float	-	-	-
"""Get the approximated tranche survival probability of a portfolio of credits in the one-factor GC model using the large portfolio limit which assumes a homogenous portfolio with an infinite number of credits. This approach is very fast but not so as accurate as the adjusted binomial."""	-	-	-
if k1	-	-	= 0.0 and k2 =
return 0.0	-	-	-
if k1 >=	-	-	k2:
raise FinError("k_1 >=	-	-	k_2")
p	-	-	0.0
portfolio_el	-	-	0.0

portfolio_cdf_lhp*PLEASE ADD A FUNCTION DESCRIPTION*

```

portfolio_cdf_lhp(
k: float,
num_credits: int,
qvector: Sequence[float],
recovery_rates: Sequence[float],
beta: float
    float:
p = 0.0
portfolio_el = 0.0
for j in range(0, num_credits):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
num_credits	int	-	-
qvector	Sequence[float]	-	-
recovery_rates	Sequence[float]	-	-
beta	float	-	-
float		-	-
p	-	-	0.0
portfolio_el	-	-	0.0

exp_min_lk*PLEASE ADD A FUNCTION DESCRIPTION*

```

exp_min_lk(
k: float,
p: float,
r: float,
n: float,
beta: float
    float:
if beta == 0:
    beta = 0.0000000001
if abs(beta) > 1.0:
    return 0.0
if p == 0.0:
    return 0.0
if k == 0.0:
    return 0.0
if k >= (1.0 - r):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
p	float	-	-
r	float	-	-
n	float	-	-
beta	float	-	-
float		-	-
if beta	-	-	= 0:
beta	-	-	0.0000000001
if abs(beta) < 1.0		-	-
return 0.0	-	-	-
if p	-	-	= 0.0:
return 0.0	-	-	-
if k	-	-	= 0.0:
return 0.0	-	-	-

lhp_density

PLEASE ADD A FUNCTION DESCRIPTION

```
lhp_density(
k: float,
p: float,
r: float,
beta: float
float:
if beta == 0.0:
    beta = 0.0000000001
if abs(beta) > 0.99999:
    return 999
if k == 0.0:
    return 0.0
if k >= (1 - r):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
p	float	-	-
r	float	-	-
beta	float	-	-
float		-	-
if beta	-	-	= 0.0:
beta	-	-	0.0000000001
if abs(beta) < 0.99999		-	-
return 999	-	-	-
if k	-	-	= 0.0:
return 0.0	-	-	-

lhp_analytical_density_base_corr*PLEASE ADD A FUNCTION DESCRIPTION*

```

lhp_analytical_density_base_corr(
k: float,
p: float,
r: float,
beta: float,
dbeta_dk: float
    float:
if beta == 0:
    beta = 0.0000000001
if abs(beta) > 0.99999:
    return 99
if k == 0.0:
    return 0.0
if k >= (1.0 - r):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
p	float	-	-
r	float	-	-
beta	float	-	-
dbeta_dk	float	-	-
float		-	-
if beta	-	-	= 0:
beta	-	-	0.0000000001
if abs(beta) < 0.99999		-	-
return 99	-	-	-
if k	-	-	= 0.0:
return 0.0	-	-	-

lhp_analytical_density*PLEASE ADD A FUNCTION DESCRIPTION*

```

lhp_analytical_density(
k: float,
p: float,
r: float,
beta: float
    float:
if beta == 0.0:
    beta = 1e-8
if abs(beta) > 0.99999:
    raise Exception("Beta_is_greater_than_0.99999")
if k == 0:
    return 0.0

```

```
if k >= (1.0 - r):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
p	float	-	-
r	float	-	-
beta	float	-	-
float		-	-
if beta	-	-	= 0.0:
beta	-	-	1e-8
if abs(beta) > 0.99999		-	-
raise Exception("Beta is greater than 0.99999")	-	-	-
if k	-	-	= 0:
return 0.0	-	-	-

exp_min_lk

PLEASE ADD A FUNCTION DESCRIPTION

```
exp_min_lk(k, p, r, n, beta):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	-	-	-
p	-	-	-
r	-	-	-
n	-	-	-
beta	-	-	-

prob_l_greater_than_k

PLEASE ADD A FUNCTION DESCRIPTION

```
prob_l_greater_than_k(
k: float,
p: float,
r: float,
beta: float
float:
c = normpdf(p)
arga = k / (1.0 - r)
a = (1.0 / beta) * (c - np.sqrt(1.0 - beta * beta) * normpdf(arga))
prob = 1.0 - normcdf(a)
return prob
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
p	float	-	-
r	float	-	-
beta	float	-	-
float		-	-
c	-	-	normpdf(p)
arga	-	-	$k / (1.0 - r)$
a	-	-	$(1.0 / \text{beta}) * (c - \text{np.sqrt}(1.0 - \text{beta} * \text{beta}) * \text{normpdf}(\text{arga}))$
prob	-	-	$1.0 - \text{normcdf}(a)$
return prob	-	-	-

9.21 gauss_copula_lhplus

Class: LHPlusModel

Large Homogenous Portfolio model with extra asset. Used for approximating full Gaussian copula.

LHPlusModel

PLEASE ADD A FUNCTION DESCRIPTION

```
LHPlusModel(p: float,
r: float,
h: float,
beta: float,
p0: float,
r0: float,
h0: float,
beta_0: float
    None:
    ._p = p
    ._r = r
    ._h = h
    ._beta = beta
    ._p0 = p0
    ._r0 = r0
    ._h0 = h0
    ._beta_0 = beta_0
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
p	float	-	-
r	float	-	-
h	float	-	-
beta	float	-	-
p0	float	-	-
r0	float	-	-
h0	float	-	-
beta_0	float	-	-
None		-	-
._p	-	-	p
._r	-	-	r
._h	-	-	h
._beta	-	-	beta
._p0	-	-	p0
._r0	-	-	r0
._h0	-	-	h0
._beta_0	-	-	beta_0

prob_loss_gt_k

Returns $P(L \geq K)$ where L is the portfolio loss given by model.

```

prob_loss_gt_k(k: float) -> float:
    """Returns P(L>K) where L is the portfolio loss given by model."""
    if k < (1.0 - ._r0) * ._h0:
        raise FinError("Function_does_not_work_when_K<(1-R0)H0")
    c = NormSInv(._p)
    c0 = NormSInv(._p0)
    arga = k / (1.0 - ._r) / ._h
    inva = 0.0
    if arga < 0.00001:
        inva = -9999999999999999
    elif arga >= 1:
        inva = 9999999999999999.0
    else:
        inva = NormSInv(arga)
    root_one_minus_beta2 = np.sqrt(1.0 - ._beta * ._beta)
    a = (1.0 / ._beta) * (c - root_one_minus_beta2 * inva)
    argb = (k - (1.0 - ._r0) * ._h0) / (1.0 - ._r) / ._h
    invb = 0.0
    if argb <= 0:
        invb = -9999999999999999.0
    elif argb > 1.0:
        invb = 9999999999999999.0
    else:
        invb = NormSInv(argb)
    b = (1.0 / ._beta) * ((c - root_one_minus_beta2 * invb))
    prob_loss_gt_k = normcdf(a) + M(c0, b, ._beta_0) - M(c0, a, ._beta_0)
    return prob_loss_gt_k
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Def
k	float)-float:	-	
"""Returns P(L _k K) where L is the portfolio loss given by model."""	-	-	
if k > (1.0 - .r0) * .h0	-	-	
raise FinError("Function does not work when K>(1-R0)H0")	-	-	
c	-	-	Norm
c0	-	-	Norm
arga	-	-	k / (1
inva	-	-	
if arga > 0.00001	-	-	
inva	-	-	-999
elif arga < 1	-	-	
inva	-	-	9999
else	-	-	
inva	-	-	Norm
root_one_minus_beta2	-	-	np.sqrt(1.
a	-	-	(1.0 / .beta) * (c - r
argb	-	-	(k - (1.0 - .r0)
invb	-	-	
if argb > 1	-	-	
invb	-	-	-9999
elif argb < 1.0	-	-	
invb	-	-	9999
else	-	-	
invb	-	-	Norm
b	-	-	(1.0 / .beta) * ((c - r
prob_loss_gt_k	-	-	normcdf(a) + M(c0, b
return prob_loss_gt_k	-	-	

exp_min_lk_integral

PLEASE ADD A FUNCTION DESCRIPTION

```
exp_min_lk_integral(k: float, dk: float) -> float:
k0 = 0.0
num_steps = int(k / dk)
dk = k / num_steps
cdf0 = 1.0
cdf1 = 0.0
exp_min_lk = 0.0
check_sum = 0.0
for _ in range(0, num_steps):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float	-	-
dk	float)-float:	-	-
k0	-	-	0.0
num_steps	-	-	int(k / dk)
dk	-	-	k / num_steps
cdf0	-	-	1.0
cdf1	-	-	0.0
exp_min_lk	-	-	0.0
check_sum	-	-	0.0

exp_min_lk

PLEASE ADD A FUNCTION DESCRIPTION

```
exp_min_lk(k: float) -> float:
if abs(k) < 1e-6:
    return k
c = NormSInv(._p)
c0 = NormSInv(._p0)
arga = k / (1.0 - ._r) / ._h
argb = (k - (1.0 - ._r0) * ._h0) / (1.0 - ._r) / ._h
if argb < 0:
    raise FinError("Tranche_too_thin_for_LHPlus")
inva = NormSInv(arga)
invb = NormSInv(argb)
rt_one_minus_beta2 = np.sqrt(1.0 - ._beta * ._beta)
a = (c - rt_one_minus_beta2 * inva) / ._beta
b = (c - rt_one_minus_beta2 * invb) / ._beta
r12 = ._beta
r13 = ._beta_0
r23 = ._beta * ._beta_0
e11 = ._p * ._h * (1.0 - ._r)
e12 = ._p0 * ._h0 * (1.0 - ._r0)
e13 = -k * (M(c0, a, ._beta_0) - normcdf(a))
e14 = -((1.0 - ._r0) * ._h0 - k) * M(c0, b, ._beta_0)
term1 = (M(c, a, ._beta)
        + phi3(b, c, c0, r12, r13, r23)
        - phi3(a, c, c0, r12, r13, r23)
        )
e15 = -(1.0 - ._r) * ._h * term1
elk1k2 = e11 + e12 + e13 + e14 + e15
return elk1k2
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float)-float:	-	-
if abs(k) $\leq$ 1e-6		-	-
return k	-	-	-
c	-	-	NormSInv(.p)
c0	-	-	NormSInv(.p0)
arga	-	-	k / (1.0 - .r) / .h
argb	-	-	(k - (1.0 - .r0) * .h0) / (1.0 - .r) / .h
if argb $\leq$ 0		-	-
raise FinError("Tranche too thin for LHPlus")	-	-	-
inva	-	-	NormSInv(arga)
invb	-	-	NormSInv(argb)
rt_one_minus_beta2	-	-	np.sqrt(1.0 - .beta * .beta)
a	-	-	(c - rt_one_minus_beta2 * inva) / .beta
b	-	-	(c - rt_one_minus_beta2 * invb) / .beta
r12	-	-	.beta
r13	-	-	.beta_0
r23	-	-	.beta * .beta_0
el1	-	-	.p * .h * (1.0 - .r)
el2	-	-	.p0 * .h0 * (1.0 - .r0)
el3	-	-	-k * (M(c0, a, .beta_0) - normcdf(a))
el4	-	-	-((1.0 - .r0) * .h0 - k) * M(c0, b, .beta_0)
+ phi3(b, c, c0, r12, r13, r23)	-	-	-
- phi3(a, c, c0, r12, r13, r23)	-	-	-
)	-	-	-
el5	-	-	-(1.0 - .r) * .h * term1
elk1k2	-	-	el1 + el2 + el3 + el4 + el5
return elk1k2	-	-	-

exp_min_lk2

PLEASE ADD A FUNCTION DESCRIPTION

```
exp_min_lk2(k: float) -> float:
if abs(k) < 1e-6:
    return k
c = NormSInv(.p)
c0 = NormSInv(.p0)
arga = k / (1.0 - .r) / .h
argb = (k - (1.0 - .r0) * .h0) / (1.0 - .r) / .h
if argb < 0.0:
    raise FinError("Tranche too thin for LHPlus")
inva = NormSInv(arga)
invb = NormSInv(argb)
root_one_minus_beta2 = np.sqrt(1.0 - .beta * .beta)
a = (1.0 / .beta) * (c - root_one_minus_beta2 * inva)
b = (1.0 / .beta) * (c - root_one_minus_beta2 * invb)
r12 = .beta
```



```

r13 = ._beta_0
r23 = ._beta * ._beta_0
el1 = ._p * ._h * (1.0 - ._r) + ._p0 * ._h0 * (1.0 - ._r0
)
el1 = el1 - k * (M(c0, a, ._beta_0) - normcdf(a))
el1 = el1 - ((1.0 - ._r0) * ._h0 - k) * M(c0, b, ._beta_0)
term = (M(c, a, ._beta)
      + phi3(b, c, c0, r12, r13, r23)
      - phi3(a, c, c0, r12, r13, r23)
)
el1 = el1 - (1.0 - ._r) * ._h * term
return el1
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	float)-float:	-	-
if abs(k) > 1e-6		-	-
return k	-	-	-
c	-	-	NormSInv(.p)
c0	-	-	NormSInv(.p0)
arga	-	-	k / (1.0 - ._r) / ._h
argb	-	-	(k - (1.0 - ._r0) * ._h0) / (1.0 - ._r) / ._h
if argb > 0.0		-	-
raise FinError("Tranche too thin for LHPlus")	-	-	-
inva	-	-	NormSInv(arga)
invb	-	-	NormSInv(argb)
root_one_minus_beta2	-	-	np.sqrt(1.0 - ._beta * ._beta)
a	-	-	(1.0 / ._beta) * (c - root_one_minus_beta2 * i
b	-	-	(1.0 / ._beta) * (c - root_one_minus_beta2 * i
r12	-	-	._beta
r13	-	-	._beta_0
r23	-	-	._beta * ._beta_0
)	-	-	-
el1	-	-	el1 - k * (M(c0, a, ._beta_0) - normcdf(a
el1	-	-	el1 - ((1.0 - ._r0) * ._h0 - k) * M(c0, b, ._bet
+ phi3(b, c, c0, r12, r13, r23)	-	-	-
- phi3(a, c, c0, r12, r13, r23)	-	-	-
)	-	-	-
el1	-	-	el1 - (1.0 - ._r) * ._h * term
return el1	-	-	-

tranche_survival_prob

PLEASE ADD A FUNCTION DESCRIPTION

```

tranche_survival_prob(k1: float, k2: float) -> float:
if k2 == k1:
    raise FinError("tranche_survival_prob:_Same_strikes")
dk = 0.00001
el_k2 = .exp_min_lk_integral(k2, dk)
el_k1 = .exp_min_lk_integral(k1, dk)
q = 1.0 - (el_k2 - el_k1) / (k2 - k1)
return q

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k1	float	-	-
k2	float)-ζfloat:	-	-
if k2	-	-	= k1:
raise FinError("tranche_survival_prob Samestrikes")	-	-	-
dk	-	-	0.00001
el_k2	-	-	.exp_min_lk_integral(k2, dk)
el_k1	-	-	.exp_min_lk_integral(k1, dk)
q	-	-	1.0 - (el_k2 - el_k1) / (k2 - k1)
return q	-	-	-

9.22 gauss_copula_onefactor

loss_dbn_recursion_gcd

PLEASE ADD A FUNCTION DESCRIPTION

```
loss_dbn_recursion_gcd(
    num_credits: int,
    default_probs: Sequence[float],
    loss_units: Sequence[float],
    beta_vector: Sequence[float],
    num_integration_steps: int
    np.ndarray:
    """Full_construction_of_the_loss_distribution_of_a_portfolio_of_credits
    where_losses_have_been_calculate_as_number_of_units_based_on_the_GCD."""
    if len(default_probs) != num_credits:
        raise FinError("Default_probability_length_must_equal_num_credits.")
    if len(loss_units) != num_credits:
        raise FinError("Loss_units_length_must_equal_num_credits.")
    if len(beta_vector) != num_credits:
        raise FinError("Beta_vector_length_must_equal_num_credits.")
    num_loss_units = 1
    for lu in loss_units:
        num_loss_units += int(lu)
    uncond_loss_dbn = np.zeros(num_loss_units)
    z = MIN_Z
    dz = 2.0 * abs(z) / num_integration_steps
    cond_default_probs = np.zeros(num_credits)
    thresholds = np.zeros(num_credits)
    for i_credit in range(0, int(num_credits)):
```

The function arguments are described in the following table.

Argument Name	Type	Description	
num_credits	int	-	
default_probs	Sequence[float]	-	
loss_units	Sequence[float]	-	
beta_vector	Sequence[float]	-	
num_integration_steps	int	-	
np.ndarray		-	
"""Full construction of the loss distribution of a portfolio of credits where losses have been calculate as number of units based on the GCD."""	-	-	
if len(default_probs) !	-	-	
raise FinError("Default probability length must equal num credits.")	-	-	
if len(loss_units) !	-	-	
raise FinError("Loss units length must equal num credits.")	-	-	
if len(beta_vector) !	-	-	
raise FinError("Beta vector length must equal num credits.")	-	-	
num_loss_units	-	-	
for lu in loss_units		-	
num_loss_units +	-	-	
uncond_loss_dbn	-	-	np.zer
z	-	-	
dz	-	-	2.0 * abs(z)
cond.default_probs	-	-	np.z
thresholds	-	-	np.z

homog_basket_loss_dbn

PLEASE ADD A FUNCTION DESCRIPTION

```

homog_basket_loss_dbn(
    survival_probs: Sequence[float],
    recovery_rates: Sequence[float],
    beta_vector: Sequence[float],
    num_integration_steps: int
    np.ndarray:
    """Calculate the loss distribution of a CDS default basket where the
    portfolio is equally weighted and the losses in the portfolio are homo-
    geneous i.e. the credits have the same recovery rates."""
    num_credits = len(survival_probs)
    if num_credits == 0:
        raise FinError("Number of credits equals zero")
    for i_credit in range(1, num_credits):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
survival_probs	Sequence[float]	-	-
recovery_rates	Sequence[float]	-	-
beta_vector	Sequence[float]	-	-
num_integration_steps	int	-	-
np.ndarray		-	-
"""Calculate the loss distribution of a CDS default basket where the portfolio is equally weighted and the losses in the portfolio are homogeneous i.e. the credits have the same recovery rates."""	-	-	-
num_credits	-	-	len(survival_probs)
if num_credits	-	-	= 0:
raise FinError("Number of credits equals zero")	-	-	-

tranche_surv_prob_recursion

PLEASE ADD A FUNCTION DESCRIPTION

```
tranche_surv_prob_recursion(
    k1: float,
    k2: float,
    num_credits: int,
    survival_probs: Sequence[float],
    recovery_rates: Sequence[float],
    beta_vector: Sequence[float],
    num_integration_steps: int
    float:
    """Get the tranche survival probability of a portfolio of credits in the
    one-factor GC model using a full recursion calculation of the loss
    distribution and survival probabilities to some time horizon."""
    if k1 == 0.0 and k2 == 0.0:
        return 0.0
    if k1 >= k2:
        raise FinError("k_1 >= k_2")
    common_recovery_flag = 1
    loss_amounts = np.zeros(num_credits)
    for i_credit in range(0, num_credits):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k1	float	-	-
k2	float	-	-
num_credits	int	-	-
survival_probs	Sequence[float]	-	-
recovery_rates	Sequence[float]	-	-
beta_vector	Sequence[float]	-	-
num_integration_steps	int	-	-
float		-	-
"""Get the tranche survival probability of a portfolio of credits in the			
one-factor GC model using a full recursion calculation of the loss			
distribution and survival probabilities to some time horizon."""			
if k1	-	-	= 0.0 and k2 ==
return 0.0	-	-	-
if k1 <	-	-	k2:
raise FinError("k_1 <	-	-	k_2")
common_recovery_flag	-	-	1
loss_amounts	-	-	np.zeros(num_cre

gauss_approx_tranche_loss

PLEASE ADD A FUNCTION DESCRIPTION

```

gauss_approx_tranche_loss(
k1: float,
k2: float,
mu: float,
sigma: float
    float:
if abs(sigma) < 1e-6:
    tranche_loss = 0.0
    if mu > k1:
        tranche_loss += mu - k1
    if mu > k2:
        tranche_loss += mu - k2
else:
    d1 = (mu - k1) / sigma
    d2 = (mu - k2) / sigma
    expd1 = np.exp(-0.5 * d1 * d1)
    expd2 = np.exp(-0.5 * d2 * d2)
    tranche_loss = (
        (mu - k1) * normcdf(d1)
        - (mu - k2) * normcdf(d2)
        + sigma * expd1 * INV_ROOT_2_PI
        - sigma * expd2 * INV_ROOT_2_PI
    )
return tranche_loss
#####
t(
float64(float64, float64, int64, float64[:,], float64[:,], float64[:,], int64),
fastmath=True,

```

```
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k1	float	-	-
k2	float	-	-
mu	float	-	-
sigma	float	-	-
float	-	-	-
if abs(sigma) > 1e-6	-	-	-
tranche_loss	-	-	0.0
if mu < k1	-	-	-
tranche_loss +	-	-	mu - k1
if mu < k2	-	-	-
tranche_loss +	-	-	mu - k2
else	-	-	-
d1	-	-	(mu - k1) / sigma
d2	-	-	(mu - k2) / sigma
expd1	-	-	np.exp(-0.5 * d1 * d1)
expd2	-	-	np.exp(-0.5 * d2 * d2)
(mu - k1) * normcdf(d1)	-	-	-
- (mu - k2) * normcdf(d2)	-	-	-
+ sigma * expd1 * INV_ROOT_2_PI	-	-	-
- sigma * expd2 * INV_ROOT_2_PI	-	-	-
)	-	-	-
return tranche_loss	-	-	-
float64(float64, float64, int64, float64[],float64[:,],float64[:,],int64)	-	-	-
fastmath	-	-	True
cache	-	-	True

tranch_surv_prob_gaussian

PLEASE ADD A FUNCTION DESCRIPTION

```
tranch_surv_prob_gaussian(
k1: float,
k2: float,
num_credits: int,
survival_probs: Sequence[float],
recovery_rates: Sequence[float],
beta_vector: Sequence[float],
num_integration_steps: int
float:
"""Get the approximated tranche survival probability of a portfolio
of credits in the one-factor GC model using a Gaussian fit of the
conditional loss distribution and survival probabilities to some time
```

```

horizon._Note_that_the_losses_in_this_fit_are_allowed_to_be_negative."""
if k1 == 0.0 and k2 == 0.0:
    return 0.0
if k1 >= k2:
    raise FinError("k_1_>=k_2")
default_probs = [0.0] * num_credits
for i_credit in range(0, num_credits):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k1	float	-	-
k2	float	-	-
num_credits	int	-	-
survival_probs	Sequence[float]	-	-
recovery_rates	Sequence[float]	-	-
beta_vector	Sequence[float]	-	-
num_integration_steps	int	-	-
float		-	-
"""Get the approximated tranche survival probability of a portfolio	-	-	-
of credits in the one-factor GC model using a Gaussian fit of the	-	-	-
conditional loss distribution and survival probabilities to some time	-	-	-
horizon. Note that the losses in this fit are allowed to be negative."""	-	-	-
if k1	-	-	= 0.0 and k2 == 0
return 0.0	-	-	-
if k1 >= k2:	-	-	k2:
raise FinError("k_1_>=k_2")	-	-	k_2")
default_probs	-	-	[0.0] * num_credits

loss_dbn_hetero_adj_binomial

PLEASE ADD A FUNCTION DESCRIPTION

```

loss_dbn_hetero_adj_binomial(
    num_credits: int,
    default_probs: Sequence[float],
    loss_ratio: Sequence[float],
    beta_vector: Sequence[float],
    num_integration_steps: int
) -> np.ndarray:
    """Get the portfolio loss distribution using the adjusted binomial
    approximation to the conditional loss distribution."""
    num_loss_units = num_credits + 1
    cond_default_probs = np.zeros(num_credits)
    uncond_loss_dbn = np.zeros(num_loss_units)
    indep_dbn = np.zeros(num_loss_units)
    # Determine default threshold for each credit
    thresholds = np.zeros(num_credits)
    for i_credit in range(0, num_credits):

```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_credits	int	-	-
default_probs	Sequence[float]	-	-
loss_ratio	Sequence[float]	-	-
beta_vector	Sequence[float]	-	-
num_integration_steps	int	-	-
np.ndarray		-	-
"""Get the portfolio loss distribution using the adjusted binomial approximation to the conditional loss distribution."""		-	-
num_loss_units	-	-	num_credits + 1
cond_default_probs	-	-	np.zeros(num_credits)
uncond_loss_dbn	-	-	np.zeros(num_loss_units)
indep_dbn	-	-	np.zeros(num_loss_units)
thresholds	-	-	np.zeros(num_credits)

tranche_surv_prob_adj_binomial

PLEASE ADD A FUNCTION DESCRIPTION

```
tranche_surv_prob_adj_binomial(
    k1: float,
    k2: float,
    num_credits: int,
    survival_probs: Sequence[float],
    recovery_rates: Sequence[float],
    beta_vector: Sequence[float],
    num_integration_steps: int
    float:
    """Get the approximated tranche survival probability of a portfolio of
    credits in the one-factor GC model using the adjusted binomial fit of the
    conditional loss distribution and survival probabilities to some time
    horizon. This approach is both fast and highly accurate."""
    if k1 == 0.0 and k2 == 0.0:
        return 0.0
    if k1 >= k2:
        raise FinError("k1 >= k2")
    default_probs = np.zeros(num_credits)
    for i_credit in range(0, num_credits):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k1	float	-	-
k2	float	-	-
num_credits	int	-	-
survival_probs	Sequence[float]	-	-
recovery_rates	Sequence[float]	-	-
beta_vector	Sequence[float]	-	-
num_integration_steps	int	-	-
float		-	-
"""Get the approximated tranche survival probability of a portfolio of	-	-	-
credits in the one-factor GC model using the adjusted binomial fit of the	-	-	-
conditional loss distribution and survival probabilities to some time	-	-	-
horizon. This approach is both fast and highly accurate."""	-	-	-
if k1	-	-	= 0.0 and k2 :
return 0.0	-	-	-
if k1 < 0	-	-	k2:
raise ValueError("k_1 < 0")	-	-	k_2")
default_probs	-	-	np.zeros(num_

9.23 gbm_process_simulator

get_paths_times

PLEASE ADD A FUNCTION DESCRIPTION

```

get_paths_times(
    num_paths: int,
    num_time_steps: int,
    t: float,
    mu: float,
    stock_price: float,
    volatility: float,
    seed: int
) Tuple[np.ndarray, np.ndarray]:
    """Get the simulated GBM process for a single asset with even num_paths and
    time_steps. Inputs include the number of time_steps, paths, the drift mu,
    stock_price, volatility and a seed.
    Parameters:
    - num_paths: int, number of paths (must be even)
    - num_time_steps: int, number of time_steps
    - t: float, total time in years
    - mu: float, annual drift (mean return)
    - stock_price: float, initial stock price
    - volatility: float, annual volatility
    - seed: int, random seed for reproducibility
    """
    if num_paths % 2 == 0:
        num_paths_even = num_paths // 2
    else:
        raise FinError("Number of paths must be an even number.")
    np.random.seed(seed)
    dt = t / num_time_steps
    vsqrt_dt = volatility * np.sqrt(dt)
    m = np.exp((mu - (volatility**2) / 2) * dt)
    t_all = np.linspace(0.0, t, num_time_steps + 1)
    s_all = np.empty((num_paths, num_time_steps + 1))
    s_all[:, 0] = stock_price
    for it in range(1, num_time_steps + 1):

```

The function arguments are described in the following table.

Argument Name	Type	Description	De
num_paths	int	-	
num_time_steps	int	-	
t	float	-	
mu	float	-	
stock_price	float	-	
volatility	float	-	
seed	int	-	
Tuple[np.ndarray, np.ndarray]		-	
"""Get the simulated GBM process for a single asset with even num paths and	-	-	
time steps. Inputs include the number of time steps	-	-	
paths	-	-	
the drift mu	-	-	
stock price	-	-	
volatility and a seed.	-	-	
Parameters		-	
- num_paths	int	-	
number of paths (must be even)	-	-	
- num_time_steps	int	-	
number of time steps	-	-	
- t	float	-	
total time in years	-	-	
- mu	float	-	
annual drift (mean return)	-	-	
- stock_price	float	-	
initial stock price	-	-	
- volatility	float	-	
annual volatility	-	-	
- seed	int	-	
random seed for reproducibility	-	-	
"""	-	-	
if num_paths % 2	-	-	
num_paths_even	-	-	num
else		-	
raise FinError("Number of paths must be an even number.")	-	-	
np.random.seed(seed)	-	-	
dt	-	-	t / nu
vsqrt_dt	-	-	volatili
m	-	-	np.exp((mu - (
t_all	-	-	np.linspace(0.0
s_all	-	-	np.empty((num_p
s_all[	,0]	-	st

get_assets_paths_times*PLEASE ADD A FUNCTION DESCRIPTION*

```

get_assets_paths_times(
    num_assets: int,
    num_paths: int,
    num_time_steps: int,
    t: float,
    mus: np.ndarray,
    stock_prices: np.ndarray,
    volatilities: np.ndarray,
    corr_matrix: np.ndarray,
    seed: int
) Tuple[np.ndarray, np.ndarray]:
    """Get the simulated GBM process for a number of assets and paths and num
    time_steps. Inputs include the number of assets, paths, the vector of mus,
    stock prices, volatilities, a correlation matrix and a seed.
    Parameters:
    -_num_assets: int, number of assets
    -_num_paths: int, number of paths (must be even)
    -_num_time_steps: int, number of time steps
    -_t: float, total time in years
    -_mus: ndarray, annual drift rates for each asset
    -_stock_prices: ndarray, initial stock prices for each asset
    -_volatilities: ndarray, annual volatilities for each asset
    -_corr_matrix: ndarray, correlation matrix between assets
    -_seed: int, random seed for reproducibility
    """
    if num_paths % 2 == 0:
        num_paths_even = num_paths // 2
    else:
        raise FinError("Number of paths must be an even number.")
    if stock_prices.shape[0] != num_assets:
        raise FinError("Stock price vector incorrect size.")
    if volatilities.shape[0] != num_assets:
        raise FinError("Volatilities vector incorrect size.")
    if mus.shape[0] != num_assets:
        raise FinError("Drift mu vector incorrect size.")
    if corr_matrix.shape[0] != num_assets:
        raise FinError("Correlation matrix incorrect size.")
    if corr_matrix.shape[1] != num_assets:
        raise FinError("Correlation matrix incorrect size.")
    np.random.seed(seed)
    dt = t / num_time_steps
    vsqrt_dts = volatilities * np.sqrt(dt)
    m = np.exp((mus - volatilities * volatilities / 2.0) * dt)
    s_all = np.empty((num_assets, num_paths, num_time_steps + 1))
    t_all = np.linspace(0, t, num_time_steps + 1)
    g = np.random.standard_normal((num_paths_even, num_time_steps + 1, num_assets))
    c = cholesky(corr_matrix)
    g_corr = np.empty((num_paths_even, num_time_steps + 1, num_assets))
    # or use g_corr = np.einsum('ijk,kl->ijl', g, c)
    # Calculate the Cholesky dot product
    for ip in nb.prange(num_paths_even):

```

The function arguments are described in the following table.

Argument Name	Type	Description
num_assets	int	-
num_paths	int	-
num_time_steps	int	-
t	float	-
mus	np.ndarray	-
stock_prices	np.ndarray	-
volatilities	np.ndarray	-
corr_matrix	np.ndarray	-
seed	int	-
Tuple[np.ndarray, np.ndarray]		-
"""Get the simulated GBM process for a number of assets and paths and num time steps. Inputs include the number of assets	-	-
paths	-	-
the vector of mus	-	-
stock prices	-	-
volatilities	-	-
a correlation matrix and a seed.	-	-
Parameters		-
- num_assets	int	-
number of assets	-	-
- num_paths	int	-
number of paths (must be even)	-	-
- num_time_steps	int	-
number of time steps	-	-
- t	float	-
total time in years	-	-
- mus	ndarray	-
annual drift rates for each asset	-	-
- stock_prices	ndarray	-
initial stock prices for each asset	-	-
- volatilities	ndarray	-
annual volatilities for each asset	-	-
- corr_matrix	ndarray	-
correlation matrix between assets	-	-
- seed	int	-
random seed for reproducibility	-	-
"""	-	-
if num_paths % 2	-	-
num_paths_even	-	-
else		-
raise FinError("Number of paths must be an even number.")	-	-
if stock_prices.shape[0] !	-	-
raise FinError("Stock price vector incorrect size.")	-	-
if volatilities.shape[0] !	-	-
raise FinError("Volatilities vector incorrect size.")	-	-
if mus.shape[0] !	-	-
raise FinError("Drift mu vector incorrect size.")	-	-
if corr_matrix.shape[0] !	-	-
raise FinError("Correlation matrix incorrect size.")	-	-
if corr_matrix.shape[1] !	-	-
raise FinError("Correlation matrix incorrect size.")	-	-

get_assets_paths*PLEASE ADD A FUNCTION DESCRIPTION*

```

get_assets_paths(
    num_assets: int,
    num_paths: int,
    t: float,
    mus: np.ndarray,
    stock_prices: np.ndarray,
    volatilities: np.ndarray,
    corr_matrix: np.ndarray,
    seed: int
) Tuple[np.ndarray, np.ndarray]:
    """Get the simulated GBM process for a number of assets and paths for one
    time step. Inputs include the number of assets, paths, the vector of mus,
    stock prices, volatilities, a correlation matrix and a seed.
    Parameters:
    - num_assets: int, number of assets
    - num_paths: int, number of paths (must be even)
    - t: float, total time in years
    - mus: ndarray, annual drift rates for each asset
    - stock_prices: ndarray, initial stock prices for each asset
    - volatilities: ndarray, annual volatilities for each asset
    - corr_matrix: ndarray, correlation matrix between assets
    - seed: int, random seed for reproducibility
    """
    if num_paths % 2 == 0:
        num_paths_even = num_paths // 2
    else:
        raise FinError("Number of paths must be an even number.")
    if stock_prices.shape[0] != num_assets:
        raise FinError("Stock price vector incorrect size.")
    if volatilities.shape[0] != num_assets:
        raise FinError("Volatilities vector incorrect size.")
    if mus.shape[0] != num_assets:
        raise FinError("Drift mu vector incorrect size.")
    if corr_matrix.shape[0] != num_assets or corr_matrix.shape[1] != num_assets:
        raise FinError("Correlation matrix incorrect size.")
    np.random.seed(seed)
    vsqrt_dts = volatilities * np.sqrt(t)
    m = np.exp((mus - volatilities * volatilities / 2.0) * t)
    s_all = np.empty((num_assets, 2 * num_paths_even))
    num_time_steps = 1
    t_all = np.linspace(0, t, num_time_steps + 1)
    g = np.random.standard_normal((num_paths_even, num_assets))
    c = cholesky(corr_matrix)
    g_corr = np.empty((num_paths_even, num_assets))
    # Calculate the dot product
    for ia in range(0, num_assets):

```

The function arguments are described in the following table.

Argument Name	Type	Description
num_assets	int	-
num_paths	int	-
t	float	-
mus	np.ndarray	-
stock_prices	np.ndarray	-
volatilities	np.ndarray	-
corr_matrix	np.ndarray	-
seed	int	-
Tuple[np.ndarray, np.ndarray]		-
"""Get the simulated GBM process for a number of assets and paths for one time step. Inputs include the number of assets	-	-
paths	-	-
the vector of mus	-	-
stock prices	-	-
volatilities	-	-
a correlation matrix and a seed.	-	-
Parameters		-
- num_assets	int	-
number of assets	-	-
- num_paths	int	-
number of paths (must be even)	-	-
- t	float	-
total time in years	-	-
- mus	ndarray	-
annual drift rates for each asset	-	-
- stock_prices	ndarray	-
initial stock prices for each asset	-	-
- volatilities	ndarray	-
annual volatilities for each asset	-	-
- corr_matrix	ndarray	-
correlation matrix between assets	-	-
- seed	int	-
random seed for reproducibility	-	-
"""	-	-
if num_paths % 2	-	-
num_paths_even	-	-
else		-
raise FinError("Number of paths must be an even number.")	-	-
if stock_prices.shape[0] !	-	-
raise FinError("Stock price vector incorrect size.")	-	-
if volatilities.shape[0] !	-	-
raise FinError("Volatilities vector incorrect size.")	-	-
if mus.shape[0] !	-	-
raise FinError("Drift mu vector incorrect size.")	-	-
if corr_matrix.shape[0] !	-	-
raise FinError("Correlation matrix incorrect size.")	-	-
np.random.seed(seed)	-	-
vsqrt_dts	-	-
m	-	np.exp((n
s_all	-	np.empty
num_time_steps		

9.24 heston

Enumerated Type: HestonNumericalScheme

This enumerated type has the following values:

- EULER
- EULERLOG
- QUADEXP
- cache
- parallel
- num_steps
- s_paths
- s_paths[:,0]
- sdt
- rhohat
- sigma2
- ifscheme
- s
- v
- z1
- z2
- z_v
- z_s
- vplus
- rtvplus
- v+
- s+
- s_paths[i_path,i_step]
- elifscheme
- x
- v
- z_v
- z_s
- vplus
- rtvplus
- x+
- v+
- s_paths[i_path,i_step]
- elifscheme
- qq
- psic
- gamma1
- gamma2
- k_0
- k_1
- k_2

- k_3
- k_4
- aa
- mu
- c1
- c2
- x
- vn
- z_v
- z_s
- m
- m2
- s2
- psi
- u
- ifpsi_i
- b2
- a
- b
- z_v
- vnp
- d
- m
- k_0
- p
- beta
- ifu_j
- vnp
- vnp
- m
- k_0
- x+
- s_paths[i_path,i_step]
- vn
- verbose
- if2.0*kappa*theta_j
- self._v0
- self._kappa
- self._theta
- self._sigma
- self._rho
- scheme:HestonNumericalScheme
- tau
- k
- dt
- scheme_value

- s_paths
- ifoption.opt_type
- path_payoff
- elifoption.opt_type
- path_payoff
- payoff
- v
- tau
- rho
- sigma
- v0
- kappa
- theta
- r
- q
- s0
- kk
- ff
- vv
- k
- b
- d
- g
- t_m
- qq
- t
- ww
- phi
- x
- i_1
- v1
- v2
- tau
- rho
- sigma
- v0
- kappa
- theta
- q
- r
- vv
- s0
- f
- k
- x
- k

- b
- e
- g
- h
- q
- qq
- hh
- integrand
- integral
- v
- tau
- rho
- sigma
- v0
- kappa
- theta
- q
- r
- s0
- k
- vv
- beta
- d
- g
- qq
- bb
- aa
- v
- area
- v
- v
- tau
- rho
- sigma
- v0
- kappa
- theta
- q
- r
- s0
- k
- f
- x0
- vv
- aa
- bb

- gg
- d
- rplus
- rminus
- rr
- qq
- dd
- cc
- phi
- area
- v

Class: Heston

class Heston:

Heston

PLEASE ADD A FUNCTION DESCRIPTION

```
Heston(v0: float, kappa: float, theta: float, sigma: float, rho: float
None:
verbose = False
if 2.0 * kappa * theta <= sigma and verbose:
    print("Feller_condition_not_satisfied._Zero_Variance_possible")
._v0 = v0
._kappa = kappa
._theta = theta
._sigma = sigma
._rho = rho
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
v0	float	-	-
kappa	float	-	-
theta	float	-	-
sigma	float	-	-
rho	float	-	-
None		-	-
verbose	-	-	False
if 2.0 * kappa * theta <=	-	-	sigma and verbose:
print("Feller condition not satisfied. Zero Variance possible")	-	-	-
._v0	-	-	v0
._kappa	-	-	kappa
._theta	-	-	theta
._sigma	-	-	sigma
._rho	-	-	rho

value_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

value_mc(value_dt: float,
option: Any,
stock_price: float,
interest_rate: float,
dividend_yield: float,
num_paths: int,
num_steps_per_year: int,
seed: int,
scheme: HestonNumericalScheme = HestonNumericalScheme.EULERLOG,
float:
tau = (option.expiry_dt - value_dt) / G_DAYS_IN_YEAR
k = option.strike_price
dt = 1.0 / num_steps_per_year
scheme_value = float(scheme.value)
s_paths = get_paths(stock_price,
interest_rate,
dividend_yield,
._v0,
._kappa,
._theta,
._sigma,
._rho,
tau,
dt,
num_paths,
seed,
scheme_value,
)
if option.opt_type == OptionTypes.EUROPEAN_CALL:
path_payoff = np.maximum(s_paths[:, -1] - k, 0.0)
elif option.opt_type == OptionTypes.EUROPEAN_PUT:
path_payoff = np.maximum(k - s_paths[:, -1], 0.0)

```

```
else:
    raise FinError("Unknown_option_type.")
payoff = np.mean(path_payoff)
v = payoff * exp(-interest_rate * tau)
return v
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	float	-	-
option	Any	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
num_paths	int	-	-
num_steps_per_year	int	-	-
seed	int	-	-
scheme	HestonNumericalScheme	-	HestonNumericalScheme.EUL
float		-	-
tau	-	-	(option.expiry_dt - value_dt) / G.DA
k	-	-	option.strike_price
dt	-	-	1.0 / num_steps_per_yea
scheme_value	-	-	float(scheme.value)
interest_rate	-	-	-
dividend_yield	-	-	-
._v0	-	-	-
._kappa	-	-	-
._theta	-	-	-
._sigma	-	-	-
._rho	-	-	-
tau	-	-	-
dt	-	-	-
num_paths	-	-	-
seed	-	-	-
scheme_value	-	-	-
)	-	-	-
if option.opt_type	-	-	= OptionTypes.EUROPEAN_
path_payoff	-	-	np.maximum(s_paths[:, -1] -
elif option.opt_type	-	-	= OptionTypes.EUROPEAN
path_payoff	-	-	np.maximum(k - s_paths[:, -1]
else	-	-	-
raise FinError("Unknown option type.")	-	-	-
payoff	-	-	np.mean(path_payoff)
v	-	-	payoff * exp(-interest_rate *
return v	-	-	-

value_lewis

PLEASE ADD A FUNCTION DESCRIPTION

```
value_lewis(value_dt: float,
option: Any,
stock_price: float,
interest_rate: float,
```

```

dividend_yield: float,
float:
tau = (option.expiry_dt - value_dt) / G_DAYS_IN_YEAR
rho = ._rho
sigma = ._sigma
v0 = ._v0
kappa = ._kappa
theta = ._theta
r = interest_rate
q = dividend_yield
s0 = stock_price
kk = option.strike_price
ff = s0 * exp((r - q) * tau)
vv = sigma * sigma

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	float	-	-
option	Any	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
float	-	-	-
tau	-	(option.expiry_dt - value_dt) / G_DAYS_IN_YEAR	
rho	-	-	._rho
sigma	-	-	._sigma
v0	-	-	._v0
kappa	-	-	._kappa
theta	-	-	._theta
r	-	-	interest_rate
q	-	-	dividend_yield
s0	-	-	stock_price
kk	-	-	option.strike_price
ff	-	-	s0 * exp((r - q) * tau)
vv	-	-	sigma * sigma

phi

PLEASE ADD A FUNCTION DESCRIPTION

```

phi(k_in,
k = k_in + 0.5 * 1j
b = kappa + 1j * rho * sigma * k
d = np.sqrt(b**2 + vv * k * (k - 1j))
g = (b - d) / (b + d)
t_m = (b - d) / vv
qq = np.exp(-d * tau)
t = t_m * (1.0 - qq) / (1.0 - g * qq)
ww = (kappa

```

```

    * theta
    * (tau * t_m - 2.0 * np.log((1.0 - g * qq) / (1.0 - g)) / vv)
)
phi = np.exp(ww + v0 * t)
return phi

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k_in	-	-	-
k	-	-	k_in + 0.5 * l_j
b	-	-	kappa + l_j * rho * sigma * k
d	-	-	np.sqrt(b**2 + vv * k * (k - l_j))
g	-	-	(b - d) / (b + d)
t_m	-	-	(b - d) / vv
qq	-	-	np.exp(-d * tau)
t	-	-	t_m * (1.0 - qq) / (1.0 - g * qq)
* theta	-	-	-
* (tau * t_m - 2.0 * np.log((1.0 - g * qq) / (1.0 - g)) / vv)	-	-	-
)	-	-	-
phi	-	-	np.exp(ww + v0 * t)
return phi	-	-	-

phi_transform

PLEASE ADD A FUNCTION DESCRIPTION

```
phi_transform(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

integrand

PLEASE ADD A FUNCTION DESCRIPTION

```
integrand(k):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k	-	-	-

value_lewis_rouah*PLEASE ADD A FUNCTION DESCRIPTION*

```

value_lewis_rouah(value_dt: float,
option: Any,
stock_price: float,
interest_rate: float,
dividend_yield: float,
float:
tau = (option.expiry_dt - value_dt) / G_DAYS_IN_YEAR
rho = ._rho
sigma = ._sigma
v0 = ._v0
kappa = ._kappa
theta = ._theta
q = dividend_yield
r = interest_rate
vv = sigma * sigma
s0 = stock_price
f = s0 * exp((r - q) * tau)
k = option.strike_price
x = log(f / k)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	float	-	-
option	Any	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
float		-	-
tau	-	(option.expiry_dt - value_dt) / G_DAYS_IN_YEAR	
rho	-	-	._rho
sigma	-	-	._sigma
v0	-	-	._v0
kappa	-	-	._kappa
theta	-	-	._theta
q	-	-	dividend_yield
r	-	-	interest_rate
vv	-	-	sigma * sigma
s0	-	-	stock_price
f	-	-	s0 * exp((r - q) * tau)
k	-	-	option.strike_price
x	-	-	log(f / k)

fn*PLEASE ADD A FUNCTION DESCRIPTION*

```
fn(k_in):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
k_in	-	-	-

value_weber

PLEASE ADD A FUNCTION DESCRIPTION

```
value_weber(value_dt: float,
option: Any,
stock_price: float,
interest_rate: float,
dividend_yield: float,
float:
tau = (option.expiry_dt - value_dt) / G_DAYS_IN_YEAR
rho = ._rho
sigma = ._sigma
v0 = ._v0
kappa = ._kappa
theta = ._theta
q = dividend_yield
r = interest_rate
s0 = stock_price
k = option.strike_price
vv = sigma**2
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	float	-	-
option	Any	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
float	-	-	-
tau	-	(option.expiry_dt - value_dt) / G_DAYS_IN_YEAR	
rho	-	-	._rho
sigma	-	-	._sigma
v0	-	-	._v0
kappa	-	-	._kappa
theta	-	-	._theta
q	-	-	dividend_yield
r	-	-	interest_rate
s0	-	-	stock_price
k	-	-	option.strike_price
vv	-	-	sigma**2

fn*PLEASE ADD A FUNCTION DESCRIPTION*

```
fn(s, b):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	-	-	-
b	-	-	-

integrand*PLEASE ADD A FUNCTION DESCRIPTION*

```
integrand(u):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
u	-	-	-

value_gatheral*PLEASE ADD A FUNCTION DESCRIPTION*

```
value_gatheral(value_dt: float,
option: Any,
stock_price: float,
interest_rate: float,
dividend_yield: float,
float:
tau = (option.expiry_dt - value_dt) / G_DAYS_IN_YEAR
rho = ._rho
sigma = ._sigma
v0 = ._v0
kappa = ._kappa
theta = ._theta
q = dividend_yield
r = interest_rate
s0 = stock_price
k = option.strike_price
f = s0 * exp((r - q) * tau)
x0 = log(f / k)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
value_dt	float	-	-
option	Any	-	-
stock_price	float	-	-
interest_rate	float	-	-
dividend_yield	float	-	-
float	-	-	-
tau	-	(option.expiry_dt - value_dt) / G_DAYS_IN_YEAR	-
rho	-	-	._rho
sigma	-	-	._sigma
v0	-	-	._v0
kappa	-	-	._kappa
theta	-	-	._theta
q	-	-	dividend_yield
r	-	-	interest_rate
s0	-	-	stock_price
k	-	-	option.strike_price
f	-	-	s0 * exp((r - q) * tau)
x0	-	-	log(f / k)

ff

PLEASE ADD A FUNCTION DESCRIPTION

```
ff(j) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
j	-	-	-

integrand

PLEASE ADD A FUNCTION DESCRIPTION

```
integrand(u) :
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
u	-	-	-

get_paths

PLEASE ADD A FUNCTION DESCRIPTION

```
get_paths(  
    s0: float,  
    r: float,  
    q: float,  
    v0: float,  
    kappa: float,  
    theta: float,  
    sigma: float,  
    rho: float,  
    t: float,  
    dt: float,  
    num_paths: int,  
    seed: int,  
    scheme: float,  
    np.ndarray:  
    np.random.seed(seed)  
    num_steps = int(t / dt)  
    s_paths = np.zeros(shape=(num_paths, num_steps))  
    s_paths[:, 0] = s0  
    sdt = np.sqrt(dt)  
    rhohat = np.sqrt(1.0 - rho * rho)  
    sigma2 = sigma * sigma  
    if scheme == HestonNumericalScheme.EULER.value:  
        # Basic scheme to first order with truncation on variance  
        for i_path in range(0, num_paths):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s0	float	-	-
r	float	-	-
q	float	-	-
v0	float	-	-
kappa	float	-	-
theta	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
dt	float	-	-
num_paths	int	-	-
seed	int	-	-
scheme	float	-	-
np.ndarray		-	-
np.random.seed(seed)	-	-	-
num_steps	-	-	int(t / dt)
s_paths	-	-	np.zeros(shape=(num_paths, num_steps))
s_paths[	,0]	-	s0
sdt	-	-	np.sqrt(dt)
rhohat	-	-	np.sqrt(1.0 - rho * rho)
sigma2	-	-	sigma * sigma
if scheme	-	-	= HestonNumericalScheme.EULER.value:

9.25 hw_tree

Enumerated Type: FinHWEuropeanCalcType

This enumerated type has the following values:

- JAMSHIDIAN
- EXPIRY_ONLY
- EXPIRY_TREE
- int:1
- ifoption_exercise_type
- ifoption_exercise_type
- ifoption_exercise_type
- @njit(fastmath
- _a
- bt_t
- bt_delta
- term1
- term2
- log_ahat
- bhatt_t
- p
- @njit(fastmath
- tree_maturity
- dt
- d_r
- j_max
- n
- pu
- pm
- pd
- r_t
- Branchingissimpletrinomialouttotimestepm
- ajdt
- j_n
- ifj
- pu[j_n]
- pm[j_n]
- pd[j_n]
- elifj
- pu[j_n]
- pm[j_n]
- pd[j_n]
- pu[j_n]
- pm[j_n]
- pd[j_n]
- qq

- alpha
- qq[0,n]
- nm
- sum_qz
- rdt
- sum_qz+
- alpha[m]
- j_n
- r_t[m,j_n]
- j_n
- rdt
- z
- ifj
- qq[m+1,j_n]+
- qq[m+1,j_n-1]+
- qq[m+1,j_n-2]+
- elifj
- qq[m+1,j_n]+
- qq[m+1,j_n+1]+
- qq[m+1,j_n+2]+
- qq[m+1,j_n+1]+
- qq[m+1,j_n]+
- qq[m+1,j_n-1]+
- @njit(fastmath
- debug
- num_time_steps,num_nodes
- dt
- j_max
- expiry_step
- tree_flows
- num_cpns
- t_cpn
- ift_cpn;
- n
- ttree
- df_flow
- df_tree
- tree_flows[n]+
- mapped_times
- mapped_amounts
- mapped_times
- mapped_amounts
- mapped_times
- mapped_amounts
- accrued
- ttree

- accrued[m]
- accrued[m]*
- accrued[m]
- call_option_values
- put_option_values
- bond_values
- pt_exp
- ptdelta
- cpn
- zcb
- nm
- k_n
- r_t
- bond_price
- tflow
- iftflow_t
- ptflow
- zcb
- cpn
- bond_price+
- bond_price+
- bond_values[expiry_step,k_n]
- nm
- k_n
- dirty_price
- clean_price
- call_exercise
- put_exercise
- call_option_values[expiry_step,k_n]
- put_option_values[expiry_step,k_n]
- m
- nm
- flow
- k_n
- r
- df
- pu
- pm
- pd
- ifk
- vu
- vm
- vd
- v
- bond_values[m,k_n]
- elifk

- vu
- vm
- vd
- v
- bond_values[m,k_n]
- vu
- vm
- vd
- v
- bond_values[m,k_n]
- bond_values[m,k_n]+
- vcall
- vput
- ifk
- vu
- vm
- vd
- vcall
- elifk
- vu
- vm
- vd
- vcall
- vu
- vm
- vd
- vcall
- call_option_values[m,k_n]
- ifk
- vu
- vm
- vd
- vput
- elifk
- vu
- vm
- vd
- vput
- vu
- vm
- vd
- vput
- put_option_values[m,k_n]
- dirty_price
- clean_price
- call_exercise

- put_exercise
- hold_call
- hold_put
- ifm
- call_option_values[m,k_n]
- put_option_values[m,k_n]
- elifexercise_type_int
- call_option_values[m,k_n]
- put_option_values[m,k_n]
- @njit(fastmath
- num_time_steps,num_nodes
- j_max
- expiry_step
- maturity_step
- fixed_leg_flows
- float_leg_values
- num_cpns
- t_cpn
- n
- ttree
- df_flow
- df_tree
- fixed_leg_flows[n]+
- float_leg_values[n]
- mapped_times
- mapped_amounts
- accd_at_expiry
- if_tree_times[n-1];t_expand_tree_times[n];
- mapped_times
- mapped_amounts
- mapped_times
- mapped_amounts
- accrued
- ttree
- accrued[m]
- accrued[m]*
- accrued[m]
- fixed_leg_values
- pay_values
- rec_values
- flow
- fixed_leg_values[maturity_step,k]
- n
- nm
- flow
- k_n

- r_t
- df
- pu
- pm
- pd
- ifk
- vu
- vm
- vd
- v
- fixed_leg_values[m,k_n]
- elifk
- vu
- vm
- vd
- v
- fixed_leg_values[m,k_n]
- vu
- vm
- vd
- v
- fixed_leg_values[m,k_n]
- fixed_leg_values[m,k_n]+
- vpay
- vrec
- ifk
- vu
- vm
- vd
- vpay
- elifk
- vu
- vm
- vd
- vpay
- vu
- vm
- vd
- vpay
- pay_values[m,k_n]
- ifk
- vu
- vm
- vd
- vrec
- elifk

- vu
- vm
- vd
- vrec
- vu
- vm
- vd
- vrec
- rec_values[m,k_n]
- hold_pay
- hold_rec
- fixed_leg_value
- float_leg_value
- pay_exercise
- rec_exercise
- ifm
- pay_values[m,k_n]
- rec_values[m,k_n]
- elifexercise_type_int
- pay_values[m,k_n]
- rec_values[m,k_n]
- elifexercise_type_int
- pay_values[m,k_n]
- rec_values[m,k_n]
- @njit(fastmath
- num_time_steps,num_nodes
- dt
- j_max
- t_mat
- maturity_step
- tree_flows
- num_cpns
- t_cpn
- n
- ttree
- df_flow
- df_tree
- tree_flows[n]+
- mapped_times
- mapped_amounts
- mapped_times
- mapped_amounts
- accrued
- ttree
- accrued[m]
- accrued[m]*

- accrued[m]
- tree_call_value
- num_calls
- call_time
- n
- tree_call_value[n]
- tree_put_value
- num_puts
- put_time
- n
- tree_put_value[n]
- call_put_bond_values
- bond_values
- debug
- df
- px
- flow
- t
- df
- pv
- px+
- px+
- m
- nm
- vcall
- vput
- vhold
- vclean
- value
- k_n
- bond_values[m,k_n]
- call_put_bond_values[m,k_n]
- nm
- flow
- vcall
- vput
- k_n
- r_t
- df
- pu
- pm
- pd
- ifk
- vu
- vm
- vd

- elifk
- vu
- vm
- vd
- vu
- vm
- vd
- v
- bond_values[m,k_n]
- bond_values[m,k_n]+
- ifk
- vu
- vm
- vd
- elifk
- vu
- vm
- vd
- vu
- vm
- vd
- vhold
- vhold
- value
- call_put_bond_values[m,k_n]
- self
- t_exp
- cpn_times
- cpn_amounts
- df_times
- df_values
- strike_price
- face
- dt
- tdelta
- pt_exp
- ptdelta
- num_flows
- pv
- t_cpn
- cpn
- pt_cpn
- zcb
- pv
- ift_cpn_i
- pv

- `accd`
- `pvclean`
- `obj`
- `num_time_steps:int`
- `european_calc_type:FinHWEuropeanCalcType`
- `dr`
- `raiseFinError("Meanreversionspeedparametersshouldbe;`
- `self.sigma`
- `self.a`
- `self.num_time_steps`
- `self.european_calc_type`
- `self.qq`
- `self.r`
- `self.tree_times`
- `self.pu`
- `self.pm`
- `self.pd`
- `self.discount_curve`
- `self.tree_built`
- `self.df_times`
- `self.dfs`
- `self.r_t`
- `self.dt`
- `pt_exp`
- `pt_mat`
- `sigma`
- `a`
- `a`
- `sigmap`
- `sigmap`
- `sigmap`
- `h`
- `call_value`
- `put_value`
- `num_cpns`
- `argtuple`
- `x0`
- `rstar`
- `x0`
- `fprime`
- `args`
- `tol`
- `maxiter`
- `fprime2`
- `dt`
- `pt_exp`

- ptdelta
- call_value
- put_value
- t_cpn
- cpn
- ift_cpn_i
- pt_cpn
- strike
- v
- call
- put
- call_value+
- put_value+
- call_value+
- put_value+
- dt
- tdelta
- pt_exp
- ptdelta
- _,num_nodes
- expiry_step
- call_value
- put_value
- num_cpns
- q
- r_t
- pv
- t_cpn
- cpn
- ift_cpn_i
- pt_cpn
- zcb
- pv+
- pv+
- accrued
- pv
- put_payoff
- call_payoff
- put_value+
- call_value+
- raiseFinError("Treeexpirymustbe_i
- dt
- tdelta
- pt_exp
- ptdelta
- pt_mat

- `_,num_nodes`
- `expiry_step`
- `call_value`
- `put_value`
- `q`
- `r_t`
- `zcb`
- `put_payoff`
- `call_payoff`
- `put_value+`
- `call_value+`
- `exercise_type_int`
- `t_mat`
- `pay_value,rec_value`
- `exercise_type_int`
- `ifexercise_type_int`
- `ifself.european_calc_type`
- `v`
- `call_value`
- `put_value`
- `elifself.european_calc_type`
- `v`
- `call_value`
- `put_value`
- `elifself.european_calc_type`
- `call_value,put_value`
- `call_value,put_value`
- `cpn_times`
- `cpn_flows`
- `call_times`
- `put_times`
- `call_prices`
- `put_prices`
- `v`
- `ift_mat`
- `_,num_nodes`
- `fn1`
- `fn2`
- `time_step`
- `p`
- `ad`
- `p+`
- `zero_rate`
- `tree_maturity`
- `tree_times`
- `self.tree_times`

- df_tree
- df_tree[0]
- t
- df_tree[i]
- self.df_times
- self.dfs
- self.qq,self.pu,self.pm,self.pd,self.r_t,self.dt
- s
- s+
- s+
- s+
- s+

Class: HWTree

class HWTree:

HWTree

*Constructs the Hull-White rate model. The speed of mean reversion a and volatility are passed in. The short rate process is given by $dr = (\theta(t) - ar) * dt + \sigma * dW$. The model will switch to use Jamshidian's approach where possible unless the useJamshidian flag is set to false in which case it uses the trinomial Tree.*

```
HWTree(sigma: float,
a: float,
num_time_steps: int = 100,
european_calc_type: FinHWEuropeanCalcType = FinHWEuropeanCalcType.EXPIRY_TREE,
None:
    """Constructs the Hull-White rate model. The speed of mean reversion
    a and volatility are passed in. The short rate process is given by
    dr = (theta(t) - ar) * dt + sigma * dW. The model will switch to use
    Jamshidian's approach where possible unless the useJamshidian flag is
    set to false in which case it uses the trinomial Tree."""
    if sigma < 0.0:
        raise FinError("Negative volatility not allowed.")
    if a < 0.0:
        raise FinError("Mean reversion speed parameter should be >= 0.")
    .sigma = sigma
    .a = a
    .num_time_steps = num_time_steps
    .european_calc_type = european_calc_type
    .qq = None
    .r = None
    .tree_times = None
    .pu = None
    .pm = None
    .pd = None
    .discount_curve = None
    .tree_built = False
    .df_times = None
    .dfs = None
```

```
.r_t = None
.dt = None
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description
sigma	float	-
a	float	-
num_time_steps	int	-
european_calc_type	FinHWEuropeanCalcType	-
None		-
"""Constructs the Hull-White rate model. The speed of mean reversion	-	-
a and volatility are passed in. The short rate process is given by	-	-
dr	-	(the
Jamshidian's approach where possible unless the useJamshidian flag is	-	-
set to false in which case it uses the trinomial Tree."""	-	-
if sigma < 0.0	-	-
raise FinError("Negative volatility not allowed.")	-	-
if a < 0.0	-	-
raise FinError("Mean reversion speed parameter should be >= 0")	-	-
.sigma	-	-
.a	-	-
.num_time_steps	-	-
.european_calc_type	-	-
.qq	-	-
.r	-	-
.tree_times	-	-
.pu	-	-
.pm	-	-
.pd	-	-
.discount_curve	-	-
.tree_built	-	-
.df_times	-	-
.dfs	-	-
.r_t	-	-
.dt	-	-

option_on_zcb

Price an option on a zero cpn bond using analytical solution of Hull-White model. User provides bond face and option strike and expiry date and maturity date.

```
option_on_zcb(t_exp: float,
t_mat: float,
```

```

strike: float,
face_amount: float,
df_times: np.ndarray,
df_values: np.ndarray,
Dict[str, float]:
    """Price an option on a zero-cpn bond using analytical solution of
    Hull-White model. User provides bond face and option strike and expiry
    date and maturity date."""
    if t_exp > t_mat:
        raise FinError("Option expiry after bond matures.")
    if t_exp < 0.0:
        raise FinError("Option expiry time negative.")
    pt_exp = _uinterpolate(t_exp, df_times, df_values, INTERP_TYPE_VALUE)
    pt_mat = _uinterpolate(t_mat, df_times, df_values, INTERP_TYPE_VALUE)
    sigma = .sigma
    a = .a
    if abs(a) < SMALL:
        a = SMALL
    sigmap = (sigma / a) * (1.0 - np.exp(-a * (t_mat - t_exp)))
    sigmap = sigmap * np.sqrt((1.0 - np.exp(-2.0 * a * t_exp)) / 2.0 / a)
    if abs(sigmap) < SMALL:
        sigmap = SMALL
    h = np.log((face_amount * pt_mat) / (strike * pt_exp)) / sigmap + sigmap / 2.0
    call_value = face_amount * pt_mat * normcdf(h) - strike * pt_exp * normcdf(h - sigmap)
    put_value = strike * pt_exp * normcdf(-h + sigmap) - face_amount * pt_mat * normcdf(-h)
    return {"call": call_value, "put": put_value}
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
t_exp	float	-
t_mat	float	-
strike	float	-
face_amount	float	-
df_times	np.ndarray	-
df_values	np.ndarray	-
Dict[str, float]		-
"""Price an option on a zero cpn bond using analytical solution of Hull-White model. User provides bond face and option strike and expiry date and maturity date."""	-	-
if t_exp > t_mat		-
raise FinError("Option expiry after bond matures.")	-	-
if t_exp < 0.0		-
raise FinError("Option expiry time negative.")	-	-
pt_exp	-	- _uinterpolate(t
pt_mat	-	- _uinterpolate(t
sigma	-	-
a	-	-
if abs(a) > SMALL		-
a	-	-
sigmap	-	- (sigm
sigmap	-	- sigmap * r
if abs(sigmap) > SMALL		-
sigmap	-	-
h	-	- np.log((face_amou
)	-	-
return "call"	call_value	-
"put"	put_value	-

european_bond_option_jamshidian

PLEASE ADD A FUNCTION DESCRIPTION

```
european_bond_option_jamshidian(t_exp: float,
strike_price: float,
face: float,
cpn_times: np.ndarray,
cpn_amounts: np.ndarray,
df_times: np.ndarray,
df_values: np.ndarray,
Dict[str, float]):
"""Valuation_of_a_European_bond_option_using_the_Jamshidian
deconstruction_of_the_bond_into_a_strip_of_zero_cpn_bonds_with_the
short_rate_that_would_make_the_bond_option_be_at_the_money_forward."""
#     print(df_times)
#     print(df_values)
num_cpns = len(cpn_times)
```

```

argtuple = (t_exp,
            cpn_times,
            cpn_amounts,
            df_times,
            df_values,
            strike_price,
            face,
            )
# Can I improve on this initial guess ?
x0 = 0.05
rstar = optimize.newton(fwd_dirty_bond_price,
                        x0=x0,
                        fprime=None,
                        args=argtuple,
                        tol=1e-10,
                        maxiter=50,
                        fprime2=None,
                        )
# Now we price a series of zero cpn bonds using this short rate
dt = 1e-6
pt_exp = _uinterpolate(t_exp, df_times, df_values, INTERP_TYPE_VALUE)
ptdelta = _uinterpolate(t_exp + dt, df_times, df_values, INTERP_TYPE_VALUE)
call_value = 0.0
put_value = 0.0
# Adjust strike to handle
for i in range(0, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description
t_exp	float	-
strike_price	float	-
face	float	-
cpn_times	np.ndarray	-
cpn_amounts	np.ndarray	-
df_times	np.ndarray	-
df_values	np.ndarray	-
Dict[str, float]		-
"""Valuation of a European bond option using the Jamshidian	-	-
deconstruction of the bond into a strip of zero cpn bonds with the	-	-
short rate that would make the bond option be at the money forward."""	-	-
num_cpns	-	-
cpn_times	-	-
cpn_amounts	-	-
df_times	-	-
df_values	-	-
strike_price	-	-
face	-	-
)	-	-
x0	-	-
x0	-	-
fprime	-	-
args	-	-
tol	-	-
maxiter	-	-
fprime2	-	-
)	-	-
dt	-	-
pt_exp	-	- _uinterpolate(t_exp
ptdelta	-	- _uinterpolate(t_exp
call_value	-	-
put_value	-	-

european_bond_option_expiry_only

PLEASE ADD A FUNCTION DESCRIPTION

```
european_bond_option_expiry_only(t_exp: float,
strike_price: float,
face_amount: float,
cpn_times: np.ndarray,
cpn_amounts: np.ndarray,
Dict[str, float]):
"""Price_a_European_option_on_a_cpn-paying_bond_using_a_tree_to
generate_short_rates_at_the_expiry_date_and_then_to_use_the_analytical
solution_of_zero_cpn_bond_prices_in_the_HW_model_to_calculate_the
```

```

corresponding_bond_price._User_provides_bond_object_and_option_details.
"""
dt = .dt
tdelta = t_exp + dt
pt_exp = _uinterpolate(t_exp, .df_times, .dfs, INTERP_TYPE_VALUE)
ptdelta = _uinterpolate(tdelta, .df_times, .dfs, INTERP_TYPE_VALUE)
_, num_nodes = .qq.shape
expiry_step = int(t_exp / dt + 0.50)
call_value = 0.0
put_value = 0.0
num_cpns = len(cpn_times)
for k in range(0, num_nodes):

```

The function arguments are described in the following table.

Argument Name	Type	Description
t_exp	float	-
strike_price	float	-
face_amount	float	-
cpn_times	np.ndarray	-
cpn_amounts	np.ndarray	-
Dict[str, float]		-
"""Price a European option on a cpn-paying bond using a tree to generate short rates at the expiry date and then to use the analytical solution of zero cpn bond prices in the HW model to calculate the corresponding bond price. User provides bond object and option details.	-	-
"""	-	-
dt	-	-
tdelta	-	-
pt_exp	-	-
ptdelta	-	-
-	-	-
num_nodes	-	-
expiry_step	-	-
call_value	-	-
put_value	-	-
num_cpns	-	-

option_on_zero_cpn_bond_tree

PLEASE ADD A FUNCTION DESCRIPTION

```

option_on_zero_cpn_bond_tree(t_exp: float,
t_mat: float,
strike_price: float,
face_amount: float,
Dict[str, float]:
"""Price an option on a zero_cpn_bond using a HW trinomial

```

```

tree._The_discount_curve_was_already_supplied_to_the_tree_build."""
if t_exp > t_mat:
    raise FinError("Option_expiry_after_bond_matures.")
if t_exp < 0.0:
    raise FinError("Option_expiry_time_negative.")
if .tree_times is None:
    raise FinError("Tree_has_not_been_constructed.")
if .tree_times[-1] < t_exp:
    raise FinError("Tree_expiry_must_be_>=option_expiry_date.")
dt = .dt
tdelta = t_exp + dt
pt_exp = _uinterpolate(t_exp, .df_times, .dfs, INTERP_TYPE_VALUE)
ptdelta = _uinterpolate(tdelta, .df_times, .dfs, INTERP_TYPE_VALUE)
pt_mat = _uinterpolate(t_mat, .df_times, .dfs, INTERP_TYPE_VALUE)
_, num_nodes = .qq.shape
expiry_step = int(t_exp / dt + 0.50)
call_value = 0.0
put_value = 0.0
for k in range(0, num_nodes):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t_exp	float	-	
t_mat	float	-	
strike_price	float	-	
face_amount	float	-	
Dict[str, float]		-	
"""Price an option on a zero cpn bond using a HW trinomial tree. The discount curve was already supplied to the tree build."""	-	-	
if t_exp > t_mat		-	
raise FinError("Option expiry after bond matures.")	-	-	
if t_exp < 0.0		-	
raise FinError("Option expiry time negative.")	-	-	
if .tree_times is None		-	
raise FinError("Tree has not been constructed.")	-	-	
if .tree_times[-1] < t_exp		-	
raise FinError("Tree expiry must be >= option expiry date.")	-	-	option expiry date
dt	-	-	.dt
tdelta	-	-	t_exp + dt
pt_exp	-	-	_uinterpolate(t_exp, .df_times, .dfs, INTERP_TYPE_VALUE)
ptdelta	-	-	_uinterpolate(tdelta, .df_times, .dfs, INTERP_TYPE_VALUE)
pt_mat	-	-	_uinterpolate(t_mat, .df_times, .dfs, INTERP_TYPE_VALUE)
-	-	-	
num_nodes	-	-	.qq.shape
expiry_step	-	-	int(t_exp / dt + 0.50)
call_value	-	-	0.0
put_value	-	-	0.0

bermudan_swaption

Swaption that can be exercised on specific dates over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time.

```
bermudan_swaption(t_exp: float,
                  t_mat: float,
                  strike: float,
                  face: float,
                  cpn_times: np.ndarray,
                  cpn_flows: np.ndarray,
                  exercise_type: Any,
                  Dict[str, float]):
    """Swaption that can be exercised on specific dates over the exercise
    period. Due to non-analytical bond price we need to extend tree out to
    bond maturity and take into account cash flows through time."""
    exercise_type_int = option_exercise_types_to_int(exercise_type)
    t_mat = cpn_times[-1]
    if t_exp > t_mat:
        raise FinError("Option_expiry_after_bond_matures.")
    if t_exp < 0.0:
        raise FinError("Option_expiry_time_negative.")
    pay_value, rec_value = bermudan_swaption_tree_fast(t_exp,
                                                       t_mat,
                                                       strike,
                                                       face,
                                                       cpn_times,
                                                       cpn_flows,
                                                       exercise_type_int,
                                                       .df_times,
                                                       .dfs,
                                                       .tree_times,
                                                       .qq,
                                                       .pu,
                                                       .pm,
                                                       .pd,
                                                       .r_t,
                                                       .dt,
                                                       .a,
                                                       )
    return {"pay": pay_value, "rec": rec_value}
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t_exp	float	-	
t_mat	float	-	
strike	float	-	
face	float	-	
cpn_times	np.ndarray	-	
cpn_flows	np.ndarray	-	
exercise_type	Any	-	
Dict[str, float]		-	
"""Swaption that can be exercised on specific dates over the exercise period. Due to non-analytical bond price we need to extend tree out to bond maturity and take into account cash flows through time."""	-	-	
exercise_type_int	-	-	option_exercise_types
t_mat	-	-	cpn_tin
if t_exp < t_mat		-	
raise FinError("Option expiry after bond matures.")	-	-	
if t_exp > 0.0		-	
raise FinError("Option expiry time negative.")	-	-	
pay_value	-	-	
t_mat	-	-	
strike	-	-	
face	-	-	
cpn_times	-	-	
cpn_flows	-	-	
exercise_type_int	-	-	
.df_times	-	-	
.dfs	-	-	
.tree_times	-	-	
.qq	-	-	
.pu	-	-	
.pm	-	-	
.pd	-	-	
.r_t	-	-	
.dt	-	-	
.a	-	-	
)	-	-	
return "pay"	pay_value	-	
"rec"	rec_value	-	

bond_option

Value a bond option that can have European or American exercise. This is done using a trinomial tree that we extend out to bond maturity. For European bond options, Jamshidian's model is faster and is used instead i.e. not this function.

```

bond_option(t_exp: float,
strike_price: float,
face_amount: float,
cpn_times: np.ndarray,
cpn_flows: np.ndarray,
exercise_type: Any,
    Dict[str, float]):
    """Value a bond option that can have European or American exercise.
    This is done using a trinomial tree that we extend out to bond
    maturity. For European bond options, Jamshidian's model is
    faster and is used instead i.e. not this function."""
    exercise_type_int = option_exercise_types_to_int(exercise_type)
    if exercise_type_int == 1:
        if .european_calc_type == FinHWEuropeanCalcType.JAMSHIDIAN:
            v = .european_bond_option_jamshidian(t_exp,
                strike_price,
                face_amount,
                cpn_times,
                cpn_flows,
                .df_times,
                .dfs,
            )
            call_value = v["call"]
            put_value = v["put"]
        elif .european_calc_type == FinHWEuropeanCalcType.EXPIRY_ONLY:
            v = .european_bond_option_expiry_only(t_exp, strike_price, face_amount,
                cpn_times, cpn_flows
            )
            call_value = v["call"]
            put_value = v["put"]
        elif .european_calc_type == FinHWEuropeanCalcType.EXPIRY_TREE:
            call_value, put_value = american_bond_option_tree_fast(t_exp,
                strike_price,
                face_amount,
                cpn_times,
                cpn_flows,
                exercise_type_int,
                .sigma,
                .a,
                .qq,
                .pu,
                .pm,
                .pd,
                .r_t,
                .dt,
                .tree_times,
                .df_times,
                .dfs,
            )
        else:
            raise FinError("Unknown_HW_model_implementation_choice.")
    else:
        call_value, put_value = american_bond_option_tree_fast(t_exp,
            strike_price,
            face_amount,
            cpn_times,
            cpn_flows,

```



```
        exercise_type_int,  
        .sigma,  
        .a,  
        .qq,  
        .pu,  
        .pm,  
        .pd,  
        .r_t,  
        .dt,  
        .tree_times,  
        .df_times,  
        .dfs,  
    )  
    return {"call": call_value, "put": put_value}  
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t_exp	float	-	
strike_price	float	-	
face_amount	float	-	
cpn_times	np.ndarray	-	
cpn_flows	np.ndarray	-	
exercise_type	Any	-	
Dict[str, float]		-	
"""Value a bond option that can have European or American exercise. This is done using a trinomial tree that we extend out to bond maturity. For European bond options Jamshidian's model is faster and is used instead i.e. not this function."""	-	-	
exercise_type_int	-	-	option_exercise_type
if exercise_type_int	-	-	:
if .european_calc_type	-	-	= FinHWEuropeanCa
strike_price	-	-	
face_amount	-	-	
cpn_times	-	-	
cpn_flows	-	-	
.df_times	-	-	
.dfs	-	-	
)	-	-	
call_value	-	-	v['
put_value	-	-	v['
elif .european_calc_type	-	-	= FinHWEuropeanCa
)	-	-	
call_value	-	-	v['
put_value	-	-	v['
elif .european_calc_type	-	-	= FinHWEuropeanCa
call_value	-	-	
strike_price	-	-	
face_amount	-	-	
cpn_times	-	-	
cpn_flows	-	-	
exercise_type_int	-	-	
.sigma	-	-	
.a	-	-	
.qq	-	-	
.pu	-	-	
.pm	-	-	
.pd	-	-	
.r_t	-	-	
.dt	-	-	
.tree_times	-	-	
.df_times	-	-	
.dfs	-	-	
)	-	-	
else		-	
raise FinError("Unknown HW model implementation choice.")	-	-	
else		-	
call_value			

callable_puttable_bond_tree

PLEASE ADD A FUNCTION DESCRIPTION

```
callable_puttable_bond_tree(cpn_times: np.ndarray,
cpn_flows: np.ndarray,
call_times: np.ndarray,
call_prices: np.ndarray,
put_times: np.ndarray,
put_prices: np.ndarray,
face_amount: float,
Dict[str, float]):
    """Value an option on a bond with cpns that can have European or
    American exercise. Some minor issues to do with handling cpns on
    the option expiry date need to be solved. Also this function should be
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cpn_times	np.ndarray	-	-
cpn_flows	np.ndarray	-	-
call_times	np.ndarray	-	-
call_prices	np.ndarray	-	-
put_times	np.ndarray	-	-
put_prices	np.ndarray	-	-
face_amount	float	-	-
Dict[str, float]		-	-
"""Value an option on a bond with cpns that can have European or	-	-	-
American exercise. Some minor issues to do with handling cpns on	-	-	-
the option expiry date need to be solved. Also this function should be	-	-	-

df_tree

PLEASE ADD A FUNCTION DESCRIPTION

```
df_tree(t_mat: float) -> Union[float, tuple]:
    """Discount_factor_as_seen_from_now_to_time_t_mat_as_long_as_the_time
    is_on_the_tree_grid."""
    if t_mat == 0.0:
        return 1.0
    _, num_nodes = .qq.shape
    fn1 = t_mat / .dt
    fn2 = float(int(t_mat / .dt))
    if abs(fn1 - fn2) > 1e-6:
        raise FinError("Time_not_on_tree_time_grid")
    time_step = int(t_mat / .dt) + 1
    p = 0.0
    for i in range(0, num_nodes):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t_mat	float or float)-; Union[fl	-	-
"""Discount factor as seen from now to time t_mat as long as the time is on the tree grid."""	-	-	-
if t_mat	-	-	-
return 1.0	-	-	-
-	-	-	-
num_nodes	-	-	.q
fn1	-	-	t.L
fn2	-	-	float(in
if abs(fn1 - fn2) < 1e-6	-	-	-
raise FinError("Time not on tree time grid")	-	-	-
time_step	-	-	int(t.n
p	-	-	-

build_tree

PLEASE ADD A FUNCTION DESCRIPTION

```

build_tree(tree_mat: float,
df_times: np.ndarray,
df_values: np.ndarray,
None:
"""Build_the_trinomial_tree."""
if isinstance(df_times, np.ndarray) is False:
    raise FinError("DF_TIMES_must_be_a_numpy_vector")
if isinstance(df_values, np.ndarray) is False:
    raise FinError("DF_VALUES_must_be_a_numpy_vector")
# I wish to add on an additional time to the tree so that the second
# last time corresponds to a maturity tree_mat. For this reason I scale
# up the maturity date of the tree as follows
tree_maturity = tree_mat * (.num_time_steps + 1) / .num_time_steps
# The vector of times goes out to this maturity
tree_times = np.linspace(0.0, tree_maturity, .num_time_steps + 2)
.tree_times = tree_times
df_tree = np.zeros(shape=.num_time_steps + 2)
df_tree[0] = 1.0
for i in range(1, .num_time_steps + 2):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
tree_mat	float	-	-
df_times	np.ndarray	-	-
df_values	np.ndarray	-	-
None	-	-	-
"""Build the trinomial tree."""	-	-	-
if isinstance(df_times, np.ndarray) is False	-	-	-
raise FinError("DF TIMES must be a numpy vector")	-	-	-
if isinstance(df_values, np.ndarray) is False	-	-	-
raise FinError("DF VALUES must be a numpy vector")	-	-	-
tree_maturity	-	-	tree_mat * (.num_time_steps + 1) / .
tree_times	-	-	np.linspace(0.0, tree_maturity, .num
.tree_times	-	-	tree_times
df_tree	-	-	np.zeros(shape=.num_time_s
df_tree[0]	-	-	1.0

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str	-	-	-

option_exercise_types_to_int

PLEASE ADD A FUNCTION DESCRIPTION

```
option_exercise_types_to_int(option_exercise_type: Any) -> int:
"""
Convert_option_exercise_type_enum_to_integer.
Args:
_____option_exercise_type_(ExerciseTypes):_The_exercise_type.
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
option_exercise_type	Any)->int:	-	-
"""	-	-	-
Convert option exercise type enum to integer.	-	-	-
Args	-	-	-

p_fast

Forward discount factor as seen at some time t which may be in the future for payment at time T where r_t is

the delta-period short rate seen at time t and pt is the discount factor to time t , ptd is the one period discount factor to time $t+dt$ and pT is the discount factor from now until the payment of the 1 dollar of the discount factor.

```
p_fast(t, t_mat, r_t, delta, pt, ptd, p_mat, _sigma, _a):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
t_mat	-	-	-
r_t	-	-	-
delta	-	-	-
pt	-	-	-
ptd	-	-	-
p_mat	-	-	-
_sigma	-	-	-
_a	-	-	-

build_tree_fast

Fast tree construction using Numba.

```
build_tree_fast(a, sigma, tree_times, num_time_steps, discount_factors):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	-	-	-
sigma	-	-	-
tree_times	-	-	-
num_time_steps	-	-	-
discount_factors	-	-	-

american_bond_option_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
american_bond_option_tree_fast(
t_exp,
strike_price,
face_amount,
cpn_times,
cpn_amounts,
exercise_type_int,
_sigma,
_a,
_qq,
```

```

_pu,
_pm,
_pd,
_r_t,
_dt,
_tree_times,
_df_times,
_df_values,
"""Value an option on a bond with cpns that can have European or
American exercise. Some minor issues to do with handling cpns on
the option expiry date need to be solved."""
debug = False
if debug:
    print("Entering_AmerBondOption")
    print("Coupon_Times", cpn_times)
    print("Coupon_Amounts", cpn_amounts)
num_time_steps, num_nodes = _qq.shape
dt = _dt
j_max = ceil(0.1835 / (_a * dt))
expiry_step = int(t_exp / dt + 0.50)
# Want to add cpns before expiry to the grid so that we can value
# their impact on the decision to exercise the option early
tree_flows = np.zeros(num_time_steps)
num_cpns = len(cpn_times)
# Flows that fall on the expiry date included. The tree only goes out to
# the expiry date so cpns after this date do not go onto the tree.
for i in range(0, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	-	-	-
strike_price	-	-	-
face_amount	-	-	-
cpn_times	-	-	-
cpn_amounts	-	-	-
exercise_type_int	-	-	-
_sigma	-	-	-
_a	-	-	-
_qq	-	-	-
_pu	-	-	-
_pm	-	-	-
_pd	-	-	-
_r_t	-	-	-
_dt	-	-	-
_tree_times	-	-	-
_df_times	-	-	-
_df_values	-	-	-
"""Value an option on a bond with cpns that can have European or	-	-	-
American exercise. Some minor issues to do with handling cpns on	-	-	-
the option expiry date need to be solved."""	-	-	-
debug	-	-	False
if debug	-	-	-
print("Entering AmerBondOption")	-	-	-
print("Coupon Times", cpn_times)	-	-	-
print("Coupon Amounts", cpn_amounts)	-	-	-
num_time_steps	-	-	-
num_nodes	-	-	_qq.shape
dt	-	-	_dt
j_max	-	-	ceil(0.1835 / (_a * dt))
expiry_step	-	-	int(t_exp / dt + 0.50)
tree_flows	-	-	np.zeros(num_time_steps)
num_cpns	-	-	len(cpn_times)

bermudan_swaption_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
bermudan_swaption_tree_fast(
t_exp,
t_mat,
strike_price,
face_amount,
cpn_times,
cpn_flows,
exercise_type_int,
_df_times,
```



```

_df_values,
_tree_times,
_qq,
_pu,
_pm,
_pd,
_r_t,
_dt,
_a,
"""Option to enter into a swap that can be exercised on cpn payment
dates after the start of the exercise period. Due to multiple exercise
times we need to extend tree out to bond maturity and take into account
cash flows through time."""
num_time_steps, num_nodes = _qq.shape
j_max = ceil(0.1835 / (_a * _dt))
expiry_step = int(t_exp / _dt + 0.50)
maturity_step = int(t_mat / _dt + 0.50)
fixed_leg_flows = np.zeros(num_time_steps)
float_leg_values = np.zeros(num_time_steps)
num_cpns = len(cpn_times)
# Tree flows go all the way out to the bond maturity date
for i in range(0, num_cpns):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t_exp	-	-	-
t_mat	-	-	-
strike_price	-	-	-
face_amount	-	-	-
cpn_times	-	-	-
cpn_flows	-	-	-
exercise_type_int	-	-	-
_df_times	-	-	-
_df_values	-	-	-
_tree_times	-	-	-
_qq	-	-	-
_pu	-	-	-
_pm	-	-	-
_pd	-	-	-
_r_t	-	-	-
_dt	-	-	-
_a	-	-	-
"""Option to enter into a swap that can be exercised on cpn payment	-	-	-
dates after the start of the exercise period. Due to multiple exercise	-	-	-
times we need to extend tree out to bond maturity and take into account	-	-	-
cash flows through time."""	-	-	-
num_time_steps	-	-	-
num_nodes	-	-	_qq.shape
j_max	-	-	ceil(0.1835 / (_a * _dt))
expiry_step	-	-	int(t_exp / _dt + 0.50)
maturity_step	-	-	int(t_mat / _dt + 0.50)
fixed_leg_flows	-	-	np.zeros(num_time_steps)
float_leg_values	-	-	np.zeros(num_time_steps)
num_cpns	-	-	len(cpn_times)

callable_puttable_bond_tree_fast

PLEASE ADD A FUNCTION DESCRIPTION

```
callable_puttable_bond_tree_fast(
    cpn_times,
    cpn_flows,
    call_times,
    call_prices,
    put_times,
    put_prices,
    face,
    _sigma,
    _a,
    _qq,  # IS SIGMA USED ?
    _pu,
    _pm,
```

```

_pd,
_r_t,
_dt,
_tree_times,
_df_times,
_df_values,
"""Value an option on a bond with cpns that can have European or
American exercise. Some minor issues to do with handling cpns on
the option expiry date need to be solved."""
# print("Coupon Times:", cpn_times)
# print("Coupon Flows:", cpn_flows)
# print("DF Times:", _df_times)
# print("DF Values:", _df_values)
if np.any(cpn_times < 0.0):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cpn_times	-	-	-
cpn_flows	-	-	-
call_times	-	-	-
call_prices	-	-	-
put_times	-	-	-
put_prices	-	-	-
face	-	-	-
_sigma	-	-	-
_a	-	-	-
-qq	-	IS SIGMA USED ?	-
_pu	-	-	-
_pm	-	-	-
_pd	-	-	-
_r_t	-	-	-
_dt	-	-	-
_tree_times	-	-	-
_df_times	-	-	-
_df_values	-	-	-
"""Value an option on a bond with cpns that can have European or	-	-	-
American exercise. Some minor issues to do with handling cpns on	-	-	-
the option expiry date need to be solved."""	-	-	-

fwd_dirty_bond_price

PLEASE ADD A FUNCTION DESCRIPTION

```

fwd_dirty_bond_price(r_t: float, *args: Any) -> float:
"""
Price a cpn bearing bond on the option expiry date and return
the difference from a strike price. This is used in a root search to

```

```

find_the_future_expiry_time_short_rate_that_makes_the_bond_price_equal
to_the_option_strike_price._It_is_a_key_step_in_the_Jamshidian_bond
decomposition_approach._The_strike_is_a_clean_price.
Args:
r_t(float):_Short_rate_at_expiry.

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r_t	float	-	-
*args	Any)-float:	-	-
"""	-	-	-
Price a cpn bearing bond on the option expiry date and return	-	-	-
the difference from a strike price. This is used in a root search to	-	-	-
find the future expiry time short rate that makes the bond price equal	-	-	-
to the option strike price. It is a key step in the Jamshidian bond	-	-	-
decomposition approach. The strike is a clean price.	-	-	-
Args	-	-	-

9.26 Imm_mc

Enumerated Type: ModelLMMMModelTypes

This enumerated type has the following values:

- LMM.ONE_FACTOR
- LMM.HW_M_FACTOR
- LMM.FULL_N_FACTOR
- num_paths
- num_times
- num_fwds
- print("Path:
- print(")
- print(")
- cache
- fastmath
- num_periods
- iflen(taus)!
- iflen(zetas)!
- iflen(rho)!
- iflen(rho[0])!
- ifa
- p
- p[0]
- p[ix]
- wts
- pv01ab
- pv01ab+
- sab
- wts[i]
- swaption_var
- wti
- wtj
- fi
- fj
- intsigmai
- intsigmai+
- term
- swaption_var+
- tau_a
- tau_a+
- tau_b
- tau_b+
- swaption_vol
- cache
- fastmath

- num_paths
- num_fwds
- ifa_i
- fwd_swap_rate_mean
- fwd_swap_rate_var
- numeraire
- numeraire*
- pv01
- df
- f
- tau
- df
- pv01
- fwd_swap_rate
- fwd_swap_rate_mean+
- fwd_swap_rate_var+
- taua
- taua+
- fwd_swap_rate_mean/
- fwd_swap_rate_var
- fwd_swap_rate_vol
- fwd_swap_rate_vol/
- cache
- fastmath
- size
- fwd_corr
- sumfwdi
- sumfwdj
- sumfwdifwdi
- sumfwdifwdj
- sumfwdjfwjdj
- dfwdi
- dfwdj
- sumfwdi+
- sumfwdj+
- sumfwdifwdi+
- sumfwdifwdj+
- sumfwdjfwjdj+
- avgfwdi
- avgfwdj
- avgfwdifwdi
- avgfwdifwdj
- avgfwdjfwjdj
- covii
- covjj
- covij

- corr
- fwd_corr[i_fwd-i_time][j_fwd-i_time]
- fwd_corr[j_fwd-i_time][i_fwd-i_time]
- fwd_corr[i_fwd-i_time][j_fwd-i_time]
- fwd_corr[j_fwd-i_time][i_fwd-i_time]
- cache
- fastmath
- caplet
- disc_fwd
- ifk_i
- disc_fwd[0]
- disc_fwd[i]
- t_exp
- k
- t_exp+
- vol
- f
- d1
- d2
- caplet[i]
- @njit(float64[:,:](float64[:,:],int64),cache
- lent
- result
- result[i-n-1][j-n-1]
- @njit(float64[:,:](float64[:,:]),cache
- chol
- cache
- fastmath
- num_paths
- half_num_paths
- fwd
- fwd_b
- disc_fwd
- disc_fwd[0]
- disc_fwd[ix]
- corr
- factors
- matrix
- chol
- g_matrix
- g
- g_matrix[i_path,j,k]
- g_matrix[i_path+half_num_paths,j,k]
- avgg
- stdg
- fwd[i_path,0,i_fwd]

- dt
- sqrt_dt
- zi
- mu_a
- rho
- fk
- zk
- tk
- mu_a+
- w
- f
- w
- avgg+
- stdg+
- fwd_b[i]
- mu_b
- rho
- fk
- zk
- tk
- mu_b+
- mu_avg
- x
- fwd[i_path,j,i]
- cache
- fastmath
- iflen(gammas)!
- iflen(fwd0)!
- iflen(taus)!
- num_paths
- half_num_paths
- fwd
- fwd_b
- num_times
- ifuse_sobol
- num_dimensions
- rands
- g_matrix
- u
- g
- g_matrix[i_path,j]
- g_matrix[i_path+half_num_paths,j]
- elifuse_sobol
- g_matrix
- g
- g_matrix[i_path,j]

- g_matrix[i_path+half_num_paths,j]
- fwd[i_path,0,i_fwd]
- dtj
- sqrt_dtj
- w
- zkj
- mu_a
- fi
- zij
- ti
- mu_a+
- x
- fwd_b[k]
- mu_b
- fi
- zij
- ti
- mu_b+
- mu_c
- x
- fwd[i_path,j+1,k]
- cache
- fastmath
- iflen(lambdas)!
- iflen(lambdas[0])!
- num_paths
- half_num_paths
- fwd
- fwd_b
- num_times
- ifuse_sobol
- num_dimensions
- rands
- g_matrix
- col
- u
- g
- g_matrix[i_path,j,q]
- g_matrix[i_path+half_num_paths,j,q]
- elifuse_sobol
- g_matrix
- g
- g_matrix[i_path,j,q]
- g_matrix[i_path+half_num_paths,j,q]
- fwd[i_path,0,i_fwd]
- dtj

- sqrt_dtj
- mu_a
- fi
- ti
- zz
- zij
- zkj
- zz+
- mu_a+
- ito_term
- ito_term+
- random_term
- wq
- random_term+
- random_term*
- x
- fwd_b[k]
- mu_b
- fi
- ti
- zz
- zij
- zkj
- zz+
- mu_b+
- mu_c
- x
- fwd[i_path,j+1,k]
- cache
- fastmath
- max_paths
- max_fwds
- df
- cap_flr_lets
- cap_flr_let_values
- numeraire
- period_roll
- libor
- cap_flr_lets[0]
- libor
- ifj
- ifis_cap
- cap_flr_lets[j]
- cap_flr_lets[j]
- numeraire[0]
- ifis_cap

- cap_flr_lets[j]
- elifis_cap
- cap_flr_lets[j]
- period_roll
- numeraire[j]
- denom
- cap_flr_let_values[i_fwd]+
- cap_flr_let_values[i_fwd]/
- cache
- fastmath
- max_paths
- max_fwds
- df
- numeraire
- sum_fixed
- sun_float
- fixed_flows
- float_flows
- df[0]
- df[ix]
- period_roll
- libor
- float_flows[0]
- fixed_flows[0]
- numeraire[0]
- libor
- ifj
- fixed_flows[j]
- float_flows[j]
- fixed_flows[j]
- float_flows[j]
- period_roll
- numeraire[j]
- sun_float+
- sum_fixed+
- sun_float/
- sum_fixed/
- v
- pv01
- swap_rate
- cache
- fastmath
- max_paths
- max_fwds
- ifa_i
- df

- pv01
- df[0]
- df[ix]
- sum_pay_rec_swaption
- numeraire
- numeraire*
- pv01
- df
- f
- tau
- df
- pv01
- fwd_swap_rate
- ifis_payer
- pay_rec_swaption
- elifis_payer
- pay_rec_swaption
- sum_pay_rec_swaption+
- pay_rec_price
- cache
- fastmath
- max_paths
- max_fwds
- df
- numeraire
- rachet_caplets
- rachet_caplet_values
- df[0]
- df[ix]
- period_roll
- libor
- rachet_caplets[0]
- prev_ibor
- k
- libor
- ifj
- rachet_caplets[j]
- numeraire[0]
- rachet_caplets[j]
- period_roll
- numeraire[j]
- rachet_caplet_values[i_fwd]+
- rachet_caplet_values[i_fwd]/
- cache
- fastmath
- max_paths

- max_fwds
- df
- numeraire
- flexi_caplets
- flexi_caplet_values
- df[0]
- df[ix]
- period_roll
- libor
- flexi_caplets[0]
- num_caplets_left
- libor
- ifj
- flexi_caplets[j]
- num_caplets_left-
- numeraire[0]
- flexi_caplets[j]
- num_caplets_left-
- period_roll
- numeraire[j]
- flexi_caplet_values[i_fwd]+
- flexi_caplet_values[i_fwd]/
- flexi_cap_value
- flexi_cap_value+
- cache
- fastmath
- max_paths
- max_fwds
- df
- numeraire
- sticky_caplets
- sticky_caplet_values
- df[0]
- df[ix]
- period_roll
- libor
- sticky_caplets[0]
- k
- prev_ibor
- k
- libor
- ifj
- sticky_caplets[j]
- numeraire[0]
- sticky_caplets[j]
- period_roll

- `numeraire[j]`
- `sticky_caplet_values[i_fwd]+`
- `sticky_caplet_values[i_fwd]/`

lmm_print_forwards

Helper function to display the simulated Ibor rates.

```
lmm_print_forwards(fwds):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwds	-	-	-

lmm_swaption_vol_approx

Implements Rebonato's approximation for the swap rate volatility to be used when pricing a swaption that expires in period a for a swap maturing at the end of period b taking into account the forward volatility term structure (zetas) and the forward-forward correlation matrix ρ .

```
lmm_swaption_vol_approx(a, b, fwd0, taus, zetas, rho):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	-	-	-
b	-	-	-
fwd0	-	-	-
taus	-	-	-
zetas	-	-	-
rho	-	-	-

lmm_sim_swaption_vol

Calculates the swap rate volatility using the forwards generated in the simulation to see how it compares to Rebonatto estimate.

```
lmm_sim_swaption_vol(a, b, fwd0, fwds, taus):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	-	-	-
b	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-

lmm_fwd_fwd_correlation

Extract forward forward correlation matrix at some future time index from the simulated forward rates and return the matrix.

```
lmm_fwd_fwd_correlation(num_fwds, num_paths, i_time, fwds):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_fwds	-	-	-
num_paths	-	-	-
i_time	-	-	-
fwds	-	-	-

lmm_price_caps_black

Price a strip of capfloorlets using Black's model using the time grid of the LMM model. The prices can be compared with the LMM model prices.

```
lmm_price_caps_black(fwd0, vol_caplet, p, k, taus):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
fwd0	-	-	-
vol_caplet	-	-	-
p	-	-	-
k	-	-	-
taus	-	-	-

sub_matrix

Returns a submatrix of correlation matrix at later time step in the LMM simulation which is then used to generate correlated Gaussian RVs.

```
sub_matrix(t, n):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t	-	-	-
n	-	-	-

cholesky_np

Numba-compliant wrapper around Numpy cholesky function.

```
cholesky_np(rho):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
rho	-	-	-

lmm_simulate_fwds_nf

Full N-Factor Arbitrage-free simulation of forward Ibor discount in the spot measure given an initial forward curve, volatility term structure and full rank correlation structure. Cholesky decomposition is used to extract the factor weights. The number of forwards at time 0 is given. The 3D matrix of forward rates by path, time and forward point is returned. WARNING: NEED TO CHECK THAT CORRECT VOLATILITY IS BEING USED (OFF BY ONE BUG NEEDS TO BE RULED OUT)

```
lmm_simulate_fwds_nf(num_fwds, num_paths, fwd0, zetas, correl, taus, seed):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_fwds	-	-	-
num_paths	-	-	-
fwd0	-	-	-
zetas	-	-	-
correl	-	-	-
taus	-	-	-
seed	-	-	-

lmm_simulate_fwds_1f

PLEASE ADD A FUNCTION DESCRIPTION

```
lmm_simulate_fwds_1f(
num_fwds, num_paths, numeraire_index, fwd0, gammas, taus, use_sobol, seed
"""One_factor_Arbitrage-free_simulation_of_forward_Ibor_discount_in_the
spot_measure_following_Hull_Page_768. Given_an_initial_forward_curve,
volatility_term_structure. The_3D_matrix_of_forward_rates_by_path, time
and_forward_point_is_returned. This_function_is_kept_mainly_for_its
simplicity_and_speed.
NB: The_Gamma_volatility_has_an_initial_entry_of_zero. This_differs_from
Hull's_indexing_by_one_and_so_is_why_I_do_not_subtract_1_from_the_index_as
Hull_does_in_his_equation_32.14.
The_Number_of_Forwards_is_the_number_of_points_on_the_initial_curve_to_the
trade_maturity_date.
But_be_careful: a_cap_that_matures_in_10_years_with_quarterly_caplets_has
40_forwards_BUT_the_last_forward_to_reset_occurs_at_9.75_years. You_should
not_simulate_beyond_this_time. If_you_give_the_model_10_years_as_in_the
Hull_examples, you_need_to_simulate_41_(or_in_this_case_11) forwards_as_the
final_cap_or_ratchet_has_its_reset_in_10_years."""
```



```

if len(gammas) != num_fwds:
    raise FinError("Gamma_vector_does_not_have_right_number_of_forwards")
if len(fwd0) != num_fwds:
    raise FinError("The_length_of_fwd0_is_not_equal_to_num_fwds")
if len(taus) != num_fwds:
    raise FinError("The_length_of_Taus_is_not_equal_to_num_fwds")
np.random.seed(seed)
# Even number of paths for antithetics
num_paths = 2 * int(num_paths / 2)
half_num_paths = int(num_paths / 2)
fwd = np.empty((num_paths, num_fwds, num_fwds))
fwd_b = np.zeros(num_fwds)
num_times = num_fwds
if use_sobol == 1:
    num_dimensions = num_times
    rand = get_uniform_sobol(half_num_paths, num_dimensions)
    g_matrix = np.empty((num_paths, num_times))
    for i_path in range(0, half_num_paths):

```

The function arguments are described in the following table.

Argument Name	Type
num_fwds	-
num_paths	-
numeraire_index	-
fwd0	-
gammas	-
taus	-
use_sobol	-
seed	-
"""One factor Arbitrage-free simulation of forward Ibor discount in the spot measure following Hull Page 768. Given an initial forward curve volatility term structure. The 3D matrix of forward rates by path	-
time	-
and forward point is returned. This function is kept mainly for its simplicity and speed.	-
NB	TheGamma volatility has an initial entry of zero.
Hull's indexing by one and so is why I do not subtract 1 from the index as Hull does in his equation 32.14.	-
The Number of Forwards is the number of points on the initial curve to the trade maturity date.	-
But be careful	acapt that matures in 10 years with quarterly
40 forwards BUT the last forward to reset occurs at 9.75 years. You should not simulate beyond this time. If you give the model 10 years as in the Hull examples	-
you need to simulate 41 (or in this case 11) forwards as the final cap or ratchet has its reset in 10 years."""	-
if len(gammas) !	-
raise FinError("Gamma vector does not have right number of forwards")	-
if len(fwd0) !	-
raise FinError("The length of fwd0 is not equal to num_fwds")	-
if len(taus) !	-
raise FinError("The length of Taus is not equal to num_fwds")	-
np.random.seed(seed)	-
num_paths	-
half_num_paths	-
fwd	-
fwd_b	-
num_times	-
if use_sobol	-
num_dimensions	-
rands	-
g_matrix	-

lmm_simulate_fwds_mf*PLEASE ADD A FUNCTION DESCRIPTION*

```

lmm_simulate_fwds_mf(
    num_fwds,
    num_factors,
    num_paths,
    numeraire_index,
    fwd0,
    lambdas,
    taus,
    use_sobol,
    seed,
    """Multi-Factor Arbitrage-free simulation of forward Ibor discount in the
    spot measure following Hull Page 768. Given an initial forward curve,
    volatility factor term structure. The 3D matrix of forward rates by path,
    time and forward point is returned."""
    np.random.seed(seed)
    if len(lambdas) != num_factors:
        raise FinError("Lambda does not have the right number of factors")
    if len(lambdas[0]) != num_fwds:
        raise FinError("Lambda does not have the right number of forwards")
    # Even number of paths for antithetics
    num_paths = 2 * int(num_paths / 2)
    half_num_paths = int(num_paths / 2)
    fwd = np.empty((num_paths, num_fwds, num_fwds))
    fwd_b = np.zeros(num_fwds)
    num_times = num_fwds
    if use_sobol == 1:
        num_dimensions = num_times * num_factors
        rands = get_uniform_sobol(half_num_paths, num_dimensions)
        g_matrix = np.empty((num_paths, num_times, num_factors))
        for i_path in range(0, half_num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_fwds	-	-	-
num_factors	-	-	-
num_paths	-	-	-
numeraire_index	-	-	-
fwd0	-	-	-
lambdas	-	-	-
taus	-	-	-
use_sobol	-	-	-
seed	-	-	-
"""Multi-Factor Arbitrage-free simulation of forward Ibor discount in the spot measure following Hull Page 768. Given an initial forward curve volatility factor term structure. The 3D matrix of forward rates by path time and forward point is returned."""			
np.random.seed(seed)	-	-	-
if len(lambdas) != num_factors:	-	-	num_factors
raise FinError("Lambda does not have the right number of factors")	-	-	-
if len(lambdas[0]) != num_paths:	-	-	num_paths
raise FinError("Lambda does not have the right number of forwards")	-	-	-
num_paths	-	-	2 * int(num_paths)
half_num_paths	-	-	int(num_paths / 2)
fwd	-	-	np.empty((num_paths, num_paths, num_paths))
fwd_b	-	-	np.zeros((num_paths, num_paths, num_paths))
num_times	-	-	num_paths
if use_sobol	-	-	-
num_dimensions	-	-	num_paths
rands	-	-	get_uniform_sobol(half_num_paths)
g_matrix	-	-	np.empty((num_paths, num_paths, num_paths))

lmm_cap_flr_pricer

Function to price a strip of cap or floorlets in accordance with the simulated forward curve dynamics.

```
lmm_cap_flr_pricer(num_fwds, num_paths, k, fwd0, fwds, taus, is_cap):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_fwds	-	-	-
num_paths	-	-	-
k	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-
is_cap	-	-	-

lmm_swap_pricer

Function to reprice a basic swap using the simulated forward Ibors.

```
lmm_swap_pricer(cpn, num_periods, num_paths, fwd0, fwds, taus):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
cpn	-	-	-
num_periods	-	-	-
num_paths	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-

lmm_swaption_pricer

Function to price a European swaption using the simulated forward discount.

```
lmm_swaption_pricer(strike, a, b, num_paths, fwd0, fwds, taus, is_payer):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
strike	-	-	-
a	-	-	-
b	-	-	-
num_paths	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-
is_payer	-	-	-

lmm_ratchet_caplet_pricer

Price a ratchet using the simulated Ibor rates.

```
lmm_ratchet_caplet_pricer(spd, num_periods, num_paths, fwd0, fwds, taus):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
spd	-	-	-
num_periods	-	-	-
num_paths	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-

lmm_flexi_cap_pricer

Price a flexicap using the simulated Ibor rates.

```
lmm_flexi_cap_pricer(max_caplets, k, num_periods, num_paths, fwd0, fwds, taus):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
max_caplets	-	-	-
k	-	-	-
num_periods	-	-	-
num_paths	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-

lmm_sticky_caplet_pricer

Price a sticky cap using the simulated Ibor rates.

```
lmm_sticky_caplet_pricer(spread, num_periods, num_paths, fwd0, fwds, taus):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
spread	-	-	-
num_periods	-	-	-
num_paths	-	-	-
fwd0	-	-	-
fwds	-	-	-
taus	-	-	-

9.27 loss_dbn_builder

indep_loss_dbn_hetero_adj_binomial

PLEASE ADD A FUNCTION DESCRIPTION

```
indep_loss_dbn_hetero_adj_binomial(
    num_credits: int,
    cond_probs: Sequence[float],
    loss_ratio: Sequence[float]
    np.ndarray:
    # Algorithm due to D. O'Kane.
    num_losses = num_credits + 1
    indep_dbn = np.zeros(num_losses)
    p = 0.0
    for i_credit in range(0, num_credits):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_credits	int	-	-
cond_probs	Sequence[float]	-	-
loss_ratio	Sequence[float]	-	-
np.ndarray		-	-
num_losses	-	-	num_credits + 1
indep_dbn	-	-	np.zeros(num_losses)
p	-	-	0.0

portfolio_gcd

PLEASE ADD A FUNCTION DESCRIPTION

```
portfolio_gcd(
    actual_losses: Sequence[float]
    float:
    num_credits = len(actual_losses)
    scaling = 1000000
    temp = (int)(actual_losses[0] * scaling)
    for i_credit in range(1, num_credits):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
actual_losses	Sequence[float]	-	-
float		-	-
num_credits	-	-	len(actual_losses)
scaling	-	-	1000000
temp	-	-	(int)(actual_losses[0] * scaling)

indep_loss_dbn_recursion_gcd*PLEASE ADD A FUNCTION DESCRIPTION*

```

indep_loss_dbn_recursion_gcd(
num_credits: int,
cond_default_probs: Sequence[float],
loss_units: Sequence[float]
    np.ndarray:
num_loss_units = 1
for i in range(0, len(loss_units)):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_credits	int	-	-
cond_default_probs	Sequence[float]	-	-
loss_units	Sequence[float]	-	-
np.ndarray		-	-
num_loss_units	-	-	1

9.28 merton_firm

Class: MertonFirm

Implementation of the Merton Firm Value Model according to the original formulation by Merton with the inputs being the asset value of the firm, the liabilities (bond face), the time to maturity in years, the risk-free rate, the asset growth rate and the asset value volatility.

MertonFirm

Create an object that holds all of the model parameters. These parameters may be vectorised.

```
MertonFirm(asset_value: Union[float, np.ndarray],
bond_face: Union[float, np.ndarray],
years_to_maturity: Union[float, np.ndarray],
risk_free_rate: Union[float, np.ndarray],
asset_growth_rate: Union[float, np.ndarray],
asset_volatility: Union[float, np.ndarray],
None:
    """Create an object that holds all of the model parameters. These
parameters may be vectorised."""
    check_argument_types(.MertonFirm, locals())
    ._a = np.array(asset_value)
    ._l = np.array(bond_face)
    ._t = np.array(years_to_maturity)
    ._r = np.array(risk_free_rate)
    ._mu = np.array(asset_growth_rate)
    ._va = np.array(asset_volatility)
    ._d = .debt_value()
    ._e = .equity_value()
    ._ve = .equity_vol()
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
asset_value	float or np.ndarray	-	-
bond_face	float or np.ndarray	-	-
years_to_maturity	float or np.ndarray	-	-
risk_free_rate	float or np.ndarray	-	-
asset_growth_rate	float or np.ndarray	-	-
asset_volatility	float or np.ndarray	-	-
None		-	-
"""Create an object that holds all of the model parameters. These parameters may be vectorised."""	-	-	-
check_argument_types(MertonFirm, locals())	-	-	-
._a	-	-	np.array(asset_value)
._l	-	-	np.array(bond_face)
._t	-	-	np.array(years_to_maturity)
._r	-	-	np.array(risk_free_rate)
._mu	-	-	np.array(asset_growth_rate)
._va	-	-	np.array(asset_volatility)
._d	-	-	.debt_value
._e	-	-	.equity_value
._ve	-	-	.equity_volatility

leverage

Calculate the leverage.

```
leverage() -> np.ndarray:
"""Calculate the leverage."""
lvg = ._a / ._l
return lvg
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
)-ζ np.ndarray		-	-
"""Calculate the leverage."""	-	-	-
lvg	-	-	._a / ._l
return lvg	-	-	-

asset_value

Calculate the asset value.

```
asset_value() -> np.ndarray:
"""Calculate the asset value."""
return ._a
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray		-	-
"""Calculate the asset value."""	-	-	-
return ._a	-	-	-

debt_face_value

Calculate the asset value.

```
debt_face_value() -> np.ndarray:
"""Calculate the asset value."""
return ._l
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray		-	-
"""Calculate the asset value."""	-	-	-
return ._l	-	-	-

asset_vol

Return the asset volatility.

```
asset_vol() -> np.ndarray:
"""Return the asset volatility."""
return ._va
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray		-	-
"""Return the asset volatility."""	-	-	-
return ._va	-	-	-

equity_vol

Calculate the equity volatility.

```
equity_vol() -> np.ndarray:
"""Calculate the equity volatility."""
e = .equity_value()
lvq = ._a / ._l
sigma_root_t = ._va * np.sqrt(._t)
dl = np.log(lvq) + (._r + 0.5 * ._va**2) * ._t
```

```

d1 = d1 / sigma_root_t
evol = (.a / e) * normcdf_vect(d1) * .va
return evol
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Calculate the equity volatility."""	-	-	-
e	-	-	.equity_value()
lv	-	-	.a / .l
sigma_root_t	-	-	.va * np.sqrt(.t)
d1	-	-	np.log(lv) + (.r + 0.5 * .va**2) * .t
d1	-	-	d1 / sigma_root_t
evol	-	-	(.a / e) * normcdf_vect(d1) * .va
return evol	-	-	-

equity_value

Calculate the equity value.

```

equity_value() -> np.ndarray:
"""Calculate the equity value."""
lv = .a / .l
sigma_root_t = .va * np.sqrt(.t)
d1 = np.log(lv) + (.r + 0.5 * .va**2) * .t
d1 = d1 / sigma_root_t
d2 = d1 - sigma_root_t
eval = .a * normcdf_vect(d1) - .l * np.exp(-.r * .t)
) * normcdf_vect(d2)
return eval
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Calculate the equity value."""	-	-	-
lv	-	-	.a / .l
sigma_root_t	-	-	.va * np.sqrt(.t)
d1	-	-	np.log(lv) + (.r + 0.5 * .va**2) * .t
d1	-	-	d1 / sigma_root_t
d2	-	-	d1 - sigma_root_t
return eval	-	-	-

debt_value

Calculate the debt value

```

debt_value() -> np.ndarray:
"""Calculate the debt value"""
lv = ._a / ._l
sigma_root_t = ._va * np.sqrt(._t)
d1 = np.log(lv) + (._r + 0.5 * ._va**2) * ._t
d1 = d1 / sigma_root_t
d2 = d1 - sigma_root_t
dvalue = ._a * normcdf_vect(-d1) + ._l * np.exp(-._r * ._t
) * normcdf_vect(d2)
return dvalue
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Calculate the debt value"""	-	-	-
lv	-	-	._a / ._l
sigma_root_t	-	-	._va * np.sqrt(._t)
d1	-	-	np.log(lv) + (._r + 0.5 * ._va**2) * ._t
d1	-	-	d1 / sigma_root_t
d2	-	-	d1 - sigma_root_t
return dvalue	-	-	-

credit_spread

Calculate the credit spread from the debt value.

```

credit_spread() -> np.ndarray:
"""Calculate the credit spread from the debt value."""
dvalue = .debt_value()
spd = -(1.0 / ._t) * np.log(dvalue / ._l) - ._r
return spd
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Calculate the credit spread from the debt value."""	-	-	-
dvalue	-	-	.debt_value()
spd	-	-	-(1.0 / ._t) * np.log(dvalue / ._l) - ._r
return spd	-	-	-

prob_default

Calculate the default probability. This is not risk-neutral so it uses the real world drift rather than the risk-free rate.

```

prob_default() -> np.ndarray:
"""Calculate the default probability. This is not risk-neutral so it
uses the real world drift rather than the risk-free rate."""
lv = .a / .l
dd = np.log(lv) + (.mu - (.va**2) / 2.0) * .t
dd = dd / .va / np.sqrt(.t)
pd = 1.0 - normcdf_vect(dd)
return pd
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Calculate the default probability. This is not risk-neutral so it uses the real world drift rather than the risk-free rate."""	-	-	-
lv	-	-	.a / .l
dd	-	-	$\text{np.log(lv)} + (.mu - (.va**2) / 2.0) * .t$
dd	-	-	$dd / .va / \text{np.sqrt}(.t)$
pd	-	-	$1.0 - \text{normcdf_vect}(dd)$
return pd	-	-	-

dist_default

Calculate the distance to default. This is not risk-neutral so it uses the real world drift rather than the risk-free rate.

```

dist_default() -> np.ndarray:
"""Calculate the distance to default. This is not risk-neutral so it
uses the real world drift rather than the risk-free rate."""
lv = .a / .l
dd = np.log(lv) + (.mu - (.va**2) / 2.0) * .t
dd = dd / .va / np.sqrt(.t)
return dd
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> np.ndarray	-	-	-
"""Calculate the distance to default. This is not risk-neutral so it uses the real world drift rather than the risk-free rate."""	-	-	-
lv	-	-	.a / .l
dd	-	-	$\text{np.log(lv)} + (.mu - (.va**2) / 2.0) * .t$
dd	-	-	$dd / .va / \text{np.sqrt}(.t)$
return dd	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("ASSET_VALUE", ._a)
s += label_to_string("BOND_FACE", ._l)
s += label_to_string("YEARS_TO_MATURITY", ._t)
s += label_to_string("ASSET_GROWTH", ._mu)
s += label_to_string("ASSET_VOLATILITY", ._va)
return s
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -> str	-	-	-
s	-	label_to_string("OBJECT TYPE", type().__name__)	
s +	-	label_to_string("ASSET VALUE", ._a)	
s +	-	label_to_string("BOND FACE", ._l)	
s +	-	label_to_string("YEARS TO MATURITY", ._t)	
s +	-	label_to_string("ASSET GROWTH", ._mu)	
s +	-	label_to_string("ASSET VOLATILITY", ._va)	
return s	-	-	-

9.29 merton_firm_mkt

Class: MertonFirmMkt(MertonFirm)

Market Extension of the Merton Firm Model according to the original formulation by Merton with the inputs being the equity value of the firm, the liabilities (bond face), the time to maturity in years, the risk-free rate, the asset growth rate and the equity volatility. The asset value and asset volatility are computed internally by solving two non-linear simultaneous equations.

MertonFirmMkt

PLEASE ADD A FUNCTION DESCRIPTION

```
MertonFirmMkt(equity_value: Union[float, np.ndarray],
bond_face: Union[float, np.ndarray],
years_to_maturity: Union[float, np.ndarray],
risk_free_rate: Union[float, np.ndarray],
asset_growth_rate: Union[float, np.ndarray],
equity_volatility: Union[float, np.ndarray],
"""Create an object that holds all of the model parameters. These
parameters may be vectorised."""
check_argument_types(.MertonFirmMkt, locals())
if isinstance(equity_value, float):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
equity_value	float or np.ndarray	-	-
bond_face	float or np.ndarray	-	-
years_to_maturity	float or np.ndarray	-	-
risk_free_rate	float or np.ndarray	-	-
asset_growth_rate	float or np.ndarray	-	-
equity_volatility	float or np.ndarray	-	-
"""Create an object that holds all of the model parameters. These parameters may be vectorised."""	-	-	-
check_argument_types(.MertonFirmMkt, locals())	-	-	-

_solve_for_asset_value_and_vol

PLEASE ADD A FUNCTION DESCRIPTION

```
_solve_for_asset_value_and_vol() -> None:
._a = []
._va = []
for i in range(0, ._nmax):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i None		-	-
..a	-	-	[]
..va	-	-	[]

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("EQUITY_VALUE", ..e)
s += label_to_string("BOND_FACE", ..l)
s += label_to_string("YEARS_TO_MATURITY", ..t)
s += label_to_string("ASSET_GROWTH", ..mu)
s += label_to_string("EQUITY_VOLATILITY", ..ve)
return s
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -i str		-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("EQUITY VALUE", ..e)
s +	-	-	label_to_string("BOND FACE", ..l)
s +	-	-	label_to_string("YEARS TO MATURITY", ..t)
s +	-	-	label_to_string("ASSET GROWTH", ..mu)
s +	-	-	label_to_string("EQUITY VOLATILITY", ..ve)
return s	-	-	-

fobj

Find value of asset value and vol that fit equity value and vol

```
_fobj(x: Any, *args: Any) -> float:
"""Find_value_of_asset_value_and_vol_that_fit_equity_value_and_vol"""
a, v_a = x
e = args[0]
v_e = args[1]
l = args[2]
t = args[3]
r = args[4]
lv_g = a / l
sigma_root_t = v_a * np.sqrt(t)
d1 = np.log(lv_g) + (r + 0.5 * v_a**2) * t
d1 = d1 / sigma_root_t
d2 = d1 - sigma_root_t
v_e_lhs = (a / e) * normcdf(d1) * v_a
e_lhs = a * normcdf(d1) - 1 * np.exp(-r * t) * normcdf(d2)
obj = (e - e_lhs) ** 2 + (v_e - v_e_lhs) ** 2
return obj
```

```
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
x	Any	-	-
*args	Any)-float:	-	-
"""Find value of asset value and vol that fit equity value and vol"""	-	-	-
a	-	-	a
v_a	-	-	v_a
e	-	-	e
v_e	-	-	v_e
l	-	-	l
t	-	-	t
r	-	-	r
lv	-	-	lv
sigma_root_t	-	-	v_a *
d1	-	-	np.log(lv) + (r
d1	-	-	d1 / si
d2	-	-	d1 - si
v_e_lhs	-	-	(a / e) * no
e_lhs	-	-	a * normcdf(d1) - 1 * n
obj	-	-	(e - e_lhs) ** 2 -
return obj	-	-	-

9.30 model

Class: Model

class Model:

Model

PLEASE ADD A FUNCTION DESCRIPTION

```
Model () :
```

The function arguments are described in the following table.

9.31 option_implied_dbn

option_implied_dbn

PLEASE ADD A FUNCTION DESCRIPTION

```
option_implied_dbn(
s: float,
t: float,
r: float,
q: float,
strikes: Sequence[float],
sigmas: Sequence[float]
    np.ndarray:
    """This function calculates the option smile/skew-implied probability
    density function times the interval width."""
    if len(strikes) != len(sigmas):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
s	float	-	-
t	float	-	-
r	float	-	-
q	float	-	-
strikes	Sequence[float]	-	-
sigmas	Sequence[float]	-	-
np.ndarray		-	-
"""This function calculates the option smile/skew-implied probability density function times the interval width."""	-	-	-

9.32 process_simulator

Enumerated Type: ProcessTypes

This enumerated type has the following values:

- GBM_PROCESS
- CIR_PROCESS
- HESTON_PROCESS
- VASICEK_PROCESS
- CEV_PROCESS
- JUMP_DIFFUSION_PROCESS
- ifprocess_type
- (stock_price,drift,volatility,scheme)
- paths
- elifprocess_type
- (stock_price,drift,v0,kappa,theta,sigma,rho,scheme)
- paths
- elifprocess_type
- (r0,kappa,theta,sigma,scheme)
- paths
- elifprocess_type
- (r0,kappa,theta,sigma,scheme)
- paths

Enumerated Type: FinHestonNumericalScheme

This enumerated type has the following values:

- EULER_SCHEME
- EULERLOG_SCHEME
- QUADEXP_SCHEME
- cache
- fastmath
- parallel
- dt
- num_steps
- s_paths
- s_paths[:,0]
- sdt
- rhohat
- sigma2
- ifscheme
- s
- v
- z1
- z2
- z_v

- z_s
- v_plus
- rtv_plus
- v+
- s+
- s_paths[i_path,i_step]
- elifscheme
- x
- v
- z_v
- z_s
- v_plus
- rtv_plus
- x+
- v+
- s_paths[i_path,i_step]
- elifscheme
- q
- psic
- gamma1
- gamma2
- k_0
- k_1
- k_2
- k_3
- k4
- a
- mu
- c1
- c2
- x
- vn
- z_v
- z_s
- m
- m2
- s2
- psi
- u
- ifpsi_i
- b2
- a
- b
- z_v
- vnp
- d

- m
- k_0
- p
- beta
- ifu_i
- vnp
- vnp
- m
- k_0
- x₊
- s_paths[i_path,i_step]
- vn

Enumerated Type: FinGBMNumericalScheme

This enumerated type has the following values:

- NORMAL_SCHEME
- ANTITHETIC_SCHEME
- cache
- parallel
- dt
- num_time_steps
- vsqrt_dt
- m
- ifscheme
- s_all
- s_all[:,0]
- gl_d
- w
- s_all[ip,it]
- elifscheme
- s_all
- s_all[:,0]
- gl_d
- w
- s_all[ip,it]
- s_all[ip+num_paths,it]
- m
- v

Enumerated Type: FinVasicekNumericalScheme

This enumerated type has the following values:

- NORMAL
- ANTITHETIC
- cache

- fastmath
- dt
- num_steps
- sigma_sqrt_dt
- ifscheme
- rate_path
- rate_path[:,0]
- r
- z
- r+
- rate_path[i_path,i_step]
- elifscheme
- rate_path
- rate_path[:,0]
- r1
- r2
- z
- r1
- r2
- rate_path[i_path,i_step]
- rate_path[i_path+num_paths,i_step]

Enumerated Type: CIRNumericalScheme

This enumerated type has the following values:

- EULER_SCHEME
- LOGNORMAL_SCHEME
- MILSTEIN_SCHEME
- KAHLJACKEL_SCHEME
- EXACT_SCHEME
- cache
- fastmath
- dt
- num_steps
- rate_path
- rate_path[:,0]
- ifscheme
- sigma_sqrt_dt
- r
- z
- rplus
- sqrt_rplus
- r
- rate_path[i_path,i_step]
- elifscheme
- x

- y
- r
- z
- mean
- var
- sig
- r
- rate_path[i_path,i_step]
- elifscheme
- sigma_sqrt_dt
- sigma2dt
- r
- z
- sqrt_rplus
- r
- r
- rate_path[i_path,i_step]
- elifscheme
- bhat
- sqrt_dt
- r
- z
- beta
- sqrt_rplus
- c
- r
- rate_path[i_path,i_step]

Class: FinProcessSimulator

class FinProcessSimulator:

FinProcessSimulator

PLEASE ADD A FUNCTION DESCRIPTION

```
FinProcessSimulator():
```

The function arguments are described in the following table.

get_process

PLEASE ADD A FUNCTION DESCRIPTION

```
get_process(process_type, t, model_params, num_annual_steps, num_paths, seed
if process_type == ProcessTypes.GBM_PROCESS:
    (stock_price, drift, volatility, scheme) = model_params
```

```

    paths = get_gbm_paths(num_paths,
                          num_annual_steps,
                          t,
                          drift,
                          stock_price,
                          volatility,
                          scheme.value,
                          seed,
                          )
    return paths
elif process_type == ProcessTypes.HESTON_PROCESS:
    (stock_price, drift, v0, kappa, theta, sigma, rho, scheme) = model_params
    paths = get_heston_paths(num_paths,
                             num_annual_steps,
                             t,
                             drift,
                             stock_price,
                             v0,
                             kappa,
                             theta,
                             sigma,
                             rho,
                             scheme.value,
                             seed,
                             )
    return paths
elif process_type == ProcessTypes.VASICEK_PROCESS:
    (r0, kappa, theta, sigma, scheme) = model_params
    paths = get_vasicek_paths(num_paths,
                              num_annual_steps,
                              t,
                              r0,
                              kappa,
                              theta,
                              sigma,
                              scheme.value,
                              seed,
                              )
    return paths
elif process_type == ProcessTypes.CIR_PROCESS:
    (r0, kappa, theta, sigma, scheme) = model_params
    paths = get_cir_paths(num_paths,
                          num_annual_steps,
                          t,
                          r0,
                          kappa,
                          theta,
                          sigma,
                          scheme.value,
                          seed,
                          )
    return paths
else:
    raise FinError("Unknown_process" + str(process_type))
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
process_type	-	-	-
t	-	-	-
model_params	-	-	-
num_annual_steps	-	-	-
num_paths	-	-	-
seed	-	-	-
if process_type	-	-	= ProcessTypes.GBM_PROCESS:
(stock_price, drift, volatility, scheme)	-	-	model_params
num_annual_steps	-	-	-
t	-	-	-
drift	-	-	-
stock_price	-	-	-
volatility	-	-	-
scheme.value	-	-	-
seed	-	-	-
)	-	-	-
return paths	-	-	-
elif process_type	-	-	= ProcessTypes.HESTON_PROCESS:
(stock_price, drift, v0, kappa, theta, sigma, rho, scheme)	-	-	model_params
num_annual_steps	-	-	-
t	-	-	-
drift	-	-	-
stock_price	-	-	-
v0	-	-	-
kappa	-	-	-
theta	-	-	-
sigma	-	-	-
rho	-	-	-
scheme.value	-	-	-
seed	-	-	-
)	-	-	-
return paths	-	-	-
elif process_type	-	-	= ProcessTypes.VASICEK_PROCESS:
(r0, kappa, theta, sigma, scheme)	-	-	model_params
num_annual_steps	-	-	-
t	-	-	-
r0	-	-	-
kappa	-	-	-
theta	-	-	-
sigma	-	-	-
scheme.value	-	-	-
seed	-	-	-
)	-	-	-
return paths	-	-	-
elif process_type	-	-	= ProcessTypes.CIR_PROCESS:
(r0, kappa, theta, sigma, scheme)	-	-	model_params
num_annual_steps	-	-	-
t	-	-	-
r0	-	-	-
kappa	-	-	-
theta	-	-	-

get_heston_paths

PLEASE ADD A FUNCTION DESCRIPTION

```
get_heston_paths(  
    num_paths,  
    num_annual_steps,  
    t,  
    drift,  
    s0,  
    v0,  
    kappa,  
    theta,  
    sigma,  
    rho,  
    scheme,  
    seed,  
    np.random.seed(seed)  
    dt = 1.0 / num_annual_steps  
    num_steps = int(t / dt)  
    s_paths = np.empty(shape=(num_paths, num_steps + 1))  
    s_paths[:, 0] = s0  
    sdt = sqrt(dt)  
    rhohat = sqrt(1.0 - rho * rho)  
    sigma2 = sigma * sigma  
    if scheme == FinHestonNumericalScheme.EULER_SCHEME.value:  
        # Basic scheme to first order with truncation on variance  
        for i_path in nb.prange(num_paths):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_paths	-	-	-
num_annual_steps	-	-	-
t	-	-	-
drift	-	-	-
s0	-	-	-
v0	-	-	-
kappa	-	-	-
theta	-	-	-
sigma	-	-	-
rho	-	-	-
scheme	-	-	-
seed	-	-	-
np.random.seed(seed)	-	-	-
dt	-	-	1.0 / num_annual_steps
num_steps	-	-	int(t / dt)
s_paths	-	-	np.empty(shape=(num_paths, num_steps + 1))
s_paths[	,0]	-	s0
sdt	-	-	sqrt(dt)
rhohat	-	-	sqrt(1.0 - rho * rho)
sigma2	-	-	sigma * sigma
if scheme	-	-	= FinHestonNumericalScheme.EULER_SCHEME.value:

get_gbm_paths

PLEASE ADD A FUNCTION DESCRIPTION

```
get_gbm_paths(num_paths, num_annual_steps, t, mu, stock_price, sigma, scheme, seed):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_paths	-	-	-
num_annual_steps	-	-	-
t	-	-	-
mu	-	-	-
stock_price	-	-	-
sigma	-	-	-
scheme	-	-	-
seed	-	-	-

get_vasicek_paths

PLEASE ADD A FUNCTION DESCRIPTION

```
get_vasicek_paths(
```

```

num_paths, num_annual_steps, t, r0, kappa, theta, sigma, scheme, seed
np.random.seed(seed)
dt = 1.0 / num_annual_steps
num_steps = int(t / dt)
sigma_sqrt_dt = sigma * sqrt(dt)
if scheme == FinVasicekNumericalScheme.NORMAL.value:
    rate_path = np.empty((num_paths, num_steps + 1))
    rate_path[:, 0] = r0
    for i_path in nb.prange(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_paths	-	-	-
num_annual_steps	-	-	-
t	-	-	-
r0	-	-	-
kappa	-	-	-
theta	-	-	-
sigma	-	-	-
scheme	-	-	-
seed	-	-	-
np.random.seed(seed)	-	-	-
dt	-	-	1.0 / num_annual_steps
num_steps	-	-	int(t / dt)
sigma_sqrt_dt	-	-	sigma * sqrt(dt)
if scheme	-	-	= FinVasicekNumericalScheme.NORMAL.value:
rate_path	-	-	np.empty((num_paths, num_steps + 1))
rate_path[	,0]	-	r0

get_cir_paths

PLEASE ADD A FUNCTION DESCRIPTION

```

get_cir_paths(
num_paths, num_annual_steps, t, r0, kappa, theta, sigma, scheme, seed
np.random.seed(seed)
dt = 1.0 / num_annual_steps
num_steps = int(t / dt)
rate_path = np.empty(shape=(num_paths, num_steps + 1))
rate_path[:, 0] = r0
if scheme == CIRNumericalScheme.EULER_SCHEME.value:
    sigma_sqrt_dt = sigma * sqrt(dt)
    for i_path in nb.prange(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_paths	-	-	-
num_annual_steps	-	-	-
t	-	-	-
r0	-	-	-
kappa	-	-	-
theta	-	-	-
sigma	-	-	-
scheme	-	-	-
seed	-	-	-
np.random.seed(seed)	-	-	-
dt	-	-	1.0 / num_annual_steps
num_steps	-	-	int(t / dt)
rate_path	-	-	np.empty(shape=(num_paths, num_steps + 1))
rate_path[	,0]	-	r0
if scheme	-	-	= CIRNumericalScheme.EULER_SCHEME.value:
sigma_sqrt_dt	-	-	sigma * sqrt(dt)

9.33 rates_ho_lee

Class: ModelRatesHoLee

class ModelRatesHoLee:

ModelRatesHoLee

Construct Ho-Lee model using single parameter of volatility. The dynamical equation is $dr = \theta(t) dt + \sigma * dW$. Any no-arbitrage fitting is done within functions below.

```
ModelRatesHoLee(sigma: float) -> None:
    """Construct Ho-Lee model using single parameter of volatility. The
    dynamical equation is  $dr = \theta(t) dt + \sigma * dW$ . Any no-arbitrage
    fitting is done within functions below."""
    if sigma < 0.0:
        raise FinError("Negative volatility not allowed.")
    ._sigma = sigma
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
sigma	float)-;None:	-	
"""Construct Ho-Lee model using single parameter of volatility. The dynamical equation is $dr = \theta(t) dt + \sigma * dW$. fitting is done within functions below."""	-	-	
if sigma < 0.0	-	-	
raise FinError("Negative volatility not allowed.")	-	-	
._sigma	-	-	si

zcb

PLEASE ADD A FUNCTION DESCRIPTION

```
zcb(rt1: float,
    t1: float,
    t2: float,
    discount_curve: Any
    float:
    delta = t2 - t1
    dt = 1e-10
    pt1 = discount_curve.df(t1)
    pt1p = discount_curve.df(t1 + dt)
    pt2 = discount_curve.df(t2)
    z = p_fast(t1, t2, rt1, delta, pt1, pt1p, pt2, ._sigma)
    return z
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
rt1	float	-	-
t1	float	-	-
t2	float	-	-
discount_curve	Any	-	-
float		-	-
delta	-	-	t2 - t1
dt	-	-	1e-10
pt1	-	-	discount_curve.df(t1)
pt1p	-	-	discount_curve.df(t1 + dt)
pt2	-	-	discount_curve.df(t2)
z	-	-	p_fast(t1, t2, rt1, delta, pt1, pt1p, pt2, .sigma)
return z	-	-	-

option_on_zcb

Price an option on a zero coupon bond using analytical solution of Hull-White model. User provides bond face and option strike and expiry date and maturity date.

```

option_on_zcb(t_exp: float,
t_mat: float,
strike_price: float,
face_amount: float,
df_times: np.ndarray,
df_values: np.ndarray
    Dict[str, float]):
    """Price an option on a zero coupon bond using analytical solution of
    Hull-White model. User provides bond face and option strike and expiry
    date and maturity date."""
    if t_exp > t_mat:
        raise FinError("Option expiry after bond matures.")
    if t_exp < 0.0:
        raise FinError("Option expiry time negative.")
    pt_exp = _uinterpolate(t_exp, df_times, df_values, INTERP)
    pt_mat = _uinterpolate(t_mat, df_times, df_values, INTERP)
    sigma = .sigma
    sigmap = sigma * (t_mat - t_exp) * np.sqrt(t_exp)
    h = (np.log((face_amount * pt_mat) / (strike_price * pt_exp)) / sigmap
        + sigmap / 2.0
    )
    call_value = face_amount * pt_mat * normcdf(h
    ) - strike_price * pt_exp * normcdf(h - sigmap)
    put_value = strike_price * pt_exp * normcdf(-h + sigmap
    ) - face_amount * pt_mat * normcdf(-h)
    return {"call": call_value, "put": put_value}
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	
t_exp	float	-	
t_mat	float	-	
strike_price	float	-	
face_amount	float	-	
df_times	np.ndarray	-	
df_values	np.ndarray	-	
Dict[str, float]		-	
"""Price an option on a zero coupon bond using analytical solution of Hull-White model. User provides bond face and option strike and expiry date and maturity date."""			
if t_exp > t_mat		-	
raise FinError("Option expiry after bond matures.")	-	-	
if t_exp < 0.0		-	
raise FinError("Option expiry time negative.")	-	-	
pt_exp	-	-	_uinterpolate(t_exp,
pt_mat	-	-	_uinterpolate(t_mat,
sigma	-	-	
sigmap	-	-	sigma * (t_mat -
+ sigmap / 2.0	-	-	
)	-	-	
return "call"	call_value	-	
"put"	put_value	-	

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
t -> str		-	-

p_fast

Forward discount factor as seen at some time t which may be in the future for payment at time T where R_t is the delta-period short rate seen at time t and p_t is the discount factor to time t , p_{t+dt} is the one period discount factor to time $t+dt$ and p_T is the discount factor from now until the payment of the 1 dollar of the discount factor.

```
p_fast(
t: float,
t_mat: float,
rt: float,
delta: float,
```

```

pt: float,
ptd: float,
p_mat: float,
sigma: float
float:
"""Forward_discount_factor_as_seen_at_some_time_t_which_may_be_in_the
future_for_payment_at_time_T_where_Rt_is_the_delta-period_short_rate
seen_at_time_t_and_pt_is_the_discount_factor_to_time_t,_ptd_is_the_one
period_discount_factor_to_time_t+dt_and_pT_is_the_discount_factor_from
now_until_the_payment_of_the_1_dollar_of_the_discount_factor."""
bt_t = t_mat - t
bt_delta = delta
term1 = np.log(p_mat / pt) - (bt_t / bt_delta) * np.log(ptd / pt)
term2 = (sigma**2) * t * bt_t * (bt_t - bt_delta) / (2.0)
log_ahat = term1 - term2
bhatt_t = (bt_t / bt_delta) * delta
p = np.exp(log_ahat - bhatt_t * rt)
return p
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default
t	float	-	
t_mat	float	-	
rt	float	-	
delta	float	-	
pt	float	-	
ptd	float	-	
p_mat	float	-	
sigma	float	-	
float		-	
"""Forward discount factor as seen at some time t which may be in the	-	-	
future for payment at time T where Rt is the delta-period short rate	-	-	
seen at time t and pt is the discount factor to time t	-	-	
ptd is the one	-	-	
period discount factor to time t+dt and pT is the discount factor from	-	-	
now until the payment of the 1 dollar of the discount factor."""	-	-	
bt_t	-	-	t_m
bt_delta	-	-	de
term1	-	-	np.log(p_mat / pt) - (bt_t /
term2	-	-	(sigma**2) * t * bt_t *
log_ahat	-	-	term1
bhatt_t	-	-	(bt_t / bt_d
p	-	-	np.exp(log_ah
return p	-	-	

9.34 sabr

Class: SABR

SABR - Stochastic alpha beta rho model by Hagan et al. which is a stochastic volatility model where alpha controls the implied volatility, beta is the exponent on the the underlying assets process so beta = 0 is normal and beta = 1 is lognormal, rho is the correlation between the underlying and the volatility process.

SABR

Create SABR with all of the model parameters. We will also provide functions below to assist with the calibration of the value of alpha.

```
SABR(alpha, beta, rho, nu):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	-	-	-
beta	-	-	-
rho	-	-	-
nu	-	-	-

black_vol

Black volatility from SABR model using Hagan et al. approx.

```
black_vol(f, k, t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
f	-	-	-
k	-	-	-
t	-	-	-

black_vol_with_alpha

PLEASE ADD A FUNCTION DESCRIPTION

```
black_vol_with_alpha(alpha, f, k, t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	-	-	-
f	-	-	-
k	-	-	-
t	-	-	-

value

Price an option using Black's model which values in the forward measure following a change of measure.

```

value(forward_rate, # Forward rate
strike_rate, # Strike Rate
time_to_expiry, # time to expiry in years
df, # Discount Factor to expiry date
call_or_put,
# Call or put
"""Price an option using Black's model which values in the forward
measure following a change of measure."""
f = forward_rate
t = time_to_expiry
k = strike_rate
sqrt_t = np.sqrt(t)
vol = .black_vol(f, k, t)
d1 = np.log(f / k) + vol * vol * t / 2
d1 = d1 / (vol * sqrt_t)
d2 = d1 - vol * sqrt_t
if call_or_put == OptionTypes.EUROPEAN_CALL:
    return df * (f * normcdf(d1) - k * normcdf(d2))
elif call_or_put == OptionTypes.EUROPEAN_PUT:
    return df * (k * normcdf(-d2) - f * normcdf(-d1))
else:
    raise Exception("Option_type_must_be_a_European_Call (C)_or_Put (P) ")
#####

```

The function arguments are described in the following table.

Argument Name	Type	Description
forward_rate	-	Forward rate
strike_rate	-	Strike Rate
time_to_expiry	-	time to expiry in years
df	-	Discount Factor to expiry date
call_or_put	-	-
"""Price an option using Black's model which values in the forward measure following a change of measure."""		
f	-	-
t	-	-
k	-	-
sqrt_t	-	-
vol	-	-
d1	-	- np.lo
d1	-	-
d2	-	-
if call_or_put	-	= Option
return df * (f * normcdf(d1) - k * normcdf(d2))	-	-
elif call_or_put	-	= Option
return df * (k * normcdf(-d2) - f * normcdf(-d1))	-	-
else	-	-
raise Exception("Option type must be a European Call(C) or Put(P)")	-	-

set_alpha_from_black_vol

Estimate the value of the alpha coefficient of the SABR model by solving for the value of alpha that makes the SABR black vol equal to the input black vol. This uses a numerical 1D solver.

```
set_alpha_from_black_vol(black_vol, forward, strike, time_to_expiry):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
black_vol	-	-	-
forward	-	-	-
strike	-	-	-
time_to_expiry	-	-	-

fn

PLEASE ADD A FUNCTION DESCRIPTION

```
fn(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

set_alpha_from_atm_black_vol

We solve cubic equation for the unknown variable alpha for the special ATM case of the strike equalling the forward following Hagan and al. equation (3.3). We take the smallest real root as the preferred solution. This is useful for calibrating the model when beta has been chosen.

```
set_alpha_from_atm_black_vol(black_vol, atm_strike, time_to_expiry):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
black_vol	-	-	-
atm_strike	-	-	-
time_to_expiry	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__():
```

The function arguments are described in the following table.

_X

Return function x used in Hagan's 2002 SABR lognormal vol expansion.

```
_x(rho, z):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
rho	-	-	-
z	-	-	-

vol_function_sabr

Black volatility implied by SABR model.

```
vol_function_sabr(params, f, k, t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	-	-	-
f	-	-	-
k	-	-	-
t	-	-	-

vol_function_sabr_beta_one

This is the SABR function with the exponent beta set equal to 1 so only 3 parameters are free. The first parameter is alpha, then nu and the third parameter is rho. Check the order as it is not the same as main SABR fn

```
vol_function_sabr_beta_one(params, f, k, t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	-	-	-
f	-	-	-
k	-	-	-
t	-	-	-

vol_function_sabr_beta_half

Black volatility implied by SABR model.

```
vol_function_sabr_beta_half(params, f, k, t):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	-	-	-
f	-	-	-
k	-	-	-
t	-	-	-

9.35 sabr_shifted

Class: SABRShifted

SABR - Shifted Stochastic alpha beta rho model by Hagan et al. is a stochastic volatility model where alpha controls the implied volatility, beta is the exponent on the the underlying assets process so beta = 0 is normal and beta = 1 is lognormal, rho is the correlation between the underlying and the volatility process. The shift allows negative rates.

SABRShifted

Create SABRShifted with all of the model parameters. We also provide functions below to assist with the calibration of the value of alpha.

```
SABRShifted(alpha: float,
beta: float,
rho: float,
nu: float,
shift: float
None:
"""Create SABRShifted with all of the model parameters. We
also provide functions below to assist with the calibration of the
value of alpha."""
._alpha = alpha
._beta = beta
._rho = rho
._nu = nu
._shift = shift
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	float	-	-
beta	float	-	-
rho	float	-	-
nu	float	-	-
shift	float	-	-
None		-	-
"""Create SABRShifted with all of the model parameters. We also provide functions below to assist with the calibration of the value of alpha."""	-	-	-
._alpha	-	-	alpha
._beta	-	-	beta
._rho	-	-	rho
._nu	-	-	nu
._shift	-	-	shift

black_vol

PLEASE ADD A FUNCTION DESCRIPTION

```
black_vol(f: Union[float, np.ndarray],
k: Union[float, np.ndarray],
t: Union[float, np.ndarray]
Union[float, np.ndarray]:
"""Black_volatility_from_SABR_model_using_Hagan_et_al._approx."""
params = np.array([._alpha, ._beta, ._rho, ._nu, ._shift])
# I wish to enable vectorisations
if isinstance(f, np.ndarray):
```

The function arguments are described in the following table.

Argument Name	Type	Description	
f	float or np.ndarray	-	
k	float or np.ndarray	-	
t	float or np.ndarray	-	
Union[float, np.ndarray]		-	
"""Black volatility from SABR model using Hagan et al. approx."""	-	-	
params	-	-	np.array([._alph

black_vol_with_alpha

PLEASE ADD A FUNCTION DESCRIPTION

```
black_vol_with_alpha(alpha: np.ndarray,
f: float,
k: float,
t: float
float:
._alpha = alpha[0]
black_vol = .black_vol(f, k, t)
return black_vol
#####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
alpha	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float		-	-
._alpha	-	-	alpha[0]
black_vol	-	-	.black_vol(f, k, t)
return black_vol	-	-	-

value

Price an option using Black's model which values in the forward measure following a change of measure.

```

value(forward_rate: float,
      strike_rate: float,
      time_to_expiry: float,
      df: float,
      call_or_put: Any
      float:
      """Price an option using Black's model which values in the forward
      measure following a change of measure."""
      f = forward_rate
      t = time_to_expiry
      k = strike_rate
      sqrt_t = np.sqrt(t)
      vol = .black_vol(f, k, t)
      d1 = (np.log((f) / (k)) + vol * vol * t / 2) / (vol * sqrt_t)
      d2 = d1 - vol * sqrt_t
      if call_or_put == OptionTypes.EUROPEAN_CALL:
          return df * (f * normcdf(d1) - k * normcdf(d2))
      elif call_or_put == OptionTypes.EUROPEAN_PUT:
          return df * (k * normcdf(-d2) - f * normcdf(-d1))
      raise FinError("Option type must be a European Call(C) or Put(P)")
      #####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
forward_rate	float	-	-
strike_rate	float	-	-
time_to_expiry	float	-	-
df	float	-	-
call_or_put	Any	-	-
float		-	-
"""Price an option using Black's model which values in the forward	-	-	-
measure following a change of measure."""	-	-	-
f	-	-	forward_rate
t	-	-	time_to_expiry
k	-	-	strike_rate
sqrt_t	-	-	np.sqrt(t)
vol	-	-	.black_vol(f, k, t)
d1	-	(np.log((f) / (k)) + vol * vol * t / 2) / (vol * sqrt_t)	
d2	-	d1 - vol * sqrt_t	
if call_or_put	-	= OptionTypes.EUROPEAN_CALL	
return df * (f * normcdf(d1) - k * normcdf(d2))	-	-	-
elif call_or_put	-	= OptionTypes.EUROPEAN_PUT	
return df * (k * normcdf(-d2) - f * normcdf(-d1))	-	-	-
raise FinError("Option type must be a European Call(C) or Put(P)")	-	-	-

set_alpha_from_black_vol

Estimate the value normcdf of the alpha coefficient of the SABR model by solving for the value of alpha that makes the SABR black vol equal to the input black vol. This uses a numerical 1D solver.

```
set_alpha_from_black_vol(black_vol: float,
                        forward: float,
                        strike: float,
                        time_to_expiry: float
                        None:
                        """Estimate the value normcdf of the alpha coefficient of the SABR model
                        by solving for the value of alpha that makes the SABR black vol equal
                        to the input black vol. This uses a numerical 1D solver."""
                        t_exp = time_to_expiry
                        f = forward
                        k = strike
                        # The starting point is based on assuming that the strike is ATM
                        .set_alpha_from_atm_black_vol(black_vol, strike, time_to_expiry)
                        init_alpha = ._alpha
                        if init_alpha != black_vol:
                            # Objective function
                            #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
black_vol	float	-	-
forward	float	-	-
strike	float	-	-
time_to_expiry	float	-	-
None		-	-
by solving for the value of alpha that makes the SABR black vol equal	-	-	-
to the input black vol. This uses a numerical 1D solver."""	-	-	-
t_exp	-	-	time_to_expiry
f	-	-	forward
k	-	-	strike
.set_alpha_from_atm_black_vol(black_vol, strike, time_to_expiry)	-	-	-
init_alpha	-	-	._alpha
if init_alpha !	-	-	black_vol:

fn

PLEASE ADD A FUNCTION DESCRIPTION

```
fn(x):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	-	-	-

set_alpha_from_atm_black_vol

We solve cubic equation for the unknown variable alpha for the special ATM case of the strike equalling the forward following Hagan and al. equation (3.3). We take the smallest real root as the preferred solution. This is useful for calibrating the model when beta has been chosen.

```

set_alpha_from_atm_black_vol(black_vol: float,
atm_strike: float,
time_to_expiry: float
    None:
    """We solve cubic equation for the unknown variable alpha for the
    special ATM case of the strike equalling the forward following Hagan
    and al. equation (3.3). We take the smallest real root as the preferred
    solution. This is useful for calibrating the model when beta has been
    chosen."""
    # For shifted SABR
    atm_strike = atm_strike + ._shift
    beta = ._beta
    rho = ._rho
    nu = ._nu
    t_exp = time_to_expiry
    k = atm_strike
    coeff0 = -black_vol * (k ** (1.0 - ._beta))
    coeff1 = 1.0 + ((2.0 - 3.0 * rho**2) / 24.0) * (nu**2) * t_exp
    coeff2 = (rho * beta * nu * t_exp) / (4.0 * (k ** (1.0 - beta)))
    coeff3 = (((1.0 - beta) ** 2) * t_exp) / (24.0 * (k ** (2.0 - 2.0 * beta)))
    coeffs = [coeff3, coeff2, coeff1, coeff0]
    roots = np.roots(coeffs)
    # Selecting the smallest positive real root
    alpha = np.min([coeff.real for coeff in roots if coeff.real > 0])
    ._alpha = alpha
    #####

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
black_vol	float	-	-
atm_strike	float	-	-
time_to_expiry	float	-	-
None	-	-	-
"""We solve cubic equation for the unknown variable alpha for the special ATM case of the strike equalling the forward following Hagan and al. equation (3.3). We take the smallest real root as the preferred solution. This is useful for calibrating the model when beta has been chosen."""			
atm_strike	-	-	atm_strike
beta	-	-	-
rho	-	-	-
nu	-	-	-
t_exp	-	-	time_to_expiry
k	-	-	a
coeff0	-	-	-black_vol * t_exp
coeff1	-	-	1.0 + ((2.0 - 3.0 * rho) * t_exp)
coeff2	-	-	(rho * beta * nu * t_exp)
coeff3	-	-	((1.0 - beta) ** 2) * t_exp
coeffs	-	-	[coeff3, coeff2, coeff1, coeff0]
roots	-	-	np.roots(coeffs)
alpha	-	-	np.min([coeff.real for coeff in roots])
..alpha	-	-	-

__repr__

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
z	float	-	-

X

Return function x used in Hagan's 2002 SABR lognormal vol expansion.

```
_x(rho: float, z: float) -> float:
"""Return function x used in Hagan's 2002 SABR lognormal vol expansion."""
a = (1.0 - 2.0 * rho * z + z**2) ** 0.5 + z - rho
b = 1.0 - rho
return np.log(a / b)
#####
t(fastmath=True, cache=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description
rho	float	-
z	float)-float:	-
"""Return function x used in Hagan's 2002 SABR lognormal vol expansion."""	-	-
a	-	- (1.0 - 2.0)
b	-	-
return np.log(a / b)	-	-
t(fastmath	-	-

vol_function_shifted_sabr

Black volatility implied by SABR model.

```

vol_function_shifted_sabr(
params: np.ndarray,
f: float,
k: float,
t: float
float:
"""Black_volatility_implied_by_SABR_model."""
alpha = params[0]
beta = params[1]
rho = params[2]
nu = params[3]
shift = params[4]
f = f + shift
k = k + shift
alpha = max(alpha, 1e-10)
# Negative strikes or forwards
if k <= 0:
    raise FinError("Strike_must_be_positive")
if f <= 0:
    raise FinError("Forward_must_be_positive")
logfk = np.log(f / k)
b = 1.0 - beta
fkb = (f * k) ** b
a = b**2 * alpha**2 / (24.0 * fkb)
b = 0.25 * rho * beta * nu * alpha / fkb**0.5
c = (2.0 - 3.0 * rho**2.0) * nu**2.0 / 24
d = fkb**0.5
v = b**2 * logfk**2 / 24.0
w = b**4 * logfk**4 / 1920.0
z = nu * fkb**0.5 * logfk / alpha
eps = 1e-07
if abs(z) > eps:
    vz = alpha * z * (1.0 + (a + b + c) * t) / (d * (1.0 + v + w) * _x(rho, z))
    return vz
v0 = alpha * (1.0 + (a + b + c) * t) / (d * (1.0 + v + w))
return v0
#####

```


The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float		-	-
"""Black volatility implied by SABR model."""	-	-	-
alpha	-	-	params[0]
beta	-	-	params[1]
rho	-	-	params[2]
nu	-	-	params[3]
shift	-	-	params[4]
f	-	-	f + shift
k	-	-	k + shift
alpha	-	-	max(alpha, 1e-10)
if k > 0	-	-	0:
raise FinError("Strike must be positive")	-	-	-
if f > 0	-	-	0:
raise FinError("Forward must be positive")	-	-	-
logfk	-	-	np.log(f / k)
b	-	-	1.0 - beta
fk b	-	-	(f * k) ** b
a	-	-	b**2 * alpha**2 / (24.0 * (1.0 - 3.0 * rho * beta * nu * alpha / (2.0 - 3.0 * rho**2.0) * nu**2.0))
b	-	-	0.25 * rho * beta * nu * alpha / (2.0 - 3.0 * rho**2.0) * nu**2.0
c	-	-	(2.0 - 3.0 * rho**2.0) * nu**2.0
d	-	-	fk b**0.5
v	-	-	b**2 * logfk**2 / 24.0
w	-	-	b**4 * logfk**4 / 1920
z	-	-	nu * fk b**0.5 * logfk / alpha
eps	-	-	1e-07
if abs(z) > eps	-	-	-
vz	-	-	alpha * z * (1.0 + (a + b + c) * t) / (d * (1.0 + (a + b + c) * t))
return vz	-	-	-
v0	-	-	alpha * (1.0 + (a + b + c) * t) / (d * (1.0 + (a + b + c) * t))
return v0	-	-	-

9.36 sobol

get_gaussian_sobol

Sobol Gaussian quasi random points generator based on graycode order. The generated points follow a normal distribution.

```
get_gaussian_sobol(num_points, dimension):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_points	-	-	-
dimension	-	-	-

get_uniform_sobol

Sobol uniform quasi random points generator based on graycode order. This function returns a 2D Numpy array of values where the number of rows is the number of draws and the number of columns is the number of dimensions of the random values. Each dimension has the same number of random draws. Each column of random numbers is ordered so as not to correlate, i.e be independent from any other column.

```
get_uniform_sobol(num_points, dimension):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
num_points	-	-	-
dimension	-	-	-

9.37 student_t_copula

Class: StudentTCopula

class StudentTCopula:

default_times

PLEASE ADD A FUNCTION DESCRIPTION

```
default_times(issuer_curves: Sequence[Any],
corr_matrix: np.ndarray,
degrees_of_freedom: float,
num_trials: int,
seed: int
    np.ndarray:
np.random.seed(seed)
num_credits = len(issuer_curves)
x = np.random.normal(0.0, 1.0, size=(num_credits, num_trials))
c = np.linalg.cholesky(corr_matrix)
y = np.dot(c, x)
corr_times = np.empty(shape=(num_credits, 2 * num_trials))
for i_trial in range(0, num_trials):
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
issuer_curves	Sequence[Any]	-	-
corr_matrix	np.ndarray	-	-
degrees_of_freedom	float	-	-
num_trials	int	-	-
seed	int	-	-
np.ndarray		-	-
np.random.seed(seed)	-	-	-
num_credits	-	-	len(issuer_curves)
x	-	-	np.random.normal(0.0, 1.0, size=(num_credits, num_trials))
c	-	-	np.linalg.cholesky(corr_matrix)
y	-	-	np.dot(c, x)
corr_times	-	-	np.empty(shape=(num_credits, 2 * num_trials))

9.38 vasicek_mc

Class: ModelRatesVasicek

class ModelRatesVasicek:

ModelRatesVasicek

PLEASE ADD A FUNCTION DESCRIPTION

```
ModelRatesVasicek(a: float, b: float, sigma: float) -> None:
    ._a = a
    ._b = b
    ._sigma = sigma
    #####
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	float	-	-
b	float	-	-
sigma	float)-;None:	-	-
._a	-	-	a
._b	-	-	b
._sigma	-	-	sigma

--repr--

PLEASE ADD A FUNCTION DESCRIPTION

```
__repr__() -> str:
s = label_to_string("OBJECT_TYPE", type().__name__)
s += label_to_string("a", ._a)
s += label_to_string("b", ._b)
s += label_to_string("sigma", ._sigma)
return s
#####
stmath=True, cache=True)
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
) -; str	-	-	-
s	-	-	label_to_string("OBJECT TYPE", type().__name__)
s +	-	-	label_to_string("a", ._a)
s +	-	-	label_to_string("b", ._b)
s +	-	-	label_to_string("sigma", ._sigma)
return s	-	-	-
stmath	-	-	True
cache	-	-	True)

meanr*Expectation of short rate at later time t*

```

meanr(
r0: float,
a: float,
b: float,
t: float
float:
"""Expectation_of_short_rate_at_later_time_t"""
mr = r0 * exp(-a * t) + b * (1 - exp(-a * t))
return mr
#####
t(fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
t	float	-	-
float		-	-
"""Expectation of short rate at later time t"""	-	-	-
mr	-	-	$r0 * \exp(-a * t) + b * (1 - \exp(-a * t))$
return mr	-	-	-
t(fastmath	-	-	True, cache=True)

variancer*Variance of short rate at later time t*

```

variancer(
a: float,
sigma: float,
t: float
float:
"""Variance_of_short_rate_at_later_time_t"""
vr = sigma * sigma * (1.0 - exp(-2.0 * a * t)) / 2.0 / a
return vr
#####
t(fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
a	float	-	-
sigma	float	-	-
t	float	-	-
float		-	-
""""Variance of short rate at later time t""""	-	-	-
vr	-	-	$\sigma * \sigma * (1.0 - \exp(-2.0 * a * t)) / 2.0 / a$
return vr	-	-	-
t(fastmath	-	-	True, cache=True)

zero_price

Generate zero price analytically using Vasicek model

```

zero_price(
r0: float,
a: float,
b: float,
sigma: float,
t: float
float:
""""Generate_zero_price_analytically_using_Vasicek_model""""
bb = (1.0 - exp(-a * t)) / a
aa = exp(
    (b - sigma * sigma / 2.0 / a / a) * (bb - t) - bb * bb * sigma * sigma / 4.0 / a
)
zcb = aa * exp(-r0 * bb)
return zcb
#####
t(
float64[:](float64, float64, float64, float64, float64, float64, int64),
parallel=False,
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type
r0	float
a	float
b	float
sigma	float
t	float
float	
"""Generate zero price analytically using Vasicek model"""	-
bb	-
(b - sigma * sigma / 2.0 / a / a) * (bb - t) - bb * bb * sigma * sigma / 4.0 / a	-
)	-
zcb	-
return zcb	-
float64[	float64 or float64,float64,float64,float64,float64
parallel	-
fastmath	-
cache	-

rate_path_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

rate_path_mc(
r0: float,
a: float,
b: float,
sigma: float,
t: float,
dt: float,
seed: int
    np.ndarray:
    """Generate a path of short rates using Vasicek model"""
    np.random.seed(seed)
    num_steps = int(t / dt)
    rate_path = np.zeros(num_steps)
    rate_path[0] = r0
    num_paths = 1
    sigmasqrt_dt = sigma * sqrt(dt)
    for _ in nb.prange(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
sigma	float	-	-
t	float	-	-
dt	float	-	-
seed	int	-	-
np.ndarray		-	-
"""Generate a path of short rates using Vasicek model"""	-	-	-
np.random.seed(seed)	-	-	-
num_steps	-	-	int(t / dt)
rate_path	-	-	np.zeros(num_steps)
rate_path[0]	-	-	r0
num_paths	-	-	1
sigmasqrt_dt	-	-	sigma * sqrt(dt)

zero_price_mc

PLEASE ADD A FUNCTION DESCRIPTION

```

zero_price_mc(
r0: float,
a: float,
b: float,
sigma: float,
t: float,
dt: float,
num_paths: int,
seed: int
    float:
    """Generate_zero_price_by_Monte_Carlo_using_Vasicek_model"""
np.random.seed(seed)
num_steps = int(t / dt)
sigmasqrt_dt = sigma * sqrt(dt)
zcb = 0.0
for _ in nb.prange(num_paths):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
r0	float	-	-
a	float	-	-
b	float	-	-
sigma	float	-	-
t	float	-	-
dt	float	-	-
num_paths	int	-	-
seed	int	-	-
float		-	-
"""Generate zero price by Monte Carlo using Vasicek model"""	-	-	-
np.random.seed(seed)	-	-	-
num_steps	-	-	int(t / dt)
sigmasqrt_dt	-	-	sigma * sqrt(dt)
zcb	-	-	0.0

9.39 volatility_fns

Enumerated Type: VolFuncTypes

This enumerated type has the following values:

- CLARK
- SABR
- SABR_BETA_ONE
- SABR_BETA_HALF
- BBG
- CLARK5
- SVI
- SSVI
- @njit(float64(float64[:],float64,float64,float64),fastmath
- x
- sigma0
- arg
- deltax
- f
- f+
- @njit(float64(float64[:],float64,float64,float64),fastmath
- coefficienti.e.sigma(K)
- b
- num_params
- RatherthanpassintheATMvol,implyitfromthedelta
- sigma
- pwr
- sigma+
- vsqrtt
- d1
- delta
- v
- pwr
- v+
- @njit(float64(float64[:],float64,float64,float64),fastmath
- x
- a
- b
- rho
- m
- sigma
- vart
- v
- @njit(float64(float64,float64),fastmath
- gamma
- theta

- phi
- fastmath
- cache
- """Thisisthetotalvariancew
- theta
- p
- px
- gg
- v
- fastmath
- cache
- theta
- p
- px
- v
- v
- fastmath
- cache
- theta
- p
- px
- v
- v
- fastmath
- cache
- eps
- ssvitplus
- ssvitminus
- deriv
- fastmath
- cache
- w
- w
- w1
- w2
- xwv
- v
- fastmath
- cache
- vsqrt
- v
- fastmath
- cache
- dm
- v
- v

- fastmath
- cache
- num
- den
- var
- @njit(float64(float64[:],float64,float64,float64),fastmath
- gamma
- sigma
- rho
- x
- vart
- vart
- sigma

vol_function_clark

PLEASE ADD A FUNCTION DESCRIPTION

```
vol_function_clark(
params: np.ndarray,
f: float,
k: float,
t: float
float:
"""Volatility_Function_in_book_by_Iain_Clark_generalised_to_allow_for
higher_than_quadratic_power._Care_needs_to_be_taken_to_avoid_overfitting.
The_exact_reference_is_Clark_Page_59."""
if f < 0.0:
    print("f:", f)
    raise FinError("Forward_is_negative")
if k < 0.0:
    print("k:", k)
    raise FinError("Strike_is_negative")
x = np.log(f / k)
sigma0 = np.exp(params[0])
arg = x / (sigma0 * np.sqrt(t))
deltax = normcdf(arg) - 0.50 # The -0.50 seems to be missing in book
f = 0.0
for i in range(0, len(params)):
```

The function arguments are described in the following table.

Argument Name	Type	Description
params	np.ndarray	-
f	float	-
k	float	-
t	float	-
float		-
"""Volatility Function in book by Iain Clark generalised to allow for higher than quadratic power. Care needs to be taken to avoid overfitting. The exact reference is Clark Page 59."""	-	-
if f < 0.0		-
print("f	",f)	-
raise FinError("Forward is negative")	-	-
if k < 0.0		-
print("k	",k)	-
raise FinError("Strike is negative")	-	-
x	-	-
sigma0	-	-
arg	-	-
deltax	-	The -0.50 seems to be missing in b
f	-	-

vol_function_bloomberg

PLEASE ADD A FUNCTION DESCRIPTION

```

vol_function_bloomberg(
params: np.ndarray,
f: float,
k: float,
t: float
    float:
    """Volatility Function similar to the one used by Bloomberg. It is
    a quadratic function in the spot delta of the option. It can therefore
    go negative so it requires a good initial guess when performing the
    fitting to avoid this happening. The first parameter is the quadratic
    coefficient i.e.  $\sigma(K) = a * D * D + b * D + c$  where  $a = \text{params}[0]$ ,
 $b = \text{params}[1]$ ,  $c = \text{params}[2]$  and  $D$  is the spot delta. """
    num_params = len(params)
    # Rather than pass in the ATM vol, I imply it from the delta=0.50 curve
    sigma = 0.0
    for i in range(0, len(params)):

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float	-	-	-
"""Volatility Function similar to the one used by Bloomberg. It is	-	-	-
a quadratic function in the spot delta of the option. It can therefore	-	-	-
go negative so it requires a good initial guess when performing the	-	-	-
fitting to avoid this happening. The first parameter is the quadratic	-	-	-
coefficient i.e. sigma(K)	-	-	$a * D * D + b * D + c$ where
b	-	-	params[1]
c	-	-	params[2] and D is the
num_params	-	-	len(params)
sigma	-	-	0.0

vol_function_svi

Volatility Function proposed by Gatheral in 2004. Increasing a results in a vertical translation of the smile in the positive direction. Increasing b decreases the angle between the put and call wing, i.e. tightens the smile. Increasing rho results in a counter-clockwise rotation of the smile. Increasing m results in a horizontal translation of the smile in the positive direction. Increasing sigma reduces the at-the-money curvature of the smile.

```

vol_function_svi(
    params: np.ndarray,
    f: float,
    k: float,
    t: float
    float:
    """Volatility Function proposed by Gatheral in 2004. Increasing a results
    in a vertical translation of the smile in the positive direction.
    Increasing b decreases the angle between the put and call wing, i.e.
    tightens the smile. Increasing rho results in a counter-clockwise rotation
    of the smile. Increasing m results in a horizontal translation of the smile
    in the positive direction. Increasing sigma reduces the at-the-money
    curvature of the smile."""
    x = np.log(f / k)
    a = params[0]
    b = params[1]
    rho = params[2]
    m = params[3]
    sigma = params[4]
    var_t = a + b * (rho * (x - m) + np.sqrt((x - m) ** 2 + sigma * sigma))
    v = np.sqrt(var_t / t)
    return v
)
"""
theral ssvi surface svi and equivalent local volatility
de from https://www.imperial.ac.uk/~ajacquie/IC_AMDP/IC_AMDP_Docs/Code/ssvi.pdf
#####
t(float64(float64, float64), fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type
params	np.ndarray
f	float
k	float
t	float
float	
"""Volatility Function proposed by Gatheral in 2004. Increasing a results in a vertical translation of the smile in the positive direction.	-
Increasing b decreases the angle between the put and call wing i.e.	-
tightens the smile. Increasing rho results in a counter-clockwise rotation of the smile. Increasing m results in a horizontal translation of the smile in the positive direction. Increasing sigma reduces the at-the-money curvature of the smile."""	-
x	-
a	-
b	-
rho	-
m	-
sigma	-
vart	-
v	-
return v	-
theral ssvi surface svi and equivalent local volatility	-
de from https	//wwwf.imperial.ac.uk/~ajacquie/IC_AMDPI/IC
t(float64(float64, float64), fastmath	-

phi_ssvi

PLEASE ADD A FUNCTION DESCRIPTION

```

phi_ssvi(
    theta: float,
    gamma: float
    float:
    if abs(gamma) < 1e-8:
        gamma = 1e-8
    if abs(theta) < 1e-8:
        theta = 1e-8
    phi = (1.0 / gamma / theta) * (1.0 - (1.0 - np.exp(-gamma * theta)) / gamma / theta)
    return phi
#####
t(
    float64(float64, float64, float64, float64, float64),
    fastmath=True,
    cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
theta	float	-	-
gamma	float	-	-
float		-	-
if abs(gamma) > 1e-8		-	-
gamma	-	-	1e-8
if abs(theta) > 1e-8		-	-
theta	-	-	1e-8
phi	-	-	$(1.0 / \text{gamma} / \text{theta}) * (1.0 - (1.0 - \text{np.exp}(-\text{gam}))$
return phi	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

ssvi

This is the total variance $w = \sigma(t) \times \sigma(t) (0,t) \times t$

```
ssvi(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
float:
"""This is the total variance  $w = \sigma(t) \times \sigma(t) (0,t) \times t$ """
theta = sigma * sigma * t
p = phi_ssvi(theta, gamma)
px = p * x
gg = px + rho
v = 0.5 * theta * (1.0 + rho * px + np.sqrt(gg**2 + 1.0 - rho * rho))
return v
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float		-	-
"""This is the total variance w	-	-	sigma(t) x sigma(t) (0,t) x t"""
theta	-	-	sigma * sigma * t
p	-	-	phi_ssvi(theta, gamma)
px	-	-	p * x
gg	-	-	px + rho
v	-	-	0.5 * theta * (1.0 + rho * px + np.sqrt(gg**2 + 1.0))
return v	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

ssvil

PLEASE ADD A FUNCTION DESCRIPTION

```

ssvil(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
    float:
# First derivative with respect to x
theta = sigma * sigma * t
p = phi_ssvi(theta, gamma)
px = p * x
v = 0.5 * theta * p * (px + rho * np.sqrt(px**2 + 2.0 * px * rho + 1.0) + rho)
v = v / np.sqrt(px**2 + 2.0 * px * rho + 1.0)
return v
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float		-	-
theta	-	-	sigma * sigma * t
p	-	-	phi_ssvi(theta, gamma)
px	-	-	p * x
v	-	-	$0.5 * \text{theta} * p * (px + \text{rho} * \text{np.sqrt}(px^{**2} + 2.0 * px * \text{rho} + 1.0))$
v	-	-	$v / \text{np.sqrt}(px^{**2} + 2.0 * px * \text{rho} + 1.0)$
return v	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

ssvi2

PLEASE ADD A FUNCTION DESCRIPTION

```

ssvi2(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
    float:
# Second derivative with respect to x
theta = sigma * sigma * t
p = phi_ssvi(theta, gamma)
px = p * x
v = 0.5 * theta * p * p * (1.0 - rho * rho)
v = v / ((px**2 + 2.0 * px * rho + 1.0) * np.sqrt(px**2 + 2.0 * px * rho + 1.0))
return v
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float		-	-
theta	-	-	$\sigma * \sigma * t$
p	-	-	$\phi_{ssvi}(\theta, \gamma)$
px	-	-	$p * x$
v	-	-	$0.5 * \theta * p * p * (1.0 - \rho)$
v	-	-	$v / ((px^{**2} + 2.0 * px * \rho + 1.0) * \text{np.sqrt}(px^{**2} + 2.0 * px * \rho + 1.0))$
return v	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

ssvit

PLEASE ADD A FUNCTION DESCRIPTION

```

ssvit(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
    float:
# First derivative with respect to t, by central difference
eps = 0.0001
ssvitplus = ssvi(x, gamma, sigma, rho, t + eps)
ssvitminus = ssvi(x, gamma, sigma, rho, t - eps)
deriv = (ssvitplus - ssvitminus) / 2.0 / eps
return deriv
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float		-	-
eps	-	-	0.0001
ssvitplus	-	-	ssvi(x, gamma, sigma, rho, t + eps)
ssvitminus	-	-	ssvi(x, gamma, sigma, rho, t - eps)
deriv	-	-	(ssvitplus - ssvitminus) / 2.0 / eps
return deriv	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

g

PLEASE ADD A FUNCTION DESCRIPTION

```

g(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
float:
w = ssvi(x, gamma, sigma, rho, t)
if abs(w) < 1e-10:
    w = 1e-10
w1 = ssvi1(x, gamma, sigma, rho, t)
w2 = ssvi2(x, gamma, sigma, rho, t)
xwv = x * w1 / w
v = (1.0 - 0.5 * xwv) ** 2 - 0.25 * w1 * w1 * (0.25 + 1.0 / w) + 0.5 * w2
return v
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float		-	-
w	-	-	ssvi(x, gamma, sigma, rho, t)
if abs(w) > 1e-10		-	-
w	-	-	1e-10
w1	-	-	ssvi1(x, gamma, sigma, rho,
w2	-	-	ssvi2(x, gamma, sigma, rho,
xwv	-	-	x * w1 / w
v	-	-	(1.0 - 0.5 * xwv) ** 2 - 0.25 * w1 * w1 * (0.25
return v	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

dminus

PLEASE ADD A FUNCTION DESCRIPTION

```
dminus(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
float:
vsqrt = np.sqrt(ssvi(x, gamma, sigma, rho, t))
v = -x / vsqrt - 0.5 * vsqrt
return v
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float		-	-
vsqrt	-	-	np.sqrt(ssvi(x, gamma, sigma, rho, t))
v	-	-	-x / vsqrt - 0.5 * vsqrt
return v	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

density_ssvi

PLEASE ADD A FUNCTION DESCRIPTION

```

density_ssvi(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
float:
dm = dminus(x, gamma, sigma, rho, t)
v = g(x, gamma, sigma, rho, t) * np.exp(-0.5 * dm * dm)
v = v / np.sqrt(2.0 * np.pi * ssvi(x, gamma, sigma, rho, t))
return v
#####
t(
float64(float64, float64, float64, float64, float64),
fastmath=True,
cache=True,

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float	-	-	-
dm	-	-	dminus(x, gamma, sigma, rho, t)
v	-	-	$g(x, \gamma, \sigma, \rho, t) * \exp(-0.5 * dm^2)$
v	-	-	$v / \sqrt{2.0 * \pi * ssvi(x, \gamma, \sigma, \rho, t)}$
return v	-	-	-
float64(float64, float64, float64, float64, float64)	-	-	-
fastmath	-	-	True
cache	-	-	True

ssvi_local_varg

PLEASE ADD A FUNCTION DESCRIPTION

```

ssvi_local_varg(
x: float,
gamma: float,
sigma: float,
rho: float,
t: float
    float:
# Compute the equivalent ssvi local variance
num = ssvit(x, gamma, sigma, rho, t)
den = g(x, gamma, sigma, rho, t)
var = num / den
return var
#####
t(float64(float64[:], float64, float64, float64), fastmath=True, cache=True)

```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
x	float	-	-
gamma	float	-	-
sigma	float	-	-
rho	float	-	-
t	float	-	-
float	-	-	-
num	-	-	ssvit(x, gamma, sigma, rho, t)
den	-	-	$g(x, \gamma, \sigma, \rho, t)$
var	-	-	num / den
return var	-	-	-
t(float64(float64[],float64,float64,float64),fastmath	-	-	True, cache=True)

vol_function_ssvi

Volatility Function proposed by Gatheral in 2004.

```
vol_function_ssvi(
params: np.ndarray,
f: float,
k: float,
t: float
float:
"""Volatility_Function_proposed_by_Gatheral_in_2004."""
gamma = params[0]
sigma = params[1]
rho = params[2]
x = np.log(f / k)
var = ssvi_local_varg(x, gamma, sigma, rho, t)
var = max(var, 0.0)
sigma = np.sqrt(var)
return sigma
```

The function arguments are described in the following table.

Argument Name	Type	Description	Default Value
params	np.ndarray	-	-
f	float	-	-
k	float	-	-
t	float	-	-
float		-	-
"""Volatility Function proposed by Gatheral in 2004."""	-	-	-
gamma	-	-	params[0]
sigma	-	-	params[1]
rho	-	-	params[2]
x	-	-	np.log(f / k)
var	-	-	ssvi_local_varg(x, gamma, sigma, rho, t)
var	-	-	max(var, 0.0)
sigma	-	-	np.sqrt(var)
return sigma	-	-	-

9.40 `__init__`